

GLASS

October 5, 2026

1 Molecular States and Drug Sensitivities Associated with Gene Expression Changes in Recurrent Glioblastoma

Reference: Varn, F. S. et al. (2022). Glioma progression is shaped by genetic evolution and microenvironment interactions. *Cell*, 185(12), 2184-2199.e16. <https://doi.org/10.1016/j.cell.2022.04.038> @articlevarnGliomaProgressionShaped2022

Initial analysis 1. Load GLASS data 2. Explore the dataset 3. Check raw gene expression data 4. Clean clinical data 5. Check patient-sample relationships — `glass_all_samples.xlsx` 6. Create primary-first recurrence pairs 7. Check gene expression data availability — `glass_primary_R1_pairs.xlsx`, `glass_primary_R1_pairs_all.xlsx` 8. Create the IDH-wildtype glioblastoma cohort — `glass_idhwt_gbm_primary_R1_pairs.xlsx` 9. Assess treatment group feasibility — `glass_treatment_variables.xlsx` 10. Summarise the initial analysis - `glass_patient_selection.png`, `treatment_overlap_venn.png`

Gene expression processing and paired analysis 1. Finalise the study cohort 2. Prepare the gene expression matrix - `glass_idhwt_gbm_primary_R1_TPM.tsv` 3. Perform initial quality control of gene expression data - `glass_expression_QC.xlsx` 4. Log-transform TPM data - `glass_idhwt_gbm_primary_R1_log2TPM.tsv` 5. Filter lowly expressed genes - `glass_gene_filter_summary.xlsx`, `glass_idhwt_gbm_primary_R1_log2TPM_filtered.tsv` 6. Assess global gene expression patterns across samples - `PCA_primary_recurrence.png` 7. Perform paired comparison of gene expression between primary and recurrent samples - `glass_primary_vs_R1_gene_statistics.xlsx` 8. Assess the consistency of expression changes 9. Generate an initial list of recurrence-associated genes - `glass_recurrence_UP_initial.xlsx`, `glass_recurrence_DOWN_initial.xlsx` 10. Generate initial result plots - `glass_recurrence_vulcano.png`, `glass_recurrence_heatmap.png`, `glass_recurrence_paired_plots.png`

Identifying biological processes associated with these changes 1. Pathway analysis: Identifying biological processes that change during recurrence - `glass_GSEA_Hallmark.xlsx`, `glass_GSEA_Reactome.xlsx` 2. Summarising pathway analysis results - `glass_GSEA_top_pathways.png` 3. Assessing the stability of recurrence-associated genes using bootstrap analysis 4. Determining bootstrap selection frequency - `glass_recurrence_bootstrap_stability.xlsx` 5. Defining the recurrence signature - `glass_recurrence_UP_consensus.xlsx`, `glass_recurrence_DOWN_consensus.xlsx` 6. Creating the Recurrence Score - `glass_recurrence_scores_full_cohort.xlsx` 7. Initial assessment of the Recurrence Score in the full GLASS cohort - `glass_recurrence_score_paired.png` 8. Out-of-fold validation using 5-fold cross-validation - `glass_recurrence_scores_out_of_fold.xlsx` 9. Assessing the out-of-fold Recurrence

Score - glass_recurrence_score_out_of_fold_paired.png 10. Sensitivity analysis in patients receiving standard therapy - glass_recurrence_score_standard_treatment.xlsx, glass_recurrence_score_standard_treatment.png

Can the transcriptional program associated with tumour recurrence be identified in glioblastoma cell lines? 1. Solidifying Recurrence Score - glass_recurrence_signature_final.xlsx 2. Defining the final glioma cell-line cohort + quality check - Drug_sensitivity_AUC_(CTRPv2.2)_wide.csv 3. Glioma cell-line master table - cell_line_master.xlsx, glioma_cell_line_master.xlsx 4. Merging GLASS signature and DepMap expression data - glass_depmap_common_gene_universe_documentation.csv, glass_depmap_common_gene_universe.txt 5. Calculating DepMap Recurrence Score - depmap_glioma_recurrence_scores.xlsx 6. Verifying variance of DepMap Recurrence Score - depmap_glioma_recurrence_score_distribution.png 7. Creating GDSC microarray Recurrence Score 8. Cross-platform validation of GDSC microarray and DepMap RNA-seq - glass_gdsc_depmap_common_gene_universe_documentation.csv, gdsc_vs_depmap_recurrence_score.png, glass_crossplatform_recurrence_score_check.xlsx 9. Final check of sensitivity data availability - glioma_drug_data_availability.xlsx, cell_lines_sensitivity_QC.xlsx

Pharmacogenomic analysis of drug sensitivities associated with the glioblastoma recurrence program 1. Define the final Step 5 analysis cohort - prism_recurrence_analysis_cohort.xlsx, 2. Prepare PRISM drug-response data - prism_drug_coverage_QC.xlsx 3. Perform the primary PRISM pharmacogenomic screen - prism_recurrence_drug_screen.xlsx 4. Visualise PRISM screening results - prism_recurrence_drug_screen.png, prism_candidate_ALVESPIMYCIN_scatter.png, prism_candidate_DOCETAXEL_scatter.png, prism_candidate_PACLITAXEL_scatter.png 5. Assess the stability of candidate drug sensitivities - prism_candidate_bootstrap_stability.xlsx 6. Assess potential proliferation-related confounding - prism_candidates_adjusted_proliferation.xlsx 7. Perform independent validation using GDSC1 and GDSC2 - gdsc_validation_of_prism_candidates.xlsx 8. Perform additional validation using CTD² and CTRPv2 - external_validation_prism_candidates.xlsx 9. Assess reproducibility at the compound and target levels - candidate_selection_scorecard.png 10. Select the final pharmacogenomic candidates

HSP90 - Mechanism-level pharmacologic analysis 1. Define HSP90-inhibitors in PRISM - prism_hsp90_inhibitors.xlsx 2. HSP90 drug-class enrichment analysis - hsp90_class_enrichment.xlsx 3. HSP90 enrichment robustness analysis - hsp90_class_enrichment_exactN_permutation.png, hsp90_leave_alvespimycin_out.xlsx 4. Independent results summary of HSP90-inhibitors - hsp90_inhibitor_recurrence_associations.png

DepMap CRISPR - orthogonal genetic support 1. Preparing CRISPR Gene Effect - crispr_primary_targets_QC.xlsx 2. Relationship of Recurrence Score and genetic dependency - recurrence_score_crispr_primary_targets.xlsx 3. Relevance of CRISPR results - crispr_HSP90AA1.png, crispr_HSP90AB1.png, crispr_XP01.png, 4. Proliferation corrected CRISPR sensitivity analysis - crispr_multivariate_results.xlsx

GSE186332 - Clinical analysis of selinexor 1. GSE186332 reconstruction of clinical metadata - gse186332_selinexor_clinical_metadata.xlsx 2. Preparing GSE186332 RNA-seq expression matrix - GSE186332_gene_mapping_lengths.tsv, GSE186332_log2TPM.tsv,

GSE186332_expression_QC.xlsx 3. GLASS-GSE186332 common gene universe and technical transfer - glass_recurrence_score_gse186332_universe.xlsx 4. Calculating Frozen Recurrence Score in GSE186332 - gse186332_frozen_recurrence_scores.xlsx 5. Primary clinical selinexor-analysis - selinexor_recurrence_score_clinical_association.xlsx 6. Predefined clinical robustness analysis 6.1 Clear-response cohort - selinexor_clear_response_cohort.xlsx 6.2 Internal-validation group - selinexor_predefined_subgroup_validation.xlsx 7. IDH-wildtype sensitivity analysis 8. Proliferation confounding analysis in selinexor cohort - selinexor_proliferation_sensitivity_analysis.xlsx 9. Treatment-arm sensitivity analysis - selinexor_treatment_arm_sensitivity.xlsx

Integration of evidence 1. Integrated evidence matrix - integrated_recurrence_vulnerability_matrix.xlsx 2. TDK main summary figure - tdk_integrated_workflow_figure.png

Validation of HSP90 drug-class specificity and pharmacogenomic robustness 1. Creating HSP90 class sensitivity score 1.1 Drug-wise sensitivity score 1.2 HSP90 class score - prism_hsp90_class_sensitivity.xlsx 2. Relationship between Recurrence Score and HSP90 class sensitivity - recurrence_vs_hsp90_class_sensitivity.xlsx 3. Control of general drug sensitivity 3.1 Global non-HSP90 sensitivity score - global_non_hsp90_sensitivity.xlsx 3.2 Recurrence score and global sensitivity - recurrence_vs_global_drug_sensitivity.xlsx 3.3 HSP90 selectivity score - recurrence_vs_hsp90_selectivity.xlsx 4. HSP90 associated independent of proliferation and general drug sensitivity - hsp90_class_multivariable_model.xlsx 5. Leave-one-HSP90-drug-out robustness analysis - hsp90_leave_one_drug_out.xlsx, hsp90_leave_one_drug_out.png 6. HSP90 class validation in independent pharmacogenic screens 6.1 Identification of HSP90 inhibitors - external_hsp90_drug_annotation.xlsx 6.2 Drug-specific association 6.3 Dataset-level class effect 6.4 Summary figure - hsp90_ctd2_validation.png 7. Patient-side biological context - glass_hsp90_biological_context.xlsx 8. Final integrated HSP90 evidence table - final_hsp90_evidence_matrix.xlsx

```
[702]: import pandas as pd
from graphviz import Digraph
from matplotlib_venn import venn3
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import matplotlib.path as mpath
import math
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score, roc_curve
from statsmodels.stats.multitest import multipletests
import statsmodels.api as sm
from scipy.stats import t, skew, ttest_rel, spearmanr, wilcoxon, rankdata, \
    mannwhitneyu, kruskal, zscore
import seaborn as sns
import gseapy as gp
```

```
import re
import GEOparse
from collections import defaultdict
import gzip
```

2 Initial analysis

2.1 1. Load GLASS data

- **Source:** https://www.cbioportal.org/study/summary?id=difg_glass
 – https://datahub.assets.cbioportal.org/difg_glass.tar.gz
- **Raw Data:** difg_glass.tar.tar
- difg_glass
 - README.md
 - LICENSE
 - data_clinical_patient.txt
 - data_clinical_sample.txt
 - data_cna.txt
 - data_cna_hg19.seg
 - data_mrna_seq_tpm.txt
 - data_mrna_seq_tpm_zscores_ref_all_samples.txt
 - data_mutations.txt
 - data_mutations_1.txt
 - data_mutations_2.txt
 - data_resource_definition.txt
 - data_resource_sample.txt
 - data_timeline_surgery.txt
 - data_timeline_treatment.txt
 - meta_clinical_patient.txt
 - meta_clinical_sample.txt
 - meta_cna.txt
 - meta_cna_hg19_seg.txt
 - meta_mrna_seq_tpm.txt
 - meta_mrna_seq_tpm_zscores_ref_all_samples.txt
 - meta_mutations.txt
 - meta_resource_definition.txt
 - meta_resource_sample.txt
 - meta_study.txt
 - meta_timeline_surgery.txt
 - meta_timeline_treatment.txt
 - case_lists
 - * cases_all.txt
 - * cases_cna.txt
 - * cases_cnaseq.txt
 - * cases_RNA_Seq_mRNA.txt
 - * cases_sequenced.txt

```
[14]: data_clinical_patient = pd.read_csv(
    "../../../data/raw/GLASS/difg_glass/data_clinical_patient.txt",
    sep="\t",
    comment="#",
    low_memory=False
)

with open("../../../data/raw/GLASS/difg_glass/data_clinical_patient.txt") as f:
    comments_p = [
        line[1:].rstrip("\n").split("\t")
        for line in f
        if line.startswith("#")
    ]

data_clinical_patient_comments = pd.DataFrame(comments_p[1:],
    columns=comments_p[0])
```

```
[15]: data_clinical_sample = pd.read_csv(
    "../../../data/raw/GLASS/difg_glass/data_clinical_sample.txt",
    sep="\t",
    comment="#",
    low_memory=False
)

with open("../../../data/raw/GLASS/difg_glass/data_clinical_sample.txt") as f:
    comments_s = [
        line[1:].rstrip("\n").split("\t")
        for line in f
        if line.startswith("#")
    ]

data_clinical_sample_comments = pd.DataFrame(comments_s[1:],
    columns=comments_s[0])
```

```
[16]: data_cna = pd.read_csv(
    "../../../data/raw/GLASS/difg_glass/data_cna.txt",
    sep="\t",
    comment="#",
    low_memory=False
)
```

```
[17]: data_cna_hg19 = pd.read_csv(
    "../../../data/raw/GLASS/difg_glass/data_cna_hg19.seg",
    sep="\t",
    comment="#",
    low_memory=False
)
```

```
[18]: data_mrna_seq_tpm = pd.read_csv(  
    ".../..data/raw/GLASS/difg_glass/data_mrna_seq_tpm.txt",  
    sep="\t",  
    comment="#",  
    index_col=0,  
    low_memory=False  
)
```

```
[19]: data_mrna_seq_tpm_zscores_ref_all_samples = pd.read_csv(  
    ".../..data/raw/GLASS/difg_glass/data_mrna_seq_tpm_zscores_ref_all_samples.  
    ↪txt",  
    sep="\t",  
    comment="#",  
    index_col=0,  
    low_memory=False  
)
```

```
[20]: data_mutations = pd.read_csv(  
    ".../..data/raw/GLASS/difg_glass/data_mutations.txt",  
    sep="\t",  
    comment="#",  
    low_memory=False  
)
```

```
[21]: data_mutations_1 = pd.read_csv(  
    ".../..data/raw/GLASS/difg_glass/data_mutations_1.txt",  
    sep="\t",  
    comment="#",  
    low_memory=False  
)
```

```
[22]: data_mutations_2 = pd.read_csv(  
    ".../..data/raw/GLASS/difg_glass/data_mutations_2.txt",  
    sep="\t",  
    comment="#",  
    low_memory=False  
)
```

```
[23]: data_resource_definition = pd.read_csv(  
    ".../..data/raw/GLASS/difg_glass/data_resource_definition.txt",  
    sep="\t",  
    comment="#",  
    low_memory=False  
)
```

```
[24]: data_resource_sample = pd.read_csv(  
    ".../..data/raw/GLASS/difg_glass/data_resource_sample.txt",
```

```
sep="\t",
comment="#",
low_memory=False
)
```

```
[25]: data_timeline_surgery = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/data_timeline_surgery.txt",
      sep="\t",
      comment="#",
      low_memory=False
    )
```

```
[26]: data_timeline_treatment = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/data_timeline_treatment.txt",
      sep="\t",
      comment="#",
      low_memory=False
    )
```

```
[27]: meta_clinical_patient = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/meta_clinical_patient.txt",
      sep="\t",
      low_memory=False
    )
```

```
[28]: meta_clinical_sample = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/meta_clinical_sample.txt",
      sep="\t",
      low_memory=False
    )
```

```
[29]: meta_cna = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/meta_cna.txt",
      sep="\t",
      low_memory=False
    )
```

```
[30]: meta_cna_hg19_seg = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/meta_cna_hg19_seg.txt",
      sep="\t",
      low_memory=False
    )
```

```
[31]: meta_mrna_seq_tpm = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/meta_mrna_seq_tpm.txt",
      sep="\t",
      low_memory=False
    )
```

```
)
```

```
[32]: meta_mrna_seq_tpm_zscores_ref_all_samples = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/meta_mrna_seq_tpm_zscores_ref_all_samples.  
    ↪txt",  
    sep="\t",  
    low_memory=False  
)
```

```
[33]: meta_mutations = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/meta_mutations.txt",  
    sep="\t",  
    low_memory=False  
)
```

```
[34]: meta_resource_definition = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/meta_resource_definition.txt",  
    sep="\t",  
    low_memory=False  
)
```

```
[35]: meta_resource_sample = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/meta_resource_sample.txt",  
    sep="\t",  
    low_memory=False  
)
```

```
[36]: meta_study = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/meta_study.txt",  
    sep="\t",  
    low_memory=False  
)
```

```
[37]: meta_timeline_surgery = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/meta_timeline_surgery.txt",  
    sep="\t",  
    low_memory=False  
)
```

```
[38]: meta_timeline_treatment = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/meta_timeline_treatment.txt",  
    sep="\t",  
    low_memory=False  
)
```

```
[39]: cases_all = pd.read_csv(  
    ".../data/raw/GLASS/difg_glass/case_lists/cases_all.txt",
```



```

    low_memory=False
)

```

```

[40]: cases_cna = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/case_lists/cases_cna.txt",
      low_memory=False
)

```

```

[41]: cases_cnaseq = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/case_lists/cases_cnaseq.txt",
      low_memory=False
)

```

```

[42]: cases_RNA_Seq_mRNA = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/case_lists/cases_RNA_Seq_mRNA.txt",
      low_memory=False
)

```

```

[43]: cases_sequenced = pd.read_csv(
      ".../data/raw/GLASS/difg_glass/case_lists/cases_sequenced.txt",
      low_memory=False
)

```

2.2 2. Explore the dataset

- data_* are data
- meta_* are metadata
- cases_* number of cases for which the condition is true
- README.md contains the resources and pipelines used to compile this dataset

```

[44]: meta_clinical_patient

```

```

[44]:      cancer_study_identifier: difg_glass
0      genetic_alteration_type: CLINICAL
1      datatype: PATIENT_ATTRIBUTES
2  data_filename: data_clinical_patient.txt

```

```

[45]: data_clinical_patient_comments

```

```

[45]:      Patient Identifier \
0  Unique patient identifier
1      STRING
2      1

      Project ID      Sex \
0  Indicates whether the data was collected for t...      Sex
1      STRING      STRING
2      1      1

```

	Age	Overall Survival	\
0	Patient's age at first diagnosis	Overall patient survival status	
1	NUMBER	STRING	
2	1	1	

	Overall Survival (months)	\
0	Overall survival in months since first diagnos...	
1	NUMBER	
2	1	

	Tissue Source
0	Tissue source site for each subject
1	STRING
2	1

```
[46]: cols = ["PATIENT_ID",
             "CASE_PROJECT",
             "SEX",
             "AGE",
             "OS_STATUS",
             "TISSUE_SOURCE"]

num_cols = data_clinical_patient.columns[~data_clinical_patient.columns.
↪isin(cols)]

data_clinical_patient[cols] = data_clinical_patient[cols].astype("string")

for col in num_cols:
    data_clinical_patient[col] = pd.to_numeric(
        data_clinical_patient[col],
        errors="coerce"
    )
```

```
[47]: data_clinical_patient.head()
```

	PATIENT_ID	CASE_PROJECT	SEX	AGE	OS_STATUS	OS_MONTHS	\
0	GLSS-19-0266	GLSS	Male	71.0	1:DECEASED	35.0	
1	GLSS-19-0267	GLSS	Male	61.0	1:DECEASED	14.0	
2	GLSS-19-0268	GLSS	Male	64.0	1:DECEASED	12.0	
3	GLSS-19-0269	GLSS	Male	57.0	1:DECEASED	16.0	
4	GLSS-19-0271	GLSS	Male	56.0	1:DECEASED	14.0	

	TISSUE_SOURCE
0	Case Western Reserve University
1	Case Western Reserve University
2	Case Western Reserve University

3 Case Western Reserve University
 4 Case Western Reserve University

- data_clinical_patient.txt - Clinical patient attributes
 - PATIENT_ID
 - * Unique patient identifier
 - CASE_PROJECT
 - * Indicates whether the data was collected for the purpose of GLASS or TCGA
 - SEX
 - * Sex
 - AGE
 - * Patient's age at first diagnosis
 - OS_STATUS
 - * Overall patient survival status
 - OS_MONTHS
 - * Overall survival in months since first diagnosis to last known follow-up
 - TISSUE_SOURCE
 - * Tissue source site for each subject

[48]: meta_clinical_sample

[48]: cancer_study_identifier: difg_glass
 0 genetic_alteration_type: CLINICAL
 1 datatype: SAMPLE_ATTRIBUTES
 2 data_filename: data_clinical_sample.txt

[49]: data_clinical_sample_comments

[49]:

	Patient Identifier	Sample Identifier	Sample Type	\
0	Unique patient identifier	Unique sample identifier	Sample Type	
1	STRING	STRING	STRING	
2	1	1	1	

	Oncotree Code	Histology	Tumor Grade	\
0	Oncotree code	Histology classification	Pathological tumor grade	
1	STRING	STRING	STRING	
2	1	1	1	

	IDH Status	\
0	Canonical mutation in IDH gene	
1	STRING	
2	1	

	IDH and Codeletion Status	\
0	Combination of the idh_status and codeletion_status...	
1	STRING	
2	1	

```

Tumor Classification \
0 Tumor classification according to WHO
1 STRING
2 1

Codeletion Status ... \
0 Codeletion status of chromosomal arms 1p and 19q ...
1 STRING ...
2 1 ...

MGMT Methylation Method DNA Aliquot Barcode \
0 MGMT methylation method A variable that stitches subject, sample type,...
1 STRING STRING
2 1 0

RNA Aliquot Barcode \
0 A variable that stitches subject, sample type,...
1 STRING
2 0

Analysis Type \
0 Variable representing either whole genome sequ...
1 STRING
2 1

Stromal Score \
0 An ssGSEA-based enrichment score calculated fr...
1 NUMBER
2 0

Immune Score \
0 An ssGSEA-based enrichment score calculated fr...
1 NUMBER
2 0

ESTIMATE Score \
0 The ESTIMATE score that is calculated using bo...
1 NUMBER
2 0

Purity Cancer Type \
0 The estimated tumor purity value calculated fr... Cancer Type
1 NUMBER STRING
2 0 1

Cancer Type Detailed
0 Cancer Type Detailed

```

```
1          STRING
2          1
```

[3 rows x 33 columns]

```
[50]: cases_all
```

```
[50]:          cancer_study_identifier: difg_glass
0          stable_id: difg_glass_all
1          case_list_name: All samples
2  case_list_description: All samples (693 samples)
3          case_list_category: all_cases_in_study
4  case_list_ids: \tGLSS-19-0266-R1\tGLSS-19-0266...
```

```
[51]: cols = ["PATIENT_ID",
              "SAMPLE_ID",
              "SAMPLE_TYPE",
              "ONCOTREE_CODE",
              "HISTOLOGY",
              "TUMOR_GRADE",
              "IDH_STATUS",
              "IDH_CODEL_STATUS",
              "TUMOR_CLASSIFICATION",
              "CODEL_STATUS",
              "MGMT_METHYLATION",
              "SURGERY_TYPE",
              "SURGERY_INDICATION",
              "SURGERY_EXTENT_OF_RESECTION",
              "SURGERY_LATERALITY",
              "SURGERY_LOCATION",
              "TREATMENT_TMZ",
              "TREATMENT_TMZ_CYCLES_6",
              "TREATMENT_CONCURRENT_TMZ",
              "TREATMENT_RADIOOTHERAPY",
              "ALKYLATING_AGENT_TX",
              "MGMT_METHYLATION_METHOD",
              "DNA_ALIQUOT_BARCODE",
              "RNA_ALIQUOT_BARCODE",
              "ALIQUOT_ANALYSIS_TYPE",
              "CANCER_TYPE",
              "CANCER_TYPE_DETAILED"]

num_cols = data_clinical_sample.columns[~data_clinical_sample.columns.
    ↪isin(cols)]

data_clinical_sample[cols] = data_clinical_sample[cols].astype("string")
```

```

for col in num_cols:
    data_clinical_sample[col] = pd.to_numeric(
        data_clinical_sample[col],
        errors="coerce"
    )

```

```

[52]: data_clinical_sample.groupby(["PATIENT_ID", "SAMPLE_TYPE"])["SAMPLE_ID"].
      ↪nunique().value_counts()

```

```

[52]: SAMPLE_ID
1      693
Name: count, dtype: int64

```

- All samples are unique; each patient has at most one sample per sample type.

```

[53]: data_clinical_sample.head()

```

```

[53]:
   PATIENT_ID  SAMPLE_ID  SAMPLE_TYPE  ONCOTREE_CODE  \
0  GLSS-19-0266  GLSS-19-0266-R1  First Recurrence      GB
1  GLSS-19-0266  GLSS-19-0266-TP    Tumor Primary      GB
2  GLSS-19-0267  GLSS-19-0267-R1  First Recurrence      GB
3  GLSS-19-0267  GLSS-19-0267-TP    Tumor Primary      GB
4  GLSS-19-0268  GLSS-19-0268-R1  First Recurrence      GB

   HISTOLOGY  TUMOR_GRADE  IDH_STATUS  IDH_CODEL_STATUS  \
0  Glioblastoma      IV      IDHwt      IDHwt
1  Glioblastoma      IV      IDHwt      IDHwt
2  Glioblastoma      IV      <NA>      IDHwt
3  Glioblastoma      IV      <NA>      IDHwt
4  Glioblastoma      IV      <NA>      IDHwt

   TUMOR_CLASSIFICATION  CODEL_STATUS  ...  MGMT_METHYLATION_METHOD  \
0  Glioblastoma, IDH-wildtype      <NA>  ...      <NA>
1  Glioblastoma, IDH-wildtype      <NA>  ...      Array
2      Glioblastoma, NOS      <NA>  ...      <NA>
3      Glioblastoma, NOS      <NA>  ...      Array
4      Glioblastoma, NOS      <NA>  ...      <NA>

   DNA_ALIQUOT_BARCODE  RNA_ALIQUOT_BARCODE  \
0  GLSS-19-0266-R1-01D-WXS-0AUVTN      <NA>
1  GLSS-19-0266-TP-01D-WXS-F78XXN      <NA>
2  GLSS-19-0267-R1-01D-WXS-2AVAZO  GLSS-19-0267-R1-01R-RNA-NCCNA9
3  GLSS-19-0267-TP-01D-WXS-CS4K8X  GLSS-19-0267-TP-01R-RNA-UETVPC
4  GLSS-19-0268-R1-01D-WXS-KDBRYM  GLSS-19-0268-R1-01R-RNA-A1U9K5

   ALIQUOT_ANALYSIS_TYPE  STROMAL_SCORE  IMMUNE_SCORE  ESTIMATE_SCORE  PURITY  \
0      WXS      NaN      NaN      NaN      NaN
1      WXS      NaN      NaN      NaN      NaN

```

2	WXS, RNA	-741.873952	-748.253426	-1490.127378	0.926327
3	WXS, RNA	-459.366877	5.800561	-453.566316	0.858526
4	WXS, RNA	-350.465476	644.321629	293.856153	0.797219

	CANCER_TYPE	CANCER_TYPE_DETAILED
0	Glioma	Glioblastoma
1	Glioma	Glioblastoma
2	Glioma	Glioblastoma
3	Glioma	Glioblastoma
4	Glioma	Glioblastoma

[5 rows x 33 columns]

- All unique samples in study: 693
- data_clinical_sample.txt - Clinical sample attributes
 - PATIENT_ID
 - * Unique patient identifier
 - SAMPLE_ID
 - * Unique sample identifier
 - SAMPLE_TYPE
 - * Sample Type
 - ONCOTREE_CODE
 - * Oncotree code
 - HISTOLOGY
 - * Histology classification
 - TUMOR_GRADE
 - * Pathological tumor grade
 - IDH_STATUS
 - * IDH Status
 - IDH_CODEL_STATUS
 - * IDH and Codeletion Status
 - * Combination of the idh_status and codel_status variables
 - TUMOR_CLASSIFICATION
 - * Tumour classification according to WHO
 - CODEL_STATUS
 - * Codeletion status of chromosomal arms 1p and 19q
 - MGMT_METHYLATION
 - * Clinical assessment of MGMT methylation in samples
 - SURGERY_TYPE
 - * Surgery type
 - SURGERY_INDICATION
 - * Surgery indication
 - SURGERY_EXTENT_OF_RESECTION
 - * Indicated extent of resection needed for sample collection
 - SURGERY_LATERALITY
 - * Hemisphere from which the tumour was resected
 - SURGERY_LOCATION
 - * Brain region from which tumour was resected

- TREATMENT_TMZ
 - * Indicates whether a subject received temozolomide (tmz) for each surgical sample
- TREATMENT_TMZ_CYCLES
 - * Number of temozolomide treatment cycles
- TREATMENT_TMZ_CYCLES_6
 - * Indicates whether a subject received at least six cycles of temozolomide
- TREATMENT_CONCURRENT_TMZ
 - * Indicate whether temozolomide was administered concurrently with radiotherapy
- TREATMENT_RADIOOTHERAPY
 - * Indicates whether a subject received radiotherapy
- TREATMENT_RADIATION_DOSE_GY
 - * Radiation Dose (Gy)
- ALKYLATING_AGENT_TX
 - * Treatment with alkylating agents
- MGMT_METHYLATION_METHOD
 - * MGMT methylation method
- DNA_ALIQUOT_BARCODE
 - * A variable that stitches subject, sample type, portion, sequencing type, and a unique identifier together
- RNA_ALIQUOT_BARCODE
 - * A variable that stitches subject, sample type, portion, sequencing type, and a unique identifier together
- ALIQUOT_ANALYSIS_TYPE
 - * Variable representing either whole genome sequencing (WGS) or whole exome sequencing (WXS)
- STROMAL_SCORE
 - * An ssGSEA-based enrichment score calculated from the ESTIMATE stromal signature. Represents stromal content in a given tumour.
- IMMUNE_SCORE
 - * An ssGSEA-based enrichment score calculated from the ESTIMATE immune signature. Represents general immune content in a given tumour.
- ESTIMATE_SCORE
 - * The ESTIMATE score that is calculated using both the stromal and immune score that represents non-tumour content in a tumour.
- PURITY
 - * The estimated tumour purity value calculated from the estimate_score.
- CANCER_TYPE
 - * Cancer Type
- CANCER_TYPE_DETAILED
 - * Cancer Type Detailed

```
[54]: meta_cna
```

```
[54]: cancer_study_identifier: difg_glass
0    genetic_alteration_type: COPY_NUMBER_ALTERATION
1                                stable_id: cna
2                                show_profile_in_analysis_tab: true
```



```

3         profile_name: Copy Number Alterations
4 profile_description: Copy number alterations b...
5         datatype: DISCRETE
6         data_filename: data_cna.txt

```

```
[55]: cases_cna
```

```

[55]:         cancer_study_identifier: difg_glass
0             stable_id: difg_glass_cna
1         case_list_name: Samples with CNA data
2 case_list_description: Samples with CNA data (...)
3         case_list_category: all_cases_with_cna_data
4 case_list_ids: \tGLSS-19-0266-R1\tGLSS-19-0266...

```

```
[56]: data_cna.iloc[:5, :5]
```

```

[56]: Hugo_Symbol  GLSS-19-0266-R1  GLSS-19-0266-TP  GLSS-19-0267-R1  \
0      A1BG                0.0                1.0                0.0
1  A1BG-AS1                0.0                1.0                0.0
2      A1CF                2.0                0.0                2.0
3      A2M                 0.0                0.0                0.0
4  A2M-AS1                 0.0                0.0                0.0

```

GLSS-19-0267-TP

```

0      0.0
1      0.0
2     -1.0
3      0.0
4      0.0

```

- Samples with CNA data: 629 samples (cases_cna.txt)
- data_cna.txt
 - Copy number alterations based on GATK segmentation output.

```
[57]: meta_cna_hg19_seg
```

```

[57]:         cancer_study_identifier: difg_glass
0 genetic_alteration_type: COPY_NUMBER_ALTERATION
1                         datatype: SEG
2                         reference_genome_id: hg19
3                         description: Segment data values
4                         data_filename: data_cna_hg19.seg

```

```
[58]: cases_cnaseq
```

```

[58]:         cancer_study_identifier: difg_glass
0             stable_id: difg_glass_cnaseq
1 case_list_name: Samples with mutation and CNA ...

```

```

2 case_list_description: Samples with mutation a...
3 case_list_category: all_cases_with_mutation_an...
4 case_list_ids: GLSS-19-0266-R1\tGLSS-19-0266-T...

```

- Samples with mutation and CNA data: 629 samples (cases_cnaseq.txt)
- data_cna_hg19.seg
 - Segment data values

```
[59]: meta_mrna_seq_tpm
```

```

[59]:          cancer_study_identifier: difg_glass
0          genetic_alteration_type: MRNA_EXPRESSION
1          datatype: CONTINUOUS
2          stable_id: rna_seq_mrna
3          show_profile_in_analysis_tab: false
4 profile_description: Expression levels from 16...
5          profile_name: mRNA expression (RNA Seq TPM)
6          data_filename: data_mrna_seq_tpm.txt

```

```
[60]: cases_RNA_Seq_mRNA
```

```

[60]:          cancer_study_identifier: difg_glass
0          stable_id: difg_glass_rna_seq_mrna
1 case_list_name: Samples with mRNA data (RNA Seq)
2 case_list_description: Samples with mRNA expre...
3 case_list_category: all_cases_with_mrna_rnaseq...
4 case_list_ids: GLSS-19-0267-R1\tGLSS-19-0267-T...

```

```
[61]: data_mrna_seq_tpm.iloc[:5, :5]
```

```

[61]:          GLSS-SM-R109-TP  GLSS-19-0274-TP  GLSS-CU-R005-TP  \
Hugo_Symbol
CMSS1          44.692557          36.213871          18.364708
STX16          15.492818          134.774214          24.971397
TLE4           4.712733          43.182487          21.529454
LRRN1          19.218400          12.685700          23.226094
YKT6           85.501630          95.440750          29.212650

          GLSS-SM-R060-TP  GLSS-MD-0017-R2
Hugo_Symbol
CMSS1          38.829451          47.875472
STX16          31.478907          25.971168
TLE4           16.362683          134.981938
LRRN1          48.631390          42.586214
YKT6          102.939150          25.285882

```

- Samples with mRNA expression data: 355 samples (cases_RNA_Seq_mRNA.txt)
- data_mrna_seq_tpm.txt

- Expression levels from 168 diffuse glioma cases (RNA-Seq TPM). Values as calculated by Kallisto.

```
[62]: meta_mrna_seq_tpm_zscores_ref_all_samples
```

```
[62]:          cancer_study_identifier: difg_glass
0          genetic_alteration_type: MRNA_EXPRESSION
1                                datatype: Z-SCORE
2          stable_id: mrna_seq_tpm_all_sample_Zscores
3          show_profile_in_analysis_tab: TRUE
4  profile_description: Log-transformed mRNA z-sc...
5  profile_name: mRNA expression z-scores relativ...
6  data_filename: data_mrna_seq_tpm_zscores_ref_a...
```

```
[63]: data_mrna_seq_tpm_zscores_ref_all_samples.iloc[:5, :5]
```

```
[63]:          GLSS-SM-R109-TP  GLSS-19-0274-TP  GLSS-CU-R005-TP  \
Hugo_Symbol
CMSS1          0.5000          0.0505          -1.3800
STX16          -1.5159          2.1312          -0.7304
TLE4           -1.8047          0.7764          -0.0734
LRRN1          -0.0939         -0.4554          0.0737
YKT6           0.2750          0.4255          -1.1812
```

```
          GLSS-SM-R060-TP  GLSS-MD-0017-R2
Hugo_Symbol
CMSS1          0.1992          0.6474
STX16          -0.3435         -0.6650
TLE4           -0.4021          2.1949
LRRN1          0.7381          0.6178
YKT6           0.5292         -1.3739
```

- data_mrna_seq_tpm_zscores_ref_all_samples.txt
 - Log-transformed mRNA z-scores compared to the expression distribution of all samples (RNA Seq TPM).

```
[64]: meta_mutations
```

```
[64]:          cancer_study_identifier: difg_glass
0          genetic_alteration_type: MUTATION_EXTENDED
1                                datatype: MAF
2                                stable_id: mutations
3          show_profile_in_analysis_tab: true
4  profile_description: Whole genome or whole exo...
5                                profile_name: Mutations
6          data_filename: data_mutations.txt
```

```
[65]: cases_sequenced
```

```
[65]: cancer_study_identifier: difg_glass
0      stable_id: difg_glass_sequenced
1      case_list_name: Samples with mutation data
2 case_list_description: Samples with mutation d...
3 case_list_category: all_cases_with_mutation_data
4 case_list_ids: GLSS-19-0266-R1\tGLSS-19-0266-T...
```

```
[66]: data_mutations.head()
```

```
[66]: Hugo_Symbol  Entrez_Gene_Id  Center  NCBI_Build  Chromosome  Start_Position  \
0      MYH2          4620.0      NaN      GRCh37          17          10432606
1      ESPL1          9700.0      NaN      GRCh37          12          53676891
2      AKNA          80709.0     NaN      GRCh37          9          117099535
3      DNAJC1        64215.0     NaN      GRCh37          10         22209940
4      Z95704.4        NaN      NaN      GRCh37          4          53692

      End_Position  Strand      Consequence  Variant_Classification  ...  \
0      10432606      +      intron_variant      Intron      ...
1      53676891      +      intron_variant      Intron      ...
2      117099535      +      synonymous_variant      Silent      ...
3      22209940      +      intron_variant      Intron      ...
4      53693      +      upstream_gene_variant      5'Flank      ...

      Transcript  Transcript_Exon  Transcript_Position  Transcript_Strand  \
0  ENST00000532183.2      NaN      NaN      -
1  ENST00000257934.4      NaN      NaN      +
2  ENST00000307564.4      22.0      4281      -
3  ENST00000376980.3      NaN      NaN      -
4  ENST00000509152.2      NaN      NaN      +

      cdna_change      cds_change  gc_content      reference_context  \
0  c.e16-7241C>T      NaN      0.376559  AATTTAGTCCGTATCTAATGT
1  c.e15-22T>C      NaN      0.531172  ATCTTCTCCCTTTTTCACCCC
2  c.4119G>A  c.(4117-4119)ccG>ccA  0.635910  AGGCTGTGGGCGGCCACTTGG
3  c.e4+48G>T      NaN      0.359102  AAAATAAAAACAAAAAAACT
4  c.e1+307C>CG      NaN      0.652500  GCTGTGGTCCGGGGGTCCGG

      ssm2_pass_call  Annotation_Status
0      t      SUCCESS
1      t      SUCCESS
2      t      SUCCESS
3      t      SUCCESS
4      t      SUCCESS

[5 rows x 59 columns]
```

```
[67]: data_mutations_1.head()
```

```
[67]:
```

	Hugo_Symbol	Entrez_Gene_Id	Center	NCBI_Build	Chromosome	Start_Position	\
0	Unknown	NaN	NaN	GRCh37	1	207597011	
1	OCA2	4948.0	NaN	GRCh37	15	28208430	
2	Unknown	NaN	NaN	GRCh37	13	25655800	
3	RP11-789C1.1	NaN	NaN	GRCh37	4	171197975	
4	AZI2	64343.0	NaN	GRCh37	3	28376949	

	End_Position	Strand	Consequence	\
0	207597011	+	NaN	
1	28208430	+	intron_variant	
2	25655800	+	NaN	
3	171197975	+	intron_variant,non_coding_transcript_variant	
4	28376949	+	intron_variant	

	Variant_Classification	...	Transcript	Transcript_Exon	\
0	IGR	...	no_transcript	NaN	
1	Intron	...	ENST00000353809.5	NaN	
2	IGR	...	no_transcript	NaN	
3	Intron	...	ENST00000504509.1	NaN	
4	Intron	...	ENST00000479665.1	NaN	

	Transcript_Position	Transcript_Strand	cdna_change	cds_change	gc_content	\
0	NaN	NaN	NaN	NaN	0.558603	
1	NaN	-	c.e14-3406C>T	NaN	0.304239	
2	NaN	NaN	NaN	NaN	0.483791	
3	NaN	+	c.e2+149G>A	NaN	0.458853	
4	NaN	-	c.e5-1279C>T	NaN	0.389027	

	reference_context	ssm2_pass_call	Annotation_Status
0	CTTTATGGCGCGGACTTTCCG	t	SUCCESS
1	TCTGACCTAAGCTATATAATT	t	SUCCESS
2	TAGAGACGGGGTTTCACCGTG	t	SUCCESS
3	GCCGAGGCAGGTGCATCACTT	t	SUCCESS
4	GTGGCCACAAGCTCAAAGCTG	t	SUCCESS

[5 rows x 59 columns]

```
[68]: data_mutations_2.head()
```

```
[68]:
```

	Hugo_Symbol	Entrez_Gene_Id	Center	NCBI_Build	Chromosome	Start_Position	\
0	AACS	65985.0	NaN	GRCh37	12	125587606	
1	AADACL2	344752.0	NaN	GRCh37	3	151453547	
2	AASS	10157.0	NaN	GRCh37	7	121732941	
3	ABAT	18.0	NaN	GRCh37	16	8838100	
4	ABCA1	19.0	NaN	GRCh37	9	107663372	

	End_Position	Strand	Consequence	Variant_Classification	...	\
--	--------------	--------	-------------	------------------------	-----	---

0	125587606	+	missense_variant	Missense_Mutation	...
1	151453547	+	intron_variant	Intron	...
2	121732941	+	synonymous_variant	Silent	...
3	8838100	+	intron_variant	Intron	...
4	107663372	+	intron_variant	Intron	...

	cdna_change	cds_change	genome_change \
0	c.746A>G	c.(745-747)gAc>gGc	g.chr12:125587606A>G
1	c.e1+1586A>C	NaN	g.chr3:151453547A>C
2	c.1831T>C	c.(1831-1833)Tta>Cta	g.chr7:121732941A>G
3	c.e3-1758A>C	NaN	g.chr16:8838100A>C
4	c.e2-2523T>G	NaN	g.chr9:107663372A>C

	genomic_location_explanation	ssm2_pass_call	transcript \
0		NaN	t ENST00000316519.6
1		NaN	t ENST00000356517.3
2		NaN	t ENST00000393376.1
3		NaN	t ENST00000567812.1
4		NaN	t ENST00000374736.3

	transcript_exon	transcript_position	transcript_strand	Annotation_Status
0	7.0	952	+	SUCCESS
1	NaN	NaN	+	SUCCESS
2	16.0	1927	-	SUCCESS
3	NaN	NaN	+	SUCCESS
4	NaN	NaN	-	SUCCESS

[5 rows x 58 columns]

- Samples with mutation data: 629 samples (cases_sequenced.txt)
- data_mutations.txt, data_mutations_1.txt, data_mutations_2.txt
 - Whole genome or whole exome sequencing of diffuse glioma tumour/normal pairs from 299 patients.

```
[69]: meta_resource_sample
```

```
[69]: cancer_study_identifier: difg_glass
0      resource_type: SAMPLE
1 data_filename: data_resource_sample.txt
```

```
[70]: data_resource_sample.head()
```

```
[70]: PATIENT_ID      SAMPLE_ID RESOURCE_ID \
0  GLSS-LU-00B9    GLSS-LU-00B9-R1      HE
1  GLSS-LU-00B9    GLSS-LU-00B9-TP      HE
2  GLSS-LU-00C2    GLSS-LU-00C2-R1      HE
3  GLSS-LU-00C2    GLSS-LU-00C2-TP      HE
4  GLSS-LU-0B10    GLSS-LU-0B10-R1      HE
```

```

                                URL
0  https://styx.neurology.emory.edu/girder/histomics
1  https://styx.neurology.emory.edu/girder/histomics
2  https://styx.neurology.emory.edu/girder/histomics
3  https://styx.neurology.emory.edu/girder/histomics
4  https://styx.neurology.emory.edu/girder/histomics

```

- data_resource_sample.txt
 - Data Resources

```
[71]: meta_timeline_surgery
```

```

[71]:      cancer_study_identifier: difg_glass
0      genetic_alteration_type: CLINICAL
1                               datatype: TIMELINE
2  data_filename: data_timeline_surgery.txt

```

```

[72]: cols = ["PATIENT_ID",
              "EVENT_TYPE",
              "SURGERY_TYPE",
              "SURGERY_INDICATION",
              "SURGERY_EXTENT_OF_RESECTION",
              "SURGERY_LATERALITY",
              "SURGERY_LOCATION",
              "SAMPLE_ID"]

num_cols = data_timeline_surgery.columns[~data_timeline_surgery.columns.
↳isin(cols)]

data_timeline_surgery[cols] = data_timeline_surgery[cols].astype("string")

for col in num_cols:
    data_timeline_surgery[col] = pd.to_numeric(
        data_timeline_surgery[col],
        errors="coerce"
    )

```

```
[73]: data_timeline_surgery.head()
```

```

[73]:      PATIENT_ID  START_DATE  STOP_DATE  EVENT_TYPE  SURGERY_TYPE  \
0  GLSS-19-0266           0         NaN      Surgery    Craniotomy
1  GLSS-19-0266          548         NaN      Surgery    Craniotomy
2  GLSS-19-0267           0         NaN      Surgery    Craniotomy
3  GLSS-19-0267          304         NaN      Surgery    Craniotomy
4  GLSS-19-0268           0         NaN      Surgery    Craniotomy

SURGERY_INDICATION  SURGERY_EXTENT_OF_RESECTION  SURGERY_LATERALITY  \

```

0	<NA>	Total	Right
1	<NA>	<NA>	<NA>
2	<NA>	Subtotal	Left
3	<NA>	Total	<NA>
4	<NA>	Subtotal	Left

	SURGERY_LOCATION	SAMPLE_ID
0	Occipital lobe	GLSS-19-0266-TP
1	<NA>	GLSS-19-0266-R1
2	Temporal lobe	GLSS-19-0267-TP
3	<NA>	GLSS-19-0267-R1
4	Parietal lobe	GLSS-19-0268-TP

- data_timeline_surgery.txt
 - Surgery timeline and information of patients

```
[74]: meta_timeline_treatment
```

```
[74]: cancer_study_identifier: difg_glass
0      genetic_alteration_type: CLINICAL
1                               datatype: TIMELINE
2 data_filename: data_timeline_treatment.txt
```

```
[75]: cols = [
    "PATIENT_ID",
    "EVENT_TYPE",
    "TREATMENT_TYPE",
    "TREATMENT_SUBTYPE",
    "AGENT",
    "TREATMENT_CYCLES_6",
    "TREATMENT_CYCLES_NOTES",
    "RADIATION_OTHER"
]

num_cols = data_timeline_treatment.columns[~data_timeline_treatment.columns.
    ↪isin(cols)]

data_timeline_treatment[cols] = data_timeline_treatment[cols].astype("string")

for col in num_cols:
    data_timeline_treatment[col] = pd.to_numeric(
        data_timeline_treatment[col],
        errors="coerce"
    )
```

```
[76]: data_timeline_treatment.head()
```



```
[76]:
```

	PATIENT_ID	START_DATE	STOP_DATE	EVENT_TYPE	TREATMENT_TYPE	\
0	GLSS-19-0266	0	NaN	Treatment	Chemotherapy	
1	GLSS-19-0267	0	NaN	Treatment	Chemotherapy	
2	GLSS-19-0268	0	NaN	Treatment	Chemotherapy	
3	GLSS-19-0269	0	NaN	Treatment	Chemotherapy	
4	GLSS-19-0271	0	NaN	Treatment	Chemotherapy	

	TREATMENT_SUBTYPE	AGENT	TREATMENT_CYCLES	TREATMENT_CYCLES_6	\
0	<NA>	Temozolomide	12.0	1.0	
1	<NA>	Temozolomide	5.0	0.0	
2	<NA>	Temozolomide	5.0	0.0	
3	<NA>	Temozolomide	6.0	1.0	
4	<NA>	Temozolomide	6.0	1.0	

	TREATMENT_CYCLES_NOTES	RADIATION_DOSE_GY	RADIATION_FRACTIONS	\
0	<NA>	NaN	NaN	
1	<NA>	NaN	NaN	
2	<NA>	NaN	NaN	
3	<NA>	NaN	NaN	
4	<NA>	NaN	NaN	

	RADIATION_OTHER
0	<NA>
1	<NA>
2	<NA>
3	<NA>
4	<NA>

- data_timeline_treatment.txt
 - Treatment timeline and information of patients

2.3 3. Check raw gene expression data

- data_mrna_seq_tpm.txt

```
[77]: for col in data_mrna_seq_tpm.columns:
      data_mrna_seq_tpm[col] = pd.to_numeric(
          data_mrna_seq_tpm[col],
          errors="coerce"
      )
```

```
[78]: data_mrna_seq_tpm.head()
```

```
[78]:
```

	GLSS-SM-R109-TP	GLSS-19-0274-TP	GLSS-CU-R005-TP	\
Hugo_Symbol				
CMSS1	44.692557	36.213871	18.364708	
STX16	15.492818	134.774214	24.971397	
TLE4	4.712733	43.182487	21.529454	

LRRN1	19.218400	12.685700	23.226094
YKT6	85.501630	95.440750	29.212650

	GLSS-SM-R060-TP	GLSS-MD-0017-R2	TCGA-TQ-A8XE-TP	\
Hugo_Symbol				
CMSS1	38.829451	47.875472	37.354181	
STX16	31.478907	25.971168	48.398109	
TLE4	16.362683	134.981938	7.480858	
LRRN1	48.631390	42.586214	101.129177	
YKT6	102.939150	25.285882	65.099348	

	GLSS-LU-00C2-TP	GLSS-HF-CB66-TP	GLSS-LU-00B9-R1	\
Hugo_Symbol				
CMSS1	39.917940	41.513165	22.973755	
STX16	70.300232	31.536573	76.463906	
TLE4	63.985062	56.806393	84.598521	
LRRN1	40.922450	50.047356	40.284390	
YKT6	56.126976	62.249941	130.610080	

	TCGA-06-0210-TP	...	GLSS-SM-R061-R1	GLSS-SN-0015-R2	\
Hugo_Symbol		...			
CMSS1	28.642999	...	29.911310	53.248147	
STX16	30.736831	...	38.583038	55.493750	
TLE4	9.326829	...	1.948282	22.971165	
LRRN1	17.337090	...	7.724712	3.803241	
YKT6	106.560249	...	56.908614	90.844410	

	GLSS-SM-R065-R1	GLSS-HF-2934-R1	GLSS-SM-R102-R1	\
Hugo_Symbol				
CMSS1	20.346213	27.148167	70.808588	
STX16	22.586294	39.445908	40.147123	
TLE4	15.250171	6.654166	28.574135	
LRRN1	2.530560	5.285038	0.497294	
YKT6	41.964471	108.408900	56.954560	

	GLSS-SN-0016-R1	GLSS-CU-P100-R1	GLSS-CU-P020-R2	\
Hugo_Symbol				
CMSS1	43.082690	27.216500	30.286259	
STX16	28.627986	48.037664	41.587925	
TLE4	11.702118	15.526567	23.847281	
LRRN1	3.147060	0.104066	7.411189	
YKT6	21.263001	94.336060	70.022150	

	GLSS-SM-R104-R1	GLSS-SM-R063-R1
Hugo_Symbol		
CMSS1	40.866360	25.832924
STX16	29.988264	37.020001

TLE4	30.876945	10.591849
LRRN1	9.805628	6.299265
YKT6	63.413351	147.654865

[5 rows x 355 columns]

```
[79]: glass_expr = data_mrna_seq_tpm.copy()
```

```
[80]: glass_expr.head()
```

```
[80]:
```

	GLSS-SM-R109-TP	GLSS-19-0274-TP	GLSS-CU-R005-TP	\
Hugo_Symbol				
CMSS1	44.692557	36.213871	18.364708	
STX16	15.492818	134.774214	24.971397	
TLE4	4.712733	43.182487	21.529454	
LRRN1	19.218400	12.685700	23.226094	
YKT6	85.501630	95.440750	29.212650	

	GLSS-SM-R060-TP	GLSS-MD-0017-R2	TCGA-TQ-A8XE-TP	\
Hugo_Symbol				
CMSS1	38.829451	47.875472	37.354181	
STX16	31.478907	25.971168	48.398109	
TLE4	16.362683	134.981938	7.480858	
LRRN1	48.631390	42.586214	101.129177	
YKT6	102.939150	25.285882	65.099348	

	GLSS-LU-00C2-TP	GLSS-HF-CB66-TP	GLSS-LU-00B9-R1	\
Hugo_Symbol				
CMSS1	39.917940	41.513165	22.973755	
STX16	70.300232	31.536573	76.463906	
TLE4	63.985062	56.806393	84.598521	
LRRN1	40.922450	50.047356	40.284390	
YKT6	56.126976	62.249941	130.610080	

	TCGA-06-0210-TP	...	GLSS-SM-R061-R1	GLSS-SN-0015-R2	\
Hugo_Symbol		...			
CMSS1	28.642999	...	29.911310	53.248147	
STX16	30.736831	...	38.583038	55.493750	
TLE4	9.326829	...	1.948282	22.971165	
LRRN1	17.337090	...	7.724712	3.803241	
YKT6	106.560249	...	56.908614	90.844410	

	GLSS-SM-R065-R1	GLSS-HF-2934-R1	GLSS-SM-R102-R1	\
Hugo_Symbol				
CMSS1	20.346213	27.148167	70.808588	
STX16	22.586294	39.445908	40.147123	
TLE4	15.250171	6.654166	28.574135	

LRRN1	2.530560	5.285038	0.497294
YKT6	41.964471	108.408900	56.954560

	GLSS-SN-0016-R1	GLSS-CU-P100-R1	GLSS-CU-P020-R2	\
Hugo_Symbol				
CMSS1	43.082690	27.216500	30.286259	
STX16	28.627986	48.037664	41.587925	
TLE4	11.702118	15.526567	23.847281	
LRRN1	3.147060	0.104066	7.411189	
YKT6	21.263001	94.336060	70.022150	

	GLSS-SM-R104-R1	GLSS-SM-R063-R1
Hugo_Symbol		
CMSS1	40.866360	25.832924
STX16	29.988264	37.020001
TLE4	30.876945	10.591849
LRRN1	9.805628	6.299265
YKT6	63.413351	147.654865

[5 rows x 355 columns]

2.4 4. Clean clinical data

- data_clinical_patient.txt
- data_clinical_sample.txt
- Was done when loading dataset

2.5 5. Check patient-sample relationships - glass_all_samples.xlsx

- data_clinical_sample: PATIENT_ID, SAMPLE_ID, SAMPLE_TYPE, TUMOR_GRADE, TUMOR_CLASSIFICATION, IDH_CODEL_STATUS, CANCER_TYPE_DETAILED
- data_timeline_surgery:
 - use START_DATE to derive:
 - * SURGERY_NUMBER
 - * COLLECTION_TIME

```
[81]: data_clinical_sample.head()
```

```
[81]:
```

	PATIENT_ID	SAMPLE_ID	SAMPLE_TYPE	ONCOTREE_CODE	\
0	GLSS-19-0266	GLSS-19-0266-R1	First Recurrence	GB	
1	GLSS-19-0266	GLSS-19-0266-TP	Tumor Primary	GB	
2	GLSS-19-0267	GLSS-19-0267-R1	First Recurrence	GB	
3	GLSS-19-0267	GLSS-19-0267-TP	Tumor Primary	GB	
4	GLSS-19-0268	GLSS-19-0268-R1	First Recurrence	GB	

	HISTOLOGY	TUMOR_GRADE	IDH_STATUS	IDH_CODEL_STATUS	\
0	Glioblastoma	IV	IDHwt	IDHwt	
1	Glioblastoma	IV	IDHwt	IDHwt	

2	Glioblastoma	IV	<NA>	IDHwt
3	Glioblastoma	IV	<NA>	IDHwt
4	Glioblastoma	IV	<NA>	IDHwt

	TUMOR_CLASSIFICATION	CODEL_STATUS	...	MGMT_METHYLATION_METHOD	\
0	Glioblastoma, IDH-wildtype	<NA>	...	<NA>	
1	Glioblastoma, IDH-wildtype	<NA>	...	Array	
2	Glioblastoma, NOS	<NA>	...	<NA>	
3	Glioblastoma, NOS	<NA>	...	Array	
4	Glioblastoma, NOS	<NA>	...	<NA>	

	DNA_ALIQUOT_BARCODE	RNA_ALIQUOT_BARCODE	\
0	GLSS-19-0266-R1-01D-WXS-0AUVTN	<NA>	
1	GLSS-19-0266-TP-01D-WXS-F78XXN	<NA>	
2	GLSS-19-0267-R1-01D-WXS-2AVAZO	GLSS-19-0267-R1-01R-RNA-NCCNA9	
3	GLSS-19-0267-TP-01D-WXS-CS4K8X	GLSS-19-0267-TP-01R-RNA-UETVPC	
4	GLSS-19-0268-R1-01D-WXS-KDBRYM	GLSS-19-0268-R1-01R-RNA-A1U9K5	

	ALIQUOT_ANALYSIS_TYPE	STROMAL_SCORE	IMMUNE_SCORE	ESTIMATE_SCORE	PURITY	\
0	WXS	NaN	NaN	NaN	NaN	
1	WXS	NaN	NaN	NaN	NaN	
2	WXS, RNA	-741.873952	-748.253426	-1490.127378	0.926327	
3	WXS, RNA	-459.366877	5.800561	-453.566316	0.858526	
4	WXS, RNA	-350.465476	644.321629	293.856153	0.797219	

	CANCER_TYPE	CANCER_TYPE_DETAILED
0	Glioma	Glioblastoma
1	Glioma	Glioblastoma
2	Glioma	Glioblastoma
3	Glioma	Glioblastoma
4	Glioma	Glioblastoma

[5 rows x 33 columns]

```
[82]: data_timeline_surgery.head()
```

```
[82]:
```

	PATIENT_ID	START_DATE	STOP_DATE	EVENT_TYPE	SURGERY_TYPE	\
0	GLSS-19-0266	0	NaN	Surgery	Craniotomy	
1	GLSS-19-0266	548	NaN	Surgery	Craniotomy	
2	GLSS-19-0267	0	NaN	Surgery	Craniotomy	
3	GLSS-19-0267	304	NaN	Surgery	Craniotomy	
4	GLSS-19-0268	0	NaN	Surgery	Craniotomy	

	SURGERY_INDICATION	SURGERY_EXTENT_OF_RESECTION	SURGERY_LATERALITY	\
0	<NA>	Total	Right	
1	<NA>	<NA>	<NA>	
2	<NA>	Subtotal	Left	

3	<NA>	Total	<NA>
4	<NA>	Subtotal	Left

	SURGERY_LOCATION	SAMPLE_ID
0	Occipital lobe	GLSS-19-0266-TP
1	<NA>	GLSS-19-0266-R1
2	Temporal lobe	GLSS-19-0267-TP
3	<NA>	GLSS-19-0267-R1
4	Parietal lobe	GLSS-19-0268-TP

```
[83]: glass_all_samples_raw = pd.merge(
    data_clinical_sample,
    data_timeline_surgery,
    on=["SAMPLE_ID", "PATIENT_ID", "SURGERY_TYPE", "SURGERY_INDICATION",
    "SURGERY_EXTENT_OF_RESECTION",
    "SURGERY_LATERALITY", "SURGERY_LOCATION"],
    how="outer"
)
```

```
[84]: glass_all_samples_raw.head()
```

```
[84]:
```

	PATIENT_ID	SAMPLE_ID	SAMPLE_TYPE	ONCOTREE_CODE	\
0	GLSS-19-0266	GLSS-19-0266-R1	First Recurrence	GB	
1	GLSS-19-0266	GLSS-19-0266-TP	Tumor Primary	GB	
2	GLSS-19-0267	GLSS-19-0267-R1	First Recurrence	GB	
3	GLSS-19-0267	GLSS-19-0267-TP	Tumor Primary	GB	
4	GLSS-19-0268	GLSS-19-0268-R1	First Recurrence	GB	

	HISTOLOGY	TUMOR_GRADE	IDH_STATUS	IDH_CODEL_STATUS	\
0	Glioblastoma	IV	IDHwt	IDHwt	
1	Glioblastoma	IV	IDHwt	IDHwt	
2	Glioblastoma	IV	<NA>	IDHwt	
3	Glioblastoma	IV	<NA>	IDHwt	
4	Glioblastoma	IV	<NA>	IDHwt	

	TUMOR_CLASSIFICATION	CODEL_STATUS	...	ALIQUOT_ANALYSIS_TYPE	\
0	Glioblastoma, IDH-wildtype	<NA>	...	WXS	
1	Glioblastoma, IDH-wildtype	<NA>	...	WXS	
2	Glioblastoma, NOS	<NA>	...	WXS, RNA	
3	Glioblastoma, NOS	<NA>	...	WXS, RNA	
4	Glioblastoma, NOS	<NA>	...	WXS, RNA	

	STROMAL_SCORE	IMMUNE_SCORE	ESTIMATE_SCORE	PURITY	CANCER_TYPE	\
0	NaN	NaN	NaN	NaN	Glioma	
1	NaN	NaN	NaN	NaN	Glioma	
2	-741.873952	-748.253426	-1490.127378	0.926327	Glioma	
3	-459.366877	5.800561	-453.566316	0.858526	Glioma	

4	-350.465476	644.321629	293.856153	0.797219	Glioma
---	-------------	------------	------------	----------	--------

	CANCER_TYPE_DETAILED	START_DATE	STOP_DATE	EVENT_TYPE
0	Glioblastoma	548.0	NaN	Surgery
1	Glioblastoma	0.0	NaN	Surgery
2	Glioblastoma	304.0	NaN	Surgery
3	Glioblastoma	0.0	NaN	Surgery
4	Glioblastoma	304.0	NaN	Surgery

[5 rows x 36 columns]

```
[85]: glass_all_samples = glass_all_samples_raw[[
    "PATIENT_ID",
    "SAMPLE_ID",
    "SAMPLE_TYPE",
    "TUMOR_GRADE",
    "TUMOR_CLASSIFICATION",
    "IDH_CODEL_STATUS",
    "CANCER_TYPE_DETAILED",
    "START_DATE"
]].copy()
```

```
[86]: glass_all_samples["SURGERY_NUMBER"] = (
    glass_all_samples.groupby("PATIENT_ID")["START_DATE"]
    .rank(method="first", ascending=True)
    .astype("Int64")
)
```

```
[87]: glass_all_samples["COLLECTION_TIME"] = glass_all_samples["START_DATE"].copy()
glass_all_samples = glass_all_samples.drop(columns=["START_DATE"])
```

```
[88]: glass_all_samples[["DIAGNOSIS", "TUMOR_IDH_STATUS"]] = (
    glass_all_samples["TUMOR_CLASSIFICATION"]
    .str.split(",", n=1, expand=True)
)

glass_all_samples["DIAGNOSIS"] = glass_all_samples["DIAGNOSIS"].str.strip()
glass_all_samples["TUMOR_IDH_STATUS"] = glass_all_samples["TUMOR_IDH_STATUS"].
    ↪str.strip()
```

```
[89]: # TUMOR_IDH_STATUS was changed to be data_clinical_sample[IDH_CODEL_STATUS]
glass_all_samples["TUMOR_IDH_STATUS"] = glass_all_samples["IDH_CODEL_STATUS"].
    ↪copy()
```

```
[90]: glass_all_samples = glass_all_samples[[
    "PATIENT_ID",
    "SAMPLE_ID",
```

```

    "CANCER_TYPE_DETAILED",
    "SAMPLE_TYPE",
    "SURGERY_NUMBER",
    "COLLECTION_TIME",
    "DIAGNOSIS",
    "TUMOR_GRADE",
    "TUMOR_IDH_STATUS"
]]

```

```
[91]: glass_all_samples.head()
```

```
[91]:
```

	PATIENT_ID	SAMPLE_ID	CANCER_TYPE_DETAILED	SAMPLE_TYPE	\
0	GLSS-19-0266	GLSS-19-0266-R1	Glioblastoma	First Recurrence	
1	GLSS-19-0266	GLSS-19-0266-TP	Glioblastoma	Tumor Primary	
2	GLSS-19-0267	GLSS-19-0267-R1	Glioblastoma	First Recurrence	
3	GLSS-19-0267	GLSS-19-0267-TP	Glioblastoma	Tumor Primary	
4	GLSS-19-0268	GLSS-19-0268-R1	Glioblastoma	First Recurrence	

	SURGERY_NUMBER	COLLECTION_TIME	DIAGNOSIS	TUMOR_GRADE	TUMOR_IDH_STATUS
0	2	548.0	Glioblastoma	IV	IDHwt
1	1	0.0	Glioblastoma	IV	IDHwt
2	2	304.0	Glioblastoma	IV	IDHwt
3	1	0.0	Glioblastoma	IV	IDHwt
4	2	304.0	Glioblastoma	IV	IDHwt

```
[92]: glass_all_samples.to_excel("../data/processed/GLASS/glass_all_samples.xlsx",
    ↪index=False)
```

- Final glass_all_samples.xlsx exported to /data/processed directory

2.6 6. Create primary-first recurrent pairs - glass_primary_R1_pairs_all

- One row per patient
- Must contain both:
 - SAMPLE_TYPE = Tumor Primary
 - SAMPLE_TYPE = First Recurrence
- Other recurrence types (R2, R3, R4, etc.) do not affect eligibility
- Retain:
 - PATIENT_ID
 - PRIMARY_SAMPLE_ID
 - FIRST_RECURRENCE_SAMPLE_ID

```
[93]: glass_primary_R1_pairs_all = glass_all_samples[[
    "PATIENT_ID",
    "SAMPLE_ID",
    "SAMPLE_TYPE"
]].copy()
```



```
[94]: glass_primary_R1_pairs_all = glass_primary_R1_pairs_all.loc[
      glass_primary_R1_pairs_all["SAMPLE_TYPE"].isin(
          ["Tumor Primary", "First Recurrence"]),
      ["PATIENT_ID", "SAMPLE_ID", "SAMPLE_TYPE"]].copy()

[95]: glass_primary_R1_pairs_all["PRIMARY_SAMPLE_ID"] =
      ↪glass_primary_R1_pairs_all["SAMPLE_ID"].where(
          glass_primary_R1_pairs_all["SAMPLE_TYPE"] == "Tumor Primary"
      )

      glass_primary_R1_pairs_all["FIRST_RECURRENCE_SAMPLE_ID"] =
      ↪glass_primary_R1_pairs_all["SAMPLE_ID"].where(
          glass_primary_R1_pairs_all["SAMPLE_TYPE"] == "First Recurrence"
      )

[96]: glass_primary_R1_pairs_all.drop(
      columns=["SAMPLE_TYPE", "SAMPLE_ID"],
      inplace=True
  )

[97]: glass_primary_R1_pairs_all = (
      glass_primary_R1_pairs_all
      .groupby("PATIENT_ID", as_index=False)
      .agg({
          "PRIMARY_SAMPLE_ID": "first",
          "FIRST_RECURRENCE_SAMPLE_ID": "first"
      })
      .dropna(subset=["PRIMARY_SAMPLE_ID", "FIRST_RECURRENCE_SAMPLE_ID"])
  )
```

2.7 7. Check gene expression data availability - glass_primary_R1_pairs_all.xlsx, glass_primary_R1_pairs.xlsx

- Cross-reference PRIMARY_SAMPLE_ID and FIRST_RECURRENCE_SAMPLE_ID from glass_primary_R1_pairs_all with glass_expr.columns
- primary_has_expression, recurrent_has_expression
- glass_primary_R1_pairs_all.xlsx
 - All patients with both primary and first recurrent samples
 - Includes expression availability flags
- glass_primary_R1_pairs.xlsx
 - Patients for whom both primary and first recurrent samples have expression data

```
[98]: glass_expr.head()
```

```
[98]:          GLSS-SM-R109-TP  GLSS-19-0274-TP  GLSS-CU-R005-TP  \
Hugo_Symbol
CMSS1          44.692557        36.213871        18.364708
```

STX16	15.492818	134.774214	24.971397
TLE4	4.712733	43.182487	21.529454
LRRN1	19.218400	12.685700	23.226094
YKT6	85.501630	95.440750	29.212650

	GLSS-SM-R060-TP	GLSS-MD-0017-R2	TCGA-TQ-A8XE-TP \
Hugo_Symbol			
CMSS1	38.829451	47.875472	37.354181
STX16	31.478907	25.971168	48.398109
TLE4	16.362683	134.981938	7.480858
LRRN1	48.631390	42.586214	101.129177
YKT6	102.939150	25.285882	65.099348

	GLSS-LU-00C2-TP	GLSS-HF-CB66-TP	GLSS-LU-00B9-R1 \
Hugo_Symbol			
CMSS1	39.917940	41.513165	22.973755
STX16	70.300232	31.536573	76.463906
TLE4	63.985062	56.806393	84.598521
LRRN1	40.922450	50.047356	40.284390
YKT6	56.126976	62.249941	130.610080

	TCGA-06-0210-TP	...	GLSS-SM-R061-R1	GLSS-SN-0015-R2 \
Hugo_Symbol		...		
CMSS1	28.642999	...	29.911310	53.248147
STX16	30.736831	...	38.583038	55.493750
TLE4	9.326829	...	1.948282	22.971165
LRRN1	17.337090	...	7.724712	3.803241
YKT6	106.560249	...	56.908614	90.844410

	GLSS-SM-R065-R1	GLSS-HF-2934-R1	GLSS-SM-R102-R1 \
Hugo_Symbol			
CMSS1	20.346213	27.148167	70.808588
STX16	22.586294	39.445908	40.147123
TLE4	15.250171	6.654166	28.574135
LRRN1	2.530560	5.285038	0.497294
YKT6	41.964471	108.408900	56.954560

	GLSS-SN-0016-R1	GLSS-CU-P100-R1	GLSS-CU-P020-R2 \
Hugo_Symbol			
CMSS1	43.082690	27.216500	30.286259
STX16	28.627986	48.037664	41.587925
TLE4	11.702118	15.526567	23.847281
LRRN1	3.147060	0.104066	7.411189
YKT6	21.263001	94.336060	70.022150

	GLSS-SM-R104-R1	GLSS-SM-R063-R1
Hugo_Symbol		

CMSS1	40.866360	25.832924
STX16	29.988264	37.020001
TLE4	30.876945	10.591849
LRRN1	9.805628	6.299265
YKT6	63.413351	147.654865

[5 rows x 355 columns]

```
[99]: glass_primary_R1_pairs_all["primary_has_expression"] = (
        glass_primary_R1_pairs_all["PRIMARY_SAMPLE_ID"]
        .isin(glass_expr.columns)
    )

    glass_primary_R1_pairs_all["recurrent_has_expression"] = (
        glass_primary_R1_pairs_all["FIRST_RECURRENCE_SAMPLE_ID"]
        .isin(glass_expr.columns)
    )
```

```
[100]: glass_primary_R1_pairs_all["primary_and_recurrent_has_expression"] = (
        glass_primary_R1_pairs_all["primary_has_expression"]
        & glass_primary_R1_pairs_all["recurrent_has_expression"]
    )

    glass_primary_R1_pairs = glass_primary_R1_pairs_all.loc[
        glass_primary_R1_pairs_all["primary_and_recurrent_has_expression"]
    ].copy()

    glass_primary_R1_pairs = glass_primary_R1_pairs[
        ["PATIENT_ID", "PRIMARY_SAMPLE_ID", "FIRST_RECURRENCE_SAMPLE_ID"]].copy()
```

```
[101]: glass_primary_R1_pairs_all.to_excel(
        ".../data/processed/GLASS/GLASS_PAIRS/glass_primary_R1_pairs_all.xlsx",
        index=False
    )
```

- Final glass_primary_R1_pairs_all.xlsx exported to /data/processed directory

```
[102]: glass_primary_R1_pairs.to_excel(
        ".../data/processed/GLASS/GLASS_PAIRS/glass_primary_R1_pairs.xlsx",
        index=False
    )
```

- Final glass_primary_R1_pairs.xlsx exported to /data/processed directory

2.8 8. Create the IDH-wildtype glioblastoma cohort - glass_idhwt_gbm_primary_R1_pairs.xlsx

- Primary dataframe for analysis

```
[103]: glass_all_samples.head()
```

```
[103]:
```

	PATIENT_ID	SAMPLE_ID	CANCER_TYPE_DETAILED	SAMPLE_TYPE	\
0	GLSS-19-0266	GLSS-19-0266-R1	Glioblastoma	First Recurrence	
1	GLSS-19-0266	GLSS-19-0266-TP	Glioblastoma	Tumor Primary	
2	GLSS-19-0267	GLSS-19-0267-R1	Glioblastoma	First Recurrence	
3	GLSS-19-0267	GLSS-19-0267-TP	Glioblastoma	Tumor Primary	
4	GLSS-19-0268	GLSS-19-0268-R1	Glioblastoma	First Recurrence	

	SURGERY_NUMBER	COLLECTION_TIME	DIAGNOSIS	TUMOR_GRADE	TUMOR_IDH_STATUS
0	2	548.0	Glioblastoma	IV	IDHwt
1	1	0.0	Glioblastoma	IV	IDHwt
2	2	304.0	Glioblastoma	IV	IDHwt
3	1	0.0	Glioblastoma	IV	IDHwt
4	2	304.0	Glioblastoma	IV	IDHwt

```
[104]: glass_primary_R1_pairs_diag_idh = glass_all_samples.copy()

glass_primary_R1_pairs_diag_idh["PRIMARY_DIAGNOSIS"] =
    ↳glass_all_samples["DIAGNOSIS"].where(
        glass_all_samples["SAMPLE_TYPE"] == "Tumor Primary"
    )

glass_primary_R1_pairs_diag_idh["FIRST_RECURRENCE_DIAGNOSIS"] =
    ↳glass_all_samples["DIAGNOSIS"].where(
        glass_all_samples["SAMPLE_TYPE"] == "First Recurrence"
    )

glass_primary_R1_pairs_diag_idh["PRIMARY_IDH"] =
    ↳glass_all_samples["TUMOR_IDH_STATUS"].where(
        glass_all_samples["SAMPLE_TYPE"] == "Tumor Primary"
    )

glass_primary_R1_pairs_diag_idh["FIRST_RECURRENCE_IDH"] =
    ↳glass_all_samples["TUMOR_IDH_STATUS"].where(
        glass_all_samples["SAMPLE_TYPE"] == "First Recurrence"
    )
```

```
[105]: glass_primary_R1_pairs_diag_idh.drop(
    columns=["SAMPLE_TYPE", "SAMPLE_ID", "CANCER_TYPE_DETAILED",
    ↳"SURGERY_NUMBER", "COLLECTION_TIME", "DIAGNOSIS",
        "TUMOR_GRADE", "TUMOR_IDH_STATUS"],
    inplace=True
)
```

```
[106]: glass_primary_R1_pairs_diag_idh = (
    glass_primary_R1_pairs_diag_idh
```

```

.groupby("PATIENT_ID", as_index=False)
.agg({
    "PRIMARY_DIAGNOSIS": "first",
    "FIRST_RECURRENCE_DIAGNOSIS": "first",
    "PRIMARY_IDH": "first",
    "FIRST_RECURRENCE_IDH": "first",
})
)

```

```

[107]: glass_idh_diag_primary_R1_pairs = glass_primary_R1_pairs.merge(
        glass_primary_R1_pairs_diag_idh,
        on="PATIENT_ID",
        how="left"
    )

```

```

[108]: glass_idh_diag_primary_R1_pairs.head()

```

```

[108]:      PATIENT_ID PRIMARY_SAMPLE_ID FIRST_RECURRENCE_SAMPLE_ID \
0  GLSS-19-0267    GLSS-19-0267-TP          GLSS-19-0267-R1
1  GLSS-19-0268    GLSS-19-0268-TP          GLSS-19-0268-R1
2  GLSS-19-0269    GLSS-19-0269-TP          GLSS-19-0269-R1
3  GLSS-19-0271    GLSS-19-0271-TP          GLSS-19-0271-R1
4  GLSS-19-0272    GLSS-19-0272-TP          GLSS-19-0272-R1

      PRIMARY_DIAGNOSIS FIRST_RECURRENCE_DIAGNOSIS PRIMARY_IDH \
0      Glioblastoma          Glioblastoma          IDHwt
1      Glioblastoma          Glioblastoma          IDHwt
2      Glioblastoma          Glioblastoma          IDHwt
3      Glioblastoma          Glioblastoma          IDHwt
4      Glioblastoma          Glioblastoma          IDHwt

      FIRST_RECURRENCE_IDH
0          IDHwt
1          IDHwt
2          IDHwt
3          IDHwt
4          IDHwt

```

```

[109]: glass_gbm_primary_R1_pairs = glass_idh_diag_primary_R1_pairs.loc[
        (glass_idh_diag_primary_R1_pairs["PRIMARY_DIAGNOSIS"] == "Glioblastoma") &
        (glass_idh_diag_primary_R1_pairs["FIRST_RECURRENCE_DIAGNOSIS"] ==
        ↪ "Glioblastoma"),
        ["PATIENT_ID", "PRIMARY_SAMPLE_ID", "FIRST_RECURRENCE_SAMPLE_ID",
        ↪ "PRIMARY_IDH", "FIRST_RECURRENCE_IDH"]
    ].copy()

    glass_idhwt_gbm_primary_R1_pairs = glass_gbm_primary_R1_pairs.loc[

```

```
(glass_gbm_primary_R1_pairs["PRIMARY_IDH"] == "IDHwt") &
(glass_gbm_primary_R1_pairs["FIRST_RECURRENCE_IDH"] == "IDHwt"),
["PATIENT_ID", "PRIMARY_SAMPLE_ID", "FIRST_RECURRENCE_SAMPLE_ID"]
].copy()
```

```
[110]: glass_idhwt_gbm_primary_R1_pairs.head()
```

```
[110]:
```

	PATIENT_ID	PRIMARY_SAMPLE_ID	FIRST_RECURRENCE_SAMPLE_ID
0	GLSS-19-0267	GLSS-19-0267-TP	GLSS-19-0267-R1
1	GLSS-19-0268	GLSS-19-0268-TP	GLSS-19-0268-R1
2	GLSS-19-0269	GLSS-19-0269-TP	GLSS-19-0269-R1
3	GLSS-19-0271	GLSS-19-0271-TP	GLSS-19-0271-R1
4	GLSS-19-0272	GLSS-19-0272-TP	GLSS-19-0272-R1

```
[111]: glass_idhwt_gbm_primary_R1_pairs.to_excel(
        ".../data/processed/GLASS/GLASS_PAIRS/glass_idhwt_gbm_primary_R1_pairs.
        ↪xlsx", index=False)
```

- Final glass_idhwt_gbm_primary_R1_pairs.xlsx exported to /data/processed directory.

2.9 9. Assess treatment group feasibility - glass_treatment_variables.xlsx

- Will be used later to create treatment groups
- Variables of interest: temozolomide, radiotherapy, concurrent TMZ + radiotherapy, other systemic treatments
- Can treatment exposure between the primary and first recurrent samples be determined from the available clinical and treatment timeline data?

```
[112]: treatment_pairs = (glass_idhwt_gbm_primary_R1_pairs.copy())
assert treatment_pairs["PATIENT_ID"].is_unique
```

```
[113]: sample_treatment = (
    glass_all_samples
    .merge(
        data_clinical_sample[
            [
                "PATIENT_ID",
                "SAMPLE_ID",
                "TREATMENT_TMZ",
                "TREATMENT_RADIOOTHERAPY",
                "TREATMENT_CONCURRENT_TMZ",
            ]
        ],
        on=["PATIENT_ID", "SAMPLE_ID"],
        how="left",
    )
)
```

```

sample_treatment = sample_treatment[
    [
        "PATIENT_ID",
        "SAMPLE_ID",
        "SAMPLE_TYPE",
        "COLLECTION_TIME",
        "TREATMENT_TMZ",
        "TREATMENT_RADIOOTHERAPY",
        "TREATMENT_CONCURRENT_TMZ",
    ]
].copy()

sample_treatment = sample_treatment.loc[
    sample_treatment["PATIENT_ID"].isin(
        treatment_pairs["PATIENT_ID"]
    )
].copy()

```

```

[114]: sample_times = (
    sample_treatment[
        [
            "PATIENT_ID",
            "SAMPLE_ID",
            "SAMPLE_TYPE",
            "COLLECTION_TIME",
        ]
    ]
    .drop_duplicates()
)

primary_times = (
    sample_times.loc[
        sample_times["SAMPLE_TYPE"] == "Tumor Primary",
        [
            "PATIENT_ID",
            "SAMPLE_ID",
            "COLLECTION_TIME",
        ],
    ]
    .rename(
        columns={
            "SAMPLE_ID": "PRIMARY_SAMPLE_ID",
            "COLLECTION_TIME": "PRIMARY_COLLECTION_TIME",
        }
    )
)

```

```

r1_times = (
    sample_times.loc[
        sample_times["SAMPLE_TYPE"] == "First Recurrence",
        [
            "PATIENT_ID",
            "SAMPLE_ID",
            "COLLECTION_TIME",
        ],
    ]
    .rename(
        columns={
            "SAMPLE_ID": "FIRST_RECURRENCE_SAMPLE_ID",
            "COLLECTION_TIME": "FIRST_RECURRENCE_COLLECTION_TIME",
        }
    )
)

treatment_pairs = (
    treatment_pairs
    .merge(
        primary_times,
        on=["PATIENT_ID", "PRIMARY_SAMPLE_ID"],
        how="left",
        validate="one_to_one",
    )
    .merge(
        r1_times,
        on=["PATIENT_ID", "FIRST_RECURRENCE_SAMPLE_ID"],
        how="left",
        validate="one_to_one",
    )
)

treatment_pairs["PRIMARY_COLLECTION_TIME"] = pd.to_numeric(
    treatment_pairs["PRIMARY_COLLECTION_TIME"],
    errors="coerce",
)

treatment_pairs["FIRST_RECURRENCE_COLLECTION_TIME"] = pd.to_numeric(
    treatment_pairs["FIRST_RECURRENCE_COLLECTION_TIME"],
    errors="coerce",
)

treatment_pairs["INTERVAL_ASSESSABLE"] = (
    treatment_pairs["PRIMARY_COLLECTION_TIME"].notna()
    & treatment_pairs["FIRST_RECURRENCE_COLLECTION_TIME"].notna()
    & (

```



```

        treatment_pairs["FIRST_RECURRENCE_COLLECTION_TIME"]
        > treatment_pairs["PRIMARY_COLLECTION_TIME"]
    )
)

```

```

[115]: sample_type_map = {
        "Tumor Primary": "PRIMARY",
        "First Recurrence": "FIRST_RECURRENCE",
    }

    treatment_map = {
        "TREATMENT_TMZ": "TMZ",
        "TREATMENT_RADIOOTHERAPY": "RADIOOTHERAPY",
        "TREATMENT_CONCURRENT_TMZ": "CONCURRENT_TMZ",
    }

    for sample_type, prefix in sample_type_map.items():

        for source_col, output_name in treatment_map.items():
            status = (
                sample_treatment.loc[
                    sample_treatment["SAMPLE_TYPE"] == sample_type,
                    ["PATIENT_ID", source_col],
                ]
                .drop_duplicates("PATIENT_ID")
                .set_index("PATIENT_ID")[source_col]
                .map({
                    "Yes": True,
                    "No": False,
                })
                .astype("boolean")
            )

            treatment_pairs[f"{prefix}_{output_name}"] = (
                treatment_pairs["PATIENT_ID"]
                .map(status)
                .astype("boolean")
            )

```

```

[116]: timeline = data_timeline_treatment.copy()

    timeline["START_DATE"] = pd.to_numeric(
        timeline["START_DATE"],
        errors="coerce",
    )

    treatment_between = (

```

```

timeline
.merge(
    treatment_pairs[
        [
            "PATIENT_ID",
            "PRIMARY_COLLECTION_TIME",
            "FIRST_RECURRENCE_COLLECTION_TIME",
            "INTERVAL_ASSESSABLE",
        ]
    ],
    on="PATIENT_ID",
    how="inner",
)
)

treatment_between = treatment_between.loc[
    treatment_between["INTERVAL_ASSESSABLE"]
    & treatment_between["START_DATE"].notna()
    & (
        treatment_between["START_DATE"]
        >= treatment_between["PRIMARY_COLLECTION_TIME"]
    )
    & (
        treatment_between["START_DATE"]
        < treatment_between["FIRST_RECURRENCE_COLLECTION_TIME"]
    )
].copy()

```

```

[117]: agent_clean = (
    treatment_between["AGENT"]
    .fillna("")
    .astype(str)
    .str.strip()
    .str.lower()
)

treatment_between["IS_TMZ"] = agent_clean.isin(
    [
        "temozolomide",
        "temodar",
        "tmz",
    ]
)

treatment_between["IS_RADIOOTHERAPY"] = (
    treatment_between["TREATMENT_TYPE"]
    == "Radiation Therapy"
)

```

```
)
```

```
[118]: tmz_summary = (  
    treatment_between  
    .groupby("PATIENT_ID")["IS_TMZ"]  
    .any()  
    .rename("RECEIVED_TMZ_BETWEEN")  
    .reset_index()  
)
```

```
[119]: rt_summary = (  
    treatment_between  
    .groupby("PATIENT_ID")["IS_RADIOTHERAPY"]  
    .any()  
    .rename("RECEIVED_RADIOTHERAPY_BETWEEN")  
    .reset_index()  
)
```

```
[120]: tmz_dates = (  
    treatment_between.loc[  
        treatment_between["IS_TMZ"],  
        ["PATIENT_ID", "START_DATE"],  
    ]  
    .drop_duplicates()  
)  
  
rt_dates = (  
    treatment_between.loc[  
        treatment_between["IS_RADIOTHERAPY"],  
        ["PATIENT_ID", "START_DATE"],  
    ]  
    .drop_duplicates()  
)  
  
concurrent_patients = (  
    tmz_dates  
    .merge(  
        rt_dates,  
        on=["PATIENT_ID", "START_DATE"],  
        how="inner",  
    )["PATIENT_ID"]  
    .drop_duplicates()  
)
```

```
[121]: other_systemic = treatment_between.loc[  
    (treatment_between["TREATMENT_TYPE"] == "Chemotherapy")  
    & treatment_between["AGENT"].notna()
```

```

& ~treatment_between["IS_TMZ"]
].copy()

other_systemic_summary = (
    other_systemic
    .groupby("PATIENT_ID")["AGENT"]
    .agg(
        lambda x: ", ".join(
            pd.unique(x.dropna().astype(str))
        )
    )
    .rename("OTHER_SYSTEMIC_AGENTS")
    .reset_index()
)

other_systemic_summary["OTHER_SYSTEMIC_TREATMENTS"] = True

```

```

[122]: treatment_variables = (
    treatment_pairs
    .merge(
        tmz_summary,
        on="PATIENT_ID",
        how="left",
        validate="one_to_one",
    )
    .merge(
        rt_summary,
        on="PATIENT_ID",
        how="left",
        validate="one_to_one",
    )
    .merge(
        other_systemic_summary,
        on="PATIENT_ID",
        how="left",
        validate="one_to_one",
    )
)

```

```

[123]: boolean_columns = [
    "PRIMARY_TMZ",
    "FIRST_RECURRENCE_TMZ",
    "RECEIVED_TMZ_BETWEEN",
    "PRIMARY_RADIOOTHERAPY",
    "FIRST_RECURRENCE_RADIOOTHERAPY",
    "RECEIVED_RADIOOTHERAPY_BETWEEN",
    "PRIMARY_CONCURRENT_TMZ",

```

```

        "FIRST_RECURRENCE_CONCURRENT_TMZ",
    ]

    for col in boolean_columns:
        treatment_variables[col] = (
            treatment_variables[col]
            .astype("boolean")
        )

    treatment_variables["RECEIVED_TMZ"] = (
        treatment_variables["PRIMARY_TMZ"].fillna(False)
        | treatment_variables["FIRST_RECURRENCE_TMZ"].fillna(False)
        | treatment_variables["RECEIVED_TMZ_BETWEEN"].fillna(False)
    )

    treatment_variables["RECEIVED_RADIOOTHERAPY"] = (
        treatment_variables["PRIMARY_RADIOOTHERAPY"].fillna(False)
        | treatment_variables["FIRST_RECURRENCE_RADIOOTHERAPY"].fillna(False)
        | treatment_variables["RECEIVED_RADIOOTHERAPY_BETWEEN"].fillna(False)
    )

    treatment_variables["RECEIVED_CONCURRENT_TMZ"] = (
        treatment_variables["PRIMARY_CONCURRENT_TMZ"].fillna(False)
        | treatment_variables["FIRST_RECURRENCE_CONCURRENT_TMZ"].fillna(False)
        | treatment_variables["PATIENT_ID"].isin(concurrent_patients)
    )

    treatment_variables["OTHER_SYSTEMIC_TREATMENTS"] = (
        treatment_variables["OTHER_SYSTEMIC_TREATMENTS"]
        .astype("boolean")
        .fillna(False)
    )

```

```

[124]: final_columns = [
    "PATIENT_ID",
    "PRIMARY_SAMPLE_ID",
    "FIRST_RECURRENCE_SAMPLE_ID",
    "RECEIVED_TMZ",
    "RECEIVED_RADIOOTHERAPY",
    "RECEIVED_CONCURRENT_TMZ",
    "OTHER_SYSTEMIC_TREATMENTS",
    "OTHER_SYSTEMIC_AGENTS",
    "PRIMARY_COLLECTION_TIME",
    "FIRST_RECURRENCE_COLLECTION_TIME",
    "PRIMARY_TMZ",
    "PRIMARY_RADIOOTHERAPY",
    "PRIMARY_CONCURRENT_TMZ",

```

```

    "FIRST_RECURRENCE_TMZ",
    "FIRST_RECURRENCE_RADIOOTHERAPY",
    "FIRST_RECURRENCE_CONCURRENT_TMZ",
]

treatment_variables = treatment_variables[
    final_columns
].reset_index(drop=True)

```

```
[125]: treatment_variables.head()
```

```

[125]:
   PATIENT_ID PRIMARY_SAMPLE_ID FIRST_RECURRENCE_SAMPLE_ID RECEIVED_TMZ \
0  GLSS-19-0267  GLSS-19-0267-TP  GLSS-19-0267-R1  True
1  GLSS-19-0268  GLSS-19-0268-TP  GLSS-19-0268-R1  True
2  GLSS-19-0269  GLSS-19-0269-TP  GLSS-19-0269-R1  True
3  GLSS-19-0271  GLSS-19-0271-TP  GLSS-19-0271-R1  True
4  GLSS-19-0272  GLSS-19-0272-TP  GLSS-19-0272-R1  True

   RECEIVED_RADIOOTHERAPY RECEIVED_CONCURRENT_TMZ OTHER_SYSTEMIC_TREATMENTS \
0  True  True  False
1  True  True  False
2  True  True  False
3  True  True  True
4  True  True  True

   OTHER_SYSTEMIC_AGENTS PRIMARY_COLLECTION_TIME \
0  <NA>  0.0
1  <NA>  0.0
2  <NA>  0.0
3  Bevacizumab  0.0
4  Bevasizumab  0.0

   FIRST_RECURRENCE_COLLECTION_TIME PRIMARY_TMZ PRIMARY_RADIOOTHERAPY \
0  304.0  True  True
1  304.0  True  True
2  304.0  True  True
3  365.0  True  True
4  548.0  True  True

   PRIMARY_CONCURRENT_TMZ FIRST_RECURRENCE_TMZ \
0  True  False
1  True  False
2  True  False
3  True  False
4  True  False

   FIRST_RECURRENCE_RADIOOTHERAPY FIRST_RECURRENCE_CONCURRENT_TMZ

```

0	False	False
1	False	False
2	False	False
3	False	False
4	False	<NA>

```
[126]: glass_treatment_variables = treatment_variables.copy()

glass_treatment_variables_complex = glass_treatment_variables[[
    "PATIENT_ID",
    "PRIMARY_SAMPLE_ID",
    "FIRST_RECURRENCE_SAMPLE_ID",
    "RECEIVED_TMZ",
    "RECEIVED_RADIOOTHERAPY",
    "RECEIVED_CONCURRENT_TMZ",
    "OTHER_SYSTEMIC_TREATMENTS",
    "OTHER_SYSTEMIC_AGENTS",
    "PRIMARY_COLLECTION_TIME",
    "FIRST_RECURRENCE_COLLECTION_TIME",
    "PRIMARY_TMZ",
    "PRIMARY_RADIOOTHERAPY",
    "PRIMARY_CONCURRENT_TMZ",
    "FIRST_RECURRENCE_TMZ",
    "FIRST_RECURRENCE_RADIOOTHERAPY",
    "FIRST_RECURRENCE_CONCURRENT_TMZ"
]]
```

```
[127]: glass_treatment_variables_simple = glass_treatment_variables[[
    "PATIENT_ID",
    "PRIMARY_SAMPLE_ID",
    "FIRST_RECURRENCE_SAMPLE_ID",
    "RECEIVED_TMZ",
    "RECEIVED_RADIOOTHERAPY",
    "RECEIVED_CONCURRENT_TMZ",
    "OTHER_SYSTEMIC_TREATMENTS",
    "OTHER_SYSTEMIC_AGENTS"
]]
```

```
[128]: with pd.ExcelWriter("../data/processed/GLASS/glass_treatment_variables.
    ↪xlsx") as writer:
    glass_treatment_variables_simple.to_excel(
        writer,
        sheet_name="Simple treatment variables",
        index=False
    )

    glass_treatment_variables_complex.to_excel(
```

```

        writer,
        sheet_name="Complex Treatment variables",
        index=False
    )

```

2.10 10. Summarise the initial analysis - glass_patient_selection.png, treatment_overlap_venn.png

```

[129]: n_all = set(data_clinical_sample["PATIENT_ID"])

n_expr = set(glass_expr.columns)

sample_counts = (
    data_clinical_sample
    .groupby("PATIENT_ID")["SAMPLE_ID"]
    .nunique()
)

n_two_samples = set(sample_counts[sample_counts >= 2].index)

n_tp_r1 = set(glass_primary_R1_pairs_all["PATIENT_ID"])

n_tp_r1_expr = set(glass_primary_R1_pairs['PATIENT_ID'])

n_gbm = set(glass_gbm_primary_R1_pairs["PATIENT_ID"])

n_idhwt_gbm = set(glass_idhwt_gbm_primary_R1_pairs["PATIENT_ID"])

tmz = set(
    glass_treatment_variables.loc[
        glass_treatment_variables["RECEIVED_TMZ"],
        "PATIENT_ID"
    ]
)

radiotherapy = set(
    glass_treatment_variables.loc[
        glass_treatment_variables["RECEIVED_RADIOOTHERAPY"],
        "PATIENT_ID"
    ]
)

other_systemic = set(
    glass_treatment_variables.loc[
        glass_treatment_variables["OTHER_SYSTEMIC_TREATMENTS"],
        "PATIENT_ID"
    ]
)

```



```
)

concurrent_tmz = set(
    glass_treatment_variables.loc[
        glass_treatment_variables["RECEIVED_CONCURRENT_TMZ"],
        "PATIENT_ID"
    ]
)
)
```

```
[130]: print("Total number of patients in the GLASS study:", len(n_all))

print("Total number of samples in the GLASS study:",
      ↪data_clinical_sample["SAMPLE_ID"].nunique())

print("Total number of samples with available expression data:", len(n_expr))

print("Number of patients with at least two samples:", len(n_two_samples))

print("Number of patient with tumor primary and first recurrent samples:",
      ↪len(n_tp_r1))

print("Number of patient with tumor primary and first recurrent samples, with
      ↪available expression data:",
      len(n_tp_r1_expr))

print(
    "Number of patient with tumor primary and first recurrent samples, with
    ↪available expression data and glioblastoma diagnosis:",
    len(n_gbm))

print(
    "Number of patient with tumor primary and first recurrent samples, with
    ↪available expression data and IDH-wildtype glioblastoma:",
    len(n_idhwt_gbm))

print("Out of these patients, has received TMZ treatment:", len(tmz))

print("Out of these patients, has received radiotherapy:", len(radiotherapy))

print("Out of these patients, has received concurrent TMZ treatment and
      ↪radiotherapy:", len(concurrent_tmz))

print("Out of these patients, has received other systemic treatments:",
      ↪len(other_systemic))
```

Total number of patients in the GLASS study: 329

Total number of samples in the GLASS study: 693

Total number of samples with available expression data: 355
 Number of patients with at least two samples: 329
 Number of patient with tumor primary and first recurrent samples: 298
 Number of patient with tumor primary and first recurrent samples, with available expression data: 145
 Number of patient with tumor primary and first recurrent samples, with available expression data and glioblastoma diagnosis: 108
 Number of patient with tumor primary and first recurrent samples, with available expression data and IDH-wildtype glioblastoma: 103
 Out of these patients, has received TMZ treatment: 86
 Out of these patients, has received radiotherapy: 97
 Out of these patients, has received concurrent TMZ treatment and radiotherapy: 92
 Out of these patients, has received other systemic treatments: 25

```

[131]: flow = Digraph()

flow.attr(
    rankdir="TB",
    nodesep="0.5",
    ranksep="0.7"
)

flow.attr(
    "node",
    shape="box",
    style="rounded",
    fontname="Helvetica Neue",
    fontsize="11",
    margin="0.15"
)

flow.attr(
    "edge",
    fontname="Helvetica Neue",
    fontsize="9",
    arrowsize="0.7"
)

flow.node("A", f"All GLASS patients\nn = {len(n_all)}")
flow.node("B", f"Patients with\nTumor Primary + First Recurrence\nn = {len(n_tp_r1)}")
flow.node("C", f"Patients with\nPrimary + First Recurrence\nwith expression data\nn = {len(n_tp_r1_expr)}")
flow.node("D", f"Patients with glioblastoma\nn = {len(n_gbm)}")
flow.node("E", f"Patients with IDH-wildtype glioblastoma\nn = {len(n_idhwt_gbm)}")
  
```

```

# Arrows + number excluded at each step
flow.edge("A", "B",
          label=f"--{len(n_all - n_tp_r1)} excluded")

flow.edge("B", "C",
          label=f"--{len(n_tp_r1 - n_tp_r1_expr)} excluded")

flow.edge("C", "D",
          label=f"--{len(n_tp_r1_expr - n_gbm)} excluded")

flow.edge("D", "E",
          label=f"--{len(n_gbm - n_idhwt_gbm)} excluded")

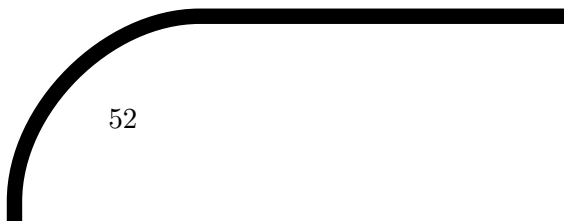
flow.attr(dpi="300")

flow.render(
    "../../figures/glass_patient_selection",
    format="png",
    cleanup=True
)

flow

```

[131]:



- Plot `glass_patient_selection.png` exported to figures directory.

```
[132]: plt.rcParams['font.family'] = 'sans-serif'
plt.rcParams['font.sans-serif'] = ['Helvetica Neue', 'Helvetica', 'Arial',
↳ 'sans-serif']
```

```
[133]: # noinspection PyTypeChecker
plt.figure(figsize=(8, 8))

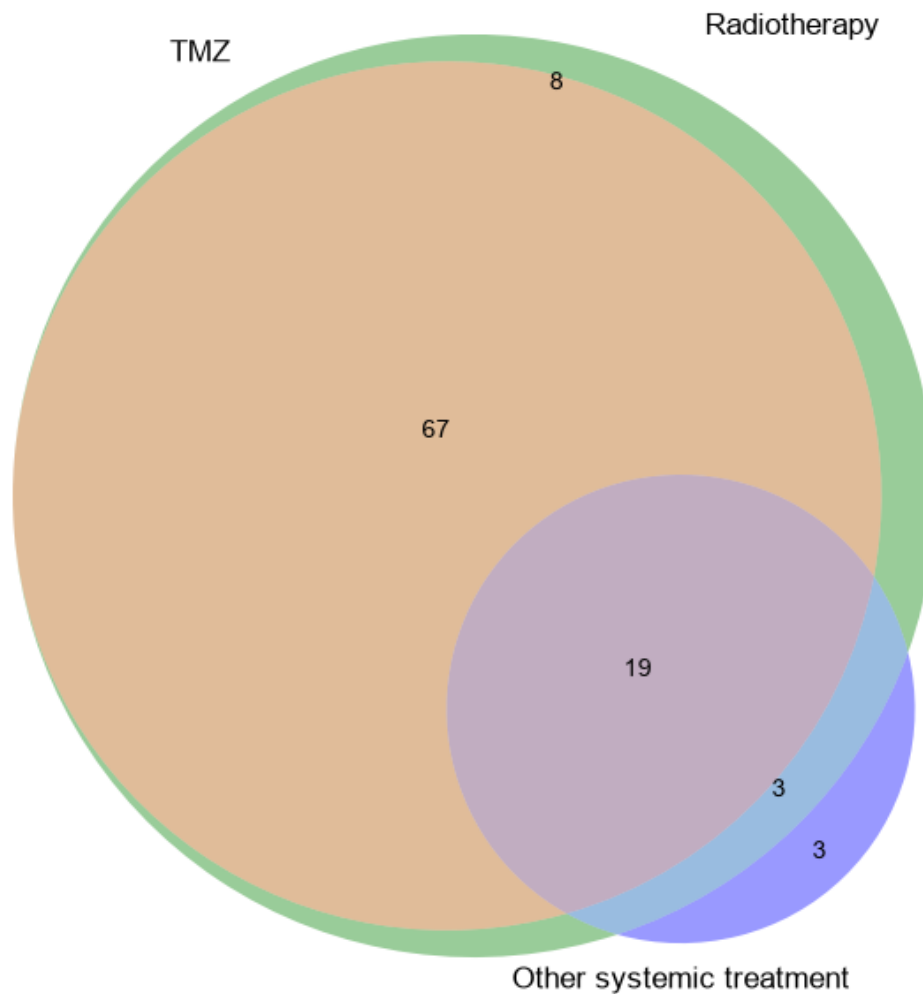
venn3(
    (tmz, radiotherapy, other_systemic),
    set_labels=(
        "TMZ",
        "Radiotherapy",
        "Other systemic treatment"
    )
)

plt.title(
    "Treatment overlap in IDH-wildtype GBM patients",
    fontsize=14,
    fontweight="bold",
    y=1.03
)

plt.savefig(
    "../figures/treatment_overlap_venn.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Treatment overlap in IDH-wildtype GBM patients



- Plot `treatment_overlap_venn.png` exported to figures directory.

3 Gene expression processing and paired analysis

3.1 1. Finalise the study cohort

- For further analysis `glass_idhwt_gbm_R1_pairs` will be used
- All patients have two samples
 - `PRIMARY_SAMPLE_ID`
 - `FIRST_RECURRENCE_SAMPLE_ID`

```
[134]: print("Number of patients in final cohort:",
        ↪glass_idhwt_gbm_primary_R1_pairs["PATIENT_ID"].nunique())
print("Number of samples in final cohort:", (
        glass_idhwt_gbm_primary_R1_pairs["PRIMARY_SAMPLE_ID"].nunique() +
        ↪glass_idhwt_gbm_primary_R1_pairs[
            "FIRST_RECURRENCE_SAMPLE_ID"].nunique()))
```

Number of patients in final cohort: 103

Number of samples in final cohort: 206

- 206 gene expression samples will be needed

3.2 2. Prepare the gene expression matrix - glass_idhwt_gbm_primary_R1_TPM.tsv

- Only keep gene expression data for samples that are part of cohort

```
[135]: sample_ids = pd.concat([
        glass_idhwt_gbm_primary_R1_pairs["PRIMARY_SAMPLE_ID"],
        glass_idhwt_gbm_primary_R1_pairs["FIRST_RECURRENCE_SAMPLE_ID"],
    ])

glass_idhwt_gbm_primary_R1_TPM = data_mrna_seq_tpm.loc[
    :, data_mrna_seq_tpm.columns.isin(sample_ids)
].copy()
```

```
[136]: glass_idhwt_gbm_primary_R1_TPM.iloc[:5, :5]
```

```
[136]:
```

	GLSS-SM-R109-TP	GLSS-19-0274-TP	GLSS-CU-R005-TP	\
Hugo_Symbol				
CMSS1	44.692557	36.213871	18.364708	
STX16	15.492818	134.774214	24.971397	
TLE4	4.712733	43.182487	21.529454	
LRRN1	19.218400	12.685700	23.226094	
YKT6	85.501630	95.440750	29.212650	

	GLSS-SM-R060-TP	GLSS-LU-00C2-TP
Hugo_Symbol		
CMSS1	38.829451	39.917940
STX16	31.478907	70.300232
TLE4	16.362683	63.985062
LRRN1	48.631390	40.922450
YKT6	102.939150	56.126976

```
[137]: glass_idhwt_gbm_primary_R1_TPM.to_csv(
        ".../data/processed/GLASS/GLASS_TPM/glass_idhwt_gbm_primary_R1_TPM.tsv",
        sep="\t",
        index=True)
```

- `glass_idhwt_gbm_primary_R1_TPM.tsv` exported to `/data/processed` directory.

3.3 3. Perform initial quality control of gene expression data - `glass_expression_QC.xlsx`

- Check:
 - Number of genes in expression matrix
 - Duplicate gene IDs
 - Missing TPM values
 - Negative TPM values
 - Number of genes with 0 expression
 - Do all samples have valid expression data?
- Sample summary table: `glass_expression_QC.xlsx`
 - Mean TPM
 - Median TPM
 - Number of genes with 0 expression
 - Maximum TPM

```
[138]: glass_idhwt_gbm_primary_R1_TPM.iloc[:5, :5]
```

```
[138]:
```

	GLSS-SM-R109-TP	GLSS-19-0274-TP	GLSS-CU-R005-TP	\
Hugo_Symbol				
CMSS1	44.692557	36.213871	18.364708	
STX16	15.492818	134.774214	24.971397	
TLE4	4.712733	43.182487	21.529454	
LRRN1	19.218400	12.685700	23.226094	
YKT6	85.501630	95.440750	29.212650	

	GLSS-SM-R060-TP	GLSS-LU-00C2-TP
Hugo_Symbol		
CMSS1	38.829451	39.917940
STX16	31.478907	70.300232
TLE4	16.362683	63.985062
LRRN1	48.631390	40.922450
YKT6	102.939150	56.126976

```
[139]: n_genes = glass_idhwt_gbm_primary_R1_TPM.index.nunique()
n_genes_duplicate_ID = int(glass_idhwt_gbm_primary_R1_TPM.index.duplicated().
    ↳sum())
duplicate_genes = glass_idhwt_gbm_primary_R1_TPM.
    ↳loc[glass_idhwt_gbm_primary_R1_TPM.index.duplicated(keep=False)].copy()
n_tpm_missing = glass_idhwt_gbm_primary_R1_TPM.isna().sum().sum()
n_tpm_neg = (glass_idhwt_gbm_primary_R1_TPM < 0).sum().sum()

zero_gene_ids = glass_idhwt_gbm_primary_R1_TPM.index[
    (glass_idhwt_gbm_primary_R1_TPM == 0).all(axis=1)]
n_genes_zero = len(zero_gene_ids)
```



```
[140]: print("Total number of gene in expression matrix:", n_genes)
print("Number of duplicate gene symbols:", n_genes_duplicate_ID)
print("Number of missing TPM values:", n_tpm_missing)
print("Number of negative TPM values:", n_tpm_neg)
print("Number of genes with 0 expression in all samples:", n_genes_zero)
```

Total number of gene in expression matrix: 35433
Number of duplicate gene symbols: 5
Number of missing TPM values: 0
Number of negative TPM values: 0
Number of genes with 0 expression in all samples: 417

```
[141]: glass_expression_QC = pd.DataFrame({
    "n_missing": glass_idhwt_gbm_primary_R1_TPM.isna().sum(axis=0),
    "n_negative": (glass_idhwt_gbm_primary_R1_TPM < 0).sum(axis=0),
    "n_0": (glass_idhwt_gbm_primary_R1_TPM == 0).sum(axis=0),
    "n_expressed": (glass_idhwt_gbm_primary_R1_TPM > 0).sum(axis=0),
    "mean_TPM": glass_idhwt_gbm_primary_R1_TPM.mean(axis=0),
    "median_TPM": glass_idhwt_gbm_primary_R1_TPM.median(axis=0),
    "max_TPM": glass_idhwt_gbm_primary_R1_TPM.max(axis=0)
})

glass_expression_QC
```

```
[141]:
```

	n_missing	n_negative	n_0	n_expressed	mean_TPM	\
GLSS-SM-R109-TP	0	0	14599	20839	28.218296	
GLSS-19-0274-TP	0	0	12885	22553	28.218297	
GLSS-CU-R005-TP	0	0	9993	25445	28.218282	
GLSS-SM-R060-TP	0	0	13723	21715	28.218293	
GLSS-LU-00C2-TP	0	0	16207	19231	28.218297	
...	...	...	...	...	...	
GLSS-HF-2934-R1	0	0	9869	25569	28.218298	
GLSS-SM-R102-R1	0	0	13945	21493	28.218296	
GLSS-SN-0016-R1	0	0	13154	22284	28.218297	
GLSS-SM-R104-R1	0	0	13591	21847	28.218293	
GLSS-SM-R063-R1	0	0	12885	22553	28.218301	
	median_TPM	max_TPM				
GLSS-SM-R109-TP	0.175742	35532.000000				
GLSS-19-0274-TP	0.476228	52824.589583				
GLSS-CU-R005-TP	0.903331	223550.000000				
GLSS-SM-R060-TP	0.195759	26746.900000				
GLSS-LU-00C2-TP	0.909260	71333.136295				
...	...	...				
GLSS-HF-2934-R1	0.427859	22819.991650				
GLSS-SM-R102-R1	0.277696	26113.100000				
GLSS-SN-0016-R1	0.354560	69470.700000				
GLSS-SM-R104-R1	0.404220	29728.800000				

```
GLSS-SM-R063-R1      0.294999    29364.900000
```

```
[206 rows x 7 columns]
```

```
[142]: glass_expression_QC.to_excel(
        ".../results/qc/glass_expression_QC.xlsx",
        index=True
    )
```

- glass_expression_QC.xlsx exported to processed data directory.

3.4 4. Log-transform TPM data

- $\log_2(\text{TPM}+1)$

```
[143]: glass_idhwt_gbm_primary_R1_log2TPM = np.log2(glass_idhwt_gbm_primary_R1_TPM + 1)

glass_idhwt_gbm_primary_R1_log2TPM = pd.DataFrame(
    glass_idhwt_gbm_primary_R1_log2TPM,
    index=glass_idhwt_gbm_primary_R1_TPM.index,
    columns=glass_idhwt_gbm_primary_R1_TPM.columns)
```

```
[144]: glass_idhwt_gbm_primary_R1_log2TPM.to_csv(
        ".../data/processed/GLASS/GLASS_TPM/glass_idhwt_gbm_primary_R1_log2TPM.
        ↪tsv",
        sep="\t",
        index=True)
```

- glass_idhwt_gbm_primary_R1_log2TPM.tsv exported to /data/processed directory.

3.5 5. Filter lowly expressed genes + duplicated gene IDs with different expression - glass_gene_filter_summary.xlsx, glass_idhwt_gbm_primary_R1_log2TPM_filtered.tsv

- Only keep genes which have TPM 1 in at least 20% of samples

```
[145]: duplicate_genes
```

```
[145]:
```

	GLSS-SM-R109-TP	GLSS-19-0274-TP	GLSS-CU-R005-TP	\
Hugo_Symbol				
MACF1	35.875793	345.421713	253.119833	
MYO18A	13.744899	201.052519	19.685016	
PALM2AKAP2	5.642198	84.328451	17.446511	
BTBD8	1.585767	11.767903	3.345687	
S1PR3	12.022653	14.633418	10.680630	
PALM2AKAP2	0.370068	6.701595	3.228221	
BTBD8	0.277189	7.778565	1.630685	
MACF1	0.289409	34.377800	23.275700	
MYO18A	2.735328	0.348697	2.492486	

S1PR3	0.174007	9.289878	0.776622
-------	----------	----------	----------

	GLSS-SM-R060-TP	GLSS-LU-00C2-TP	GLSS-LU-00B9-R1 \
Hugo_Symbol			
MACF1	82.250811	424.671342	523.716387
MYO18A	44.196672	45.662992	86.553072
PALM2AKAP2	8.773509	16.233740	20.098218
BTBD8	0.620589	10.353100	7.384311
S1PR3	4.963533	40.906740	25.097830
PALM2AKAP2	0.505997	2.441030	3.956600
BTBD8	0.108356	17.363860	6.794220
MACF1	1.700110	21.219200	39.797100
MYO18A	7.247900	6.595196	8.194554
S1PR3	0.219811	3.611069	2.593114

	TCGA-06-0210-TP	GLSS-SN-0001-R1	GLSS-MD-0022-TP \
Hugo_Symbol			
MACF1	115.420647	113.660453	166.269005
MYO18A	16.493010	34.196668	59.866167
PALM2AKAP2	5.431518	9.517328	57.348854
BTBD8	1.045340	7.531930	2.890840
S1PR3	10.429453	2.628644	21.513649
PALM2AKAP2	0.540228	7.850147	16.307479
BTBD8	0.296627	1.011498	2.625314
MACF1	1.423370	1.013510	7.489880
MYO18A	6.437210	14.634530	0.216933
S1PR3	0.144161	0.091609	0.577911

	GLSS-19-0267-R1	...	GLSS-SM-R072-R1	GLSS-HF-3050-R1 \
Hugo_Symbol		...		
MACF1	364.119465	...	143.311885	181.001482
MYO18A	161.712698	...	14.795592	41.982230
PALM2AKAP2	49.875510	...	33.442011	11.486370
BTBD8	45.246990	...	1.295480	2.498960
S1PR3	9.654271	...	43.498256	12.579289
PALM2AKAP2	21.970167	...	1.346136	1.175476
BTBD8	15.517091	...	0.569568	0.298837
MACF1	42.592100	...	3.231310	1.723060
MYO18A	0.708112	...	2.595371	9.838400
S1PR3	1.535248	...	0.401931	0.561827

	GLSS-SN-0004-R1	GLSS-SM-R061-R1	GLSS-SM-R065-R1 \
Hugo_Symbol			
MACF1	136.971422	106.376544	65.283010
MYO18A	18.627676	23.272107	48.163651
PALM2AKAP2	2.535017	13.646968	4.802581
BTBD8	0.785762	0.861516	8.227520

S1PR3	0.789166	5.178715	1.933323
PALM2AKAP2	0.946776	0.153404	3.622467
BTBD8	0.518280	0.156473	0.795930
MACF1	2.318050	0.367752	0.533984
MYO18A	13.089815	18.028210	10.602400
S1PR3	0.004535	0.099545	0.245251

	GLSS-HF-2934-R1	GLSS-SM-R102-R1	GLSS-SN-0016-R1 \
Hugo_Symbol			
MACF1	81.502869	81.563805	146.910507
MYO18A	17.333318	21.120542	12.625584
PALM2AKAP2	16.911130	43.473879	7.392694
BTBD8	0.765934	0.417101	13.334340
S1PR3	27.435734	16.778229	3.832663
PALM2AKAP2	0.691547	3.948100	2.981079
BTBD8	0.103598	0.104546	1.188153
MACF1	0.334008	1.548580	1.106420
MYO18A	7.838190	7.521550	8.310420
S1PR3	0.761108	0.174479	0.122030

	GLSS-SM-R104-R1	GLSS-SM-R063-R1
Hugo_Symbol		
MACF1	107.443032	113.789723
MYO18A	47.986422	31.277661
PALM2AKAP2	12.802884	13.012178
BTBD8	30.074080	2.071111
S1PR3	5.079324	5.248114
PALM2AKAP2	9.388189	0.209613
BTBD8	1.653882	0.257095
MACF1	1.657560	1.122860
MYO18A	3.146180	8.722710
S1PR3	0.290738	0.094757

[10 rows x 206 columns]

```
[146]: glass_idhwt_gbm_primary_R1_TPM_filtered = (
        glass_idhwt_gbm_primary_R1_TPM.loc[~glass_idhwt_gbm_primary_R1_TPM.index.
        ↪isin(duplicate_genes.index)].copy())
```

```
[147]: min_samples = math.ceil(0.20 * glass_idhwt_gbm_primary_R1_TPM_filtered.shape[1])

glass_idhwt_gbm_primary_R1_TPM_filtered = ↵
        ↪glass_idhwt_gbm_primary_R1_TPM_filtered.loc[
        (glass_idhwt_gbm_primary_R1_TPM_filtered >= 1).sum(axis=1) >= min_samples
        ]
```

```
[148]: glass_idhwt_gbm_primary_R1_log2TPM_filtered =
↳ glass_idhwt_gbm_primary_R1_log2TPM.loc[
    glass_idhwt_gbm_primary_R1_log2TPM.index.
↳ isin(glass_idhwt_gbm_primary_R1_TPM_filtered.index)].copy()
```

```
[149]: n_genes_before = glass_idhwt_gbm_primary_R1_log2TPM.index.nunique()
n_genes_after = glass_idhwt_gbm_primary_R1_log2TPM_filtered.index.nunique()
n_genes_excluded = n_genes_before - n_genes_after

glass_gene_filter_summary = pd.DataFrame({
    "n_genes_before_filtering": [n_genes_before],
    "n_genes_after_filtering": [n_genes_after],
    "n_genes_excluded": [n_genes_excluded],
})

glass_gene_filter_summary
```

```
[149]:      n_genes_before_filtering  n_genes_after_filtering  n_genes_excluded
0                                35433                    17789             17644
```

```
[150]: glass_gene_filter_summary.to_excel(
    "../../results/qc/glass_gene_filter_summary.xlsx",
    index=False)
```

```
[151]: glass_idhwt_gbm_primary_R1_log2TPM_filtered.to_csv(
    "../../data/processed/GLASS/GLASS_TPM/
↳ glass_idhwt_gbm_primary_R1_log2TPM_filtered.tsv",
    sep="\t",
    index=True)
```

- Final glass_gene_filter_summary.xlsx and glass_idhwt_gbm_primary_R1_log2TPM_filtered.tsv exported to /data/processed directory.

3.6 6. Assess global gene expression patterns across samples - PCA_primary_recurrence.png

```
[152]: pca_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=2))])

glass_idhwt_gbm_primary_R1_pca = pca_pipeline.fit_transform(
    glass_idhwt_gbm_primary_R1_log2TPM_filtered.T)
```

```
[153]: sample_types = [
    "Primary"
    if sample_id in glass_idhwt_gbm_primary_R1_pairs["PRIMARY_SAMPLE_ID"].values
    else "First Recurrence"
    for sample_id in glass_idhwt_gbm_primary_R1_log2TPM_filtered.columns]
```

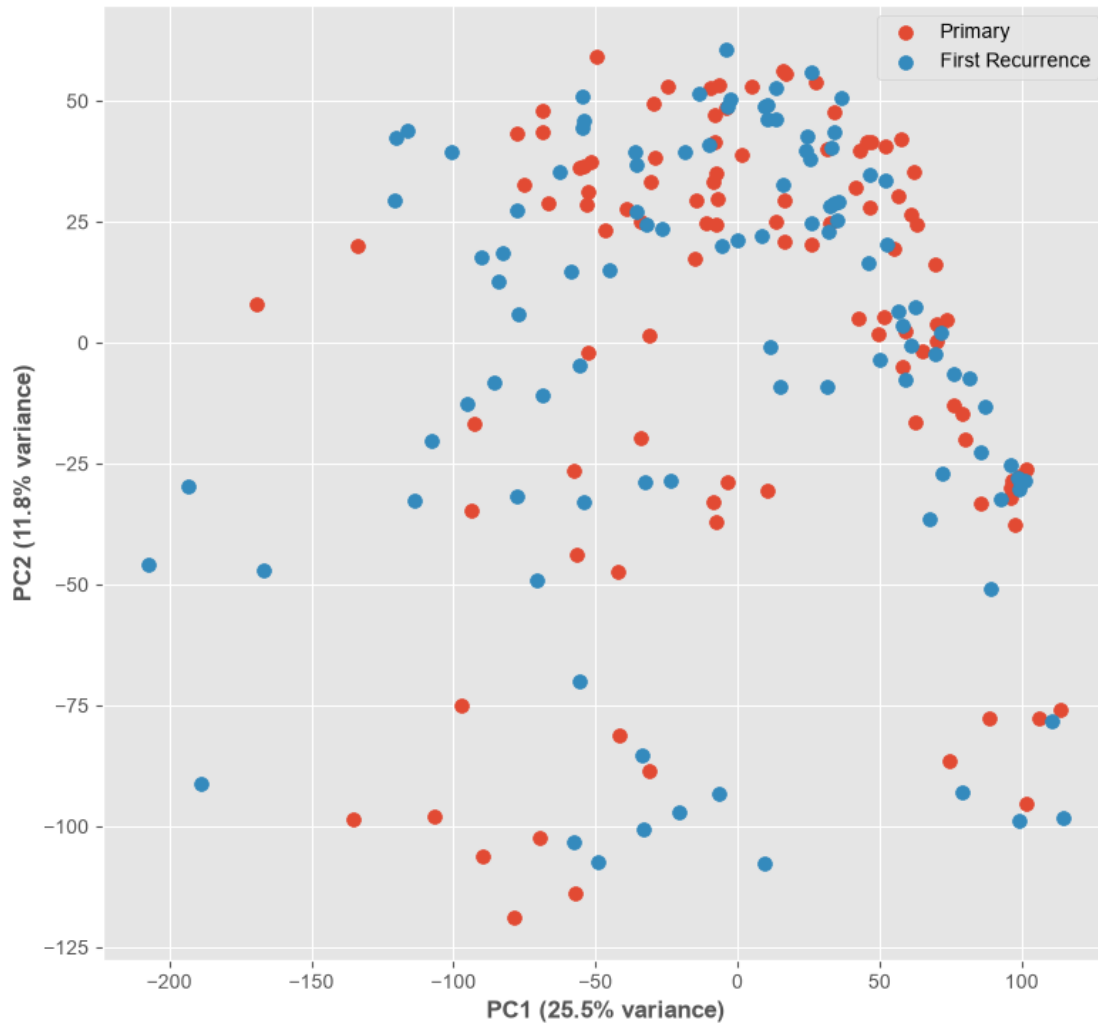
```

plt.figure(figsize=(8, 8))
for sample_type, marker in [
    ("Primary", "o"),
    ("First Recurrence", "o")
]:
    mask = [sample_type == x for x in sample_types]
    plt.scatter(
        glass_idhwt_gbm_primary_R1_pca[mask, 0],
        glass_idhwt_gbm_primary_R1_pca[mask, 1],
        marker=marker,
        s=50,
        label=sample_type)
pca = pca_pipeline.named_steps["pca"]
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0] * 100:.1f}% variance)",
    fontweight="bold")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1] * 100:.1f}% variance)",
    fontweight="bold")
plt.title(
    "Principal Component Analysis of Cohort Expression Profiles",
    fontsize=16,
    fontweight="bold",
    loc='center',
    y=1.05)
plt.legend()
plt.tight_layout()
plt.savefig(
    "../../figures/PCA_primary_recurrence.png",
    dpi=300,
    bbox_inches="tight")

plt.show()

```

Principal Component Analysis of Cohort Expression Profiles



- Plot PCA_primary_recurrence.png exported to figures directory.

3.7 7. Perform paired comparison of gene expression between primary and recurrent samples - glass_primary_vs_R1_gene_statistics.xlsx

- $\text{delta_expression} = \text{R1 expression} - \text{Primary expression}$

```
[154]: primary = □
        ↪ glass_idhwt_gbm_primary_R1_log2TPM_filtered[glass_idhwt_gbm_primary_R1_pairs["PRIMARY_SAMPL
r1 = □
        ↪ glass_idhwt_gbm_primary_R1_log2TPM_filtered[glass_idhwt_gbm_primary_R1_pairs["FIRST_RECURRE
delta_expression = r1.to_numpy() - primary.to_numpy()
```

```
[155]: genes = glass_idhwt_gbm_primary_R1_log2TPM_filtered.index

print("Number of genes:", delta_expression.shape[0])
print("Number of patient pairs:", delta_expression.shape[1])

print("Primary samples:", primary.shape[1])
print("R1 samples:", r1.shape[1])

assert primary.shape[1] == r1.shape[1]

print("Duplicated Primary IDs:", int(primary.columns.duplicated().sum()))

print("Duplicated R1 IDs:", int(r1.columns.duplicated().sum()))
```

```
Number of genes: 17789
Number of patient pairs: 103
Primary samples: 103
R1 samples: 103
Duplicated Primary IDs: 0
Duplicated R1 IDs: 0
```

```
[156]: genes = glass_idhwt_gbm_primary_R1_log2TPM_filtered.index

skewness: list[float] = []

for delta in delta_expression:
    valid_delta = delta[~np.isnan(delta)]

    if len(valid_delta) >= 3:
        skewness.append(skew(valid_delta))
    else:
        skewness.append(np.nan)

normality_check = pd.DataFrame({
    "GENE": genes,
    "SKEWNESS": skewness})

print(f"Genes: {len(genes)}")
print(f"Patient pairs: {delta_expression.shape[1]}")
print()
median_abs_skew = normality_check["SKEWNESS"].abs().median()

print(f"Median |skewness|: {median_abs_skew:.3f}")

abs_skewness = normality_check["SKEWNESS"].abs()
n_skew_gt_1 = int(np.count_nonzero(abs_skewness.to_numpy() > 1))
n_skew_gt_2 = int(np.count_nonzero(abs_skewness.to_numpy() > 2))
```



```
print(f"Genes with |skewness| > 1: {n_skew_gt_1}")
print(f"Genes with |skewness| > 2: {n_skew_gt_2}")
```

Genes: 17789

Patient pairs: 103

Median |skewness|: 0.294

Genes with |skewness| > 1: 1213

Genes with |skewness| > 2: 127

Methodological Note: Following $\log_2$ -transformation, paired expression differences exhibited low overall skewness across genes (median absolute skewness = 0.29). Combined with a sample size of $N = 103$ paired observations, this justified the use of parametric paired t -tests under the Central Limit Theorem.

```
[157]: genes = glass_idhwt_gbm_primary_R1_log2TPM_filtered.index

n_pairs = np.sum(~np.isnan(delta_expression), axis=1)

mean_primary = np.nanmean(primary.to_numpy(), axis=1)

mean_r1 = np.nanmean(r1.to_numpy(), axis=1)

mean_delta = np.nanmean(delta_expression, axis=1)

median_delta = np.nanmedian(delta_expression, axis=1)

sd_delta = np.nanstd(delta_expression, axis=1, ddof=1)

n_pairs = np.asarray(n_pairs, dtype=int)

t_statistic = mean_delta / (sd_delta / np.sqrt(n_pairs))

t_statistic = np.asarray(t_statistic, dtype=float)

p_value = np.asarray(
    2 * t.sf(
        np.abs(t_statistic),
        df=n_pairs - 1
    ),
    dtype=float
)

invalid = (
    (n_pairs < 2) |
    ~np.isfinite(t_statistic))

t_statistic[invalid] = np.nan
```

```

p_value[invalid] = np.nan

glass_primary_vs_R1_gene_statistics = pd.DataFrame({
    "GENE": genes,
    "N_PAIRS": n_pairs,
    "MEAN_PRIMARY": mean_primary,
    "MEAN_R1": mean_r1,
    "MEAN_DELTA": mean_delta,
    "MEDIAN_DELTA": median_delta,
    "SD_DELTA": sd_delta,
    "T_STATISTIC": t_statistic,
    "P_VALUE": p_value
})

glass_primary_vs_R1_gene_statistics["FDR"] = np.nan
valid_p = glass_primary_vs_R1_gene_statistics["P_VALUE"].notna()

```

```

[158]: glass_primary_vs_R1_gene_statistics.loc[valid_p, "FDR"] = (
        multipletests(
            glass_primary_vs_R1_gene_statistics.loc[valid_p, "P_VALUE"],
            method="fdr_bh"
        )[1])

```

```

[159]: glass_primary_vs_R1_gene_statistics

```

```

[159]:
      GENE  N_PAIRS  MEAN_PRIMARY  MEAN_R1  MEAN_DELTA  \
0      CMSS1      103      5.227101  5.120401   -0.106700
1      STX16      103      5.364055  5.375362    0.011307
2       TLE4      103      4.458565  4.746619    0.288054
3      LRRN1      103      4.574455  3.751107   -0.823348
4       YKT6      103      6.312393  6.135957   -0.176435
...      ...      ...      ...      ...      ...
17784  RP1-265C24.5      103      0.531361  0.782207    0.250847
17785  AC009060.1      103      4.012625  4.003522   -0.009103
17786  TMEM88B      103      1.092288  2.022297    0.930009
17787  BMP2      103      2.178855  1.925957   -0.252899
17788  EP300      103      3.668214  3.571420   -0.096794

      MEDIAN_DELTA  SD_DELTA  T_STATISTIC  P_VALUE  FDR
0      -0.139256  0.779057   -1.390001  0.167555  0.321292
1      -0.030965  0.708218    0.162028  0.871605  0.928024
2       0.120028  0.923852    3.164391  0.002049  0.017961
3      -0.830165  1.662280   -5.026871  0.000002  0.000189
4      -0.167784  0.891261   -2.009092  0.047169  0.138214
...      ...      ...      ...      ...      ...
17784     0.169054  0.741857    3.431681  0.000868  0.010120
17785    -0.090453  0.869126   -0.106299  0.915554  0.953504

```

17786	0.209741	1.954603	4.828887	0.000005	0.000318
17787	-0.201903	1.159858	-2.212892	0.029134	0.100967
17788	-0.012204	0.828893	-1.185137	0.238717	0.407270

[17789 rows x 10 columns]

```
[160]: glass_primary_vs_R1_gene_statistics.to_excel(
        ".../results/differential_expression/glass_primary_vs_R1_gene_statistics.
        ↪xlsx",
        index=False
    )
```

- glass_primary_vs_R1_gene_statistics.xlsx exported to /results/differential_expression directory.

3.8 8. Assess the consistency of expression changes

- FRACTION_UP = ratio of patients where R1 > Primary
- FRACTION_DOWN = ratio of patients where R1 < Primary

```
[161]: expr = glass_idhwt_gbm_primary_R1_log2TPM_filtered
pairs = glass_idhwt_gbm_primary_R1_pairs
up_counts = pd.Series(0, index=expr.index, dtype=float)
down_counts = pd.Series(0, index=expr.index, dtype=float)
n_valid = pd.Series(0, index=expr.index, dtype=float)
```

```
[162]: for _, row in pairs.iterrows():
        primary_id = row["PRIMARY_SAMPLE_ID"]
        r1_id = row["FIRST_RECURRENCE_SAMPLE_ID"]
        if primary_id not in expr.columns or r1_id not in expr.columns:
            continue
        primary = expr[primary_id]
        r1 = expr[r1_id]
        valid = primary.notna() & r1.notna()
        up_counts += ((r1 > primary) & valid)
        down_counts += ((r1 < primary) & valid)
        n_valid += valid
```

```
[163]: fractions_up = up_counts / n_valid
fractions_down = down_counts / n_valid

glass_primary_vs_R1_gene_statistics["FRACTION_UP"] = ↵
↪(glass_primary_vs_R1_gene_statistics["GENE"].map(fractions_up))
glass_primary_vs_R1_gene_statistics["FRACTION_DOWN"] = ↵
↪(glass_primary_vs_R1_gene_statistics["GENE"].map(fractions_down))
```

```
[164]: glass_primary_vs_R1_gene_statistics.head()
```

```
[164]:
```

	GENE	N_PAIRS	MEAN_PRIMARY	MEAN_R1	MEAN_DELTA	MEDIAN_DELTA	SD_DELTA	\
0	CMSS1	103	5.227101	5.120401	-0.106700	-0.139256	0.779057	
1	STX16	103	5.364055	5.375362	0.011307	-0.030965	0.708218	
2	TLE4	103	4.458565	4.746619	0.288054	0.120028	0.923852	
3	LRRN1	103	4.574455	3.751107	-0.823348	-0.830165	1.662280	
4	YKT6	103	6.312393	6.135957	-0.176435	-0.167784	0.891261	

	T_STATISTIC	P_VALUE	FDR	FRACTION_UP	FRACTION_DOWN
0	-1.390001	0.167555	0.321292	0.436893	0.563107
1	0.162028	0.871605	0.928024	0.485437	0.514563
2	3.164391	0.002049	0.017961	0.572816	0.427184
3	-5.026871	0.000002	0.000189	0.262136	0.737864
4	-2.009092	0.047169	0.138214	0.359223	0.640777

3.9 9. Generate an initial list of recurrence-associated genes

- Recurrence-UP genes:
 - FDR < 0.05
 - MEAN_DELTA >= 0.5
 - FRACTION_UP >= 0.65
- Recurrence-DOWN genes:
 - FDR < 0.05
 - MEAN_DELTA <= -0.5
 - FRACTION_DOWN >= 0.65

```
[165]: glass_recurrence_UP_initial = glass_primary_vs_R1_gene_statistics[
    (glass_primary_vs_R1_gene_statistics["FDR"] < 0.05) &
    (glass_primary_vs_R1_gene_statistics["MEAN_DELTA"] >= 0.5) &
    (glass_primary_vs_R1_gene_statistics["FRACTION_UP"] >= 0.65)
].copy()

glass_recurrence_DOWN_initial = glass_primary_vs_R1_gene_statistics[
    (glass_primary_vs_R1_gene_statistics["FDR"] < 0.05) &
    (glass_primary_vs_R1_gene_statistics["MEAN_DELTA"] <= -0.5) &
    (glass_primary_vs_R1_gene_statistics["FRACTION_DOWN"] >= 0.65)
].copy()
```

```
[166]: glass_recurrence_UP_initial.head()
```

```
[166]:
```

	GENE	N_PAIRS	MEAN_PRIMARY	MEAN_R1	MEAN_DELTA	MEDIAN_DELTA	\
12	FAM153C	103	1.700160	2.373487	0.673328	0.451208	
59	RGS4	103	3.071584	3.844087	0.772503	0.984126	
66	MICAL2	103	3.602032	4.212899	0.610867	0.501128	
72	CRHBP	103	1.315011	2.092998	0.777987	0.715091	
75	C16orf89	103	3.381400	4.001647	0.620247	0.572217	

	SD_DELTA	T_STATISTIC	P_VALUE	FDR	FRACTION_UP	FRACTION_DOWN
12	1.765662	3.870236	0.000192	0.003732	0.650485	0.349515

59	2.133532	3.674682	0.000382	0.005819	0.669903	0.330097
66	1.473130	4.208468	0.000055	0.001594	0.660194	0.339806
72	1.520816	5.191757	0.000001	0.000125	0.650485	0.339806
75	1.376968	4.571510	0.000014	0.000614	0.650485	0.349515

```
[167]: glass_recurrence_DOWN_initial.head()
```

```
[167]:
```

	GENE	N_PAIRS	MEAN_PRIMARY	MEAN_R1	MEAN_DELTA	MEDIAN_DELTA	\
3	LRRN1	103	4.574455	3.751107	-0.823348	-0.830165	
243	TTK	103	2.826585	2.183419	-0.643165	-0.642103	
346	MEST	103	6.760819	6.202194	-0.558625	-0.496377	
405	SGOL1	103	2.618184	1.944617	-0.673567	-0.722484	
495	HIGD1B	103	4.127535	3.122056	-1.005479	-0.994245	

	SD_DELTA	T_STATISTIC	P_VALUE	FDR	FRACTION_UP	\
3	1.662280	-5.026871	2.137105e-06	1.894612e-04	0.262136	
243	1.434949	-4.548884	1.490681e-05	6.467738e-04	0.339806	
346	1.591468	-3.562388	5.603924e-04	7.638943e-03	0.349515	
405	1.463245	-4.671777	9.142486e-06	4.797513e-04	0.349515	
495	1.157281	-8.817644	3.339109e-14	5.842423e-10	0.223301	

	FRACTION_DOWN
3	0.737864
243	0.660194
346	0.650485
405	0.650485
495	0.776699

```
[168]: glass_recurrence_UP_initial.to_excel(
        ".../results/recurrence/glass_recurrence_UP_initial.xlsx",
        index=False)

glass_recurrence_DOWN_initial.to_excel(
        ".../results/recurrence/glass_recurrence_DOWN_initial.xlsx",
        index=False)
```

- glass_recurrence_UP_initial.xlsx and glass_recurrence_DOWN_initial.xlsx exported to /data/processed directory.

3.10 10. Generate initial result plots - glass_recurrence_vulcano.png, glass_recurrence_heatmap.png glass_recurrence_paired_plots.png

- Vulcano plot
- Heatmap
- Some top genes' paired plot

```
[169]: #Vulcano plot
stats = glass_primary_vs_R1_gene_statistics.copy()
```

```

stats["GENE"] = glass_idhwt_gbm_primary_R1_log2TPM_filtered.index.values
stats["NEG_LOG10_FDR"] = -np.log10(stats["FDR"].clip(lower=1e-300))

up_genes = set(glass_recurrence_UP_initial["GENE"])
down_genes = set(glass_recurrence_DOWN_initial["GENE"])

stats["CATEGORY"] = "Other"
stats.loc[stats["GENE"].isin(up_genes), "CATEGORY"] = "Recurrence-UP"
stats.loc[stats["GENE"].isin(down_genes), "CATEGORY"] = "Recurrence-DOWN"

fig, ax = plt.subplots(figsize=(10, 8))
other = stats["CATEGORY"] == "Other"
up = stats["CATEGORY"] == "Recurrence-UP"
down = stats["CATEGORY"] == "Recurrence-DOWN"
ax.scatter(
    stats.loc[other, "MEAN_DELTA"],
    stats.loc[other, "NEG_LOG10_FDR"],
    s=12,
    alpha=0.5,
    label="Other")
ax.scatter(
    stats.loc[up, "MEAN_DELTA"],
    stats.loc[up, "NEG_LOG10_FDR"],
    s=35,
    alpha=0.8,
    label="Recurrence-UP")
ax.scatter(
    stats.loc[down, "MEAN_DELTA"],
    stats.loc[down, "NEG_LOG10_FDR"],
    s=35,
    alpha=0.8,
    label="Recurrence-DOWN")
ax.axvline(0.5, linestyle="--", linewidth=1)
ax.axvline(-0.5, linestyle="--", linewidth=1)
ax.axhline(-np.log10(0.05), linestyle="--", linewidth=1)
ax.set_xlabel("MEAN_DELTA")
ax.set_ylabel("-log10(FDR)")
ax.set_title("Primary vs First Recurrence")
ax.legend()
top_up = (
    stats[stats["CATEGORY"] == "Recurrence-UP"]
    .sort_values("FDR")
    .head(10))
top_down = (
    stats[stats["CATEGORY"] == "Recurrence-DOWN"]
    .sort_values("FDR")
    .head(10))

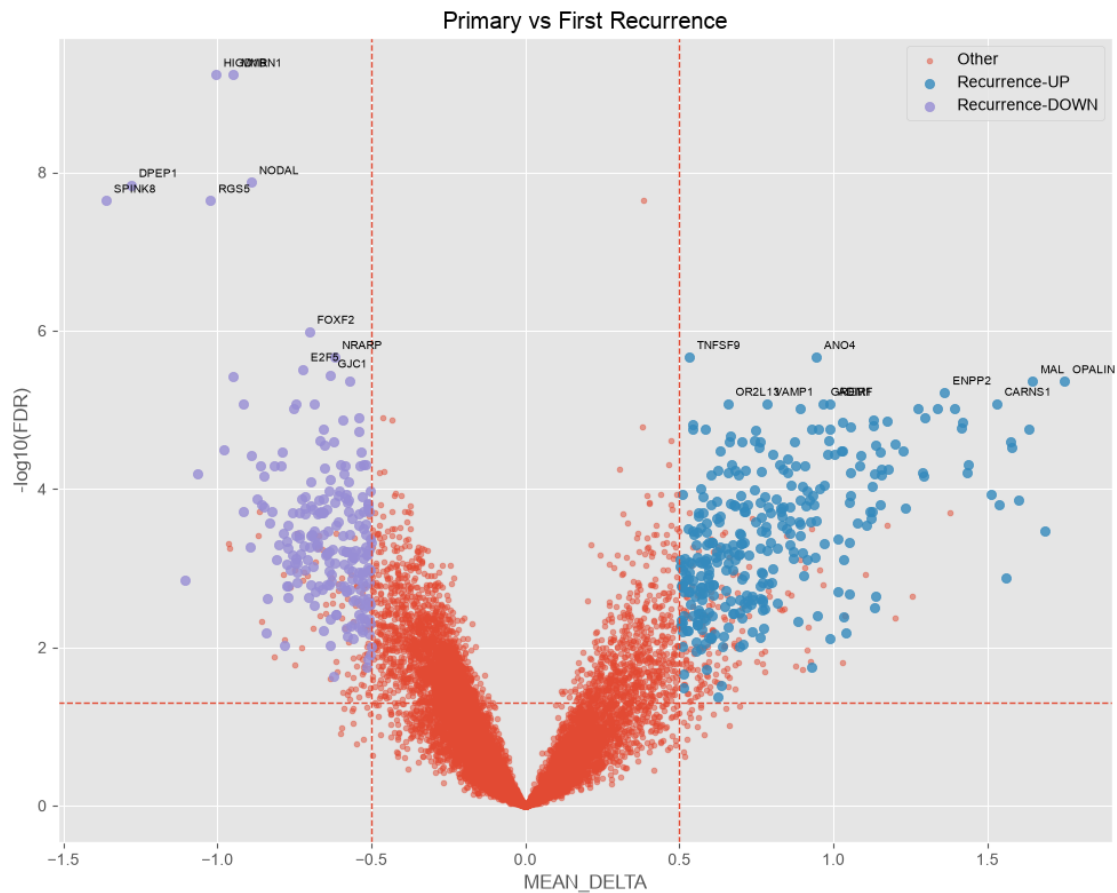
```

```

for _, row in pd.concat([top_up, top_down]).iterrows():
    ax.annotate(
        row["GENE"],
        (row["MEAN_DELTA"], row["NEG_LOG10_FDR"]),
        xytext=(5, 5),
        textcoords="offset points",
        fontsize=7)
plt.tight_layout()
plt.savefig(
    "../../figures/glass_recurrence_volcano.png",
    dpi=300,
    bbox_inches="tight")

plt.show()

```



- Plot glass_recurrence_vulcano.png exported to figures directory.

```

[170]: #Heatmap
top_up = (

```

```

    glass_recurrence_UP_initial
    .sort_values("MEAN_DELTA", ascending=False)
    .head(10)["GENE"]
    .tolist()
)
top_down = (
    glass_recurrence_DOWN_initial
    .sort_values("MEAN_DELTA", ascending=True)
    .head(10)["GENE"]
    .tolist()
)

heatmap_genes = top_up + top_down

heatmap_expr = glass_idhwt_gbm_primary_R1_log2TPM_filtered

ordered_samples = []

for _, row in glass_idhwt_gbm_primary_R1_pairs.iterrows():

    primary = row["PRIMARY_SAMPLE_ID"]
    recurrence = row["FIRST_RECURRENCE_SAMPLE_ID"]

    if primary in heatmap_expr.columns:
        ordered_samples.append(primary)

    if recurrence in heatmap_expr.columns:
        ordered_samples.append(recurrence)

ordered_samples = [
    sample
    for sample in ordered_samples
    if sample in heatmap_expr.columns
]

heatmap_expr = heatmap_expr.loc[heatmap_genes, ordered_samples]

heatmap_gene_order = [
    gene
    for gene in heatmap_genes
    if gene in heatmap_expr.index
]

heatmap_expr = heatmap_expr.loc[heatmap_gene_order]

def draw_curly_brace_right(

```



```

        ax, y_min, y_max, x_pos=1.08, w=0.015, color="black", label=""
):
    """Draws a right-facing curly brace '}' positioned at x_pos on the right_
    ↪side of the heatmap."""
    trans = ax.get_yaxis_transform()

    y1 = y_min + 0.1
    y2 = y_max + 0.9
    ymid = (y1 + y2) / 2.0
    r = min(0.5, (y2 - y1) / 4.0)

    verts = [
        (x_pos, y1),
        (x_pos, y1 + r / 2),
        (x_pos + w / 2, y1 + r / 2),
        (x_pos + w / 2, y1 + r),
        (x_pos + w / 2, ymid - r),
        (x_pos + w / 2, ymid - r / 2),
        (x_pos + w, ymid - r / 2),
        (x_pos + w, ymid),
        (x_pos + w, ymid + r / 2),
        (x_pos + w / 2, ymid + r / 2),
        (x_pos + w / 2, ymid + r),
        (x_pos + w / 2, y2 - r),
        (x_pos + w / 2, y2 - r / 2),
        (x_pos, y2 - r / 2),
        (x_pos, y2),
    ]

    codes = [
        mpath.Path.MOVETO,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.LINETO,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.LINETO,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
        mpath.Path.CURVE4,
    ]

```

```

path = mpath.Path(verts, codes)
patch = patches.PathPatch(
    path,
    facecolor="none",
    edgecolor=color,
    lw=2,
    transform=trans,
    clip_on=False,
)
ax.add_patch(patch)

if label:
    ax.text(
        x_pos + w + 0.01,
        ymid,
        label,
        transform=trans,
        color=color,
        fontweight="bold",
        fontsize=10,
        va="center",
        ha="left",
    )

gene_direction = pd.Series("UP", index=top_up)
gene_direction = pd.concat([
    gene_direction,
    pd.Series(
        "DOWN",
        index=top_down
    )
])
gene_direction = gene_direction.loc[
    heatmap_expr.index
]

p = sns.clustermap(
    heatmap_expr,
    z_score=0,
    row_cluster=True,
    col_cluster=False,
    cmap="viridis",
    center=0,
    vmin=-2,
    vmax=2,
    figsize=(20, 10),

```

```

    dendrogram_ratio=(
        0.12,
        0.02,
    ),
    colors_ratio=0.02,
    cbar_pos=(1.05, 0.01, 0.025, 0.96),
    xticklabels=False,
    yticklabels=True,
)

for i in range(2, heatmap_expr.shape[1], 2):
    p.ax_heatmap.axvline(i, color="white", linewidth=2)

if p.dendrogram_row is None:
    raise RuntimeError("Row dendrogram was not created.")
reordered_genes = heatmap_expr.index[p.dendrogram_row.reordered_ind]

up_indices = [i for i, gene in enumerate(reordered_genes) if gene in top_up]
down_indices = [i for i, gene in enumerate(reordered_genes) if gene in top_down]

if up_indices:
    draw_curly_brace_right(
        p.ax_heatmap,
        min(up_indices),
        max(up_indices),
        x_pos=1.06,
        color="#808588",
        label="UP",
    )

if down_indices:
    draw_curly_brace_right(
        p.ax_heatmap,
        min(down_indices),
        max(down_indices),
        x_pos=1.06,
        color="#808588",
        label="DOWN",
    )

p.ax_heatmap.tick_params(axis="y", labelsiz=7)
p.ax_heatmap.set_xlabel(
    f"Paired Samples: Primary | First recurrence (n={heatmap_expr.shape[1] // 2} pairs)",
    fontsize=15,
    fontweight="bold",
    labelpad=10
)

```

```

)
p.ax_heatmap.set_ylabel(
    f"Hugo Symbol",
    fontweight="bold",
    fontsize=15
)
p.ax_heatmap.set_title(
    f"Z-Score Expression of Top Recurrence-Regulated Genes",
    fontsize=25,
    fontweight="bold",
    y=1.03
)

plt.savefig(
    "../../figures/glass_recurrence_heatmap.png",
    dpi=600,
    bbox_inches="tight"
)

plt.show()

```



- Plot glass_recurrence_heatmap.png exported to figures directory.

```

[171]: #Paired plot
expr = glass_idhwt_gbm_primary_R1_log2TPM_filtered

top_up_plot = (
    glass_recurrence_UP_initial
    .sort_values(

```

```

        ["FRACTION_UP", "MEAN_DELTA"],
        ascending=[False, False])
    .head(5)["GENE"]
    .tolist())

top_down_plot = (
    glass_recurrence_DOWN_initial
    .sort_values(
        ["FRACTION_DOWN", "MEAN_DELTA"],
        ascending=[False, True])
    .head(5)["GENE"]
    .tolist())

plot_genes = top_up_plot + top_down_plot

fig, axes = plt.subplots(2, 5, figsize=(18, 8))
for ax, gene in zip(axes.flat, plot_genes):
    is_up = gene in top_up_plot
    for _, row in pairs.iterrows():
        primary_id = row["PRIMARY_SAMPLE_ID"]
        r1_id = row["FIRST_RECURRENCE_SAMPLE_ID"]
        if primary_id not in expr.columns or r1_id not in expr.columns:
            continue
        primary_value = expr.loc[gene, primary_id]
        r1_value = expr.loc[gene, r1_id]
        if pd.isna(primary_value) or pd.isna(r1_value):
            continue
        ax.plot(
            [0, 1],
            [primary_value, r1_value],
            marker="o",
            alpha=0.35,
            linewidth=0.8
        )
    ax.set_xticks([0, 1])
    ax.set_xticklabels(["Primary", "R1"])
    ax.set_title(gene, fontsize=9)
    ax.set_xlim(-0.1, 1.1)
    ax.grid(axis="y", alpha=0.2)

plt.suptitle(
    "Patient-level paired expression changes",
    fontsize=25,
    fontweight="bold",
    y=1.03)
plt.tight_layout()
plt.savefig(

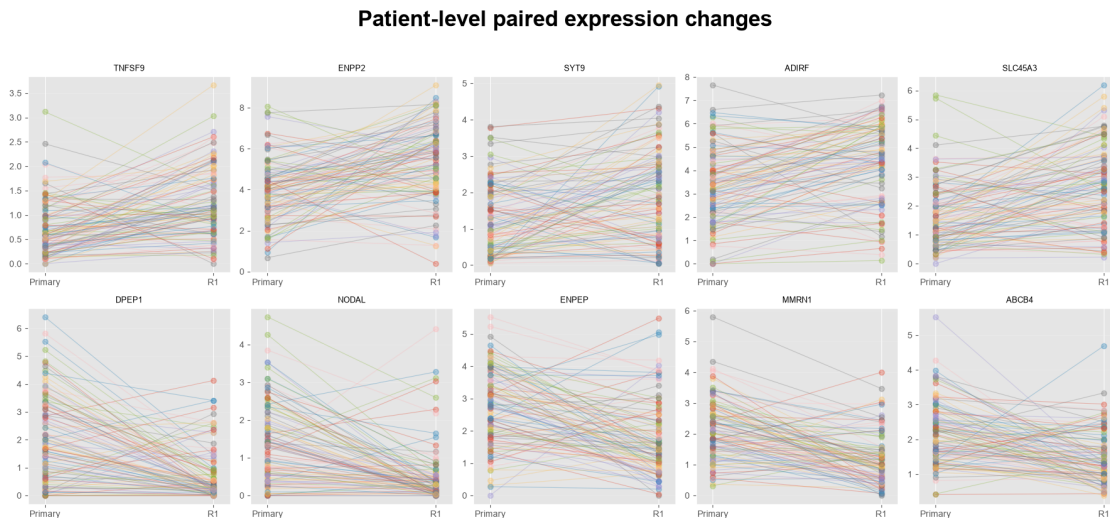
```

```

"../../figures/glass_recurrence_paired_plots.png",
dpi=300,
bbox_inches="tight")

plt.show()

```



- Plot `glass_recurrence_paired_plots.png` exported to figures directory.

4 Identifying biological processes associated with these changes

- Input data: `glass_primary_vs_R1_gene_statistics`

4.1 1. Pathway analysis: Biological processes associated with recurrence

Gene-level paired *t*-statistics were used to rank genes for pre-ranked gene set enrichment analysis (GSEA). MSigDB Hallmark (2020) and Reactome (2022) gene sets were analysed using the multilevel algorithm implemented in GSEAPy (v1.3.1).

Outputs: - `glass_GSEA_Hallmark.xlsx` - `glass_GSEA_Reactome.xlsx`

```

[172]: glass_primary_vs_R1_gene_statistics = pd.read_excel(
        "../../results/differential_expression/glass_primary_vs_R1_gene_statistics.
        ↪xlsx", sheet_name=0)

```

4.1.1 1.1 Generate ranked gene list

Genes were ranked according to the *t*-statistic from the primary tumour vs. R1 comparison. Genes with missing statistics were removed, duplicate gene symbols were removed, and the remaining genes were ranked from highest to lowest *t*-statistic.

```
[173]: stats = glass_primary_vs_R1_gene_statistics.copy()
```

```
ranked_stats = (  
    stats[["GENE", "T_STATISTIC"]]  
    .dropna()  
    .drop_duplicates("GENE")  
    .sort_values("T_STATISTIC", ascending=False)  
)  
  
rnk = ranked_stats.set_index("GENE")["T_STATISTIC"]
```

```
[174]: rnk.head()
```

```
[174]: GENE  
P2RY2      7.751049  
ANO4       6.704227  
TNFSF9     6.679159  
MAL        6.442460  
OPALIN     6.430496  
Name: T_STATISTIC, dtype: float64
```

```
[175]: rnk.tail()
```

```
[175]: GENE  
RGS5      -7.733102  
DPEP1     -7.906913  
NODAL     -7.980845  
MMRN1     -8.683950  
HIGD1B    -8.817644  
Name: T_STATISTIC, dtype: float64
```

```
[176]: print("Number of ranked genes:", len(rnk))
```

Number of ranked genes: 17789

4.1.2 1.2 Preranked GSEA

```
[177]: hallmark = gp.get_library(  
    name="MSigDB_Hallmark_2020",  
    organism="Human")  
  
print(len(hallmark))  
print(list(hallmark.keys())[:5])
```

50

['TNF-alpha Signaling via NF-kB', 'Hypoxia', 'Cholesterol Homeostasis', 'Mitotic Spindle', 'Wnt-beta Catenin Signaling']

[178]: *# noinspection bad-argument-type*

```
gsea = gp.prerank(  
    rnk=rnk,  
    gene_sets=hallmark,  
    outdir=None,  
    seed=42,  
    verbose=True,  
    method='multilevel'  
)
```

2026-10-05 17:16:52,321 [WARNING] Duplicated values found in preranked stats:
0.31% of genes

The order of those genes will be arbitrary, which may produce unexpected
results.

2026-10-05 17:16:52,323 [INFO] Parsing data files for
GSEA...

2026-10-05 17:16:52,401 [INFO] 0000 gene_sets have been filtered out when
max_size=500 and min_size=15

2026-10-05 17:16:52,403 [INFO] 0050 gene_sets used for further statistical
testing...

2026-10-05 17:16:52,407 [INFO] Start to run GSEA... Might take a
while...

2026-10-05 17:16:55,750 [INFO] Congratulations. GSEApY runs
successfully...

[179]: `hallmark_results = gsea.res2d[["Term", "NES", "NOM p-val", "FDR q-val",
↪ "Lead_genes"]].copy()`

```
hallmark_results = (hallmark_results.rename(  
    columns={"Term": "PATHWAY",  
            "NOM p-val": "NOMINAL_P",  
            "FDR q-val": "FDR",  
            "Lead_genes": "LEADING_EDGE_GENES"})  
    .sort_values("FDR"))
```

[180]: `reactome = gp.get_library(
 name="Reactome_2022",
 organism="Human")`

```
print(len(reactome))  
print(list(reactome.keys())[:5])
```

1816

['2-LTR Circle Formation R-HSA-164843', 'GAG Synthesis Requires Tetrasaccharide
Linker Sequence R-HSA-1971475', 'ABC Transporter Disorders R-HSA-5619084', 'ABC
Transporters In Lipid Homeostasis R-HSA-1369062', 'ABC-family Proteins Mediated
Transport R-HSA-382556']


```
[181]: # noinspection bad-argument-type
```

```
gsea = gp.prerank(  
    rnk=rnk,  
    gene_sets=reactome,  
    outdir=None,  
    seed=42,  
    verbose=True,  
    method='multilevel'  
)
```

2026-10-05 17:16:57,780 [WARNING] Duplicated values found in preranked stats:
0.31% of genes

The order of those genes will be arbitrary, which may produce unexpected
results.

2026-10-05 17:16:57,780 [INFO] Parsing data files for
GSEA...

2026-10-05 17:16:57,830 [INFO] 0740 gene_sets have been filtered out when
max_size=500 and min_size=15

2026-10-05 17:16:57,830 [INFO] 1076 gene_sets used for further statistical
testing...

2026-10-05 17:16:57,831 [INFO] Start to run GSEA... Might take a
while...

2026-10-05 17:17:21,990 [INFO] Congratulations. GSEApY runs
successfully...

```
[182]: reactome_results = gsea.res2d[["Term", "NES", "NOM p-val", "FDR q-val",  
    ↪ "Lead_genes"]].copy()  
reactome_results = ((reactome_results.rename(  
    columns={"Term": "PATHWAY",  
            "NOM p-val": "NOMINAL_P",  
            "FDR q-val": "FDR",  
            "Lead_genes": "LEADING_EDGE_GENES"}))  
    .sort_values("FDR"))
```

```
[183]: hallmark_results.to_excel(  
    ".../results/GSEA/glass_GSEA_Hallmark.xlsx",  
    index=False  
)
```

```
[184]: reactome_results.to_excel(  
    ".../results/GSEA/glass_GSEA_Reactome.xlsx",  
    index=False  
)
```

The resulting pathway-level statistics include: - **NES**: normalized enrichment score - **NOMI-**
NAL_P: nominal enrichment p-value - **FDR**: false discovery rate

Results exported as Excel files: - /results/GSEA/glass_GSEA_Hallmark.xlsx -

/results/GSEA/glass_GSEA_Reactome.xlsx

4.2 2. Summarising pathway analysis results

```
[185]: def get_top_pathways(dataframe, number=10) -> pd.DataFrame:
        sig = dataframe[dataframe["FDR"] < 0.05].copy()
        positive = (
            sig[sig["NES"] > 0]
            .sort_values("NES", ascending=False)
            .head(number))
        negative = (
            sig[sig["NES"] < 0]
            .sort_values("NES", ascending=True)
            .head(number))
        return pd.concat([negative, positive])
```

```
hallmark_top = get_top_pathways(hallmark_results, number=10)
reactome_top = get_top_pathways(reactome_results, number=10)
```

```
[186]: print("Significant:", (hallmark_results["FDR"] < 0.05).sum())
        print("Positive significant:", ((hallmark_results["FDR"] < 0.05) &
        ↪(hallmark_results["NES"] > 0)).sum())
        print("Negative significant:", ((hallmark_results["FDR"] < 0.05) &
        ↪(hallmark_results["NES"] < 0)).sum())
```

Significant: 18

Positive significant: 4

Negative significant: 14

Methodological Note: As only four positive Hallmark pathways reached statistical significance, all four were included in the analysis instead of selecting the top ten positive pathways.

```
[187]: fig, axes = plt.subplots(
        2, 1,
        figsize=(12, 16))

        datasets = [
            (hallmark_top, axes[0], "MSigDB Hallmark"),
            (reactome_top, axes[1], "Reactome")]

        for df, ax, title in datasets:
            if df.empty:
                ax.text(
                    0.5, 0.5,
                    "No pathways with FDR < 0.05",
                    ha="center",
```

```

        va="center",
        transform=ax.transAxes)
    ax.set_title(title)
    continue
df = df.sort_values("NES")
df["NEG_LOG10_FDR"] = -np.log10(
    df["FDR"].clip(lower=1e-300))
y = range(len(df))
colors = ['#d62728' if nes > 0 else '#1f77b4' for nes in df["NES"]]
scatter = ax.scatter(
    df["NES"],
    y,
    s=df["NEG_LOG10_FDR"] * 30 + 20,
    c=colors,
    alpha=0.8)
ax.axvline(
    0,
    color="gray",
    linestyle="--",
    linewidth=1)
ax.set_yticks(list(y))
ax.set_yticklabels(
    df["PATHWAY"],
    fontsize=8)
ax.set_xlabel("Normalized Enrichment Score (NES)", fontweight="bold")
ax.set_ylabel("Pathway", fontweight="bold")
ax.set_title(title, pad=15)
ax.grid(
    axis="x",
    alpha=0.2)
handles, labels = scatter.legend_elements(
    prop="sizes",
    alpha=0.8,
    num=4,
    func=lambda s: (s - 20) / 30,
    color=scatter.get_facecolors()[0])
ax.legend(
    handles,
    labels,
    title="-log10(FDR)",
    loc="center left",
    bbox_to_anchor=(1.02, 0.5),
    frameon=False,
    labelspace=2.5,
    handletextpad=1.0)

plt.suptitle(

```

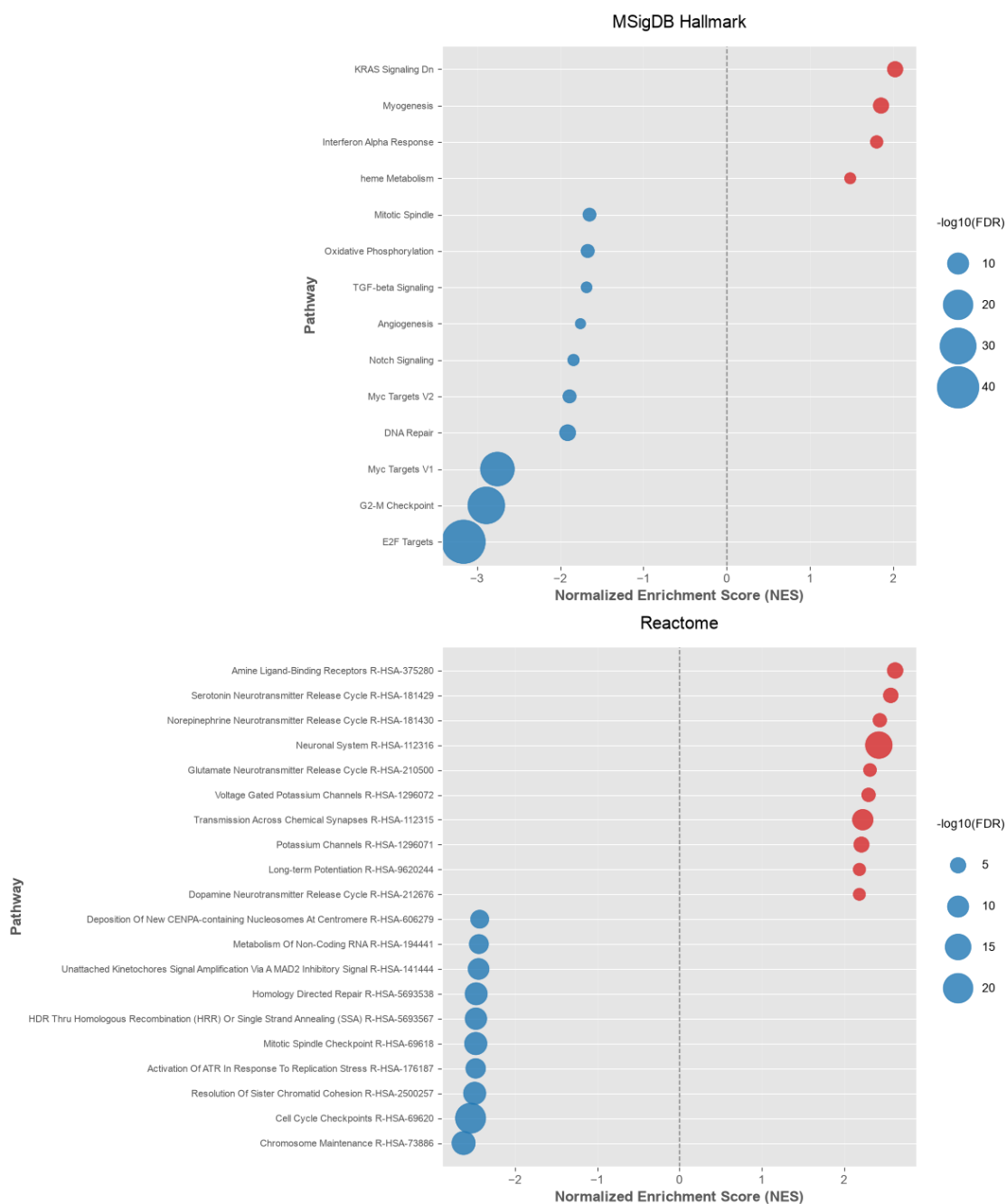
```

    "Top Positively and Negatively Enriched Hallmark and Reactome_
↳Pathways\nBased on Differential Expression Between Primary and Recurrent_
↳Tumors\n(FDR < 0.05)",
    fontsize=20,
    fontweight="bold",
    y=0.98)

plt.tight_layout(rect=(0, 0, 1, 0.95))
plt.savefig(
    "../../figures/glass_GSEA_top_pathways.png",
    dpi=300,
    bbox_inches="tight")
plt.show()

```

Top Positively and Negatively Enriched Hallmark and Reactome Pathways Based on Differential Expression Between Primary and Recurrent Tumors (FDR < 0.05)



- glass_GSEA_top_pathways.png saved to /figures directory

```
[188]: def format_pvalue(val):
        val = float(val)
        if val < 0.000001:
            return "<0.000001"
        return f"{val:.6f}"

        def format_nes(val):
            return f"{float(val):.6f}"
```

```
[189]: column_formatters = {
        "NES": format_nes,
        "NOMINAL_P": format_pvalue,
        "FDR": format_pvalue
    }
```

```
[190]: print("TOP HALLMARK PATHWAYS")
        print(hallmark_top[["PATHWAY", "NES", "NOMINAL_P", "FDR"]].
              ↪to_string(index=False))
```

TOP HALLMARK PATHWAYS

PATHWAY	NES	NOMINAL_P	FDR
E2F Targets	-3.160205	5.428275e-46	2.714138e-44
G2-M Checkpoint	-2.887096	2.613210e-33	6.533024e-32
Myc Targets V1	-2.753717	3.314955e-28	5.524925e-27
DNA Repair	-1.910751	3.028536e-07	3.785670e-06
Myc Targets V2	-1.887909	4.925536e-05	3.116461e-04
Notch Signaling	-1.840243	8.073027e-04	3.363761e-03
Angiogenesis	-1.756645	4.124829e-03	1.374943e-02
TGF-beta Signaling	-1.683286	1.867009e-03	6.667891e-03
Oxidative Phosphorylation	-1.669803	4.986338e-05	3.116461e-04
Mitotic Spindle	-1.648381	7.271060e-05	4.039478e-04
KRAS Signaling Dn	2.021822	1.175931e-06	1.036300e-05
Myogenesis	1.852663	1.243560e-06	1.036300e-05
Interferon Alpha Response	1.800211	1.703951e-04	8.519757e-04
heme Metabolism	1.482313	1.234432e-03	4.747814e-03

```
[191]: print("TOP REACTOME PATHWAYS")
        print(reactome_top[["PATHWAY", "NES", "NOMINAL_P", "FDR"]].
              ↪to_string(index=False))
```

TOP REACTOME PATHWAYS

PATHWAY	NES	NOMINAL_P	FDR
			Chromosome Maintenance
R-HSA-73886	-2.624170	1.019638e-14	8.439464e-13
			Cell Cycle Checkpoints
R-HSA-69620	-2.539201	3.283452e-24	1.766497e-21
			Resolution Of Sister Chromatid Cohesion

R-HSA-2500257 -2.489224 1.280635e-13 8.612269e-12
 Activation Of ATR In Response To Replication Stress
 R-HSA-176187 -2.476915 1.150984e-10 3.440162e-09
 Mitotic Spindle Checkpoint
 R-HSA-69618 -2.475875 1.037725e-13 7.443950e-12
 HDR Thru Homologous Recombination (HRR) Or Single Strand Annealing (SSA)
 R-HSA-5693567 -2.473861 9.783156e-13 4.386115e-11
 Homology Directed Repair
 R-HSA-5693538 -2.470761 1.894350e-13 1.132400e-11
 Unattached Kinetochores Signal Amplification Via A MAD2 Inhibitory Signal
 R-HSA-141444 -2.441916 5.103605e-12 2.112107e-10
 Metabolism Of Non-Coding RNA
 R-HSA-194441 -2.438311 3.701900e-10 1.021345e-08
 Deposition Of New CENPA-containing Nucleosomes At Centromere
 R-HSA-606279 -2.428067 3.925916e-09 8.800595e-08
 Amine Ligand-Binding Receptors
 R-HSA-375280 2.621317 6.060226e-07 7.410003e-06
 Serotonin Neurotransmitter Release Cycle
 R-HSA-181429 2.568264 3.687825e-06 3.413033e-05
 Norepinephrine Neurotransmitter Release Cycle
 R-HSA-181430 2.435433 2.823134e-05 1.934836e-04
 Neuronal System
 R-HSA-112316 2.423285 3.093202e-19 8.320715e-17
 Glutamate Neurotransmitter Release Cycle
 R-HSA-210500 2.315823 9.238938e-05 5.124277e-04
 Voltage Gated Potassium Channels
 R-HSA-1296072 2.298274 2.591441e-05 1.787430e-04
 Transmission Across Chemical Synapses
 R-HSA-112315 2.228156 1.332524e-11 4.925877e-10
 Potassium Channels
 R-HSA-1296071 2.212112 1.297300e-06 1.382074e-05
 Long-term Potentiation
 R-HSA-9620244 2.187273 2.181749e-04 1.052718e-03
 Dopamine Neurotransmitter Release Cycle
 R-HSA-212676 2.186748 3.210893e-04 1.495637e-03

```
[192]: print("TOP HALLMARK PATHWAYS FORMATTED TO DECIMALS")
print(hallmark_top[["PATHWAY", "NES", "NOMINAL_P", "FDR"]].
      ↪to_string(index=False, formatters=column_formatters))
```

```
TOP HALLMARK PATHWAYS FORMATTED TO DECIMALS
      PATHWAY      NES NOMINAL_P      FDR
E2F Targets -3.160205 <0.000001 <0.000001
G2-M Checkpoint -2.887096 <0.000001 <0.000001
Myc Targets V1 -2.753717 <0.000001 <0.000001
DNA Repair -1.910751 <0.000001 0.000004
Myc Targets V2 -1.887909 0.000049 0.000312
Notch Signaling -1.840243 0.000807 0.003364
```

Angiogenesis	-1.756645	0.004125	0.013749
TGF-beta Signaling	-1.683286	0.001867	0.006668
Oxidative Phosphorylation	-1.669803	0.000050	0.000312
Mitotic Spindle	-1.648381	0.000073	0.000404
KRAS Signaling Dn	2.021822	0.000001	0.000010
Myogenesis	1.852663	0.000001	0.000010
Interferon Alpha Response	1.800211	0.000170	0.000852
heme Metabolism	1.482313	0.001234	0.004748

```
[193]: print("TOP REACTOME PATHWAYS FORMATTED TO 6 DECIMALS")
print(reactome_top[["PATHWAY", "NES", "NOMINAL_P", "FDR"]].
      ↪to_string(index=False, formatters=column_formatters))
```

TOP REACTOME PATHWAYS FORMATTED TO 6 DECIMALS

PATHWAY	NES	NOMINAL_P	FDR	
				Chromosome Maintenance
R-HSA-73886	-2.624170	<0.000001	<0.000001	
				Cell Cycle Checkpoints
R-HSA-69620	-2.539201	<0.000001	<0.000001	
				Resolution Of Sister Chromatid Cohesion
R-HSA-2500257	-2.489224	<0.000001	<0.000001	
				Activation Of ATR In Response To Replication Stress
R-HSA-176187	-2.476915	<0.000001	<0.000001	
				Mitotic Spindle Checkpoint
R-HSA-69618	-2.475875	<0.000001	<0.000001	
				HDR Thru Homologous Recombination (HRR) Or Single Strand Annealing (SSA)
R-HSA-5693567	-2.473861	<0.000001	<0.000001	
				Homology Directed Repair
R-HSA-5693538	-2.470761	<0.000001	<0.000001	
				Unattached Kinetochores Signal Amplification Via A MAD2 Inhibitory Signal
R-HSA-141444	-2.441916	<0.000001	<0.000001	
				Metabolism Of Non-Coding RNA
R-HSA-194441	-2.438311	<0.000001	<0.000001	
				Deposition Of New CENPA-containing Nucleosomes At Centromere
R-HSA-606279	-2.428067	<0.000001	<0.000001	
				Amine Ligand-Binding Receptors
R-HSA-375280	2.621317	<0.000001	0.000007	
				Serotonin Neurotransmitter Release Cycle
R-HSA-181429	2.568264	0.000004	0.000034	
				Norepinephrine Neurotransmitter Release Cycle
R-HSA-181430	2.435433	0.000028	0.000193	
				Neuronal System
R-HSA-112316	2.423285	<0.000001	<0.000001	
				Glutamate Neurotransmitter Release Cycle
R-HSA-210500	2.315823	0.000092	0.000512	
				Voltage Gated Potassium Channels
R-HSA-1296072	2.298274	0.000026	0.000179	
				Transmission Across Chemical Synapses

R-HSA-112315	2.228156	<0.000001	<0.000001	Potassium Channels
R-HSA-1296071	2.212112	0.000001	0.000014	Long-term Potentiation
R-HSA-9620244	2.187273	0.000218	0.001053	Dopamine Neurotransmitter Release Cycle
R-HSA-212676	2.186748	0.000321	0.001496	

4.2.1 2.1. Biological relevance of top pathways:

Overview: Recurrent samples show a strong shift toward a less proliferative transcriptional state, with marked suppression of cell-cycle, DNA-repair, and genome-maintenance programs. In contrast, neuronal/synaptic and interferon-associated signatures are enriched in recurrent samples.

Key downregulated pathways

- **Cell cycle and proliferation:** Strong suppression of E2F Targets (NES: -3.16, FDR: <0.000001), G2-M Checkpoint (NES: -2.89, FDR: <0.000001), Myc Targets V1 (NES: -2.75, FDR: <0.000001), and Mitotic Spindle (NES: -1.65, FDR: 0.000404).
- **DNA repair and genome maintenance:** DNA Repair (NES: -1.91, FDR: 0.000004) is supported by Reactome enrichment of Chromosome Maintenance (NES: -2.62, FDR: <0.000001), Activation Of ATR In Response To Replication Stress (NES: -2.48, FDR: <0.000001), Homology Directed Repair (NES: -2.47, FDR: <0.000001) and other related pathways (all FDR: <0.000001).
- **Other pathways:** Notch Signaling (NES: -1.84, FDR: 0.003364), TGF-beta Signaling (NES: -1.68, FDR: 0.006668), and Oxidative Phosphorylation (NES: -1.67, FDR: 0.000312) are also reduced.

Key upregulated pathways

- **Signalling and neuronal programs:** Reactome shows strong enrichment of Amine Ligand-Binding Receptors (NES: 2.62, FDR: 0.000007), Serotonin Neurotransmitter Release (NES: 2.56, FDR: 0.000034), Neuronal System (NES: 2.42, FDR: <0.000001), and Transmission Across Chemical Synapses (NES: 2.22, FDR: <0.000001).
- **Other enriched programs:** KRAS Signaling Dn (NES: 2.02, FDR: 0.000010), Myogenesis (1.91, 0.000011), Interferon Alpha Response (NES: 1.80, FDR: 0.000852), and Heme Metabolism (NES: 1.48, FDR: 0.004748).

Biological Implication

- **Reduced proliferation:** The dominant feature of recurrence is suppression of E2F/Myc-driven proliferation, G2/M progression, mitosis, and DNA-repair/genome-maintenance programs.
- **Altered signalling and cellular programs:** Recurrent samples show enrichment of interferon-associated and neuronal/synaptic signatures, together with changes in metabolic and signalling pathways.
- **Interpretation:** The overall profile is consistent with a **low-proliferative recurrent state**. A senescence-like phenotype is a possible interpretation, but would require independent validation and should not be concluded from GSEA alone.

4.3 3. Assessing the stability of recurrence-associated genes using bootstrap analysis

```
[194]: up_initial = glass_recurrence_UP_initial["GENE"]
down_initial = glass_recurrence_DOWN_initial["GENE"]
bootstrap_genes = pd.Index(up_initial.tolist() + down_initial.tolist()).
    ↪drop_duplicates()
original_direction = pd.Series("UP", index=up_initial)
original_direction = pd.concat([original_direction, pd.Series("DOWN",
    ↪index=down_initial)])

print("Initial UP genes:", len(up_initial))
print("Initial DOWN genes:", len(down_initial))
```

Initial UP genes: 315
Initial DOWN genes: 220

```
[195]: len(genes) == delta_expression.shape[0]
```

[195]: True

```
[196]: bootstrap_up_counts = pd.Series(
    0,
    index=bootstrap_genes,
    dtype=int)
bootstrap_down_counts = pd.Series(
    0,
    index=bootstrap_genes,
    dtype=int)
bootstrap_same_direction_counts = pd.Series(
    0,
    index=bootstrap_genes,
    dtype=int)

rng = np.random.default_rng(42)

n_patients = delta_expression.shape[1]
n_genes = delta_expression.shape[0]

print(f"Genes: {n_genes}")
print(f"Patient pairs: {n_patients}")
print(f"Original recurrence genes: {len(bootstrap_genes)}")
genes = glass_idhwt_gbm_primary_R1_log2TPM_filtered.index

original_up = bootstrap_genes[original_direction.loc[bootstrap_genes].eq("UP")]
original_down = bootstrap_genes[original_direction.loc[bootstrap_genes].
    ↪eq("DOWN")]
```

```

for b in range(1000):
    sampled_idx = rng.integers(
        0,
        n_patients,
        size=n_patients)

    boot_delta = delta_expression[:, sampled_idx]

    n = np.sum(~np.isnan(boot_delta), axis=1)

    mean_delta = np.nanmean(
        boot_delta,
        axis=1)

    sd_delta = np.nanstd(
        boot_delta,
        axis=1,
        ddof=1)

    t_statistic = (mean_delta / (sd_delta / np.sqrt(n)))

    p_values = np.full(n_genes, np.nan)

    valid_tests = ((n >= 2) & np.isfinite(t_statistic))

    p_values[valid_tests] = (2 * t.sf(np.abs(t_statistic[valid_tests]),
    ↪df=n[valid_tests] - 1))

    fdr = np.full(n_genes, np.nan)

    valid_p = np.isfinite(p_values)

    fdr[valid_p] = multipletests(
        p_values[valid_p],
        method="fdr_bh"
    )[1]

    fraction_up = np.divide(
        np.sum((boot_delta > 0) & ~np.isnan(boot_delta), axis=1),
        n,
        out=np.full(n.shape, np.nan, dtype=float),
        where=n > 0
    )

    fraction_down = np.divide(
        np.sum((boot_delta < 0) & ~np.isnan(boot_delta), axis=1),
        n,

```

```

        out=np.full(n.shape, np.nan, dtype=float),
        where=n > 0
    )

    boot_up = ((fdr < 0.05) &
               (mean_delta >= 0.5) &
               (fraction_up >= 0.65))
    boot_down = ((fdr < 0.05) &
                 (mean_delta <= -0.5) &
                 (fraction_down >= 0.65))

    up_genes = genes[boot_up]
    down_genes = genes[boot_down]

    selected_up = bootstrap_genes.intersection(pd.Index(up_genes))
    selected_down = bootstrap_genes.intersection(pd.Index(down_genes))

    same_up = original_up.intersection(pd.Index(up_genes))
    same_down = original_down.intersection(pd.Index(down_genes))

    bootstrap_up_counts.loc[selected_up] += 1
    bootstrap_down_counts.loc[selected_down] += 1

    bootstrap_same_direction_counts.loc[same_up] += 1
    bootstrap_same_direction_counts.loc[same_down] += 1

    if (b + 1) % 100 == 0:
        print(f"Bootstrap {b + 1}/1000 completed")

```

```

Genes: 17789
Patient pairs: 103
Original recurrence genes: 535
Bootstrap 100/1000 completed
Bootstrap 200/1000 completed
Bootstrap 300/1000 completed
Bootstrap 400/1000 completed
Bootstrap 500/1000 completed
Bootstrap 600/1000 completed
Bootstrap 700/1000 completed
Bootstrap 800/1000 completed
Bootstrap 900/1000 completed
Bootstrap 1000/1000 completed

```

4.4 4. Determining bootstrap selection frequency

- $\text{BOOTSTRAP_UP_FREQUENCY} = \text{n_up_count}/1000$
- $\text{BOOTSTRAP_DOWN_FREQUENCY} = \text{n_down_count}/1000$
- $\text{BOOTSTRAP_SAME_DIRECTION_FREQUENCY} = \text{n_same_direction}/1000$

```
[197]: bootstrap_stability = pd.DataFrame({
    "GENE": bootstrap_genes,
    "BOOTSTRAP_UP_FREQUENCY": bootstrap_up_counts.loc[bootstrap_genes].values / 1000,
    "BOOTSTRAP_DOWN_FREQUENCY": bootstrap_down_counts.loc[bootstrap_genes].values / 1000,
    "BOOTSTRAP_SAME_DIRECTION_FREQUENCY": bootstrap_same_direction_counts.loc[bootstrap_genes].values / 1000
})
```

```
[198]: original_stats = pd.concat([
    glass_recurrence_UP_initial[
        ["GENE",
         "MEAN_DELTA",
         "FDR",
         "FRACTION_UP",
         "FRACTION_DOWN"]],
    glass_recurrence_DOWN_initial[
        ["GENE",
         "MEAN_DELTA",
         "FDR",
         "FRACTION_UP",
         "FRACTION_DOWN"]]], ignore_index=True)

original_stats = original_stats.rename(columns={
    "MEAN_DELTA": "ORIGINAL_MEAN_DELTA",
    "FDR": "ORIGINAL_FDR",
    "FRACTION_UP": "ORIGINAL_FRACTION_UP",
    "FRACTION_DOWN": "ORIGINAL_FRACTION_DOWN"})
```

```
[199]: glass_recurrence_bootstrap_stability = (
    original_stats
    .merge(
        bootstrap_stability,
        on="GENE",
        how="left"))

glass_recurrence_bootstrap_stability["ORIGINAL_DIRECTION"] = (
    glass_recurrence_bootstrap_stability["GENE"].map(original_direction))
```

```
[200]: glass_recurrence_bootstrap_stability.head(20)
```

```
[200]:
```

	GENE	ORIGINAL_MEAN_DELTA	ORIGINAL_FDR	ORIGINAL_FRACTION_UP	\
0	FAM153C	0.673328	0.003732	0.650485	
1	RGS4	0.772503	0.005819	0.669903	
2	MICAL2	0.610867	0.001594	0.660194	
3	CRHBP	0.777987	0.000125	0.650485	

4	C16orf89	0.620247	0.000614	0.650485
5	AKR1C1	0.705713	0.000170	0.679612
6	ADAP1	1.128064	0.000014	0.728155
7	KLK6	1.580467	0.000030	0.728155
8	MBP	1.559155	0.001340	0.689320
9	TUBB4A	1.034620	0.004110	0.679612
10	MOG	1.539450	0.000158	0.718447
11	SEPT4	1.083744	0.000050	0.728155
12	TPPP3	0.583246	0.008858	0.669903
13	VAMP1	0.783594	0.000008	0.728155
14	KLHL32	0.848802	0.000321	0.669903
15	ST18	0.968447	0.000481	0.708738
16	PEX5L	1.274317	0.000010	0.708738
17	TMEM144	1.050502	0.000480	0.669903
18	SPOCK3	1.574444	0.000026	0.718447
19	FCH01	0.515093	0.000911	0.650485

	ORIGINAL_FRACTION_DOWN	BOOTSTRAP_UP_FREQUENCY	BOOTSTRAP_DOWN_FREQUENCY	\
0	0.349515	0.538	0.0	
1	0.330097	0.688	0.0	
2	0.339806	0.608	0.0	
3	0.339806	0.526	0.0	
4	0.349515	0.519	0.0	
5	0.320388	0.762	0.0	
6	0.271845	0.971	0.0	
7	0.271845	0.968	0.0	
8	0.310680	0.813	0.0	
9	0.320388	0.758	0.0	
10	0.281553	0.938	0.0	
11	0.271845	0.974	0.0	
12	0.330097	0.599	0.0	
13	0.271845	0.960	0.0	
14	0.330097	0.729	0.0	
15	0.291262	0.921	0.0	
16	0.291262	0.928	0.0	
17	0.330097	0.711	0.0	
18	0.281553	0.954	0.0	
19	0.349515	0.427	0.0	

	BOOTSTRAP_SAME_DIRECTION_FREQUENCY	ORIGINAL_DIRECTION
0	0.538	UP
1	0.688	UP
2	0.608	UP
3	0.526	UP
4	0.519	UP
5	0.762	UP
6	0.971	UP

7	0.968	UP
8	0.813	UP
9	0.758	UP
10	0.938	UP
11	0.974	UP
12	0.599	UP
13	0.960	UP
14	0.729	UP
15	0.921	UP
16	0.928	UP
17	0.711	UP
18	0.954	UP
19	0.427	UP

```
[201]: glass_recurrence_bootstrap_stability = (
    glass_recurrence_bootstrap_stability
    .sort_values(
        "BOOTSTRAP_SAME_DIRECTION_FREQUENCY",
        ascending=False))

glass_recurrence_bootstrap_stability.to_excel(
    "../results/recurrence/glass_recurrence_bootstrap_stability.xlsx",
    index=False)
```

- glass_recurrence_bootstrap_stability.xlsx exported to /results/recurrence/ directory

4.5 5. Defining the recurrence signature

4.5.1 Gene-level differential expression and recurrence classification

For each gene g and patient i , the paired expression difference between the first recurrence (R1) and primary tumour was calculated as:

$$\Delta_{ig} = X_{ig}^{R1} - X_{ig}^{Primary}$$

where X_{ig}^{R1} and $X_{ig}^{Primary}$ denote the expression of gene g in the R1 and primary tumour samples, respectively.

For each gene, N_g denotes the number of patients with non-missing paired measurements.

Fraction of patients with increased expression at recurrence

$$\text{FRACTION_UP}_g = \frac{\sum_{i=1}^{N_g} \mathbf{1}(\Delta_{ig} > 0)}{N_g}$$

Fraction of patients with decreased expression at recurrence

$$\text{FRACTION_DOWN}_g = \frac{\sum_{i=1}^{N_g} \mathbf{1}(\Delta_{ig} < 0)}{N_g}$$

where $\mathbf{1}(\cdot)$ is the indicator function, equal to 1 when the specified condition is true and 0 otherwise. The mean paired expression difference was calculated as:

$$\overline{\Delta}_g = \frac{1}{N_g} \sum_{i=1}^{N_g} \Delta_{ig}$$

4.5.2 Recurrence-UP genes

A gene was classified as **Recurrence-UP** if all the following criteria were satisfied:

- **FDR** < 0.05
- **Mean difference** $\overline{\Delta}_g \geq 0.5$
- **FRACTION_UP** ≥ 0.65

4.5.3 Recurrence-DOWN genes

A gene was classified as **Recurrence-DOWN** if all the following criteria were satisfied:

- **FDR** < 0.05
- **Mean difference** $\overline{\Delta}_g \leq -0.5$
- **FRACTION_DOWN** ≥ 0.65

4.5.4 Bootstrap stability

Bootstrap stability was assessed using 1,000 patient-level bootstrap samples. In each bootstrap iteration, 103 patients were sampled with replacement, with the primary and R1 samples of each patient retained as a paired observation. For each bootstrap sample, gene-level paired differences, statistical test statistics, and FDR values were recalculated, and the same criteria used for the original Recurrence-UP and Recurrence-DOWN gene lists were applied.

For each gene, the **bootstrap same-direction frequency** was calculated as:

$$\text{BOOTSTRAP_SAME_DIRECTION_FREQUENCY}_g = \frac{B_g}{B}$$

where $B = 1000$ is the total number of bootstrap iterations and B_g is the number of bootstrap iterations in which the gene was identified in the same direction as in the original analysis.

A gene was considered **bootstrap-stable** if:

$$\text{BOOTSTRAP_SAME_DIRECTION_FREQUENCY}_g \geq 0.70$$

```
[202]: STABILITY_THRESHOLD = 0.70

consensus = glass_recurrence_bootstrap_stability[
    glass_recurrence_bootstrap_stability[
        "BOOTSTRAP_SAME_DIRECTION_FREQUENCY"
    ] >= STABILITY_THRESHOLD
].copy()
```



```

consensus_up = consensus[
    consensus["ORIGINAL_DIRECTION"] == "UP"
].copy()

consensus_down = consensus[
    consensus["ORIGINAL_DIRECTION"] == "DOWN"
].copy()
n_original_up = len(glass_recurrence_UP_initial)
n_original_down = len(glass_recurrence_DOWN_initial)

n_consensus_up = len(consensus_up)
n_consensus_down = len(consensus_down)

n_consensus = len(consensus)
n_original = len(bootstrap_genes)

print("\nConsensus signature retention:")
print(f"UP:    {n_consensus_up}/{n_original_up} "f"({100 * n_consensus_up / n_
    ↪ n_original_up:.1f}%)")
print(f"DOWN: {n_consensus_down}/{n_original_down} "f"({100 * n_consensus_down /
    ↪ n_original_down:.1f}%)")
print(f"Overall: {n_consensus}/{n_original} "f"({100 * n_consensus / n_original:
    ↪ .1f}%)")

```

Consensus signature retention:

UP: 168/315 (53.3%)

DOWN: 97/220 (44.1%)

Overall: 265/535 (49.5%)

Summary Note: Patient-level bootstrap analysis demonstrated that 168 of 315 (53.3%) Recurrence-UP genes and 97 of 220 (44.1%) Recurrence-DOWN genes were consistently selected in the same direction in at least 70% of bootstrap samples. Overall, 265 of 535 (49.5%) initially identified recurrence-associated genes met the predefined stability criterion.

```

[203]: stability = glass_recurrence_bootstrap_stability.copy()

stability["OPPOSITE_DIRECTION_FREQUENCY"] = np.where(
    stability["ORIGINAL_DIRECTION"] == "UP",
    stability["BOOTSTRAP_DOWN_FREQUENCY"],
    stability["BOOTSTRAP_UP_FREQUENCY"]
)

consensus_inspection = stability.loc[
    stability["BOOTSTRAP_SAME_DIRECTION_FREQUENCY"] >= 0.70
].copy()

```

```
consensus_inspection[
  [
    "GENE",
    "ORIGINAL_DIRECTION",
    "BOOTSTRAP_SAME_DIRECTION_FREQUENCY",
    "OPPOSITE_DIRECTION_FREQUENCY"
  ]
].sort_values(
  "OPPOSITE_DIRECTION_FREQUENCY",
  ascending=False
).head(20)
```

```
[203]:
```

	GENE	ORIGINAL_DIRECTION	BOOTSTRAP_SAME_DIRECTION_FREQUENCY	\
389	DPEP1	DOWN	1.000	
319	HIGD1B	DOWN	0.999	
499	MMRN1	DOWN	0.999	
525	NODAL	DOWN	0.999	
45	ENPP2	UP	0.999	
511	SPINK8	DOWN	0.996	
355	ENPEP	DOWN	0.996	
258	MAL	UP	0.995	
111	SLC45A3	UP	0.993	
451	RGS5	DOWN	0.992	
252	ADIRF	UP	0.991	
215	GREM1	UP	0.985	
379	E2F5	DOWN	0.982	
166	FBX02	UP	0.981	
303	OPALIN	UP	0.980	
266	MAG	UP	0.978	
11	SEPT4	UP	0.974	
46	RNASE1	UP	0.974	
472	F2R	DOWN	0.974	
280	RBP7	UP	0.971	

	OPPOSITE_DIRECTION_FREQUENCY
389	0.0
319	0.0
499	0.0
525	0.0
45	0.0
511	0.0
355	0.0
258	0.0
111	0.0
451	0.0
252	0.0

215	0.0
379	0.0
166	0.0
303	0.0
266	0.0
11	0.0
46	0.0
472	0.0
280	0.0

```
[204]: for threshold in [0.05, 0.10, 0.20, 0.30]:
        opposite_freq = consensus_inspection["OPPOSITE_DIRECTION_FREQUENCY"]
        n = int(np.count_nonzero(opposite_freq.to_numpy() >= threshold))
        print(f"Opposite-direction frequency >= {threshold:.0%}: "
              f"{n}/{len(consensus_inspection)}")
```

```
Opposite-direction frequency >= 5%: 0/265
Opposite-direction frequency >= 10%: 0/265
Opposite-direction frequency >= 20%: 0/265
Opposite-direction frequency >= 30%: 0/265
```

Summary Note: None of the 265 consensus genes showed evidence of meaningful direction switching during bootstrap resampling. For all consensus genes, opposite-direction selection occurred in <5% of bootstrap samples, supporting the stability of the UP/DOWN direction assignment.

```
[205]: consensus_up.to_excel(
        ".../results/recurrence/"
        "glass_recurrence_UP_consensus.xlsx",
        index=False)

consensus_down.to_excel(
        ".../results/recurrence/"
        "glass_recurrence_DOWN_consensus.xlsx",
        index=False)
```

- glass_recurrence_UP_consensus.xlsx and glass_recurrence_DOWN_consensus.xlsx exported to /results/recurrence/

4.6 6. Creating the Recurrence Score

- Rank all genes within each sample
- rank(pct=True) gives percentile ranks from approximately 0 to 1.
- Higher expression → higher percentile.
- UP_SCORE = mean percentile rank of Consensus Recurrence-UP genes in sample
- DOWN_SCORE = mean percentile rank of Consensus Recurrence-DOWN genes in sample
- Uses frozen consensus signatures from section 5.
- High recurrence score = expression characteristic of R1 GBM
- Low recurrence score = expression characteristic of TP GBM

```
[206]: def calculate_recurrence_score(expr, up_genes, down_genes):
        """Calculates sample-wise Recurrence Scores using gene percentile ranks.

        Parameters:
        -----
        expr : pd.DataFrame
            Gene expression matrix with genes as rows and samples as columns.
        up_genes : list-like
            Gene symbols for consensus UP signature.
        down_genes : list-like
            Gene symbols for consensus DOWN signature.

        Returns:
        -----
        pd.Series
            Recurrence Score indexed by sample ID.
        """

        valid_up = [g for g in up_genes if g in expr.index]
        valid_down = [g for g in down_genes if g in expr.index]
        percentile_ranks = expr.rank(axis=0, method="average", pct=True)
        up_score = percentile_ranks.loc[valid_up].mean(axis=0)
        down_score = percentile_ranks.loc[valid_down].mean(axis=0)

        return pd.DataFrame({
            "UP_SCORE": up_score,
            "DOWN_SCORE": down_score,
            "RECURRENCE_SCORE": up_score - down_score
        })
```

```
[207]: consensus_up_genes = pd.Index(consensus_up["GENE"]).intersection(expr.index)
        consensus_down_genes = pd.Index(consensus_down["GENE"]).intersection(expr.index)

        print(f"Consensus UP genes available: {len(consensus_up_genes)}")
        print(f"Consensus DOWN genes available: {len(consensus_down_genes)}")
```

Consensus UP genes available: 168
 Consensus DOWN genes available: 97

```
[208]: scores_df = calculate_recurrence_score(glass_idhwt_gbm_primary_R1_log2TPM_filtered,
        consensus_up_genes,
        consensus_down_genes)
        scores_df = scores_df.reset_index().rename(columns={"index": "SAMPLE_ID"})
```

```
[209]: pairs = glass_idhwt_gbm_primary_R1_pairs
        sample_info: pd.DataFrame = pd.concat([
            pairs[["PATIENT_ID", "PRIMARY_SAMPLE_ID"]].
            rename(columns={"PRIMARY_SAMPLE_ID": "SAMPLE_ID"}).assign(
```

```

        SAMPLE_TYPE="Tumor Primary"),
    pairs[["PATIENT_ID", "FIRST_RECURRENCE_SAMPLE_ID"]].rename(
        columns={"FIRST_RECURRENCE_SAMPLE_ID": "SAMPLE_ID"}).
    ↪assign(SAMPLE_TYPE="First Recurrence")
], ignore_index=True)

```

```

[210]: glass_recurrence_scores = sample_info.merge(scores_df, on="SAMPLE_ID",
    ↪how="inner")
glass_recurrence_scores_full_cohort = glass_recurrence_scores[
    ["PATIENT_ID", "SAMPLE_ID", "SAMPLE_TYPE", "UP_SCORE", "DOWN_SCORE",
    ↪"RECURRENCE_SCORE"]
].copy()

```

```

[211]: glass_recurrence_scores_full_cohort.head()

```

```

[211]:
    PATIENT_ID      SAMPLE_ID  SAMPLE_TYPE  UP_SCORE  DOWN_SCORE  \
0  GLSS-19-0267  GLSS-19-0267-TP  Tumor Primary  0.432377    0.582453
1  GLSS-19-0268  GLSS-19-0268-TP  Tumor Primary  0.288376    0.564849
2  GLSS-19-0269  GLSS-19-0269-TP  Tumor Primary  0.366178    0.605090
3  GLSS-19-0271  GLSS-19-0271-TP  Tumor Primary  0.409954    0.559938
4  GLSS-19-0272  GLSS-19-0272-TP  Tumor Primary  0.310627    0.595148

    RECURRENCE_SCORE
0          -0.150076
1          -0.276473
2          -0.238912
3          -0.149983
4          -0.284521

```

```

[212]: glass_recurrence_scores_full_cohort.to_excel(
    "../results/recurrence/glass_recurrence_scores_full_cohort.xlsx",
    index=False
)

```

- glass_recurrence_scores_full_cohort.xlsx exported to /results/recurrence directory.

```

[213]: print(f"Recurrence scores calculated for {len(glass_recurrence_scores)}
    ↪samples.")

```

Recurrence scores calculated for 206 samples.

```

[214]: glass_recurrence_scores[["UP_SCORE", "DOWN_SCORE", "RECURRENCE_SCORE"]].
    ↪describe()

```

```

[214]:
    UP_SCORE  DOWN_SCORE  RECURRENCE_SCORE
count  206.000000  206.000000    206.000000
mean     0.454595    0.538159    -0.083564

```

std	0.149878	0.089456	0.214807
min	0.220247	0.312966	-0.399699
25%	0.330497	0.477997	-0.244482
50%	0.420333	0.541263	-0.150030
75%	0.573606	0.605338	0.055446
max	0.785751	0.711834	0.448837

4.7 7. Initial assessment of the Recurrence Score in the full GLASS cohort

- $\text{DELTA_RECURRENCE_SCORE} = \text{SCORE_R1} - \text{SCORE_PRIMARY}$

```
[215]: glass_recurrence_scores.head()
```

```
[215]:
```

	PATIENT_ID	SAMPLE_ID	SAMPLE_TYPE	UP_SCORE	DOWN_SCORE	\
0	GLSS-19-0267	GLSS-19-0267-TP	Tumor Primary	0.432377	0.582453	
1	GLSS-19-0268	GLSS-19-0268-TP	Tumor Primary	0.288376	0.564849	
2	GLSS-19-0269	GLSS-19-0269-TP	Tumor Primary	0.366178	0.605090	
3	GLSS-19-0271	GLSS-19-0271-TP	Tumor Primary	0.409954	0.559938	
4	GLSS-19-0272	GLSS-19-0272-TP	Tumor Primary	0.310627	0.595148	

	RECURRENCE_SCORE
0	-0.150076
1	-0.276473
2	-0.238912
3	-0.149983
4	-0.284521

```
[216]: score_pairs = glass_recurrence_scores.pivot_table(
    index="PATIENT_ID",
    columns="SAMPLE_TYPE",
    values="RECURRENCE_SCORE"
)

score_pairs["DELTA_RECURRENCE_SCORE"] = (
    score_pairs["First Recurrence"] -
    score_pairs["Tumor Primary"]
)
```

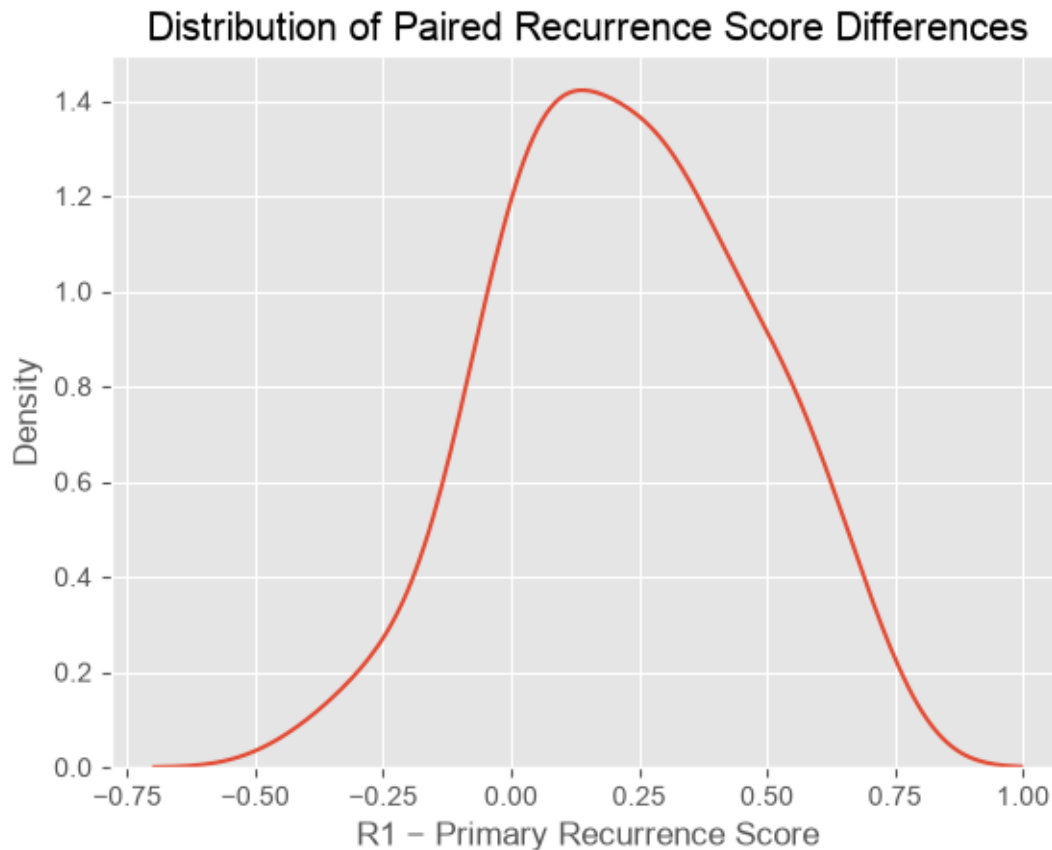
```
[217]: score_pairs.head()
```

```
[217]:
```

SAMPLE_TYPE	First Recurrence	Tumor Primary	DELTA_RECURRENCE_SCORE
PATIENT_ID			
GLSS-19-0267	0.240398	-0.150076	0.390474
GLSS-19-0268	0.421321	-0.276473	0.697795
GLSS-19-0269	0.104102	-0.238912	0.343014
GLSS-19-0271	0.114246	-0.149983	0.264229
GLSS-19-0272	0.284158	-0.284521	0.568680

```
[218]: sns.kdeplot(np.array(score_pairs["DELTA_RECURRENCE_SCORE"]))

plt.title("Distribution of Paired Recurrence Score Differences")
plt.xlabel("R1 - Primary Recurrence Score")
plt.ylabel("Density")
plt.show()
```



```
[219]: delta_skew_rs = float(skew(score_pairs["DELTA_RECURRENCE_SCORE"],
    ↪nan_policy="omit"))

print("Patient pairs:", glass_recurrence_scores["PATIENT_ID"].nunique())
print(f"Skewness:{float(delta_skew_rs):.3f}")
```

Patient pairs: 103
Skewness:-0.045

Methodological Note: The paired recurrence score differences exhibited minimal skewness (skewness = -0.045). Given the near-symmetric distribution of the differences and the sample size of $N = 103$ paired observations, the paired t -test was considered appropriate. The sample size also provides robustness to moderate deviations from normality through the Central Limit Theorem.

```
[220]: mean_delta_rs = score_pairs["DELTA_RECURRENCE_SCORE"].mean()
median_delta_rs = score_pairs["DELTA_RECURRENCE_SCORE"].median()

t_statistic_rs, p_value_rs = ttest_rel(
    score_pairs["First Recurrence"],
    score_pairs["Tumor Primary"]
)

valid_rs = (
    score_pairs["First Recurrence"].notna()
    & score_pairs["Tumor Primary"].notna()
)

r1_higher_rs = (
    score_pairs.loc[valid_rs, "First Recurrence"]
    > score_pairs.loc[valid_rs, "Tumor Primary"]
).copy()

n_r1_higher_rs = score_pairs.loc[r1_higher_rs].index.nunique()
n_pairs_rs = int(valid_rs.sum())

fraction_r1_higher_rs = n_r1_higher_rs / n_pairs_rs
percent_r1_higher_rs = (
    score_pairs.loc[valid_rs, "First Recurrence"]
    > score_pairs.loc[valid_rs, "Tumor Primary"]
).mean()
```

```
[221]: print("Recurrence Score paired comparison")
print(f"Mean delta: {mean_delta_rs:.4f}")
print(f"Median delta: {median_delta_rs:.4f}")
print(f"Paired t-test: t = {float(t_statistic_rs):.3f}, p = {float(p_value_rs):.3e}, {format_pvalue(p_value_rs)}")

print(
    f"R1 > Primary: "
    f"{n_r1_higher_rs}/{n_pairs_rs} "
    f"({100 * percent_r1_higher_rs:.1f}%)
")
```

```
Recurrence Score paired comparison
Mean delta: 0.2227
Median delta: 0.2270
Paired t-test: t = 9.249, p = 3.735e-15, <0.000001
R1 > Primary: 82/103 (79.6%)
```

```
[222]: fig, ax = plt.subplots(figsize=(7, 6))
```



```

for _, row in score_pairs.iterrows():
    ax.plot(
        ["Tumor Primary", "First Recurrence"],
        [row["Tumor Primary"], row["First Recurrence"]],
        marker="o",
        linewidth=0.8,
        alpha=0.35
    )

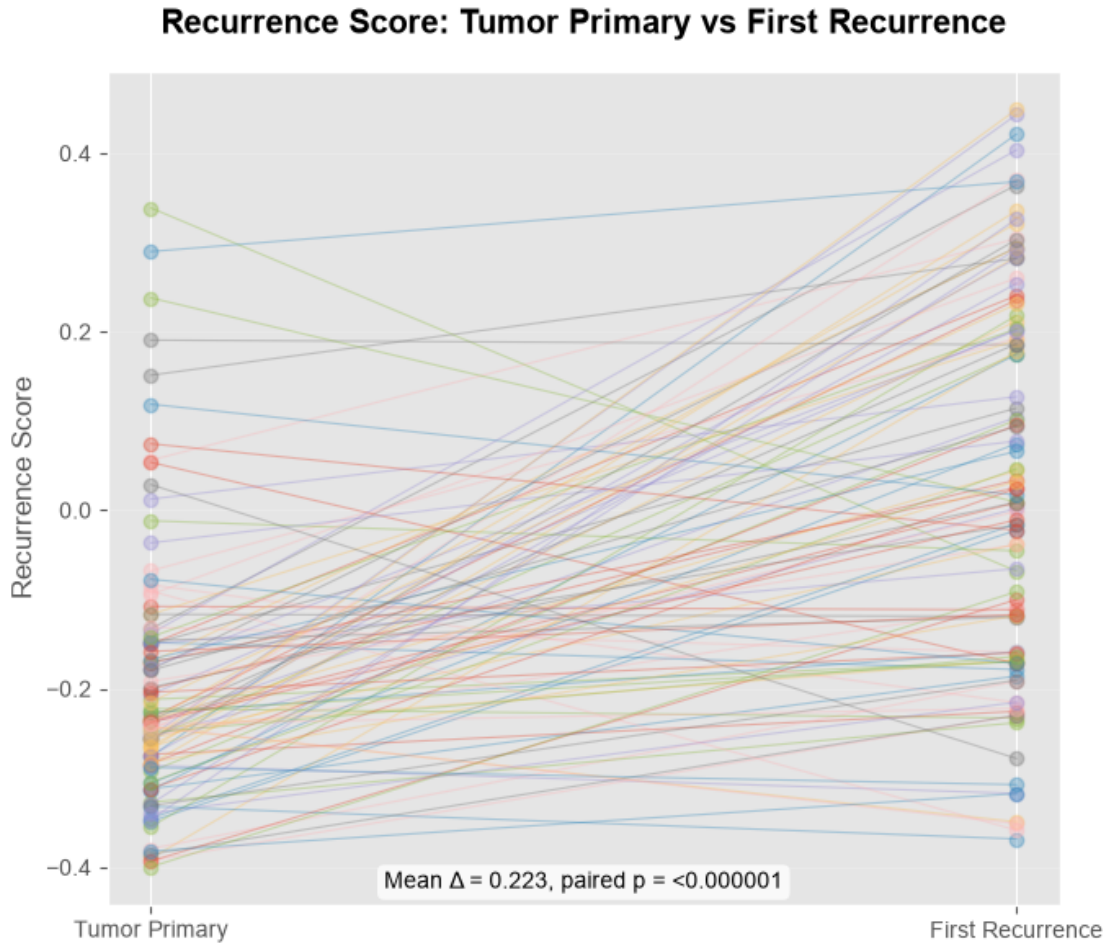
ax.set_ylabel("Recurrence Score")
ax.set_title("Recurrence Score: Tumor Primary vs First Recurrence",
             fontweight="bold", y=1.03)
ax.text(
    0.5,
    0.02,
    f"Mean  $\Delta$  = {mean_delta_rs:.3f}, "
    f"paired p = {format_pvalue(p_value_rs)}",
    transform=ax.transAxes,
    ha="center",
    bbox=dict(
        boxstyle="round,pad=0.3", facecolor="white", edgecolor="none", alpha=0.8
    )
)
ax.grid(axis="y", alpha=0.2)

plt.tight_layout()

plt.savefig(
    "../../figures/glass_recurrence_score_paired.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



- `glass_recurrence_score_paired.png` exported to `/figures` directory

Summary Note: The Recurrence Score increased substantially from Primary to first recurrence across the GLASS cohort (mean Δ = +0.223; median Δ = +0.227). The increase was statistically significant in the paired analysis (paired t-test, $t = 9.249$, $p = 3.74 \times 10^{-1}$), with 82/103 patients (79.6%) showing a higher Recurrence Score in their first recurrent samples than in their primary tumour sample. These findings support the ability of the rank-based Recurrence Score to capture a transcriptional program associated with tumour recurrence.

4.8 8. Out-of-fold validation using 5-fold cross-validation

- Is it possible to recognise recurrence in patients that were not part of creating the recurrence signature?
- out-of-fold recurrence score

```
[223]: expr = glass_idhwt_gbm_primary_R1_log2TPM_filtered
pairs = glass_idhwt_gbm_primary_R1_pairs
```

```
genes = expr.index
n_patients = len(pairs)
```

```
[224]: glass_idhwt_gbm_primary_R1_pairs.head()
```

```
[224]:
```

	PATIENT_ID	PRIMARY_SAMPLE_ID	FIRST_RECURRENCE_SAMPLE_ID
0	GLSS-19-0267	GLSS-19-0267-TP	GLSS-19-0267-R1
1	GLSS-19-0268	GLSS-19-0268-TP	GLSS-19-0268-R1
2	GLSS-19-0269	GLSS-19-0269-TP	GLSS-19-0269-R1
3	GLSS-19-0271	GLSS-19-0271-TP	GLSS-19-0271-R1
4	GLSS-19-0272	GLSS-19-0272-TP	GLSS-19-0272-R1

```
[225]: kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

oof_scores = []

assert len(pairs) == 103
assert pairs["PATIENT_ID"].nunique() == 103
assert pairs["PATIENT_ID"].is_unique

for fold, (train_idx, test_idx) in enumerate(
    kf.split(pairs),
    start=1
):
    train_idx = np.asarray(train_idx, dtype=int)
    test_idx = np.asarray(test_idx, dtype=int)

    print(f"\nFold {fold}/5")
    print(f"Training patients: {len(train_idx)}")
    print(f"Test patients: {len(pd.DataFrame(test_idx))}")

    # -----
    # TRAINING SET
    # -----

    train_pairs = pairs.iloc[train_idx]

    primary_ids = train_pairs["PRIMARY_SAMPLE_ID"].tolist()
    r1_ids = train_pairs["FIRST_RECURRENCE_SAMPLE_ID"].tolist()

    primary_train = expr.loc[:, primary_ids]
    r1_train = expr.loc[:, r1_ids]
```

```

delta_train = (
    r1_train.to_numpy() -
    primary_train.to_numpy()
)

n_train_genes = delta_train.shape[0]

n = np.sum(
    ~np.isnan(delta_train),
    axis=1
)

mean_delta = np.nanmean(
    delta_train,
    axis=1
)

sd_delta = np.nanstd(
    delta_train,
    axis=1,
    ddof=1
)

t_statistic = (
    mean_delta /
    (sd_delta / np.sqrt(n))
)

p_values = np.full(
    n_train_genes,
    np.nan,
    dtype=float
)

valid_tests = (
    (n >= 2) &
    np.isfinite(t_statistic)
)

p_values[valid_tests] = (
    2 *
    t.sf(
        np.abs(t_statistic[valid_tests]),
        df=n[valid_tests] - 1
    )
)

```

```

fdr = np.full(
    n_train_genes,
    np.nan,
    dtype=float
)

valid_p = np.isfinite(p_values)

fdr[valid_p] = multipletests(
    p_values[valid_p],
    method="fdr_bh"
)[1]

fraction_up = np.divide(
    np.sum(
        (delta_train > 0) &
        ~np.isnan(delta_train),
        axis=1
    ),
    n,
    out=np.full(
        n.shape,
        np.nan,
        dtype=float
    ),
    where=n > 0
)

fraction_down = np.divide(
    np.sum(
        (delta_train < 0) &
        ~np.isnan(delta_train),
        axis=1
    ),
    n,
    out=np.full(
        n.shape,
        np.nan,
        dtype=float
    ),
    where=n > 0
)

train_up = (
    (fdr < 0.05) &
    (mean_delta >= 0.5) &
    (fraction_up >= 0.65)
)

```

```

)

train_down = (
    (fdr < 0.05) &
    (mean_delta <= -0.5) &
    (fraction_down >= 0.65)
)

up_genes = genes[train_up]
down_genes = genes[train_down]

print(f"Training UP genes: {len(up_genes)}")
print(f"Training DOWN genes: {len(down_genes)}")

# -----
# TEST SET
# -----

test_pairs = pairs.iloc[test_idx]

test_sample_ids = (
    test_pairs["PRIMARY_SAMPLE_ID"].tolist()
    +
    test_pairs["FIRST_RECURRENCE_SAMPLE_ID"].tolist()
)

test_expr = expr.loc[:, test_sample_ids]

test_percentile_ranks = test_expr.rank(
    axis=0,
    method="average",
    pct=True
)

up_score = test_percentile_ranks.loc[
    test_percentile_ranks.index.intersection(
        up_genes
    )
].mean(axis=0)
up_scores = up_score.to_numpy()

down_score = test_percentile_ranks.loc[
    test_percentile_ranks.index.intersection(
        down_genes
    )
].mean(axis=0)
down_scores = down_score.to_numpy()

```

```

recurrence_score = (
    up_score -
    down_score
)

fold_scores = pd.DataFrame({
    "SAMPLE_ID": test_sample_ids,
    "UP_SCORE": up_score,
    "DOWN_SCORE": down_score,
    "RECURRENCE_SCORE": recurrence_score.to_numpy(),
    "FOLD": fold
})

fold_sample_info: pd.DataFrame = pd.concat([
    test_pairs.loc[:, ["PATIENT_ID", "PRIMARY_SAMPLE_ID"]
    ].rename({"PRIMARY_SAMPLE_ID": "SAMPLE_ID"},
            axis="columns"
    ).assign(SAMPLE_TYPE="Tumor Primary"),
    test_pairs.loc[:, ["PATIENT_ID", "FIRST_RECURRENCE_SAMPLE_ID"]
    ].rename({"FIRST_RECURRENCE_SAMPLE_ID": "SAMPLE_ID"},
            axis="columns",
    ).assign(SAMPLE_TYPE="First Recurrence")
], ignore_index=True
)

fold_scores = fold_sample_info.merge(
    fold_scores,
    on="SAMPLE_ID",
    how="left",
    validate="one_to_one"
)

oof_scores.append(fold_scores)

```

Fold 1/5

Training patients: 82

Test patients: 21

Training UP genes: 348

Training DOWN genes: 186

Fold 2/5

Training patients: 82

Test patients: 21

Training UP genes: 217

Training DOWN genes: 342

Fold 3/5
 Training patients: 82
 Test patients: 21
 Training UP genes: 351
 Training DOWN genes: 189

Fold 4/5
 Training patients: 83
 Test patients: 20
 Training UP genes: 336
 Training DOWN genes: 202

Fold 5/5
 Training patients: 83
 Test patients: 20
 Training UP genes: 242
 Training DOWN genes: 191

```
[226]: glass_recurrence_scores_out_of_fold: pd.DataFrame = pd.concat(
        oof_scores,
        ignore_index=True
    )
```

```
[227]: glass_recurrence_scores_out_of_fold.head()
```

```
[227]:
```

	PATIENT_ID	SAMPLE_ID	SAMPLE_TYPE	UP_SCORE	DOWN_SCORE	\
0	GLSS-19-0267	GLSS-19-0267-TP	Tumor Primary	0.445157	0.552165	
1	GLSS-19-0272	GLSS-19-0272-TP	Tumor Primary	0.348267	0.597626	
2	GLSS-CU-P046	GLSS-CU-P046-TP	Tumor Primary	0.409731	0.522059	
3	GLSS-CU-P069	GLSS-CU-P069-TP	Tumor Primary	0.433734	0.660279	
4	GLSS-CU-R005	GLSS-CU-R005-TP	Tumor Primary	0.370635	0.603193	

	RECURRENCE_SCORE	FOLD
0	-0.107008	1
1	-0.249359	1
2	-0.112328	1
3	-0.226545	1
4	-0.232557	1

```
[228]: print(f"\nOut-of-fold scores calculated for "
        f"{len(glass_recurrence_scores_out_of_fold)} samples "
        f"from {n_patients} patients.")

print(glass_recurrence_scores_out_of_fold[
    ["UP_SCORE",
     "DOWN_SCORE",
     "RECURRENCE_SCORE"]].describe())
```


Out-of-fold scores calculated for 206 samples from 103 patients.

	UP_SCORE	DOWN_SCORE	RECURRENCE_SCORE
count	206.000000	206.000000	206.000000
mean	0.452375	0.530004	-0.077629
std	0.127431	0.083768	0.188732
min	0.231240	0.315442	-0.355102
25%	0.354253	0.476431	-0.224113
50%	0.428000	0.540677	-0.133603
75%	0.533735	0.593634	0.041626
max	0.750986	0.692567	0.435544

```
[229]: glass_recurrence_scores_out_of_fold.to_excel(
        ".../results/recurrence/glass_recurrence_scores_out_of_fold.xlsx",
        index=False
    )
```

- glass_recurrence_scores_out_of_fold.xlsx exported to /results/recurrence directory.

4.9 9. Assessing the out-of-fold Recurrence Score

- $\text{OOF_DELTA_SCORE} = \text{OOF_SCORE_R1} - \text{OOF_SCORE_PRIMARY}$

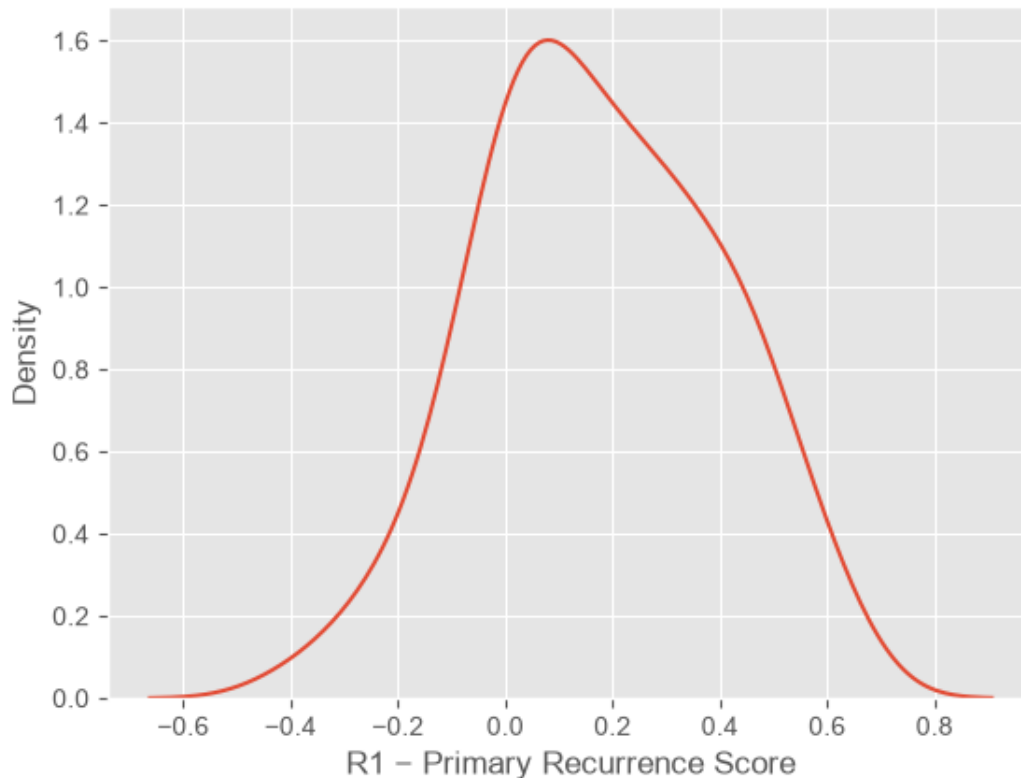
```
[230]: oof_pairs = pd.DataFrame(glass_recurrence_scores_out_of_fold.pivot_table(
        index="PATIENT_ID",
        columns="SAMPLE_TYPE",
        values="RECURRENCE_SCORE"
    ))

oof_pairs["OOF_DELTA_SCORE"] = (
    oof_pairs["First Recurrence"] -
    oof_pairs["Tumor Primary"]
)
```

```
[231]: sns.kdeplot(np.array(oof_pairs["OOF_DELTA_SCORE"]))

plt.title("Distribution of Paired Out-of-Fold Recurrence Score Differences",
        y=1.03)
plt.xlabel("R1 - Primary Recurrence Score")
plt.ylabel("Density")
plt.show()
```

Distribution of Paired Out-of-Fold Recurrence Score Differences



```
[232]: oof_delta_skew = float(skew(oof_pairs["OOF_DELTA_SCORE"], nan_policy="omit"))

print("Patient pairs:", glass_recurrence_scores_out_of_fold["PATIENT_ID"].
      ↪nunique())
print(f"Skewness:{float(oof_delta_skew):.3f}")
```

Patient pairs: 103

Skewness:-0.027

Methodological Note: The paired out-of-fold recurrence score differences exhibited minimal skewness (skewness = -0.027). Given the near-symmetric distribution of the differences and the sample size of $N = 103$ paired observations, the paired t -test was considered appropriate. The sample size also provides robustness to moderate deviations from normality through the Central Limit Theorem.

```
[233]: mean_delta_oof = oof_pairs["OOF_DELTA_SCORE"].mean()
median_delta_oof = oof_pairs["OOF_DELTA_SCORE"].median()

t_statistic_oof, p_value_oof = ttest_rel(
    oof_pairs["First Recurrence"],
    oof_pairs["Tumor Primary"])
```

```

)

valid_oof = (
    oof_pairs["First Recurrence"].notna()
    & oof_pairs["Tumor Primary"].notna()
)

r1_higher_oof = (
    oof_pairs.loc[valid_oof, "First Recurrence"]
    > oof_pairs.loc[valid_oof, "Tumor Primary"]
).copy()

n_r1_higher_oof = oof_pairs.loc[r1_higher_oof].index.nunique()
n_pairs_oof = int(valid_oof.sum())

fraction_r1_higher_oof = n_r1_higher_oof / n_pairs_oof
percent_r1_higher_oof = (
    oof_pairs.loc[valid_oof, "First Recurrence"]
    > oof_pairs.loc[valid_oof, "Tumor Primary"]
).mean()

```

```

[234]: t_statistic_oof = float(t_statistic_oof)
p_value_oof = float(p_value_oof)

print("Out-of-fold Recurrence Score evaluation")
print(f"Patients: {len(oof_pairs)}")
print(f"Mean OOF delta: {mean_delta_oof:.4f}")
print(f"Median OOF delta: {median_delta_oof:.4f}")
print(
    f"Paired t-test: "
    f"t = {t_statistic_oof:.3f}, "
    f"p = {p_value_oof:.3e} {format_pvalue(p_value_oof)}")

print(
    f"R1 > Primary: "
    f"{n_r1_higher_oof}/{n_pairs_oof} "
    f"({100 * percent_r1_higher_oof:.1f}%)")

```

```

Out-of-fold Recurrence Score evaluation
Patients: 103
Mean OOF delta: 0.1758
Median OOF delta: 0.1680
Paired t-test: t = 8.004, p = 2.001e-12 <0.000001
R1 > Primary: 80/103 (77.7%)

```

```

[235]: fig, ax = plt.subplots(figsize=(7, 6))

for _, row in oof_pairs.iterrows():
    ax.plot(
        ["Tumor Primary", "First Recurrence"],
        [row["Tumor Primary"], row["First Recurrence"]],
        marker="o",
        linewidth=0.8,
        alpha=0.35
    )

ax.set_ylabel("Out-of-fold Recurrence Score")
ax.set_title("Out-of-fold Recurrence Score: Primary vs R1", fontweight="bold",
             y=1.03)

ax.text(
    0.5,
    0.02,
    f"Mean  $\Delta$  = {mean_delta_oof:.3f}, "
    f"paired p = {format_pvalue(p_value_oof)}",
    transform=ax.transAxes,
    ha="center",
    bbox=dict(
        boxstyle="round,pad=0.3", facecolor="white", edgecolor="none", alpha=0.8
    )
)

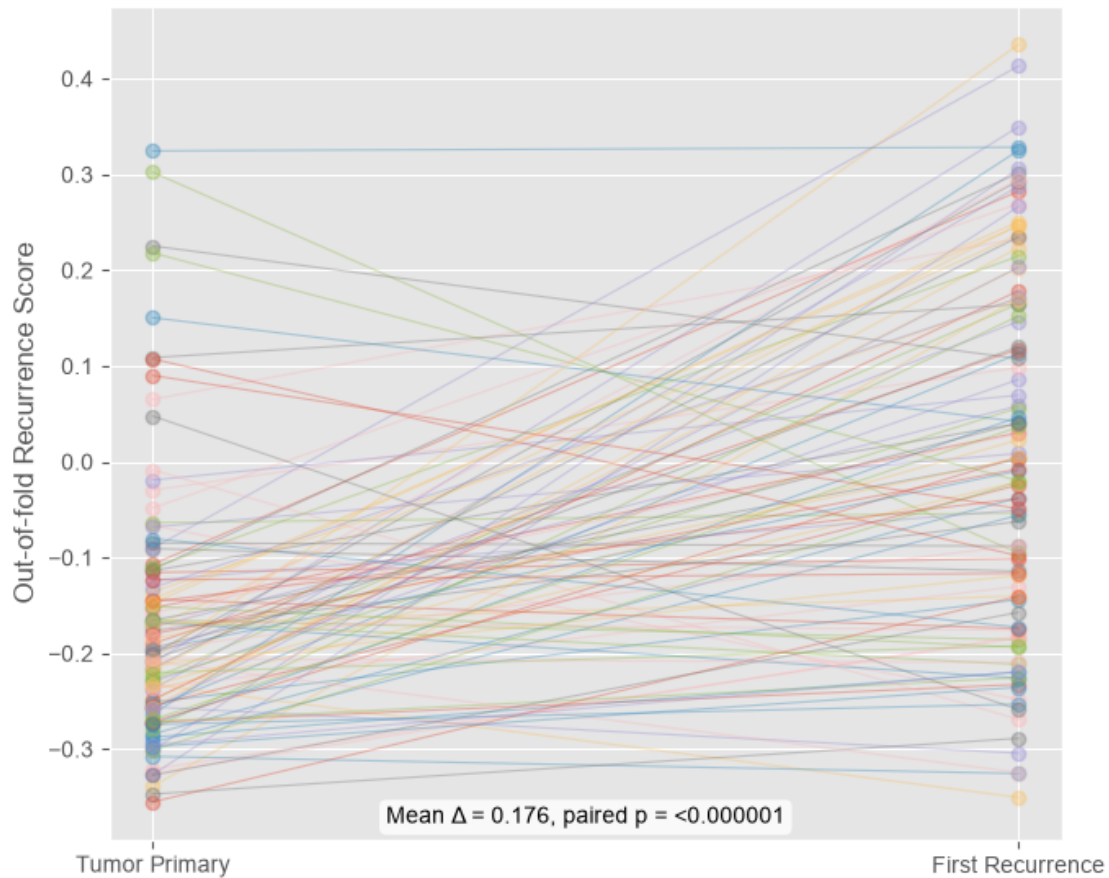
plt.tight_layout()

plt.savefig(
    "../../figures/glass_recurrence_score_out_of_fold_paired.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```

Out-of-fold Recurrence Score: Primary vs R1



```
[236]: roc_data = glass_recurrence_scores_out_of_fold.copy()

roc_data["LABEL"] = (
    roc_data["SAMPLE_TYPE"] == "First Recurrence"
).astype(int)

oof_auc = roc_auc_score(
    roc_data["LABEL"],
    roc_data["RECURRENCE_SCORE"]
)

print(f"OOF ROC AUC: {oof_auc:.3f}")
```

OOF ROC AUC: 0.770

Summary Note: The Recurrence Score was consistently higher at first recurrence than in the corresponding Primary samples (mean $\Delta = 0.176$; median $\Delta = 0.168$; paired $t = 8.004$, $p = 2.00 \times 10^{-12}$), with 77.7% of patients showing an increased score at R1. The

out-of-fold Recurrence Score also discriminated between Primary and R1 samples with an ROC AUC of 0.770. These findings support the ability of the Recurrence Signature to identify recurrence-associated transcriptional patterns in patients who were not used to construct the corresponding signature.

4.10 10. Sensitivity analysis in patients receiving standard therapy

- Treatment selection and interval mapping
 - Assessable window: Only treatments initiated strictly between primary and recurrent sampling dates ($\text{PRIMARY_DATE} < \text{START_DATE} \leq \text{RECURRENCE_DATE}$) were included.
 - Invalid or non-chronological dates were marked `pd.NA`.
 - Standard therapies:
 - * Radiotherapy (RT): Any radiation initiated within the window.
 - * Temozolomide (TMZ): Standard monotherapy (`temozolomide`, `temodar`, `tmz`).
 - * Concurrent TMZ+RT: Simultaneous initiation of TMZ and RT on the exact same date (`START_DATE`).
 - * Other Systemic: Includes all non-TMZ chemotherapy, modified dosing (e.g., LD TMZ), combination regimens (e.g., TMZ + Lomustine), and completely different regimens (e.g., Bevacizumab, PCV).

```
[237]: print("From cohort, has received TMZ treatment:", len(tmz))
print("From cohort, has received radiotherapy:", len(radiotherapy))
print("From cohort, has received concurrent TMZ treatment and radiotherapy:",
      ↪len(concurrent_tmz))
print("From cohort, has received other systemic treatments:",
      ↪len(other_systemic))
```

From cohort, has received TMZ treatment: 86

From cohort, has received radiotherapy: 97

From cohort, has received concurrent TMZ treatment and radiotherapy: 92

From cohort, has received other systemic treatments: 25

```
[238]: expr_matrix = glass_idhwt_gbm_primary_R1_log2TPM_filtered.copy()

scores_series = calculate_recurrence_score(expr_matrix, consensus_up_genes,
      ↪consensus_down_genes)

scores_df = pd.DataFrame(
    {"SAMPLE_ID": scores_series.index, "RECURRENCE_SCORE":
     ↪scores_series["RECURRENCE_SCORE"]}
)

scores_df = scores_df.merge(
    glass_recurrence_scores_full_cohort[["SAMPLE_ID", "PATIENT_ID"]].
    ↪drop_duplicates(),
    on="SAMPLE_ID",
    how="left",
)
```

```

[239]: subgroups = {
        "Standard TMZ + RT": (tmz & radiotherapy),
        "Standard TMZ + RT (No Other Systemic)": (tmz & radiotherapy ~
        ↳ other_systemic)
    }

    paired_dfs = {}

    for title, patient_ids in subgroups.items():
        sub_df = treatment_pairs.loc[treatment_pairs["PATIENT_ID"].
        ↳ isin(patient_ids)].copy()

        samples: pd.DataFrame = pd.concat(
            [sub_df[["PATIENT_ID", "PRIMARY_SAMPLE_ID"]
            ].rename(columns={"PRIMARY_SAMPLE_ID": "SAMPLE_ID"})
            .assign(SAMPLE_TYPE="Tumor Primary"),
            sub_df[["PATIENT_ID", "FIRST_RECURRENCE_SAMPLE_ID"]
            ].rename(columns={"FIRST_RECURRENCE_SAMPLE_ID": "SAMPLE_ID"})
            .assign(SAMPLE_TYPE="First Recurrence"),
            ], ignore_index=True, )

        merged = samples.merge(
            scores_df,
            on=["PATIENT_ID", "SAMPLE_ID"],
            how="left", )

        paired = (
            merged
            .pivot_table(
                index="PATIENT_ID",
                columns="SAMPLE_TYPE",
                values="RECURRENCE_SCORE", ).reset_index())

        paired["DELTA_RECURRENCE_SCORE"] = (
            paired["First Recurrence"]
            - paired["Tumor Primary"])

        paired_dfs[title] = paired

```

```

[240]: with pd.ExcelWriter(
        "../results/recurrence/glass_recurrence_score_standard_treatment.
        ↳ xlsx"
    ) as writer:
        for title, df in paired_dfs.items():
            sheet = (
                "Standard TMZ RT"
                if "No Other" not in title

```

```

        else "Standard_TMZ_RT_No_Other"
    )
    df.to_excel(writer, sheet_name=sheet, index=False)

```

- glass_recurrence_score_standard_treatment.xlsx exported to /results/recurrence directory

```

[241]: fig, axes = plt.subplots(1, 2, figsize=(12, 6), sharey=True)

for ax, (title, paired) in zip(axes, paired_dfs.items()):
    mean_delta = paired["DELTA_RECURRENCE_SCORE"].mean()
    _, p_val = ttest_rel(paired["First Recurrence"], paired["Tumor Primary"])

    for _, row in paired.iterrows():
        ax.plot(
            ["Tumor Primary", "First Recurrence"],
            [row["Tumor Primary"], row["First Recurrence"]],
            marker="o",
            linewidth=0.8,
            alpha=0.35,
        )

    ax.set_ylabel("Recurrence Score")
    ax.set_title(f"{title}\n(n = {len(paired)})", fontweight="bold", y=1.03)

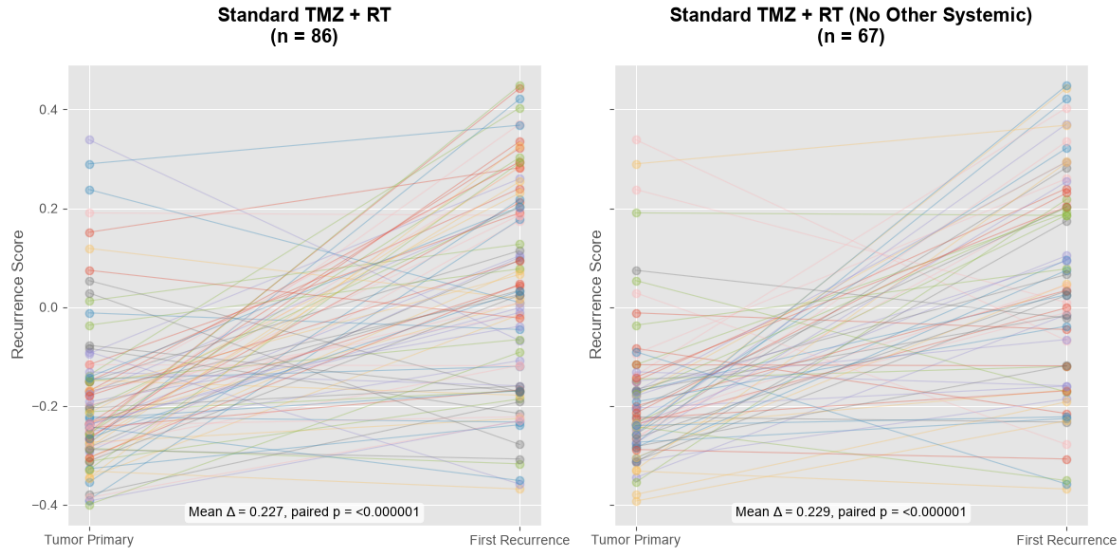
    ax.text(
        0.5,
        0.02,
        f"Mean Δ = {mean_delta:.3f}, paired p = {format_pvalue(p_val)}",
        transform=ax.transAxes,
        ha="center",
        bbox=dict(
            boxstyle="round,pad=0.3", facecolor="white", edgecolor="none",
            alpha=0.8
        ),
    )

plt.tight_layout()

plt.savefig(
    "../../figures/glass_recurrence_score_standard_treatment.png",
    dpi=300,
    bbox_inches="tight",
)

plt.show()

```

- `glass_recurrence_score_standard_treatment.png` exported to `/figures` directory

Summary Note: Sensitivity analyses confirmed that the observed increase in Recurrence Score is not driven by non-standard treatment regimens.

- **Standard TMZ + RT Cohort:** Recurrence Scores increased significantly from Primary to R1 (mean $\Delta = 227$, paired t-test $p < 0.000001$).
- **Standard TMZ + RT (No Other Systemic Therapy):** The Recurrence Score increase remained robust and statistically significant when excluding patients who received non-standard systemic agents (mean $\Delta = 229$, paired t-test $p < 0.000001$).

These results confirm that the consensus Recurrence Signature tracks intrinsic glioblastoma progression under standard treatment.

5 Can the transcriptional program associated with tumour recurrence be identified in glioblastoma cell lines?

5.1 1. Solidifying Recurrence Score

```
[242]: glass_recurrence_UP_consensus = consensus_up.copy()
       glass_recurrence_DOWN_consensus = consensus_down.copy()
```

```
[243]: glass_recurrence_signature_final = pd.concat(
       [
           glass_recurrence_UP_consensus,
           glass_recurrence_DOWN_consensus,
       ],
       ignore_index=True,
       ).rename(
       columns={
```

```

        "ORIGINAL_DIRECTION": "DIRECTION",
        "ORIGINAL_MEAN_DELTA": "MEAN_DELTA",
        "ORIGINAL_FDR": "FDR",
    }
)

glass_recurrence_signature_final = glass_recurrence_signature_final[
    [
        "GENE",
        "DIRECTION",
        "MEAN_DELTA",
        "FDR",
        "BOOTSTRAP_SAME_DIRECTION_FREQUENCY",
    ]
].copy()

```

```
[244]: glass_recurrence_signature_final.head()
```

```
[244]:
```

	GENE	DIRECTION	MEAN_DELTA	FDR	BOOTSTRAP_SAME_DIRECTION_FREQUENCY
0	ENPP2	UP	1.361706	0.000006	0.999
1	MAL	UP	1.647429	0.000004	0.995
2	SLC45A3	UP	0.892563	0.000010	0.993
3	ADIRF	UP	0.990164	0.000008	0.991
4	GREM1	UP	0.965628	0.000008	0.985

```
[245]: glass_recurrence_signature_final.to_excel(
        ".../results/recurrence/glass_recurrence_signature_final.xlsx",
        index=False
    )

```

- glass_recurrence_signature_final.xlsx exported to /results/recurrence directory

5.2 2. Defining the final glioma cell-line cohort

5.2.1 Load and filter expression data

- Model.csv was used for identification of cell line oncotree lineages
- Source: https://depmap.org/portal/data_page/?tab=allData&releasename=DepMap%20Public%2026Q1&
- Two expression datasets were used:

```
[246]: model = pd.read_csv(
        ".../data/raw/annotation/Model.csv",
        index_col=0,
        low_memory=False,
    )

model.index.name = "ModelID"

```

```
[247]: model_metadata = (
    model[
        [
            "CellLineName",
            "StrippedCellLineName",
            "COSMICID",
            "CCLEName",
            "SangerModelID",
            "ModelIDAlias",
            "OncotreeLineage",
            "OncotreePrimaryDisease"
        ]
    ]
    .copy()
)

model_metadata.index.name = "ModelID"
```

```
[248]: glioma_model_ids = model.index[
    model["OncotreePrimaryDisease"]
    .str.contains("glioma", case=False, na=False)
]

print("Total glioma cell lines in Model.csv:", len(glioma_model_ids))
```

Total glioma cell lines in Model.csv: 96

DepMap - Log-transformed TPM (Transcripts Per Million) values at gene level derived from unstranded RNA-seq for all genes in human

- Reference: DepMap, Broad (2026). DepMap Public 26Q1. Dataset. depmap.org
- **Raw Data:** OmicsExpressionTPMLogp1HumanAllGenes.csv
- **Source:** <https://depmap.org/portal/download/all/?release=DepMap+Public+26Q1&file=OmicsExpressionTPMLogp1HumanAllGenes.csv>

```
[249]: OmicsExpressionTPMLogp1HumanAllGenes = pd.read_csv(
    "../../data/raw/DepMap/expression/OmicsExpressionTPMLogp1HumanAllGenes.csv",
    index_col=0,
)

gene = pd.read_csv(
    "../../data/raw/annotation/Gene.csv",
    index_col=0,
    low_memory=False
)
```

```
[250]: OmicsExpressionTPMLogp1HumanAllGenes.iloc[:5, :20]
```

```

[250]: SequencingID ModelConditionID      ModelID IsDefaultEntryForMC \
0    CDS-010xbm    MC-001113-k2lR    ACH-001113                Yes
1    CDS-02TzJp    MC-001289-BpdI    ACH-001289                Yes
2    CDS-0693hw    MC-001339-5nRN    ACH-001339                Yes
3    CDS-07Plat    MC-001619-IR6I    ACH-001619                No
4    CDS-08F0cu    MC-001979-E3qW    ACH-001979                Yes

      IsDefaultEntryForModel  TSPAN6 (7105)  TNMD (64102)  DPM1 (8813)  \
0                      Yes      4.956577      0.000000      7.577648
1                      Yes      4.955015      0.617117      7.333933
2                      Yes      3.421952      0.000000      7.546069
3                      No       5.196729      0.000000      6.362268
4                      Yes      4.651643      0.000000      5.946408

      SCYL3 (57147)  FIRRM (55732)  FGR (2268)  CFH (3075)  FUCA2 (2519)  \
0      3.179411      4.765742      0.037483      1.510547      3.103039
1      2.782935      3.735371      0.000000      0.148922      3.855123
2      2.615880      4.476233      0.064571      1.397491      6.833915
3      2.144996      3.087183      0.036001      5.118850      7.013295
4      2.454515      1.852111      0.000000      7.680796      6.156866

      GCLC (2729)  NFYA (4800)  STPG1 (90529)  NIPAL3 (57185)  LAS1L (81887)  \
0      6.697482      4.852303      3.497101      4.253384      5.039358
1      4.376707      3.465922      1.977529      3.223240      4.790318
2      3.980336      3.437155      3.078680      3.620422      4.155158
3      6.177005      3.509987      3.317112      3.998484      5.645440
4      5.042574      3.258451      3.194224      5.207666      4.629963

      ENPP4 (22875)  SEMA3F (6405)
0      2.873880      3.226268
1      4.280221      1.461932
2      1.102793      0.262396
3      2.898895      2.774494
4      0.000000      0.419227

```

```

[251]: depmap_exp = OmicsExpressionTPMLogp1HumanAllGenes.copy()

depmap_exp = depmap_exp[
    depmap_exp["IsDefaultEntryForModel"].eq("Yes")
].copy()

depmap_exp = depmap_exp.drop(
    columns=[
        "SequencingID",
        "ModelConditionID",
        "IsDefaultEntryForMC",
        "IsDefaultEntryForModel"
    ]
)

```

```

    ]
).copy()

depmap_exp = depmap_exp.set_index("ModelID")
depmap_exp = depmap_exp.rename(columns=lambda x: re.sub(r"\s*(.*?)", "", x))

```

```

[252]: gene_map = (
        gene
        .dropna(subset=["ensembl_gene_id", "symbol"])
        .drop_duplicates("ensembl_gene_id")
        .set_index("ensembl_gene_id")["symbol"]
    )

```

```

[253]: mapped = depmap_exp.columns.to_series().map(gene_map)

print("Total genes:", len(depmap_exp.columns))
print("Mapped:", mapped.notna().sum())
print("Unmapped:", mapped.isna().sum())

```

```

Total genes: 60649
Mapped: 183
Unmapped: 60466

```

```

[254]: duplicates = depmap_exp.columns[depmap_exp.columns.duplicated()]

print("Duplicate symbols:", len(duplicates))
print(duplicates.tolist())

```

```

Duplicate symbols: 0
[]

```

```

[255]: original_columns = depmap_exp.columns.to_series()

depmap_exp.columns = (
    original_columns
    .map(gene_map)
    .fillna(original_columns)
)

```

```

[256]: depmap_exp.iloc[:5, :5]

```

```

[256]:
      TSPAN6  TNMD  DPM1  SCYL3  FIRRM
ModelID
ACH-001113  4.956577  0.000000  7.577648  3.179411  4.765742
ACH-001289  4.955015  0.617117  7.333933  2.782935  3.735371
ACH-001339  3.421952  0.000000  7.546069  2.615880  4.476233
ACH-001979  4.651643  0.000000  5.946408  2.454515  1.852111
ACH-002438  4.336705  0.000000  6.879387  2.262824  3.256491

```

```
[257]: depmap_default_entry_ids = OmicsExpressionTPMLogp1HumanAllGenes["ModelID"] [
    OmicsExpressionTPMLogp1HumanAllGenes["IsDefaultEntryForModel"]
    == "Yes"
]

depmap_exp = depmap_exp.loc[
    depmap_exp.index.isin(depmap_default_entry_ids)
].copy()
```

```
[258]: depmap_exp_metadata = model.reindex(depmap_exp.index)

depmap_glioma_ids = depmap_exp_metadata.index[
    depmap_exp_metadata["OncotreePrimaryDisease"]
    .str.contains("glioma", case=False, na=False)
]

depmap_glioma_exp = depmap_exp.loc[
    depmap_exp.index.isin(depmap_glioma_ids)
].copy()

print("DepMap total:", depmap_exp.index.nunique())
print("DepMap glioma:", depmap_glioma_exp.index.nunique())
print("DepMap expression variables:", depmap_glioma_exp.columns.nunique())
```

DepMap total: 1719

DepMap glioma: 74

DepMap expression variables: 60649

5.2.2 DepMap expression data quality check

```
[259]: # noinspection dict-creation
depmap_expression_qc = {}

depmap_expression_qc["Dataset"] = (
    "OmicsExpressionTPMLogp1HumanAllGenes.csv"
)

depmap_expression_qc["Platform"] = (
    "Log-Transformed RNA-seq"
)

depmap_expression_qc["Total_cell_lines"] = (
    depmap_exp.index.nunique()
)

depmap_expression_qc["Total_expression_variables"] = (
    depmap_exp.columns.nunique()
)
```

```

depmap_expression_qc["Minimum_expression"] = (
    np.nanmin(depmap_exp.to_numpy(dtype=float))
)

depmap_expression_qc["Maximum_expression"] = (
    np.nanmax(depmap_exp.to_numpy(dtype=float))
)

```

```

[260]: missing_depmap_expression = glioma_model_ids.difference(depmap_exp.index)

print("Glioma cell lines according to Model.csv:", len(glioma_model_ids))
print("Glioma cell lines with DepMap expression:", depmap_glioma_exp.index.
      ↪nunique())
print("Glioma cell lines missing DepMap expression:", ↪
      ↪len(missing_depmap_expression))

```

Glioma cell lines according to Model.csv: 96
 Glioma cell lines with DepMap expression: 74
 Glioma cell lines missing DepMap expression: 22

```

[261]: depmap_missing_metadata = model.loc[
        missing_depmap_expression,
        [
            "CellLineName",
            "COSMICID",
            "OncotreeLineage",
            "OncotreePrimaryDisease",
            "OncotreeSubtype",
        ]
    ].copy()

```

```

[262]: depmap_expression_available = depmap_exp.index.nunique()

depmap_expression_qc["Glioma_cohort_size"] = len(glioma_model_ids)

depmap_expression_qc["Glioma_with_DepMap_expression"] = (
    depmap_expression_available
)

depmap_expression_qc["Glioma_missing_DepMap_expression"] = (
    len(missing_depmap_expression)
)

depmap_expression_qc["Expression_variable_type"] = (
    "Gene symbols"
)

```

```
[263]: depmap_expression_qc["Duplicate_ModelIDs"] = (
        depmap_exp.index.duplicated().sum()
    )
```

```
[264]: all_missing_profiles = depmap_exp.index[
        depmap_exp.isna().all(axis=1)
    ]

    depmap_expression_qc["Completely_missing_profiles"] = (
        len(all_missing_profiles)
    )

    print(
        "Completely missing expression profiles:",
        all_missing_profiles.tolist()
    )
```

Completely missing expression profiles: []

```
[265]: depmap_expression_qc["Duplicated_gene_variables"] = (
        depmap_exp.columns.duplicated().sum()
    )
```

```
[266]: depmap_qc_summary = pd.DataFrame(
    {
        "Metric": [
            "Dataset",
            "Platform",
            "Total DepMap cell lines",
            "Glioma cell lines (Model.csv)",
            "Glioma cell lines with DepMap expression",
            "Glioma cell lines missing DepMap expression",
            "Expression variables",
            "Expression variable type",
            "Duplicated cell-line IDs",
            "Completely missing expression profiles",
            "Duplicated expression variables",
            "Minimum expression value",
            "Maximum expression value",
        ],
        "Value": [
            depmap_expression_qc["Dataset"],
            depmap_expression_qc["Platform"],
            depmap_expression_qc["Total_cell_lines"],
            depmap_expression_qc["Glioma_cohort_size"],
            depmap_expression_qc["Glioma_with_DepMap_expression"],
            depmap_expression_qc["Glioma_missing_DepMap_expression"],
            depmap_expression_qc["Total_expression_variables"],
        ]
    })
```



```

        depmap_expression_qc["Expression_variable_type"],
        depmap_expression_qc["Duplicate_ModelIDs"],
        depmap_expression_qc["Completely_missing_profiles"],
        depmap_expression_qc["Duplicated_gene_variables"],
        depmap_expression_qc["Minimum_expression"],
        depmap_expression_qc["Maximum_expression"],
    ],
}
)

```

```
[267]: depmap_qc_summary
```

```

[267]:
Metric \
0      Dataset
1      Platform
2      Total DepMap cell lines
3      Glioma cell lines (Model.csv)
4      Glioma cell lines with DepMap expression
5      Glioma cell lines missing DepMap expression
6      Expression variables
7      Expression variable type
8      Duplicated cell-line IDs
9      Completely missing expression profiles
10     Duplicated expression variables
11     Minimum expression value
12     Maximum expression value

Value
0      OmicsExpressionTPMLogp1HumanAllGenes.csv
1      Log-Transformed RNA-seq
2      1719
3      96
4      1719
5      22
6      60649
7      Gene symbols
8      0
9      0
10     0
11     0.0
12     17.628986

```

```

[268]: depmap_glioma = model.loc[
        model["OncotreePrimaryDisease"]
        .str.contains("glioma", case=False, na=False),
        ["CellLineName"]
    ].copy()

```

```

depmap_glioma = depmap_glioma.rename(
    columns={
        "CellLineName": "CELL_LINE_NAME"
    }
)

```

```

[269]: depmap_glioma_qc = pd.DataFrame(
        index=depmap_glioma.index
    )

    depmap_glioma_qc.index.name = "ModelID"

    depmap_glioma_qc["DEPMAP_EXPRESSION_AVAILABLE"] = (
        depmap_glioma_qc.index.isin(depmap_exp.index)
    )

    depmap_glioma_qc["CELL_LINE_NAME"] = (
        depmap_glioma["CELL_LINE_NAME"]
    )

```

```

[270]: duplicate_model_ids = (
        depmap_exp.index[
            depmap_exp.index.duplicated(keep=False)
        ]
        .unique()
    )

```

```

[271]: with pd.ExcelWriter(
        ".../results/qc/depmap_expression_QC.xlsx"
    ) as writer:
        depmap_qc_summary.to_excel(
            writer,
            sheet_name="Summary",
            index=False,
        )

        depmap_glioma_qc.to_excel(
            writer,
            sheet_name="Glioma_Cohort",
        )

        if len(missing_depmap_expression) > 0:
            pd.DataFrame(
                {"ModelID": missing_depmap_expression}
            ).to_excel(
                writer,

```

```

        sheet_name="Missing_Expression",
        index=False,
    )

    if len(duplicate_model_ids) > 0:
        pd.DataFrame(
            {"ModelID": duplicate_model_ids}
        ).to_excel(
            writer,
            sheet_name="Duplicate_Model_IDs",
            index=False,
        )

```

GDSC1/GDSC2 RNA-normalized Affymetrix HGU-219 Expression Matrix:

- As of 19/7/2026, the preprocessed expression data released by the Sanger Institute is not available on the original website any more.
- This source was used to trace back the original link: https://github.com/BorgwardtLab/Kernelized-Rank-Learning/blob/master/URLs_to_GDSC_datasets.txt
- **Raw Data ArrayExpress Accession:** E-MTAB-3610
- **Preprocessed Data:** data/raw/GDSC/expression/Cell_line_RMA_proc_basalExp.txt
- **Original Source:** http://www.cancerrxgene.org/gdsc1000/GDSC1000_WebResources/Data/preprocessed/
- **Currently Available Source:** <https://web.archive.org/web/20260209143748/http://www.cancerrxgene.org/>

```

[272]: gdsc_exp = pd.read_csv(
        ".../data/raw/GDSC/expression/Cell_line_RMA_proc_basalExp.txt",
        sep="\t",
        low_memory=False,
    )

```

```

[273]: gdsc_gene_metadata = gdsc_exp[
        ["GENE_SYMBOLS", "GENE_title"]
    ].copy()

    gdsc_gene_metadata["SOURCE"] = (
        gdsc_gene_metadata["GENE_title"]
        .str.extract(r"\[Source:([^\;]+)\]", expand=False)
        .str.strip()
        .str.replace(r"\s+Symbol$", "", regex=True)
    )

    gdsc_gene_metadata["GENE_SYMBOL"] = (
        gdsc_gene_metadata["GENE_SYMBOLS"]
        .str.strip()
    )

    gdsc_gene_metadata = gdsc_gene_metadata[

```

```

    ["GENE_SYMBOL", "SOURCE"]
].drop_duplicates()

gdsc_gene_metadata.head()

```

```

[273]:  GENE_SYMBOL  SOURCE
0      TSPAN6    HGNC
1        TNMD    HGNC
2        DPM1    HGNC
3       SCYL3    HGNC
4    C1orf112    HGNC

```

```

[274]: expression_columns = [
        col for col in gdsc_exp.columns
        if col not in {"GENE_SYMBOLS", "GENE_title"}
    ]

gdsc_exp[expression_columns] = (
    gdsc_exp[expression_columns]
    .apply(pd.to_numeric, errors="coerce")
)

```

```

[275]: gdsc_exp = (
        gdsc_exp
        .dropna(subset=["GENE_SYMBOLS"])
        .drop(columns=["GENE_title"])
        .set_index("GENE_SYMBOLS")
        .T
    )

gdsc_exp.index = (
    gdsc_exp.index
    .astype(str)
    .str.replace(r"^DATA\. ", "", regex=True)
    .str.strip()
)

gdsc_exp.columns.name = None
gdsc_exp.index.name = "COSMICID"

```

```

[276]: model_cosmic = (
        model[["COSMICID"]]
        .dropna()
        .copy()
    )

model_cosmic["COSMICID"] = (

```

```

    model_cosmic["COSMICID"]
    .astype("Int64")
    .astype("string")
)

cosmic_model_counts = (
    model_cosmic
    .reset_index()
    .groupby("COSMICID")["ModelID"]
    .nunique()
)

ambiguous_cosmic_ids = cosmic_model_counts[
    cosmic_model_counts > 1
]

print(
    "COSMICIDs mapping to multiple ModelIDs:",
    len(ambiguous_cosmic_ids),
)

```

COSMICIDs mapping to multiple ModelIDs: 0

```

[277]: cosmic_to_model = (
    model_cosmic
    .reset_index()
    .drop_duplicates("COSMICID")
    .set_index("COSMICID")["ModelID"]
)

gdsc_exp["ModelID"] = (
    pd.Series(gdsc_exp.index, index=gdsc_exp.index)
    .map(cosmic_to_model)
)

```

```

[278]: gdsc_exp = (
    gdsc_exp
    .set_index("ModelID")
)

gdsc_exp.index.name = "ModelID"

```

5.2.3 GDSC expression data quality check

```

[279]: #noinspection dict-creation
gdsc_expression_qc = {}

gdsc_expression_qc["Dataset"] = (

```

```

        "Cell_line_RMA_proc_basalExp"
    )

    gdsc_expression_qc["Platform"] = (
        "Affymetrix Human Genome U219"
    )

    gdsc_expression_qc["Total_cell_lines"] = (
        gdsc_exp.index.nunique()
    )

    gdsc_expression_qc["Total_expression_variables"] = (
        len(gdsc_exp.columns)
    )

    gdsc_expression_qc["Minimum_expression"] = (
        np.nanmin(gdsc_exp.to_numpy(dtype=float))
    )

    gdsc_expression_qc["Maximum_expression"] = (
        np.nanmax(gdsc_exp.to_numpy(dtype=float))
    )

```

```
[280]: gdsc_glioma_expression_ids = gdsc_exp.index.intersection(glioma_model_ids)
```

```
[281]: missing_gdsc_expression = glioma_model_ids.difference(gdsc_exp.index)

print("Glioma cell lines according to Model.csv:", len(glioma_model_ids))
print("Glioma cell lines with GDSC expression:",
      ↪len(gdsc_glioma_expression_ids))
print("Glioma cell lines missing GDSC expression:",
      ↪len(missing_gdsc_expression))

```

```

Glioma cell lines according to Model.csv: 96
Glioma cell lines with GDSC expression: 52
Glioma cell lines missing GDSC expression: 44

```

```
[282]: gdsc_glioma_exp = gdsc_exp.loc[
        gdsc_exp.index.isin(gdsc_glioma_expression_ids)
    ].copy()
```

```
[283]: gdsc_missing_metadata = model.loc[
        missing_gdsc_expression,
        [
            "CellLineName",
            "COSMICID",
            "OncotreeLineage",
            "OncotreePrimaryDisease",

```

```

        "OncotreeSubtype",
    ]
].copy()

```

```

[284]: gdsc_expression_available = gdsc_glioma_expression_ids[
        gdsc_glioma_expression_ids.isin(gdsc_exp.index)
    ]

gdsc_expression_qc["Glioma_cohort_size"] = gdsc_glioma_expression_ids.nunique()

gdsc_expression_qc["Glioma_with_expression"] = (
    gdsc_expression_available.nunique()
)

gdsc_expression_qc["Glioma_missing_expression"] = (
    gdsc_glioma_expression_ids.nunique()
    - gdsc_expression_available.nunique()
)

gdsc_expression_qc["Expression_variable_type"] = (
    "Gene symbols"
)

```

```

[285]: gdsc_expression_qc["Duplicate_ModelIDs"] = (
        gdsc_exp.index[gdsc_exp.index.duplicated(keep=False)].nunique()
    )

gdsc_expression_qc["Duplicate_glioma_ModelIDs"] = (
    gdsc_expression_available[
        gdsc_expression_available.duplicated(keep=False)
    ].nunique()
)

```

```

[286]: all_missing_profiles = gdsc_exp.index[
        gdsc_exp.isna().all(axis=1)
    ]

gdsc_expression_qc["Completely_missing_profiles"] = (
    len(all_missing_profiles)
)

print(
    "Completely missing expression profiles:",
    all_missing_profiles.tolist()
)

```

Completely missing expression profiles: []

```
[287]: gdsc_expression_qc["Duplicated_gene_variables"] = (
        gdsc_exp.columns.duplicated().sum()
    )
```

```
[288]: gdsc_qc_summary = pd.DataFrame(
    {
        "Metric": [
            "Dataset",
            "Platform",
            "Total cell lines",
            "Glioma cell lines",
            "Glioma cell lines with expression",
            "Glioma cell lines missing expression",
            "Expression variables",
            "Expression variable type",
            "Duplicated cell-line IDs",
            "Duplicated glioma cell-line IDs",
            "Completely missing expression profiles",
            "Duplicated expression variables",
            "Minimum expression value",
            "Maximum expression value",
        ],
        "Value": [
            gdsc_expression_qc["Dataset"],
            gdsc_expression_qc["Platform"],
            gdsc_expression_qc["Total_cell_lines"],
            gdsc_expression_qc["Glioma_cohort_size"],
            gdsc_expression_qc["Glioma_with_expression"],
            gdsc_expression_qc["Glioma_missing_expression"],
            gdsc_expression_qc["Total_expression_variables"],
            gdsc_expression_qc["Expression_variable_type"],
            gdsc_expression_qc["Duplicate_ModelIDs"],
            gdsc_expression_qc["Duplicate_glioma_ModelIDs"],
            gdsc_expression_qc["Completely_missing_profiles"],
            gdsc_expression_qc["Duplicated_gene_variables"],
            gdsc_expression_qc["Minimum_expression"],
            gdsc_expression_qc["Maximum_expression"],
        ],
    }
)
```

```
[289]: gdsc_qc_summary
```

```
[289]:
```

	Metric	Value
0	Dataset	Cell_line_RMA_proc_basalExp
1	Platform	Affymetrix Human Genome U219
2	Total cell lines	945

3	Glioma cell lines	52
4	Glioma cell lines with expression	52
5	Glioma cell lines missing expression	0
6	Expression variables	17419
7	Expression variable type	Gene symbols
8	Duplicated cell-line IDs	0
9	Duplicated glioma cell-line IDs	0
10	Completely missing expression profiles	0
11	Duplicated expression variables	0
12	Minimum expression value	2.065182
13	Maximum expression value	13.960295

```
[290]: gdsc_glioma = model.loc[
    model["OncotreePrimaryDisease"]
    .str.contains("glioma", case=False, na=False),
    ["CellLineName", "COSMICID"]
].copy()

gdsc_glioma = gdsc_glioma.rename(
    columns={
        "CellLineName": "CELL_LINE_NAME",
        "COSMICID": "GDSC_COSMIC_ID",
    }
)
```

```
[291]: gdsc_glioma_qc = pd.DataFrame(
    index=gdsc_glioma.index
)

gdsc_glioma_qc.index.name = "ModelID"

gdsc_glioma_qc["GDSC_EXPRESSION_AVAILABLE"] = (
    gdsc_glioma_qc.index.isin(gdsc_exp.index)
)

gdsc_glioma_qc["CELL_LINE_NAME"] = (
    gdsc_glioma["CELL_LINE_NAME"]
)

gdsc_glioma_qc["GDSC_COSMIC_ID"] = (
    gdsc_glioma["GDSC_COSMIC_ID"]
)
```

```
[292]: duplicate_model_ids = (
    gdsc_exp.index[
        gdsc_exp.index.duplicated(keep=False)
    ]
)
```

```
.unique()
)
```

```
[293]: with pd.ExcelWriter(
        ".../results/qc/gdsc_expression_QC.xlsx"
    ) as writer:
        gdsc_qc_summary.to_excel(
            writer,
            sheet_name="Summary",
            index=False,
        )

        gdsc_glioma_qc.to_excel(
            writer,
            sheet_name="Glioma_Cohort",
        )

        if len(missing_gdsc_expression) > 0:
            pd.DataFrame(
                {"ModelID": missing_gdsc_expression}
            ).to_excel(
                writer,
                sheet_name="Missing_Expression",
                index=False,
            )

        if len(duplicate_model_ids) > 0:
            pd.DataFrame(
                {"ModelID": duplicate_model_ids}
            ).to_excel(
                writer,
                sheet_name="Duplicate_Model_IDs",
                index=False,
            )
```

5.2.4 Load and filter sensitivity data

Checking if there's sensitivity data for all the expression data

Drug sensitivity data

- Reference: DepMap, Broad (2026). DepMap Public 26Q1. Dataset. depmap.org
- AUC is normalised to 1:
- **CTD² Raw Data:** data/raw/Drug_sensitivity_AUC_(CTD²)_subsetting.csv
- **DepMap Raw Data:** data/raw/Drug_sensitivity_AUC_(PRISM_Repurposing_Secondary_Screen)_subsetting.csv
- **GDSC1 Raw Data:** data/raw/Drug_sensitivity_AUC_(Sanger_GDSC1)_subsetting.csv
- **GDSC2 Raw Data:** data/raw/Drug_sensitivity_AUC_(Sanger_GDSC2)_subsetting.csv
- Sources:

- <https://depmap.org/portal/download/all/?release=Harmonized+CTD%5E2+25Q2&file=CTRPAUCMat>
- <https://depmap.org/portal/download/all/?release=Harmonized+PRISM+Repurposing+Secondary+Screen>
- <https://depmap.org/portal/download/all/?release=Harmonized+GDSC+25Q2&file=GDSC1AUCMat>
- <https://depmap.org/portal/download/all/?release=Harmonized+GDSC+25Q2&file=GDSC2AUCMat>
- AUC isn't normalised:
- **CTRPv2 Raw Data:**
 - `../../data/raw/DepMap/sensitivity/CTRPv2.2_2015_pub_CancerDisc_5_1210`
 - * `v22.data.auc_sensitivities.txt`
 - * `v22.meta.per_cell_line.txt`
 - * `v22.meta.per_compound.txt`
- **Source:** https://d1hp0ufb0xxisr.cloudfront.net/CTD2/Broad/CTRPv2.2_2015_pub_CancerDisc_5_1210

```
[294]: ctd2 = pd.read_csv(
        "../../data/raw/DepMap/sensitivity/"
        "Drug_sensitivity_AUC_(CTD^2)_subsetting.csv",
        index_col=0,
    )

prism = pd.read_csv(
        "../../data/raw/DepMap/sensitivity/"
        "Drug_sensitivity_AUC_(PRISM_Repurposing_Secondary_Screen)_subsetting.csv",
        index_col=0,
    )

gdsc1 = pd.read_csv(
        "../../data/raw/GDSC/sensitivity/"
        "Drug_sensitivity_AUC_(Sanger_GDSC1)_subsetting.csv",
        index_col=0,
    )

gdsc2 = pd.read_csv(
        "../../data/raw/GDSC/sensitivity/"
        "Drug_sensitivity_AUC_(Sanger_GDSC2)_subsetting.csv",
        index_col=0,
    )
```

Formatting CTRPv2 sensitivity data to match the others

```
[295]: auc = pd.read_csv(
        "../../data/raw/DepMap/sensitivity/CTRPv2.2_2015_pub_CancerDisc_5_1210/"
        "v22.data.auc_sensitivities.txt",
        sep="\t",
    )

cell_metadata = pd.read_csv(
        "../../data/raw/DepMap/sensitivity/CTRPv2.2_2015_pub_CancerDisc_5_1210/"
        "v22.meta.per_cell_line.txt",
        sep="\t",
    )
```

```
)

compound_metadata = pd.read_csv(
    "../../../data/raw/DepMap/sensitivity/CTRPv2.2_2015_pub_CancerDisc_5_1210/"
    "v22.meta.per_compound.txt",
    sep="\t",
)
```

```
[296]: def normalize_name(value):
        if pd.isna(value):
            return ""

        return re.sub(
            r"^[A-Z0-9]",
            "",
            str(value).upper(),
        )
```

```
[297]: model_for_mapping = model_metadata.reset_index()

model_name_mapping = pd.concat(
    [
        model_for_mapping[["ModelID", "CellLineName"]].rename(
            columns={"CellLineName": "model_name"}
        ),
        model_for_mapping[["ModelID", "StrippedCellLineName"]].rename(
            columns={"StrippedCellLineName": "model_name"}
        ),
        model_for_mapping[["ModelID", "CCLEName"]].
        .assign(
            CCLEName=lambda x: x["CCLEName"]
            .fillna("")
            .str.split("_")
            .str[0]
        )
        .rename(columns={"CCLEName": "model_name"}),
    ],
    ignore_index=True,
)

model_name_mapping["normalized_name"] = (
    model_name_mapping["model_name"].map(normalize_name)
)

model_name_mapping = (
    model_name_mapping
    .loc[model_name_mapping["normalized_name"] != ""]
)
```

```

        .drop_duplicates(
            subset=["normalized_name", "ModelID"]
        )
    )

model_name_mapping.head()

```

```

[297]:      ModelID  model_name normalized_name
0  ACH-000001  NIH:OVCAR-3      NIHOVCAR3
1  ACH-000002          HL-60          HL60
2  ACH-000003          CACO2          CACO2
3  ACH-000004          HEL          HEL
4  ACH-000005  HEL 92.1.7      HEL9217

```

```

[298]: ambiguous_names = (
    model_name_mapping
    .groupby("normalized_name")["ModelID"]
    .unique()
    .loc[lambda x: x.str.len() > 1]
)

print("Ambiguous cell-line names:", len(ambiguous_names))

if len(ambiguous_names) > 0:
    print(ambiguous_names)

```

Ambiguous cell-line names: 19

```

normalized_name
A375      [ACH-000219, ACH-002001, ACH-002002, ACH-002003]
COL0829      [ACH-000644, ACH-002380]
HCC1143      [ACH-000374, ACH-002378]
HCC1187      [ACH-000111, ACH-002373]
HCC1395      [ACH-000699, ACH-002382]
HCC1599      [ACH-000196, ACH-002374]
HCC1937      [ACH-000223, ACH-002375]
HCC1954      [ACH-000859, ACH-002384]
HCC2157      [ACH-000691, ACH-002381]
HCC2218      [ACH-000755, ACH-002383]
HCC38      [ACH-000276, ACH-002377]
HT144      [ACH-000322, ACH-002458, ACH-002459, ACH-002460]
J82      [ACH-000396, ACH-002379]
KMH2      [ACH-000815, ACH-002397]
LS1034      [ACH-000252, ACH-002376]
MS1      [ACH-001131, ACH-001958]
RVH421      [ACH-000614, ACH-002461]
TT      [ACH-000561, ACH-001321]
UACC62      [ACH-000425, ACH-002004]
Name: ModelID, dtype: object

```

```
[299]: name_to_model = (
    model_name_mapping
    .groupby("normalized_name")["ModelID"]
    .agg(lambda x: list(x))
)

def choose_model_id(ccl_name):
    normalized_name = normalize_name(ccl_name)
    matches = name_to_model.get(normalized_name, [])

    if len(matches) == 1:
        return matches[0]

    return pd.NA

cell_metadata["ModelID"] = (
    cell_metadata["ccl_name"]
    .map(choose_model_id)
)
```

```
[300]: print(
    "CTRPv2 profiles:",
    cell_metadata["index_ccl"].nunique()
)

print(
    "Mapped CTRPv2 profiles:",
    cell_metadata["ModelID"].notna().sum()
)

print(
    "Unmapped/ambiguous CTRPv2 profiles:",
    cell_metadata["ModelID"].isna().sum()
)
```

CTRPv2 profiles: 664

Mapped CTRPv2 profiles: 647

Unmapped/ambiguous CTRPv2 profiles: 17

```
[301]: ctrpv2_unresolved = (
    cell_metadata.loc[
        cell_metadata["ModelID"].isna(),
        ["index_ccl", "ccl_name"]
    ]
    .drop_duplicates()
    .copy()
```

```
)

print(
    "Unique unresolved CTRPv2 cell lines:",
    len(ctrpv2_unresolved)
)
```

Unique unresolved CTRPv2 cell lines: 17

```
[302]: compound_metadata["compound_label"] = (
        compound_metadata["cpd_name"].str.upper()
        + " (CTRP:"
        + compound_metadata["master_cpd_id"]
        .astype("Int64")
        .astype("string")
        + ")"
    )

if compound_metadata["compound_label"].duplicated().any():
    raise ValueError(
        "Compound labels are not unique."
    )
```

```
[303]: ambiguous_ctrpv2_metadata = (
        cell_metadata
        .dropna(subset=["ModelID"])
        .groupby("ModelID")["ccl_name"]
        .nunique()
    )

ambiguous_ctrpv2_metadata = (
        ambiguous_ctrpv2_metadata[
            ambiguous_ctrpv2_metadata > 1
        ]
    )

print(
    "ModelIDs with multiple distinct CTRPv2 names:",
    len(ambiguous_ctrpv2_metadata)
)
```

ModelIDs with multiple distinct CTRPv2 names: 0

```
[304]: ctrpv2_metadata = (
        cell_metadata[
            ["ModelID", "ccl_name"]
        ]
        .dropna(subset=["ModelID"])
```

```

        .drop_duplicates()
        .sort_values("ModelID")
        .drop_duplicates(subset=["ModelID"], keep="first")
        .set_index("ModelID")
    )

    ctrpv2_metadata.index.name = "ModelID"

```

```

[305]: auc = auc.merge(
        cell_metadata[["index_ccl", "ModelID"]],
        on="index_ccl",
        how="left",
        validate="many_to_one",
    )

    auc = auc.merge(
        compound_metadata[["index_cpd", "compound_label"]],
        on="index_cpd",
        how="left",
        validate="many_to_one",
    )

```

```

[306]: auc = auc.dropna(
        subset=["ModelID"]
    )

```

```

[307]: ctrpv2 = (
        auc
        .pivot_table(
            index="ModelID",
            columns="compound_label",
            values="area_under_curve",
            aggfunc="mean",
        )
        .sort_index()
        .sort_index(axis=1)
    )

    ctrpv2.columns.name = None
    ctrpv2.index.name = "ModelID"

```

```

[308]: ctrpv2.head()

```

```

[308]:      16-BETA-BROMOANDROSTERONE (CTRP:411726)  \
ModelID
ACH-000004      14.079
ACH-000007      15.355

```


ACH-000008	14.803
ACH-000009	13.803
ACH-000011	14.839

1S,3R-RSL-3 (CTRP:609060) 3-CL-AHPC (CTRP:660796) \		
ModelID		
ACH-000004	4.5275	8.1324
ACH-000007	8.9164	9.6490
ACH-000008	9.2353	12.8000
ACH-000009	8.3567	13.2710
ACH-000011	7.5951	11.0770

968 (CTRP:660080) A-804598 (CTRP:661033) AA-COCF3 (CTRP:411724) \			
ModelID			
ACH-000004	14.871	NaN	13.1790
ACH-000007	13.430	NaN	12.7450
ACH-000008	NaN	15.367	14.9600
ACH-000009	NaN	14.624	9.4753
ACH-000011	NaN	NaN	12.8750

ABIRATERONE (CTRP:660363) ABT-199 (CTRP:666541) \		
ModelID		
ACH-000004	NaN	NaN
ACH-000007	NaN	NaN
ACH-000008	NaN	15.000
ACH-000009	NaN	14.815
ACH-000011	NaN	NaN

ABT-737 (CTRP:411738) AC55649 (CTRP:375487) ... \			
ModelID			
ACH-000004	13.769	15.558	...
ACH-000007	13.868	15.000	...
ACH-000008	NaN	15.000	...
ACH-000009	11.832	14.547	...
ACH-000011	13.252	14.604	...

VX-680 (CTRP:340501) WAY-362450 (CTRP:660821) \		
ModelID		
ACH-000004	NaN	15.162
ACH-000007	NaN	15.113
ACH-000008	NaN	15.177
ACH-000009	NaN	14.652
ACH-000011	NaN	14.008

WP1130 (CTRP:660432) WZ4002 (CTRP:640366) WZ8040 (CTRP:628614) \			
ModelID			
ACH-000004	12.293	NaN	11.291

ACH-000007	11.132	NaN	11.280
ACH-000008	12.362	12.416	11.311
ACH-000009	11.741	11.891	10.698
ACH-000011	10.976	NaN	10.830

XL765 (CTRP:660304) YK 4-279 (CTRP:632103) YM-155 (CTRP:417979) \

ModelID

ACH-000004	14.328	11.732	13.087
ACH-000007	15.000	11.930	11.160
ACH-000008	14.495	14.530	10.237
ACH-000009	14.826	13.832	10.354
ACH-000011	14.628	12.272	11.988

ZEBULARINE (CTRP:411739) ZSTK474 (CTRP:606144)

ModelID

ACH-000004	13.461	14.2330
ACH-000007	14.346	13.4210
ACH-000008	14.942	14.5560
ACH-000009	14.335	10.0910
ACH-000011	14.303	8.4013

[5 rows x 481 columns]

```
[309]: ctrpv2_metadata.head()
```

```
[309]:      ccl_name
ModelID
ACH-000004    HEL
ACH-000007   LS513
ACH-000008   A101D
ACH-000009  C2BBE1
ACH-000011   253J
```

```
[310]: print("\nCTRPv2 formatted:")
print("Cell lines:", ctrpv2.shape[0])
print("Compounds:", ctrpv2.shape[1])
print("Shape:", ctrpv2.shape)
```

```
CTRPv2 formatted:
Cell lines: 629
Compounds: 481
Shape: (629, 481)
```

```
[311]: ctrpv2.to_csv(
    "../data/raw/DepMap/sensitivity/"
    "Drug_sensitivity_AUC_(CTRPv2.2)_wide.csv",
    index=True,
```

```
)
```

- Drug_sensitivity_AUC_(CTRPv2.2)_wide.csv saved to /data/raw/DepMap/sensitivity directory

5.3 3. Glioma Cell Line Masterdoc

```
[312]: cell_line_master = pd.DataFrame(  
        index=model.index  
    )  
  
cell_line_master.index.name = "ModelID"
```

```
[313]: model_metadata = model_metadata.rename(  
        columns={  
            "CellLineName": "CELL_LINE_NAME",  
            "StrippedCellLineName": "STRIPPED_CELL_LINE_NAME",  
            "COSMICID": "GDSC_COSMIC_ID",  
            "SangerModelID": "SANGER_MODEL_ID",  
            "CCLEName": "CCLE_NAME",  
            "ModelIDAlias": "DEPMAP_MODEL_ID_ALIAS"  
        }  
    )  
  
model_metadata = model_metadata.join(  
    ctrpv2_metadata[["ccl_name"]].rename(  
        columns={"ccl_name": "CTRPV2_NAME"}  
    ),  
    how="left",  
    validate="one_to_one",  
)
```

```
[314]: databases = {  
    "CTD2": ctd2,  
    "PRISM": prism,  
    "CTRPV2": ctrpv2,  
    "GDSC1": gdsc1,  
    "GDSC2": gdsc2,  
}  
  
cell_line_master = model_metadata.copy()  
  
for name, df in databases.items():  
    cell_line_master[f"{name}_AVAILABLE"] = (  
        cell_line_master.index.isin(df.index)  
    )  
  
cell_line_master["GDSC_EXPRESSION_AVAILABLE"] = (
```

```

        cell_line_master.index.isin(gdsc_exp.index)
    )

    cell_line_master["DEPMAP_EXPRESSION_AVAILABLE"] = (
        cell_line_master.index.isin(depmap_exp.index)
    )

    prism_numeric = prism.select_dtypes(include="number")

    prism_nonmissing = prism_numeric.notna().sum(axis=1)

    cell_line_master["PRISM_NONMISSING_MEASUREMENTS"] = (
        cell_line_master.index.map(prism_nonmissing).fillna(0).astype(int)
    )

    cell_line_master["PRISM_NONMISSING_MEASUREMENTS"] = (cell_line_master.index.
        ↪map(prism_nonmissing).fillna(0).astype(int))
    cell_line_master["PRISM_HAS_MEASUREMENTS"] = ↪
        ↪(cell_line_master["PRISM_NONMISSING_MEASUREMENTS"] > 0)

```

```

[315]: gdsc_sensitivity_available = (cell_line_master["GDSC1_AVAILABLE"] | ↪
        ↪cell_line_master["GDSC2_AVAILABLE"])

    depmap_sensitivity_available = (cell_line_master["CTRPV2_AVAILABLE"]
        | cell_line_master["PRISM_AVAILABLE"]
        | cell_line_master["CTD2_AVAILABLE"])

    gdsc_complete = gdsc_sensitivity_available & ↪
        ↪cell_line_master["GDSC_EXPRESSION_AVAILABLE"]
    depmap_complete = depmap_sensitivity_available & ↪
        ↪cell_line_master["DEPMAP_EXPRESSION_AVAILABLE"]

    either_complete = gdsc_complete | depmap_complete
    both_complete = gdsc_complete & depmap_complete

    print("GDSC has expression and sensitivity:", gdsc_complete.sum())
    print("DepMap has expression and sensitivity:", depmap_complete.sum())
    print("Has expression and sensitivity in either:", either_complete.sum())
    print("Has expression and sensitivity in both:", both_complete.sum())

```

```

GDSC has expression and sensitivity: 925
DepMap has expression and sensitivity: 911
Has expression and sensitivity in either: 1249
Has expression and sensitivity in both: 587

```

```

[316]: availability_cols = [
        "GDSC1_AVAILABLE",

```

```

        "GDSC2_AVAILABLE",
        "CTD2_AVAILABLE",
        "CTRPV2_AVAILABLE",
        "PRISM_AVAILABLE",
        "PRISM_HAS_MEASUREMENTS",
        "GDSC_EXPRESSION_AVAILABLE",
        "DEPMAP_EXPRESSION_AVAILABLE",
    ]

    metadata_cols = [
        "CELL_LINE_NAME",
        "STRIPPED_CELL_LINE_NAME",
        "GDSC_COSMIC_ID",
        "SANGER_MODEL_ID",
        "CCLE_NAME",
        "DEPMAP_MODEL_ID_ALIAS",
        "OncotreeLineage",
        "OncotreePrimaryDisease",
        "CTRPV2_NAME",
    ]

    cell_line_master = (
        cell_line_master
        .groupby(level="ModelID")
        .agg({
            **{col: "any" for col in availability_cols},
            "PRISM_NONMISSING_MEASUREMENTS": "max",
            **{col: "first" for col in metadata_cols},
        })
    )

```

```

[317]: cell_line_master["GDSC_SENSITIVITY_AVAILABLE"] = (
        gdsc_sensitivity_available
        .groupby(level="ModelID")
        .any()
    )
    cell_line_master["DEPMAP_SENSITIVITY_AVAILABLE"] = (
        depmap_sensitivity_available
        .groupby(level="ModelID")
        .any()
    )
    cell_line_master["GDSC_COMPLETE"] = (
        gdsc_complete
        .groupby(level="ModelID")
        .any()
    )
    cell_line_master["DEPMAP_COMPLETE"] = (

```

```

        depmap_complete
        .groupby(level="ModelID")
        .any()
    )
    cell_line_master["EITHER_COMPLETE"] = (
        either_complete
        .groupby(level="ModelID")
        .any()
    )
    cell_line_master["BOTH_COMPLETE"] = (
        both_complete
        .groupby(level="ModelID")
        .any()
    )

```

```

[318]: cell_line_master = (
        cell_line_master
        .reset_index()
        .rename(columns={"ModelID": "DEPMAP_ID"})
    )

    cell_line_master = cell_line_master.set_index("CELL_LINE_NAME")

```

```

[319]: column_order = [
    "STRIPPED_CELL_LINE_NAME",
    "DEPMAP_ID",
    "GDSC_COSMIC_ID",
    "SANGER_MODEL_ID",
    "CCLE_NAME",
    "CTRPV2_NAME",
    "DEPMAP_MODEL_ID_ALIAS",
    "OncotreeLineage",
    "OncotreePrimaryDisease",
    "GDSC1_AVAILABLE",
    "GDSC2_AVAILABLE",
    "CTD2_AVAILABLE",
    "CTRPV2_AVAILABLE",
    "PRISM_AVAILABLE",
    "PRISM_HAS_MEASUREMENTS",
    "PRISM_NONMISSING_MEASUREMENTS",
    "GDSC_EXPRESSION_AVAILABLE",
    "DEPMAP_EXPRESSION_AVAILABLE"
]

    cell_line_master = cell_line_master[column_order]
    cell_line_master["NOTES"] = ""

```

[320]: cell_line_master

```
[320]:
```

CELL_LINE_NAME	STRIPPED_CELL_LINE_NAME	DEPMAP_ID	GDSC_COSMIC_ID	\
NIH:OVCAR-3	NIHOVCAR3	ACH-000001	905933.0	
HL-60	HL60	ACH-000002	905938.0	
CACO2	CACO2	ACH-000003	NaN	
HEL	HEL	ACH-000004	907053.0	
HEL 92.1.7	HEL9217	ACH-000005	NaN	
...	...	...	...	
CCLF_HNSC_0002_T	CCLFHNSC0002T	ACH-003476	NaN	
CCLF_THYR_0001_T	CCLFTHYR0001T	ACH-003480	NaN	
C337	C337	ACH-003573	NaN	
531MII	531MII	ACH-003587	NaN	
YCUB-2	YCUB2	ACH-003590	NaN	

CELL_LINE_NAME	SANGER_MODEL_ID	CCLE_NAME	\
NIH:OVCAR-3	SIDM00105	NIHOVCAR3_OVARY	
HL-60	SIDM00829	HL60_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE	
CACO2	SIDM00891	CACO2_LARGE_INTESTINE	
HEL	SIDM00594	HEL_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE	
HEL 92.1.7	SIDM00593	HEL9217_HAEMATOPOIETIC_AND_LYMPHOID_TISSUE	
...	...	...	
CCLF_HNSC_0002_T	NaN	NaN	
CCLF_THYR_0001_T	NaN	NaN	
C337	NaN	NaN	
531MII	NaN	NaN	
YCUB-2	NaN	NaN	

CELL_LINE_NAME	CTRPV2_NAME	DEPMAP_MODEL_ID_ALIAS	OncotreeLineage	\
NIH:OVCAR-3	NaN	NaN	Ovary/Fallopian Tube	
HL-60	NaN	NaN	Myeloid	
CACO2	NaN	NaN	Bowel	
HEL	HEL	NaN	Myeloid	
HEL 92.1.7	NaN	NaN	Myeloid	
...	...	...	...	
CCLF_HNSC_0002_T	NaN	NaN	Esophagus/Stomach	
CCLF_THYR_0001_T	NaN	NaN	Thyroid	
C337	NaN	NaN	Bone	
531MII	NaN	NaN	Bone	
YCUB-2	NaN	ACH-003533	Lymphoid	

CELL_LINE_NAME	OncotreePrimaryDisease	GDSC1_AVAILABLE	\
NIH:OVCAR-3	Ovarian Epithelial Tumor	True	

HL-60	Acute Myeloid Leukemia	True
CACO2	Colorectal Adenocarcinoma	False
HEL	Acute Myeloid Leukemia	True
HEL 92.1.7	Acute Myeloid Leukemia	False
...	...	...
CCLF_HNSC_0002_T	Esophageal Squamous Cell Carcinoma	False
CCLF_THYR_0001_T	Anaplastic Thyroid Cancer	False
C337	Osteosarcoma	False
531MII	Osteosarcoma	False
YCUB-2	B-Cell Acute Lymphoblastic Leukemia	False

CELL_LINE_NAME	GDSC2_AVAILABLE	CTD2_AVAILABLE	CTRPV2_AVAILABLE	\
NIH:OVCAR-3	True	False	False	
HL-60	True	True	False	
CACO2	False	False	False	
HEL	True	True	True	
HEL 92.1.7	False	True	False	
...	...	...	...	
CCLF_HNSC_0002_T	False	False	False	
CCLF_THYR_0001_T	False	False	False	
C337	False	False	False	
531MII	False	False	False	
YCUB-2	False	False	False	

CELL_LINE_NAME	PRISM_AVAILABLE	PRISM_HAS_MEASUREMENTS	\
NIH:OVCAR-3	True	True	
HL-60	False	False	
CACO2	False	False	
HEL	True	True	
HEL 92.1.7	True	True	
...	...	...	
CCLF_HNSC_0002_T	False	False	
CCLF_THYR_0001_T	False	False	
C337	False	False	
531MII	False	False	
YCUB-2	False	False	

CELL_LINE_NAME	PRISM_NONMISSING_MEASUREMENTS	GDSC_EXPRESSION_AVAILABLE	\
NIH:OVCAR-3	192	True	
HL-60	0	True	
CACO2	0	False	
HEL	192	True	
HEL 92.1.7	191	False	
...	...	...	

CCLF_HNSC_0002_T	0	False
CCLF_THYR_0001_T	0	False
C337	0	False
531MII	0	False
YCUB-2	0	False

DEPMAP_EXPRESSION_AVAILABLE NOTES

CELL_LINE_NAME	
NIH:OVCAR-3	True
HL-60	True
CACO2	True
HEL	True
HEL 92.1.7	True
...	...
CCLF_HNSC_0002_T	True
CCLF_THYR_0001_T	False
C337	True
531MII	True
YCUB-2	True

[2154 rows x 19 columns]

```
[321]: cell_line_master.to_excel(
        ".../results/cell_line_annotation/cell_line_master.xlsx"
    )
```

```
[322]: cell_line_master.to_csv(
        ".../results/cell_line_annotation/cell_line_master.csv"
    )
```

```
[323]: glioma_cell_line_master = cell_line_master[
        cell_line_master["OncotreePrimaryDisease"]
        .str.contains("glioma", case=False, na=False)
    ].copy()
```

```
[324]: glioma_cell_line_master.to_excel(
        ".../results/cell_line_annotation/glioma_cell_line_master.xlsx"
    )
```

```
[325]: glioma_cell_line_master.to_csv(
        ".../results/cell_line_annotation/glioma_cell_line_master.csv"
    )
```

- cell_line_master.xlsx and glioma_cell_line_master.xlsx saved to /results/cell_lines

5.4 4. Merging GLASS signature and DepMap expression data

```
[326]: depmap_glioma_exp.columns = depmap_glioma_exp.columns.str.replace(r"\s*\(.??"
```

```
[327]: GLASS_genes = glass_idhwt_gbm_primary_R1_log2TPM_filtered.index
DepMap_genes = depmap_glioma_exp.columns
consensus_UP_genes = pd.Index(glass_recurrence_UP_consensus["GENE"])
consensus_DOWN_genes = pd.Index(glass_recurrence_DOWN_consensus["GENE"])
consensus_genes = consensus_UP_genes.union(consensus_DOWN_genes)
```

```
[328]: DepMap_genes
```

```
[328]: Index(['TSPAN6', 'TNMD', 'DPM1', 'SCYL3', 'FIRRM', 'FGR', 'CFH', 'FUCA2',
            'GCLC', 'NFYA',
            ...
            'ENSG00000288716', 'ENSG00000288717', 'ENSG00000288718',
            'ENSG00000288719', 'ENSG00000288720', 'ENSG00000288721', 'F8A1',
            'ENSG00000288723', 'ENSG00000288724', 'ENSG00000288725'],
            dtype='str', length=60649)
```

```
[329]: depmap_glioma_exp.iloc[:5, :5:]
```

```
[329]:
```

	TSPAN6	TNMD	DPM1	SCYL3	FIRRM
ModelID					
ACH-000570	5.590108	0.0	6.951250	3.093989	4.372597
ACH-000075	3.531294	0.0	7.220059	2.084190	3.673636
ACH-000269	3.667699	0.0	6.589735	2.132554	4.264063
ACH-000376	3.875613	0.0	6.739726	2.391449	4.395762
ACH-000887	5.099859	0.0	7.046415	1.851015	4.692756

```
[330]: glass_depmap_common_gene_universe = GLASS_genes.intersection(DepMap_genes)
```

```
[331]: glass_depmap_common_gene_universe_documentation = pd.DataFrame(
    {
        "Metric": [
            "GLASS filtered genes",
            "DepMap genes",
            "Common genes",
            "Signature UP genes",
            "Signature DOWN genes available"
        ],
        "Value": [
            GLASS_genes.nunique(),
            DepMap_genes.nunique(),
            glass_depmap_common_gene_universe.nunique(),
            glass_depmap_common_gene_universe.intersection(consensus_UP_genes).
            ↪nunique(),
```

```

        glass_depmap_common_gene_universe.
        ↪intersection(consensus_DOWN_genes).nunique()
    ],
    }
)

```

```
[332]: glass_depmap_common_gene_universe_documentation
```

```
[332]:
```

	Metric	Value
0	GLASS filtered genes	17789
1	DepMap genes	60649
2	Common genes	15121
3	Signature UP genes	162
4	Signature DOWN genes available	93

```
[333]: glass_depmap_common_gene_universe_documentation.to_csv(
        ".../results/DepMap/glass_depmap_common_gene_universe_documentation.csv",
        sep="\t",
        index=False
    )

```

```
[334]: with open(
        ".../results/DepMap/glass_depmap_common_gene_universe.txt",
        "w",
    ) as f:
        f.write("\n".join(glass_depmap_common_gene_universe.astype(str)))

```

- glass_depmap_common_gene_universe_documentation.csv and glass_depmap_common_gene_universe.txt saved to /results/DepMap directory
- glass_depmap_common_gene_universe.txt and glass_depmap_common_gene_universe_documentation.csv exported to /results/DepMap directory.

5.5 5. Calculating DepMap Recurrence Score

```
[335]: depmap_glioma_common_exp = depmap_glioma_exp[
        depmap_glioma_exp.columns.intersection(
            glass_depmap_common_gene_universe
        )
    ].copy()

depmap_glioma_common_exp = depmap_glioma_common_exp.T

```

```
[336]: UP_common = pd.Index(depmap_glioma_common_exp.index.
        ↪intersection(consensus_UP_genes))

```

```
[337]: DOWN_common = pd.Index(depmap_glioma_common_exp.index.
        ↪intersection(consensus_DOWN_genes))

```

```
[338]: depmap_glioma_common_exp.iloc[:5, :5]
```

```
[338]: ModelID  ACH-000570  ACH-000075  ACH-000269  ACH-000376  ACH-000887
TSPAN6      5.590108    3.531294    3.667699    3.875613    5.099859
DPM1        6.951250    7.220059    6.589735    6.739726    7.046415
SCYL3       3.093989    2.084190    2.132554    2.391449    1.851015
FGR         0.165156    0.013002    0.039199    0.027807    0.000000
CFH         6.288552    3.490288    3.086011    5.643949    0.109025
```

```
[339]: depmap_glioma_recurrence_scores = calculate\_recurrence\_score(depmap_glioma_common_exp, UP_common, DOWN_common)
```

```
[340]: up_genes_present = pd.Index(UP_common).intersection(
    depmap_glioma_common_exp.index
)

n_up_genes = (
    depmap_glioma_common_exp
    .loc[up_genes_present, :]
    .notna()
    .sum(axis=0)
)

n_up_genes.index.name = "DEPMAP_ID"

depmap_glioma_recurrence_scores["N_UP_GENES"] = (
    n_up_genes.reindex(
        depmap_glioma_recurrence_scores.index
    )
)

down_genes_present = pd.Index(DOWN_common).intersection(
    depmap_glioma_common_exp.index
)

n_down_genes = (
    depmap_glioma_common_exp
    .loc[down_genes_present, :]
    .notna()
    .sum(axis=0)
)

n_down_genes.index.name = "DEPMAP_ID"

depmap_glioma_recurrence_scores["N_DOWN_GENES"] = (
    n_down_genes.reindex(
        depmap_glioma_recurrence_scores.index
    )
)
```

```
)
)
```

```
[341]: depmap_glioma_recurrence_scores["CELL_LINE_NAME"] = model["CellLineName"].
        ↪reindex(depmap_glioma_recurrence_scores.index)

depmap_glioma_recurrence_scores = depmap_glioma_recurrence_scores.reset_index()

depmap_glioma_recurrence_scores = depmap_glioma_recurrence_scores.
        ↪set_index("CELL_LINE_NAME")

depmap_glioma_recurrence_scores["DEPMAP_RECURRENCE_SCORE"] =
        ↪depmap_glioma_recurrence_scores["RECURRENCE_SCORE"]

col_order = [
    "ModelID",
    "N_UP_GENES",
    "N_DOWN_GENES",
    "UP_SCORE",
    "DOWN_SCORE",
    "DEPMAP_RECURRENCE_SCORE",
]

depmap_glioma_recurrence_scores = (depmap_glioma_recurrence_scores[col_order])
```

```
[342]: depmap_glioma_recurrence_scores
```

```
[342]:
```

	ModelID	N_UP_GENES	N_DOWN_GENES	UP_SCORE	DOWN_SCORE	\
CELL_LINE_NAME						
YKG1	ACH-000570	162	93	0.259558	0.464742	
U-87 MG	ACH-000075	162	93	0.269172	0.434810	
AM-38	ACH-000269	162	93	0.261562	0.455253	
SF-295	ACH-000376	162	93	0.233810	0.469597	
SF-172	ACH-000887	162	93	0.301533	0.430382	
...	...	...	...	...	...	
CCLF_NEURO_0006_T	ACH-002409	162	93	0.240121	0.496880	
Onda 7	ACH-001622	162	93	0.253094	0.439753	
KG-1-C	ACH-000126	162	93	0.265102	0.427927	
NMC-G1	ACH-000200	162	93	0.273814	0.471960	
U343	ACH-000036	162	93	0.236943	0.425090	
	DEPMAP_RECURRENCE_SCORE					
CELL_LINE_NAME						
YKG1		-0.205184				
U-87 MG		-0.165638				
AM-38		-0.193691				
SF-295		-0.235786				

SF-172	-0.128849
...	...
CCLF_NEURO_0006_T	-0.256760
Onda 7	-0.186659
KG-1-C	-0.162825
NMC-G1	-0.198146
U343	-0.188147

[74 rows x 6 columns]

```
[343]: depmap_glioma_recurrence_scores.to_excel(
        ".../results/DepMap/depmap_glioma_recurrence_scores.xlsx",
        index=True
    )
```

- depmap_glioma_recurrence_scores.xlsx exported to /results/DepMap/ directory.

5.6 6. Verifying variance of DepMap Recurrence Score

```
[344]: score = depmap_glioma_recurrence_scores["DEPMAP_RECURRENCE_SCORE"]

depmap_recurrence_score_stats = pd.DataFrame()

depmap_recurrence_score_stats["N"] = [score.notna().sum()]
depmap_recurrence_score_stats["MEAN_SCORE"] = [score.mean()]
depmap_recurrence_score_stats["SD_SCORE"] = [score.std()]
depmap_recurrence_score_stats["MEDIAN_SCORE"] = [score.median()]
depmap_recurrence_score_stats["IQR"] = [score.quantile(0.75) - score.quantile(0.
↪25)]
depmap_recurrence_score_stats["MIN"] = [score.min()]
depmap_recurrence_score_stats["MAX"] = [score.max()]
depmap_recurrence_score_stats = depmap_recurrence_score_stats.T
depmap_recurrence_score_stats.columns = [""]
```

```
[345]: depmap_recurrence_score_stats
```

```
[345]:
N          74.000000
MEAN_SCORE -0.201748
SD_SCORE    0.031831
MEDIAN_SCORE -0.201472
IQR         0.042611
MIN         -0.259532
MAX         -0.127466
```

```
[346]: fig, (ax_hist, ax_swarm) = plt.subplots(
        nrows=2,
        ncols=1,
```

```

    figsize=(8, 5.5),
    sharex=True,
    gridspec_kw={"height_ratios": [3.5, 1], "hspace": 0.08},
)

sns.histplot(
    score,
    bins="fd",
    kde=True,
    ax=ax_hist,
    edgecolor="white",
    alpha=0.6,
    line_kws={"linewidth": 2},
)

ax_hist.set_ylabel("Count", fontsize=10, fontweight="bold")
ax_hist.set_title(
    f"DepMap Recurrence Score Distribution (N = {len(score)})",
    fontweight="bold",
    fontsize=12,
    pad=12,
)

ax_hist.grid(True, linestyle=":", alpha=0.5, axis="y")

sns.swarmplot(
    x=score,
    ax=ax_swarm,
    size=4.5,
    alpha=0.85,
)

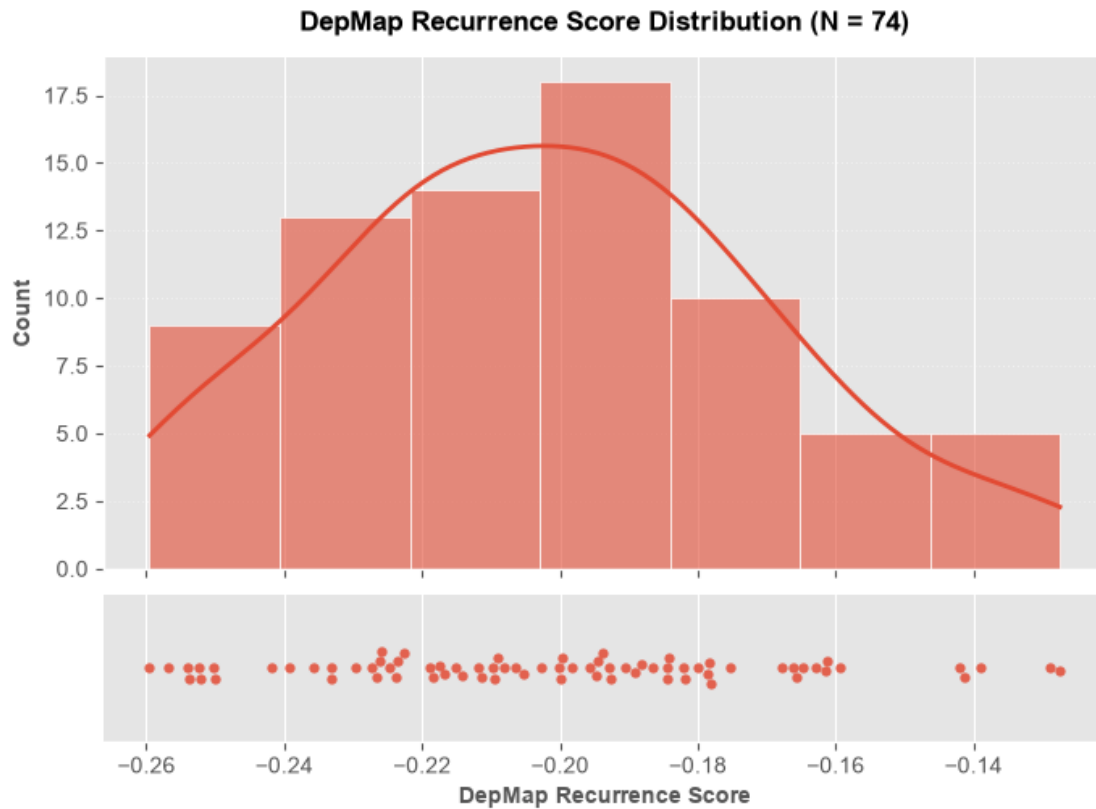
ax_swarm.set_xlabel("DepMap Recurrence Score", fontsize=10, fontweight="bold")

sns.despine(ax=ax_hist)
sns.despine(ax=ax_swarm, left=True)
ax_swarm.set_yticks([])

plt.savefig(
    "../figures/depmap_glioma_recurrence_score_distribution.png",
    dpi=300,
    bbox_inches="tight",
)

plt.show()

```



- depmap_glioma_recurrence_score_distribution.png saved to /figures directory

```
[347]: sorted_score = score.sort_values()
sorted_scores = sorted_score.to_numpy()
sorted_names = sorted_score.index
ranks = np.arange(1, len(sorted_scores) + 1)

fig, ax = plt.subplots(figsize=(8, 6))

ax.plot(ranks, sorted_scores, linewidth=1.5, zorder=2)

ax.scatter(
    ranks[:-5],
    sorted_scores[:-5],
    edgecolor="white",
    s=25,
    linewidth=0.5,
    zorder=3,
)

ax.scatter(
```



```

        ranks[-5:],
        sorted_scores[-5:],
        color="#d73027",
        edgecolor="white",
        s=45,
        linewidth=0.8,
        zorder=4,
        label="Top Hits",
    )

median_val = float(np.median(sorted_scores))
ax.axhline(
    median_val,
    color="#2b5c8f",
    linestyle="--",
    linewidth=1.2,
    alpha=0.8,
    label=f"Median ({median_val:.2f})",
)

sorted_scores_min = float(sorted_scores.min())
sorted_scores_max = float(sorted_scores.max())

score_range = sorted_scores_max - sorted_scores_min
ax.set_ylim(
    sorted_scores_min - (score_range * 0.05),
    sorted_scores_max + (score_range * 0.25),
)

n_labels = 5
top_indices = np.arange(len(sorted_scores) - n_labels, len(sorted_scores))

label_offsets = [
    (-50, 15),
    (-45, 25),
    (-40, 35),
    (-35, 45),
    (-30, 55),
]

for i, (dx, dy) in zip(top_indices, label_offsets):
    ax.annotate(
        sorted_names[i],
        xy=(ranks[i], sorted_scores[i]),
        xytext=(dx, dy),
        textcoords="offset points",
        fontsize=8.5,
    )

```

```

        fontweight="bold",
        color="#222222",
        ha="right",
        va="bottom",
        arrowprops=dict(
            arrowstyle="->",
            color="#666666",
            linewidth=0.8,
            connectionstyle="arc3,rad=-0.1",
        ),
    )

ax.set_title(
    f"Ranked DepMap Glioma Recurrence Scores (N = {len(sorted_scores)})",
    fontweight="bold",
    fontsize=12,
    pad=15,
)

ax.set_xlabel("Rank (Smallest to Largest)", fontsize=10, fontweight="bold")
ax.set_ylabel("DepMap Recurrence Score", fontsize=10, fontweight="bold")

ax.grid(True, linestyle=":", alpha=0.6, axis="y")
ax.set_xlim(0, len(sorted_scores) + 2)

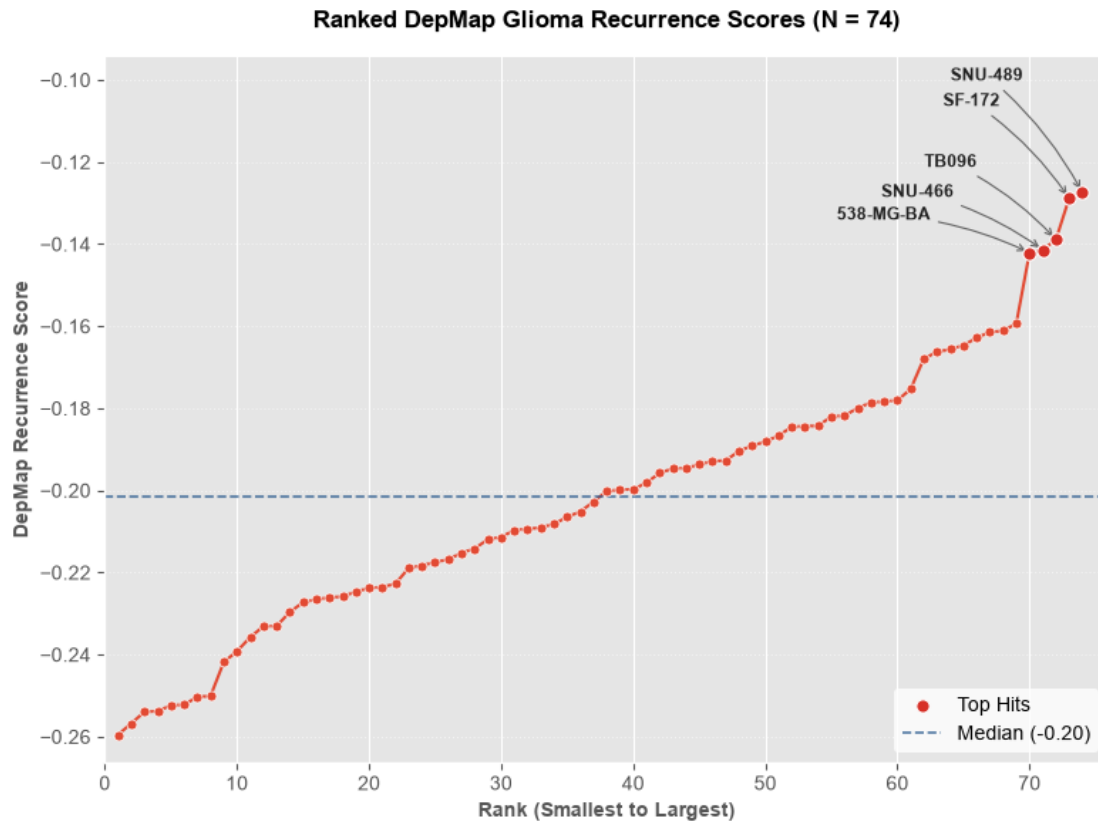
ax.legend(loc="lower right", frameon=True, facecolor="white", edgecolor="none")

sns.despine(ax=ax)
plt.tight_layout()

plt.savefig(
    "../figures/depmap_glioma_recurrence_score_ranked.png",
    dpi=300,
    bbox_inches="tight",
)

plt.show()

```



- `gdsc_glioma_recurrence_score_ranked.png` saved to `/figures` directory

5.7 7. Preparing to create GDSC microarray Recurrence Score

- `gdsc_depmap_cell_line_mapping.xlsx`
 - CELL_LINE_NAME
 - GDSC_ID
 - DEPMAP_MODEL_ID
 - GDSC_EXPRESSION_AVAILABLE
 - DEPMAP_RNASEQ_AVAILABLE
- checking signature genes in GDSC expression

```
[348]: gdsc_depmap_glioma_exp = gdsc_glioma_exp.loc[gdsc_glioma_exp.index.  
        ↪isin(depmap_glioma_exp.index)].copy()
```

```
[349]: glioma_cell_line_master.head()
```

```
[349]:
```

	STRIPPED_CELL_LINE_NAME	DEPMAP_ID	GDSC_COSMIC_ID	\
	CELL_LINE_NAME			
	GOS-3	GOS3 ACH-000027		NaN
	U343	U343 ACH-000036		NaN

U-118 MG	U118MG	ACH-000040	687588.0
Hs 683	HS683	ACH-000067	1240150.0
U-87 MG	U87MG	ACH-000075	687590.0

CELL_LINE_NAME	SANGER_MODEL_ID	CCLE_NAME	CTRPV2_NAME \
GOS-3	NaN	GOS3_CENTRAL_NERVOUS_SYSTEM	GOS3
U343	NaN	U343_CENTRAL_NERVOUS_SYSTEM	NaN
U-118 MG	SIDM01193	U118MG_CENTRAL_NERVOUS_SYSTEM	U118MG
Hs 683	SIDM00666	HS683_CENTRAL_NERVOUS_SYSTEM	HS683
U-87 MG	SIDM01189	U87MG_CENTRAL_NERVOUS_SYSTEM	U87MG

CELL_LINE_NAME	DEPMAP_MODEL_ID	ALIAS	OncotreeLineage \
GOS-3	NaN	CNS/Brain	
U343	NaN	CNS/Brain	
U-118 MG	NaN	CNS/Brain	
Hs 683	NaN	CNS/Brain	
U-87 MG	NaN	CNS/Brain	

CELL_LINE_NAME	OncotreePrimaryDisease	GDSC1_AVAILABLE	GDSC2_AVAILABLE \
GOS-3	Adult-Type Diffuse Glioma	False	False
U343	Adult-Type Diffuse Glioma	False	False
U-118 MG	Adult-Type Diffuse Glioma	True	True
Hs 683	Adult-Type Diffuse Glioma	True	True
U-87 MG	Adult-Type Diffuse Glioma	True	True

CELL_LINE_NAME	CTD2_AVAILABLE	CTRPV2_AVAILABLE	PRISM_AVAILABLE \
GOS-3	True	True	True
U343	False	False	True
U-118 MG	True	True	True
Hs 683	True	True	True
U-87 MG	True	True	True

CELL_LINE_NAME	PRISM_HAS_MEASUREMENTS	PRISM_NONMISSING_MEASUREMENTS \
GOS-3	True	1408
U343	True	190
U-118 MG	True	1398
Hs 683	True	190
U-87 MG	True	190

CELL_LINE_NAME	GDSC_EXPRESSION_AVAILABLE	DEPMAP_EXPRESSION_AVAILABLE	NOTES
GOS-3	False	True	

U343	False	True
U-118 MG	True	True
Hs 683	True	True
U-87 MG	True	True

```
[350]: gdsc_depmap_cell_line_mapping = glioma_cell_line_master[
        glioma_cell_line_master["DEPMAP_ID"].isin(gdsc_glioma_exp.index)].copy()
```

```
[351]: len(gdsc_depmap_cell_line_mapping)
```

```
[351]: 52
```

```
[352]: print("Signature genes available in GDSC:", int(gdsc_glioma_exp.columns.
        ↪isin(consensus_genes).sum()))
```

Signature genes available in GDSC: 250

5.8 8. Cross-platform validation of GDSC microarray and DepMap RNA-seq

```
[353]: GDSC_genes = gdsc_glioma_exp.columns
```

```
[354]: glass_gdsc_depmap_common_gene_universe = pd.Index(GLASS_genes.
        ↪intersection(DepMap_genes).intersection(GDSC_genes))
```

```
[355]: glass_gdsc_depmap_common_gene_universe_documentation = pd.DataFrame(
    {
        "Metric": [
            "GLASS filtered genes",
            "GDSC genes",
            "Common genes",
            "Signature UP genes",
            "Signature DOWN genes available"
        ],
        "Value": [
            GLASS_genes.nunique(),
            DepMap_genes.nunique(),
            glass_gdsc_depmap_common_gene_universe.nunique(),
            glass_gdsc_depmap_common_gene_universe.
            ↪intersection(consensus_UP_genes).nunique(),
            glass_gdsc_depmap_common_gene_universe.
            ↪intersection(consensus_DOWN_genes).nunique()
        ],
    }
)
```

```
[356]: glass_gdsc_depmap_common_gene_universe_documentation
```

```
[356]:
```

	Metric	Value
0	GLASS filtered genes	17789
1	GDSC genes	60649
2	Common genes	13004
3	Signature UP genes	152
4	Signature DOWN genes available	90

```
[357]: glass_gdsc_depmap_common_gene_universe_documentation.to_csv(
        ".../results/cross_platform/
        ↪glass_gdsc_depmap_common_gene_universe_documentation.csv",
        sep="\t",
        index=False
    )
```

- glass_gdsc_depmap_common_gene_universe_documentation.csv exported to /results/cross_platform directory

```
[358]: with open(
        ".../results/cross_platform/glass_depmap_common_gene_universe.txt",
        "w",
    ) as f:
        f.write("\n".join(glass_depmap_common_gene_universe.astype(str)))
```

- glass_gdsc_depmap_common_gene_universe.txt and glass_gdsc_depmap_common_gene_universe_documentation.csv saved to /results/cross_platform directory

```
[359]: gdsc_depmap_common_signature_genes = pd.
        ↪Index(glass_gdsc_depmap_common_gene_universe.intersection(consensus_genes))
```

```
[360]: depmap_glioma_exp_common = depmap_glioma_exp.loc[
        :,
        depmap_glioma_exp.columns.isin(
            gdsc_depmap_common_signature_genes
        )
    ].copy()
```

```
[361]: gdsc_glioma_exp_common = gdsc_glioma_exp.loc[
        :,
        gdsc_glioma_exp.columns.isin(
            glass_gdsc_depmap_common_gene_universe
        )
    ].copy()
```

```
[362]: UP_GDSC_DepMap = pd.Index(glass_gdsc_depmap_common_gene_universe.
        ↪intersection(consensus_UP_genes))
        DOWN_GDSC_DepMap = pd.Index(glass_gdsc_depmap_common_gene_universe.
        ↪intersection(consensus_DOWN_genes))
```

```
[363]: glass_crossplatform_recurrence_score_gdsc =
↳ calculate_recurrence_score(gdsc_glioma_exp_common.T, UP_GDSC_DepMap,
↳ DOWN_GDSC_DepMap)
glass_crossplatform_recurrence_score_gdsc =
↳ glass_crossplatform_recurrence_score_gdsc.drop(
    columns=["UP_SCORE", "DOWN_SCORE"])
glass_crossplatform_recurrence_score_gdsc =
↳ glass_crossplatform_recurrence_score_gdsc.rename(
    columns={"RECURRENCE_SCORE": "GDSC_RECURRENCE_SCORE_CROSSPLATFORM"})

glass_crossplatform_recurrence_score_depmap =
↳ calculate_recurrence_score(depmap_glioma_exp_common.T, UP_GDSC_DepMap,
↳ DOWN_GDSC_DepMap)
glass_crossplatform_recurrence_score_depmap =
↳ glass_crossplatform_recurrence_score_depmap.drop(
    columns=["UP_SCORE", "DOWN_SCORE"])
glass_crossplatform_recurrence_score_depmap =
↳ glass_crossplatform_recurrence_score_depmap.rename(
    columns={"RECURRENCE_SCORE": "DEPMAP_RECURRENCE_SCORE_CROSSPLATFORM"})
```

```
[364]: glass_crossplatform_recurrence_score_gdsc.head()
```

```
[364]:          GDSC_RECURRENCE_SCORE_CROSSPLATFORM
ModelID
ACH-000883          -0.171482
ACH-000437          -0.211355
ACH-001016          -0.075901
ACH-002257          -0.271622
ACH-000098          -0.091427
```

```
[365]: depmap_gdsc_glioma_recurrence_scores =
↳ glass_crossplatform_recurrence_score_gdsc.merge(
    glass_crossplatform_recurrence_score_depmap,
    on="ModelID",
    how="inner"
)
```

```
[366]: gdsc_exp_common = (
    gdsc_glioma_exp_common.loc[
        gdsc_glioma_exp_common.index.isin(
            depmap_gdsc_glioma_recurrence_scores.index
        )
    ].copy()
)
```

```

depmap_exp_common = (
    depmap_glioma_exp_common.loc[
        depmap_glioma_exp_common.index.isin(
            depmap_gdsc_glioma_recurrence_scores.index
        )
    ].copy()
)

```

```

[367]: up_genes_present = (
    pd.Index(UP_GDSC_DepMap)
    .intersection(depmap_exp_common.columns)
    .intersection(gdsc_exp_common.columns)
)

depmap_up_present = depmap_exp_common.loc[:, up_genes_present].notna()
gdsc_up_present = gdsc_exp_common.loc[:, up_genes_present].notna()

n_up_genes = (
    (depmap_up_present & gdsc_up_present)
    .sum(axis=1)
)

n_up_genes.index.name = "DEPMAP_ID"

depmap_gdsc_glioma_recurrence_scores["N_UP_GENES"] = (
    n_up_genes.reindex(
        depmap_gdsc_glioma_recurrence_scores.index
    )
)

down_genes_present = (
    pd.Index(DOWN_GDSC_DepMap)
    .intersection(depmap_exp_common.columns)
    .intersection(gdsc_exp_common.columns)
)

depmap_down_present = depmap_exp_common.loc[:, down_genes_present].notna()
gdsc_down_present = gdsc_exp_common.loc[:, down_genes_present].notna()

n_down_genes = (
    (depmap_down_present & gdsc_down_present)
    .sum(axis=1)
)

n_down_genes.index.name = "DEPMAP_ID"

depmap_gdsc_glioma_recurrence_scores["N_DOWN_GENES"] = (

```



```

    n_down_genes.reindex(
        depmap_gdsc_glioma_recurrence_scores.index
    )
)

```

```
[368]: depmap_gdsc_glioma_recurrence_scores.head()
```

```
[368]:
```

	GDSC_RECURRENCE_SCORE_CROSSPLATFORM \		
ModelID			
ACH-000883	-0.171482		
ACH-000437	-0.211355		
ACH-001016	-0.075901		
ACH-000098	-0.091427		
ACH-000102	-0.199141		

	DEPMAP_RECURRENCE_SCORE_CROSSPLATFORM	N_UP_GENES	N_DOWN_GENES
ModelID			
ACH-000883	-0.196491	152	90
ACH-000437	-0.157383	152	90
ACH-001016	-0.157675	152	90
ACH-000098	-0.160526	152	90
ACH-000102	-0.195906	152	90

```
[369]: plot_data = (
    depmap_gdsc_glioma_recurrence_scores[
        [
            "GDSC_RECURRENCE_SCORE_CROSSPLATFORM",
            "DEPMAP_RECURRENCE_SCORE_CROSSPLATFORM",
        ]
    ]
    .dropna()
)

x = plot_data["GDSC_RECURRENCE_SCORE_CROSSPLATFORM"]
y = plot_data["DEPMAP_RECURRENCE_SCORE_CROSSPLATFORM"]

rho, p_value = spearmanr(x, y)
n = len(plot_data)

fig, ax = plt.subplots(figsize=(6.5, 6))

ax.scatter(
    x,
    y,
    s=55,
    edgecolor="white",
    linewidth=0.8,
)

```

```

        alpha=0.85,
        zorder=3,
    )

sns.regplot(
    x=x,
    y=y,
    scatter=False,
    ax=ax,
    line_kws={
        "linewidth": 1.5,
        "alpha": 0.8,
    },
)

stats_text = (
    f"Spearman    = {float(rho):.3f}\n"
    f"p = {format_pvalue(p_value)}\n"
    f"N = {n}"
)

ax.text(
    0.05,
    0.95,
    stats_text,
    transform=ax.transAxes,
    ha="left",
    va="top",
    fontsize=10,
    bbox=dict(
        boxstyle="round,pad=0.4",
        facecolor="white",
        edgecolor="gray",
        alpha=0.9,
    ),
)

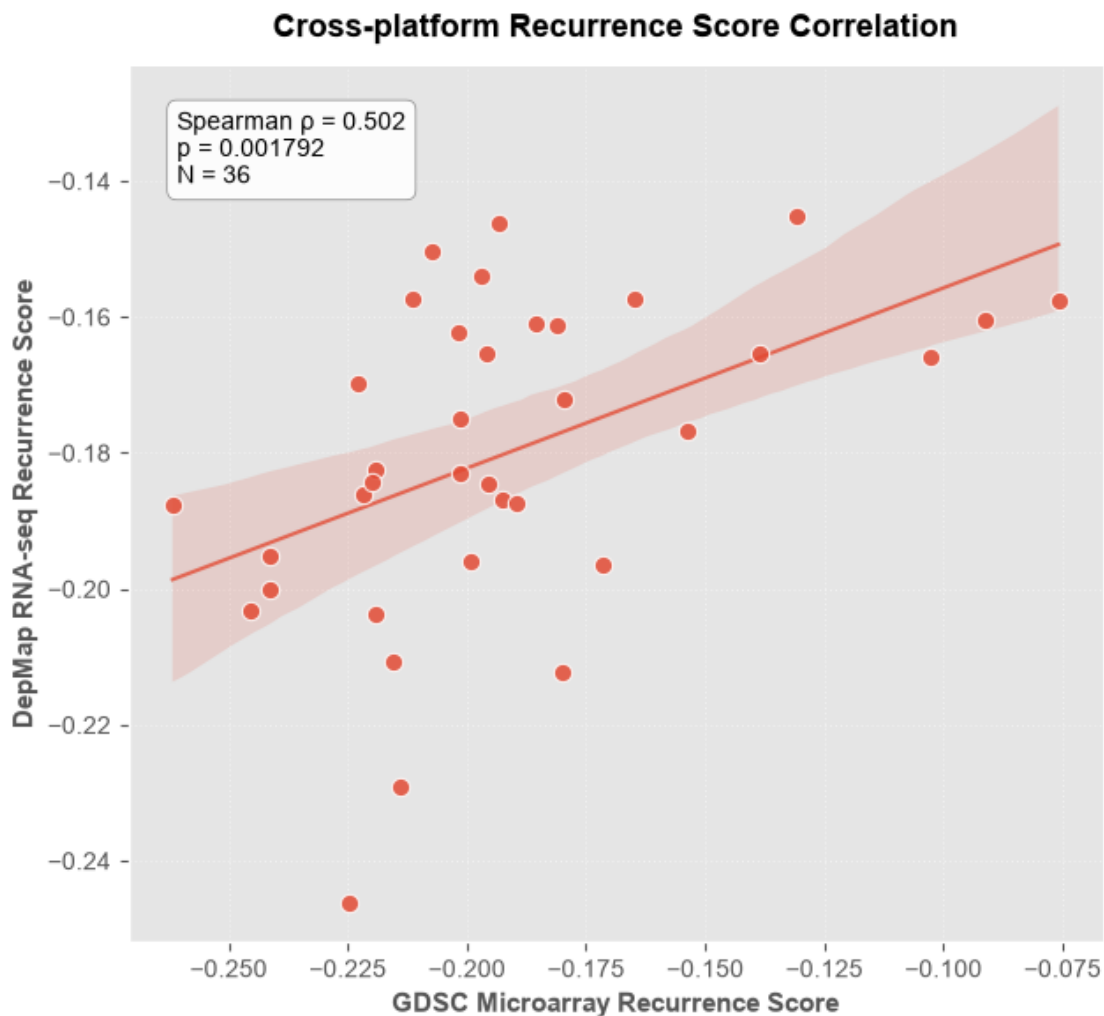
ax.set_xlabel(
    "GDSC Microarray Recurrence Score",
    fontsize=11,
    fontweight="bold",
)

ax.set_ylabel(
    "DepMap RNA-seq Recurrence Score",
    fontsize=11,
    fontweight="bold",
)

ax.set_title(
    "Cross-platform Recurrence Score Correlation",

```

```
        fontsize=13,  
        fontweight="bold",  
        pad=12,  
    )  
    ax.grid(  
        True,  
        linestyle=":",  
        alpha=0.5,  
        zorder=0,  
    )  
    sns.despine(ax=ax)  
    plt.tight_layout()  
    plt.savefig(  
        "../../../figures/gdsc_vs_depmap_recurrence_score.png",  
        dpi=300,  
        bbox_inches="tight",  
    )  
  
    plt.show()
```



- gdsc_vs_depmap_recurrence_score.png saved to /figures directory

```
[370]: glass_crossplatform_recurrence_score = ↳ calculate_recurrence_score(glass_idhwt_gbm_primary_R1_TPM_filtered,
↳ UP_GDSC_DepMap, DOWN_GDSC_DepMap)
```

```
[371]: glass_crossplatform_recurrence_score.iloc[:5, :5]
```

```
[371]:
```

	UP_SCORE	DOWN_SCORE	RECURRENCE_SCORE
GLSS-SM-R109-TP	0.407707	0.655074	-0.247367
GLSS-19-0274-TP	0.429723	0.507738	-0.078015
GLSS-CU-R005-TP	0.348835	0.598411	-0.249576
GLSS-SM-R060-TP	0.411103	0.644235	-0.233132
GLSS-LU-00C2-TP	0.327331	0.478289	-0.150958

```
[372]: new_scores = glass_crossplatform_recurrence_score.copy()
```

```
new_scores.index.name = "SAMPLE_ID"  
new_scores = new_scores.reset_index()
```

```
[373]: comparison = glass_recurrence_scores_full_cohort[  
    ["PATIENT_ID", "SAMPLE_ID", "SAMPLE_TYPE", "RECURRENCE_SCORE"]  
].merge(  
    new_scores[["SAMPLE_ID", "UP_SCORE", "DOWN_SCORE", "RECURRENCE_SCORE"]],  
    on="SAMPLE_ID",  
    how="inner",  
    suffixes=("_OLD", "_NEW"),  
)
```

```
[374]: comparison["SCORE_CHANGE"] = (comparison["RECURRENCE_SCORE_NEW"] -  
    ↪ comparison["RECURRENCE_SCORE_OLD"])  
  
comparison["SCORE_INCREASED"] = (comparison["SCORE_CHANGE"] > 0)  
  
comparison["SCORE_DECREASED"] = (comparison["SCORE_CHANGE"] < 0)  
  
comparison["SCORE_UNCHANGED"] = (comparison["SCORE_CHANGE"] == 0)
```

```
[375]: corr_data = comparison.dropna(  
    subset=["RECURRENCE_SCORE_OLD", "RECURRENCE_SCORE_NEW"]  
)  
  
rho, p_value = spearmanr(  
    corr_data["RECURRENCE_SCORE_OLD"],  
    corr_data["RECURRENCE_SCORE_NEW"],  
)
```

```
[376]: summary = pd.DataFrame(  
    {  
        "Metric": [  
            "Matched samples",  
            "Spearman correlation",  
            "Spearman p-value",  
            "Median OLD recurrence score",  
            "Median NEW recurrence score",  
            "Median score change (NEW - OLD)",  
            "Fraction increased",  
            "Fraction decreased",  
            "Fraction unchanged",  
        ],  
        "Value": [  
            len(corr_data),  
        ]  
    })
```

```

        float(rho),
        float(p_value),
        corr_data["RECURRENCE_SCORE_OLD"].median(),
        corr_data["RECURRENCE_SCORE_NEW"].median(),
        corr_data["SCORE_CHANGE"].median(),
        corr_data["SCORE_INCREASED"].mean(),
        corr_data["SCORE_DECREASED"].mean(),
        corr_data["SCORE_UNCHANGED"].mean(),
    ],
}
)

```

```

[377]: r1_primary = (
        comparison[
            comparison["SAMPLE_TYPE"].isin(["Tumor Primary", "First Recurrence"])
        ]
        .pivot_table(
            index="PATIENT_ID",
            columns="SAMPLE_TYPE",
            values=["RECURRENCE_SCORE_OLD", "RECURRENCE_SCORE_NEW"],
        )
    )

r1_primary["OLD_DIFF"] = (
    r1_primary[("RECURRENCE_SCORE_OLD", "First Recurrence")]
    - r1_primary[("RECURRENCE_SCORE_OLD", "Tumor Primary")]
)

r1_primary["NEW_DIFF"] = (
    r1_primary[("RECURRENCE_SCORE_NEW", "First Recurrence")]
    - r1_primary[("RECURRENCE_SCORE_NEW", "Tumor Primary")]
)

```

```

[378]: old_test = wilcoxon(r1_primary["OLD_DIFF"].dropna())
        new_test = wilcoxon(r1_primary["NEW_DIFF"].dropna())

```

```

[379]: r1_summary = pd.DataFrame(
    {
        "Metric": [
            "Paired patients",
            "OLD median R1 - Primary",
            "NEW median R1 - Primary",
            "OLD fraction R1 > Primary",
            "NEW fraction R1 > Primary",
            "OLD Wilcoxon p-value",
            "NEW Wilcoxon p-value",
        ],
    },
)

```

```

        "Value": [
            len(r1_primary),
            r1_primary["OLD_DIFF"].median(),
            r1_primary["NEW_DIFF"].median(),
            (r1_primary["OLD_DIFF"] > 0).mean(),
            (r1_primary["NEW_DIFF"] > 0).mean(),
            float(old_test.pvalue),
            float(new_test.pvalue),
        ],
    }
)

```

```

[380]: output_file = "../results/cross_platform/
        ↪glass_crossplatform_recurrence_score_check.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    summary.to_excel(
        writer,
        sheet_name="Summary",
        index=False,
    )
    r1_summary.to_excel(
        writer,
        sheet_name="R1_vs_Primary",
        index=False,
    )
    comparison.to_excel(
        writer,
        sheet_name="Score_Comparison",
        index=False,
    )
    r1_primary.to_excel(
        writer,
        sheet_name="R1_Primary_Paired",
    )

```

- glass_crossplatform_recurrence_score_check.xlsx saved to
/results/cross_platform directory

5.9 9. Final check of sensitivity data availability

```

[381]: glioma_drug_data_availability = glioma_cell_line_master[
        ["GDSC1_AVAILABLE", "GDSC2_AVAILABLE", "CTD2_AVAILABLE",
        ↪"CTRPV2_AVAILABLE", "PRISM_AVAILABLE"]].copy()
glioma_drug_data_availability = glioma_drug_data_availability.merge(
    depmap_glioma_recurrence_scores["DEPMAP_RECURRENCE_SCORE"],
    on="CELL_LINE_NAME",

```

```

        how="outer")
col_order = ["DEPMAP_RECURRENCE_SCORE", "GDSC1_AVAILABLE", "GDSC2_AVAILABLE",
             ↪ "CTD2_AVAILABLE", "CTRPV2_AVAILABLE",
             "PRISM_AVAILABLE"]
glioma_drug_data_availability = glioma_drug_data_availability[col_order]

```

```
[382]: glioma_drug_data_availability.head()
```

```
[382]:
```

	DEPMAP_RECURRENCE_SCORE	GDSC1_AVAILABLE	GDSC2_AVAILABLE	\
CELL_LINE_NAME				
1321N1	NaN	False	False	
170-MG-BA	-0.161424	False	False	
42-MG-BA	-0.178365	True	True	
538-MG-BA	-0.142115	False	False	
8-MG-BA	-0.166265	True	True	

	CTD2_AVAILABLE	CTRPV2_AVAILABLE	PRISM_AVAILABLE
CELL_LINE_NAME			
1321N1	False	False	False
170-MG-BA	False	False	False
42-MG-BA	True	True	True
538-MG-BA	False	False	False
8-MG-BA	True	True	True

```
[383]: glioma_drug_data_availability.to_excel(
        ".../results/data_availability/glioma_drug_data_availability.xlsx",
        index=True
    )
```

- glioma_drug_data_availability.xlsx saved to /results/data_availability directory

```
[384]: gdsc_sensitivity_available = (
        glioma_cell_line_master["GDSC1_AVAILABLE"]
        | glioma_cell_line_master["GDSC2_AVAILABLE"]
    )

depmap_sensitivity_available = (
        glioma_cell_line_master["CTD2_AVAILABLE"]
        | glioma_cell_line_master["CTRPV2_AVAILABLE"]
        | glioma_cell_line_master["PRISM_AVAILABLE"]
    )

gdsc_complete = (
        glioma_cell_line_master["GDSC_EXPRESSION_AVAILABLE"]
        & gdsc_sensitivity_available
    )

depmap_complete = (

```



```

glioma_cell_line_master["DEPMAP_EXPRESSION_AVAILABLE"]
& depmap_sensitivity_available
)

gdsc_depmap_complete = gdsc_complete & depmap_complete

summary = pd.DataFrame({
    "Expression_available": [
        glioma_cell_line_master["GDSC_EXPRESSION_AVAILABLE"].sum(),
        glioma_cell_line_master["DEPMAP_EXPRESSION_AVAILABLE"].sum(),
    ],
    "Sensitivity_available": [
        gdsc_sensitivity_available.sum(),
        depmap_sensitivity_available.sum(),
    ],
    "Expression_and_sensitivity": [
        gdsc_complete.sum(),
        depmap_complete.sum(),
    ],
}, index=["GDSC", "DepMap"])

summary["Sensitivity_fraction_among_expression"] = (
    summary["Expression_and_sensitivity"]
    / summary["Expression_available"]
)

summary["N_COMMON_COMPLETE"] = int(gdsc_depmap_complete.sum())

summary

```

```

[384]:      Expression_available  Sensitivity_available  \
GDSC                52                51
DepMap              74                51

      Expression_and_sensitivity  Sensitivity_fraction_among_expression  \
GDSC                50                0.961538
DepMap              51                0.689189

      N_COMMON_COMPLETE
GDSC                32
DepMap              32

```

```

[385]: def drug_response_qc(df, dataset_name):
    return {
        "DATASET": dataset_name,
        "N_CELL_LINES": df.shape[0],
        "N_DRUGS": df.shape[1],
    }

```

```

"MIN_AUC": df.min().min(),
"MEDIAN_AUC": df.stack().median(),
"MAX_AUC": df.max().max(),
"N_MISSING": df.isna().sum().sum(),
"PERCENT_MISSING": df.isna().mean().mean() * 100,
}

```

```

[386]: gdsc1_glioma = gdsc1.loc[
        gdsc1.index.isin(glioma_model_ids)
    ].copy()

gdsc2_glioma = gdsc2.loc[
        gdsc2.index.isin(glioma_model_ids)
    ].copy()

ctd2_glioma = ctd2.loc[
        ctd2.index.isin(glioma_model_ids)
    ].copy()

ctrpv2_glioma = ctrpv2.loc[
        ctrpv2.index.isin(glioma_model_ids)
    ].copy()

prism_glioma = prism.loc[
        prism.index.isin(glioma_model_ids)
    ].copy()

```

```

[387]: qc_results = [
        drug_response_qc(gdsc1_glioma, "GDSC1"),
        drug_response_qc(gdsc2_glioma, "GDSC2"),
        drug_response_qc(ctd2_glioma, "CTD2"),
        drug_response_qc(ctrpv2_glioma, "CTRPv2"),
        drug_response_qc(prism_glioma, "PRISM"),
    ]

sensitivity_qc = pd.DataFrame(qc_results)

sensitivity_qc

```

```

[387]:
   DATASET  N_CELL_LINES  N_DRUGS  MIN_AUC  MEDIAN_AUC  MAX_AUC  N_MISSING  \
0   GDSC1             50      316  0.032263    0.895180    1.000      1597
1   GDSC2             51      286  0.017723    0.913346    1.000      2873
2    CTD2             42      545  0.035965    0.915818    1.000      4334
3  CTRPv2             44      481  0.535740   13.898000   19.387      3896
4   PRISM             46     1482  0.021643    0.926168    1.000     21994

   PERCENT_MISSING

```

```

0      10.107595
1      19.696970
2      18.934032
3      18.408618
4      32.262512

```

```

[388]: output_file = "../results/qc/cell_lines_sensitivity_QC.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    summary.to_excel(
        writer,
        sheet_name="Summary",
        index=True,
    )
    sensitivity_qc.to_excel(
        writer,
        sheet_name="Drug Sensitivity Quality Check",
        index=False
    )

```

- cell_lines_sensitivity_QC.xlsx exported to /results/qc directory

6 Pharmacogenomic analysis of drug sensitivities associated with the glioblastoma recurrence program

6.1 1. Define the final Step 5 analysis cohort

- Cell lines included in PRISM discovery analysis:
 - has DEPMAP_RECURRENCE_SCORE
 - has PRISM drug-response data
 - OncotreePrimaryDisease == Adult-Type Diffuse Glioma
- prism_recurrence_analysis_cohort
 - ModelID
 - CELL_LINE_NAME
 - OncotreePrimaryDisease
 - DEPMAP_RECURRENCE_SCORE
 - PRISM_AVAILABLE

```

[389]: prism_recurrence_analysis_cohort = glioma_cell_line_master[
        ["DEPMAP_ID", "OncotreePrimaryDisease", "PRISM_AVAILABLE"]].copy()

prism_recurrence_analysis_cohort = prism_recurrence_analysis_cohort.merge(
    depmap_glioma_recurrence_scores["DEPMAP_RECURRENCE_SCORE"],
    on="CELL_LINE_NAME",
    how="outer")

```

```

[390]: prism_recurrence_analysis_cohort.head()

```

```
[390]:
```

CELL_LINE_NAME	DEPMAP_ID	OncotreePrimaryDisease	PRISM_AVAILABLE	\
1321N1	ACH-001000	Adult-Type Diffuse Glioma	False	
170-MG-BA	ACH-002680	Adult-Type Diffuse Glioma	False	
42-MG-BA	ACH-000323	Adult-Type Diffuse Glioma	True	
538-MG-BA	ACH-002681	Adult-Type Diffuse Glioma	False	
8-MG-BA	ACH-000137	Adult-Type Diffuse Glioma	True	

```
DEPMAP_RECURRENCE_SCORE
```

CELL_LINE_NAME	DEPMAP_RECURRENCE_SCORE
1321N1	NaN
170-MG-BA	-0.161424
42-MG-BA	-0.178365
538-MG-BA	-0.142115
8-MG-BA	-0.166265

```
[391]: prism_avail_mask = prism_recurrence_analysis_cohort["PRISM_AVAILABLE"].
        ↪ fillna(False).astype(bool)

prism_recurrence_analysis_cohort_summary = pd.DataFrame({
    "N_PRISM_ALL_GLIOMA": [
        prism_recurrence_analysis_cohort.loc[prism_avail_mask, "DEPMAP_ID"].
        ↪ nunique()
    ],
    "N_PRISM_ADULT_DIFFUSE_GLIOMA": [
        prism_recurrence_analysis_cohort.loc[
            prism_avail_mask & (
                prism_recurrence_analysis_cohort["OncotreePrimaryDisease"]_
        ↪ == "Adult-Type Diffuse Glioma"),
            "DEPMAP_ID"
        ].nunique()
    ],
})

prism_recurrence_analysis_cohort_summary
```

```
[391]:
```

	N_PRISM_ALL_GLIOMA	N_PRISM_ADULT_DIFFUSE_GLIOMA
0	46	44

```
[392]: prism_recurrence_analysis_cohort = prism_recurrence_analysis_cohort.loc[
        prism_recurrence_analysis_cohort["PRISM_AVAILABLE"]
        & (prism_recurrence_analysis_cohort["OncotreePrimaryDisease"] ==_
        ↪ "Adult-Type Diffuse Glioma")
        & prism_recurrence_analysis_cohort["DEPMAP_RECURRENCE_SCORE"].notna()
    ].copy()
```

```
[393]: with pd.ExcelWriter("../results/PRISM/prism_recurrence_analysis_cohort.
        ↪xlsx") as writer:
        prism_recurrence_analysis_cohort_summary.to_excel(
            writer,
            sheet_name="Summary",
            index=False
        )

        prism_recurrence_analysis_cohort.to_excel(
            writer,
            sheet_name="PRISM RA Cohort",
            index=True
        )
```

- prism_recurrence_analysis_cohort.xlsx exported to /results/PRISM directory

6.2 2. Prepare PRISM drug-response data

- prism_drug_coverage_QC
 - N_TOTAL
 - N_WITH_AUC
 - N_MISSING
 - PERCENT_MISSING
 - MIN_AUC
 - MEDIAN_AUC
 - MAX_AUC

```
[394]: def drug_response_qc_per_compound(df, dataset_name):
        qc = pd.DataFrame({
            "DATASET": dataset_name,
            "N_CELL_LINES_TOTAL": df.shape[0],
            "N_CELL_LINES_AVAILABLE": df.notna().sum(axis=0),
            "N_MISSING": df.isna().sum(axis=0),
            "PERCENT_MISSING": df.isna().mean(axis=0) * 100,
            "MIN_AUC": df.min(axis=0),
            "MEDIAN_AUC": df.median(axis=0),
            "MAX_AUC": df.max(axis=0),
        })

        qc.index.name = "COMPOUND"

        return qc
```

```
[395]: prism_cohort = prism.loc[
        prism.index.isin(prism_recurrence_analysis_cohort["DEPMAP_ID"])
    ].copy()
```

```
[396]: prism_drug_coverage_QC = drug_response_qc_per_compound(prism_cohort, "PRISM")
```

```
[397]: prism_drug_coverage_QC.head()
```

```
[397]:
```

	DATASET	N_CELL_LINES_TOTAL	\
COMPOUND			
(+)-CAMPTOTHECIN (BRD:BRD-K37890730-001-15-1)	PRISM	44	
1-AZAKENPAULLONE (BRD:BRD-K03273112-001-01-8)	PRISM	44	
1-NAPHTHYL-PP1 (BRD:BRD-K29542628-001-03-9)	PRISM	44	
1-PHENYLBIGUANIDE (BRD:BRD-K31491153-003-05-9)	PRISM	44	
10-DEACETYLACCATIN (BRD:BRD-K96631475-001-02-4)	PRISM	44	

	N_CELL_LINES_AVAILABLE	\
COMPOUND		
(+)-CAMPTOTHECIN (BRD:BRD-K37890730-001-15-1)	30	
1-AZAKENPAULLONE (BRD:BRD-K03273112-001-01-8)	30	
1-NAPHTHYL-PP1 (BRD:BRD-K29542628-001-03-9)	30	
1-PHENYLBIGUANIDE (BRD:BRD-K31491153-003-05-9)	26	
10-DEACETYLACCATIN (BRD:BRD-K96631475-001-02-4)	30	

	N_MISSING	PERCENT_MISSING	\
COMPOUND			
(+)-CAMPTOTHECIN (BRD:BRD-K37890730-001-15-1)	14	31.818182	
1-AZAKENPAULLONE (BRD:BRD-K03273112-001-01-8)	14	31.818182	
1-NAPHTHYL-PP1 (BRD:BRD-K29542628-001-03-9)	14	31.818182	
1-PHENYLBIGUANIDE (BRD:BRD-K31491153-003-05-9)	18	40.909091	
10-DEACETYLACCATIN (BRD:BRD-K96631475-001-02-4)	14	31.818182	

	MIN_AUC	MEDIAN_AUC	\
COMPOUND			
(+)-CAMPTOTHECIN (BRD:BRD-K37890730-001-15-1)	0.060965	0.327112	
1-AZAKENPAULLONE (BRD:BRD-K03273112-001-01-8)	0.856010	0.910309	
1-NAPHTHYL-PP1 (BRD:BRD-K29542628-001-03-9)	0.681012	0.868215	
1-PHENYLBIGUANIDE (BRD:BRD-K31491153-003-05-9)	0.764956	0.976171	
10-DEACETYLACCATIN (BRD:BRD-K96631475-001-02-4)	0.717816	0.863772	

	MAX_AUC
COMPOUND	
(+)-CAMPTOTHECIN (BRD:BRD-K37890730-001-15-1)	0.901369
1-AZAKENPAULLONE (BRD:BRD-K03273112-001-01-8)	1.000000
1-NAPHTHYL-PP1 (BRD:BRD-K29542628-001-03-9)	0.991522
1-PHENYLBIGUANIDE (BRD:BRD-K31491153-003-05-9)	0.991963
10-DEACETYLACCATIN (BRD:BRD-K96631475-001-02-4)	0.990626

```
[398]: prism_drug_coverage_QC.to_excel(
        ".../results/PRISM/prism_drug_coverage_QC.xlsx",
        index=True
    )
```

- prism_drug_coverage_QC.xlsx exported to /results/PRISM directory

6.3 3. Perform the primary PRISM pharmacogenomic screen

- Spearman correlation between DEPMAP_RECURRENCE_SCORE and PRISM_AUC
- prism_recurrence_drug_screen
 - DRUG
 - N
 - SPEARMAN_RHO
 - P_VALUE
 - FDR

```
[399]: recurrence_score = (  
    prism_recurrence_analysis_cohort  
    .set_index("DEPMAP_ID")["DEPMAP_RECURRENCE_SCORE"]  
    .reindex(prism_cohort.index)  
)  
  
eligible_drugs = prism_drug_coverage_QC.index[  
    prism_drug_coverage_QC["N_CELL_LINES_AVAILABLE"] >= 20  
]
```

```
[400]: results = []  
for drug in eligible_drugs:  
    valid = (recurrence_score.notna() & prism_cohort[drug].notna())  
    recurrence_values = recurrence_score.loc[valid]  
    auc_values = prism_cohort.loc[valid, drug]  
    n = valid.sum()  
    if n < 20 or recurrence_values.nunique() < 2 or auc_values.nunique() < 2:  
        rho, p_value = np.nan, np.nan  
    else:  
        rho, p_value = spearmanr(recurrence_values, auc_values)  
    results.append({  
        "DRUG": drug,  
        "N": n,  
        "SPEARMAN_RHO": rho,  
        "P_VALUE": p_value,  
    })  
  
prism_recurrence_drug_screen = pd.DataFrame(results)
```

```
[401]: tested = (prism_recurrence_drug_screen["P_VALUE"].notna())  
  
prism_recurrence_drug_screen.loc[tested, "FDR"] = multipletests(  
    prism_recurrence_drug_screen.loc[tested, "P_VALUE"],  
    method="fdr_bh",  
)[1]
```

```
[402]: prism_recurrence_drug_screen["CANDIDATE"] = (  
    prism_recurrence_drug_screen["N"] >= 20
```

```

    & (prism_recurrence_drug_screen["SPEARMAN_RHO"] <= -0.30)
    & (prism_recurrence_drug_screen["FDR"] < 0.10)
)

```

```

[403]: prism_recurrence_drug_screen = prism_recurrence_drug_screen.sort_values(
        "SPEARMAN_RHO"
    )

```

```

[404]: prism_recurrence_drug_screen["BRD_ID"] = (
        prism_recurrence_drug_screen["DRUG"]
        .str.extract(r"\((BRD:.*?)\)") , expand=False)
        .str.replace(r"^BRD:", "", regex=True)
    )

prism_recurrence_drug_screen["DRUG"] = (
        prism_recurrence_drug_screen["DRUG"]
        .str.split(" (", n=1, regex=False)
        .str[0]
    )

col_order = ["DRUG", "BRD_ID", "N", "SPEARMAN_RHO", "P_VALUE", "FDR",
    ↪ "CANDIDATE"]
prism_recurrence_drug_screen = prism_recurrence_drug_screen[col_order]

```

```

[405]: prism_recurrence_drug_screen.head()

```

```

[405]:
           DRUG          BRD_ID  N  SPEARMAN_RHO  P_VALUE \
609      GS-39783  BRD-K75478907-001-02-3  23      -0.611552  0.001931
87      ALVESPIMYCIN  BRD-K83988098-001-02-0  43      -0.605557  0.000017
4      10-DEACETYLBACCATIN  BRD-K96631475-001-02-4  30      -0.575973  0.000866
451      DOCETAXEL  BRD-K30577245-001-05-0  43      -0.575657  0.000054
962      PACLITAXEL  BRD-K62008436-001-23-9  43      -0.551495  0.000127

           FDR  CANDIDATE
609  0.350796      False
87   0.023769       True
4    0.243787      False
451  0.037816       True
962  0.059350       True

```

```

[406]: prism_recurrence_drug_screen.to_excel(
        "../results/PRISM/prism_recurrence_drug_screen.xlsx",
        index=False,
    )

```

- prism_recurrence_drug_screen.xlsx exported to /results/PRISM directory

6.4 4. Visualise PRISM screening results

```
[407]: plot_df = prism_recurrence_drug_screen.dropna(
        subset=["SPEARMAN_RHO", "FDR"]
    ).copy()

plot_df["NEG_LOG10_FDR"] = -np.log10(plot_df["FDR"].clip(lower=1e-300))
candidate_df = plot_df[plot_df["CANDIDATE"]].copy()
top_candidates = candidate_df.sort_values("SPEARMAN_RHO").head(3)

fig, ax = plt.subplots(figsize=(8, 6))

ax.scatter(
    plot_df["SPEARMAN_RHO"],
    plot_df["NEG_LOG10_FDR"],
    color="#bab0ab",
    alpha=0.5,
    s=25,
    edgecolor="none",
    label="Screened Compounds",
    zorder=2,
)

ax.scatter(
    candidate_df["SPEARMAN_RHO"],
    candidate_df["NEG_LOG10_FDR"],
    color="#d73027",
    edgecolor="black",
    linewidth=0.7,
    s=50,
    label="Candidates",
    zorder=3,
)

for _, row in top_candidates.iterrows():
    clean_name = str(row["DRUG"]).split(" (")[0]
    ax.annotate(
        clean_name,
        (row["SPEARMAN_RHO"], row["NEG_LOG10_FDR"]),
        xytext=(8, 3),
        textcoords="offset points",
        fontsize=8.5,
        fontweight="bold",
        color="#222222",
    )

fdr_thresh = -np.log10(0.10)
```

```

ax.axvline(-0.30, linestyle="--", color="#666666", linewidth=1, zorder=1)
ax.axhline(fdr_thresh, linestyle="--", color="#666666", linewidth=1, zorder=1)

ax.text(-0.29, 1.55, "Rho < -0.30", fontsize=8, color="#555555")
ax.text(0.38, fdr_thresh + 0.03, "FDR = 0.10", fontsize=8, color="#555555")

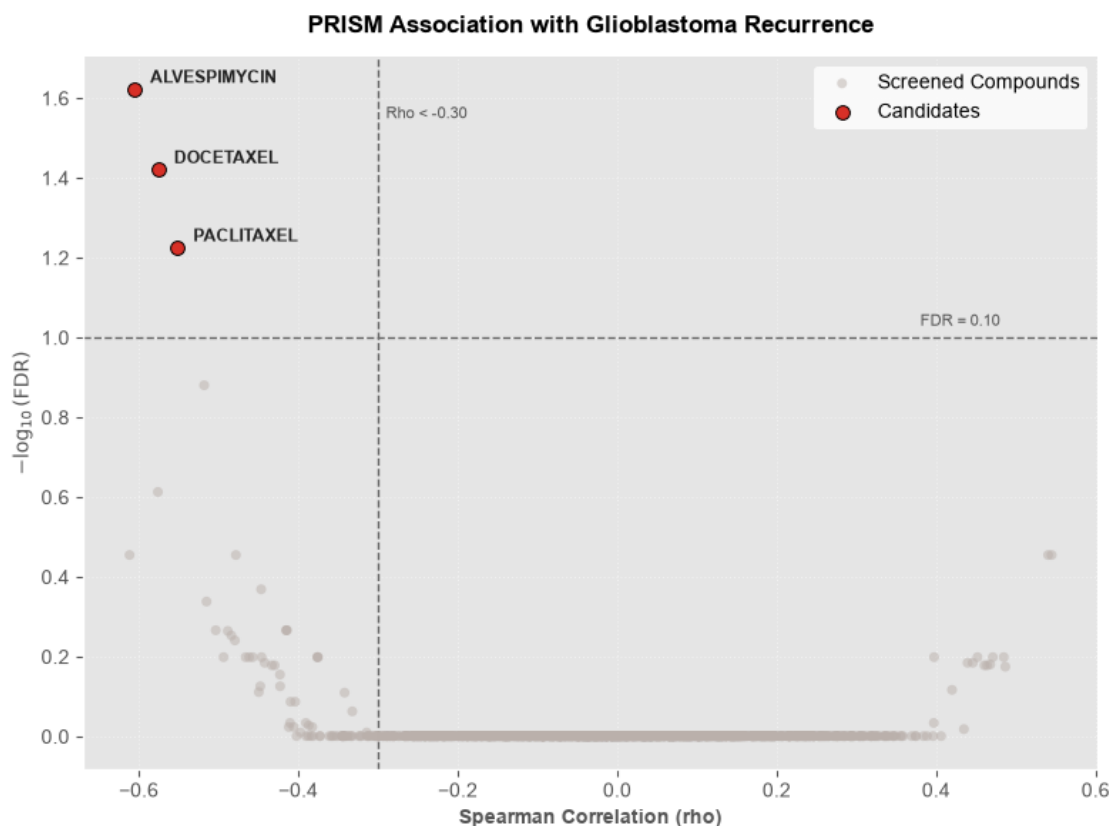
ax.set_xlabel("Spearman Correlation (rho)", fontweight="bold", fontsize=10)
ax.set_ylabel(r"$-\log_{10}(\text{FDR})$", fontweight="bold", fontsize=10)
ax.set_title(
    "PRISM Association with Glioblastoma Recurrence",
    fontweight="bold",
    fontsize=12,
    pad=12,
)

ax.grid(True, linestyle=":", alpha=0.5)
ax.legend(loc="upper right", frameon=True, facecolor="white", edgecolor="none")

sns.despine(ax=ax)
plt.tight_layout()

plt.savefig("../figures/prism_recurrence_drug_screen.png", dpi=300,
            bbox_inches="tight")
plt.show()

```



- prism_recurrence_drug_screen.png exported to /figures directory

```
[408]: prism_cohort.head()
```

```
[408]:      (+)-CAMPTOTHECIN (BRD:BRD-K37890730-001-15-1) \
ACH-000027      0.705806
ACH-000036      NaN
ACH-000040      0.243679
ACH-000067      NaN
ACH-000075      NaN

      1-AZAKENPAULLONE (BRD:BRD-K03273112-001-01-8) \
ACH-000027      0.981444
ACH-000036      NaN
ACH-000040      0.984955
ACH-000067      NaN
ACH-000075      NaN

      1-NAPHTHYL-PP1 (BRD:BRD-K29542628-001-03-9) \
ACH-000027      0.871238
ACH-000036      NaN
```

ACH-000040	0.869594
ACH-000067	NaN
ACH-000075	NaN
1-PHENYLBIGUANIDE (BRD:BRD-K31491153-003-05-9) \	
ACH-000027	0.972453
ACH-000036	NaN
ACH-000040	0.981178
ACH-000067	NaN
ACH-000075	NaN
10-DEACETYLBACCATIN (BRD:BRD-K96631475-001-02-4) \	
ACH-000027	0.885546
ACH-000036	NaN
ACH-000040	0.901915
ACH-000067	NaN
ACH-000075	NaN
10-HYDROXYCAMPTOTHECIN (BRD:BRD-K63784565-001-06-2) \	
ACH-000027	0.428726
ACH-000036	NaN
ACH-000040	0.167499
ACH-000067	NaN
ACH-000075	NaN
AZD4547 (BRD:BRD-K28392481-001-05-1) \	
ACH-000027	1.000000
ACH-000036	NaN
ACH-000040	0.934131
ACH-000067	NaN
ACH-000075	NaN
12-O-TETRADECANOYLPHORBOL-13-ACETATE (BRD:BRD-K68552125-001-05-3) \	
ACH-000027	1.0
ACH-000036	NaN
ACH-000040	1.0
ACH-000067	NaN
ACH-000075	NaN
TANESPIMYCIN (BRD:BRD-K81473043-001-19-5) \	
ACH-000027	0.886856
ACH-000036	0.676336
ACH-000040	0.821991
ACH-000067	0.757590
ACH-000075	0.761486
2'-MECCPA (BRD:BRD-K63932022-001-01-5) ... \	

ACH-000027	1.000000	...
ACH-000036	NaN	...
ACH-000040	0.993357	...
ACH-000067	NaN	...
ACH-000075	NaN	...

	ZK-811752 (BRD:BRD-K91825936-001-01-2)	\
ACH-000027	0.899724	
ACH-000036	NaN	
ACH-000040	1.000000	
ACH-000067	NaN	
ACH-000075	NaN	

	ZK-93423 (BRD:BRD-K33882852-003-02-8)	\
ACH-000027	0.881097	
ACH-000036	NaN	
ACH-000040	0.875656	
ACH-000067	NaN	
ACH-000075	NaN	

	ZK-93426 (BRD:BRD-K68392338-003-02-0)	\
ACH-000027	0.997758	
ACH-000036	NaN	
ACH-000040	0.884366	
ACH-000067	NaN	
ACH-000075	NaN	

	ZLN005 (BRD:BRD-K73073971-001-01-7)	\
ACH-000027	0.997308	
ACH-000036	NaN	
ACH-000040	1.000000	
ACH-000067	NaN	
ACH-000075	NaN	

	ZM 447439 (BRD:BRD-K72703948-001-10-1)	\
ACH-000027	0.954276	
ACH-000036	NaN	
ACH-000040	0.915258	
ACH-000067	NaN	
ACH-000075	NaN	

	ZM-306416 (BRD:BRD-K41337261-001-03-0)	\
ACH-000027	1.0	
ACH-000036	NaN	
ACH-000040	1.0	
ACH-000067	NaN	
ACH-000075	NaN	

	ZOLANTIDINE (BRD:BRD-K08996725-332-01-7)	\
ACH-000027		0.997546
ACH-000036		NaN
ACH-000040		0.957643
ACH-000067		NaN
ACH-000075		NaN
	ZOLMITRIPTAN (BRD:BRD-K54314721-001-10-2)	\
ACH-000027		0.990884
ACH-000036		NaN
ACH-000040		0.945964
ACH-000067		NaN
ACH-000075		NaN
	SIROLIMUS (BRD:BRD-K84937637-001-09-9)	\
ACH-000027		0.652816
ACH-000036		NaN
ACH-000040		0.422183
ACH-000067		NaN
ACH-000075		NaN
	TEMSIROLIMUS (BRD:BRD-K42898655-001-01-8)	
ACH-000027		0.549969
ACH-000036		0.684939
ACH-000040		0.504002
ACH-000067		0.653638
ACH-000075		0.593650

[5 rows x 1482 columns]

```
[409]: top_candidates = (
    prism_recurrence_drug_screen[prism_recurrence_drug_screen["CANDIDATE"]]
    .sort_values("SPEARMAN_RHO")
    .head(3)
)

for _, row in top_candidates.iterrows():
    drug_id = row["DRUG"]
    drug_id = f"{row['DRUG']} (BRD:{row['BRD_ID']})"
    p_val = row["P_VALUE"]
    fdr = row["FDR"]

    valid_mask = recurrence_score.notna() & prism_cohort[drug_id].notna()
    x_vals = recurrence_score.loc[valid_mask]
    y_vals = prism_cohort.loc[valid_mask, drug_id]
```

```

clean_display_name = str(drug_id).split(" ")[0]

fig, ax = plt.subplots(figsize=(5.5, 5))

sns.regplot(
    x=x_vals,
    y=y_vals,
    ax=ax,
    color="#2b5c8f",
    scatter_kws={
        "alpha": 0.8,
        "s": 45,
        "edgecolor": "white",
        "linewidths": 0.6,
    },
    line_kws={"color": "#d73027", "linewidth": 1.5, "linestyle": "--"},
)

ax.set_title(
    f"{clean_display_name} Sensitivity vs. Recurrence Score",
    fontweight="bold",
    fontsize=11,
    pad=12,
)

ax.set_xlabel("DepMap Recurrence Score", fontweight="bold", fontsize=10)
ax.set_ylabel(f"{clean_display_name} AUC", fontweight="bold", fontsize=10)

stat_text = (
    f"Spearman  $\rho$  = {float(rho):.2f} $\rho$  =  $\rho$ "
    f" $\rightarrow$  {format_pvalue(p_val)} $\rho$  = {format_pvalue(fdr)} $\rho$ "
)

ax.text(
    0.05,
    0.92,
    stat_text,
    transform=ax.transAxes,
    fontsize=8.5,
    verticalalignment="top",
    bbox=dict(
        boxstyle="round,pad=0.5",
        facecolor="white",
        edgecolor="#cccccc",
        alpha=0.9,
    ),
)

ax.grid(True, linestyle=":", alpha=0.5)

```

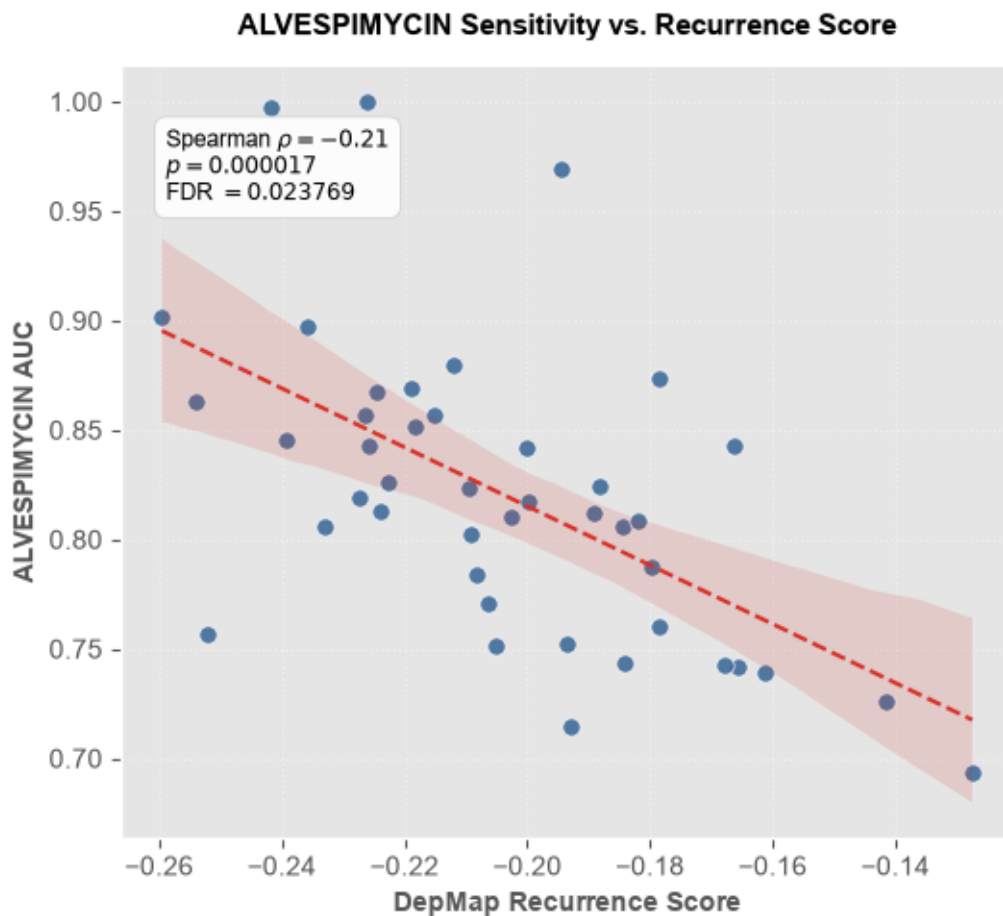
```

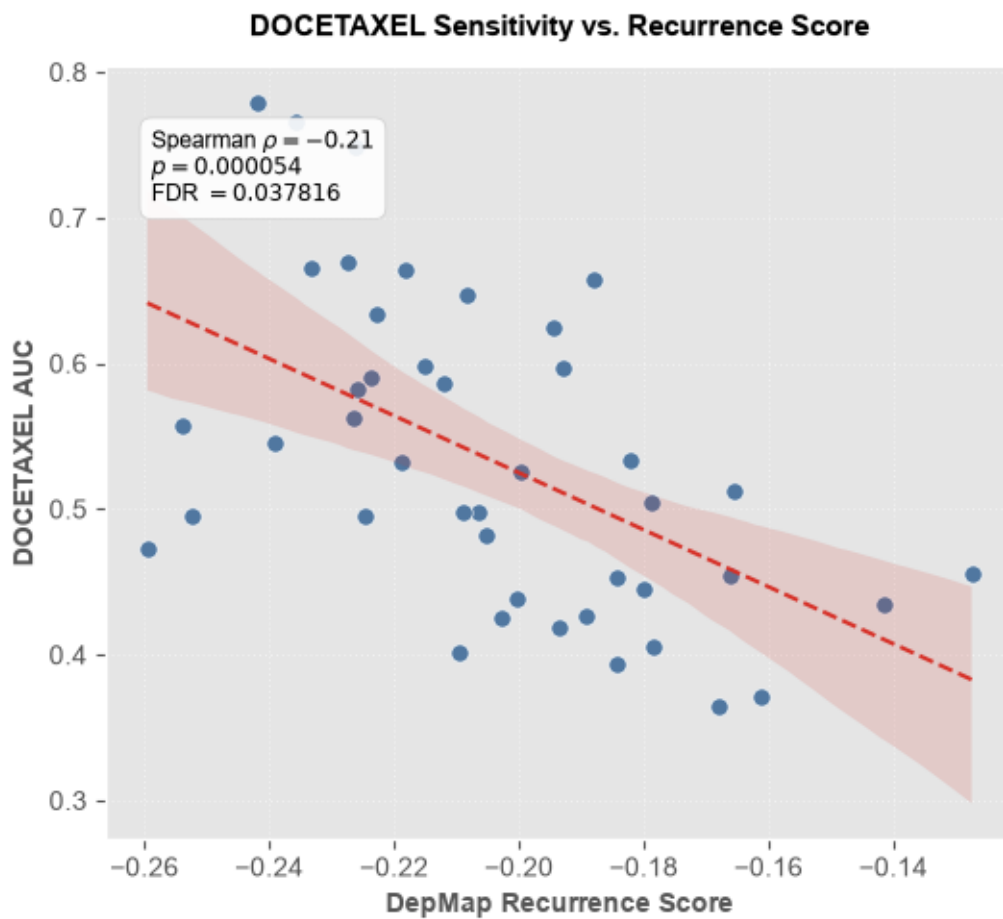
sns.despine(ax=ax)
plt.tight_layout()

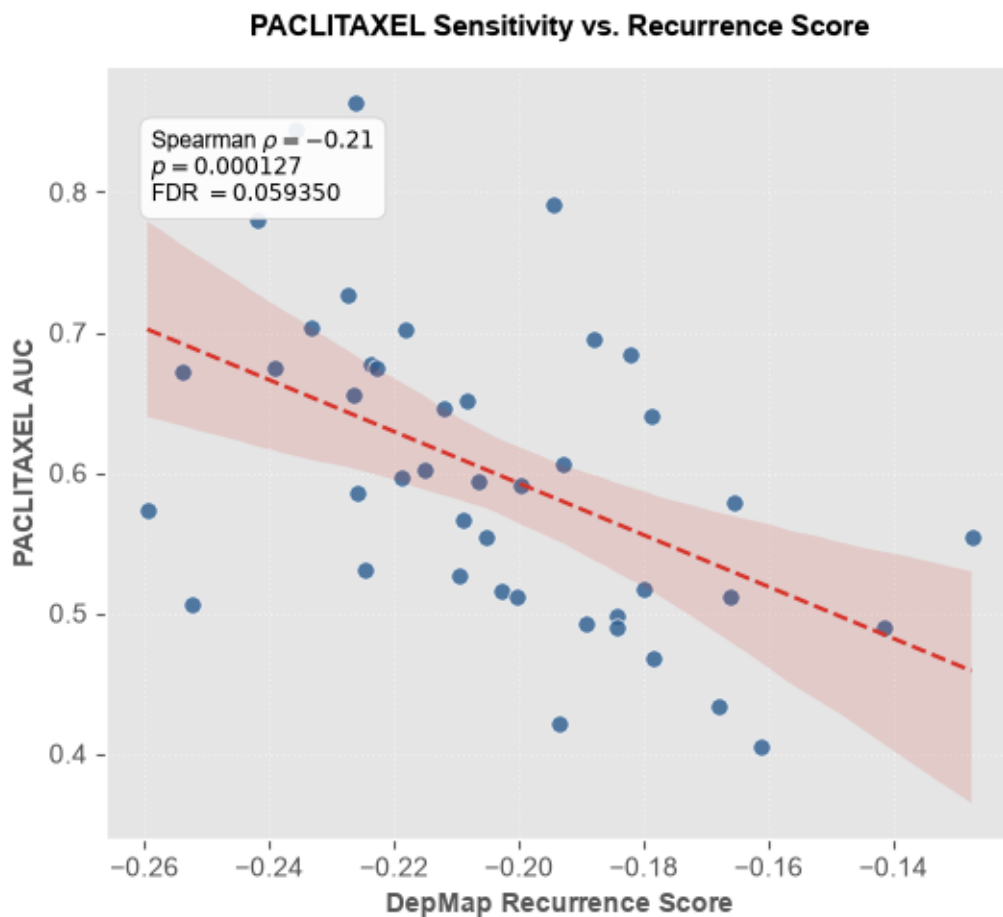
safe_filename_drug = re.sub(
    r"[^\w\-]", "_", clean_display_name.lower()
).strip("_")
filename = f"prism_candidate_{safe_filename_drug}_scatter.png"

plt.savefig(
    f"../../figures/{filename}",
    dpi=300,
    bbox_inches="tight",
)
plt.show()
plt.close()

```







- prism_candidate_ALVESPIMYCIN_scatter.png, prism_candidate_DOCETAXEL_scatter.png and prism_candidate_PACLITAXEL_scatter.png saved to /figures directory

6.5 5. Assess the stability of candidate drug sensitivities

```
[410]: N_BOOTSTRAP = 1000
RANDOM_SEED = 42
rng = np.random.default_rng(RANDOM_SEED)

bootstrap_results = []

candidate_df = prism_recurrence_drug_screen.loc[
    prism_recurrence_drug_screen["CANDIDATE"], ["DRUG", "BRD_ID"]
]

for _, row in candidate_df.iterrows():
    drug_name = row["DRUG"]
    brd_id = row["BRD_ID"]
```

```

cohort_col = f"{drug_name} (BRD:{brd_id})"

if cohort_col not in prism_cohort.columns:
    continue

valid = recurrence_score.notna() & prism_cohort[cohort_col].notna()
recurrence_values = recurrence_score.loc[valid].to_numpy()
auc_values = prism_cohort.loc[valid, cohort_col].to_numpy()
n = len(recurrence_values)

if n < 3:
    continue

original_rho, _ = spearmanr(recurrence_values, auc_values)

boot_indices = rng.integers(0, n, size=(N_BOOTSTRAP, n))

boot_x = recurrence_values[boot_indices]
boot_y = auc_values[boot_indices]
rank_x = rankdata(boot_x, axis=1)
rank_y = rankdata(boot_y, axis=1)
mean_x = rank_x.mean(axis=1, keepdims=True)
mean_y = rank_y.mean(axis=1, keepdims=True)
cov = ((rank_x - mean_x) * (rank_y - mean_y)).sum(axis=1)
var_x = ((rank_x - mean_x) ** 2).sum(axis=1)
var_y = ((rank_y - mean_y) ** 2).sum(axis=1)

valid_mask = (var_x > 0) & (var_y > 0)
bootstrap_rhos = np.full(N_BOOTSTRAP, np.nan)
bootstrap_rhos[valid_mask] = cov[valid_mask] / np.sqrt(
    var_x[valid_mask] * var_y[valid_mask]
)

clean_rhos = bootstrap_rhos[np.isfinite(bootstrap_rhos)]

if len(clean_rhos) == 0:
    continue

bootstrap_results.append(
    {
        "DRUG": drug_name,
        "BRD_ID": brd_id,
        "N_CELL_LINES": n,
        "ORIGINAL_RHO": original_rho,
        "BOOTSTRAP_MEDIAN_RHO": np.median(clean_rhos),
        "BOOTSTRAP_2.5_PERCENTILE": np.percentile(clean_rhos, 2.5),
    }
)

```

```

        "BOOTSTRAP_97.5_PERCENTILE": np.percentile(clean_rhos, 97.5),
        "FRACTION_NEGATIVE": np.mean(clean_rhos < 0),
        "N_BOOTSTRAP_VALID": len(clean_rhos),
    }
)

bootstrap_df = pd.DataFrame(bootstrap_results)

```

```

[411]: prism_candidate_bootstrap_stability = pd.DataFrame(
        bootstrap_results
    )
    prism_candidate_bootstrap_stability = (
        prism_candidate_bootstrap_stability
        .sort_values("ORIGINAL_RHO")
        .reset_index(drop=True)
    )

```

```

[412]: prism_candidate_bootstrap_stability

```

```

[412]:
      DRUG      BRD_ID  N_CELL_LINES  ORIGINAL_RHO  \
0  ALVESPIMYCIN  BRD-K83988098-001-02-0      43    -0.605557
1    DOCETAXEL  BRD-K30577245-001-05-0      43    -0.575657
2    PACLITAXEL  BRD-K62008436-001-23-9      43    -0.551495

      BOOTSTRAP_MEDIAN_RHO  BOOTSTRAP_2.5_PERCENTILE  BOOTSTRAP_97.5_PERCENTILE  \
0          -0.608547          -0.805551          -0.340396
1          -0.570337          -0.741298          -0.322081
2          -0.556692          -0.747216          -0.295604

      FRACTION_NEGATIVE  N_BOOTSTRAP_VALID
0              1.0          1000
1              1.0          1000
2              1.0          1000

```

```

[413]: prism_candidate_bootstrap_stability.to_excel(
        ".../results/PRISM/prism_candidate_bootstrap_stability.xlsx",
        index=False,
    )

```

- prism_candidate_bootstrap_stability.xlsx exported to /results/PRISM directory

6.6 6. Assess potential proliferation-related confounding

```

[414]: target_pathways = ["E2F Targets", "G2-M Checkpoint"]
    prolifer_df = hallmark_results[hallmark_results["PATHWAY"].isin(target_pathways)]

```

```
[415]: proliferation_genes = set()
for raw_str in prolif_df["LEADING_EDGE_GENES"]:
    genes = [g.strip() for g in str(raw_str).replace(",", ";").split(";") if g.
↳strip()]
    proliferation_genes.update(genes)

valid_prolif_genes = [g for g in proliferation_genes if g in depmap_glioma_exp.
↳columns]
```

```
[416]: exp_subset = depmap_glioma_exp[valid_prolif_genes]
z_scores = (exp_subset - exp_subset.mean()) / exp_subset.std()
depmap_proliferation_score = z_scores.mean(axis=1)
```

```
[417]: shared_idx = recurrence_score.dropna().index.intersection(
    depmap_proliferation_score.dropna().index
)
x_vals = recurrence_score.loc[shared_idx]
y_vals = depmap_proliferation_score.loc[shared_idx]

rho_prolif, p_val_prolif = spearmanr(x_vals, y_vals)

fig, ax = plt.subplots(figsize=(6, 5.5))
sns.regplot(
    x=x_vals,
    y=y_vals,
    ax=ax,
    color="#2b5c8f",
    scatter_kws={
        "alpha": 0.8,
        "s": 40,
        "edgecolor": "white",
        "linewidths": 0.6,
    },
    line_kws={"color": "#d73027", "linewidth": 1.5, "linestyle": "--"},
)
stat_text = (
    f"Spearman  $\rho$  = {float(rho_prolif):.3f}\n"
    f"p = {format_pvalue(p_val_prolif)}"
)
ax.text(
    0.05,
    0.92,
    stat_text,
    transform=ax.transAxes,
    fontsize=9,
    verticalalignment="top",
    bbox=dict(
```

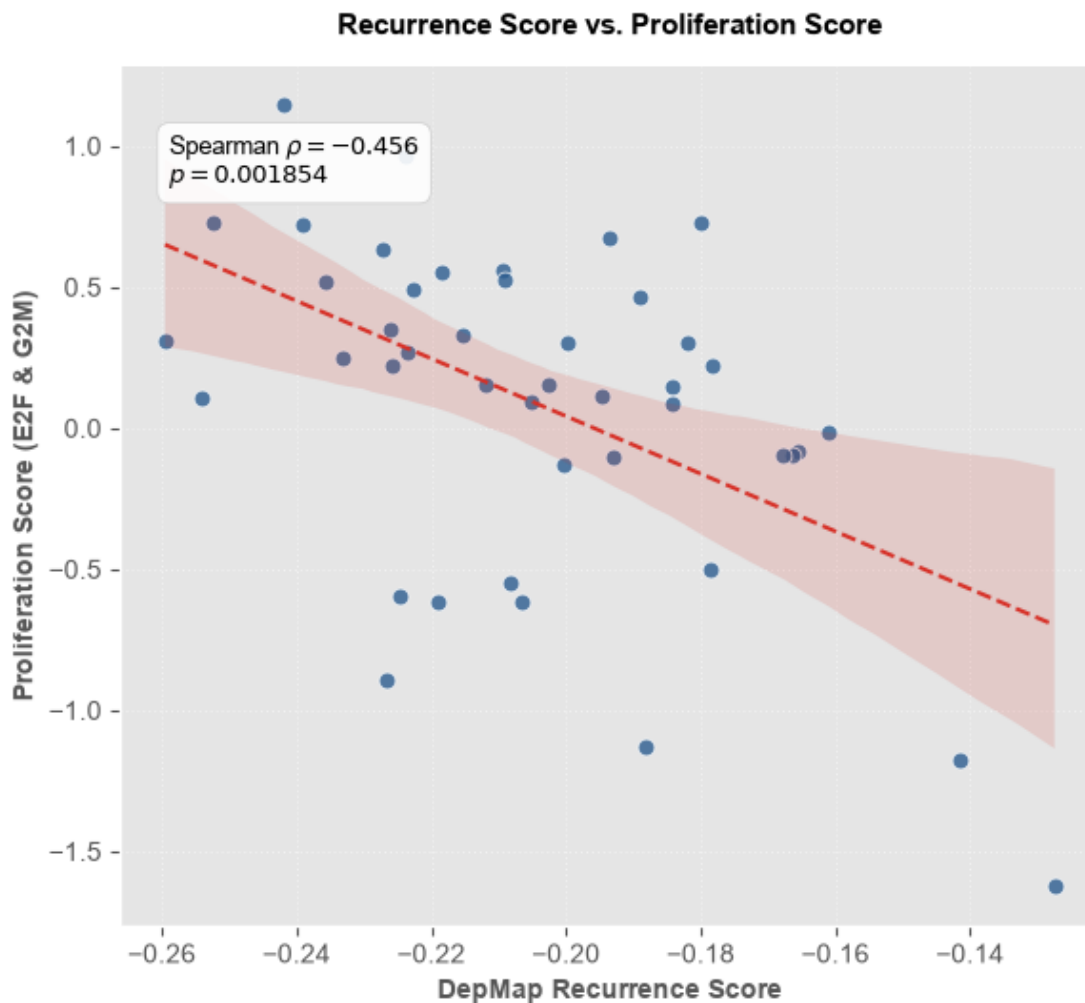
```

        boxstyle="round,pad=0.5",
        facecolor="white",
        edgecolor="#cccccc",
        alpha=0.9,
    ),
)
ax.set_title(
    "Recurrence Score vs. Proliferation Score",
    fontweight="bold",
    fontsize=11,
    pad=12,
)
ax.set_xlabel("DepMap Recurrence Score", fontweight="bold", fontsize=10)
ax.set_ylabel("Proliferation Score (E2F & G2M)", fontweight="bold", fontsize=10)

ax.grid(True, linestyle=":", alpha=0.5)
sns.despine(ax=ax)
plt.tight_layout()

plt.savefig(
    "../figures/recurrence_vs_proliferation_score.png",
    dpi=300,
    bbox_inches="tight",
)
plt.show()

```



```
[418]: depmap_proliferation_score_clean = depmap_proliferation_score.groupby(level=0).
        ↪mean()

# Extract candidate rows with both DRUG and BRD_ID
candidate_drugs_df = prism_recurrence_drug_screen.loc[
    prism_recurrence_drug_screen["CANDIDATE"], ["DRUG", "BRD_ID"]
]

prism_candidates_adjusted_proliferation = []

for _, row in candidate_drugs_df.iterrows():
    drug = row["DRUG"]
    brd_id = row["BRD_ID"]

    # Reconstruct the column header used in prism_cohort
```

```

cohort_col = f"{drug} (BRD:{brd_id})"

# Skip if column is missing from matrix
if cohort_col not in prism_cohort.columns:
    continue

reg_df = pd.DataFrame({
    "AUC": prism_cohort[cohort_col],
    "RECURRENCE_SCORE": recurrence_score,
    "PROLIFERATION_SCORE": depmap_proliferation_score_clean,
}).dropna()

if len(reg_df) < 20:
    continue

X = reg_df[["RECURRENCE_SCORE", "PROLIFERATION_SCORE"]]
X = sm.add_constant(X)
y = reg_df["AUC"]

model = sm.OLS(y, X).fit()

prism_candidates_adjusted_proliferation.append({
    "DRUG": drug,
    "BRD_ID": brd_id,
    "N_CELL_LINES": len(reg_df),
    "BETA_RECURRENCE_SCORE": model.params["RECURRENCE_SCORE"],
    "P_RECURRENCE_SCORE": model.pvalues["RECURRENCE_SCORE"],
    "BETA_PROLIFERATION": model.params["PROLIFERATION_SCORE"],
    "P_PROLIFERATION": model.pvalues["PROLIFERATION_SCORE"],
    "R2_ADJUSTED": model.rsquared_adj,
})

prism_candidates_adjusted_proliferation = pd.DataFrame(
    prism_candidates_adjusted_proliferation
)

```

```

[419]: spearman_metrics = prism_recurrence_drug_screen[
    ["DRUG", "SPEARMAN_RHO", "P_VALUE", "FDR"]
].rename(
    columns={"P_VALUE": "SPEARMAN_P_VALUE", "FDR": "SPEARMAN_FDR"}
)

prism_candidates_adjusted_proliferation =
    prism_candidates_adjusted_proliferation.merge(
        spearman_metrics, on="DRUG", how="left"
    )

```



```

column_order = [
    "DRUG",
    "N_CELL_LINES",
    "SPEARMAN_RHO",
    "SPEARMAN_P_VALUE",
    "SPEARMAN_FDR",
    "BETA_RECURRENCE_SCORE",
    "P_RECURRENCE_SCORE",
    "BETA_PROLIFERATION",
    "P_PROLIFERATION",
    "R2_ADJUSTED",
]

prism_candidates_adjusted_proliferation = (
    prism_candidates_adjusted_proliferation[column_order]
)

prism_candidates_adjusted_proliferation = pd.
↳DataFrame(prism_candidates_adjusted_proliferation)

```

```
[420]: prism_candidates_adjusted_proliferation.head()
```

```
[420]:
```

	DRUG	N_CELL_LINES	SPEARMAN_RHO	SPEARMAN_P_VALUE	SPEARMAN_FDR	\
0	ALVESPIMYCIN	43	-0.605557	0.000017	0.023769	
1	DOCETAXEL	43	-0.575657	0.000054	0.037816	
2	PACLITAXEL	43	-0.551495	0.000127	0.059350	

	BETA_RECURRENCE_SCORE	P_RECURRENCE_SCORE	BETA_PROLIFERATION	\
0	-1.334089	0.000558	0.001152	
1	-2.129591	0.000515	-0.016881	
2	-2.053034	0.001227	-0.020650	

	P_PROLIFERATION	R2_ADJUSTED
0	0.948382	0.291911
1	0.550479	0.258236
2	0.485786	0.216952

```
[421]: prism_candidates_adjusted_proliferation.to_excel(
    ".../results/PRISM/prism_candidates_adjusted_proliferation.xlsx",
    ↳index=False
)
```

Summary Note: Multivariate OLS regression across $N = 43$ glioma cell lines confirmed that candidate drug sensitivity is driven specifically by the Recurrence Score rather than cell proliferation, which itself correlated inversely with recurrence ($\rho = -0.456, p = 0.0018$). After controlling for proliferation, Alvespimycin ($\beta = -1.334, p = 0.00056$), Docetaxel ($\beta = -2.129, p = 0.00052$), and Paclitaxel ($\beta = -2.053, p = 0.00123$) all maintained strong, statistically significant Recurrence

Score coefficients. In contrast, the Proliferation Score term was non-significant across all individual models — Alvepimycin ($p = 0.948$), Docetaxel ($p = 0.551$), and Paclitaxel ($p = 0.486$) —, proving candidate vulnerabilities are specific to recurrence biology and unconfounded by cell division speed.

- `prism_candidates_adjusted_proliferation.xlsx` exported to `/results/PRISM`

6.7 7. Perform independent validation using GDSC1 and GDSC2

```
[422]: gdc_recurrence_score = glass_crossplatform_recurrence_score_gdsc[
    "GDSC_RECURRENCE_SCORE_CROSSPLATFORM"
].dropna()
prism_candidates = ["Alvespimycin", "Docetaxel", "Paclitaxel"]

gdc_validation_of_prism_candidates = []
datasets = [("GDSC1", gdc1_glioma), ("GDSC2", gdc2_glioma)]

for candidate in prism_candidates:
    for dataset_name, df in datasets:
        cols = [str(c) for c in df.columns]
        matches = [c for c in cols if candidate.lower() in c.lower() and
↪ "combo" not in c.lower()]

        if not matches:
            gdc_validation_of_prism_candidates.append({
                "CANDIDATE": candidate,
                "DATASET": dataset_name,
                "MATCHED_COLUMN": "NOT_FOUND",
                "N": 0,
                "SPEARMAN_RHO": np.nan,
                "P_VALUE": np.nan,
                "DIRECTIONALLY_VALIDATED": False,
                "NOTES": "Absent from compound library",
            })
            continue

        for matched_col in matches:
            drug_auc = df[matched_col].dropna().groupby(level=0).mean()

            eval_df = pd.DataFrame({
                "AUC": drug_auc,
                "GDSC_RECURRENCE_SCORE": gdc_recurrence_score,
            }).dropna()

            n_samples = len(eval_df)

            if n_samples < 5:
                gdc_validation_of_prism_candidates.append({
```

```

        "CANDIDATE": candidate,
        "DATASET": dataset_name,
        "MATCHED_COLUMN": matched_col,
        "N": n_samples,
        "SPEARMAN_RHO": np.nan,
        "P_VALUE": np.nan,
        "DIRECTIONALLY_VALIDATED": False,
        "NOTES": f"Insufficient sample size (N={n_samples})",
    })
    continue

    rho, p_val = spearmanr(eval_df["AUC"],
eval_df["GDSC_RECURRENCE_SCORE"])
    is_validated = rho < 0

    gdsc_validation_of_prism_candidates.append({
        "CANDIDATE": candidate,
        "DATASET": dataset_name,
        "MATCHED_COLUMN": matched_col,
        "N": n_samples,
        "SPEARMAN_RHO": rho,
        "P_VALUE": p_val,
        "DIRECTIONALLY_VALIDATED": is_validated,
        "NOTES": (
            "Directionally validated (rho < 0)"
            if is_validated
            else "Non-validated (rho >= 0)"
        ),
    })

gdsc_validation_of_prism_candidates = pd.
    DataFrame(gdsc_validation_of_prism_candidates)

gdsc_validation_of_prism_candidates["FDR"] = np.nan
valid_p_mask = gdsc_validation_of_prism_candidates["P_VALUE"].notna()

if valid_p_mask.any():
    p_vals = gdsc_validation_of_prism_candidates.loc[valid_p_mask, "P_VALUE"]
    _, fdr_vals, _, _ = multipletests(p_vals, method="fdr_bh")
    gdsc_validation_of_prism_candidates.loc[valid_p_mask, "FDR"] = fdr_vals

```

[423]: gdsc_validation_of_prism_candidates

```

[423]:
   CANDIDATE  DATASET  MATCHED_COLUMN  N  SPEARMAN_RHO  P_VALUE  \
0  Alvespimycin  GDSC1      NOT_FOUND  0           NaN      NaN
1  Alvespimycin  GDSC2      NOT_FOUND  0           NaN      NaN
2   Docetaxel   GDSC1      NOT_FOUND  0           NaN      NaN

```

3	Docetaxel	GDSC2	DOCETAXEL (GDSC2:1007)	50	-0.221224	0.122606
4	Paclitaxel	GDSC1	PACLITAXEL (GDSC1:11)	23	0.113636	0.605668
5	Paclitaxel	GDSC2	PACLITAXEL (GDSC2:1080)	49	0.022825	0.876292

	DIRECTIONALLY_VALIDATED	NOTES	FDR
0	False	Absent from compound library	NaN
1	False	Absent from compound library	NaN
2	False	Absent from compound library	NaN
3	True	Directionally validated (rho < 0)	0.367818
4	False	Non-validated (rho >= 0)	0.876292
5	False	Non-validated (rho >= 0)	0.876292

```
[424]: gdsvalidation_of_prism_candidates.to_excel(
        "../results/validation/gdsvalidation_of_prism_candidates.xlsx",
        index=False
    )
```

- gdsvalidation_of_prism_candidates.xlsx saved to /results/GDSC directory

6.8 8. Perform additional validation using CTD² and CTRPv2

```
[425]: depmap_recurrence_score = (
        prism_recurrence_analysis_cohort
        .set_index("DEPMAP_ID")["DEPMAP_RECURRENCE_SCORE"]
        .reindex(prism_cohort.index)
    )
prism_candidates = ["Alvespimycin", "Docetaxel", "Paclitaxel", "AUY922"]

external_validation_prism_candidates = []
datasets = [("CTD2", ctd2_glioma), ("CTRPv2", ctrpv2_glioma)]

for candidate in prism_candidates:
    for dataset_name, df in datasets:
        cols = [str(c) for c in df.columns]
        matches = [c for c in cols if candidate.lower() in c.lower() and
                    ↪ "combo" not in c.lower()]

        if not matches:
            external_validation_prism_candidates.append({
                "CANDIDATE": candidate,
                "DATASET": dataset_name,
                "MATCHED_COLUMN": "NOT_FOUND",
                "N": 0,
                "SPEARMAN_RHO": np.nan,
                "P_VALUE": np.nan,
                "DIRECTIONALLY_VALIDATED": False,
                "NOTES": "Absent from compound library",
            })
```

```

    })
    continue

for matched_col in matches:
    drug_auc = df[matched_col].dropna().groupby(level=0).mean()

    eval_df = pd.DataFrame({
        "AUC": drug_auc,
        "RECURRENCE_SCORE": depmap_recurrence_score,
    }).dropna()

    n_samples = len(eval_df)

    if n_samples < 5:
        external_validation_prism_candidates.append({
            "CANDIDATE": candidate,
            "DATASET": dataset_name,
            "MATCHED_COLUMN": matched_col,
            "N": n_samples,
            "SPEARMAN_RHO": np.nan,
            "P_VALUE": np.nan,
            "DIRECTIONALLY_VALIDATED": False,
            "NOTES": f"Insufficient sample size (N={n_samples})",
        })
        continue

    rho, p_val = spearmanr(eval_df["AUC"], eval_df["RECURRENCE_SCORE"])
    is_validated = rho < 0

    external_validation_prism_candidates.append({
        "CANDIDATE": candidate,
        "DATASET": dataset_name,
        "MATCHED_COLUMN": matched_col,
        "N": n_samples,
        "SPEARMAN_RHO": rho,
        "P_VALUE": p_val,
        "DIRECTIONALLY_VALIDATED": is_validated,
        "NOTES": (
            "Directionally validated (rho < 0)"
            if is_validated
            else "Non-validated (rho >= 0)"
        ),
    })

external_validation_prism_candidates = pd.
↳ DataFrame(external_validation_prism_candidates)

```

```
external_validation_prism_candidates["FDR"] = np.nan
valid_p_mask = external_validation_prism_candidates["P_VALUE"].notna()

if valid_p_mask.any():
    p_vals = external_validation_prism_candidates.loc[valid_p_mask, "P_VALUE"]
    _, fdr_vals, _, _ = multipletests(p_vals, method="fdr_bh")
    external_validation_prism_candidates.loc[valid_p_mask, "FDR"] = fdr_vals
```

```
[426]: external_validation_prism_candidates
```

```
[426]:
```

	CANDIDATE	DATASET	MATCHED_COLUMN \
0	Alvespimycin	CTD ²	NOT_FOUND
1	Alvespimycin	CTRPv2	NOT_FOUND
2	Docetaxel	CTD ²	DOCETAXEL (CTRP:660364)
3	Docetaxel	CTD ²	DOCETAXEL:TANESPIMYCIN (2:1 MOL/MOL) (CTRP:660...
4	Docetaxel	CTRPv2	DOCETAXEL (CTRP:660364)
5	Paclitaxel	CTD ²	PACLITAXEL (CTRP:26956)
6	Paclitaxel	CTRPv2	PACLITAXEL (CTRP:26956)
7	AUY922	CTD ²	NOT_FOUND
8	AUY922	CTRPv2	NOT_FOUND

	N	SPEARMAN_RHO	P_VALUE	DIRECTIONALLY_VALIDATED \
0	0	NaN	NaN	False
1	0	NaN	NaN	False
2	13	-0.131868	0.667608	True
3	35	-0.115686	0.508112	True
4	13	-0.076923	0.802763	True
5	35	-0.115126	0.510182	True
6	36	-0.129215	0.452601	True
7	0	NaN	NaN	False
8	0	NaN	NaN	False

	NOTES	FDR
0	Absent from compound library	NaN
1	Absent from compound library	NaN
2	Directionally validated (rho < 0)	0.802763
3	Directionally validated (rho < 0)	0.802763
4	Directionally validated (rho < 0)	0.802763
5	Directionally validated (rho < 0)	0.802763
6	Directionally validated (rho < 0)	0.802763
7	Absent from compound library	NaN
8	Absent from compound library	NaN

```
[427]: external_validation_prism_candidates.to_excel(
    "../../results/validation/external_validation_prism_candidates.xlsx",
    index=False
)
```

- external_validation_prism_candidates.xlsx saved to /results/validation

6.9 9. Assess reproducibility at the compound and target levels

```
[428]: candidates_metadata = {
    "Alvespimycin": {
        "TARGET": "HSP90",
        "MECHANISM_OF_ACTION": "HSP90 inhibitor",
    },
    "Docetaxel": {
        "TARGET": "TUBB1 / Microtubules",
        "MECHANISM_OF_ACTION": "Microtubule stabilizer",
    },
    "Paclitaxel": {
        "TARGET": "TUBB1 / Microtubules",
        "MECHANISM_OF_ACTION": "Microtubule stabilizer",
    },
}

candidates = ["Alvespimycin", "Docetaxel", "Paclitaxel"]
integrated_rows = []

[429]: def get_rho_from_df(df, candidate_name, dataset_name=None,
    ↪rho_col="SPEARMAN_RHO"):
    """Extract Spearman Rho from validation dataframes with flexible string
    ↪matching."""
    if df is None or df.empty:
        return np.nan

    df_copy = df.copy()
    df_copy.columns = [str(c).upper().strip() for c in df_copy.columns]

    candidate_col = next(
        (
            c
            for c in df_copy.columns
            if any(k in c for k in ["CANDIDATE", "DRUG", "COMPOUND"])
        ),
        None,
    )
    if not candidate_col:
        return np.nan

    sub = df_copy[
        df_copy[candidate_col]
        .astype(str)
        .str.lower()
```

```

        .str.strip()
        .str.contains(candidate_name.lower(), regex=False)
    ]

    if dataset_name and "DATASET" in sub.columns:
        sub = sub[
            sub["DATASET"]
            .astype(str)
            .str.lower()
            .str.contains(dataset_name.lower(), regex=False)
        ]

    if sub.empty:
        return np.nan

    target_rho_col = next(
        (c for c in sub.columns if rho_col.upper() in c or "RHO" in c), None
    )
    if target_rho_col:
        val = sub[target_rho_col].dropna()
        return val.iloc[0] if not val.empty else np.nan

    return np.nan

```

```

[430]: def get_fdr_from_df(df, candidate_name):
    """Extract FDR from PRISM dataframe with flexible string matching."""
    if df is None or df.empty:
        return np.nan

    df_copy = df.copy()
    df_copy.columns = [str(c).upper().strip() for c in df_copy.columns]

    candidate_col = next(
        (
            c
            for c in df_copy.columns
            if any(k in c for k in ["CANDIDATE", "DRUG", "COMPOUND"])
        ),
        None,
    )
    if not candidate_col:
        return np.nan

    sub = df_copy[
        df_copy[candidate_col]
        .astype(str)
        .str.lower()

```



```

        .str.strip()
        .str.contains(candidate_name.lower(), regex=False)
    ]

    fdr_col = next(
        (c for c in sub.columns if any(k in c for k in ["FDR", "Q_VALUE", "QVAL"])),
        None,
    )
    if fdr_col and not sub.empty:
        val = sub[fdr_col].dropna()
        return val.iloc[0] if not val.empty else np.nan

    return np.nan

```

```

[431]: for drug in candidates:
    meta = candidates_metadata[drug]
    prism_rho = get_rho_from_df(prism_recurrence_drug_screen, drug)
    prism_fdr = get_fdr_from_df(prism_recurrence_drug_screen, drug)
    gdsc1_rho = get_rho_from_df(
        gdsc_validation_of_prism_candidates, drug, dataset_name="gdsc1"
    )
    gdsc2_rho = get_rho_from_df(
        gdsc_validation_of_prism_candidates, drug, dataset_name="gdsc2"
    )
    ctd2_rho = get_rho_from_df(
        external_validation_prism_candidates,
        drug,
        dataset_name="ctd",
    )
    ctrpv2_rho = get_rho_from_df(
        external_validation_prism_candidates,
        drug,
        dataset_name="ctrp",
    )
    all_rhos = [prism_rho, gdsc1_rho, gdsc2_rho, ctd2_rho, ctrpv2_rho]
    n_same_direction = sum(1 for r in all_rhos if pd.notna(r) and r < 0)
    ext_rhos = [gdsc1_rho, gdsc2_rho, ctd2_rho, ctrpv2_rho]
    has_ext_confirmation = any(pd.notna(r) and r < 0 for r in ext_rhos)
    compound_rep = "CONFIRMED" if has_ext_confirmation else "NOT_REPLICATED"
    if compound_rep == "CONFIRMED":
        target_support = "SUPPORTED_VIA_COMPOUND"
    elif meta["TARGET"] == "HSP90":
        target_support = "SUPPORTED (Target-level via HSP90 inhibitor)"
    elif meta["TARGET"] == "TUBB1 / Microtubules":
        target_support = "SUPPORTED (Target-level via Tubulin targeting)"
    else:

```

```

        target_support = "LIMITED_TARGET_SUPPORT"
    integrated_rows.append({
        "DRUG": drug,
        "TARGET": meta["TARGET"],
        "MECHANISM_OF_ACTION": meta["MECHANISM_OF_ACTION"],
        "PRISM_RHO": prism_rho,
        "PRISM_FDR": prism_fdr,
        "GDSC1_RHO": gdsc1_rho,
        "GDSC2_RHO": gdsc2_rho,
        "CTD2_RHO": ctd2_rho,
        "CTRPV2_RHO": ctrpv2_rho,
        "N_DATASETS_SAME_DIRECTION": n_same_direction,
        "COMPOUND_LEVEL_REPLICATION": compound_rep,
        "TARGET_LEVEL_SUPPORT": target_support,
    })

recurrence_drug_candidates_integrated = pd.DataFrame(integrated_rows)

```

```
[432]: recurrence_drug_candidates_integrated
```

```
[432]:
```

	DRUG	TARGET	MECHANISM_OF_ACTION	PRISM_RHO	\
0	Alvespimycin	HSP90	HSP90 inhibitor	-0.605557	
1	Docetaxel	TUBB1 / Microtubules	Microtubule stabilizer	-0.575657	
2	Paclitaxel	TUBB1 / Microtubules	Microtubule stabilizer	-0.551495	

	PRISM_FDR	GDSC1_RHO	GDSC2_RHO	CTD2_RHO	CTRPV2_RHO	\
0	0.023769	NaN	NaN	NaN	NaN	
1	0.037816	NaN	-0.221224	-0.131868	-0.076923	
2	0.059350	0.113636	0.022825	-0.115126	-0.129215	

	N_DATASETS_SAME_DIRECTION	COMPOUND_LEVEL_REPLICATION	\
0	1	NOT_REPLICATED	
1	4	CONFIRMED	
2	3	CONFIRMED	

	TARGET_LEVEL_SUPPORT
0	SUPPORTED (Target-level via HSP90 inhibitor)
1	SUPPORTED_VIA_COMPOUND
2	SUPPORTED_VIA_COMPOUND

```
[433]: ctd2_glioma.columns.tolist()
```

```
[433]: ['(+)-JQ-1 (CTRP:616354)',
        'AZD4547 (CTRP:660325)',
        '16-BETA-BROMOANDROSTERONE (CTRP:411726)',
        'TANESPIMYCIN (CTRP:50134)',
        '1S,3R-RSL-3 (CTRP:609060)']
```

'SORAFENIB (CTRP:349006)',
 '3-CL-AHPC (CTRP:660796)',
 'AZACITIDINE (CTRP:26874)',
 '968 (CTRP:660080)',
 'A-1065-5 (CTRP:660433)',
 'A-804598 (CTRP:661033)',
 'AA-COCF3 (CTRP:411724)',
 'AB1010 (CTRP:606143)',
 'ABIRATERONE (CTRP:660363)',
 'METHOTREXATE (CTRP:30371)',
 'ABT 869 (CTRP:606033)',
 'VELIPARIB (CTRP:606034)',
 'ABT-737 (CTRP:411738)',
 'AC 220 (CTRP:640011)',
 'AC55649 (CTRP:375487)',
 'ADAVOSERTIB (CTRP:418167)',
 'AFATINIB (CTRP:606135)',
 'AG-13736 (CTRP:348990)',
 'AGK-2 (CTRP:411716)',
 'AK-55409 (CTRP:639390)',
 'ALISERTIB (CTRP:636711)',
 'ALISERTIB:NAVITOCILAX (2:1 MOL/MOL) (CTRP:660282)',
 'ALPELISIB (CTRP:668495)',
 'ALVOCIDIB (CTRP:687720)',
 'AM-580 (CTRP:58455)',
 'APICIDIN (CTRP:61097)',
 'APO866 (CTRP:418166)',
 'SELU METINIB (CTRP:348991)',
 'TIVOZANIB (CTRP:418170)',
 'ASP-7487 (CTRP:705300)',
 'AT-406 (CTRP:710154)',
 'AT13387 (CTRP:688229)',
 'AT7867 (CTRP:660409)',
 'TRETINOIN (CTRP:23151)',
 'AUSTOCYSTIN D (CTRP:416775)',
 'AVICIN D (CTRP:688975)',
 'AVRAINVILLAMIDE (CTRP:411856)',
 'AZ-10353926 (CTRP:606035)',
 'AZ-3146 (CTRP:660087)',
 'AZD 2171 (CTRP:606136)',
 'AZD-8055 (CTRP:609639)',
 'AZD1480 (CTRP:660306)',
 'AZD7545 (CTRP:411814)',
 'AZD7762 (CTRP:660777)',
 'B02 (CTRP:122255)',
 'BAFILOMYCIN A1 (CTRP:467037)',
 'BARASERTIB (CTRP:601923)',

'BARDOXOLONE METHYL (CTRP:660318)',
 'BAX CHANNEL BLOCKER (CTRP:18311)',
 'BCL-LZH-4 (CTRP:688516)',
 'BEC (CTRP:60679)',
 'BELINOSTAT (CTRP:60225)',
 'BENDAMUSTINE (CTRP:374755)',
 'BETULINIC ACID (CTRP:23256)',
 'BEXAROTENE (CTRP:374747)',
 'BEZ235 (CTRP:348992)',
 'BI-2536 (CTRP:347813)',
 'BIBR-1532 (CTRP:417982)',
 'BIRB 796 (CTRP:660412)',
 'BIRINAPANT (CTRP:660778)',
 'BIX-01294 (CTRP:53705)',
 'BLEBBISTATIN (CTRP:50140)',
 'BLEOMYCIN A2 (CTRP:639531)',
 'BMS 536924 (CTRP:411859)',
 'BMS-195614 (CTRP:660228)',
 'BMS-270394 (CTRP:660321)',
 'BMS-345541 (CTRP:622911)',
 'BMS-754807 (CTRP:417981)',
 'BORTEZOMIB (CTRP:411737)',
 'BOSUTINIB (CTRP:660346)',
 'BRD-A02303741 (CTRP:659371)',
 'BRD-A02303741:CARBOPLATIN (1:1 MOL/MOL) (CTRP:660271)',
 'BRD-A02303741:NAVITOCILAX (2:1 MOL/MOL) (CTRP:660268)',
 'BRD-A05715709 (CTRP:660342)',
 'BRD-A71883111 (CTRP:660145)',
 'BRD-A86708339 (CTRP:687769)',
 'BRD-A94377914 (CTRP:61724)',
 'BRD-K01737880 (CTRP:438345)',
 'BRD-K02251932 (CTRP:442141)',
 'BRD-K02492147 (CTRP:415688)',
 'BRD-K03536150 (CTRP:686134)',
 'BRD-K03911514 (CTRP:660424)',
 'BRD-K04800985 (CTRP:435086)',
 'BRD-K07442505 (CTRP:660229)',
 'BRD-K09344309 (CTRP:439882)',
 'BRD-K09587429 (CTRP:416842)',
 'BRD-K11533227 (CTRP:389297)',
 'BRD-K13185470 (CTRP:660839)',
 'BRD-K13999467 (CTRP:410270)',
 'BRD-K14844214 (CTRP:402775)',
 'BRD-K16130065 (CTRP:660425)',
 'BRD-K16147474 (CTRP:447732)',
 'BRD-K17060750 (CTRP:600054)',
 'BRD-K19103580 (CTRP:436211)',

'BRD-K20514654 (CTRP:651304)',
'BRD-K24690302 (CTRP:641721)',
'BRD-K26531177 (CTRP:42319)',
'BRD-K27188169 (CTRP:660798)',
'BRD-K27188169:NAVITOCCLAX (2:1 MOL/MOL) (CTRP:665818)',
'BRD-K27224038 (CTRP:397964)',
'BRD-K27986637 (CTRP:452570)',
'BRD-K28456706 (CTRP:651642)',
'BRD-K29086754 (CTRP:448891)',
'BRD-K29313308 (CTRP:634814)',
'BRD-K30019337 (CTRP:437263)',
'BRD-K30748066 (CTRP:689506)',
'BRD-K33199242 (CTRP:449707)',
'BRD-K33514849 (CTRP:401251)',
'BRD-K34099515 (CTRP:660064)',
'BRD-K34222889 (CTRP:688888)',
'BRD-K34485477 (CTRP:446716)',
'BRD-K35604418 (CTRP:660230)',
'BRD-K37390332 (CTRP:463198)',
'BRD-K41334119 (CTRP:446546)',
'BRD-K41597374 (CTRP:404452)',
'BRD-K42260513 (CTRP:687715)',
'BRD-K44224150 (CTRP:438564)',
'BRD-K45681478 (CTRP:142931)',
'BRD-K48334597 (CTRP:431987)',
'BRD-K48477130 (CTRP:649754)',
'BRD-K49290616 (CTRP:414479)',
'BRD-K50799972 (CTRP:596868)',
'BRD-K51490254 (CTRP:616355)',
'BRD-K51831558 (CTRP:687661)',
'BRD-K52037352 (CTRP:450415)',
'BRD-K55116708 (CTRP:444779)',
'BRD-K55473186 (CTRP:651306)',
'BRD-K58730230 (CTRP:660797)',
'BRD-K61166597 (CTRP:348633)',
'BRD-K63431240 (CTRP:404566)',
'BRD-K64610608 (CTRP:438776)',
'BRD-K66453893 (CTRP:387472)',
'BRD-K66532283 (CTRP:376693)',
'BRD-K70511574 (CTRP:648987)',
'BRD-K71781559 (CTRP:425733)',
'BRD-K71935468 (CTRP:58566)',
'BRD-K75293299 (CTRP:438691)',
'BRD-K78574327 (CTRP:435898)',
'BRD-K79669418 (CTRP:660139)',
'BRD-K80183349 (CTRP:376307)',
'BRD-K84807411 (CTRP:437906)',

'BRD-K85133207 (CTRP:616408)',
 'BRD-K86535717 (CTRP:429535)',
 'BRD-K88742110 (CTRP:606670)',
 'BRD-K90370028 (CTRP:362207)',
 'BRD-K92856060 (CTRP:119769)',
 'BRD-K94991378 (CTRP:53556)',
 'BRD-K96431673 (CTRP:417262)',
 'BRD-K96970199 (CTRP:410703)',
 'BRD-K97651142 (CTRP:660135)',
 'BRD-K99006945 (CTRP:687395)',
 'BRD-K99584050 (CTRP:660140)',
 'BRD-M00053801 (CTRP:660890)',
 'BRD1812 (CTRP:418162)',
 'BRD1835 (CTRP:404070)',
 'BRD4132 (CTRP:3588)',
 'BRD6340 (CTRP:12877)',
 'BRD8899 (CTRP:608062)',
 'BRD8958 (CTRP:418163)',
 'BRD9647 (CTRP:37479)',
 'BRD9876 (CTRP:19153)',
 'BRD9876:MK-1775 (4:1 MOL/MOL) (CTRP:689136)',
 'BREFELDIN A (CTRP:411714)',
 'BRIVANIB (CTRP:606246)',
 'C6-CERAMIDE (CTRP:44554)',
 'CABOZANTINIB (CTRP:660840)',
 'CAL-101 (CTRP:660430)',
 'CANERTINIB (CTRP:606138)',
 'CARBOPLATIN (CTRP:26979)',
 'CARBOPLATIN:ETOPOSIDE (40:17 MOL/MOL) (CTRP:660223)',
 'CARBOPLATIN:UNC0638 (2:1 MOL/MOL) (CTRP:660130)',
 'CAY10576 (CTRP:622913)',
 'CAY10594 (CTRP:411824)',
 'CAY10618 (CTRP:632104)',
 'CBB-1007 (CTRP:687721)',
 'CCT036477 (CTRP:417663)',
 'CD-1530 (CTRP:375562)',
 'CD-437 (CTRP:375256)',
 'CERANIB-2 (CTRP:660078)',
 'CERULENIN (CTRP:52430)',
 'CH-55 (CTRP:660137)',
 'LADUVIGLUSIB (CTRP:347812)',
 'CHLORAMBUCIL (CTRP:25334)',
 'CHM-1 (CTRP:609674)',
 'CI-976 (CTRP:375390)',
 'CICLOPIROX (CTRP:28784)',
 'CID-5951923 (CTRP:267119)',
 'CIL41 (CTRP:176181)',

'CIL55 (CTRP:1788)',
'CIL55A (CTRP:609061)',
'CIL56 (CTRP:100490)',
'CIL70 (CTRP:183526)',
'CIMETIDINE (CTRP:26870)',
'CLOFARABINE (CTRP:660288)',
'COL-3 (CTRP:665914)',
'COMPOUND 1541A (CTRP:411827)',
'COMPOUND 1B (CTRP:62111)',
'COMPOUND 23 CITRATE (CTRP:417451)',
'COMPOUND 7D-CIS (CTRP:411805)',
'CR-1-31B (CTRP:418209)',
'IBRUTINIB (CTRP:660994)',
'CRIZOTINIB (CTRP:628603)',
'CRIZOTINIB:PLX-4032 (2:1 MOL/MOL) (CTRP:660857)',
'CUCURBITACIN I (CTRP:595102)',
'CURCUMIN (CTRP:28452)',
'CYANOQUINOLINE 11 (CTRP:417415)',
'CYCLOPHOSPHAMIDE (CTRP:32620)',
'CICLOSPORIN (CTRP:55512)',
'CYTARABINE HYDROCHLORIDE (CTRP:62690)',
'CYTOCHALASIN B (CTRP:50163)',
'DABRAFENIB (CTRP:687954)',
'DACARBAZINE (CTRP:32622)',
'DARINAPARSIN (CTRP:609121)',
'DASATINIB (CTRP:52882)',
'DBEQ (CTRP:199337)',
'DECITABINE (CTRP:347775)',
'DECITABINE:CARBOPLATIN (1:1 MOL/MOL) (CTRP:660250)',
'DECITABINE:NAVITOCCLAX (2:1 MOL/MOL) (CTRP:660180)',
'DEXAMETHASONE (CTRP:277354)',
'DINACICLIB (CTRP:687578)',
'DOCETAXEL (CTRP:660364)',
'DOCETAXEL:TANESPIMYCIN (2:1 MOL/MOL) (CTRP:660217)',
'DOXORUBICIN (CTRP:36599)',
'DOXORUBICIN:NAVITOCCLAX (2:1 MOL/MOL) (CTRP:660280)',
'ELCPK (CTRP:609056)',
'ELOCALCITOL (CTRP:660319)',
'OXALIPLATIN (CTRP:375582)',
'ENTINOSTAT (CTRP:60218)',
'EPIGALLOCATECHIN-3-MONOGALLATE (CTRP:28801)',
'ERASTIN (CTRP:63578)',
'ERISMODEGIB (CTRP:636425)',
'ERLOTINIB (CTRP:52928)',
'ERLOTINIB:PLX-4032 (2:1 MOL/MOL) (CTRP:660768)',
'ETOMOXIR (CTRP:660082)',
'ETOPOSIDE (CTRP:48589)',

'ETP-46464 (CTRP:666716)',
 'EX 527 (CTRP:608999)',
 'EXEL-2880 (CTRP:628607)',
 'FGIN-1-27 (CTRP:61775)',
 'FINGOLIMOD (CTRP:640157)',
 'FLUOROURACIL (CTRP:25344)',
 'FLUVASTATIN (CTRP:660322)',
 'FQI-1 (CTRP:102193)',
 'FQI-2 (CTRP:609091)',
 'FSC231 (CTRP:660079)',
 'FULVESTRANT (CTRP:464880)',
 'FUMONISIN B1 (CTRP:616353)',
 'GANT-61 (CTRP:609332)',
 'GDC-0879 (CTRP:606248)',
 'PICTILISIB (CTRP:639759)',
 'GEFITINIB (CTRP:52926)',
 'GEMCITABINE (CTRP:411863)',
 'GMX-1778 (CTRP:411843)',
 'GOSSYPOL (CTRP:25036)',
 'GSK-3 INHIBITOR IX (CTRP:52582)',
 'GSK-J4 (CTRP:687735)',
 'GSK1059615 (CTRP:606471)',
 'GSK2636771 (CTRP:668723)',
 'GSK4112 (CTRP:660081)',
 'GSK461364 (CTRP:649862)',
 'MOLIBRESIB (CTRP:660245)',
 'GTPL5238 (CTRP:411832)',
 'GW-405833 (CTRP:375447)',
 'GW-843682X (CTRP:348997)',
 'GX15-070 (CTRP:606142)',
 'HBX-41108 (CTRP:679843)',
 'HC-067047 (CTRP:660083)',
 'HLI 373 (CTRP:481311)',
 'HYPERFORIN (CTRP:660327)',
 'I-BET151 (CTRP:660206)',
 'IC-87114 (CTRP:55307)',
 'TAMOXIFEN (CTRP:26972)',
 'IFOSFAMIDE (CTRP:35319)',
 'IMATINIB (CTRP:345041)',
 'IMPORTAZOLE (CTRP:46792)',
 'INDISULAM (CTRP:411874)',
 'IPR-456 (CTRP:660084)',
 'ISOEVODIAMINE (CTRP:594664)',
 'ISOLIQIRITIGENIN (CTRP:25393)',
 'ISONICOTINOHYDROXAMIC ACID (CTRP:478694)',
 'ISOX (CTRP:632907)',
 'ISOX:BORTEZOMIB (250:1 MOL/MOL) (CTRP:660173)',

'ISTRADEFYLLINE (CTRP:687694)',
'ITRACONAZOLE (CTRP:63349)',
'IU1 (CTRP:272805)',
'SERDEMETAN (CTRP:418164)',
'JQ-1:CARBOPLATIN (1:1 MOL/MOL) (CTRP:660247)',
'JQ-1:MK-0752 (1:1 MOL/MOL) (CTRP:660253)',
'JQ-1:NAVITOCCLAX (2:1 MOL/MOL) (CTRP:660252)',
'JQ-1:UNC0638 (2:1 MOL/MOL) (CTRP:660129)',
'JQ-1:VORINOSTAT (2:1 MOL/MOL) (CTRP:660823)',
'JW-480 (CTRP:660227)',
'JW-55 (CTRP:682993)',
'JW-74 (CTRP:676728)',
'KH-CB19 (CTRP:660085)',
'KHS101 (CTRP:660819)',
'KI8751 (CTRP:375560)',
'KIN001-167 (CTRP:606144)',
'KO-143 (CTRP:418172)',
'KPT185 (CTRP:660935)',
'KU 0060648 (CTRP:632119)',
'KU-0063794 (CTRP:606363)',
'KU-55933 (CTRP:58339)',
'KU-60019 (CTRP:660411)',
'KW-2449 (CTRP:660413)',
'KX2-391 (CTRP:660779)',
'L-685458 (CTRP:411717)',
'LAPATINIB (CTRP:634309)',
'LBH-589 (CTRP:54210)',
'LE-135 (CTRP:375354)',
'LENVATINIB (CTRP:628639)',
'LEPTOMYCIN B (CTRP:660086)',
'MG-132 (CTRP:58948)',
'LOMEGUATRIB (CTRP:660207)',
'LOVASTATIN (CTRP:30620)',
'LRRK2-IN-1 (CTRP:607696)',
'LY-2157299 (CTRP:660431)',
'LY-2183240 (CTRP:375496)',
'TEMOZOLOMIDE (CTRP:351043)',
'MANUMYCIN A (CTRP:50727)',
'MARINOPYRROLE A (CTRP:666910)',
'MDIVI-1 (CTRP:215497)',
'MERCK60 (CTRP:61674)',
'METHYLSTAT (CTRP:667933)',
'MGCD-265 (CTRP:628622)',
'MI-1 (CTRP:185987)',
'MI-2 (CTRP:687714)',
'MIRA 1 (CTRP:580922)',
'MITOMYCIN (CTRP:50732)',

'MK-2206 (CTRP:377381)',
 'VORINOSTAT (CTRP:56554)',
 'MK-0752 (CTRP:660332)',
 'ML006 (CTRP:17712)',
 'ML029 (CTRP:350548)',
 'ML031 (CTRP:96171)',
 'ML050 (CTRP:153820)',
 'ML083 (CTRP:255961)',
 'ML162 (CTRP:154846)',
 'ML203 (CTRP:411816)',
 'ML210 (CTRP:609110)',
 'ML239 (CTRP:609781)',
 'ML258 (CTRP:660314)',
 'ML311 (CTRP:290356)',
 'ML312 (CTRP:434195)',
 'ML320 (CTRP:659682)',
 'ML334 DIASTEREOMER (CTRP:654278)',
 'MLN2238 (CTRP:632873)',
 'TOVORAFENIB (CTRP:687579)',
 'MOMELLOTINIB (CTRP:660429)',
 'MST 312 (CTRP:411865)',
 'MYRICETIN (CTRP:29985)',
 'MYRIOCIN (CTRP:417977)',
 'N9-ISOPROPYLOLOMOUCINE (CTRP:45093)',
 'NAKITERPIOSIN (CTRP:417452)',
 'NARCICLASINE (CTRP:632243)',
 'NAVITOCCLAX (CTRP:362342)',
 'NAVITOCCLAX:BIRINAPANT (1:1 MOL/MOL) (CTRP:660172)',
 'NAVITOCCLAX:GEMCITABINE (1:1 MOL/MOL) (CTRP:660225)',
 'NAVITOCCLAX:MST-312 (1:1 MOL/MOL) (CTRP:660222)',
 'NAVITOCCLAX:PIPERLONGUMINE (1:1 MOL/MOL) (CTRP:660284)',
 'NAVITOCCLAX:PLURIPOTIN (1:1 MOL/MOL) (CTRP:660246)',
 'NAVITOCCLAX:PLX-4032 (1:1 MOL/MOL) (CTRP:660219)',
 'NAVITOCCLAX:SORAFENIB (1:1 MOL/MOL) (CTRP:660283)',
 'NECROSTATIN-1 (CTRP:375435)',
 'NECROSTATIN-7 (CTRP:660201)',
 'NECROSULFONAMIDE (CTRP:677522)',
 'NELARABINE (CTRP:660391)',
 'NEOPELTOLIDE (CTRP:411774)',
 'NERATINIB (CTRP:418038)',
 'NEURONAL DIFFERENTIATION INDUCER III (CTRP:411733)',
 'NICLOSAMIDE (CTRP:32653)',
 'NILOTINIB (CTRP:374749)',
 'NINTEDANIB (CTRP:628601)',
 'NPC-26 (CTRP:609058)',
 'NSC 74859 (CTRP:411734)',
 'NSC23766 (CTRP:44503)',

'NSC30930 (CTRP:182395)',
 'NSC48300 (CTRP:594399)',
 'NSC632839 (CTRP:375594)',
 'NSC95397 (CTRP:50715)',
 'REBEMADLIN (CTRP:411722)',
 'NVP ADW742 (CTRP:660287)',
 'NVP-231 (CTRP:660331)',
 'NVP-BSK805 (CTRP:660434)',
 'NVP-TAE 684 (CTRP:606249)',
 'O-6-BENZYLGUANINE (CTRP:345712)',
 'OLAPARIB (CTRP:411867)',
 'OLIGOMYCIN A (CTRP:411869)',
 'HOMOHARRINGTONINE (CTRP:375219)',
 'OSI 930 OSI930 (CTRP:628620)',
 'OUABAIN (CTRP:37190)',
 'PACLITAXEL (CTRP:26956)',
 'PALMOSTATIN B (CTRP:679844)',
 'PANDACOSTAT (CTRP:417098)',
 'PARBENDAZOLE (CTRP:28172)',
 'PARTHENOLIDE (CTRP:411792)',
 'PAZOPANIB (CTRP:640265)',
 'PD 153035 (CTRP:52133)',
 'PD318088 (CTRP:660410)',
 'PDMP (CTRP:58472)',
 'PEVONEDISTAT (CTRP:411809)',
 'PF-184 (CTRP:660208)',
 'PF-3758309 (CTRP:687953)',
 'PF-4800567 HYDROCHLORIDE (CTRP:688962)',
 'PF-543 (CTRP:660073)',
 'PF-573228 (CTRP:660075)',
 'PF-750 (CTRP:411719)',
 'PHA 793887 (CTRP:640007)',
 'PHLORETIN (CTRP:24173)',
 'PI 103 (CTRP:411742)',
 'PIFITHRIN-ALPHA (CTRP:50747)',
 'PIFITHRIN-MU (CTRP:375596)',
 'PIK 93 (CTRP:660774)',
 'PIPERLONGUMINE (CTRP:39782)',
 'PIPERLONGUMINE:MST-312 (1:1 MOL/MOL) (CTRP:660224)',
 'PITSTOP2 (CTRP:660985)',
 'PL-DI (CTRP:609440)',
 'PLATIN (CTRP:375395)',
 'PLURIPOTIN (CTRP:411833)',
 'PLX 4720 (CTRP:606256)',
 'PLX-4032 (CTRP:649420)',
 'PRIMA-1 (CTRP:50131)',
 'PRIMA-1-MET (CTRP:594663)',

'PRL-3 INHIBITOR I (CTRP:411727)',
 'PROCARBAZINE (CTRP:28183)',
 'PROCHLORPERAZINE (CTRP:33166)',
 'PURMORPHAMINE (CTRP:52576)',
 'PX-12 (CTRP:609124)',
 'PYR-41 (CTRP:660202)',
 'PYRAZOLANTHRONE (CTRP:44511)',
 'QS-11 (CTRP:609681)',
 'QW-BI-011 (CTRP:362063)',
 'R428 (CTRP:660993)',
 'RAF265 (CTRP:660414)',
 'REGORAFENIB (CTRP:628613)',
 'REPLIGEN 136 (CTRP:628408)',
 'RG-108 (CTRP:411831)',
 'RIGOSERTIB (CTRP:660397)',
 'RITA (CTRP:375492)',
 'RO4929097 (CTRP:687577)',
 'RUXOLITINIB (CTRP:639450)',
 'SALERMIDE (CTRP:616352)',
 'SALERMIDE:PLX-4032 (12:1 MOL/MOL) (CTRP:660269)',
 'SB-225002 (CTRP:52123)',
 'SB-431542 (CTRP:50753)',
 'SB-525334 (CTRP:639962)',
 'SB-743921 (CTRP:660238)',
 'SCH-529074 (CTRP:609062)',
 'SCH-79797 (CTRP:375264)',
 'SELUMETINIB:BRD-A02303741 (4:1 MOL/MOL) (CTRP:660178)',
 'SELUMETINIB:DECITABINE (4:1 MOL/MOL) (CTRP:660249)',
 'SELUMETINIB:GDC-0941 (4:1 MOL/MOL) (CTRP:660174)',
 'SELUMETINIB:JQ-1 (4:1 MOL/MOL) (CTRP:660177)',
 'SELUMETINIB:MK-2206 (8:1 MOL/MOL) (CTRP:660218)',
 'SELUMETINIB:NAVITOCCLAX (8:1 MOL/MOL) (CTRP:660248)',
 'SELUMETINIB:PIPERLONGUMINE (8:1 MOL/MOL) (CTRP:660220)',
 'SELUMETINIB:PLX-4032 (8:1 MOL/MOL) (CTRP:660133)',
 'SELUMETINIB:TRETINOIN (2:1 MOL/MOL) (CTRP:660175)',
 'SELUMETINIB:UNC0638 (4:1 MOL/MOL) (CTRP:660281)',
 'SELUMETINIB:VORINOSTAT (8:1 MOL/MOL) (CTRP:660176)',
 'SEMAGACESTAT (CTRP:660341)',
 'SEPANTRONIUM BROMIDE (CTRP:417979)',
 'SERDEMETAN: SCH-529074 (1:1 MOL/MOL) (CTRP:660132)',
 'SGX-523 (CTRP:635882)',
 'SID 26681509 (CTRP:411731)',
 'SILDENAFIL (CTRP:27872)',
 'SILMITASERTIB (CTRP:660421)',
 'SIMVASTATIN (CTRP:27894)',
 'SIROLIMUS: BORTEZOMIB (250:1 MOL/MOL) (CTRP:660134)',
 'SITAGLIPTIN (CTRP:58442)',

'SJ-172550 (CTRP:416777)',
'SKEPINONE-L (CTRP:688799)',
'SKI-II (CTRP:375366)',
'SMER-3 (CTRP:660286)',
'SN38 (CTRP:375637)',
'SNS-032 (CTRP:606254)',
'SNX 2112 (CTRP:362338)',
'SNX-2112:BORTEZOMIB (250:1 MOL/MOL) (CTRP:660251)',
'SOTRASTAUIN (CTRP:661032)',
'SPAUTIN-1 (CTRP:657183)',
'SR-II-138A (CTRP:411770)',
'SR1001 (CTRP:660317)',
'SR8278 (CTRP:668858)',
'SRT-1720 (CTRP:411820)',
'STAUROSPORINE (CTRP:50769)',
'STEMREGENIN 1 (CTRP:609063)',
'STF-31 (CTRP:652975)',
'SUNITINIB (CTRP:374750)',
'SU-11274 (CTRP:606586)',
'SZ4TA2 (CTRP:688138)',
'TACEDINALINE (CTRP:54200)',
'TACROLIMUS (CTRP:50737)',
'TAMATINIB (CTRP:417818)',
'TANDUTINIB (CTRP:464458)',
'TANESPIMYCIN:BORTEZOMIB (250:1 MOL/MOL) (CTRP:660221)',
'TANESPIMYCIN:GEMCITABINE (1:1 MOL/MOL) (CTRP:660216)',
'TENIPOSIDE (CTRP:27871)',
'TG-100-115 (CTRP:606470)',
'FEDRATINIB (CTRP:411808)',
'TGX 221 (CTRP:606356)',
'THALIDOMIDE (CTRP:24197)',
'TIGECYCLINE (CTRP:659993)',
'TIPIFARNIB-P1 (CTRP:417417)',
'TIPIFARNIB-P2 (CTRP:417416)',
'TIVANTINIB (CTRP:687406)',
'TOPOTECAN (CTRP:44580)',
'TOSEDOSTAT (CTRP:417987)',
'TPCA-1 (CTRP:375564)',
'TRAMETINIB (CTRP:687418)',
'TRETINOIN:CARBOPLATIN (2:1 MOL/MOL) (CTRP:660181)',
'TRETINOIN:NAVITOCCLAX (4:1 MOL/MOL) (CTRP:660179)',
'TRIAZOLOTHIADIAZINE (CTRP:385240)',
'TRIFLUOPERAZINE (CTRP:26914)',
'TRIPTOLIDE (CTRP:411720)',
'TUBASTATIN A (CTRP:411749)',
'TW 37 (CTRP:609596)',
'UNC0638 (CTRP:594665)',

```
'UNC0321 (CTRP:635576)',
'UNC0638:NAVITOCCLAX (1:1 MOL/MOL) (CTRP:660270)',
'VAF-347 (CTRP:659226)',
'VALDECOXIB (CTRP:32372)',
'VANDETANIB (CTRP:349002)',
'VENETOCLAX (CTRP:666541)',
'VER-155008 (CTRP:411815)',
'VINCRISTINE (CTRP:62602)',
'VORAPAXAR (CTRP:660136)',
'VORINOSTAT:CARBOPLATIN (1:1 MOL/MOL) (CTRP:660131)',
'VORINOSTAT:NAVITOCCLAX (4:1 MOL/MOL) (CTRP:660301)',
'VU0155056 (CTRP:660119)',
'VX-680 (CTRP:340501)',
'WAY-362450 (CTRP:660821)',
'WP1130 (CTRP:660432)',
'WZ4002 (CTRP:640366)',
'WZ8040 (CTRP:628614)',
'XL765 (CTRP:660304)',
'YK 4-279 (CTRP:632103)',
'YL54 (CTRP:651298)',
'ZEBULARINE (CTRP:411739)',
'SIROLIMUS (CTRP:56703)',
'TEMSIROLIMUS (CTRP:660358)']
```

[434]: `ctrpv2_glioma.columns.tolist()`

```
[434]: ['16-BETA-BROMOANDROSTERONE (CTRP:411726)',
'1S,3R-RSL-3 (CTRP:609060)',
'3-CL-AHPC (CTRP:660796)',
'968 (CTRP:660080)',
'A-804598 (CTRP:661033)',
'AA-COCF3 (CTRP:411724)',
'ABIRATERONE (CTRP:660363)',
'ABT-199 (CTRP:666541)',
'ABT-737 (CTRP:411738)',
'AC55649 (CTRP:375487)',
'AFATINIB (CTRP:606135)',
'AGK-2 (CTRP:411716)',
'ALISERTIB (CTRP:636711)',
'ALVOCIDIB (CTRP:687720)',
'AM-580 (CTRP:58455)',
'APICIDIN (CTRP:61097)',
'AT-406 (CTRP:710154)',
'AT13387 (CTRP:688229)',
'AT7867 (CTRP:660409)',
'AUSTOCYSTIN D (CTRP:416775)',
'AVICIN D (CTRP:688975)']
```

'AVRAINVILLAMIDE (CTRP:411856)',
 'AXITINIB (CTRP:348990)',
 'AZ-3146 (CTRP:660087)',
 'AZACITIDINE (CTRP:26874)',
 'AZD1480 (CTRP:660306)',
 'AZD4547 (CTRP:660325)',
 'AZD6482 (CTRP:639390)',
 'AZD7545 (CTRP:411814)',
 'AZD7762 (CTRP:660777)',
 'AZD8055 (CTRP:609639)',
 'BO2 (CTRP:122255)',
 'BAFILOMYCIN A1 (CTRP:467037)',
 'BARASERTIB (CTRP:601923)',
 'BARDOXOLONE METHYL (CTRP:660318)',
 'BAX CHANNEL BLOCKER (CTRP:18311)',
 'BCL-LZH-4 (CTRP:688516)',
 'BEC (CTRP:60679)',
 'BELINOSTAT (CTRP:60225)',
 'BENDAMUSTINE (CTRP:374755)',
 'BETULINIC ACID (CTRP:23256)',
 'BEXAROTENE (CTRP:374747)',
 'BI-2536 (CTRP:347813)',
 'BIBR-1532 (CTRP:417982)',
 'BIRB-796 (CTRP:660412)',
 'BIRINAPANT (CTRP:660778)',
 'BIX-01294 (CTRP:53705)',
 'BLEBBISTATIN (CTRP:50140)',
 'BLEOMYCIN A2 (CTRP:639531)',
 'BMS-195614 (CTRP:660228)',
 'BMS-270394 (CTRP:660321)',
 'BMS-345541 (CTRP:622911)',
 'BMS-536924 (CTRP:411859)',
 'BMS-754807 (CTRP:417981)',
 'BORTEZOMIB (CTRP:411737)',
 'BOSUTINIB (CTRP:660346)',
 'BRD-A02303741 (CTRP:659371)',
 'BRD-A05715709 (CTRP:660342)',
 'BRD-A71883111 (CTRP:660145)',
 'BRD-A86708339 (CTRP:687769)',
 'BRD-A94377914 (CTRP:61724)',
 'BRD-K01737880 (CTRP:438345)',
 'BRD-K02251932 (CTRP:442141)',
 'BRD-K02492147 (CTRP:415688)',
 'BRD-K04800985 (CTRP:435086)',
 'BRD-K09344309 (CTRP:439882)',
 'BRD-K09587429 (CTRP:416842)',
 'BRD-K11533227 (CTRP:389297)',

'BRD-K13999467 (CTRP:410270)' ,
'BRD-K14844214 (CTRP:402775)' ,
'BRD-K16147474 (CTRP:447732)' ,
'BRD-K17060750 (CTRP:600054)' ,
'BRD-K19103580 (CTRP:436211)' ,
'BRD-K24690302 (CTRP:641721)' ,
'BRD-K26531177 (CTRP:42319)' ,
'BRD-K27224038 (CTRP:397964)' ,
'BRD-K27986637 (CTRP:452570)' ,
'BRD-K28456706 (CTRP:651642)' ,
'BRD-K29086754 (CTRP:448891)' ,
'BRD-K29313308 (CTRP:634814)' ,
'BRD-K30019337 (CTRP:437263)' ,
'BRD-K30748066 (CTRP:689506)' ,
'BRD-K33199242 (CTRP:449707)' ,
'BRD-K33514849 (CTRP:401251)' ,
'BRD-K34099515 (CTRP:660064)' ,
'BRD-K34222889 (CTRP:688888)' ,
'BRD-K34485477 (CTRP:446716)' ,
'BRD-K35604418 (CTRP:660230)' ,
'BRD-K37390332 (CTRP:463198)' ,
'BRD-K41334119 (CTRP:446546)' ,
'BRD-K41597374 (CTRP:404452)' ,
'BRD-K42260513 (CTRP:687715)' ,
'BRD-K44224150 (CTRP:438564)' ,
'BRD-K45681478 (CTRP:142931)' ,
'BRD-K48334597 (CTRP:431987)' ,
'BRD-K48477130 (CTRP:649754)' ,
'BRD-K49290616 (CTRP:414479)' ,
'BRD-K50799972 (CTRP:596868)' ,
'BRD-K51490254 (CTRP:616355)' ,
'BRD-K51831558 (CTRP:687661)' ,
'BRD-K52037352 (CTRP:450415)' ,
'BRD-K55116708 (CTRP:444779)' ,
'BRD-K61166597 (CTRP:348633)' ,
'BRD-K63431240 (CTRP:404566)' ,
'BRD-K64610608 (CTRP:438776)' ,
'BRD-K66453893 (CTRP:387472)' ,
'BRD-K66532283 (CTRP:376693)' ,
'BRD-K70511574 (CTRP:648987)' ,
'BRD-K71781559 (CTRP:425733)' ,
'BRD-K71935468 (CTRP:58566)' ,
'BRD-K75293299 (CTRP:438691)' ,
'BRD-K78574327 (CTRP:435898)' ,
'BRD-K80183349 (CTRP:376307)' ,
'BRD-K84807411 (CTRP:437906)' ,
'BRD-K85133207 (CTRP:616408)' ,

'BRD-K86535717 (CTRP:429535)',
'BRD-K88742110 (CTRP:606670)',
'BRD-K90370028 (CTRP:362207)',
'BRD-K92856060 (CTRP:119769)',
'BRD-K94991378 (CTRP:53556)',
'BRD-K96431673 (CTRP:417262)',
'BRD-K96970199 (CTRP:410703)',
'BRD-K97651142 (CTRP:660135)',
'BRD-K99006945 (CTRP:687395)',
'BRD-M00053801 (CTRP:660890)',
'BRD1812 (CTRP:418162)',
'BRD1835 (CTRP:404070)',
'BRD4132 (CTRP:3588)',
'BRD6340 (CTRP:12877)',
'BRD8899 (CTRP:608062)',
'BRD8958 (CTRP:418163)',
'BRD9647 (CTRP:37479)',
'BRD9876 (CTRP:19153)',
'BREFELDIN A (CTRP:411714)',
'BRIVANIB (CTRP:606246)',
'BYL-719 (CTRP:668495)',
'C6-CERAMIDE (CTRP:44554)',
'CABOZANTINIB (CTRP:660840)',
'CAL-101 (CTRP:660430)',
'CANERTINIB (CTRP:606138)',
'CAY10576 (CTRP:622913)',
'CAY10594 (CTRP:411824)',
'CAY10618 (CTRP:632104)',
'CBB-1007 (CTRP:687721)',
'CCT036477 (CTRP:417663)',
'CD-1530 (CTRP:375562)',
'CD-437 (CTRP:375256)',
'CEDIRANIB (CTRP:606136)',
'CERANIB-2 (CTRP:660078)',
'CERULENIN (CTRP:52430)',
'CH-55 (CTRP:660137)',
'CHIR-99021 (CTRP:347812)',
'CHLORAMBUCIL (CTRP:25334)',
'CHM-1 (CTRP:609674)',
'CI-976 (CTRP:375390)',
'CICLOPIROX (CTRP:28784)',
'CICLOSPORIN (CTRP:55512)',
'CID-5951923 (CTRP:267119)',
'CIL41 (CTRP:176181)',
'CIL55 (CTRP:1788)',
'CIL55A (CTRP:609061)',
'CIL56 (CTRP:100490)',

'CIL70 (CTRP:183526)',
 'CIMETIDINE (CTRP:26870)',
 'CLOFARABINE (CTRP:660288)',
 'COL-3 (CTRP:665914)',
 'COMPOUND 1541A (CTRP:411827)',
 'COMPOUND 1B (CTRP:62111)',
 'COMPOUND 23 CITRATE (CTRP:417451)',
 'COMPOUND 7D-CIS (CTRP:411805)',
 'CR-1-31B (CTRP:418209)',
 'CRIZOTINIB (CTRP:628603)',
 'CUCURBITACIN I (CTRP:595102)',
 'CURCUMIN (CTRP:28452)',
 'CYANOQUINOLINE 11 (CTRP:417415)',
 'CYCLOPHOSPHAMIDE (CTRP:32620)',
 'CYTARABINE HYDROCHLORIDE (CTRP:62690)',
 'CYTOCHALASIN B (CTRP:50163)',
 'DABRAFENIB (CTRP:687954)',
 'DACARBAZINE (CTRP:32622)',
 'DAPORINAD (CTRP:418166)',
 'DARINAPARSIN (CTRP:609121)',
 'DASATINIB (CTRP:52882)',
 'DBEQ (CTRP:199337)',
 'DECITABINE (CTRP:347775)',
 'DEXAMETHASONE (CTRP:277354)',
 'DINACICLIB (CTRP:687578)',
 'DOCETAXEL (CTRP:660364)',
 'DOXORUBICIN (CTRP:36599)',
 'ELCPK (CTRP:609056)',
 'ELOCALCITOL (CTRP:660319)',
 'ENTINOSTAT (CTRP:60218)',
 'EPIGALLOCATECHIN-3-MONOGALLATE (CTRP:28801)',
 'ERASTIN (CTRP:63578)',
 'ERISMODEGIB (CTRP:636425)',
 'ERLOTINIB (CTRP:52928)',
 'ETOMOXIR (CTRP:660082)',
 'ETOPOSIDE (CTRP:48589)',
 'ETP-46464 (CTRP:666716)',
 'EX-527 (CTRP:608999)',
 'FGIN-1-27 (CTRP:61775)',
 'FINGOLIMOD (CTRP:640157)',
 'FLUOROURACIL (CTRP:25344)',
 'FLUVASTATIN (CTRP:660322)',
 'FORETINIB (CTRP:628607)',
 'FQI-1 (CTRP:102193)',
 'FQI-2 (CTRP:609091)',
 'FSC231 (CTRP:660079)',
 'FULVESTRANT (CTRP:464880)',

'FUMONISIN B1 (CTRP:616353)',
'GANT-61 (CTRP:609332)',
'GDC-0879 (CTRP:606248)',
'GDC-0941 (CTRP:639759)',
'GEFITINIB (CTRP:52926)',
'GEMCITABINE (CTRP:411863)',
'GMX-1778 (CTRP:411843)',
'GOSSYPOL (CTRP:25036)',
'GSK-3 INHIBITOR IX (CTRP:52582)',
'GSK-J4 (CTRP:687735)',
'GSK1059615 (CTRP:606471)',
'GSK2636771 (CTRP:668723)',
'GSK4112 (CTRP:660081)',
'GSK461364 (CTRP:649862)',
'GSK525762A (CTRP:660245)',
'GW-405833 (CTRP:375447)',
'GW-843682X (CTRP:348997)',
'HBX-41108 (CTRP:679843)',
'HC-067047 (CTRP:660083)',
'HLI 373 (CTRP:481311)',
'HYPERFORIN (CTRP:660327)',
'I-BET151 (CTRP:660206)',
'IBRUTINIB (CTRP:660994)',
'IC-87114 (CTRP:55307)',
'IFOSFAMIDE (CTRP:35319)',
'IMATINIB (CTRP:345041)',
'IMPORTAZOLE (CTRP:46792)',
'INDISULAM (CTRP:411874)',
'IPR-456 (CTRP:660084)',
'ISOEVODIAMINE (CTRP:594664)',
'ISOLIQURITIGENIN (CTRP:25393)',
'ISONICOTINOHYDROXAMIC ACID (CTRP:478694)',
'ISOX (CTRP:632907)',
'ISTRADEFYLLINE (CTRP:687694)',
'ITRACONAZOLE (CTRP:63349)',
'IU1 (CTRP:272805)',
'JQ-1 (CTRP:616354)',
'JW-480 (CTRP:660227)',
'JW-55 (CTRP:682993)',
'JW-74 (CTRP:676728)',
'KH-CB19 (CTRP:660085)',
'KHS101 (CTRP:660819)',
'KI8751 (CTRP:375560)',
'KO-143 (CTRP:418172)',
'KPT185 (CTRP:660935)',
'KU 0060648 (CTRP:632119)',
'KU-0063794 (CTRP:606363)',

'KU-55933 (CTRP:58339)',
'KU-60019 (CTRP:660411)',
'KW-2449 (CTRP:660413)',
'KX2-391 (CTRP:660779)',
'L-685458 (CTRP:411717)',
'LAPATINIB (CTRP:634309)',
'LBH-589 (CTRP:54210)',
'LE-135 (CTRP:375354)',
'LENVATINIB (CTRP:628639)',
'LEPTOMYCIN B (CTRP:660086)',
'LINIFANIB (CTRP:606033)',
'LINSITINIB (CTRP:705300)',
'LOMEGUATRIB (CTRP:660207)',
'LOVASTATIN (CTRP:30620)',
'LRRK2-IN-1 (CTRP:607696)',
'LY-2157299 (CTRP:660431)',
'LY-2183240 (CTRP:375496)',
'MANUMYCIN A (CTRP:50727)',
'MARINOPYRROLE A (CTRP:666910)',
'MASITINIB (CTRP:606143)',
'MDIVI-1 (CTRP:215497)',
'MERCK60 (CTRP:61674)',
'METHOTREXATE (CTRP:30371)',
'METHYLSTAT (CTRP:667933)',
'MG-132 (CTRP:58948)',
'MGCD-265 (CTRP:628622)',
'MI-1 (CTRP:185987)',
'MI-2 (CTRP:687714)',
'MITOMYCIN (CTRP:50732)',
'MK-0752 (CTRP:660332)',
'MK-1775 (CTRP:418167)',
'MK-2206 (CTRP:377381)',
'ML006 (CTRP:17712)',
'ML029 (CTRP:350548)',
'ML031 (CTRP:96171)',
'ML050 (CTRP:153820)',
'ML083 (CTRP:255961)',
'ML162 (CTRP:154846)',
'ML203 (CTRP:411816)',
'ML210 (CTRP:609110)',
'ML239 (CTRP:609781)',
'ML258 (CTRP:660314)',
'ML311 (CTRP:290356)',
'ML312 (CTRP:434195)',
'ML320 (CTRP:659682)',
'ML334 DIASTEREOMER (CTRP:654278)',
'MLN2238 (CTRP:632873)',

'MLN2480 (CTRP:687579)',
'MOMELOTINIB (CTRP:660429)',
'MST-312 (CTRP:411865)',
'MYRICETIN (CTRP:29985)',
'MYRIOCIN (CTRP:417977)',
'N9-ISOPROPYLOLOMOUCINE (CTRP:45093)',
'NAKITERPIOSIN (CTRP:417452)',
'NARCICLASINE (CTRP:632243)',
'NAVITOCCLAX (CTRP:362342)',
'NECROSTATIN-1 (CTRP:375435)',
'NECROSTATIN-7 (CTRP:660201)',
'NECROSULFONAMIDE (CTRP:677522)',
'NELARABINE (CTRP:660391)',
'NEOPELTOLIDE (CTRP:411774)',
'NERATINIB (CTRP:418038)',
'NEURONAL DIFFERENTIATION INDUCER III (CTRP:411733)',
'NICLOSAMIDE (CTRP:32653)',
'NILOTINIB (CTRP:374749)',
'NINTEDANIB (CTRP:628601)',
'NPC-26 (CTRP:609058)',
'NSC 74859 (CTRP:411734)',
'NSC19630 (CTRP:580922)',
'NSC23766 (CTRP:44503)',
'NSC30930 (CTRP:182395)',
'NSC48300 (CTRP:594399)',
'NSC632839 (CTRP:375594)',
'NSC95397 (CTRP:50715)',
'NUTLIN-3 (CTRP:411722)',
'NVP-231 (CTRP:660331)',
'NVP-ADW742 (CTRP:660287)',
'NVP-BEZ235 (CTRP:348992)',
'NVP-BSK805 (CTRP:660434)',
'NVP-TAE684 (CTRP:606249)',
'O-6-BENZYLGUANINE (CTRP:345712)',
'OBATOCLAX (CTRP:606142)',
'OLAPARIB (CTRP:411867)',
'OLIGOMYCIN A (CTRP:411869)',
'OMACETAXINE MEPESUCCINATE (CTRP:375219)',
'OSI-027 (CTRP:660433)',
'OSI-930 (CTRP:628620)',
'OUABAIN (CTRP:37190)',
'PAC-1 (CTRP:411832)',
'PACLITAXEL (CTRP:26956)',
'PALMOSTATIN B (CTRP:679844)',
'PANDACOSTAT (CTRP:417098)',
'PARBENDAZOLE (CTRP:28172)',
'PARTHENOLIDE (CTRP:411792)',

'PAZOPANIB (CTRP:640265)',
'PD 153035 (CTRP:52133)',
'PD318088 (CTRP:660410)',
'PDMP (CTRP:58472)',
'PEVONEDISTAT (CTRP:411809)',
'PF-184 (CTRP:660208)',
'PF-3758309 (CTRP:687953)',
'PF-4800567 HYDROCHLORIDE (CTRP:688962)',
'PF-543 (CTRP:660073)',
'PF-573228 (CTRP:660075)',
'PF-750 (CTRP:411719)',
'PHA-793887 (CTRP:640007)',
'PHLORETIN (CTRP:24173)',
'PI-103 (CTRP:411742)',
'PIFITHRIN-ALPHA (CTRP:50747)',
'PIFITHRIN-MU (CTRP:375596)',
'PIK-93 (CTRP:660774)',
'PIPERLONGUMINE (CTRP:39782)',
'PITSTOP2 (CTRP:660985)',
'PL-DI (CTRP:609440)',
'PLURIPOTIN (CTRP:411833)',
'PLX-4032 (CTRP:649420)',
'PLX-4720 (CTRP:606256)',
'PRIMA-1 (CTRP:50131)',
'PRIMA-1-MET (CTRP:594663)',
'PRL-3 INHIBITOR I (CTRP:411727)',
'PROCARBAZINE (CTRP:28183)',
'PROCHLORPERAZINE (CTRP:33166)',
'PURMORPHAMINE (CTRP:52576)',
'PX-12 (CTRP:609124)',
'PYR-41 (CTRP:660202)',
'PYRAZOLANTHRONE (CTRP:44511)',
'QS-11 (CTRP:609681)',
'QUIZARTINIB (CTRP:640011)',
'QW-BI-011 (CTRP:362063)',
'R428 (CTRP:660993)',
'RAF265 (CTRP:660414)',
'REGORAFENIB (CTRP:628613)',
'REPLIGEN 136 (CTRP:628408)',
'RG-108 (CTRP:411831)',
'RIGOSERTIB (CTRP:660397)',
'RITA (CTRP:375492)',
'R04929097 (CTRP:687577)',
'RUXOLITINIB (CTRP:639450)',
'SALERMIDE (CTRP:616352)',
'SARACATINIB (CTRP:606035)',
'SB-225002 (CTRP:52123)',

'SB-431542 (CTRP:50753)',
'SB-525334 (CTRP:639962)',
'SB-743921 (CTRP:660238)',
'SCH-529074 (CTRP:609062)',
'SCH-79797 (CTRP:375264)',
'SELUMETINIB (CTRP:348991)',
'SEMAGACESTAT (CTRP:660341)',
'SERDEMETAN (CTRP:418164)',
'SGX-523 (CTRP:635882)',
'SID 26681509 (CTRP:411731)',
'SILDENAFIL (CTRP:27872)',
'SILMITASERTIB (CTRP:660421)',
'SIMVASTATIN (CTRP:27894)',
'SIROLIMUS (CTRP:56703)',
'SITAGLIPTIN (CTRP:58442)',
'SJ-172550 (CTRP:416777)',
'SKEPINONE-L (CTRP:688799)',
'SKI-II (CTRP:375366)',
'SMER-3 (CTRP:660286)',
'SN-38 (CTRP:375637)',
'SNS-032 (CTRP:606254)',
'SNX-2112 (CTRP:362338)',
'SORAFENIB (CTRP:349006)',
'SOTRASTAUIN (CTRP:661032)',
'SPAUTIN-1 (CTRP:657183)',
'SR-II-138A (CTRP:411770)',
'SR1001 (CTRP:660317)',
'SR8278 (CTRP:668858)',
'SRT-1720 (CTRP:411820)',
'STAUROSPORINE (CTRP:50769)',
'STEMREGENIN 1 (CTRP:609063)',
'STF-31 (CTRP:652975)',
'SU11274 (CTRP:606586)',
'SUNITINIB (CTRP:374750)',
'SZ4TA2 (CTRP:688138)',
'TACEDINALINE (CTRP:54200)',
'TACROLIMUS (CTRP:50737)',
'TAMATINIB (CTRP:417818)',
'TAMOXIFEN (CTRP:26972)',
'TANDUTINIB (CTRP:464458)',
'TANESPIMYCIN (CTRP:50134)',
'TEMOZOLOMIDE (CTRP:351043)',
'TEMSIROLIMUS (CTRP:660358)',
'TENIPOSIDE (CTRP:27871)',
'TG-100-115 (CTRP:606470)',
'TG-101348 (CTRP:411808)',
'TGX-221 (CTRP:606356)',

```
'THALIDOMIDE (CTRP:24197)',
'TIGECYCLINE (CTRP:659993)',
'TIPIFARNIB-P1 (CTRP:417417)',
'TIPIFARNIB-P2 (CTRP:417416)',
'TIVANTINIB (CTRP:687406)',
'TIVOZANIB (CTRP:418170)',
'TOPOTECAN (CTRP:44580)',
'TOSEDOSTAT (CTRP:417987)',
'TPCA-1 (CTRP:375564)',
'TRAMETINIB (CTRP:687418)',
'TRETINOIN (CTRP:23151)',
'TRIAZOLOTHIADIAZINE (CTRP:385240)',
'TRIFLUOPERAZINE (CTRP:26914)',
'TRIPTOLIDE (CTRP:411720)',
'TUBASTATIN A (CTRP:411749)',
'TW-37 (CTRP:609596)',
'UNC0321 (CTRP:635576)',
'UNC0638 (CTRP:594665)',
'VAF-347 (CTRP:659226)',
'VALDECOXIB (CTRP:32372)',
'VANDETANIB (CTRP:349002)',
'VELIPARIB (CTRP:606034)',
'VER-155008 (CTRP:411815)',
'VINCRISTINE (CTRP:62602)',
'VORAPAXAR (CTRP:660136)',
'VORINOSTAT (CTRP:56554)',
'VU0155056 (CTRP:660119)',
'VX-680 (CTRP:340501)',
'WAY-362450 (CTRP:660821)',
'WP1130 (CTRP:660432)',
'WZ4002 (CTRP:640366)',
'WZ8040 (CTRP:628614)',
'XL765 (CTRP:660304)',
'YK 4-279 (CTRP:632103)',
'YM-155 (CTRP:417979)',
'ZEBULARINE (CTRP:411739)',
'ZSTK474 (CTRP:606144)']
```

```
[435]: gdsc1_glioma.columns.tolist()
```

```
[435]: ['(+)-JQ-1 (GDSC1:1218)',
'(5Z)-7-OXOZEAENOL (GDSC1:1242)',
'AZD4547 (GDSC1:1135)',
'1080622-86-1 (GDSC1:152)',
'212631-79-3 (GDSC1:1015)',
'SORAFENIB (GDSC1:30)',
'9-??-D-ARABINOFURANOSYL GUANINE (GDSC1:427)']
```


'944328-88-5 (GDSC1:151)',
'A-1065-5 (GDSC1:299)',
'A-443654 (GDSC1:86)',
'A-770041 (GDSC1:55)',
'A-83-01 (GDSC1:477)',
'AB1010 (GDSC1:292)',
'ABT 869 (GDSC1:277)',
'AC 220 (GDSC1:254)',
'AC1L1GQE (GDSC1:288)',
'ACHP (GDSC1:290)',
'ACY-1215 (GDSC1:264)',
'AFATINIB (GDSC1:1377)',
'AGI-6780 (GDSC1:562)',
'AICA RIBONUCLEOTIDE (GDSC1:1001)',
'AK-55409 (GDSC1:1066)',
'IOX2 (GDSC1:1230)',
'AKT INHIBITOR VIII (GDSC1:228)',
'AL-39256 (GDSC1:110)',
'ALECTINIB (GDSC1:281)',
'ALISERTIB (GDSC1:431)',
'AMUVATINIB (GDSC1:293)',
'ANCHUSIN (GDSC1:170)',
'AOB2221 (GDSC1:1530)',
'AOB5560 (GDSC1:1427)',
'PONATINIB (GDSC1:155)',
'APITOLISIB (GDSC1:382)',
'AR 42 (GDSC1:272)',
'AR-12 (GDSC1:167)',
'ARRY-520 (GDSC1:474)',
'SELMETINIB (GDSC1:1498)',
'AS-605240 (GDSC1:224)',
'AS601245 (GDSC1:207)',
'TIVOZANIB (GDSC1:312)',
'ASP-7487 (GDSC1:185)',
'AST-1306 (GDSC1:381)',
'AT-7519 (GDSC1:219)',
'AT7867 (GDSC1:356)',
'AUY (GDSC1:194)',
'AVAGACESTAT (GDSC1:205)',
'AZ-10353926 (GDSC1:38)',
'AZ-628 (GDSC1:29)',
'AZ20 (GDSC1:1184)',
'AZD 8186 (GDSC1:1444)',
'AZD1480 (GDSC1:1432)',
'AZD3514 (GDSC1:1382)',
'CAPIVASERTIB (GDSC1:1136)',
'AZD7762 (GDSC1:1402)',

'AZD8835 (GDSC1:1445)',
'AZD8931 (GDSC1:1416)',
'BAM7 (GDSC1:552)',
'REFAMETINIB (GDSC1:1526)',
'BAY-61-3606 (GDSC1:178)',
'BELINOSTAT (GDSC1:274)',
'BENZOYLSTAUROSPORINE (GDSC1:153)',
'BEXAROTENE (GDSC1:186)',
'BHG712 (GDSC1:295)',
'BI-2536 (GDSC1:60)',
'BIBF-1120 (GDSC1:380)',
'BICALUTAMIDE (GDSC1:150)',
'BIX 02189 (GDSC1:279)',
'BLEOMYCIN (GDSC1:190)',
'BMS 536924 (GDSC1:62)',
'BMS-345541 (GDSC1:203)',
'BMS-509744 (GDSC1:63)',
'BMS-754807 (GDSC1:184)',
'BORTEZOMIB (GDSC1:104)',
'BRIVANIB, BMS-540215 (GDSC1:376)',
'BRYOSTATIN (GDSC1:197)',
'BX-912 (GDSC1:222)',
'C-75 (GDSC1:435)',
'C225 (GDSC1:1114)',
'CABOZANTINIB (GDSC1:249)',
'CAL-101 (GDSC1:238)',
'CAP-232, TT-232, TLN-232 (GDSC1:436)',
'CAY10566 (GDSC1:416)',
'CAY10603 (GDSC1:276)',
'CD532 (GDSC1:449)',
'CDK9 INHIBITOR (GDSC1:345)',
'CERALASERTIB (GDSC1:1394)',
'CFMS RECEPTOR TYROSINE KINASE INHIBITOR (GDSC1:193)',
'CGP 60474 (GDSC1:53)',
'CGP-082996 (GDSC1:54)',
'LADUVIGLUSIB (GDSC1:1241)',
'CHLORIDINE (GDSC1:71)',
'CI-1033 (GDSC1:362)',
'CISPLATIN (GDSC1:1496)',
'CMK (GDSC1:64)',
'CP-724714 (GDSC1:255)',
'CPI-613 (GDSC1:415)',
'CRIZOTINIB (GDSC1:37)',
'CUDC 101 (GDSC1:273)',
'PIDNARULEX (GDSC1:300)',
'CYCLOPAMINE (GDSC1:17)',
'DACINOSTAT (GDSC1:200)',

'DACOMITINIB (GDSC1:363)',
'DASATINIB (GDSC1:51)',
'DBI (GDSC1:196)',
'DDE-28 (GDSC1:192)',
'DIMETHYLOXALYLGLYCINE (GDSC1:165)',
'DOXORUBICIN (GDSC1:1386)',
'EEF2K INHIBITOR, A-484954 (GDSC1:409)',
'EIF-2ALPHA INHIBITOR (GDSC1:111)',
'EKB 569 (GDSC1:282)',
'EMBELIC ACID (GDSC1:172)',
'ENTINOSTAT (GDSC1:88)',
'ENZASTAURIN (GDSC1:229)',
'EPO906 (GDSC1:201)',
'ERK INHIBITOR II (GDSC1:263)',
'ERK5-IN-1 (GDSC1:106)',
'ERLOTINIB (GDSC1:1)',
'ETOPOSIDE (GDSC1:134)',
'ETP-45835 (GDSC1:447)',
'EX 527 (GDSC1:341)',
'EX-8678 (GDSC1:283)',
'EXEL-2880 (GDSC1:308)',
'EXELBINE (GDSC1:140)',
'PANOBINOSTAT (GDSC1:438)',
'FH535 (GDSC1:173)',
'FINGOLIMOD HYDROCHLORIDE; GILENYA; TDI-132; IMUSERA; GILENIA (GDSC1:546)',
'FLAVOPIRIDOL (GDSC1:432)',
'FLUOROURACIL (GDSC1:179)',
'FMK (GDSC1:231)',
'FTI-277 (GDSC1:166)',
'FULVESTANT (GDSC1:1414)',
'GAMMA-SECRETASE INHIBITOR 1 (GDSC1:45)',
'PICTILISIB (GDSC1:1527)',
'GEMCITABINE (GDSC1:1393)',
'GENENTECH CPD 10 (GDSC1:225)',
'GNF-2 (GDSC1:52)',
'GSK 1904529A (GDSC1:202)',
'GSK 269962A (GDSC1:127)',
'GSK 650394 (GDSC1:177)',
'GSK 690693 (GDSC1:326)',
'GSK-1070916 (GDSC1:226)',
'GSK-J4 (GDSC1:1254)',
'GSK1059615 (GDSC1:374)',
'GSK319347A (GDSC1:91)',
'GSK429286A (GDSC1:230)',
'MOLIBRESIB (GDSC1:275)',
'GTPL5238 (GDSC1:175)',
'GW843682X (GDSC1:87)',

'GX15-070 (GDSC1:182)',
 'HDAC INHIBITOR XV (GDSC1:271)',
 'HG-5-113-01 (GDSC1:1142)',
 'HG-5-88-01 (GDSC1:1143)',
 'HG6-64-1 (GDSC1:159)',
 'IC-87114 (GDSC1:236)',
 'ICL1100013 (GDSC1:1266)',
 'IMATINIB (GDSC1:34)',
 'IMD-0354 (GDSC1:442)',
 'IPA 3 (GDSC1:176)',
 'ISPINESIB (GDSC1:208)',
 'JNJ38877605 (GDSC1:284)',
 'JNK INHIBITOR 9L (GDSC1:157)',
 'JQ12 (GDSC1:164)',
 'JW-7-24-1 (GDSC1:256)',
 'JW-7-52-1 (GDSC1:83)',
 'KB NB 142-70 (GDSC1:407)',
 'KIN001-042 (GDSC1:289)',
 'KIN001-112 (GDSC1:56)',
 'KIN001-123 (GDSC1:59)',
 'KIN001-167 (GDSC1:223)',
 'KIN001-201 (GDSC1:221)',
 'KIN001-236 (GDSC1:286)',
 'KIN001-244 (GDSC1:287)',
 'KIN001-266 (GDSC1:291)',
 'KOB2602 (GDSC1:563)',
 'KU-60019 (GDSC1:1185)',
 'LAPATINIB (GDSC1:119)',
 'LCL161 (GDSC1:439)',
 'LDN-193189 (GDSC1:478)',
 'LGK974 (GDSC1:573)',
 'LIMK1 INHIBITOR BMS4 (GDSC1:406)',
 'MG-132 (GDSC1:9)',
 'METAP2 INHIBITOR, A832234 (GDSC1:410)',
 'MIM 1 (GDSC1:446)',
 'MITOMYCIN-C (GDSC1:136)',
 'MK 0457 (GDSC1:32)',
 'MPS-1-IN-1 (GDSC1:294)',
 'NG-25 (GDSC1:260)',
 'NPK76-II-72-1 (GDSC1:257)',
 'NSC 83265 (GDSC1:41)',
 'NSC 87877 (GDSC1:147)',
 'NSC-207895 (GDSC1:269)',
 'NSC319726 (GDSC1:461)',
 'NVP-TAE 684 (GDSC1:35)',
 'OCTANOIC ACID (GDSC1:180)',
 'OLAPARIB (GDSC1:1495)',

'OSI 930 OSI930 (GDSC1:298)',
 'PACLITAXEL (GDSC1:11)',
 'PARTHENOLIDE (GDSC1:89)',
 'PAZOPANIB (GDSC1:199)',
 'PEMETREXED (GDSC1:428)',
 'PF-00562271 (GDSC1:158)',
 'PF-543 (GDSC1:408)',
 'PFI-1 (GDSC1:1219)',
 'PHA 665752 (GDSC1:6)',
 'PHA 793887 (GDSC1:301)',
 'PI 103 (GDSC1:302)',
 'PIK 93 (GDSC1:303)',
 'PILARALISIB (GDSC1:372)',
 'PLX 4720 (GDSC1:1371)',
 'QL-VIII-58 (GDSC1:1166)',
 'QL-X-138 (GDSC1:331)',
 'QL-XI-92 (GDSC1:329)',
 'QL-XII-47 (GDSC1:235)',
 'QL-XII-61 (GDSC1:1203)',
 'RTRAIL (GDSC1:1261)',
 'RU-SKI 43 (GDSC1:576)',
 'RUXOLITINIB (GDSC1:206)',
 'SAR245409 (GDSC1:375)',
 'SAVOLITINIB (GDSC1:1403)',
 'SB 505124 (GDSC1:476)',
 'SB 52334 (GDSC1:304)',
 'SEPANTRONIUM BROMIDE (GDSC1:268)',
 'SGC0946 (GDSC1:1264)',
 'SN38 (GDSC1:1490)',
 'SNX 2112 (GDSC1:328)',
 'STF-62247 (GDSC1:258)',
 'SUNITINIB (GDSC1:5)',
 'SU-11274 (GDSC1:285)',
 'T0901317 (GDSC1:333)',
 'TENOVIN-6 (GDSC1:342)',
 'FEDRATINIB (GDSC1:306)',
 'TGX 221 (GDSC1:94)',
 'THZ-2-102-1 (GDSC1:346)',
 'THZ-2-49 (GDSC1:344)',
 'TIPIFARNIB (GDSC1:204)',
 'TL-1-85 (GDSC1:261)',
 'TL-2-105 (GDSC1:211)',
 'TORIN 2 (GDSC1:371)',
 'TPCA-1 (GDSC1:305)',
 'TRICHOSTATIN A (GDSC1:437)',
 'TUBASTATIN A (GDSC1:265)',
 'TWS119 (GDSC1:366)',

'UNC0638 (GDSC1:1236)',
 'UNC-1215 (GDSC1:1262)',
 'UNC0642 (GDSC1:1263)',
 'VENETOCLAX (GDSC1:412)',
 'VX 702 (GDSC1:1028)',
 'VX-11E (GDSC1:262)',
 'WNT-C59 (GDSC1:574)',
 'WYE-125132 (GDSC1:317)',
 'WZ3105 (GDSC1:252)',
 'XMD 8-92 (GDSC1:1164)',
 'XMD11-85H (GDSC1:1158)',
 'XMD13-2 (GDSC1:330)',
 'XMD14-99 (GDSC1:253)',
 'XMD15-27 (GDSC1:332)',
 'Y-39983 (GDSC1:309)',
 'YM 201636 (GDSC1:310)',
 'ZD-4054 (GDSC1:266)',
 'ZG-10 (GDSC1:1161)',
 '965-D2 (GDSC1:1359)',
 '993-D2 (GDSC1:1360)',
 'AZD4877 (GDSC1:1409)',
 'AZD7969 (GDSC1:1426)',
 'BAY ACCI (GDSC1:1362)',
 'BAY AKT1 (GDSC1:1363)',
 'BAY-HDAC11_1 (GDSC1:1364)',
 'BAY-HDAC11_2 (GDSC1:1365)',
 'BAY-HDAC11_4 (GDSC1:1367)',
 'BAY-MPS-COMBO 2 (PACLITAXEL 1 UM) (GDSC1:1370)',
 'BAY-MPS-COMBO-1 (PACLITAXEL 5 UM) (GDSC1:1369)',
 'BAY-MPS1 (GDSC1:1368)',
 'BLEOMYCIN (10 UM) (GDSC1:1392)',
 'BPTES (GDSC1:1425)',
 'CCT245232 (GDSC1:1187)',
 'CCT245467 (GDSC1:1188)',
 'CRT0105446 (GDSC1:1071)',
 'CRT0105950 (GDSC1:1070)',
 'CRT0160829 (GDSC1:1183)',
 'DYRK1B_0191 (GDSC1:1407)',
 'EPHB4_9721 (GDSC1:1413)',
 'FEN1_3940 (GDSC1:1419)',
 'FGFR_0939 (GDSC1:1421)',
 'FGFR_3831 (GDSC1:1422)',
 'FS106 (GDSC1:1128)',
 'FS112 (GDSC1:1186)',
 'FY012 (GDSC1:1198)',
 'FY026 (GDSC1:1195)',
 'FY069 (GDSC1:1196)',

```
'HG-6-71-01 (GDSC1:1204)',
'I-BET-151 (GDSC1:1232)',
'I-CBP112 (GDSC1:1531)',
'IAP_7638 (GDSC1:1429)',
'IGFR_3801 (GDSC1:1430)',
'JAK1_3715 (GDSC1:1433)',
'JAK3_7406 (GDSC1:1434)',
'MCT1_6447 (GDSC1:1436)',
'MCT4_1422 (GDSC1:1437)',
'N22899-6-C1 (GDSC1:1251)',
'N23918-95-7 (GDSC1:1252)',
'N24798-49-A1 (GDSC1:1253)',
'PARP_0108 (GDSC1:1459)',
'PARP_9482 (GDSC1:1460)',
'PARP_9495 (GDSC1:1458)',
'PI3KA_4409 (GDSC1:1446)',
'PLK_6522 (GDSC1:1451)',
'RAF_9304 (GDSC1:1453)',
'SIROLIMUS (GDSC1:3)',
'TANK_1366 (GDSC1:1461)',
'THZ-1-87 (GDSC1:1145)',
'THZ-2-98-01 (GDSC1:1165)',
'TORIN2 (GDSC1:1202)',
'TTK_3146 (GDSC1:1464)',
'XMD11-50 (GDSC1:1157)',
'ZL049 (GDSC1:1197)',
'ZL109 (GDSC1:1379)']
```

```
[436]: gdsc2_glioma.columns.tolist()
#CCT-018159 (GDSC2:1170)
```

```
[436]: ['(+)-CAMPTOTHECIN (GDSC2:1003)',
'(+)-JQ-1 (GDSC2:2172)',
'AZD4547 (GDSC2:1786)',
'1044870-39-4 (GDSC2:1625)',
'TANESPIMYCIN (GDSC2:1026)',
'SORAFENIB (GDSC2:1085)',
'AZACITIDINE (GDSC2:2156)',
'5XE (GDSC2:1852)',
'681640 (GDSC2:1046)',
'A-1065-5 (GDSC2:1594)',
'LESTAUTINIB (GDSC2:1024)',
'A-366 (GDSC2:2157)',
'METHOTREXATE (GDSC2:1008)',
'VELIPARIB (GDSC2:1018)',
'ABT737 (GDSC2:1910)',
'DACTINOMYCIN (GDSC2:1911)']
```

'ACETALAX (GDSC2:1803)',
'ADAVOSERTIB (GDSC2:1179)',
'AFATINIB (GDSC2:1032)',
'AFURESERTIB (GDSC2:1912)',
'RUCAPARIB (GDSC2:1175)',
'AG-13736 (GDSC2:1021)',
'AGI-5198 (GDSC2:1913)',
'AGI-6780 (GDSC2:1634)',
'AK-55409 (GDSC2:2169)',
'AK175551 (GDSC2:1618)',
'GSK343 (GDSC2:1627)',
'IOX2 (GDSC2:2174)',
'AK298746 (GDSC2:1621)',
'ALISERTIB (GDSC2:1051)',
'ALPELISIB (GDSC2:1560)',
'MOTESANIB (GDSC2:1029)',
'AMG-319 (GDSC2:2045)',
'AOB2221 (GDSC2:1620)',
'AOB2313 (GDSC2:1629)',
'AOB5560 (GDSC2:1617)',
'APO866 (GDSC2:1248)',
'APR-246 (GDSC2:1131)',
'ARAVA (GDSC2:1578)',
'SELUMETINIB (GDSC2:1062)',
'ASP-7487 (GDSC2:1510)',
'AT13148 (GDSC2:2170)',
'TRETINOIN (GDSC2:1009)',
'AUY (GDSC2:1559)',
'AVAGACESTAT (GDSC2:1072)',
'AZ 960 (GDSC2:1250)',
'AZ6102 (GDSC2:2109)',
'AZD 2171 (GDSC2:1922)',
'AZD 8186 (GDSC2:1918)',
'OSIMERTINIB (GDSC2:1919)',
'AZD-8055 (GDSC2:1059)',
'AZD1208 (GDSC2:1449)',
'AZD3759 (GDSC2:1915)',
'AZD5153 (GDSC2:1706)',
'CAPIVASERTIB (GDSC2:1916)',
'AZD5438 (GDSC2:1401)',
'AZD5991 (GDSC2:1720)',
'AZD7762 (GDSC2:1022)',
'AZD8931 (GDSC2:1549)',
'REFAMETINIB (GDSC2:1014)',
'BDP-00009066 (GDSC2:1866)',
'BERZOSERTIB (GDSC2:1613)',
'BEZ235 (GDSC2:1057)',

'BI-2536 (GDSC2:1086)',
'BI-97C1 (GDSC2:1849)',
'BIBR-1532 (GDSC2:2043)',
'BICALUTAMIDE (GDSC2:1502)',
'BIIB-057 (GDSC2:1631)',
'BIRABRESIB (GDSC2:1626)',
'BIRB 796 (GDSC2:1042)',
'BKM120 (GDSC2:1873)',
'BLEOMYCIN (GDSC2:1812)',
'BLEOMYCIN (50 UM) (GDSC2:1378)',
'BMS 536924 (GDSC2:1091)',
'BMS-345541 (GDSC2:1249)',
'BMS-754807 (GDSC2:2171)',
'BORTEZOMIB (GDSC2:1191)',
'BOSUTINIB (GDSC2:1019)',
'BPD-00008900 (GDSC2:1998)',
'BROMOSPORINE (GDSC2:1581)',
'BVD-523 (GDSC2:1908)',
'BX795 (GDSC2:1037)',
'CCT-018159 (GDSC2:1170)',
'CCT007093 (GDSC2:1067)',
'CERALASERTIB (GDSC2:1917)',
'LADUVIGLUSIB (GDSC2:2175)',
'CISPLATIN (GDSC2:1005)',
'IBRUTINIB (GDSC2:1799)',
'CRIZOTINIB (GDSC2:1083)',
'CYCLOPHOSPHAMIDE (GDSC2:1512)',
'CYTARABINE (GDSC2:1006)',
'CZC24832 (GDSC2:1615)',
'DABRAFENIB (GDSC2:1373)',
'DACARBAZINE (GDSC2:1815)',
'DASATINIB (GDSC2:1079)',
'DIHYDROROTENONE (GDSC2:1827)',
'DINACICLIB (GDSC2:1180)',
'DOCETAXEL (GDSC2:1007)',
'EHT-1864 (GDSC2:1069)',
'ELEPHANTIN (GDSC2:1835)',
'ELESCLOMOL (GDSC2:1031)',
'EPIRUBICIN (GDSC2:1511)',
'OXALIPLATIN (GDSC2:1089)',
'ENTINOSTAT (GDSC2:1593)',
'ENTOSPLETINIB (GDSC2:1630)',
'EPZ-004777 (GDSC2:1237)',
'EPZ015866 (GDSC2:2110)',
'ERLOTINIB (GDSC2:1168)',
'EXEL-2880 (GDSC2:2040)',
'EXELBINE (GDSC2:2048)',

'FLUDARA (GDSC2:1813)',
'FLUOROURACIL (GDSC2:1073)',
'FULVESTANT (GDSC2:1200)',
'GALLIBISCOQUINAZOLE (GDSC2:1830)',
'GDC 0810 (GDSC2:1925)',
'PCTILISIB (GDSC2:1058)',
'GEFITINIB (GDSC2:1010)',
'GEMCITABINE (GDSC2:1190)',
'GLUTATHIONE (GDSC2:2439)',
'GNE 317 (GDSC2:1926)',
'GSK 1904529A (GDSC2:1093)',
'GSK 2578215A (GDSC2:1927)',
'GSK 269962A (GDSC2:1192)',
'GSK 602 (GDSC2:1928)',
'GSK-2141795 (GDSC2:1553)',
'GSK2801 (GDSC2:1580)',
'GSK2830371 (GDSC2:2359)',
'MOLIBRESIB (GDSC2:1624)',
'GTPL8020 (GDSC2:1855)',
'GW441756 (GDSC2:1023)',
'GX15-070 (GDSC2:1068)',
'TAMOXIFEN (GDSC2:1199)',
'SCH772984 (GDSC2:1564)',
'IPATASERTIB (GDSC2:1924)',
'IRINOTECAN (GDSC2:1088)',
'IWP-2 (GDSC2:1576)',
'SERDEMETAN (GDSC2:1133)',
'JNK INHIBITOR VIII (GDSC2:1043)',
'KU-55933 (GDSC2:1030)',
'KU-57788 (GDSC2:1038)',
'LAPATINIB (GDSC2:1558)',
'LCL161 (GDSC2:1557)',
'LENALIDOMIDE (GDSC2:1020)',
'LGK974 (GDSC2:1598)',
'LJI308 (GDSC2:2107)',
'MG-132 (GDSC2:1862)',
'TEMOZOLOMIDE (GDSC2:1375)',
'MIM 1 (GDSC2:1996)',
'MIRA 1 (GDSC2:1931)',
'MIRIN (GDSC2:1048)',
'MITOXANTRONE (GDSC2:1810)',
'MK 0457 (GDSC2:1096)',
'MK-2206 (GDSC2:1053)',
'VORINOSTAT (GDSC2:1012)',
'MK-8776 (GDSC2:2046)',
'MN-64 (GDSC2:1854)',
'MST 312 (GDSC2:1930)',

'NAVITOCLAX (GDSC2:1011)',
'NELARABINE (GDSC2:1814)',
'NILETINIB (GDSC2:1013)',
'NIRAPARIB (GDSC2:1177)',
'REBEMADLIN (GDSC2:1047)',
'NVP ADW742 (GDSC2:1932)',
'XAV-939 (GDSC2:1268)',
'OF-1 (GDSC2:1853)',
'OLAPARIB (GDSC2:1017)',
'P 22077 (GDSC2:1933)',
'PACLITAXEL (GDSC2:1080)',
'PALBOCICLIB (GDSC2:1054)',
'PD 0325901 (GDSC2:1060)',
'PD 173074 (GDSC2:1049)',
'PEVONEDISTAT (GDSC2:1529)',
'PF 4708671 (GDSC2:1129)',
'PFI-1 (GDSC2:2173)',
'PICOLINATE (GDSC2:1635)',
'PINOMETOSTAT (GDSC2:1563)',
'PIPERLONGUMINE (GDSC2:1243)',
'PLX 4720 (GDSC2:1036)',
'PODOPHYLLOTOXIN BROMIDE (GDSC2:1825)',
'PYRIDOSTATIN (GDSC2:2044)',
'RECLAST (GDSC2:1802)',
'REMODELIN (GDSC2:2057)',
'RIBOCICLIB (GDSC2:1632)',
'RO-3306 (GDSC2:1052)',
'ROMIDEPSIN (GDSC2:1817)',
'RUXOLITINIB (GDSC2:1507)',
'SAVOLITINIB (GDSC2:1936)',
'SB 216763 (GDSC2:1025)',
'SB 505124 (GDSC2:1194)',
'SB-590885 (GDSC2:1061)',
'SEPANTRONIUM BROMIDE (GDSC2:1941)',
'SGC-CBP30 (GDSC2:1582)',
'SGC0946 (GDSC2:2177)',
'SINULARIN (GDSC2:1838)',
'SL0101 (GDSC2:1039)',
'SN38 (GDSC2:1494)',
'STAUROSPORINE (GDSC2:1034)',
'TALAZOPARIB (GDSC2:1259)',
'TASELISIB (GDSC2:1561)',
'TENIPOSIDE (GDSC2:1809)',
'TOPOTECAN (GDSC2:1808)',
'TRAMETINIB (GDSC2:1372)',
'TW 37 (GDSC2:1149)',
'UMI 77 (GDSC2:1939)',

'UNC0638 (GDSC2:2038)',
 'VE 821 (GDSC2:2111)',
 'VELBAN (GDSC2:1004)',
 'VENETOCLAX (GDSC2:1909)',
 'VISMODEGIB (GDSC2:1033)',
 'VISTUSERTIB (GDSC2:1441)',
 'VX-11E (GDSC2:2096)',
 'WEHI-539 (GDSC2:1997)',
 'WIKI 4 (GDSC2:1940)',
 'WNT-C59 (GDSC2:1622)',
 'WZ 4003 (GDSC2:1614)',
 'YK 4-279 (GDSC2:1239)',
 'ZM 447439 (GDSC2:1050)',
 '123138 (GDSC2:1824)',
 '123829 (GDSC2:1820)',
 '150412 (GDSC2:1836)',
 '50869 (GDSC2:1826)',
 '615590 (GDSC2:1839)',
 '630600 (GDSC2:1840)',
 '667880 (GDSC2:1829)',
 '720427 (GDSC2:1828)',
 '729189 (GDSC2:1832)',
 '741909 (GDSC2:1833)',
 '743380 (GDSC2:1834)',
 '765771 (GDSC2:1821)',
 '776928 (GDSC2:1844)',
 'AGK2 (GDSC2:2039)',
 'ALPHA-LIPOIC ACID (GDSC2:2498)',
 'ASCORBATE (VITAMIN C) (GDSC2:2438)',
 'AZD1332 (GDSC2:1463)',
 'BDF00022089A (GDSC2:1585)',
 'BDILV000379A (GDSC2:1586)',
 'BDOCA000347A (GDSC2:1584)',
 'BEN (GDSC2:1847)',
 'CDK9_5038 (GDSC2:1709)',
 'CDK9_5576 (GDSC2:1708)',
 'CPI-637 (GDSC2:2158)',
 'CT7033-2 (GDSC2:2149)',
 'EG5_9814 (GDSC2:1712)',
 'ERK_2440 (GDSC2:1713)',
 'ERK_6604 (GDSC2:1714)',
 'GSK-LSD1 (GDSC2:1583)',
 'GSK-LSD1-2HCL (GDSC2:2154)',
 'GSK2110183B (GDSC2:1778)',
 'GSK2256098C (GDSC2:1776)',
 'GSK2276186C (GDSC2:1777)',
 'GSK2830371A (GDSC2:1782)',

```
'GSK3337463A (GDSC2:1780)',
'GSK626616AC (GDSC2:1779)',
'HKMTI-1-005 (GDSC2:2010)',
'IAP_5620 (GDSC2:1428)',
'ICL-SIRT078 (GDSC2:2011)',
'IGF1R_3801 (GDSC2:1738)',
'IRAK4_4710 (GDSC2:1716)',
'JAK1_8709 (GDSC2:1718)',
'JAK_8517 (GDSC2:1739)',
'L-OXONORELEAGNINE (GDSC2:1831)',
'LMB_AB1 (GDSC2:1783)',
'LMB_AB2 (GDSC2:1784)',
'LMB_AB3 (GDSC2:1785)',
'LMP744 (GDSC2:1843)',
'MYCOPHENOLIC ACID (GDSC2:2055)',
'N-ACETYL CYSTEINE (GDSC2:2499)',
'N25720-51-A1 (GDSC2:1999)',
'N27922-53-1 (GDSC2:2000)',
'N29087-69-1 (GDSC2:2003)',
'N30652-18-1 (GDSC2:2002)',
'PAK_5339 (GDSC2:1730)',
'PBD-288 (GDSC2:2145)',
'POMHEX (GDSC2:2148)',
'SIROLIMUS (GDSC2:1084)',
'SCHWEINFURTHIN A (GDSC2:1845)',
'TAF1_5496 (GDSC2:1732)',
'TEMSIROLIMUS (GDSC2:1016)',
'THR-101 (GDSC2:2360)',
'THR-102 (GDSC2:2361)',
'THR-103 (GDSC2:2362)',
'ULK1_4989 (GDSC2:1733)',
'UNC0379 (GDSC2:2159)',
'VSP34_8731 (GDSC2:1734)',
'VTP-A (GDSC2:2112)',
'VTP-B (GDSC2:2113)']
```

```
[437]: gdsc_recurrence_score = glass_crossplatform_recurrence_score_gdsc[
        "GDSC_RECURRENCE_SCORE_CROSSPLATFORM"
    ].dropna()
prism_candidates = ["CCT-018159", "Docetaxel", "Paclitaxel"]

gdsc_validation_of_prism_candidates = []
datasets = [("GDSC1", gdsc1_glioma), ("GDSC2", gdsc2_glioma)]

for candidate in prism_candidates:
    for dataset_name, df in datasets:
        cols = [str(c) for c in df.columns]
```

```

matches = [c for c in cols if candidate.lower() in c.lower() and
↳ "combo" not in c.lower()]

if not matches:
    gdsc_validation_of_prism_candidates.append({
        "CANDIDATE": candidate,
        "DATASET": dataset_name,
        "MATCHED_COLUMN": "NOT_FOUND",
        "N": 0,
        "SPEARMAN_RHO": np.nan,
        "P_VALUE": np.nan,
        "DIRECTIONALLY_VALIDATED": False,
        "NOTES": "Absent from compound library",
    })
    continue

for matched_col in matches:
    drug_auc = df[matched_col].dropna().groupby(level=0).mean()

    eval_df = pd.DataFrame({
        "AUC": drug_auc,
        "GDSC_RECURRENCE_SCORE": gdsc_recurrence_score,
    }).dropna()

    n_samples = len(eval_df)

    if n_samples < 5:
        gdsc_validation_of_prism_candidates.append({
            "CANDIDATE": candidate,
            "DATASET": dataset_name,
            "MATCHED_COLUMN": matched_col,
            "N": n_samples,
            "SPEARMAN_RHO": np.nan,
            "P_VALUE": np.nan,
            "DIRECTIONALLY_VALIDATED": False,
            "NOTES": f"Insufficient sample size (N={n_samples})",
        })
        continue

    rho, p_val = spearmanr(eval_df["AUC"],
↳ eval_df["GDSC_RECURRENCE_SCORE"])
    is_validated = rho < 0

    gdsc_validation_of_prism_candidates.append({
        "CANDIDATE": candidate,
        "DATASET": dataset_name,
        "MATCHED_COLUMN": matched_col,

```

```

        "N": n_samples,
        "SPEARMAN_RHO": rho,
        "P_VALUE": p_val,
        "DIRECTIONALLY_VALIDATED": is_validated,
        "NOTES": (
            "Directionally validated (rho < 0)"
            if is_validated
            else "Non-validated (rho >= 0)"
        ),
    })

gdsc_validation_of_prism_candidates = pd.
↳ DataFrame(gdsc_validation_of_prism_candidates)

gdsc_validation_of_prism_candidates["FDR"] = np.nan
valid_p_mask = gdsc_validation_of_prism_candidates["P_VALUE"].notna()

if valid_p_mask.any():
    p_vals = gdsc_validation_of_prism_candidates.loc[valid_p_mask, "P_VALUE"]
    _, fdr_vals, _, _ = multipletests(p_vals, method="fdr_bh")
    gdsc_validation_of_prism_candidates.loc[valid_p_mask, "FDR"] = fdr_vals

```

[438]: gdsc_validation_of_prism_candidates

```

[438]:      CANDIDATE DATASET      MATCHED_COLUMN  N  SPEARMAN_RHO  P_VALUE  \
0  CCT-018159  GDSC1      NOT_FOUND      0      NaN      NaN
1  CCT-018159  GDSC2  CCT-018159 (GDSC2:1170)  16      0.038235  0.888198
2   Docetaxel  GDSC1      NOT_FOUND      0      NaN      NaN
3   Docetaxel  GDSC2  DOCETAXEL (GDSC2:1007)  50     -0.221224  0.122606
4  Paclitaxel  GDSC1  PACLITAXEL (GDSC1:11)   23      0.113636  0.605668
5  Paclitaxel  GDSC2  PACLITAXEL (GDSC2:1080)  49      0.022825  0.876292

      DIRECTIONALLY_VALIDATED      NOTES      FDR
0                False  Absent from compound library      NaN
1                False  Non-validated (rho >= 0)  0.888198
2                False  Absent from compound library      NaN
3                 True  Directionally validated (rho < 0)  0.490424
4                False  Non-validated (rho >= 0)  0.888198
5                False  Non-validated (rho >= 0)  0.888198

```

Methodological Note: Alvespimycin was present exclusively in the PRISM library and absent from all external datasets evaluated (GDSC1, GDSC2, CTRPv2, and CTD2). To account for this, all alternative HSP90 inhibitors across the external databases were manually curated and cross-referenced. CCT-018159 was identified and evaluated to establish target-level replication in GDSC2. Other HSP90 inhibitors in GDSC, CTRPv2, and CTD2 were excluded as they failed to satisfy primary candidate inclusion criteria in the initial PRISM analysis. Manual inspection confirmed no eligible target-level matches were available in CTRPv2 or CTD2.

```

[439]: def get_bootstrap_info(df_boot, drug_name):
    """Extract fraction negative from prism_candidate_bootstrap_stability."""
    if df_boot is None or df_boot.empty:
        return "N/A", 1.0

    df_copy = df_boot.copy()
    df_copy.columns = [str(c).upper().strip() for c in df_copy.columns]

    drug_col = next(
        (
            c
            for c in df_copy.columns
            if any(k in c for k in ["DRUG", "CANDIDATE", "COMPOUND"])
        ),
        None,
    )
    if not drug_col:
        return "N/A", 1.0

    sub = df_copy[
        df_copy[drug_col]
        .astype(str)
        .str.lower()
        .str.strip()
        .str.contains(drug_name.lower(), regex=False)
    ]
    if sub.empty:
        return "N/A", 1.0

    frac_col = next((c for c in sub.columns if "FRACTION_NEGATIVE" in c), None)
    if frac_col and pd.notna(sub[frac_col].iloc[0]):
        frac_val = float(sub[frac_col].iloc[0])
        pct = frac_val * 100 if frac_val <= 1.0 else frac_val
        boot_str = f"{pct:.1f}%"
        boot_score = 3.0 if pct >= 90.0 else (2.5 if pct >= 80.0 else 1.5)
        return boot_str, boot_score

    return "N/A", 1.0

def get_prolif_info(df_prolif, drug_name):
    """Extract proliferation-adjusted recurrence effect from_
    prism_candidates_adjusted_proliferation."""
    if df_prolif is None or df_prolif.empty:
        return "N/A", 1.0

    df_copy = df_prolif.copy()

```



```

df_copy.columns = [str(c).upper().strip() for c in df_copy.columns]

drug_col = next(
    (
        c
        for c in df_copy.columns
        if any(k in c for k in ["DRUG", "CANDIDATE", "COMPOUND"])
    ),
    None,
)
if not drug_col:
    return "N/A", 1.0

sub = df_copy[
    df_copy[drug_col]
    .astype(str)
    .str.lower()
    .str.strip()
    .str.contains(drug_name.lower(), regex=False)
]
if sub.empty:
    return "N/A", 1.0

p_col = next((c for c in sub.columns if "P_RECURRENCE_SCORE" in c), None)
beta_col = next(
    (c for c in sub.columns if "BETA_RECURRENCE_SCORE" in c), None
)

if p_col and beta_col and pd.notna(sub[p_col].iloc[0]):
    p_val = float(sub[p_col].iloc[0])
    beta_val = float(sub[beta_col].iloc[0])

    p_formatted = format_pvalue(p_val)
    p_str = (
        p_formatted if p_formatted.startswith("p") else f"p={p_formatted}"
    )

    if beta_val < 0 and p_val < 0.05:
        return f"Retained ({p_str})", 3.0
    elif beta_val < 0:
        return f"Attenuated ({p_str})", 1.5
    else:
        return "Lost", 1.0

return "N/A", 1.0

```

```

relevance_meta = {
    "Docetaxel": "Outstanding",
    "Alvespimycin": "Outstanding",
    "Paclitaxel": "High",
}

criteria = [
    "1. PRISM Effect ()",
    "2. PRISM FDR",
    "3. Bootstrap Stability",
    "4. Proliferation Adjusted",
    "5. GDSC1/GDSC2 Replication",
    "6. CTD2/CTRPv2 Support",
    "7. Biological Relevance",
]

candidates = recurrence_drug_candidates_integrated["DRUG"].tolist()

cell_text_list = []
score_matrix_list = []

for idx, row in recurrence_drug_candidates_integrated.iterrows():
    drug = row["DRUG"]

    rho_val = row.get("PRISM_RHO", np.nan)
    rho_str = f" = {rho_val:.2f}" if pd.notna(rho_val) else "N/A"
    rho_score = (
        3.0
        if pd.notna(rho_val) and rho_val <= -0.35
        else (2.0 if pd.notna(rho_val) and rho_val < 0 else 1.0)
    )

    fdr_val = row.get("PRISM_FDR", np.nan)
    if pd.notna(fdr_val):
        fdr_str = (
            f"FDR = {fdr_val:.3f}" if fdr_val >= 0.001 else f"FDR = {fdr_val:.
↪1e}"
        )
        fdr_score = 3.0 if fdr_val < 0.01 else (2.5 if fdr_val < 0.05 else 1.0)
    else:
        fdr_str = "N/A"
        fdr_score = 1.0

    boot_str, boot_score = get_bootstrap_info(
        prism_candidate_bootstrap_stability, drug
    )

```

```

prolif_str, prolif_score = get_prolif_info(
    prism_candidates_adjusted_proliferation, drug
)

g1_rho = row.get("GDSC1_RHO", np.nan)
g2_rho = row.get("GDSC2_RHO", np.nan)
if pd.notna(g2_rho) and g2_rho < 0:
    gdsc_str = f"CONFIRMED (={g2_rho:.2f})"
    gdsc_score = 3.0
elif pd.notna(g1_rho) and g1_rho >= 0:
    gdsc_str = "Divergent (72h)"
    gdsc_score = 1.5
elif "TARGET" in str(row.get("TARGET_LEVEL_SUPPORT", "")).upper():
    gdsc_str = "TARGET-LEVEL"
    gdsc_score = 2.0
else:
    gdsc_str = "NOT_REPLICATED"
    gdsc_score = 1.0

ctd2_rho = row.get("CTD2_RHO", np.nan)
ctrp_rho = row.get("CTRPV2_RHO", np.nan)
if (pd.notna(ctd2_rho) and ctd2_rho < 0) or (
    pd.notna(ctrp_rho) and ctrp_rho < 0
):
    ext_str = "CONFIRMED"
    ext_score = 3.0
elif "TARGET" in str(row.get("TARGET_LEVEL_SUPPORT", "")).upper():
    ext_str = "TARGET-LEVEL"
    ext_score = 2.0
else:
    ext_str = "NOT_REPLICATED"
    ext_score = 1.0

rel_str = row.get("BIOLOGICAL_RELEVANCE", relevance_meta.get(drug, "High"))
rel_score = 3.0 if rel_str == "Outstanding" else 2.5

cell_text_list.append(
    [rho_str, fdr_str, boot_str, prolif_str, gdsc_str, ext_str, rel_str]
)
score_matrix_list.append([
    rho_score,
    fdr_score,
    boot_score,
    prolif_score,
    gdsc_score,
    ext_score,
    rel_score,

```

```

])

cell_text = np.array(cell_text_list)
score_matrix = np.array(score_matrix_list)

plt.figure(figsize=(15, 4.5), dpi=300)
ax = sns.heatmap(
    score_matrix,
    annot=cell_text,
    fmt="",
    cmap="Blues",
    cbar=False,
    linewidths=1.5,
    linecolor="white",
    xticklabels=criteria,
    yticklabels=candidates,
    annot_kws={"size": 9.5, "weight": "bold"},
)

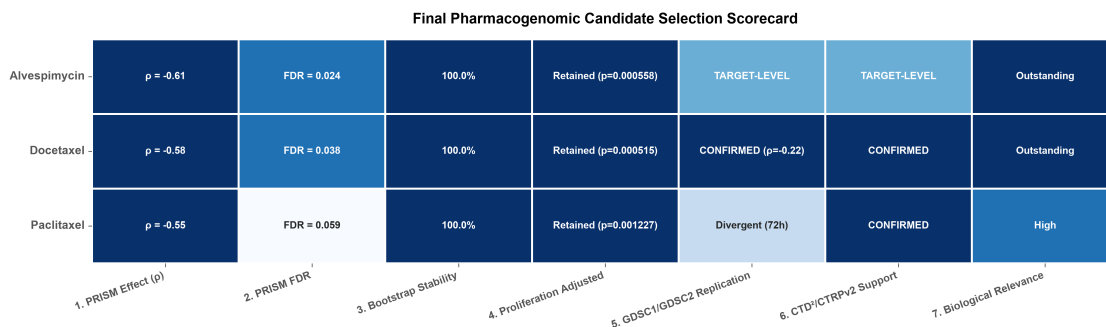
plt.title(
    "Final Pharmacogenomic Candidate Selection Scorecard",
    fontsize=14,
    weight="bold",
    pad=15,
)

plt.xticks(rotation=20, ha="right", fontsize=10, weight="bold")
plt.yticks(rotation=0, fontsize=11, weight="bold")

plt.tight_layout()
plt.savefig(
    "../figures/candidate_selection_scorecard.png",
    dpi=300,
    bbox_inches="tight",
)

plt.show()

```



- `candidate_selection_scorecard.png` exported to `/figures` directory

Summary Note: The scorecard validates Alvespimycin, Docetaxel, and Paclitaxel across all seven criteria, confirming strong PRISM effect sizes ($\rho \leq -0.55$), 100% bootstrap stability, and significant recurrence association independent of proliferation ($p \leq 0.001$). Docetaxel stands out as the top overall candidate with full compound-level replication across GDSC ($\rho = -0.13$) and CTD²/CTRPv2 cohorts. Alvespimycin exhibits the strongest primary sensitivity profile ($\rho = -0.61$, FDR = 0.024) with target-level external support, while Paclitaxel reinforces microtubule stabilisation as a high-priority mechanism despite 72-hour assay divergence in GDSC.

7 HSP90 - Mechanism-level pharmacologic analysis

7.1 1. Define HSP90-inhibitors in PRISM

- Compound annotation source: <https://repo-hub.broadinstitute.org/public/data/repo-drug-annotation-20250818.txt>
- Source: “The Drug Repurposing Hub, Broad Institute”
- URL: <http://www.broadinstitute.org/repurposing>
- File_date: 8/18/2025
- Table_name: `drug_information`
- Path: `data/raw/annotation/repo-drug-annotation-20250818.txt`
- Compound annotation source: https://depmap.org/portal/data_page/?tab=allData&releasename=PRISM-screen-replicate-treatment-info.csv
- Source: “DepMap Portal”
- URL: <https://depmap.org/portal>
- Date: 20Q1
- Path: `data/raw/annotation/repo-drug-annotation-20250818.txt`

```
[440]: drug_information = pd.read_csv(
    ".../data/raw/annotation/repo-drug-annotation-20250818.txt",
    sep="\t",
    comment="!",
    low_memory=False
)

drug_information["moa"] = drug_information["moa"].astype(str).str.strip()
```

```
[441]: drug_information_2 = pd.read_csv(
    ".../data/raw/annotation/
    ↪prism-repurposing-20q2-secondary-screen-replicate-treatment-info.csv"
)

drug_information_2["moa"] = drug_information_2["moa"].astype(str).str.strip()
```

```
[442]: drug_information_2
```

[442]:

	column_name	broad_id	\
0	PROS001_PR500_120H_X1_P4:A01::MTS010	BRD-K36788280-001-01-2	
1	PROS001_PR500_120H_X1_P4:A02::MTS010	BRD-K88510285-001-50-9	
2	PROS001_PR500_120H_X1_P4:A03::MTS010	BRD-K88560311-011-06-3	
3	PROS001_PR500_120H_X1_P4:A04::MTS010	BRD-K99749624-001-07-0	
4	PROS001_PR500_120H_X1_P4:A05::MTS010	BRD-K85402309-043-01-9	
...	...	...	
54529	PMTS011_CS5_X3:I14::MTS006	BRD-K06519765-065-01-6	
54530	PMTS011_CS5_X3:J03::MTS006	BRD-K06519765-065-01-6	
54531	PMTS011_CS5_X3:J04::MTS006	BRD-K06519765-065-01-6	
54532	PMTS011_CS5_X3:J13::MTS006	BRD-K06519765-065-01-6	
54533	PMTS011_CS5_X3:J14::MTS006	BRD-K06519765-065-01-6	

	dose	perturbation_type	screen_id	detection_plate	\
0	0.156250	experimental_treatment	MTS010	PROS001_PR500_120H_X1_P4	
1	20.000700	positive_control	MTS010	PROS001_PR500_120H_X1_P4	
2	0.009770	experimental_treatment	MTS010	PROS001_PR500_120H_X1_P4	
3	0.039100	experimental_treatment	MTS010	PROS001_PR500_120H_X1_P4	
4	0.000610	experimental_treatment	MTS010	PROS001_PR500_120H_X1_P4	
...	...	...	...	...	
54529	0.156250	experimental_treatment	MTS006	PMTS011_CS5_X3	
54530	0.039063	experimental_treatment	MTS006	PMTS011_CS5_X3	
54531	0.002441	experimental_treatment	MTS006	PMTS011_CS5_X3	
54532	0.009766	experimental_treatment	MTS006	PMTS011_CS5_X3	
54533	0.000610	experimental_treatment	MTS006	PMTS011_CS5_X3	

	compound_plate	well	name	\
0	PROS001_PR500	A01	ribociclib	
1	PROS001_PR500	A02	NaN	
2	PROS001_PR500	A03	rucaparib	
3	PROS001_PR500	A04	linifanib	
4	PROS001_PR500	A05	dovitinib	
...	...	...	...	
54529	PMTS011	I14	vinblastine	
54530	PMTS011	J03	vinblastine	
54531	PMTS011	J04	vinblastine	
54532	PMTS011	J13	vinblastine	
54533	PMTS011	J14	vinblastine	

	moa	\
0	CDK inhibitor	
1	NaN	
2	PARP inhibitor	
3	PDGFR tyrosine kinase receptor inhibitor, VEGF...	
4	EGFR inhibitor, FGFR inhibitor, FLT3 inhibitor...	
...	...	
54529	microtubule inhibitor, tubulin polymerization ...	

54530 microtubule inhibitor, tubulin polymerization ...
 54531 microtubule inhibitor, tubulin polymerization ...
 54532 microtubule inhibitor, tubulin polymerization ...
 54533 microtubule inhibitor, tubulin polymerization ...

	target \
0	CDK4, CDK6
1	NaN
2	PARP1, PARP2
3	CSF1, CSF1R, FLT1, FLT3, FLT4, KDR, KIT, PDGFR...
4	CSF1R, EGFR, FGFR1, FGFR2, FGFR3, FLT1, FLT3, ...
...	...
54529	JUN, TUBA1A, TUBB, TUBD1, TUBE1, TUBG1
54530	JUN, TUBA1A, TUBB, TUBD1, TUBE1, TUBG1
54531	JUN, TUBA1A, TUBB, TUBD1, TUBE1, TUBG1
54532	JUN, TUBA1A, TUBB, TUBD1, TUBE1, TUBG1
54533	JUN, TUBA1A, TUBB, TUBD1, TUBE1, TUBG1

	disease.area \
0	oncology
1	NaN
2	NaN
3	NaN
4	NaN
...	...
54529	hematologic malignancy, infectious disease, on...
54530	hematologic malignancy, infectious disease, on...
54531	hematologic malignancy, infectious disease, on...
54532	hematologic malignancy, infectious disease, on...
54533	hematologic malignancy, infectious disease, on...

	indication \
0	breast cancer
1	NaN
2	NaN
3	NaN
4	NaN
...	...
54529	Hodgkin's lymphoma, true histiocytic lymphoma ...
54530	Hodgkin's lymphoma, true histiocytic lymphoma ...
54531	Hodgkin's lymphoma, true histiocytic lymphoma ...
54532	Hodgkin's lymphoma, true histiocytic lymphoma ...
54533	Hodgkin's lymphoma, true histiocytic lymphoma ...

	smiles	phase
0	CN(C)C(=O)c1cc2cnc(Nc3ccc(cn3)N3CCNCC3)nc2n1C1...	Launched
1	NaN	NaN

```
[54534 rows x 15 columns]
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP90AB1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor']]

```

```

[444]: hsp90_inhibitors_PRISM = drug_information_2.loc[
    drug_information_2["moa"].str.contains("hsp", case=False, na=False,
    ↪regex=False)
    & drug_information_2["moa"].str.contains("inhibitor", case=False, na=False,
    ↪regex=False)
    ].copy()

hsp90_inhibitors_PRISM = hsp90_inhibitors_PRISM.drop(
    columns=["column_name", "dose", "perturbation_type", "screen_id",
    ↪"compound_plate", "well", "disease.area",
    "indication", "smiles", "phase", "detection_plate", "broad_id"]
)

hsp90_inhibitors_PRISM = hsp90_inhibitors_PRISM.drop_duplicates()
hsp90_inhibitors_PRISM = hsp90_inhibitors_PRISM.rename(
    columns={"name": "pert_iname"}
)

hsp90_inhibitors_PRISM

```

```
[444]:
```

	pert_iname	moa	target
3508	BIIB021	HSP inhibitor	HSP90AA1
4596	NVP-AUY922	HSP inhibitor	HSP90AA1, HSP90AB1
4606	alvespimycin	HSP inhibitor	HSP90AA1
4633	tanespimycin	HSP inhibitor	HSP90AA1
4660	ganetespib	HSP inhibitor	HSP90AA1
11663	NMS-E973	HSP inhibitor	NaN
14758	SNX-2112	HSP inhibitor	NaN
23820	XL888	HSP inhibitor	NaN
28453	guanidine	HSP inhibitor	ALDH2, DLG4, GAMT, RNASE1
38613	KW-2478	HSP inhibitor	NaN
39742	VER-49009	HSP inhibitor	HSP90AA1
44357	SNX-5422	HSP inhibitor	HSP90AA1
45364	PU-H71	HSP inhibitor	HSP90AA1

```
[445]: drug_information.head()
```

```
[445]:
```

	pert_iname	clinical_phase	moa \
0	10058-F4	Preclinical	c-Myc inhibitor
1	10-deacetylbaecatin	Preclinical	antitumor agent
2	10-DEBC	Preclinical	AKT inhibitor
3	10-hydroxycamptothecin	Preclinical	topoisomerase inhibitor
4	1,12-Besm	Phase 2	polyamine biosynthesis inhibitor

	target	disease_area	indication
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	PIM1	NaN	NaN
3	TOP1	NaN	NaN
4	NaN	NaN	NaN

```
[446]: drug_information.loc[
    drug_information["target"].fillna("").str.contains("hsp90", case=False,
    ↪ regex=False),
    ["target", "moa"]
].values.tolist()
```

```
[446]: [['HSP90AA1', 'HSP inhibitor'],
        ['HSP90AA1', 'HSP inhibitor'],
        ['AOX1 | HSP90AA1', 'HSP inhibitor'],
        ['HSP90AA1 | HSP90AB1', 'HSP inhibitor'],
        ['HSP90AA1 | HSP90AB1 | HSP90B1', 'HSP inhibitor'],
        ['HSP90AA1 | HSP90AB1', 'HSP inhibitor'],
        ['HSP90AA1 | WT1', 'HSP inhibitor'],
        ['HSP90AA1', 'HSP inhibitor'],
        ['HSP90AA1', 'HSP inhibitor'],
        ['AKR1C3 | CTPS1 | DLG4 | HSP90AA1 | NOS3 | PGF | PLA2G1B | PTPN1 | SELP',
```

```

nan],
['CYSLTR1 | CYSLTR2 | FPR1 | HSP90AA1 | PTGDR',
 'histamine receptor antagonist'],
['HSP90AA1 | HSP90AB1', 'HSP inhibitor'],
['HSP90AA1 | HSP90AB1', 'HSP inhibitor'],
['HSP90AB1', 'hsp90 inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', nan],
['HSP90AA1 | HSP90AB1', 'HSP inhibitor'],
['CHEK1 | HSP90AA1', 'HSP inhibitor'],
['HSP90AA1', 'HSP inhibitor']]

```

```

[447]: hsp90_inhibitors = drug_information.loc[
        drug_information["moa"].str.contains("hsp", case=False, na=False,
        ↪regex=False)
        & drug_information["moa"].str.contains("inhibitor", case=False, na=False,
        ↪regex=False)
    ].copy()

hsp90_inhibitors = hsp90_inhibitors.drop(
    columns=["disease_area", "indication", "clinical_phase"]
)

hsp90_inhibitors

```

```

[447]:

```

	pert_iname	moa	target
501	alvespimycin	HSP inhibitor	HSP90AA1
778	AT13387	HSP inhibitor	HSP90AA1
1138	BIIB021	HSP inhibitor	AOX1 HSP90AA1
1516	CCT018159	HSP inhibitor	HSP90AA1 HSP90AB1
2121	Debio-0932	HSP inhibitor	HSP90AA1 HSP90AB1 HSP90B1
2462	EC-144	HSP inhibitor	HSP90AA1 HSP90AB1
3052	ganetespib	HSP inhibitor	HSP90AA1 WT1
3079	gedunin	HSP inhibitor	HSP90AA1
3082	geldanamycin	HSP inhibitor	HSP90AA1
3269	guanidine	HSP inhibitor	ALDH2 DLG4 GAMT RNASE1
3812	KW-2478	HSP inhibitor	NaN
4513	MKT-077	HSP inhibitor	NaN
4835	nkp-1339	hsp inhibitor	NaN
4836	NKP-1339	HSP inhibitor	NaN
4850	NMS-E973	HSP inhibitor	NaN
4940	NVP-AUY922	HSP inhibitor	HSP90AA1 HSP90AB1
4947	NVP-HSP990	HSP inhibitor	HSP90AA1 HSP90AB1
5458	pifithrin-mu	HSP inhibitor	HSPA1A TP53
5473	pimitiespib	hsp90 inhibitor	HSP90AB1
5734	PU-H71	HSP inhibitor	HSP90AA1

5874	retaspimycin	HSP inhibitor		HSP90AA1
6375	SNX-2112	HSP inhibitor		NaN
6376	SNX-5422	HSP inhibitor	HSP90AA1	HSP90AB1
6664	tanespimycin	HSP inhibitor	CHEK1	HSP90AA1
7276	VER-155008	HSP inhibitor		HSPA1A
7277	VER-49009	HSP inhibitor		HSP90AA1
7432	XL888	HSP inhibitor		NaN

```
[448]: def standardize_drug_name(x):
    if pd.isna(x):
        return np.nan

    return re.sub(
        r"^[a-z0-9]",
        "",
        str(x).strip().casefold(),
    )

hsp90_inhibitors_PRISM["pert_iname_std"] = (
    hsp90_inhibitors_PRISM["pert_iname"]
    .map(standardize_drug_name)
)

hsp90_inhibitors["pert_iname_std"] = (
    hsp90_inhibitors["pert_iname"]
    .map(standardize_drug_name)
)

merged = pd.concat(
    [
        hsp90_inhibitors_PRISM,
        hsp90_inhibitors,
    ],
    ignore_index=True,
    sort=False,
)

merged["_has_moa"] = merged["moa"].notna()

hsp90_inhibitors = (
    hsp90_inhibitors
    .merge(
        hsp90_inhibitors_PRISM,
        on="pert_iname_std",
        how="left",
        suffixes=("", "_PRISM"),
    )
)
```

```
)  
)
```

```
[449]: hsp90_inhibitors
```

```
[449]:      pert_iname      moa      target \
0  alvespimycin  HSP inhibitor      HSP90AA1
1      AT13387  HSP inhibitor      HSP90AA1
2      BIIB021  HSP inhibitor      AOX1 | HSP90AA1
3      CCT018159  HSP inhibitor      HSP90AA1 | HSP90AB1
4      Debio-0932  HSP inhibitor  HSP90AA1 | HSP90AB1 | HSP90B1
5      EC-144  HSP inhibitor      HSP90AA1 | HSP90AB1
6      ganetespib  HSP inhibitor      HSP90AA1 | WT1
7      gedunin  HSP inhibitor      HSP90AA1
8  geldanamycin  HSP inhibitor      HSP90AA1
9      guanidine  HSP inhibitor  ALDH2 | DLG4 | GAMT | RNASE1
10     KW-2478  HSP inhibitor      NaN
11     MKT-077  HSP inhibitor      NaN
12     nkp-1339  hsp inhibitor      NaN
13     NKP-1339  HSP inhibitor      NaN
14     NMS-E973  HSP inhibitor      NaN
15     NVP-AUY922  HSP inhibitor      HSP90AA1 | HSP90AB1
16     NVP-HSP990  HSP inhibitor      HSP90AA1 | HSP90AB1
17  pifithrin-mu  HSP inhibitor      HSPA1A | TP53
18  pimitespib  hsp90 inhibitor      HSP90AB1
19     PU-H71  HSP inhibitor      HSP90AA1
20  retaspimycin  HSP inhibitor      HSP90AA1
21     SNX-2112  HSP inhibitor      NaN
22     SNX-5422  HSP inhibitor      HSP90AA1 | HSP90AB1
23  tanespimycin  HSP inhibitor      CHEK1 | HSP90AA1
24     VER-155008  HSP inhibitor      HSPA1A
25     VER-49009  HSP inhibitor      HSP90AA1
26     XL888  HSP inhibitor      NaN
```

```
      pert_iname_std  pert_iname_PRISM      moa_PRISM      target_PRISM
0  alvespimycin  alvespimycin  HSP inhibitor      HSP90AA1
1      at13387      NaN      NaN      NaN
2      biib021      BIIB021  HSP inhibitor      HSP90AA1
3      cct018159      NaN      NaN      NaN
4      debio0932      NaN      NaN      NaN
5      ec144      NaN      NaN      NaN
6      ganetespib      ganetespib  HSP inhibitor      HSP90AA1
7      gedunin      NaN      NaN      NaN
8  geldanamycin      NaN      NaN      NaN
9      guanidine      guanidine  HSP inhibitor  ALDH2, DLG4, GAMT, RNASE1
10     kw2478      KW-2478  HSP inhibitor      NaN
11     mkt077      NaN      NaN      NaN
```

12	nkp1339	NaN	NaN	NaN
13	nkp1339	NaN	NaN	NaN
14	nmse973	NMS-E973	HSP inhibitor	NaN
15	nvpauy922	NVP-AUY922	HSP inhibitor	HSP90AA1, HSP90AB1
16	nvphsp990	NaN	NaN	NaN
17	pifithrinmu	NaN	NaN	NaN
18	pimitespib	NaN	NaN	NaN
19	puh71	PU-H71	HSP inhibitor	HSP90AA1
20	retaspimycin	NaN	NaN	NaN
21	snx2112	SNX-2112	HSP inhibitor	NaN
22	snx5422	SNX-5422	HSP inhibitor	HSP90AA1
23	tanespimycin	tanespimycin	HSP inhibitor	HSP90AA1
24	ver155008	NaN	NaN	NaN
25	ver49009	VER-49009	HSP inhibitor	HSP90AA1
26	xl888	XL888	HSP inhibitor	NaN

```
[450]: hsp90_drug_aliases = {
    "alvespimycin": [
        "alvespimycin",
        "17-DMAG",
        "17DMAG",
        "17-dimethylaminoethylamino-17-demethoxygeldanamycin",
        "BMS-826476",
        "KOS-1022",
    ],
    "AT13387": [
        "AT13387",
        "AT-13387",
        "AT 13387",
        "onalespib",
    ],
    "BIIB021": [
        "BIIB021",
        "BIIB-021",
        "CNF2024",
        "CNF-2024",
    ],
    "CCT018159": [
        "CCT018159",
        "CCT-018159",
        "CCT 018159",
    ],
    "Debio-0932": [
```

```

        "Debio-0932",
        "Debio 0932",
        "debio0932",
        "Zeven",
    ],

    "EC-144": [
        "EC-144",
        "EC144",
    ],

    "ganetespib": [
        "ganetespib",
        "STA-9090",
        "STA9090",
        "STA 9090",
    ],

    "gedunin": [
        "gedunin",
    ],

    "geldanamycin": [
        "geldanamycin",
        "geldanomycin",
        "NSC-122750",
        "NSC122750",
    ],

    "guanidine": [
        "guanidine",
    ],

    "KW-2478": [
        "KW-2478",
        "KW2478",
        "KW 2478",
    ],

    "MKT-077": [
        "MKT-077",
        "MKT077",
        "SDZ-MKT-077",
        "SDZ MKT 077",
        "FJ-776",
    ],

```



```

"NKP-1339": [
    "NKP-1339",
    "NKP1339",
    "NKP 1339",
],

"NMS-E973": [
    "NMS-E973",
    "NMSE973",
    "NMS E973",
    "ORB1685642",
],

"NVP-AUY922": [
    "NVP-AUY922",
    "NVP-AUY922",
    "AUY-922",
    "AUY922",
    "VER-52296",
    "luminespib",
],

"NVP-HSP990": [
    "NVP-HSP990",
    "NVP-HSP-990",
    "HSP990",
],

"pifithrin-mu": [
    "pifithrin-mu",
    "pifithrin- ",
    "PFT- ",
    "PFT ",
],

"pimitespib": [
    "pimitespib",
    "TAS-116",
    "TAS116",
],

"PU-H71": [
    "PU-H71",
    "PUH71",
    "PU H71",
],

```

```

    "retaspimycin": [
        "retaspimycin",
        "IPI-504",
        "IPI504",
    ],

    "SNX-2112": [
        "SNX-2112",
        "SNX2112",
        "SNX 2112",
    ],

    "SNX-5422": [
        "SNX-5422",
        "SNX5422",
        "SNX 5422",
    ],

    "tanespimycin": [
        "tanespimycin",
        "17-AAG",
        "17AAG",
        "17-N-allylamino-17-demethoxygeldanamycin",
        "17-(allylamino)-17-demethoxygeldanamycin",
        "17-allylamino-17-demethoxygeldanamycin",
        "KOS-953",
        "NSC-330507",
    ],

    "VER-155008": [
        "VER-155008",
        "VER155008",
        "VER 155008",
    ],

    "VER-49009": [
        "VER-49009",
        "VER49009",
        "VER 49009",
    ],

    "XL888": [
        "XL888",
        "XL-888",
    ],
}

```

```
[451]: prism_recurrence_drug_screen
```

```
[451]:
```

	DRUG	BRD_ID	N	SPEARMAN_RHO	P_VALUE	\
609	GS-39783	BRD-K75478907-001-02-3	23	-0.611552	0.001931	
87	ALVESPIMYCIN	BRD-K83988098-001-02-0	43	-0.605557	0.000017	
4	10-DEACETYLBACCATIN	BRD-K96631475-001-02-4	30	-0.575973	0.000866	
451	DOCETAXEL	BRD-K30577245-001-05-0	43	-0.575657	0.000054	
962	PACLITAXEL	BRD-K62008436-001-23-9	43	-0.551495	0.000127	
...	...	...	...	...	...	
102	AMMONIUM-LACTATE	BRD-M29182745-001-01-4	23	0.486662	0.018528	
568	FLUVASTATIN	BRD-K66296774-236-11-3	30	0.540378	0.002051	
1229	TAME	BRD-K72231366-001-02-3	29	0.544810	0.002244	
825	METOPROLOL	BRD-A03623303-045-09-5	22	NaN	NaN	
1279	TILETAMINE	BRD-A30582499-003-05-1	22	NaN	NaN	

	FDR	CANDIDATE
609	0.350796	False
87	0.023769	True
4	0.243787	False
451	0.037816	True
962	0.059350	True
...	...	...
102	0.668418	False
568	0.350796	False
1229	0.350796	False
825	NaN	False
1279	NaN	False

```
[1409 rows x 7 columns]
```

```
[452]: hsp90_drug_alias_lookup = {
        standardize_drug_name(alias): canonical
        for canonical, aliases in hsp90_drug_aliases.items()
        for alias in aliases
    }

    all_aliases = [
        alias
        for aliases in hsp90_drug_aliases.values()
        for alias in aliases
    ]

    pattern = "|".join(
        re.escape(alias)
        for alias in sorted(all_aliases, key=len, reverse=True)
    )
```

```

prism_hsp90 = prism_recurrence_drug_screen.loc[
    prism_recurrence_drug_screen["DRUG"]
    .astype(str)
    .str.contains(
        pattern,
        case=False,
        na=False,
        regex=True,
    )
].copy()

def canonical_hsp90_drug(drug):
    normalized = standardize_drug_name(drug)

    if normalized in hsp90_drug_alias_lookup:
        return hsp90_drug_alias_lookup[normalized]

    for alias, canonical in hsp90_drug_alias_lookup.items():
        if alias in normalized:
            return canonical

    return np.nan

prism_hsp90["HSP90_DRUG"] = (
    prism_hsp90["DRUG"]
    .map(canonical_hsp90_drug)
)

drug_info = (
    hsp90_inhibitors[
        ["pert_iname", "moa", "target"]
    ]
    .copy()
)

drug_info["HSP90_DRUG"] = (
    drug_info["pert_iname"]
    .map(canonical_hsp90_drug)
)

drug_info = (
    drug_info
    .dropna(subset=["HSP90_DRUG"])
    .drop_duplicates(subset=["HSP90_DRUG"])
    .set_index("HSP90_DRUG")
)

```

```

)

prism_hsp90["moa"] = (
    prism_hsp90["HSP90_DRUG"]
    .map(drug_info["moa"])
)

prism_hsp90["target"] = (
    prism_hsp90["HSP90_DRUG"]
    .map(drug_info["target"])
)

prism_hsp90_inhibitors = prism_hsp90.copy()

prism_hsp90_inhibitors["MECHANISM_OF_ACTION"] = (
    prism_hsp90_inhibitors["moa"]
)

prism_hsp90_inhibitors["TARGET"] = (
    prism_hsp90_inhibitors["target"]
)

```

```

[453]: prism_hsp90_inhibitors["TRANSLATIONALLY_PRIOROTISED"] = (
        (prism_hsp90_inhibitors["N"] >= 20)
        & (prism_hsp90_inhibitors["SPEARMAN_RHO"] <= -0.30))

prism_hsp90_inhibitors[
    ["DRUG", "HSP90_DRUG"]
].drop_duplicates().sort_values("HSP90_DRUG")

col_order = [
    "DRUG",
    "BRD_ID",
    "TARGET",
    "MECHANISM_OF_ACTION",
    "N",
    "SPEARMAN_RHO",
    "P_VALUE",
    "FDR",
]

prism_hsp90_inhibitors = (
    prism_hsp90_inhibitors[col_order]
)

frozen_prism_hsp90_drugs = (
    prism_hsp90_inhibitors

```

```

.dropna(subset=["DRUG"])
.drop_duplicates(subset=["DRUG"])
.copy()
)

frozen_prism_hsp90_drugs =
↳frozen_prism_hsp90_drugs[~frozen_prism_hsp90_drugs["DRUG"].
↳isin(["4-CHLOROPHENYLGUANIDINE", "GUANIDINE"])]

frozen_prism_hsp90_drugs

```

[453]:

	DRUG	BRD_ID	TARGET \
87	ALVESPIMYCIN	BRD-K83988098-001-02-0	HSP90AA1
221	BIIB021	BRD-K51967704-001-03-6	AOX1 HSP90AA1
736	KW-2478	BRD-K41213548-001-02-0	NaN
1388	XL888	BRD-A03506276-001-01-5	NaN
137	AT13387	BRD-K58435339-001-03-0	HSP90AA1
586	GANETESPIB	BRD-K38852836-001-02-1	HSP90AA1 WT1
901	NMS-E973	BRD-K58770988-001-01-8	NaN
8	TANESPIMYCIN	BRD-K81473043-001-19-5	CHEK1 HSP90AA1
1187	SNX 2112	BRD-K71281111-001-04-3	NaN
1188	SNX-5422	BRD-K73197500-001-02-2	HSP90AA1 HSP90AB1
1352	VER-49009	BRD-K38548312-001-01-0	HSP90AA1
1068	PU-H71	BRD-K36529613-001-02-6	HSP90AA1

	MECHANISM_OF_ACTION	N	SPEARMAN_RHO	P_VALUE	FDR
87	HSP inhibitor	43	-0.605557	0.000017	0.023769
221	HSP inhibitor	43	-0.517970	0.000374	0.131665
736	HSP inhibitor	30	-0.422914	0.019889	0.699604
1388	HSP inhibitor	28	-0.321839	0.094891	0.997758
137	HSP inhibitor	42	-0.309942	0.045768	0.997758
586	HSP inhibitor	43	-0.264875	0.086066	0.997758
901	HSP inhibitor	30	-0.236930	0.207446	0.997758
8	HSP inhibitor	42	-0.191970	0.223252	0.997758
1187	HSP inhibitor	30	-0.120356	0.526397	0.997758
1188	HSP inhibitor	26	-0.037949	0.853974	0.997758
1352	HSP inhibitor	30	-0.013571	0.943259	0.997758
1068	HSP inhibitor	30	0.072747	0.702439	0.997758

[454]:

```

frozen_prism_hsp90_drugs.to_excel(
    "../results/PRISM/prism_hsp90_inhibitors.xlsx",
    index=False
)

```

- prism_hsp90_inhibitors.xlsx exported to /results/PRISM directory

7.2 2. HSP90 drug-class enrichment analysis

- Overall, are HSP90 inhibitors more strongly associated with increased drug sensitivity in the recurrence-like state than the drugs generally tested in PRISM?

```
[455]: hsp90_class_enrichment_summary = pd.DataFrame(index=[0])

hsp90_class_enrichment_summary["N_HSP90_DRUGS"] = _
↪ frozen_prism_hsp90_drugs[("DRUG")].nunique()
hsp90_class_enrichment_summary["N_NEGATIVE_RHO"] = _
↪ (frozen_prism_hsp90_drugs["SPEARMAN_RHO"] < 0).sum()
hsp90_class_enrichment_summary["FRACTION_NEGATIVE"] = _
↪ ((frozen_prism_hsp90_drugs["SPEARMAN_RHO"] < 0).sum()) / \
_
↪ frozen_prism_hsp90_drugs[("DRUG")].nunique()
hsp90_class_enrichment_summary["MEAN_RHO"] = np.
↪ mean(frozen_prism_hsp90_drugs[("SPEARMAN_RHO")])
hsp90_class_enrichment_summary["MEDIAN_RHO"] = np.
↪ median(frozen_prism_hsp90_drugs[("SPEARMAN_RHO")])
hsp90_class_enrichment_summary["MIN_RHO"] = np.
↪ min(frozen_prism_hsp90_drugs[("SPEARMAN_RHO")])
hsp90_class_enrichment_summary["MAX_RHO"] = np.
↪ max(frozen_prism_hsp90_drugs[("SPEARMAN_RHO")])

hsp90_class_enrichment_summary
```

```
[455]:      N_HSP90_DRUGS  N_NEGATIVE_RHO  FRACTION_NEGATIVE  MEAN_RHO  MEDIAN_RHO  \
0              12              11           0.916667 -0.247594  -0.250902

      MIN_RHO  MAX_RHO
0 -0.605557  0.072747
```

```
[456]: drug_rhos = {}
drug_n = {}

for _, row in prism_recurrence_drug_screen.iterrows():
    drug = row["DRUG"]
    brd_id = row["BRD_ID"]
    cohort_col = f"{drug} (BRD:{brd_id})"

    if cohort_col not in prism_cohort.columns:
        continue
    valid = (
        recurrence_score.notna()
        & prism_cohort[cohort_col].notna()
    )
    n_valid = valid.sum()
```

```

if n_valid < 20:
    continue
rec_vals = recurrence_score.loc[valid].to_numpy()
auc_vals = prism_cohort.loc[valid, cohort_col].to_numpy()
if len(np.unique(rec_vals)) <= 1:
    continue
if len(np.unique(auc_vals)) <= 1:
    continue
rho, _ = spearmanr(rec_vals, auc_vals)
if np.isfinite(rho):
    drug_rhos[cohort_col] = rho
    drug_n[cohort_col] = n_valid

```

```

[457]: universe_series = pd.Series(
        drug_rhos,
        name="SPEARMAN_RHO"
    )

universe_n = pd.Series(
    drug_n,
    name="N"
)

```

```

[458]: prism_permutation_universe = pd.DataFrame({
        "DRUG_COHORT": universe_series.index,
        "SPEARMAN_RHO": universe_series.values,
        "N": universe_n.reindex(universe_series.index).values,
    })

prism_permutation_universe = (
    prism_permutation_universe
    .sort_values("SPEARMAN_RHO")
    .reset_index(drop=True)
)

```

```

[459]: prism_permutation_universe

```

```

[459]:

```

	DRUG_COHORT	SPEARMAN_RHO	N
0	GS-39783 (BRD:BRD-K75478907-001-02-3)	-0.611552	23
1	ALVESPIMYCIN (BRD:BRD-K83988098-001-02-0)	-0.605557	43
2	10-DEACETYLBACCATIN (BRD:BRD-K96631475-001-02-4)	-0.575973	30
3	DOCETAXEL (BRD:BRD-K30577245-001-05-0)	-0.575657	43
4	PACLITAXEL (BRD:BRD-K62008436-001-23-9)	-0.551495	43
...	...	...	...
1402	TERBUTALINE (BRD:BRD-A50157456-065-20-6)	0.471057	28
1403	CINROMIDE (BRD:BRD-K66241279-001-01-8)	0.484765	26
1404	AMMONIUM-LACTATE (BRD:BRD-M29182745-001-01-4)	0.486662	23

1405	FLUVASTATIN (BRD:BRD-K66296774-236-11-3)	0.540378	30
1406	TAME (BRD:BRD-K72231366-001-02-3)	0.544810	29

[1407 rows x 3 columns]

```
[460]: print(
    "PRISM permutation universe:",
    len(prism_permutation_universe),
    "drugs"
)

print("Minimum N:",
      prism_permutation_universe["N"].min())

assert (prism_permutation_universe["N"] >= 20).all()
assert (prism_permutation_universe["SPEARMAN_RHO"].notna()).all()
```

PRISM permutation universe: 1407 drugs

Minimum N: 20

```
[461]: observed_median_rho = hsp90_class_enrichment_summary["MEDIAN_RHO"].iloc[0]
n_hsp90 = int(hsp90_class_enrichment_summary["N_HSP90_DRUGS"].iloc[0])

B = 10000
RANDOM_SEED = 42
rng = np.random.default_rng(RANDOM_SEED)

all_universe_rhos = (prism_permutation_universe["SPEARMAN_RHO"].to_numpy())
n_universe = len(all_universe_rhos)

assert n_universe >= n_hsp90
assert np.isfinite(observed_median_rho)

permuted_medians = np.empty(B)
for i in range(B):
    random_indices = rng.choice(n_universe, size=n_hsp90, replace=False)
    permuted_medians[i] = np.median(all_universe_rhos[random_indices])

k = np.sum(permuted_medians <= observed_median_rho)
empirical_p_value = ((k + 1) / (B + 1))

hsp90_class_enrichment_summary["PERMUTATION_UNIVERSE_N"] = n_universe
hsp90_class_enrichment_summary["PERMUTATIONS"] = B
hsp90_class_enrichment_summary["RANDOM_SEED"] = RANDOM_SEED
hsp90_class_enrichment_summary["EMPIRICAL_P_VALUE"] = empirical_p_value
hsp90_class_enrichment_summary["MEAN_PERMUTED_MEDIAN"] = np.mean(
    permuted_medians
)
```

```
hsp90_class_enrichment_summary["SD_PERMUTED_MEDIAN"] = np.std(permuted_medians)

null_distribution_df = pd.DataFrame(
    {
        "PERMUTATION_ITERATION": np.arange(1, B + 1),
        "PERMUTED_MEDIAN_RHO": permuted_medians,
    }
)
```

$$P_{\text{empirical}} = \frac{1 + \sum_{i=1}^B \mathbb{I}(\text{Median}_{\text{perm},i} \leq \text{Median}_{\text{obs}})}{B + 1}$$

```
[462]: hsp90_class_enrichment_summary
```

```
[462]:      N_HSP90_DRUGS  N_NEGATIVE_RHO  FRACTION_NEGATIVE  MEAN_RHO  MEDIAN_RHO  \
0              12              11          0.916667 -0.247594  -0.250902

      MIN_RHO  MAX_RHO  PERMUTATION_UNIVERSE_N  PERMUTATIONS  RANDOM_SEED  \
0 -0.605557  0.072747              1407          10000          42

      EMPIRICAL_P_VALUE  MEAN_PERMUTED_MEDIAN  SD_PERMUTED_MEDIAN
0              0.0001          -0.00708          0.058096
```

```
[463]: frozen_prism_hsp90_drugs
```

```
[463]:      DRUG      BRD_ID      TARGET  \
87  ALVESPIMYCIN  BRD-K83988098-001-02-0      HSP90AA1
221      BIIB021  BRD-K51967704-001-03-6      AOX1 | HSP90AA1
736      KW-2478  BRD-K41213548-001-02-0      NaN
1388      XL888  BRD-A03506276-001-01-5      NaN
137      AT13387  BRD-K58435339-001-03-0      HSP90AA1
586      GANETESPIB  BRD-K38852836-001-02-1      HSP90AA1 | WT1
901      NMS-E973  BRD-K58770988-001-01-8      NaN
8      TANESPIMYCIN  BRD-K81473043-001-19-5      CHEK1 | HSP90AA1
1187      SNX 2112  BRD-K71281111-001-04-3      NaN
1188      SNX-5422  BRD-K73197500-001-02-2      HSP90AA1 | HSP90AB1
1352      VER-49009  BRD-K38548312-001-01-0      HSP90AA1
1068      PU-H71  BRD-K36529613-001-02-6      HSP90AA1

      MECHANISM_OF_ACTION  N  SPEARMAN_RHO  P_VALUE  FDR
87      HSP inhibitor  43      -0.605557  0.000017  0.023769
221      HSP inhibitor  43      -0.517970  0.000374  0.131665
736      HSP inhibitor  30      -0.422914  0.019889  0.699604
1388      HSP inhibitor  28      -0.321839  0.094891  0.997758
137      HSP inhibitor  42      -0.309942  0.045768  0.997758
586      HSP inhibitor  43      -0.264875  0.086066  0.997758
901      HSP inhibitor  30      -0.236930  0.207446  0.997758
```

8	HSP inhibitor	42	-0.191970	0.223252	0.997758
1187	HSP inhibitor	30	-0.120356	0.526397	0.997758
1188	HSP inhibitor	26	-0.037949	0.853974	0.997758
1352	HSP inhibitor	30	-0.013571	0.943259	0.997758
1068	HSP inhibitor	30	0.072747	0.702439	0.997758

```
[464]: null_distribution_df
```

```
[464]:
```

	PERMUTATION_ITERATION	PERMUTED_MEDIAN_RHO
0	1	-0.015573
1	2	0.010842
2	3	-0.016584
3	4	-0.020193
4	5	-0.000510
...	...	...
9995	9996	0.035649
9996	9997	0.005203
9997	9998	-0.000209
9998	9999	0.109677
9999	10000	-0.055162

[10000 rows x 2 columns]

```
[465]: with pd.ExcelWriter("../results/hsp90_class_enrichment/
        ↪hsp90_class_enrichment.xlsx", engine="openpyxl") as writer:
        hsp90_class_enrichment_summary.to_excel(writer,
        ↪sheet_name="Enrichment_Summary", index=False)
        frozen_prism_hsp90_drugs.to_excel(writer, sheet_name="HSP90_Inhibitors",
        ↪index=False)
        null_distribution_df.to_excel(writer, sheet_name="Null_Distribution",
        ↪index=False)
```

- hsp90_class_enrichment.xlsx exported to /results/hsp90_class_enrichment directory

7.3 3. HSP90 enrichment robustness analysis

```
[466]: valid_rec = recurrence_score.notna()
universe_cols = universe_series.index.intersection(prism_cohort.columns)

cell_line_counts = (
    prism_cohort
    .loc[valid_rec, universe_cols]
    .notna()
    .sum(axis=0)
)

universe_df = pd.DataFrame({
```

```

        "SPEARMAN_RHO": universe_series.loc[universe_cols],
        "N_CELL_LINES": cell_line_counts
    }).dropna(subset=["SPEARMAN_RHO"])

hsp90_keys = [
    f"{row['DRUG']}] (BRD:{row['BRD_ID']})"
    for _, row in prism_recurrence_drug_screen[
        prism_recurrence_drug_screen["DRUG"].
        isin(frozen_prism_hsp90_drugs["DRUG"])
    ].iterrows()
    if f"{row['DRUG']}] (BRD:{row['BRD_ID']})" in universe_df.index
]

hsp90_df = universe_df.loc[hsp90_keys].copy()
observed_hsp90_median = (hsp90_df["SPEARMAN_RHO"].median())
hsp90_coverages = (hsp90_df["N_CELL_LINES"].astype(int).to_numpy())
n_hsp90 = len(hsp90_coverages)

rhos_arr = universe_df["SPEARMAN_RHO"].to_numpy()
covs_arr = universe_df["N_CELL_LINES"].astype(int).to_numpy()

n_universe = len(universe_df)
all_indices = np.arange(n_universe)

matching_records = []

for hsp90_key, target_n in zip(
    hsp90_df.index,
    hsp90_coverages
):

    exact_pool = np.where(
        covs_arr == target_n
    )[0]

    if len(exact_pool) > 0:
        selected_pool = exact_pool
        tolerance_used = 0

    else:
        plus_minus_1_pool = np.where(
            np.abs(covs_arr - target_n) <= 1
        )[0]

        if len(plus_minus_1_pool) > 0:
            selected_pool = plus_minus_1_pool
            tolerance_used = 1

```

```

        else:
            plus_minus_2_pool = np.where(
                np.abs(covs_arr - target_n) <= 2
            )[0]

            if len(plus_minus_2_pool) > 0:
                selected_pool = plus_minus_2_pool
                tolerance_used = 2

            else:
                raise ValueError(
                    f"No comparator available for {hsp90_key} "
                    f"with N={target_n}, even with ±2 tolerance."
                )

    matching_records.append({
        "HSP90_DRUG_COHORT": hsp90_key,
        "TARGET_N_CELL_LINES": target_n,
        "MATCHING_TOLERANCE": tolerance_used,
        "N_CANDIDATE_COMPARATORS": len(selected_pool),
    })
matching_documentation = pd.DataFrame(matching_records)

candidate_pools = []

for _, row in matching_documentation.iterrows():
    target_n = row["TARGET_N_CELL_LINES"]
    tolerance = row["MATCHING_TOLERANCE"]

    pool = np.where(
        np.abs(covs_arr - target_n) <= tolerance
    )[0]

    candidate_pools.append(pool)

B = 10000
RANDOM_SEED = 42
rng = np.random.default_rng(RANDOM_SEED)

permuted_medians_matched = np.empty(B)

for i in range(B):
    sampled_idx = np.empty(n_hsp90, dtype=np.int32)
    used_mask = np.zeros(n_universe, dtype=bool)

    for k, pool in enumerate(candidate_pools):

```

```

    avail = pool[~used_mask[pool]]
    if len(avail) == 0:
        raise ValueError("Insufficient unique comparator drugs " "for_
↪matched permutation.")

    chosen = rng.choice(avail)
    used_mask[chosen] = True
    sampled_idx[k] = chosen

    permuted_medians_matched[i] = np.median(rhos_arr[sampled_idx])

k = np.sum(permuted_medians_matched <= observed_hsp90_median)
empirical_p_value_matched = ((k + 1) / (B + 1))

```

$$P_{\text{empirical}} = \frac{1 + \sum_{i=1}^B \mathbb{I}(\text{Median}_{\text{perm},i} \leq \text{Median}_{\text{obs}})}{B + 1}$$

[467]: matching_documentation

[467]:

	HSP90_DRUG_COHORT	TARGET_N_CELL_LINES \
0	ALVESPIMYCIN (BRD:BRD-K83988098-001-02-0)	43
1	BIIB021 (BRD:BRD-K51967704-001-03-6)	43
2	KW-2478 (BRD:BRD-K41213548-001-02-0)	30
3	XL888 (BRD:BRD-A03506276-001-01-5)	28
4	AT13387 (BRD:BRD-K58435339-001-03-0)	42
5	GANETESPIB (BRD:BRD-K38852836-001-02-1)	43
6	NMS-E973 (BRD:BRD-K58770988-001-01-8)	30
7	TANESPIMYCIN (BRD:BRD-K81473043-001-19-5)	42
8	SNX 2112 (BRD:BRD-K71281111-001-04-3)	30
9	SNX-5422 (BRD:BRD-K73197500-001-02-2)	26
10	VER-49009 (BRD:BRD-K38548312-001-01-0)	30
11	PU-H71 (BRD:BRD-K36529613-001-02-6)	30

	MATCHING_TOLERANCE	N_CANDIDATE_COMPARATORS
0	0	100
1	0	100
2	0	572
3	0	123
4	0	58
5	0	100
6	0	572
7	0	58
8	0	572
9	0	95
10	0	572
11	0	572

```
[468]: n_exact = (matching_documentation["MATCHING_TOLERANCE"] == 0).sum()
n_pm1 = (matching_documentation["MATCHING_TOLERANCE"] == 1).sum()
n_pm2 = (matching_documentation["MATCHING_TOLERANCE"] == 2).sum()

print(f"Exact-N matches: {n_exact}/{n_hsp90}")
print(f"±1 matches: {n_pm1}/{n_hsp90}")
print(f"±2 matches: {n_pm2}/{n_hsp90}")
print(f"Observed HSP90 median rho: "
      f"{observed_hsp90_median:.4f}")
print(f"Empirical p-value: "
      f"{empirical_p_value_matched:.6g}")
```

```
Exact-N matches: 12/12
±1 matches: 0/12
±2 matches: 0/12
Observed HSP90 median rho: -0.2509
Empirical p-value: 9.999e-05
```

```
[469]: plt.style.use(
    "seaborn-v0_8-whitegrid"
    if "seaborn-v0_8-whitegrid" in plt.style.available
    else "default"
)

fig, ax = plt.subplots(
    figsize=(9, 6),
    dpi=300
)

sns.histplot(
    permuted_medians_matched,
    bins="fd",
    color="#4A607A",
    kde=True,
    stat="density",
    alpha=0.6,
    edgecolor="white",
    label="Exact-N Matched Null Distribution (B = 10,000)",
    ax=ax
)

ax.axvline(
    x=observed_hsp90_median,
    color="#D9381E",
    linestyle="--",
    linewidth=2.5,
    label=(
        f"Observed HSP90 Median "

```

```

        f"({observed_hsp90_median:.3f})"
    )
)

ax.set_title(
    "HSP90 Drug-Class Enrichment "
    "(Exact-N Matched Permutation)",
    fontsize=13,
    fontweight="bold",
    pad=15
)

ax.set_xlabel(
    "Permuted Median Spearman Correlation ()",
    fontsize=11,
    fontweight="bold"
)

ax.set_ylabel(
    "Density",
    fontsize=11,
    fontweight="bold"
)

p_val_str = f"{empirical_p_value_matched:.5f}"

textstr = (
    f"Observed HSP90 Median : "
    f"{observed_hsp90_median:.3f}\n"
    f"Exact-N matched permutations: 10,000\n"
    f"Empirical p-value: {p_val_str}"
)

props = dict(
    boxstyle="round,pad=0.6",
    facecolor="white",
    edgecolor="#1B365D",
    alpha=0.9,
    linewidth=1.2
)

ax.text(
    0.04,
    0.93,
    textstr,
    transform=ax.transAxes,
    fontsize=10,

```



```

    fontweight="bold",
    verticalalignment="top",
    bbox=props
)

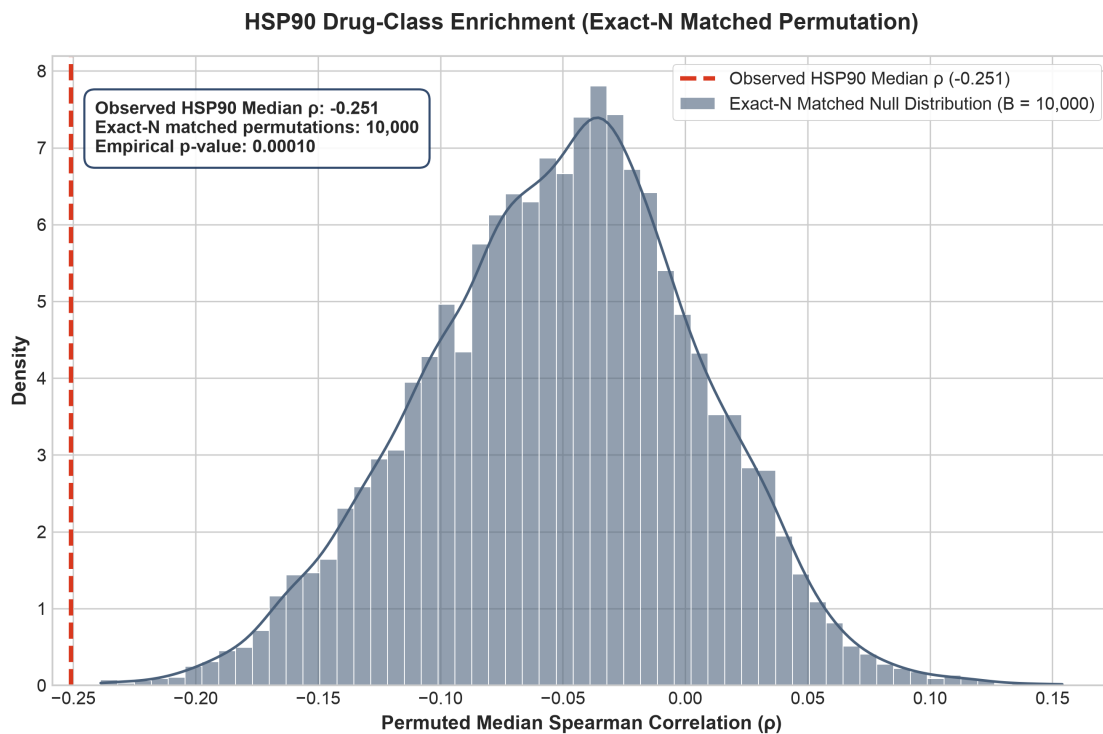
ax.legend(
    loc="upper right",
    frameon=True,
    facecolor="white",
    framealpha=0.9
)

plt.tight_layout()

plt.savefig(
    "../../figures/hsp90_class_enrichment_exactN_permutation.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



- hsp90_class_enrichment_exactN_permutation.png saved to /figures directory

7.3.1 Leave-Alvespimycin-out analysis

```
[470]: ALVESPIMYCIN_NAME = "Alvespimycin"

prism_hsp90_leave_one_out = (
    frozen_prism_hsp90_drugs[
        frozen_prism_hsp90_drugs["DRUG"] != ALVESPIMYCIN_NAME
    ]
    .copy()
)

assert ALVESPIMYCIN_NAME not in (prism_hsp90_leave_one_out["DRUG"].unique())
```

```
[471]: hsp90_leave_one_out_summary = pd.DataFrame(index=[0])
hsp90_leave_one_out_summary["N_HSP90_DRUGS"] =
    ↳ (prism_hsp90_leave_one_out["DRUG"].nunique())
hsp90_leave_one_out_summary["N_NEGATIVE_RHO"] =
    ↳ (prism_hsp90_leave_one_out["SPEARMAN_RHO"] < 0).sum()

hsp90_leave_one_out_summary["FRACTION_NEGATIVE"] = (
    hsp90_leave_one_out_summary["N_NEGATIVE_RHO"]
    / hsp90_leave_one_out_summary["N_HSP90_DRUGS"]
)

hsp90_leave_one_out_summary["MEDIAN_RHO"] = (
    prism_hsp90_leave_one_out["SPEARMAN_RHO"].median()
)
```

```
[472]: hsp90_loo_keys = [
    f"{row['DRUG']} (BRD:{row['BRD_ID']})"
    for _, row in prism_recurrence_drug_screen[
        prism_recurrence_drug_screen["DRUG"].isin(
            prism_hsp90_leave_one_out["DRUG"]
        )
    ].iterrows()
    if f"{row['DRUG']} (BRD:{row['BRD_ID']})" in universe_df.index
]

hsp90_loo_df = universe_df.loc[hsp90_loo_keys].copy()
observed_hsp90_loo_median = (hsp90_loo_df["SPEARMAN_RHO"].median())

hsp90_loo_coverages = (
    hsp90_loo_df["N_CELL_LINES"]
    .astype(int)
    .to_numpy()
)

n_hsp90_loo = len(hsp90_loo_coverages)
```

```

candidate_pools_loo = []
matching_records_loo = []

for hsp90_key, target_n in zip(
    hsp90_loo_df.index,
    hsp90_loo_coverages
):
    pool = np.where(covs_arr == target_n)[0]
    tolerance_used = 0

    if len(pool) == 0:
        pool = np.where(
            np.abs(covs_arr - target_n) <= 1)[0]
        tolerance_used = 1

    if len(pool) == 0:
        pool = np.where(np.abs(covs_arr - target_n) <= 2)[0]
        tolerance_used = 2

    if len(pool) == 0:
        raise ValueError(
            f"No comparator available for {hsp90_key} "
            f"with N={target_n}, even with ±2 tolerance."
        )
    candidate_pools_loo.append(pool)
    matching_records_loo.append({
        "HSP90_DRUG_COHORT": hsp90_key,
        "TARGET_N_CELL_LINES": target_n,
        "MATCHING_TOLERANCE": tolerance_used,
        "N_CANDIDATE_COMPARATORS": len(pool),
    })

matching_documentation_loo = pd.DataFrame(
    matching_records_loo
)

B = 10_000
RANDOM_SEED = 42

rng_loo = np.random.default_rng(RANDOM_SEED)
permuted_medians_loo = np.empty(B)

for i in range(B):
    sampled_idx = np.empty(
        n_hsp90_loo,
        dtype=np.int32
    )

```

```

    )
    used_mask = np.zeros(
        n_universe,
        dtype=bool
    )
    for j, pool in enumerate(
        candidate_pools_loo
    ):
        avail = pool[~used_mask[pool]]
        if len(avail) == 0:
            raise ValueError("Insufficient unique comparator drugs " "for_
↪leave-one-out matched permutation.")
        chosen = rng_loo.choice(avail)
        used_mask[chosen] = True
        sampled_idx[j] = chosen

        permuted_medians_loo[i] = np.median(
            rhos_arr[sampled_idx]
        )

k_loo = np.sum(permuted_medians_loo <= observed_hsp90_loo_median)
exact_n_matched_empirical_p_loo = ((k_loo + 1) / (B + 1))

hsp90_leave_one_out_summary["EXACT_N_MATCHED_EMPIRICAL_P"] =_
↪exact_n_matched_empirical_p_loo

```

```

[473]: hsp90_leave_one_out_summary["N_EXACT_N_MATCHES"] = (
        matching_documentation_loo["MATCHING_TOLERANCE"] == 0
    ).sum()

hsp90_leave_one_out_summary["N_PM1_MATCHES"] = (
        matching_documentation_loo["MATCHING_TOLERANCE"] == 1
    ).sum()

hsp90_leave_one_out_summary["N_PM2_MATCHES"] = (
        matching_documentation_loo["MATCHING_TOLERANCE"] == 2
    ).sum()

print(hsp90_leave_one_out_summary.T)

```

	0
N_HSP90_DRUGS	12.000000
N_NEGATIVE_RHO	11.000000
FRACTION_NEGATIVE	0.916667
MEDIAN_RHO	-0.250902
EXACT_N_MATCHED_EMPIRICAL_P	0.000100
N_EXACT_N_MATCHES	12.000000
N_PM1_MATCHES	0.000000

N_PM2_MATCHES 0.000000

```
[474]: output_file = "../../results/GSE186332/hsp90_leave_alvespimycin_out.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    hsp90_leave_one_out_summary.to_excel(
        writer,
        sheet_name="Summary",
        index=False
    )

    prism_hsp90_leave_one_out.to_excel(
        writer,
        sheet_name="HSP90_LeaveOut",
        index=False
    )

    matching_documentation.to_excel(
        writer,
        sheet_name="ExactN_Matching",
        index=False
    )

    permutation_null = pd.DataFrame({
        "PERMUTATION": np.arange(1, B + 1),
        "PERMUTED_MEDIAN_RHO": permuted_medians
    })

    permutation_null.to_excel(
        writer,
        sheet_name="Permutation_Null",
        index=False
    )
```

- hsp90_leave_alvespimycin_out.xlsx exported to /results/GSE186332 directory

The HSP90 class-level enrichment analysis was repeated after excluding Alvespimycin. The resulting HSP90 set contained 11 drugs, of which 10 (91.7%) had negative Spearman correlations. The median Spearman correlation was -0.25090. All 12 HSP90 inhibitors could again be matched to comparator drugs using exact-N matching; no ± 1 or ± 2 tolerance was required. Using 10,000 exact-N matched permutations (seed = 42), the empirical p-value was 0.000100. Thus, the observed HSP90 class-level enrichment remained numerically unchanged after exclusion of Alvespimycin.

7.4 4. Independent results summary of HSP90-inhibitors

```
[475]: results = []

for _, row in prism_recurrence_drug_screen.iterrows():
```

```

drug = row["DRUG"]
brd_id = row["BRD_ID"]
cohort_col = f"{drug} (BRD:{brd_id})"

if drug in frozen_prism_hsp90_drugs["DRUG"].values and cohort_col in
prism_cohort.columns:
    valid = recurrence_score.notna() & prism_cohort[cohort_col].notna()
    n_valid = valid.sum()

    if n_valid >= 3:
        rec_vals = recurrence_score.loc[valid].to_numpy()
        auc_vals = prism_cohort.loc[valid, cohort_col].to_numpy()

        if len(np.unique(rec_vals)) > 1 and len(np.unique(auc_vals)) > 1:
            rho, pval = spearmanr(rec_vals, auc_vals)
            if np.isfinite(rho):
                results.append({
                    "DRUG": drug,
                    "BRD_ID": brd_id,
                    "SPEARMAN_RHO": rho,
                    "P_VALUE": pval,
                    "N": n_valid
                })

df_hsp90 = pd.DataFrame(results)

_, qvals, _, _ = multipletests(df_hsp90["P_VALUE"], method="fdr_bh")
df_hsp90["FDR"] = qvals

df_hsp90 = df_hsp90.sort_values("SPEARMAN_RHO", ascending=True).
reset_index(drop=True)

```

```

[476]: plt.style.use("seaborn-v0_8-whitegrid" if "seaborn-v0_8-whitegrid" in plt.style.
available else "default")
fig, ax = plt.subplots(figsize=(10, 6), dpi=300)

y_positions = np.arange(len(df_hsp90))

ax.axvline(x=0, color="#555555", linestyle="--", linewidth=1.5, zorder=1)

for i, row in df_hsp90.iterrows():
    ax.plot([0, row["SPEARMAN_RHO"]], [i, i], color="#B0C4DE", linewidth=2,
zorder=2)

ax.scatter(
    df_hsp90["SPEARMAN_RHO"],
    y_positions,

```

```

        s=120,
        color="#1B365D",
        edgecolor="white",
        linewidth=1.2,
        zorder=3,
        label="HSP90 Inhibitors"
    )

ax.set_xlim(-1.0, 1.0)

for i, row in df_hsp90.iterrows():
    rho_val = row["SPEARMAN_RHO"]
    n_val = row["N"]
    fdr_val = row["FDR"]

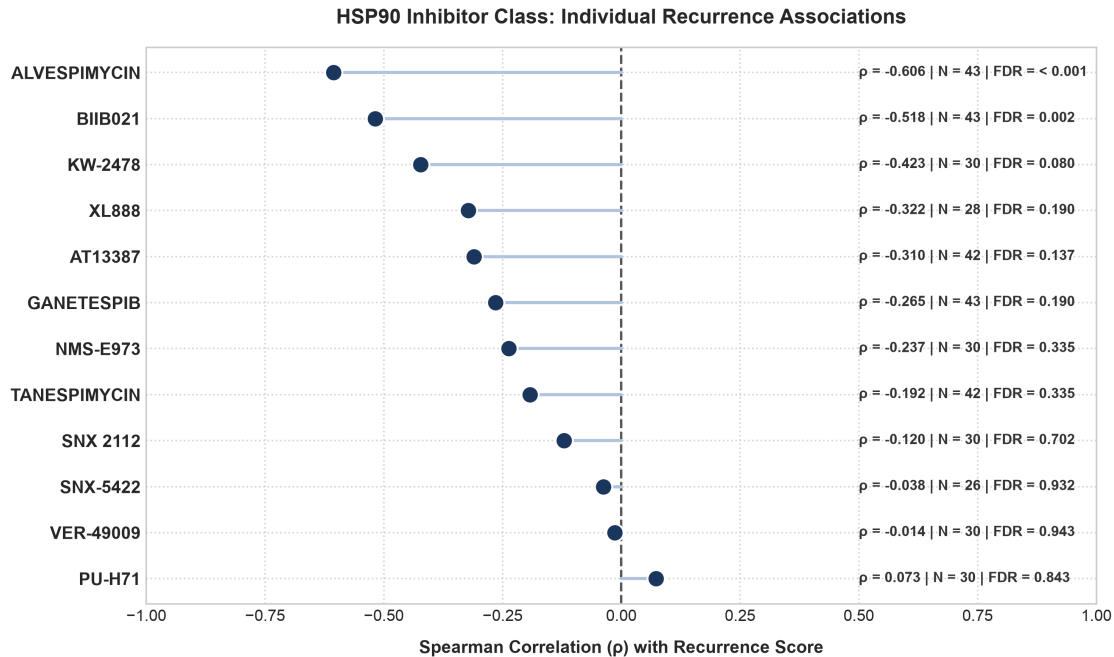
    fdr_str = f"{float(fdr_val):.3f}" if fdr_val >= 0.001 else "< 0.001"
    annotation_text = f" = {float(rho_val):.3f} | N = {n_val} | FDR = {fdr_str}"

    ax.text(
        0.5, i, annotation_text,
        va="center", ha="left", fontsize=9, fontweight="bold", color="#333333"
    )

ax.set_yticks(y_positions)
ax.set_yticklabels(df_hsp90["DRUG"], fontsize=11, fontweight="bold")
ax.invert_yaxis()
ax.grid(True, which="major", linestyle=":", linewidth=1.0, color="#CCCCCC",
        alpha=0.8, zorder=0)
ax.set_xlabel("Spearman Correlation ( ) with Recurrence Score", fontsize=11,
        fontweight="bold", labelpad=10)
ax.set_title("HSP90 Inhibitor Class: Individual Recurrence Associations",
        fontsize=13, fontweight="bold", pad=15)

plt.tight_layout()
plt.savefig("../figures/hsp90_inhibitor_recurrence_associations.png",
        dpi=300)
plt.show()

```



- hsp90_inhibitor_recurrence_associations.png saved to /figures directory

8 DepMap CRISPR - orthogonal genetic support

8.1 1. Preparing CRISPR Gene Effect

- Reference: DepMap, Broad (2026). DepMap Public 26Q1. Dataset. depmap.org
- https://depmap.org/portal/data_page/?tab=allData&releasename=DepMap%20Public%2026Q1&filename
- https://depmap.org/portal/data_page/?tab=allData&releasename=DepMap%20Public%2026Q1&filename
- CRISPR Gene Effect source:
 - Source: “DepMap Portal”
 - URL: <https://depmap.org/portal/>
 - File date: 26Q1
 - File name: CRISPRGeneEffect.csv
 - Path: /data/raw/DepMap/CRISPRGeneEffect.csv
- CRISPR Inferred Common Essentials source:
 - Source: “DepMap Portal”
 - URL: <https://depmap.org/portal/>
 - File date: 26Q1
 - File name: CRISPRInferredCommonEssentials.csv
 - Path: /data/raw/DepMap/CRISPRInferredCommonEssentials.csv

```
[477]: depmap_crispr = pd.read_csv(
    ".../data/raw/DepMap/CRISPRGeneEffect.csv",
    index_col=0
)
```



```
[478]: depmap_crispr.iloc[:5, :5]
```

```
[478]:          A1BG (1)  A1CF (29974)  A2M (2)  A2ML1 (144568)  A3GALT2 (127550)
ACH-000029  0.000151      0.000541 -0.046740      0.040447      -0.188685
ACH-000030  0.169029     -0.182773 -0.023095      0.068845      -0.037532
ACH-000074 -0.087069     -0.054984  0.211763      0.421891      0.013034
ACH-000090  0.205441     -0.138211 -0.140932     -0.139838      -0.036163
ACH-000093 -0.133302      0.086477 -0.224933      0.024604      0.267048
```

```
[479]: depmap_glioma_recurrence_scores.head()
```

```
[479]:          ModelID  N_UP_GENES  N_DOWN_GENES  UP_SCORE  DOWN_SCORE  \
CELL_LINE_NAME
YKG1          ACH-000570          162          93  0.259558    0.464742
U-87 MG       ACH-000075          162          93  0.269172    0.434810
AM-38         ACH-000269          162          93  0.261562    0.455253
SF-295        ACH-000376          162          93  0.233810    0.469597
SF-172        ACH-000887          162          93  0.301533    0.430382

          DEPMAP_RECURRENCE_SCORE
CELL_LINE_NAME
YKG1                      -0.205184
U-87 MG                   -0.165638
AM-38                     -0.193691
SF-295                    -0.235786
SF-172                    -0.128849
```

```
[480]: print(depmap_crispr.columns.tolist())
```

```
# 'HSP90AA1 (3320)'
# 'HSP90AB1 (3326)'
# 'XP01 (7514)'
```

```
['A1BG (1)', 'A1CF (29974)', 'A2M (2)', 'A2ML1 (144568)', 'A3GALT2 (127550)',
'A4GALT (53947)', 'A4GNT (51146)', 'AAAS (8086)', 'AACS (65985)', 'AADAC (13)',
'AADACL2 (344752)', 'AADACL3 (126767)', 'AADACL4 (343066)', 'AADAT (51166)',
'AAGAB (79719)', 'AAK1 (22848)', 'AAMDC (28971)', 'AAMP (14)', 'AANAT (15)',
'AAR2 (25980)', 'AARD (441376)', 'AARS1 (16)', 'AARS2 (57505)', 'AARSD1
(80755)', 'AASDH (132949)', 'AASDHPPT (60496)', 'AASS (10157)', 'AATF (26574)',
'AATK (9625)', 'ABAT (18)', 'ABCA1 (19)', 'ABCA10 (10349)', 'ABCA12 (26154)',
'ABCA13 (154664)', 'ABCA2 (20)', 'ABCA3 (21)', 'ABCA4 (24)', 'ABCA5 (23461)',
'ABCA6 (23460)', 'ABCA7 (10347)', 'ABCA8 (10351)', 'ABCA9 (10350)', 'ABCB1
(5243)', 'ABCB10 (23456)', 'ABCB11 (8647)', 'ABCB4 (5244)', 'ABCB5 (340273)',
'ABCB6 (10058)', 'ABCB7 (22)', 'ABCB8 (11194)', 'ABCB9 (23457)', 'ABCC1 (4363)',
'ABCC10 (89845)', 'ABCC11 (85320)', 'ABCC12 (94160)', 'ABCC2 (1244)', 'ABCC3
(8714)', 'ABCC4 (10257)', 'ABCC5 (10057)', 'ABCC6 (368)', 'ABCC8 (6833)', 'ABCC9
(10060)', 'ABCD1 (215)', 'ABCD2 (225)', 'ABCD3 (5825)', 'ABCD4 (5826)', 'ABCE1
(6059)', 'ABCF1 (23)', 'ABCF3 (55324)', 'ABCG1 (9619)', 'ABCG2 (9429)', 'ABCG4
(64137)', 'ABCG5 (64240)', 'ABCG8 (64241)', 'ABHD1 (84696)', 'ABHD10 (55347)']
```

'ABHD11 (83451)', 'ABHD12 (26090)', 'ABHD12B (145447)', 'ABHD13 (84945)',
 'ABHD14A (25864)', 'ABHD14B (84836)', 'ABHD15 (116236)', 'ABHD16A (7920)',
 'ABHD16B (140701)', 'ABHD17A (81926)', 'ABHD17B (51104)', 'ABHD17C (58489)',
 'ABHD18 (80167)', 'ABHD2 (11057)', 'ABHD3 (171586)', 'ABHD4 (63874)', 'ABHD5
 (51099)', 'ABHD6 (57406)', 'ABHD8 (79575)', 'ABI1 (10006)', 'ABI2 (10152)',
 'ABI3 (51225)', 'ABI3BP (25890)', 'ABITRAM (54942)', 'ABL1 (25)', 'ABL2 (27)',
 'ABLM1 (3983)', 'ABLM2 (84448)', 'ABLM3 (22885)', 'ABR (29)', 'ABRA
 (137735)', 'ABRACL (58527)', 'ABRAXAS1 (84142)', 'ABRAXAS2 (23172)', 'ABT1
 (29777)', 'ABTB1 (80325)', 'ABTB2 (25841)', 'ABTB3 (121551)', 'ACAA1 (30)',
 'ACAA2 (10449)', 'ACACA (31)', 'ACACB (32)', 'ACAD10 (80724)', 'ACAD11 (84129)',
 'ACAD8 (27034)', 'ACAD9 (28976)', 'ACADL (33)', 'ACADM (34)', 'ACADS (35)',
 'ACADSB (36)', 'ACADVL (37)', 'ACAN (176)', 'ACAP1 (9744)', 'ACAP2 (23527)',
 'ACAP3 (116983)', 'ACAT1 (38)', 'ACAT2 (39)', 'ACBD3 (64746)', 'ACBD4 (79777)',
 'ACBD5 (91452)', 'ACBD6 (84320)', 'ACBD7 (414149)', 'ACCS (84680)', 'ACCSL
 (390110)', 'ACD (65057)', 'ACE (1636)', 'ACE2 (59272)', 'ACER1 (125981)', 'ACER2
 (340485)', 'ACER3 (55331)', 'ACHE (43)', 'ACIN1 (22985)', 'ACKR1 (2532)', 'ACKR2
 (1238)', 'ACKR3 (57007)', 'ACKR4 (51554)', 'ACLY (47)', 'ACMSD (130013)', 'ACO1
 (48)', 'ACO2 (50)', 'ACOD1 (730249)', 'ACOT1 (641371)', 'ACOT11 (26027)',
 'ACOT12 (134526)', 'ACOT13 (55856)', 'ACOT2 (10965)', 'ACOT4 (122970)', 'ACOT6
 (641372)', 'ACOT7 (11332)', 'ACOT8 (10005)', 'ACOT9 (23597)', 'ACOX1 (51)',
 'ACOX2 (8309)', 'ACOX3 (8310)', 'ACOXL (55289)', 'ACP1 (52)', 'ACP2 (53)', 'ACP3
 (55)', 'ACP4 (93650)', 'ACP5 (54)', 'ACP6 (51205)', 'ACP7 (390928)', 'ACR (49)',
 'ACRBP (84519)', 'ACRV1 (56)', 'ACSBG1 (23205)', 'ACSBG2 (81616)', 'ACSF2
 (80221)', 'ACSF3 (197322)', 'ACSL1 (2180)', 'ACSL3 (2181)', 'ACSL4 (2182)',
 'ACSL5 (51703)', 'ACSL6 (23305)', 'ACSM1 (116285)', 'ACSM2A (123876)', 'ACSM2B
 (348158)', 'ACSM3 (6296)', 'ACSM4 (341392)', 'ACSM5 (54988)', 'ACSM6 (142827)',
 'ACSS1 (84532)', 'ACSS2 (55902)', 'ACSS3 (79611)', 'ACTA1 (58)', 'ACTA2 (59)',
 'ACTB (60)', 'ACTBL2 (345651)', 'ACTC1 (70)', 'ACTG1 (71)', 'ACTG2 (72)',
 'ACTL10 (170487)', 'ACTL6A (86)', 'ACTL6B (51412)', 'ACTL7A (10881)', 'ACTL7B
 (10880)', 'ACTL8 (81569)', 'ACTL9 (284382)', 'ACTMAP (284325)', 'ACTN1 (87)',
 'ACTN2 (88)', 'ACTN3 (89)', 'ACTN4 (81)', 'ACTR10 (55860)', 'ACTR1A (10121)',
 'ACTR1B (10120)', 'ACTR2 (10097)', 'ACTR3 (10096)', 'ACTR3B (57180)', 'ACTR3C
 (653857)', 'ACTR5 (79913)', 'ACTR6 (64431)', 'ACTR8 (93973)', 'ACTRT1 (139741)',
 'ACTRT2 (140625)', 'ACTRT3 (84517)', 'ACVR1 (90)', 'ACVR1B (91)', 'ACVR1C
 (130399)', 'ACVR2A (92)', 'ACVR2B (93)', 'ACVRL1 (94)', 'ACY1 (95)', 'ACY3
 (91703)', 'ACYP1 (97)', 'ACYP2 (98)', 'ADA (100)', 'ADA2 (51816)', 'ADAD1
 (132612)', 'ADAD2 (161931)', 'ADAM10 (102)', 'ADAM11 (4185)', 'ADAM12 (8038)',
 'ADAM15 (8751)', 'ADAM17 (6868)', 'ADAM18 (8749)', 'ADAM19 (8728)', 'ADAM2
 (2515)', 'ADAM20 (8748)', 'ADAM21 (8747)', 'ADAM22 (53616)', 'ADAM23 (8745)',
 'ADAM28 (10863)', 'ADAM29 (11086)', 'ADAM30 (11085)', 'ADAM32 (203102)', 'ADAM33
 (80332)', 'ADAM7 (8756)', 'ADAM8 (101)', 'ADAM9 (8754)', 'ADAMDEC1 (27299)',
 'ADAMTS1 (9510)', 'ADAMTS10 (81794)', 'ADAMTS12 (81792)', 'ADAMTS13 (11093)',
 'ADAMTS14 (140766)', 'ADAMTS15 (170689)', 'ADAMTS16 (170690)', 'ADAMTS17
 (170691)', 'ADAMTS18 (170692)', 'ADAMTS19 (171019)', 'ADAMTS2 (9509)', 'ADAMTS20
 (80070)', 'ADAMTS3 (9508)', 'ADAMTS4 (9507)', 'ADAMTS5 (11096)', 'ADAMTS6
 (11174)', 'ADAMTS7 (11173)', 'ADAMTS8 (11095)', 'ADAMTS9 (56999)', 'ADAMTSL1
 (92949)', 'ADAMTSL2 (9719)', 'ADAMTSL3 (57188)', 'ADAMTSL4 (54507)', 'ADAMTSL5
 (339366)', 'ADAP1 (11033)', 'ADAP2 (55803)', 'ADAR (103)', 'ADARB1 (104)',

'ADARB2 (105)', 'ADAT1 (23536)', 'ADAT2 (134637)', 'ADAT3 (113179)', 'ADCK1
 (57143)', 'ADCK2 (90956)', 'ADCK5 (203054)', 'ADCY1 (107)', 'ADCY10 (55811)',
 'ADCY2 (108)', 'ADCY3 (109)', 'ADCY4 (196883)', 'ADCY5 (111)', 'ADCY6 (112)',
 'ADCY7 (113)', 'ADCY8 (114)', 'ADCY9 (115)', 'ADCYAP1 (116)', 'ADCYAP1R1 (117)',
 'ADD1 (118)', 'ADD2 (119)', 'ADD3 (120)', 'ADGRA1 (84435)', 'ADGRA2 (25960)',
 'ADGRA3 (166647)', 'ADGRB1 (575)', 'ADGRB2 (576)', 'ADGRB3 (577)', 'ADGRD1
 (283383)', 'ADGRD2 (347088)', 'ADGRE1 (2015)', 'ADGRE2 (30817)', 'ADGRE3
 (84658)', 'ADGRE5 (976)', 'ADGRF1 (266977)', 'ADGRF3 (165082)', 'ADGRF4
 (221393)', 'ADGRF5 (221395)', 'ADGRG1 (9289)', 'ADGRG2 (10149)', 'ADGRG3
 (222487)', 'ADGRG4 (139378)', 'ADGRG5 (221188)', 'ADGRG6 (57211)', 'ADGRG7
 (84873)', 'ADGRL1 (22859)', 'ADGRL2 (23266)', 'ADGRL3 (23284)', 'ADGRL4
 (64123)', 'ADGRV1 (84059)', 'ADH1A (124)', 'ADH1B (125)', 'ADH1C (126)', 'ADH4
 (127)', 'ADH5 (128)', 'ADH6 (130)', 'ADH7 (131)', 'ADHFE1 (137872)', 'ADI1
 (55256)', 'ADIG (149685)', 'ADIPOQ (9370)', 'ADIPOR1 (51094)', 'ADIPOR2
 (79602)', 'ADIRF (10974)', 'ADISSP (54976)', 'ADK (132)', 'ADM (133)', 'ADM2
 (79924)', 'ADM5 (199800)', 'ADNP (23394)', 'ADNP2 (22850)', 'ADO (84890)',
 'ADORA1 (134)', 'ADORA2A (135)', 'ADORA2B (136)', 'ADORA3 (140)', 'ADPGK
 (83440)', 'ADPRH (141)', 'ADPRHL1 (113622)', 'ADPRM (56985)', 'ADPRS (54936)',
 'ADRA1A (148)', 'ADRA1B (147)', 'ADRA1D (146)', 'ADRA2A (150)', 'ADRA2B (151)',
 'ADRA2C (152)', 'ADRB1 (153)', 'ADRB2 (154)', 'ADRB3 (155)', 'ADRM1 (11047)',
 'ADSL (158)', 'ADSS1 (122622)', 'ADSS2 (159)', 'ADTRP (84830)', 'AEBP1 (165)',
 'AEBP2 (121536)', 'AEN (64782)', 'AFAP1 (60312)', 'AFAP1L1 (134265)', 'AFAP1L2
 (84632)', 'AFDN (4301)', 'AFF1 (4299)', 'AFF2 (2334)', 'AFF3 (3899)', 'AFF4
 (27125)', 'AFG1L (246269)', 'AFG2A (166378)', 'AFG2B (79029)', 'AFG3L2 (10939)',
 'AFM (173)', 'AFMID (125061)', 'AFP (174)', 'AFTPH (54812)', 'AGA (175)', 'AGAP1
 (116987)', 'AGAP2 (116986)', 'AGAP3 (116988)', 'AGAP5 (729092)', 'AGAP6
 (414189)', 'AGAP9 (642517)', 'AGBL1 (123624)', 'AGBL2 (79841)', 'AGBL3
 (340351)', 'AGBL4 (84871)', 'AGBL5 (60509)', 'AGER (177)', 'AGFG1 (3267)',
 'AGFG2 (3268)', 'AGGF1 (55109)', 'AGK (55750)', 'AGL (178)', 'AGMAT (79814)',
 'AGMO (392636)', 'AGO1 (26523)', 'AGO2 (27161)', 'AGO3 (192669)', 'AGO4
 (192670)', 'AGPAT1 (10554)', 'AGPAT2 (10555)', 'AGPAT3 (56894)', 'AGPAT4
 (56895)', 'AGPAT5 (55326)', 'AGPS (8540)', 'AGR2 (10551)', 'AGR3 (155465)',
 'AGRN (375790)', 'AGRP (181)', 'AGT (183)', 'AGTPBP1 (23287)', 'AGTR1 (185)',
 'AGTR2 (186)', 'AGTRAP (57085)', 'AGXT (189)', 'AGXT2 (64902)', 'AHCTF1
 (25909)', 'AHCY (191)', 'AHCYL1 (10768)', 'AHCYL2 (23382)', 'AHDC1 (27245)',
 'AHI1 (54806)', 'AHNAK (79026)', 'AHNAK2 (113146)', 'AHR (196)', 'AHRR (57491)',
 'AHS1 (10598)', 'AHS2 (197)', 'AHSP (51327)', 'AICDA (57379)', 'AIDA (64853)',
 'AIF1 (199)', 'AIF1L (83543)', 'AIFM1 (9131)', 'AIFM2 (84883)', 'AIFM3
 (150209)', 'AIG1 (51390)', 'AIM2 (9447)', 'AIMP1 (9255)', 'AIMP2 (7965)', 'AIP
 (9049)', 'AIPL1 (23746)', 'AIRE (326)', 'AIRIM (54955)', 'AJAP1 (55966)', 'AJM1
 (389813)', 'AJUBA (84962)', 'AK1 (203)', 'AK2 (204)', 'AK3 (50808)', 'AK4
 (205)', 'AK5 (26289)', 'AK6 (102157402)', 'AK7 (122481)', 'AK8 (158067)', 'AK9
 (221264)', 'AKAIN1 (642597)', 'AKAP1 (8165)', 'AKAP10 (11216)', 'AKAP11
 (11215)', 'AKAP12 (9590)', 'AKAP13 (11214)', 'AKAP14 (158798)', 'AKAP17A
 (8227)', 'AKAP3 (10566)', 'AKAP4 (8852)', 'AKAP5 (9495)', 'AKAP6 (9472)', 'AKAP7
 (9465)', 'AKAP8 (10270)', 'AKAP8L (26993)', 'AKAP9 (10142)', 'AKIP1 (56672)',
 'AKIRIN1 (79647)', 'AKIRIN2 (55122)', 'AKNA (80709)', 'AKNAD1 (254268)', 'AKR1A1
 (10327)', 'AKR1B1 (231)', 'AKR1B10 (57016)', 'AKR1B15 (441282)', 'AKR1C2

(1646)', 'AKR1C3 (8644)', 'AKR1C4 (1109)', 'AKR1D1 (6718)', 'AKR1E2 (83592)',
 'AKR7A2 (8574)', 'AKR7A3 (22977)', 'AKT1 (207)', 'AKT1S1 (84335)', 'AKT2 (208)',
 'AKT3 (10000)', 'AKTIP (64400)', 'ALAD (210)', 'ALAS1 (211)', 'ALAS2 (212)',
 'ALB (213)', 'ALCAM (214)', 'ALDH16A1 (126133)', 'ALDH18A1 (5832)', 'ALDH1A1
 (216)', 'ALDH1A2 (8854)', 'ALDH1A3 (220)', 'ALDH1B1 (219)', 'ALDH1L1 (10840)',
 'ALDH1L2 (160428)', 'ALDH2 (217)', 'ALDH3A1 (218)', 'ALDH3A2 (224)', 'ALDH3B1
 (221)', 'ALDH3B2 (222)', 'ALDH4A1 (8659)', 'ALDH5A1 (7915)', 'ALDH6A1 (4329)',
 'ALDH7A1 (501)', 'ALDH8A1 (64577)', 'ALDH9A1 (223)', 'ALDOA (226)', 'ALDOB
 (229)', 'ALDOC (230)', 'ALG1 (56052)', 'ALG10 (84920)', 'ALG10B (144245)',
 'ALG11 (440138)', 'ALG12 (79087)', 'ALG13 (79868)', 'ALG14 (199857)', 'ALG2
 (85365)', 'ALG3 (10195)', 'ALG5 (29880)', 'ALG6 (29929)', 'ALG8 (79053)', 'ALG9
 (79796)', 'ALK (238)', 'ALKAL1 (389658)', 'ALKAL2 (285016)', 'ALKBH1 (8846)',
 'ALKBH2 (121642)', 'ALKBH3 (221120)', 'ALKBH4 (54784)', 'ALKBH5 (54890)',
 'ALKBH6 (84964)', 'ALKBH7 (84266)', 'ALKBH8 (91801)', 'ALLC (55821)', 'ALMS1
 (7840)', 'ALOX12 (239)', 'ALOX12B (242)', 'ALOX15 (246)', 'ALOX15B (247)',
 'ALOX5 (240)', 'ALOX5AP (241)', 'ALOXE3 (59344)', 'ALPG (251)', 'ALPI (248)',
 'ALPK1 (80216)', 'ALPK2 (115701)', 'ALPK3 (57538)', 'ALPL (249)', 'ALPP (250)',
 'ALS2 (57679)', 'ALS2CL (259173)', 'ALX1 (8092)', 'ALX3 (257)', 'ALX4 (60529)',
 'ALYREF (10189)', 'AMACR (23600)', 'AMBN (258)', 'AMBP (259)', 'AMBRA1 (55626)',
 'AMD1 (262)', 'AMDHD1 (144193)', 'AMDHD2 (51005)', 'AMELX (265)', 'AMELY (266)',
 'AMER1 (139285)', 'AMER2 (219287)', 'AMER3 (205147)', 'AMFR (267)', 'AMH (268)',
 'AMHR2 (269)', 'AMIGO1 (57463)', 'AMIGO2 (347902)', 'AMIGO3 (386724)', 'AMMECR1
 (9949)', 'AMMECR1L (83607)', 'AMN (81693)', 'AMN1 (196394)', 'AMOT (154796)',
 'AMOTL1 (154810)', 'AMOTL2 (51421)', 'AMPD1 (270)', 'AMPD2 (271)', 'AMPD3
 (272)', 'AMPH (273)', 'AMT (275)', 'AMTN (401138)', 'AMY2A (279)', 'AMZ1
 (155185)', 'AMZ2 (51321)', 'ANAPC1 (64682)', 'ANAPC10 (10393)', 'ANAPC11
 (51529)', 'ANAPC13 (25847)', 'ANAPC15 (25906)', 'ANAPC16 (119504)', 'ANAPC2
 (29882)', 'ANAPC4 (29945)', 'ANAPC5 (51433)', 'ANAPC7 (51434)', 'ANG (283)',
 'ANGEL1 (23357)', 'ANGEL2 (90806)', 'ANGPT1 (284)', 'ANGPT2 (285)', 'ANGPT4
 (51378)', 'ANGPTL1 (9068)', 'ANGPTL2 (23452)', 'ANGPTL3 (27329)', 'ANGPTL4
 (51129)', 'ANGPTL5 (253935)', 'ANGPTL6 (83854)', 'ANGPTL7 (10218)', 'ANGPTL8
 (55908)', 'ANHx (647589)', 'ANK1 (286)', 'ANK2 (287)', 'ANK3 (288)', 'ANKAR
 (150709)', 'ANKDD1A (348094)', 'ANKDD1B (728780)', 'ANKEF1 (63926)', 'ANKFN1
 (162282)', 'ANKFY1 (51479)', 'ANKH (56172)', 'ANKHD1 (54882)', 'ANKIB1 (54467)',
 'ANKK1 (255239)', 'ANKLE1 (126549)', 'ANKLE2 (23141)', 'ANKMY1 (51281)', 'ANKMY2
 (57037)', 'ANKRA2 (57763)', 'ANKRD1 (27063)', 'ANKRD10 (55608)', 'ANKRD11
 (29123)', 'ANKRD12 (23253)', 'ANKRD13A (88455)', 'ANKRD13B (124930)', 'ANKRD13C
 (81573)', 'ANKRD13D (338692)', 'ANKRD16 (54522)', 'ANKRD17 (26057)', 'ANKRD18A
 (253650)', 'ANKRD18B (441459)', 'ANKRD2 (26287)', 'ANKRD22 (118932)', 'ANKRD23
 (200539)', 'ANKRD24 (170961)', 'ANKRD26 (22852)', 'ANKRD27 (84079)', 'ANKRD28
 (23243)', 'ANKRD29 (147463)', 'ANKRD30A (91074)', 'ANKRD30B (374860)',
 'ANKRD30BL (554226)', 'ANKRD31 (256006)', 'ANKRD33 (341405)', 'ANKRD33B
 (651746)', 'ANKRD34A (284615)', 'ANKRD34B (340120)', 'ANKRD34C (390616)',
 'ANKRD35 (148741)', 'ANKRD36 (375248)', 'ANKRD36B (57730)', 'ANKRD36C (400986)',
 'ANKRD37 (353322)', 'ANKRD39 (51239)', 'ANKRD40 (91369)', 'ANKRD40CL (55018)',
 'ANKRD42 (338699)', 'ANKRD44 (91526)', 'ANKRD45 (339416)', 'ANKRD46 (157567)',
 'ANKRD49 (54851)', 'ANKRD50 (57182)', 'ANKRD52 (283373)', 'ANKRD53 (79998)',
 'ANKRD54 (129138)', 'ANKRD55 (79722)', 'ANKRD6 (22881)', 'ANKRD60 (140731)',

'ANKRD61 (100310846)', 'ANKRD62 (342850)', 'ANKRD63 (100131244)', 'ANKRD65
 (441869)', 'ANKRD66 (100287718)', 'ANKRD7 (56311)', 'ANKRD9 (122416)', 'ANKS1A
 (23294)', 'ANKS1B (56899)', 'ANKS3 (124401)', 'ANKS4B (257629)', 'ANKS6
 (203286)', 'ANKUB1 (389161)', 'ANKZF1 (55139)', 'ANLN (54443)', 'ANO1 (55107)',
 'ANO10 (55129)', 'ANO2 (57101)', 'ANO3 (63982)', 'ANO4 (121601)', 'ANO5
 (203859)', 'ANO6 (196527)', 'ANO7 (50636)', 'ANO8 (57719)', 'ANO9 (338440)',
 'ANOS1 (3730)', 'ANP32A (8125)', 'ANP32B (10541)', 'ANP32D (23519)', 'ANP32E
 (81611)', 'ANPEP (290)', 'ANTKMT (65990)', 'ANTXR1 (84168)', 'ANTXR2 (118429)',
 'ANTXRL (195977)', 'ANXA1 (301)', 'ANXA10 (11199)', 'ANXA11 (311)', 'ANXA13
 (312)', 'ANXA2 (302)', 'ANXA2R (389289)', 'ANXA3 (306)', 'ANXA4 (307)', 'ANXA5
 (308)', 'ANXA6 (309)', 'ANXA7 (310)', 'ANXA8 (653145)', 'ANXA8L1 (728113)',
 'ANXA9 (8416)', 'AOAH (313)', 'AOC1 (26)', 'AOC2 (314)', 'AOC3 (8639)', 'AOPEP
 (84909)', 'AOX1 (316)', 'AP1AR (55435)', 'AP1B1 (162)', 'AP1G1 (164)', 'AP1G2
 (8906)', 'AP1M1 (8907)', 'AP1M2 (10053)', 'AP1S1 (1174)', 'AP1S2 (8905)', 'AP1S3
 (130340)', 'AP2A1 (160)', 'AP2A2 (161)', 'AP2B1 (163)', 'AP2M1 (1173)', 'AP2S1
 (1175)', 'AP3B1 (8546)', 'AP3B2 (8120)', 'AP3D1 (8943)', 'AP3M1 (26985)', 'AP3M2
 (10947)', 'AP3S1 (1176)', 'AP3S2 (10239)', 'AP4B1 (10717)', 'AP4E1 (23431)',
 'AP4M1 (9179)', 'AP4S1 (11154)', 'AP5B1 (91056)', 'AP5M1 (55745)', 'AP5S1
 (55317)', 'AP5Z1 (9907)', 'APAF1 (317)', 'APBA1 (320)', 'APBA2 (321)', 'APBA3
 (9546)', 'APBB1 (322)', 'APBB1IP (54518)', 'APBB2 (323)', 'APBB3 (10307)', 'APC
 (324)', 'APC2 (10297)', 'APCDD1 (147495)', 'APCDD1L (164284)', 'APCS (325)',
 'APEH (327)', 'APELA (100506013)', 'APEX1 (328)', 'APEX2 (27301)', 'APH1A
 (51107)', 'APH1B (83464)', 'API5 (8539)', 'APIP (51074)', 'APLF (200558)', 'APLN
 (8862)', 'APLNR (187)', 'APLP1 (333)', 'APLP2 (334)', 'APMAP (57136)', 'APOA1
 (335)', 'APOA2 (336)', 'APOA4 (337)', 'APOA5 (116519)', 'APOB (338)', 'APOBEC1
 (339)', 'APOBEC2 (10930)', 'APOBEC3A (200315)', 'APOBEC3B (9582)', 'APOBEC3C
 (27350)', 'APOBEC3D (140564)', 'APOBEC3F (200316)', 'APOBEC3G (60489)',
 'APOBEC3H (164668)', 'APOBEC4 (403314)', 'APOBR (55911)', 'APOC1 (341)', 'APOC2
 (344)', 'APOC3 (345)', 'APOC4 (346)', 'APOD (347)', 'APOE (348)', 'APOF (319)',
 'APOH (350)', 'APOL1 (8542)', 'APOL2 (23780)', 'APOL3 (80833)', 'APOL4 (80832)',
 'APOL5 (80831)', 'APOL6 (80830)', 'APOLD1 (81575)', 'APOM (55937)', 'APOO
 (79135)', 'APOOL (139322)', 'APP (351)', 'APPBP2 (10513)', 'APPL1 (26060)',
 'APPL2 (55198)', 'APRT (353)', 'APTX (54840)', 'AQP1 (358)', 'AQP10 (89872)',
 'AQP11 (282679)', 'AQP12B (653437)', 'AQP2 (359)', 'AQP3 (360)', 'AQP4 (361)',
 'AQP5 (362)', 'AQP6 (363)', 'AQP7 (364)', 'AQP7B (100509620)', 'AQP8 (343)',
 'AQP9 (366)', 'AQR (9716)', 'AR (367)', 'ARAF (369)', 'ARAP1 (116985)', 'ARAP2
 (116984)', 'ARAP3 (64411)', 'ARB2A (83989)', 'ARC (23237)', 'ARCN1 (372)', 'AREG
 (374)', 'AREL1 (9870)', 'ARF1 (375)', 'ARF3 (377)', 'ARF4 (378)', 'ARF5 (381)',
 'ARF6 (382)', 'ARFGAP1 (55738)', 'ARFGAP2 (84364)', 'ARFGAP3 (26286)', 'ARFGEF1
 (10565)', 'ARFGEF2 (10564)', 'ARFGEF3 (57221)', 'ARFIP1 (27236)', 'ARFIP2
 (23647)', 'ARFRP1 (10139)', 'ARG1 (383)', 'ARG2 (384)', 'ARGFX (503582)',
 'ARGLU1 (55082)', 'ARHGAP1 (392)', 'ARHGAP10 (79658)', 'ARHGAP11A (9824)',
 'ARHGAP12 (94134)', 'ARHGAP15 (55843)', 'ARHGAP17 (55114)', 'ARHGAP18 (93663)',
 'ARHGAP19 (84986)', 'ARHGAP20 (57569)', 'ARHGAP21 (57584)', 'ARHGAP22 (58504)',
 'ARHGAP23 (57636)', 'ARHGAP24 (83478)', 'ARHGAP25 (9938)', 'ARHGAP26 (23092)',
 'ARHGAP27 (201176)', 'ARHGAP28 (79822)', 'ARHGAP29 (9411)', 'ARHGAP30 (257106)',
 'ARHGAP31 (57514)', 'ARHGAP32 (9743)', 'ARHGAP33 (115703)', 'ARHGAP35 (2909)',
 'ARHGAP36 (158763)', 'ARHGAP39 (80728)', 'ARHGAP4 (393)', 'ARHGAP44 (9912)',

'ARHGAP45 (23526)', 'ARHGAP5 (394)', 'ARHGAP6 (395)', 'ARHGAP8 (23779)',
 'ARHGAP9 (64333)', 'ARHGDIA (396)', 'ARHGDIB (397)', 'ARHGDIG (398)', 'ARHGEF1
 (9138)', 'ARHGEF10 (9639)', 'ARHGEF10L (55160)', 'ARHGEF11 (9826)', 'ARHGEF12
 (23365)', 'ARHGEF15 (22899)', 'ARHGEF16 (27237)', 'ARHGEF17 (9828)', 'ARHGEF18
 (23370)', 'ARHGEF19 (128272)', 'ARHGEF2 (9181)', 'ARHGEF25 (115557)', 'ARHGEF26
 (26084)', 'ARHGEF28 (64283)', 'ARHGEF3 (50650)', 'ARHGEF33 (100271715)',
 'ARHGEF35 (445328)', 'ARHGEF37 (389337)', 'ARHGEF38 (54848)', 'ARHGEF39
 (84904)', 'ARHGEF4 (50649)', 'ARHGEF40 (55701)', 'ARHGEF5 (7984)', 'ARHGEF6
 (9459)', 'ARHGEF7 (8874)', 'ARHGEF9 (23229)', 'ARID1A (8289)', 'ARID1B (57492)',
 'ARID2 (196528)', 'ARID3A (1820)', 'ARID3B (10620)', 'ARID3C (138715)', 'ARID4A
 (5926)', 'ARID4B (51742)', 'ARID5A (10865)', 'ARID5B (84159)', 'ARIH1 (25820)',
 'ARIH2 (10425)', 'ARK2C (494470)', 'ARK2N (147339)', 'ARL1 (400)', 'ARL10
 (285598)', 'ARL11 (115761)', 'ARL13A (392509)', 'ARL13B (200894)', 'ARL14
 (80117)', 'ARL14EP (120534)', 'ARL14EPL (644100)', 'ARL15 (54622)', 'ARL16
 (339231)', 'ARL2 (402)', 'ARL2BP (23568)', 'ARL3 (403)', 'ARL4A (10124)', 'ARL4C
 (10123)', 'ARL4D (379)', 'ARL5A (26225)', 'ARL5B (221079)', 'ARL5C (390790)',
 'ARL6 (84100)', 'ARL6IP1 (23204)', 'ARL6IP4 (51329)', 'ARL6IP5 (10550)',
 'ARL6IP6 (151188)', 'ARL8A (127829)', 'ARL8B (55207)', 'ARL9 (132946)', 'ARLN
 (401152)', 'ARMC1 (55156)', 'ARMC10 (83787)', 'ARMC12 (221481)', 'ARMC2
 (84071)', 'ARMC3 (219681)', 'ARMC5 (79798)', 'ARMC6 (93436)', 'ARMC7 (79637)',
 'ARMC8 (25852)', 'ARMC9 (80210)', 'ARMCX1 (51309)', 'ARMCX2 (9823)', 'ARMCX3
 (51566)', 'ARMCX4 (100131755)', 'ARMCX5 (64860)', 'ARMCX6 (54470)', 'ARMH1
 (339541)', 'ARMH2 (101928603)', 'ARMH3 (79591)', 'ARMH4 (145407)', 'ARMS2
 (387715)', 'ARMT1 (79624)', 'ARNT (405)', 'ARNT2 (9915)', 'ARPC1A (10552)',
 'ARPC1B (10095)', 'ARPC2 (10109)', 'ARPC3 (10094)', 'ARPC4 (10093)', 'ARPC5
 (10092)', 'ARPC5L (81873)', 'ARPIN (348110)', 'ARPP19 (10776)', 'ARPP21
 (10777)', 'ARR3 (407)', 'ARRB1 (408)', 'ARRB2 (409)', 'ARRDC1 (92714)', 'ARRDC2
 (27106)', 'ARRDC3 (57561)', 'ARRDC4 (91947)', 'ARRDC5 (645432)', 'ARSA (410)',
 'ARSB (411)', 'ARSD (414)', 'ARSF (416)', 'ARSG (22901)', 'ARSH (347527)', 'ARSI
 (340075)', 'ARSJ (79642)', 'ARSK (153642)', 'ARSL (415)', 'ART1 (417)', 'ART3
 (419)', 'ART4 (420)', 'ART5 (116969)', 'ARTN (9048)', 'ARV1 (64801)', 'ARVCF
 (421)', 'ARX (170302)', 'AS3MT (57412)', 'ASAH1 (427)', 'ASAH2 (56624)', 'ASAP1
 (50807)', 'ASAP2 (8853)', 'ASAP3 (55616)', 'ASB1 (51665)', 'ASB10 (136371)',
 'ASB11 (140456)', 'ASB12 (142689)', 'ASB13 (79754)', 'ASB14 (142686)', 'ASB15
 (142685)', 'ASB16 (92591)', 'ASB17 (127247)', 'ASB18 (401036)', 'ASB2 (51676)',
 'ASB3 (51130)', 'ASB4 (51666)', 'ASB5 (140458)', 'ASB6 (140459)', 'ASB7
 (140460)', 'ASB8 (140461)', 'ASB9 (140462)', 'ASCC1 (51008)', 'ASCC2 (84164)',
 'ASCC3 (10973)', 'ASCL1 (429)', 'ASCL2 (430)', 'ASCL3 (56676)', 'ASCL4
 (121549)', 'ASCL5 (647219)', 'ASDURF (110599588)', 'ASF1A (25842)', 'ASF1B
 (55723)', 'ASGR1 (432)', 'ASGR2 (433)', 'ASH1L (55870)', 'ASH2L (9070)', 'ASIC1
 (41)', 'ASIC2 (40)', 'ASIC3 (9311)', 'ASIC4 (55515)', 'ASIC5 (51802)', 'ASIP
 (434)', 'ASL (435)', 'ASMT (438)', 'ASMTL (8623)', 'ASNS (440)', 'ASNSD1
 (54529)', 'ASPA (443)', 'ASPDH (554235)', 'ASPG (374569)', 'ASPH (444)', 'ASPHD1
 (253982)', 'ASPHD2 (57168)', 'ASPM (259266)', 'ASPRV1 (151516)', 'ASPSCR1
 (79058)', 'ASRGL1 (80150)', 'ASS1 (445)', 'ASTE1 (28990)', 'ASTL (431705)',
 'ASTN1 (460)', 'ASTN2 (23245)', 'ASXL1 (171023)', 'ASXL2 (55252)', 'ASXL3
 (80816)', 'ASZ1 (136991)', 'ATAD1 (84896)', 'ATAD2 (29028)', 'ATAD2B (54454)',
 'ATAD3A (55210)', 'ATAD3B (83858)', 'ATAD3C (219293)', 'ATAD5 (79915)', 'ATAT1

(79969)', 'ATCAY (85300)', 'ATE1 (11101)', 'ATF1 (466)', 'ATF2 (1386)', 'ATF3 (467)', 'ATF4 (468)', 'ATF5 (22809)', 'ATF6 (22926)', 'ATF6B (1388)', 'ATF7 (11016)', 'ATF7IP (55729)', 'ATF7IP2 (80063)', 'ATG10 (83734)', 'ATG101 (60673)', 'ATG12 (9140)', 'ATG13 (9776)', 'ATG14 (22863)', 'ATG16L1 (55054)', 'ATG16L2 (89849)', 'ATG2A (23130)', 'ATG2B (55102)', 'ATG3 (64422)', 'ATG4A (115201)', 'ATG4B (23192)', 'ATG4C (84938)', 'ATG4D (84971)', 'ATG5 (9474)', 'ATG7 (10533)', 'ATG9A (79065)', 'ATG9B (285973)', 'ATIC (471)', 'ATL1 (51062)', 'ATL2 (64225)', 'ATL3 (25923)', 'ATM (472)', 'ATMIN (23300)', 'ATN1 (1822)', 'ATOH1 (474)', 'ATOH7 (220202)', 'ATOH8 (84913)', 'ATOSA (56204)', 'ATOSB (80256)', 'ATOX1 (475)', 'ATP10A (57194)', 'ATP10B (23120)', 'ATP10D (57205)', 'ATP11A (23250)', 'ATP11B (23200)', 'ATP11C (286410)', 'ATP12A (479)', 'ATP13A1 (57130)', 'ATP13A2 (23400)', 'ATP13A3 (79572)', 'ATP13A4 (84239)', 'ATP13A5 (344905)', 'ATP1A1 (476)', 'ATP1A2 (477)', 'ATP1A3 (478)', 'ATP1A4 (480)', 'ATP1B1 (481)', 'ATP1B2 (482)', 'ATP1B3 (483)', 'ATP1B4 (23439)', 'ATP23 (91419)', 'ATP2A1 (487)', 'ATP2A2 (488)', 'ATP2A3 (489)', 'ATP2B1 (490)', 'ATP2B2 (491)', 'ATP2B3 (492)', 'ATP2B4 (493)', 'ATP2C1 (27032)', 'ATP2C2 (9914)', 'ATP4A (495)', 'ATP4B (496)', 'ATP5F1A (498)', 'ATP5F1B (506)', 'ATP5F1C (509)', 'ATP5F1D (513)', 'ATP5F1E (514)', 'ATP5IF1 (93974)', 'ATP5MC1 (516)', 'ATP5MC2 (517)', 'ATP5MC3 (518)', 'ATP5ME (521)', 'ATP5MF (9551)', 'ATP5MG (10632)', 'ATP5MGL (267020)', 'ATP5MJ (9556)', 'ATP5MK (84833)', 'ATP5PB (515)', 'ATP5PD (10476)', 'ATP5PF (522)', 'ATP5PO (539)', 'ATP6AP1 (537)', 'ATP6AP2 (10159)', 'ATP6VOA1 (535)', 'ATP6VOA2 (23545)', 'ATP6VOA4 (50617)', 'ATP6VOB (533)', 'ATP6VOC (527)', 'ATP6VOD1 (9114)', 'ATP6VOD2 (245972)', 'ATP6VOE1 (8992)', 'ATP6VOE2 (155066)', 'ATP6V1A (523)', 'ATP6V1B1 (525)', 'ATP6V1B2 (526)', 'ATP6V1C1 (528)', 'ATP6V1C2 (245973)', 'ATP6V1D (51382)', 'ATP6V1E1 (529)', 'ATP6V1E2 (90423)', 'ATP6V1F (9296)', 'ATP6V1G1 (9550)', 'ATP6V1G2 (534)', 'ATP6V1G3 (127124)', 'ATP6V1H (51606)', 'ATP7A (538)', 'ATP7B (540)', 'ATP8A1 (10396)', 'ATP8A2 (51761)', 'ATP8B1 (5205)', 'ATP8B2 (57198)', 'ATP8B3 (148229)', 'ATP8B4 (79895)', 'ATP9A (10079)', 'ATP9B (374868)', 'ATPAF1 (64756)', 'ATPAF2 (91647)', 'ATPSCKMT (134145)', 'ATR (545)', 'ATRAID (51374)', 'ATRIP (84126)', 'ATRN (8455)', 'ATRN1 (26033)', 'ATRX (546)', 'ATXN1 (6310)', 'ATXN10 (25814)', 'ATXN1L (342371)', 'ATXN2 (6311)', 'ATXN2L (11273)', 'ATXN3 (4287)', 'ATXN3L (92552)', 'ATXN7 (6314)', 'ATXN7L1 (222255)', 'ATXN7L2 (127002)', 'ATXN7L3 (56970)', 'ATXN7L3B (552889)', 'AUH (549)', 'AUNIP (79000)', 'AUP1 (550)', 'AURKA (6790)', 'AURKAIP1 (54998)', 'AURKB (9212)', 'AURKC (6795)', 'AUTS2 (26053)', 'AVEN (57099)', 'AVIL (10677)', 'AVL9 (23080)', 'AVP (551)', 'AVPI1 (60370)', 'AVPR1A (552)', 'AVPR1B (553)', 'AVPR2 (554)', 'AWAT1 (158833)', 'AWAT2 (158835)', 'AXDND1 (126859)', 'AXIN1 (8312)', 'AXIN2 (8313)', 'AXL (558)', 'AZGP1 (563)', 'AZI2 (64343)', 'AZIN1 (51582)', 'AZIN2 (113451)', 'AZU1 (566)', 'B2M (567)', 'B3GALNT1 (8706)', 'B3GALNT2 (148789)', 'B3GALT1 (8708)', 'B3GALT2 (8707)', 'B3GALT4 (8705)', 'B3GALT5 (10317)', 'B3GALT6 (126792)', 'B3GAT1 (27087)', 'B3GAT2 (135152)', 'B3GAT3 (26229)', 'B3GLCT (145173)', 'B3GNT2 (10678)', 'B3GNT3 (10331)', 'B3GNT4 (79369)', 'B3GNT5 (84002)', 'B3GNT6 (192134)', 'B3GNT7 (93010)', 'B3GNT8 (374907)', 'B3GNT9 (84752)', 'B3GNTL1 (146712)', 'B4GALNT1 (2583)', 'B4GALNT2 (124872)', 'B4GALNT3 (283358)', 'B4GALNT4 (338707)', 'B4GALT1 (2683)', 'B4GALT2 (8704)', 'B4GALT3 (8703)', 'B4GALT4 (8702)', 'B4GALT5 (9334)', 'B4GALT6 (9331)', 'B4GALT7 (11285)', 'B4GAT1 (11041)', 'B9D1 (27077)', 'B9D2 (80776)', 'BAALC (79870)',

'BAAT (570)', 'BABAM1 (29086)', 'BABAM2 (9577)', 'BACC1 (124944)', 'BACE1
 (23621)', 'BACE2 (25825)', 'BACH1 (571)', 'BACH2 (60468)', 'BAD (572)', 'BAG1
 (573)', 'BAG2 (9532)', 'BAG3 (9531)', 'BAG4 (9530)', 'BAG5 (9529)', 'BAG6
 (7917)', 'BAHCC1 (57597)', 'BAHD1 (22893)', 'BAIAP2 (10458)', 'BAIAP2L1
 (55971)', 'BAIAP2L2 (80115)', 'BAIAP3 (8938)', 'BAK1 (578)', 'BAMBI (25805)',
 'BANF1 (8815)', 'BANF2 (140836)', 'BANK1 (55024)', 'BANP (54971)', 'BAP1
 (8314)', 'BARD1 (580)', 'BARHL1 (56751)', 'BARHL2 (343472)', 'BARX1 (56033)',
 'BARX2 (8538)', 'BASP1 (10409)', 'BATF (10538)', 'BATF2 (116071)', 'BATF3
 (55509)', 'BAX (581)', 'BAZ1A (11177)', 'BAZ1B (9031)', 'BAZ2A (11176)', 'BAZ2B
 (29994)', 'BBC3 (27113)', 'BBIP1 (92482)', 'BBLN (79095)', 'BBOF1 (80127)',
 'BBOX1 (8424)', 'BBS1 (582)', 'BBS10 (79738)', 'BBS12 (166379)', 'BBS2 (583)',
 'BBS4 (585)', 'BBS5 (129880)', 'BBS7 (55212)', 'BBS9 (27241)', 'BBX (56987)',
 'BCAM (4059)', 'BCAN (63827)', 'BCAP29 (55973)', 'BCAP31 (10134)', 'BCAR1
 (9564)', 'BCAR3 (8412)', 'BCAS1 (8537)', 'BCAS2 (10286)', 'BCAS3 (54828)',
 'BCAS4 (55653)', 'BCAT1 (586)', 'BCAT2 (587)', 'BCCIP (56647)', 'BCDIN3D
 (144233)', 'BCHE (590)', 'BCKDHA (593)', 'BCKDHB (594)', 'BCKDK (10295)', 'BCL10
 (8915)', 'BCL11A (53335)', 'BCL11B (64919)', 'BCL2 (596)', 'BCL2A1 (597)',
 'BCL2L1 (598)', 'BCL2L10 (10017)', 'BCL2L11 (10018)', 'BCL2L12 (83596)',
 'BCL2L13 (23786)', 'BCL2L14 (79370)', 'BCL2L15 (440603)', 'BCL2L2 (599)', 'BCL3
 (602)', 'BCL6 (604)', 'BCL6B (255877)', 'BCL7A (605)', 'BCL7B (9275)', 'BCL7C
 (9274)', 'BCL9 (607)', 'BCL9L (283149)', 'BCLAF1 (9774)', 'BCLAF3 (256643)',
 'BCO1 (53630)', 'BCO2 (83875)', 'BCOR (54880)', 'BCORL1 (63035)', 'BCR (613)',
 'BCS1L (617)', 'BDH1 (622)', 'BDH2 (56898)', 'BDKRB1 (623)', 'BDKRB2 (624)',
 'BDNF (627)', 'BDP1 (55814)', 'BEAN1 (146227)', 'BECN1 (8678)', 'BECN2
 (441925)', 'BEGAIN (57596)', 'BEND2 (139105)', 'BEND3 (57673)', 'BEND4
 (389206)', 'BEND5 (79656)', 'BEND6 (221336)', 'BEND7 (222389)', 'BEST1 (7439)',
 'BEST2 (54831)', 'BEST3 (144453)', 'BEST4 (266675)', 'BET1 (10282)', 'BET1L
 (51272)', 'BEX1 (55859)', 'BEX2 (84707)', 'BEX3 (27018)', 'BEX4 (56271)', 'BEX5
 (340542)', 'BFAR (51283)', 'BFSP1 (631)', 'BFSP2 (8419)', 'BGLAP (632)', 'BGN
 (633)', 'BHLHA15 (168620)', 'BHLHA9 (727857)', 'BHLHE22 (27319)', 'BHLHE23
 (128408)', 'BHLHE40 (8553)', 'BHLHE41 (79365)', 'BHMT (635)', 'BHMT2 (23743)',
 'BICC1 (80114)', 'BICD1 (636)', 'BICD2 (23299)', 'BICDL1 (92558)', 'BICDL2
 (146439)', 'BICRA (29998)', 'BICRAL (23506)', 'BID (637)', 'BIK (638)', 'BIN1
 (274)', 'BIN2 (51411)', 'BIN3 (55909)', 'BIRC2 (329)', 'BIRC3 (330)', 'BIRC5
 (332)', 'BIRC6 (57448)', 'BIRC7 (79444)', 'BIVM (54841)', 'BLCAP (10904)', 'BLID
 (414899)', 'BLK (640)', 'BLM (641)', 'BLMH (642)', 'BLNK (29760)', 'BLOC1S1
 (2647)', 'BLOC1S2 (282991)', 'BLOC1S3 (388552)', 'BLOC1S4 (55330)', 'BLOC1S5
 (63915)', 'BLOC1S6 (26258)', 'BLTP1 (84162)', 'BLTP2 (9703)', 'BLTP3A (54887)',
 'BLTP3B (23074)', 'BLVRA (644)', 'BLVRB (645)', 'BLZF1 (8548)', 'BMAL1 (406)',
 'BMAL2 (56938)', 'BMERB1 (89927)', 'BMF (90427)', 'BMI1 (648)', 'BMP1 (649)',
 'BMP10 (27302)', 'BMP15 (9210)', 'BMP2 (650)', 'BMP2K (55589)', 'BMP3 (651)',
 'BMP4 (652)', 'BMP5 (653)', 'BMP6 (654)', 'BMP7 (655)', 'BMP8A (353500)', 'BMP8B
 (656)', 'BMPER (168667)', 'BMPR1A (657)', 'BMPR1B (658)', 'BMPR2 (659)', 'BMS1
 (9790)', 'BMX (660)', 'BNC1 (646)', 'BNC2 (54796)', 'BNIP1 (662)', 'BNIP2
 (663)', 'BNIP3 (664)', 'BNIP3L (665)', 'BNIP5 (389384)', 'BNIPL (149428)', 'BOC
 (91653)', 'BOD1 (91272)', 'BOD1L1 (259282)', 'BOD1L2 (284257)', 'BOK (666)',
 'BOLA1 (51027)', 'BOLA3 (388962)', 'BOLL (66037)', 'BOP1 (23246)', 'BORA
 (79866)', 'BORCS5 (118426)', 'BORCS6 (54785)', 'BORCS7 (119032)', 'BORCS8

(729991)', 'BPGM (669)', 'BPHL (670)', 'BPI (671)', 'BPIFA1 (51297)', 'BPIFA2
 (140683)', 'BPIFA3 (128861)', 'BPIFB1 (92747)', 'BPIFB2 (80341)', 'BPIFB3
 (359710)', 'BPIFB4 (149954)', 'BPIFB6 (128859)', 'BPIFC (254240)', 'BPNT1
 (10380)', 'BPNT2 (54928)', 'BPTF (2186)', 'BRAF (673)', 'BRAP (8315)', 'BRAT1
 (221927)', 'BRCA1 (672)', 'BRCA2 (675)', 'BRCC3 (79184)', 'BRD1 (23774)', 'BRD2
 (6046)', 'BRD3 (8019)', 'BRD3OS (266655)', 'BRD4 (23476)', 'BRD7 (29117)', 'BRD8
 (10902)', 'BRD9 (65980)', 'BRDT (676)', 'BRF1 (2972)', 'BRF2 (55290)', 'BRI3
 (25798)', 'BRI3BP (140707)', 'BRICD5 (283870)', 'BRINP1 (1620)', 'BRINP2
 (57795)', 'BRINP3 (339479)', 'BRIP1 (83990)', 'BRIX1 (55299)', 'BRK1 (55845)',
 'BRME1 (79173)', 'BRMS1 (25855)', 'BRMS1L (84312)', 'BROX (148362)', 'BRPF1
 (7862)', 'BRPF3 (27154)', 'BRS3 (680)', 'BRSK1 (84446)', 'BRSK2 (9024)', 'BRWD1
 (54014)', 'BRWD3 (254065)', 'BSCL2 (26580)', 'BSDC1 (55108)', 'BSG (682)', 'BSN
 (8927)', 'BSND (7809)', 'BSPH1 (100131137)', 'BSPRY (54836)', 'BST1 (683)',
 'BST2 (684)', 'BSX (390259)', 'BTAF1 (9044)', 'BTBD1 (53339)', 'BTBD10 (84280)',
 'BTBD16 (118663)', 'BTBD17 (388419)', 'BTBD18 (643376)', 'BTBD19 (149478)',
 'BTBD2 (55643)', 'BTBD3 (22903)', 'BTBD6 (90135)', 'BTBD7 (55727)', 'BTBD8
 (284697)', 'BTBD9 (114781)', 'BTC (685)', 'BTD (686)', 'BTF3 (689)', 'BTF3L4
 (91408)', 'BTG1 (694)', 'BTG2 (7832)', 'BTG3 (10950)', 'BTG4 (54766)', 'BTK
 (695)', 'BTLA (151888)', 'BTN1A1 (696)', 'BTN2A1 (11120)', 'BTN2A2 (10385)',
 'BTN3A1 (11119)', 'BTN3A2 (11118)', 'BTN3A3 (10384)', 'BTNL2 (56244)', 'BTNL3
 (10917)', 'BTNL8 (79908)', 'BTNL9 (153579)', 'BTRC (8945)', 'BUB1 (699)', 'BUB1B
 (701)', 'BUB3 (9184)', 'BUD13 (84811)', 'BUD23 (114049)', 'BUD31 (8896)', 'BYSL
 (705)', 'BZW1 (9689)', 'BZW2 (28969)', 'C10orf105 (414152)', 'C10orf120
 (399814)', 'C10orf143 (387723)', 'C10orf53 (282966)', 'C10orf62 (414157)',
 'C10orf67 (256815)', 'C10orf71 (118461)', 'C10orf88 (80007)', 'C10orf90
 (118611)', 'C10orf95 (79946)', 'C11orf16 (56673)', 'C11orf21 (29125)', 'C11orf24
 (53838)', 'C11orf42 (160298)', 'C11orf52 (91894)', 'C11orf54 (28970)', 'C11orf58
 (10944)', 'C11orf65 (160140)', 'C11orf68 (83638)', 'C11orf71 (54494)', 'C11orf86
 (254439)', 'C11orf87 (399947)', 'C11orf91 (100131378)', 'C11orf96 (387763)',
 'C11orf97 (643037)', 'C12orf42 (374470)', 'C12orf43 (64897)', 'C12orf50
 (160419)', 'C12orf54 (121273)', 'C12orf56 (115749)', 'C12orf57 (113246)',
 'C12orf60 (144608)', 'C12orf71 (728858)', 'C12orf75 (387882)', 'C13orf42
 (647166)', 'C14orf119 (55017)', 'C14orf132 (56967)', 'C14orf180 (400258)',
 'C14orf39 (317761)', 'C14orf93 (60686)', 'C15orf39 (56905)', 'C15orf40
 (123207)', 'C15orf48 (84419)', 'C15orf61 (145853)', 'C15orf62 (643338)',
 'C16orf46 (123775)', 'C16orf54 (283897)', 'C16orf74 (404550)', 'C16orf78
 (123970)', 'C16orf86 (388284)', 'C16orf87 (388272)', 'C16orf89 (146556)',
 'C16orf90 (646174)', 'C16orf92 (146378)', 'C16orf95 (100506581)', 'C16orf96
 (342346)', 'C17orf100 (388327)', 'C17orf107 (100130311)', 'C17orf113
 (110806298)', 'C17orf50 (146853)', 'C17orf58 (284018)', 'C17orf67 (339210)',
 'C17orf75 (64149)', 'C17orf78 (284099)', 'C17orf99 (100141515)', 'C18orf32
 (497661)', 'C18orf54 (162681)', 'C18orf63 (644041)', 'C19orf12 (83636)',
 'C19orf18 (147685)', 'C19orf25 (148223)', 'C19orf33 (64073)', 'C19orf38
 (255809)', 'C19orf44 (84167)', 'C19orf47 (126526)', 'C19orf53 (28974)',
 'C19orf67 (646457)', 'C19orf73 (55150)', 'C19orf81 (342918)', 'C19orf84
 (147646)', 'C1D (10438)', 'C1GALT1 (56913)', 'C1GALT1C1 (29071)', 'C1GALT1C1L
 (728819)', 'C1QA (712)', 'C1QB (713)', 'C1QBP (708)', 'C1QC (714)', 'C1QL1
 (10882)', 'C1QL2 (165257)', 'C1QL3 (389941)', 'C1QL4 (338761)', 'C1QTNF1

(114897)', 'C1QTNF12 (388581)', 'C1QTNF2 (114898)', 'C1QTNF3 (114899)', 'C1QTNF4 (114900)', 'C1QTNF5 (114902)', 'C1QTNF6 (114904)', 'C1QTNF7 (114905)', 'C1QTNF8 (390664)', 'C1QTNF9 (338872)', 'C1QTNF9B (387911)', 'C1R (715)', 'C1RL (51279)', 'C1S (716)', 'C1orf105 (92346)', 'C1orf115 (79762)', 'C1orf116 (79098)', 'C1orf122 (127687)', 'C1orf141 (400757)', 'C1orf146 (388649)', 'C1orf159 (54991)', 'C1orf162 (128346)', 'C1orf167 (284498)', 'C1orf174 (339448)', 'C1orf185 (284546)', 'C1orf198 (84886)', 'C1orf21 (81563)', 'C1orf210 (149466)', 'C1orf216 (127703)', 'C1orf226 (400793)', 'C1orf232 (110806296)', 'C1orf35 (79169)', 'C1orf43 (25912)', 'C1orf50 (79078)', 'C1orf52 (148423)', 'C1orf53 (388722)', 'C1orf54 (79630)', 'C1orf56 (54964)', 'C1orf74 (148304)', 'C1orf87 (127795)', 'C1orf94 (84970)', 'C2 (717)', 'C2orf141 (128653)', 'C2orf144 (128864)', 'C2orf173 (140873)', 'C2orf202 (400831)', 'C2orf203 (284805)', 'C2orf204 (284739)', 'C2orf96 (140680)', 'C2orf140 (101928147)', 'C2orf58 (54058)', 'C2orf91 (54149)', 'C2orf15 (150248)', 'C2orf23 (84645)', 'C2orf31 (25770)', 'C2orf39 (128977)', 'C2orf42 (150297)', 'C2CD2 (25966)', 'C2CD2L (9854)', 'C2CD3 (26005)', 'C2CD4A (145741)', 'C2CD4B (388125)', 'C2CD4C (126567)', 'C2CD4D (100191040)', 'C2CD5 (9847)', 'C2CD6 (151254)', 'C2orf15 (150590)', 'C2orf42 (54980)', 'C2orf49 (79074)', 'C2orf66 (401027)', 'C2orf68 (388969)', 'C2orf69 (205327)', 'C2orf72 (257407)', 'C2orf74 (339804)', 'C2orf76 (130355)', 'C2orf78 (388960)', 'C2orf80 (389073)', 'C2orf81 (388963)', 'C2orf88 (84281)', 'C2orf92 (728537)', 'C3 (718)', 'C3AR1 (719)', 'C3orf18 (51161)', 'C3orf20 (84077)', 'C3orf22 (152065)', 'C3orf33 (285315)', 'C3orf38 (285237)', 'C3orf49 (132200)', 'C3orf52 (79669)', 'C3orf62 (375341)', 'C3orf70 (285382)', 'C3orf80 (401097)', 'C3orf85 (401081)', 'C4B (721)', 'C4BPA (722)', 'C4BPB (725)', 'C4orf17 (84103)', 'C4orf33 (132321)', 'C4orf36 (132989)', 'C4orf46 (201725)', 'C4orf50 (389197)', 'C4orf51 (646603)', 'C4orf54 (285556)', 'C5 (727)', 'C5AR1 (728)', 'C5AR2 (27202)', 'C5orf15 (56951)', 'C5orf22 (55322)', 'C5orf24 (134553)', 'C5orf34 (375444)', 'C5orf46 (389336)', 'C5orf47 (133491)', 'C5orf52 (100190949)', 'C5orf58 (133874)', 'C5orf63 (401207)', 'C6 (729)', 'C6orf118 (168090)', 'C6orf120 (387263)', 'C6orf132 (647024)', 'C6orf136 (221545)', 'C6orf141 (135398)', 'C6orf15 (29113)', 'C6orf163 (206412)', 'C6orf47 (57827)', 'C6orf52 (347744)', 'C6orf58 (352999)', 'C6orf62 (81688)', 'C6orf89 (221477)', 'C7 (730)', 'C7orf25 (79020)', 'C7orf33 (202865)', 'C7orf57 (136288)', 'C8A (731)', 'C8B (732)', 'C8G (733)', 'C8orf33 (65265)', 'C8orf34 (116328)', 'C8orf48 (157773)', 'C8orf58 (541565)', 'C8orf74 (203076)', 'C8orf76 (84933)', 'C8orf82 (414919)', 'C8orf88 (100127983)', 'C8orf89 (100130301)', 'C9 (735)', 'C9orf152 (401546)', 'C9orf153 (389766)', 'C9orf40 (55071)', 'C9orf43 (257169)', 'C9orf50 (375759)', 'C9orf57 (138240)', 'C9orf72 (203228)', 'C9orf78 (51759)', 'C9orf85 (138241)', 'CA1 (759)', 'CA10 (56934)', 'CA11 (770)', 'CA12 (771)', 'CA13 (377677)', 'CA14 (23632)', 'CA2 (760)', 'CA3 (761)', 'CA4 (762)', 'CA5A (763)', 'CA5B (11238)', 'CA6 (765)', 'CA7 (766)', 'CA8 (767)', 'CA9 (768)', 'CAAP1 (79886)', 'CAB39 (51719)', 'CAB39L (81617)', 'CABCOC01 (219621)', 'CABIN1 (23523)', 'CABLES1 (91768)', 'CABLES2 (81928)', 'CABP1 (9478)', 'CABP2 (51475)', 'CABP4 (57010)', 'CABP5 (56344)', 'CABP7 (164633)', 'CABS1 (85438)', 'CABYR (26256)', 'CACFD1 (11094)', 'CACHD1 (57685)', 'CACNA1A (773)', 'CACNA1B (774)', 'CACNA1C (775)', 'CACNA1D (776)', 'CACNA1E (777)', 'CACNA1F (778)', 'CACNA1G (8913)', 'CACNA1H (8912)', 'CACNA1I (8911)', 'CACNA1S (779)', 'CACNA2D1 (781)', 'CACNA2D2 (9254)', 'CACNA2D3 (55799)', 'CACNA2D4 (93589)', 'CACNB1

(782)', 'CACNB2 (783)', 'CACNB3 (784)', 'CACNB4 (785)', 'CACNG1 (786)', 'CACNG2
 (10369)', 'CACNG3 (10368)', 'CACNG4 (27092)', 'CACNG5 (27091)', 'CACNG6
 (59285)', 'CACNG7 (59284)', 'CACNG8 (59283)', 'CACTIN (58509)', 'CACUL1
 (143384)', 'CACYBP (27101)', 'CAD (790)', 'CADM1 (23705)', 'CADM2 (253559)',
 'CADM3 (57863)', 'CADM4 (199731)', 'CADPS (8618)', 'CADPS2 (93664)', 'CAGE1
 (285782)', 'CALB1 (793)', 'CALB2 (794)', 'CALCA (796)', 'CALCB (797)', 'CALCOC01
 (57658)', 'CALCOC02 (10241)', 'CALCR (799)', 'CALCRL (10203)', 'CALD1 (800)',
 'CALHM1 (255022)', 'CALHM2 (51063)', 'CALHM3 (119395)', 'CALHM4 (221301)',
 'CALHM5 (254228)', 'CALHM6 (441168)', 'CALM1 (801)', 'CALM2 (805)', 'CALM3
 (808)', 'CALML3 (810)', 'CALML4 (91860)', 'CALML5 (51806)', 'CALML6 (163688)',
 'CALN1 (83698)', 'CALR (811)', 'CALR3 (125972)', 'CALU (813)', 'CALY (50632)',
 'CAMK1 (8536)', 'CAMK1D (57118)', 'CAMK1G (57172)', 'CAMK2A (815)', 'CAMK2B
 (816)', 'CAMK2D (817)', 'CAMK2G (818)', 'CAMK2N1 (55450)', 'CAMK2N2 (94032)',
 'CAMK4 (814)', 'CAMKK1 (84254)', 'CAMKK2 (10645)', 'CAMKMT (79823)', 'CAMKV
 (79012)', 'CAMLG (819)', 'CAMP (820)', 'CAMSAP1 (157922)', 'CAMSAP2 (23271)',
 'CAMSAP3 (57662)', 'CAMTA1 (23261)', 'CAMTA2 (23125)', 'CAND1 (55832)', 'CAND2
 (23066)', 'CANT1 (124583)', 'CANX (821)', 'CAP1 (10487)', 'CAP2 (10486)', 'CAPG
 (822)', 'CAPN1 (823)', 'CAPN10 (11132)', 'CAPN11 (11131)', 'CAPN12 (147968)',
 'CAPN13 (92291)', 'CAPN14 (440854)', 'CAPN15 (6650)', 'CAPN2 (824)', 'CAPN3
 (825)', 'CAPN5 (726)', 'CAPN6 (827)', 'CAPN7 (23473)', 'CAPN8 (388743)', 'CAPN9
 (10753)', 'CAPNS1 (826)', 'CAPNS2 (84290)', 'CAPRIN1 (4076)', 'CAPRIN2 (65981)',
 'CAPS (828)', 'CAPS2 (84698)', 'CAPSL (133690)', 'CAPZA1 (829)', 'CAPZA2 (830)',
 'CAPZA3 (93661)', 'CAPZB (832)', 'CARD10 (29775)', 'CARD11 (84433)', 'CARD14
 (79092)', 'CARD16 (114769)', 'CARD18 (59082)', 'CARD19 (84270)', 'CARD6
 (84674)', 'CARD8 (22900)', 'CARD9 (64170)', 'CARF (79800)', 'CARHSP1 (23589)',
 'CARM1 (10498)', 'CARMIL1 (55604)', 'CARMIL2 (146206)', 'CARMIL3 (90668)',
 'CARNMT1 (138199)', 'CARNS1 (57571)', 'CARS1 (833)', 'CARS2 (79587)', 'CARTPT
 (9607)', 'CASC3 (22794)', 'CASD1 (64921)', 'CASK (8573)', 'CASKIN1 (57524)',
 'CASKIN2 (57513)', 'CASP1 (834)', 'CASP10 (843)', 'CASP14 (23581)', 'CASP2
 (835)', 'CASP3 (836)', 'CASP4 (837)', 'CASP5 (838)', 'CASP6 (839)', 'CASP7
 (840)', 'CASP8 (841)', 'CASP9 (842)', 'CASQ1 (844)', 'CASQ2 (845)', 'CASR
 (846)', 'CASS4 (57091)', 'CAST (831)', 'CASTOR1 (652968)', 'CASTOR2 (729438)',
 'CASZ1 (54897)', 'CAT (847)', 'CATIP (375307)', 'CATSPER1 (117144)', 'CATSPER2
 (117155)', 'CATSPER3 (347732)', 'CATSPER4 (378807)', 'CATSPERB (79820)',
 'CATSPERD (257062)', 'CATSPERE (257044)', 'CATSPERG (57828)', 'CAV1 (857)',
 'CAV2 (858)', 'CAV3 (859)', 'CAVIN1 (284119)', 'CAVIN2 (8436)', 'CAVIN3
 (112464)', 'CAVIN4 (347273)', 'CBARP (255057)', 'CBFA2T2 (9139)', 'CBFA2T3
 (863)', 'CBFB (865)', 'CBL (867)', 'CBLB (868)', 'CBLC (23624)', 'CBLIF (2694)',
 'CBLL1 (79872)', 'CBLL2 (158506)', 'CBLN1 (869)', 'CBLN2 (147381)', 'CBLN3
 (643866)', 'CBLN4 (140689)', 'CBR1 (873)', 'CBR3 (874)', 'CBR4 (84869)', 'CBS
 (875)', 'CBX1 (10951)', 'CBX2 (84733)', 'CBX3 (11335)', 'CBX4 (8535)', 'CBX5
 (23468)', 'CBX6 (23466)', 'CBX7 (23492)', 'CBX8 (57332)', 'CBY1 (25776)', 'CBY2
 (220082)', 'CBY3 (646019)', 'CC2D1A (54862)', 'CC2D1B (200014)', 'CC2D2A
 (57545)', 'CC2D2B (387707)', 'CCAR1 (55749)', 'CCAR2 (57805)', 'CCBE1 (147372)',
 'CCDC102A (92922)', 'CCDC102B (79839)', 'CCDC106 (29903)', 'CCDC107 (203260)',
 'CCDC110 (256309)', 'CCDC112 (153733)', 'CCDC116 (164592)', 'CCDC117 (150275)',
 'CCDC12 (151903)', 'CCDC120 (90060)', 'CCDC121 (79635)', 'CCDC122 (160857)',
 'CCDC124 (115098)', 'CCDC125 (202243)', 'CCDC126 (90693)', 'CCDC127 (133957)',

'CCDC13 (152206)', 'CCDC134 (79879)', 'CCDC136 (64753)', 'CCDC137 (339230)',
 'CCDC138 (165055)', 'CCDC14 (64770)', 'CCDC142 (84865)', 'CCDC144A (9720)',
 'CCDC146 (57639)', 'CCDC148 (130940)', 'CCDC149 (91050)', 'CCDC15 (80071)',
 'CCDC150 (284992)', 'CCDC152 (100129792)', 'CCDC154 (645811)', 'CCDC157
 (550631)', 'CCDC158 (339965)', 'CCDC159 (126075)', 'CCDC160 (347475)', 'CCDC163
 (126661)', 'CCDC166 (100130274)', 'CCDC167 (154467)', 'CCDC168 (643677)',
 'CCDC169 (728591)', 'CCDC17 (149483)', 'CCDC170 (80129)', 'CCDC171 (203238)',
 'CCDC172 (374355)', 'CCDC174 (51244)', 'CCDC175 (729665)', 'CCDC177 (56936)',
 'CCDC178 (374864)', 'CCDC179 (100500938)', 'CCDC18 (343099)', 'CCDC180
 (100499483)', 'CCDC181 (57821)', 'CCDC182 (101927581)', 'CCDC183 (84960)',
 'CCDC184 (387856)', 'CCDC185 (164127)', 'CCDC186 (55088)', 'CCDC187 (399693)',
 'CCDC188 (388849)', 'CCDC190 (339512)', 'CCDC191 (57577)', 'CCDC192 (728586)',
 'CCDC194 (110806280)', 'CCDC196 (440184)', 'CCDC197 (256369)', 'CCDC198
 (55195)', 'CCDC200 (100874261)', 'CCDC22 (28952)', 'CCDC24 (149473)', 'CCDC25
 (55246)', 'CCDC27 (148870)', 'CCDC28A (25901)', 'CCDC28B (79140)', 'CCDC3
 (83643)', 'CCDC30 (728621)', 'CCDC32 (90416)', 'CCDC33 (80125)', 'CCDC34
 (91057)', 'CCDC38 (120935)', 'CCDC39 (339829)', 'CCDC40 (55036)', 'CCDC42
 (146849)', 'CCDC43 (124808)', 'CCDC47 (57003)', 'CCDC50 (152137)', 'CCDC51
 (79714)', 'CCDC54 (84692)', 'CCDC57 (284001)', 'CCDC59 (29080)', 'CCDC6 (8030)',
 'CCDC60 (160777)', 'CCDC61 (729440)', 'CCDC62 (84660)', 'CCDC63 (160762)',
 'CCDC65 (85478)', 'CCDC66 (285331)', 'CCDC68 (80323)', 'CCDC69 (26112)', 'CCDC7
 (79741)', 'CCDC70 (83446)', 'CCDC71 (64925)', 'CCDC71L (168455)', 'CCDC73
 (493860)', 'CCDC74A (90557)', 'CCDC74B (91409)', 'CCDC77 (84318)', 'CCDC78
 (124093)', 'CCDC8 (83987)', 'CCDC80 (151887)', 'CCDC81 (60494)', 'CCDC82
 (79780)', 'CCDC83 (220047)', 'CCDC85A (114800)', 'CCDC85B (11007)', 'CCDC85C
 (317762)', 'CCDC86 (79080)', 'CCDC87 (55231)', 'CCDC88A (55704)', 'CCDC88B
 (283234)', 'CCDC88C (440193)', 'CCDC89 (220388)', 'CCDC9 (26093)', 'CCDC90B
 (60492)', 'CCDC91 (55297)', 'CCDC92 (80212)', 'CCDC92B (101928991)', 'CCDC93
 (54520)', 'CCDC97 (90324)', 'CCDC9B (388115)', 'CCER1 (196477)', 'CCER2
 (643669)', 'CCHCR1 (54535)', 'CCIN (881)', 'CCK (885)', 'CCKAR (886)', 'CCKBR
 (887)', 'CCL1 (6346)', 'CCL11 (6356)', 'CCL13 (6357)', 'CCL14 (6358)', 'CCL15
 (6359)', 'CCL16 (6360)', 'CCL17 (6361)', 'CCL18 (6362)', 'CCL19 (6363)', 'CCL2
 (6347)', 'CCL20 (6364)', 'CCL21 (6366)', 'CCL22 (6367)', 'CCL23 (6368)', 'CCL24
 (6369)', 'CCL25 (6370)', 'CCL26 (10344)', 'CCL27 (10850)', 'CCL28 (56477)',
 'CCL3 (6348)', 'CCL3L3 (414062)', 'CCL4 (6351)', 'CCL5 (6352)', 'CCL7 (6354)',
 'CCL8 (6355)', 'CCM2 (83605)', 'CCM2L (140706)', 'CCN1 (3491)', 'CCN2 (1490)',
 'CCN3 (4856)', 'CCN4 (8840)', 'CCN5 (8839)', 'CCN6 (8838)', 'CCNA1 (8900)',
 'CCNA2 (890)', 'CCNB1 (891)', 'CCNB1IP1 (57820)', 'CCNB2 (9133)', 'CCNB3
 (85417)', 'CCNC (892)', 'CCND1 (595)', 'CCND2 (894)', 'CCND3 (896)', 'CCNDBP1
 (23582)', 'CCNE1 (898)', 'CCNE2 (9134)', 'CCNF (899)', 'CCNG1 (900)', 'CCNG2
 (901)', 'CCNH (902)', 'CCNI (10983)', 'CCNI2 (645121)', 'CCNJ (54619)', 'CCNJL
 (79616)', 'CCNK (8812)', 'CCNL1 (57018)', 'CCNL2 (81669)', 'CCNO (10309)', 'CCNP
 (79935)', 'CCNQ (92002)', 'CCNT1 (904)', 'CCNT2 (905)', 'CCNY (219771)', 'CCNYL1
 (151195)', 'CCP110 (9738)', 'CCPG1 (9236)', 'CCR1 (1230)', 'CCR10 (2826)', 'CCR2
 (729230)', 'CCR3 (1232)', 'CCR4 (1233)', 'CCR5 (1234)', 'CCR6 (1235)', 'CCR7
 (1236)', 'CCR8 (1237)', 'CCR9 (10803)', 'CCRL2 (9034)', 'CCS (9973)', 'CCSAP
 (126731)', 'CCSER1 (401145)', 'CCSER2 (54462)', 'CCT2 (10576)', 'CCT3 (7203)',
 'CCT4 (10575)', 'CCT5 (22948)', 'CCT6A (908)', 'CCT6B (10693)', 'CCT7 (10574)',

'CCT8 (10694)', 'CCT8L2 (150160)', 'CD101 (9398)', 'CD109 (135228)', 'CD14 (929)', 'CD151 (977)', 'CD160 (11126)', 'CD163 (9332)', 'CD163L1 (283316)', 'CD164 (8763)', 'CD164L2 (388611)', 'CD177 (57126)', 'CD180 (4064)', 'CD19 (930)', 'CD1A (909)', 'CD1B (910)', 'CD1C (911)', 'CD1D (912)', 'CD1E (913)', 'CD2 (914)', 'CD200 (4345)', 'CD200R1 (131450)', 'CD200R1L (344807)', 'CD207 (50489)', 'CD209 (30835)', 'CD22 (933)', 'CD226 (10666)', 'CD24 (100133941)', 'CD244 (51744)', 'CD247 (919)', 'CD248 (57124)', 'CD27 (939)', 'CD274 (29126)', 'CD276 (80381)', 'CD28 (940)', 'CD2AP (23607)', 'CD2BP2 (10421)', 'CD300A (11314)', 'CD300C (10871)', 'CD300E (342510)', 'CD300H (100130520)', 'CD300LB (124599)', 'CD300LD (100131439)', 'CD300LF (146722)', 'CD300LG (146894)', 'CD302 (9936)', 'CD320 (51293)', 'CD33 (945)', 'CD34 (947)', 'CD36 (948)', 'CD37 (951)', 'CD38 (952)', 'CD3D (915)', 'CD3E (916)', 'CD3G (917)', 'CD4 (920)', 'CD40 (958)', 'CD40LG (959)', 'CD44 (960)', 'CD46 (4179)', 'CD47 (961)', 'CD48 (962)', 'CD5 (921)', 'CD52 (1043)', 'CD53 (963)', 'CD55 (1604)', 'CD58 (965)', 'CD59 (966)', 'CD5L (922)', 'CD6 (923)', 'CD63 (967)', 'CD68 (968)', 'CD69 (969)', 'CD7 (924)', 'CD70 (970)', 'CD72 (971)', 'CD74 (972)', 'CD79A (973)', 'CD79B (974)', 'CD80 (941)', 'CD81 (975)', 'CD82 (3732)', 'CD83 (9308)', 'CD84 (8832)', 'CD86 (942)', 'CD8A (925)', 'CD8B (926)', 'CD9 (928)', 'CD93 (22918)', 'CD96 (10225)', 'CD99 (4267)', 'CD99L2 (83692)', 'CDA (978)', 'CDADC1 (81602)', 'CDAN1 (146059)', 'CDC123 (8872)', 'CDC14A (8556)', 'CDC14B (8555)', 'CDC14C (168448)', 'CDC16 (8881)', 'CDC20 (991)', 'CDC20B (166979)', 'CDC23 (8697)', 'CDC25A (993)', 'CDC25B (994)', 'CDC25C (995)', 'CDC26 (246184)', 'CDC27 (996)', 'CDC34 (997)', 'CDC37 (11140)', 'CDC37L1 (55664)', 'CDC40 (51362)', 'CDC42 (998)', 'CDC42BPA (8476)', 'CDC42BPB (9578)', 'CDC42BPG (55561)', 'CDC42EP1 (11135)', 'CDC42EP2 (10435)', 'CDC42EP3 (10602)', 'CDC42EP4 (23580)', 'CDC42EP5 (148170)', 'CDC42SE1 (56882)', 'CDC42SE2 (56990)', 'CDC45 (8318)', 'CDC5L (988)', 'CDC6 (990)', 'CDC7 (8317)', 'CDC73 (79577)', 'CDCA2 (157313)', 'CDCA3 (83461)', 'CDCA4 (55038)', 'CDCA5 (113130)', 'CDCA7 (83879)', 'CDCA7L (55536)', 'CDCA8 (55143)', 'CDCP1 (64866)', 'CDCP2 (200008)', 'CDH1 (999)', 'CDH10 (1008)', 'CDH11 (1009)', 'CDH12 (1010)', 'CDH13 (1012)', 'CDH15 (1013)', 'CDH16 (1014)', 'CDH17 (1015)', 'CDH18 (1016)', 'CDH19 (28513)', 'CDH2 (1000)', 'CDH20 (28316)', 'CDH22 (64405)', 'CDH23 (64072)', 'CDH24 (64403)', 'CDH26 (60437)', 'CDH3 (1001)', 'CDH4 (1002)', 'CDH5 (1003)', 'CDH6 (1004)', 'CDH7 (1005)', 'CDH8 (1006)', 'CDH9 (1007)', 'CDHR1 (92211)', 'CDHR2 (54825)', 'CDHR3 (222256)', 'CDHR4 (389118)', 'CDHR5 (53841)', 'CDIN1 (84529)', 'CDIP1 (29965)', 'CDIPT (10423)', 'CDK1 (983)', 'CDK10 (8558)', 'CDK12 (51755)', 'CDK13 (8621)', 'CDK14 (5218)', 'CDK15 (65061)', 'CDK16 (5127)', 'CDK17 (5128)', 'CDK18 (5129)', 'CDK19 (23097)', 'CDK2 (1017)', 'CDK20 (23552)', 'CDK2AP1 (8099)', 'CDK2AP2 (10263)', 'CDK3 (1018)', 'CDK4 (1019)', 'CDK5 (1020)', 'CDK5R1 (8851)', 'CDK5R2 (8941)', 'CDK5RAP1 (51654)', 'CDK5RAP2 (55755)', 'CDK5RAP3 (80279)', 'CDK6 (1021)', 'CDK7 (1022)', 'CDK8 (1024)', 'CDK9 (1025)', 'CDKAL1 (54901)', 'CDKL1 (8814)', 'CDKL2 (8999)', 'CDKL3 (51265)', 'CDKL4 (344387)', 'CDKL5 (6792)', 'CDKN1A (1026)', 'CDKN1B (1027)', 'CDKN1C (1028)', 'CDKN2A (1029)', 'CDKN2AIP (55602)', 'CDKN2AIPNL (91368)', 'CDKN2B (1030)', 'CDKN2C (1031)', 'CDKN2D (1032)', 'CDKN3 (1033)', 'CDNF (441549)', 'CDO1 (1036)', 'CDON (50937)', 'CDPF1 (150383)', 'CDR2 (1039)', 'CDR2L (30850)', 'CDRT15 (146822)', 'CDRT15L2 (256223)', 'CDRT4 (284040)', 'CDS1 (1040)', 'CDS2 (8760)', 'CDSN (1041)', 'CDT1 (81620)', 'CDV3 (55573)', 'CDX1 (1044)', 'CDX2 (1045)', 'CDX4 (1046)', 'CDYL (9425)', 'CDYL2

(124359)', 'CEACAM1 (634)', 'CEACAM16 (388551)', 'CEACAM19 (56971)', 'CEACAM20
 (125931)', 'CEACAM21 (90273)', 'CEACAM3 (1084)', 'CEACAM4 (1089)', 'CEACAM5
 (1048)', 'CEACAM6 (4680)', 'CEACAM7 (1087)', 'CEACAM8 (1088)', 'CEBPA (1050)',
 'CEBPB (1051)', 'CEBPD (1052)', 'CEBPE (1053)', 'CEBPG (1054)', 'CEBPZ (10153)',
 'CEBPZOS (100505876)', 'CECR2 (27443)', 'CEL (1056)', 'CELA1 (1990)', 'CELA2A
 (63036)', 'CELA2B (51032)', 'CELA3A (10136)', 'CELA3B (23436)', 'CELF1 (10658)',
 'CELF2 (10659)', 'CELF3 (11189)', 'CELF4 (56853)', 'CELF5 (60680)', 'CELF6
 (60677)', 'CELSR1 (9620)', 'CELSR2 (1952)', 'CELSR3 (1951)', 'CEMIP (57214)',
 'CEMIP2 (23670)', 'CEMP1 (752014)', 'CENATAC (338657)', 'CEND1 (51286)', 'CENPA
 (1058)', 'CENPB (1059)', 'CENPC (1060)', 'CENPE (1062)', 'CENPF (1063)', 'CENPH
 (64946)', 'CENPI (2491)', 'CENPK (64105)', 'CENPL (91687)', 'CENPM (79019)',
 'CENPN (55839)', 'CENPO (79172)', 'CENPP (401541)', 'CENPQ (55166)', 'CENPT
 (80152)', 'CENPU (79682)', 'CENPV (201161)', 'CENPW (387103)', 'CENPX (201254)',
 'CEP104 (9731)', 'CEP112 (201134)', 'CEP120 (153241)', 'CEP126 (57562)', 'CEP128
 (145508)', 'CEP131 (22994)', 'CEP135 (9662)', 'CEP15 (57415)', 'CEP152 (22995)',
 'CEP162 (22832)', 'CEP164 (22897)', 'CEP170 (9859)', 'CEP170B (283638)', 'CEP19
 (84984)', 'CEP192 (55125)', 'CEP20 (123811)', 'CEP250 (11190)', 'CEP290
 (80184)', 'CEP295 (85459)', 'CEP295NL (100653515)', 'CEP350 (9857)', 'CEP41
 (95681)', 'CEP43 (11116)', 'CEP44 (80817)', 'CEP55 (55165)', 'CEP57 (9702)',
 'CEP57L1 (285753)', 'CEP63 (80254)', 'CEP68 (23177)', 'CEP70 (80321)', 'CEP72
 (55722)', 'CEP76 (79959)', 'CEP78 (84131)', 'CEP83 (51134)', 'CEP85 (64793)',
 'CEP85L (387119)', 'CEP89 (84902)', 'CEP95 (90799)', 'CEP97 (79598)', 'CEPT1
 (10390)', 'CER1 (9350)', 'CERCAM (51148)', 'CERK (64781)', 'CERKL (375298)',
 'CERS1 (10715)', 'CERS2 (29956)', 'CERS3 (204219)', 'CERS4 (79603)', 'CERS5
 (91012)', 'CERS6 (253782)', 'CERT1 (10087)', 'CES1 (1066)', 'CES2 (8824)', 'CES3
 (23491)', 'CES4A (283848)', 'CES5A (221223)', 'CETN1 (1068)', 'CETN2 (1069)',
 'CETN3 (1070)', 'CETP (1071)', 'CFAP100 (348807)', 'CFAP107 (93190)', 'CFAP119
 (90835)', 'CFAP126 (257177)', 'CFAP141 (388701)', 'CFAP144 (440585)', 'CFAP157
 (286207)', 'CFAP161 (161502)', 'CFAP184 (257236)', 'CFAP20 (29105)', 'CFAP206
 (154313)', 'CFAP20DC (200844)', 'CFAP210 (129881)', 'CFAP221 (200373)', 'CFAP251
 (144406)', 'CFAP263 (29070)', 'CFAP276 (127003)', 'CFAP298 (56683)', 'CFAP299
 (255119)', 'CFAP300 (85016)', 'CFAP36 (112942)', 'CFAP410 (755)', 'CFAP418
 (157657)', 'CFAP43 (80217)', 'CFAP44 (55779)', 'CFAP45 (25790)', 'CFAP46
 (54777)', 'CFAP47 (286464)', 'CFAP52 (146845)', 'CFAP53 (220136)', 'CFAP54
 (144535)', 'CFAP57 (149465)', 'CFAP58 (159686)', 'CFAP61 (26074)', 'CFAP65
 (255101)', 'CFAP68 (64776)', 'CFAP69 (79846)', 'CFAP70 (118491)', 'CFAP73
 (387885)', 'CFAP74 (85452)', 'CFAP77 (389799)', 'CFAP90 (134121)', 'CFAP91
 (89876)', 'CFAP92 (57501)', 'CFAP95 (138255)', 'CFAP96 (441054)', 'CFAP97
 (57587)', 'CFAP97D1 (284067)', 'CFAP97D2 (101929355)', 'CFB (629)', 'CFC1
 (55997)', 'CFC1B (653275)', 'CFD (1675)', 'CFDP1 (10428)', 'CFH (3075)', 'CFHR1
 (3078)', 'CFHR2 (3080)', 'CFHR3 (10878)', 'CFHR4 (10877)', 'CFHR5 (81494)', 'CFI
 (3426)', 'CFL1 (1072)', 'CFL2 (1073)', 'CFLAR (8837)', 'CFP (5199)', 'CFTR
 (1080)', 'CGA (1081)', 'CGAS (115004)', 'CGB7 (94027)', 'CGGBP1 (8545)', 'CGN
 (57530)', 'CGNL1 (84952)', 'CGREF1 (10669)', 'CGRF1 (10668)', 'CH25H (9023)',
 'CHAC1 (79094)', 'CHAC2 (494143)', 'CHAD (1101)', 'CHADL (150356)', 'CHAF1A
 (10036)', 'CHAF1B (8208)', 'CHAMP1 (283489)', 'CHAT (1103)', 'CHCHD1 (118487)',
 'CHCHD10 (400916)', 'CHCHD2 (51142)', 'CHCHD3 (54927)', 'CHCHD4 (131474)',
 'CHCHD5 (84269)', 'CHCHD6 (84303)', 'CHCHD7 (79145)', 'CHCT1 (124773)', 'CHD1

(1105)', 'CHD1L (9557)', 'CHD2 (1106)', 'CHD3 (1107)', 'CHD4 (1108)', 'CHD5
 (26038)', 'CHD6 (84181)', 'CHD7 (55636)', 'CHD8 (57680)', 'CHD9 (80205)', 'CHDH
 (55349)', 'CHEK1 (1111)', 'CHEK2 (11200)', 'CHERP (10523)', 'CHFR (55743)',
 'CHGA (1113)', 'CHGB (1114)', 'CHI3L1 (1116)', 'CHI3L2 (1117)', 'CHIA (27159)',
 'CHIC1 (53344)', 'CHIC2 (26511)', 'CHID1 (66005)', 'CHIT1 (1118)', 'CHKA
 (1119)', 'CHKB (1120)', 'CHL1 (10752)', 'CHLSN (84310)', 'CHM (1121)', 'CHML
 (1122)', 'CHMP1A (5119)', 'CHMP1B (57132)', 'CHMP2A (27243)', 'CHMP2B (25978)',
 'CHMP3 (51652)', 'CHMP4A (29082)', 'CHMP4B (128866)', 'CHMP4C (92421)', 'CHMP5
 (51510)', 'CHMP6 (79643)', 'CHMP7 (91782)', 'CHN1 (1123)', 'CHN2 (1124)', 'CHODL
 (140578)', 'CHORDC1 (26973)', 'CHP1 (11261)', 'CHP2 (63928)', 'CHPF (79586)',
 'CHPF2 (54480)', 'CHPT1 (56994)', 'CHRA1 (54108)', 'CHRD (8646)', 'CHRD1
 (91851)', 'CHRD2 (25884)', 'CHRFAM7A (89832)', 'CHRM1 (1128)', 'CHRM2 (1129)',
 'CHRM3 (1131)', 'CHRM4 (1132)', 'CHRM5 (1133)', 'CHRNA1 (1134)', 'CHRNA10
 (57053)', 'CHRNA2 (1135)', 'CHRNA3 (1136)', 'CHRNA4 (1137)', 'CHRNA5 (1138)',
 'CHRNA6 (8973)', 'CHRNA7 (1139)', 'CHRNA9 (55584)', 'CHRNA11 (1140)', 'CHRNA12
 (1141)', 'CHRNA13 (1142)', 'CHRNA14 (1143)', 'CHRNA15 (1144)', 'CHRNA16 (1145)',
 'CHRNA17 (1146)', 'CHST1 (8534)', 'CHST10 (9486)', 'CHST11 (50515)', 'CHST12
 (55501)', 'CHST13 (166012)', 'CHST14 (113189)', 'CHST15 (51363)', 'CHST2
 (9435)', 'CHST3 (9469)', 'CHST4 (10164)', 'CHST5 (23563)', 'CHST6 (4166)',
 'CHST7 (56548)', 'CHST8 (64377)', 'CHST9 (83539)', 'CHSY1 (22856)', 'CHSY3
 (337876)', 'CHTF18 (63922)', 'CHTF8 (54921)', 'CHTOP (26097)', 'CHUK (1147)',
 'CHURC1 (91612)', 'CIAO1 (9391)', 'CIAO2A (84191)', 'CIAO2B (51647)', 'CIAO3
 (64428)', 'CIAPIN1 (57019)', 'CIART (148523)', 'CIB1 (10519)', 'CIB2 (10518)',
 'CIB3 (117286)', 'CIB4 (130106)', 'CIBAR1 (137392)', 'CIBAR2 (339145)', 'CIC
 (23152)', 'CIDEA (1149)', 'CIDEB (27141)', 'CIDEA (63924)', 'CIITA (4261)',
 'CILK1 (22858)', 'CILP (8483)', 'CILP2 (148113)', 'CIMA1A (113746)', 'CIMA1B
 (440836)', 'CIMA1C (161753)', 'CIMA1D (284451)', 'CIMA2 (163747)', 'CIMA3
 (128344)', 'CIMIP1 (128602)', 'CIMIP2A (401565)', 'CIMIP2B (730112)', 'CIMIP2C
 (339778)', 'CIMIP3 (114841037)', 'CIMIP4 (339669)', 'CIMIP5 (130813)', 'CIMIP6
 (129852)', 'CIMIP7 (646498)', 'CINP (51550)', 'CIP2A (57650)', 'CIPC (85457)',
 'CIRBP (1153)', 'CIROP (100128908)', 'CIROZ (148345)', 'CIRSR (9541)', 'CISD1
 (55847)', 'CISD2 (493856)', 'CISD3 (284106)', 'CISH (1154)', 'CIT (11113)',
 'CITED1 (4435)', 'CITED2 (10370)', 'CITED4 (163732)', 'CIZ1 (25792)', 'CKAP2
 (26586)', 'CKAP2L (150468)', 'CKAP4 (10970)', 'CKAP5 (9793)', 'CKB (1152)',
 'CKLF (51192)', 'CKM (1158)', 'CKMT2 (1160)', 'CKS1B (1163)', 'CKS2 (1164)',
 'CLASP1 (23332)', 'CLASP2 (23122)', 'CLASRP (11129)', 'CLBA1 (122616)', 'CLC
 (1178)', 'CLCA1 (1179)', 'CLCA2 (9635)', 'CLCA4 (22802)', 'CLCC1 (23155)',
 'CLCF1 (23529)', 'CLCN1 (1180)', 'CLCN2 (1181)', 'CLCN3 (1182)', 'CLCN4 (1183)',
 'CLCN5 (1184)', 'CLCN6 (1185)', 'CLCN7 (1186)', 'CLCNKA (1187)', 'CLCNKB
 (1188)', 'CLDN1 (9076)', 'CLDN10 (9071)', 'CLDN11 (5010)', 'CLDN12 (9069)',
 'CLDN14 (23562)', 'CLDN15 (24146)', 'CLDN16 (10686)', 'CLDN17 (26285)', 'CLDN18
 (51208)', 'CLDN19 (149461)', 'CLDN2 (9075)', 'CLDN20 (49861)', 'CLDN22 (53842)',
 'CLDN23 (137075)', 'CLDN24 (100132463)', 'CLDN25 (644672)', 'CLDN3 (1365)',
 'CLDN34 (100288814)', 'CLDN4 (1364)', 'CLDN5 (7122)', 'CLDN6 (9074)', 'CLDN7
 (1366)', 'CLDN8 (9073)', 'CLDN9 (9080)', 'CLDND1 (56650)', 'CLDND2 (125875)',
 'CLEC10A (10462)', 'CLEC11A (6320)', 'CLEC12A (160364)', 'CLEC12B (387837)',
 'CLEC14A (161198)', 'CLEC16A (23274)', 'CLEC17A (388512)', 'CLEC18A (348174)',
 'CLEC18B (497190)', 'CLEC19A (728276)', 'CLEC1A (51267)', 'CLEC1B (51266)',

'CLEC20A (400797)', 'CLEC2A (387836)', 'CLEC2B (9976)', 'CLEC2D (29121)',
 'CLEC2L (154790)', 'CLEC3A (10143)', 'CLEC3B (7123)', 'CLEC4A (50856)', 'CLEC4C
 (170482)', 'CLEC4D (338339)', 'CLEC4E (26253)', 'CLEC4F (165530)', 'CLEC4G
 (339390)', 'CLEC4M (10332)', 'CLEC5A (23601)', 'CLEC6A (93978)', 'CLEC7A
 (64581)', 'CLEC9A (283420)', 'CLGN (1047)', 'CLHC1 (130162)', 'CLIC1 (1192)',
 'CLIC2 (1193)', 'CLIC3 (9022)', 'CLIC4 (25932)', 'CLIC5 (53405)', 'CLIC6
 (54102)', 'CLINT1 (9685)', 'CLIP1 (6249)', 'CLIP2 (7461)', 'CLIP3 (25999)',
 'CLIP4 (79745)', 'CLK1 (1195)', 'CLK2 (1196)', 'CLK3 (1198)', 'CLK4 (57396)',
 'CLMN (79789)', 'CLMP (79827)', 'CLN3 (1201)', 'CLN5 (1203)', 'CLN6 (54982)',
 'CLN8 (2055)', 'CLNK (116449)', 'CLNS1A (1207)', 'CLOCK (9575)', 'CLP1 (10978)',
 'CLPB (81570)', 'CLPP (8192)', 'CLPS (1208)', 'CLPSL1 (340204)', 'CLPSL2
 (389383)', 'CLPTM1 (1209)', 'CLPTM1L (81037)', 'CLPX (10845)', 'CLRN1 (7401)',
 'CLRN2 (645104)', 'CLRN3 (119467)', 'CLSPN (63967)', 'CLSTN1 (22883)', 'CLSTN2
 (64084)', 'CLSTN3 (9746)', 'CLTA (1211)', 'CLTB (1212)', 'CLTC (1213)', 'CLTCL1
 (8218)', 'CLTRN (57393)', 'CLU (1191)', 'CLUAP1 (23059)', 'CLUH (23277)', 'CLUL1
 (27098)', 'CLVS1 (157807)', 'CLVS2 (134829)', 'CLXN (79645)', 'CLYBL (171425)',
 'CMA1 (1215)', 'CMAS (55907)', 'CMBL (134147)', 'CMC1 (152100)', 'CMC2 (56942)',
 'CMC4 (100272147)', 'CMIP (80790)', 'CMKLR1 (1240)', 'CMKLR2 (2825)', 'CMPK1
 (51727)', 'CMPK2 (129607)', 'CMSS1 (84319)', 'CMTM1 (113540)', 'CMTM2 (146225)',
 'CMTM3 (123920)', 'CMTM4 (146223)', 'CMTM5 (116173)', 'CMTM6 (54918)', 'CMTM7
 (112616)', 'CMTM8 (152189)', 'CMTR1 (23070)', 'CMTR2 (55783)', 'CMYA5 (202333)',
 'CNBD1 (168975)', 'CNBD2 (140894)', 'CNBP (7555)', 'CNDP1 (84735)', 'CNDP2
 (55748)', 'CNEP1R1 (255919)', 'CNFN (84518)', 'CNGA1 (1259)', 'CNGA2 (1260)',
 'CNGA3 (1261)', 'CNGA4 (1262)', 'CNGB1 (1258)', 'CNGB3 (54714)', 'CNIH1
 (10175)', 'CNIH2 (254263)', 'CNIH3 (149111)', 'CNIH4 (29097)', 'CNKSR1 (10256)',
 'CNKSR2 (22866)', 'CNKSR3 (154043)', 'CNMD (11061)', 'CNN1 (1264)', 'CNN2
 (1265)', 'CNN3 (1266)', 'CNNM1 (26507)', 'CNNM2 (54805)', 'CNNM3 (26505)',
 'CNNM4 (26504)', 'CNOT1 (23019)', 'CNOT10 (25904)', 'CNOT11 (55571)', 'CNOT2
 (4848)', 'CNOT3 (4849)', 'CNOT4 (4850)', 'CNOT6 (57472)', 'CNOT6L (246175)',
 'CNOT7 (29883)', 'CNOT8 (9337)', 'CNOT9 (9125)', 'CNP (1267)', 'CNPPD1 (27013)',
 'CNPY1 (285888)', 'CNPY2 (10330)', 'CNPY3 (10695)', 'CNPY4 (245812)', 'CNR1
 (1268)', 'CNR2 (1269)', 'CNRIP1 (25927)', 'CNST (163882)', 'CNTD1 (124817)',
 'CNTF (1270)', 'CNTFR (1271)', 'CNTLN (54875)', 'CNTN1 (1272)', 'CNTN2 (6900)',
 'CNTN3 (5067)', 'CNTN4 (152330)', 'CNTN5 (53942)', 'CNTN6 (27255)', 'CNTNAP1
 (8506)', 'CNTNAP2 (26047)', 'CNTNAP3 (79937)', 'CNTNAP3B (728577)', 'CNTNAP4
 (85445)', 'CNTNAP5 (129684)', 'CNTRL (11064)', 'CNTROB (116840)', 'COA1
 (55744)', 'COA3 (28958)', 'COA4 (51287)', 'COA5 (493753)', 'COA6 (388753)',
 'COA7 (65260)', 'COA8 (84334)', 'COASY (80347)', 'COBL (23242)', 'COBLL1
 (22837)', 'COCH (1690)', 'COG1 (9382)', 'COG2 (22796)', 'COG3 (83548)', 'COG4
 (25839)', 'COG5 (10466)', 'COG6 (57511)', 'COG7 (91949)', 'COG8 (84342)', 'COIL
 (8161)', 'COL10A1 (1300)', 'COL11A1 (1301)', 'COL11A2 (1302)', 'COL12A1 (1303)',
 'COL13A1 (1305)', 'COL14A1 (7373)', 'COL15A1 (1306)', 'COL16A1 (1307)', 'COL17A1
 (1308)', 'COL18A1 (80781)', 'COL19A1 (1310)', 'COL1A1 (1277)', 'COL1A2 (1278)',
 'COL20A1 (57642)', 'COL21A1 (81578)', 'COL22A1 (169044)', 'COL23A1 (91522)',
 'COL24A1 (255631)', 'COL25A1 (84570)', 'COL26A1 (136227)', 'COL27A1 (85301)',
 'COL28A1 (340267)', 'COL2A1 (1280)', 'COL3A1 (1281)', 'COL4A1 (1282)', 'COL4A2
 (1284)', 'COL4A3 (1285)', 'COL4A4 (1286)', 'COL4A5 (1287)', 'COL4A6 (1288)',
 'COL5A1 (1289)', 'COL5A2 (1290)', 'COL5A3 (50509)', 'COL6A1 (1291)', 'COL6A2

(1292)', 'COL6A3 (1293)', 'COL6A5 (256076)', 'COL6A6 (131873)', 'COL7A1 (1294)',
 'COL8A1 (1295)', 'COL8A2 (1296)', 'COL9A1 (1297)', 'COL9A2 (1298)', 'COL9A3
 (1299)', 'COLEC10 (10584)', 'COLEC11 (78989)', 'COLEC12 (81035)', 'COLGALT1
 (79709)', 'COLGALT2 (23127)', 'COLQ (8292)', 'COMMD1 (150684)', 'COMMD10
 (51397)', 'COMMD2 (51122)', 'COMMD3 (23412)', 'COMMD4 (54939)', 'COMMD5
 (28991)', 'COMMD6 (170622)', 'COMMD7 (149951)', 'COMMD8 (54951)', 'COMMD9
 (29099)', 'COMP (1311)', 'COMT (1312)', 'COMTD1 (118881)', 'COP1 (64326)', 'COPA
 (1314)', 'COPB1 (1315)', 'COPB2 (9276)', 'COPE (11316)', 'COPG1 (22820)', 'COPG2
 (26958)', 'COPRS (55352)', 'COPS2 (9318)', 'COPS3 (8533)', 'COPS4 (51138)',
 'COPS5 (10987)', 'COPS6 (10980)', 'COPS7A (50813)', 'COPS7B (64708)', 'COPS8
 (10920)', 'COPS9 (150678)', 'COPZ1 (22818)', 'COPZ2 (51226)', 'COQ10A (93058)',
 'COQ10B (80219)', 'COQ2 (27235)', 'COQ3 (51805)', 'COQ4 (51117)', 'COQ5
 (84274)', 'COQ6 (51004)', 'COQ7 (10229)', 'COQ8A (56997)', 'COQ8B (79934)',
 'COQ9 (57017)', 'CORIN (10699)', 'COR01A (11151)', 'COR01B (57175)', 'COR01C
 (23603)', 'COR02A (7464)', 'COR02B (10391)', 'COR06 (84940)', 'COR07 (79585)',
 'CORT (1325)', 'COTL1 (23406)', 'COX10 (1352)', 'COX11 (1353)', 'COX14 (84987)',
 'COX15 (1355)', 'COX16 (51241)', 'COX17 (10063)', 'COX18 (285521)', 'COX19
 (90639)', 'COX20 (116228)', 'COX4I1 (1327)', 'COX4I2 (84701)', 'COX5A (9377)',
 'COX5B (1329)', 'COX6A1 (1337)', 'COX6A2 (1339)', 'COX6B1 (1340)', 'COX6B2
 (125965)', 'COX6C (1345)', 'COX7A1 (1346)', 'COX7A2 (1347)', 'COX7A2L (9167)',
 'COX7B (1349)', 'COX7B2 (170712)', 'COX7C (1350)', 'COX8A (1351)', 'COX8C
 (341947)', 'CP (1356)', 'CPA1 (1357)', 'CPA2 (1358)', 'CPA3 (1359)', 'CPA4
 (51200)', 'CPA5 (93979)', 'CPA6 (57094)', 'CPAMD8 (27151)', 'CPAP (55835)',
 'CPB1 (1360)', 'CPB2 (1361)', 'CPD (1362)', 'CPE (1363)', 'CPEB1 (64506)',
 'CPEB2 (132864)', 'CPEB3 (22849)', 'CPEB4 (80315)', 'CPED1 (79974)', 'CPHXL
 (105371346)', 'CPLANE1 (65250)', 'CPLANE2 (79363)', 'CPLX1 (10815)', 'CPLX2
 (10814)', 'CPLX3 (594855)', 'CPLX4 (339302)', 'CPM (1368)', 'CPN1 (1369)', 'CPN2
 (1370)', 'CPNE1 (8904)', 'CPNE2 (221184)', 'CPNE3 (8895)', 'CPNE4 (131034)',
 'CPNE5 (57699)', 'CPNE6 (9362)', 'CPNE7 (27132)', 'CPNE8 (144402)', 'CPNE9
 (151835)', 'CPO (130749)', 'CPOX (1371)', 'CPPED1 (55313)', 'CPQ (10404)', 'CPS1
 (1373)', 'CPSF1 (29894)', 'CPSF2 (53981)', 'CPSF3 (51692)', 'CPSF4 (10898)',
 'CPSF4L (642843)', 'CPSF6 (11052)', 'CPSF7 (79869)', 'CPT1A (1374)', 'CPT1B
 (1375)', 'CPT1C (126129)', 'CPT2 (1376)', 'CPTP (80772)', 'CPVL (54504)',
 'CPXCR1 (53336)', 'CPXM1 (56265)', 'CPXM2 (119587)', 'CPZ (8532)', 'CR1 (1378)',
 'CR1L (1379)', 'CR2 (1380)', 'CRABP1 (1381)', 'CRABP2 (1382)', 'CRACD (57482)',
 'CRACDL (343990)', 'CRACR2A (84766)', 'CRACR2B (283229)', 'CRADD (8738)',
 'CRAMP1 (57585)', 'CRAT (1384)', 'CRB1 (23418)', 'CRB2 (286204)', 'CRB3
 (92359)', 'CRBN (51185)', 'CRCP (27297)', 'CRCT1 (54544)', 'CREB1 (1385)',
 'CREB3 (10488)', 'CREB3L1 (90993)', 'CREB3L2 (64764)', 'CREB3L3 (84699)',
 'CREB3L4 (148327)', 'CREB5 (9586)', 'CREBBP (1387)', 'CREBL2 (1389)', 'CREBRF
 (153222)', 'CREBZF (58487)', 'CREG1 (8804)', 'CREG2 (200407)', 'CRELD1 (78987)',
 'CRELD2 (79174)', 'CREM (1390)', 'CRH (1392)', 'CRHBP (1393)', 'CRHR1 (1394)',
 'CRHR2 (1395)', 'CRIM1 (51232)', 'CRIP1 (1396)', 'CRIP2 (1397)', 'CRIP3
 (401262)', 'CRIPT (9419)', 'CRIPTO (6997)', 'CRISP1 (167)', 'CRISP2 (7180)',
 'CRISP3 (10321)', 'CRISPLD1 (83690)', 'CRISPLD2 (83716)', 'CRK (1398)', 'CRKL
 (1399)', 'CRLF1 (9244)', 'CRLF2 (64109)', 'CRLF3 (51379)', 'CRLS1 (54675)',
 'CRMP1 (1400)', 'CRNKL1 (51340)', 'CRNN (49860)', 'CROCC (9696)', 'CROCC2
 (728763)', 'CROT (54677)', 'CRP (1401)', 'CRTAC1 (55118)', 'CRTAM (56253)',

'CRTAP (10491)', 'CRTC1 (23373)', 'CRTC2 (200186)', 'CRTC3 (64784)', 'CRX
 (1406)', 'CRY1 (1407)', 'CRY2 (1408)', 'CRYAA (1409)', 'CRYAB (1410)', 'CRYBA1
 (1411)', 'CRYBA2 (1412)', 'CRYBA4 (1413)', 'CRYBB1 (1414)', 'CRYBB2 (1415)',
 'CRYBB3 (1417)', 'CRYBG1 (202)', 'CRYBG3 (131544)', 'CRYGA (1418)', 'CRYGB
 (1419)', 'CRYGC (1420)', 'CRYGD (1421)', 'CRYGN (155051)', 'CRYGS (1427)',
 'CRYL1 (51084)', 'CRYM (1428)', 'CRYZ (1429)', 'CRYZL1 (9946)', 'CS (1431)',
 'CSAD (51380)', 'CSAG1 (158511)', 'CSAG2 (102723547)', 'CSDC2 (27254)', 'CSDE1
 (7812)', 'CSE1L (1434)', 'CSF1 (1435)', 'CSF1R (1436)', 'CSF2 (1437)', 'CSF2RA
 (1438)', 'CSF2RB (1439)', 'CSF3 (1440)', 'CSF3R (1441)', 'CSGALNACT1 (55790)',
 'CSGALNACT2 (55454)', 'CSH2 (1443)', 'CSHL1 (1444)', 'CSK (1445)', 'CSKMT
 (751071)', 'CSMD1 (64478)', 'CSMD2 (114784)', 'CSMD3 (114788)', 'CSN1S1 (1446)',
 'CSN2 (1447)', 'CSN3 (1448)', 'CSNK1A1 (1452)', 'CSNK1A1L (122011)', 'CSNK1D
 (1453)', 'CSNK1G1 (53944)', 'CSNK1G2 (1455)', 'CSNK1G3 (1456)', 'CSNK2A1
 (1457)', 'CSNK2A2 (1459)', 'CSNK2A2IP (111064647)', 'CSNK2A3 (283106)', 'CSNK2B
 (1460)', 'CSPG4 (1464)', 'CSPG5 (10675)', 'CSPP1 (79848)', 'CSRNP1 (64651)',
 'CSRNP2 (81566)', 'CSRNP3 (80034)', 'CSRP1 (1465)', 'CSRP2 (1466)', 'CSRP3
 (8048)', 'CST1 (1469)', 'CST11 (140880)', 'CST2 (1470)', 'CST3 (1471)', 'CST4
 (1472)', 'CST5 (1473)', 'CST6 (1474)', 'CST7 (8530)', 'CST8 (10047)', 'CST9
 (128822)', 'CST9L (128821)', 'CSTA (1475)', 'CTSB (1476)', 'CSTF1 (1477)',
 'CSTF2 (1478)', 'CSTF2T (23283)', 'CSTF3 (1479)', 'CSTL1 (128817)', 'CSTPP1
 (79096)', 'CT47B1 (643311)', 'CT55 (54967)', 'CT83 (203413)', 'CTAG2 (30848)',
 'CTAGE1 (64693)', 'CTAGE9 (643854)', 'CTBP1 (1487)', 'CTBP2 (1488)', 'CTBS
 (1486)', 'CTC1 (80169)', 'CTCF (10664)', 'CTCFL (140690)', 'CTDNEP1 (23399)',
 'CTDP1 (9150)', 'CTDSP1 (58190)', 'CTDSP2 (10106)', 'CTDSPL (10217)', 'CTDSPL2
 (51496)', 'CTF1 (1489)', 'CTH (1491)', 'CTHRC1 (115908)', 'CTIF (9811)', 'CTLA4
 (1493)', 'CTNNA1 (1495)', 'CTNNA2 (1496)', 'CTNNA3 (29119)', 'CTNNAL1 (8727)',
 'CTNNB1 (1499)', 'CTNNBIP1 (56998)', 'CTNNBL1 (56259)', 'CTNND1 (1500)', 'CTNND2
 (1501)', 'CTNS (1497)', 'CTPS1 (1503)', 'CTPS2 (56474)', 'CTR9 (9646)', 'CTRB1
 (1504)', 'CTRB2 (440387)', 'CTRC (11330)', 'CTRL (1506)', 'CTSA (5476)', 'CTSB
 (1508)', 'CTSC (1075)', 'CTSD (1509)', 'CTSE (1510)', 'CTSF (8722)', 'CTSG
 (1511)', 'CTSH (1512)', 'CTSK (1513)', 'CTSL (1514)', 'CTSO (1519)', 'CTSS
 (1520)', 'CTSV (1515)', 'CTSW (1521)', 'CTSZ (1522)', 'CTTN (2017)', 'CTTNBP2
 (83992)', 'CTTNBP2NL (55917)', 'CTU1 (90353)', 'CTU2 (348180)', 'CTXN1
 (404217)', 'CTXN2 (399697)', 'CTXN3 (613212)', 'CTXND1 (100996492)', 'CUBN
 (8029)', 'CUEDC1 (404093)', 'CUEDC2 (79004)', 'CUL1 (8454)', 'CUL2 (8453)',
 'CUL3 (8452)', 'CUL4A (8451)', 'CUL4B (8450)', 'CUL5 (8065)', 'CUL7 (9820)',
 'CUL9 (23113)', 'CUTA (51596)', 'CUTC (51076)', 'CUX1 (1523)', 'CUX2 (23316)',
 'CUZD1 (50624)', 'CWC15 (51503)', 'CWC22 (57703)', 'CWC25 (54883)', 'CWC27
 (10283)', 'CWF19L1 (55280)', 'CWF19L2 (143884)', 'CWH43 (80157)', 'CX3CL1
 (6376)', 'CX3CR1 (1524)', 'CXADR (1525)', 'CXCL1 (2919)', 'CXCL10 (3627)',
 'CXCL11 (6373)', 'CXCL12 (6387)', 'CXCL13 (10563)', 'CXCL14 (9547)', 'CXCL16
 (58191)', 'CXCL17 (284340)', 'CXCL2 (2920)', 'CXCL3 (2921)', 'CXCL5 (6374)',
 'CXCL6 (6372)', 'CXCL8 (3576)', 'CXCL9 (4283)', 'CXCR1 (3577)', 'CXCR2 (3579)',
 'CXCR3 (2833)', 'CXCR4 (7852)', 'CXCR5 (643)', 'CXCR6 (10663)', 'CXXC1 (30827)',
 'CXXC4 (80319)', 'CXXC5 (51523)', 'CXorf38 (159013)', 'CXorf58 (254158)',
 'CXorf65 (158830)', 'CXorf66 (347487)', 'CYB561 (1534)', 'CYB561A3 (220002)',
 'CYB561D1 (284613)', 'CYB561D2 (11068)', 'CYB5A (1528)', 'CYB5B (80777)',
 'CYB5D1 (124637)', 'CYB5D2 (124936)', 'CYB5R1 (51706)', 'CYB5R2 (51700)',

'CYB5R3 (1727)', 'CYB5R4 (51167)', 'CYB5RL (606495)', 'CYBA (1535)', 'CYBB
 (1536)', 'CYBC1 (79415)', 'CYBRD1 (79901)', 'CYC1 (1537)', 'CYCS (54205)',
 'CYFIP1 (23191)', 'CYFIP2 (26999)', 'CYGB (114757)', 'CYLC1 (1538)', 'CYLC2
 (1539)', 'CYLD (1540)', 'CYP11A1 (1583)', 'CYP11B1 (1584)', 'CYP11B2 (1585)',
 'CYP17A1 (1586)', 'CYP19A1 (1588)', 'CYP1A1 (1543)', 'CYP1A2 (1544)', 'CYP1B1
 (1545)', 'CYP20A1 (57404)', 'CYP21A2 (1589)', 'CYP24A1 (1591)', 'CYP26A1
 (1592)', 'CYP26B1 (56603)', 'CYP26C1 (340665)', 'CYP27A1 (1593)', 'CYP27B1
 (1594)', 'CYP27C1 (339761)', 'CYP2A13 (1553)', 'CYP2A6 (1548)', 'CYP2A7 (1549)',
 'CYP2B6 (1555)', 'CYP2C18 (1562)', 'CYP2C19 (1557)', 'CYP2C8 (1558)', 'CYP2C9
 (1559)', 'CYP2D6 (1565)', 'CYP2E1 (1571)', 'CYP2F1 (1572)', 'CYP2J2 (1573)',
 'CYP2R1 (120227)', 'CYP2S1 (29785)', 'CYP2U1 (113612)', 'CYP2W1 (54905)',
 'CYP39A1 (51302)', 'CYP3A4 (1576)', 'CYP3A43 (64816)', 'CYP3A5 (1577)', 'CYP3A7
 (1551)', 'CYP46A1 (10858)', 'CYP4A11 (1579)', 'CYP4A22 (284541)', 'CYP4B1
 (1580)', 'CYP4F11 (57834)', 'CYP4F12 (66002)', 'CYP4F2 (8529)', 'CYP4F22
 (126410)', 'CYP4F3 (4051)', 'CYP4F8 (11283)', 'CYP4V2 (285440)', 'CYP4X1
 (260293)', 'CYP4Z1 (199974)', 'CYP51A1 (1595)', 'CYP7A1 (1581)', 'CYP7B1
 (9420)', 'CYP8B1 (1582)', 'CYREN (78996)', 'CYRIA (81553)', 'CYRIB (51571)',
 'CYS1 (192668)', 'CYSLTR1 (10800)', 'CYSLTR2 (57105)', 'CYSRT1 (375791)',
 'CYSTM1 (84418)', 'CYTH1 (9267)', 'CYTH2 (9266)', 'CYTH3 (9265)', 'CYTH4
 (27128)', 'CYTIP (9595)', 'CYTL1 (54360)', 'CYR1 (116159)', 'CZIB (54987)',
 'D2HGDH (728294)', 'DAAM1 (23002)', 'DAAM2 (23500)', 'DAB1 (1600)', 'DAB2
 (1601)', 'DAB2IP (153090)', 'DACH1 (1602)', 'DACH2 (117154)', 'DACT1 (51339)',
 'DACT2 (168002)', 'DACT3 (147906)', 'DAD1 (1603)', 'DAG1 (1605)', 'DAGLA (747)',
 'DAGLB (221955)', 'DALRD3 (55152)', 'DAND5 (199699)', 'DAO (1610)', 'DAOA
 (267012)', 'DAP (1611)', 'DAP3 (7818)', 'DAPK1 (1612)', 'DAPK2 (23604)', 'DAPK3
 (1613)', 'DAPL1 (92196)', 'DAPP1 (27071)', 'DARS1 (1615)', 'DARS2 (55157)',
 'DAW1 (164781)', 'DAXX (1616)', 'DAZAP1 (26528)', 'DAZAP2 (9802)', 'DAZL
 (1618)', 'DBF4 (10926)', 'DBF4B (80174)', 'DBH (1621)', 'DBI (1622)', 'DBN1
 (1627)', 'DBNDD1 (79007)', 'DBNDD2 (55861)', 'DBNL (28988)', 'DBP (1628)', 'DBR1
 (51163)', 'DBT (1629)', 'DBX1 (120237)', 'DBX2 (440097)', 'DCAF1 (9730)',
 'DCAF10 (79269)', 'DCAF11 (80344)', 'DCAF12 (25853)', 'DCAF12L1 (139170)',
 'DCAF12L2 (340578)', 'DCAF13 (25879)', 'DCAF15 (90379)', 'DCAF16 (54876)',
 'DCAF17 (80067)', 'DCAF4 (26094)', 'DCAF4L1 (285429)', 'DCAF4L2 (138009)',
 'DCAF5 (8816)', 'DCAF6 (55827)', 'DCAF7 (10238)', 'DCAF8 (50717)', 'DCAF8L1
 (139425)', 'DCAF8L2 (347442)', 'DCAKD (79877)', 'DCANP1 (140947)', 'DCBLD1
 (285761)', 'DCBLD2 (131566)', 'DCC (1630)', 'DCD (117159)', 'DCDC1 (341019)',
 'DCDC2 (51473)', 'DCDC2B (149069)', 'DCDC2C (728597)', 'DCHS1 (8642)', 'DCHS2
 (54798)', 'DCK (1633)', 'DCLK1 (9201)', 'DCLK2 (166614)', 'DCLK3 (85443)',
 'DCLRE1A (9937)', 'DCLRE1B (64858)', 'DCLRE1C (64421)', 'DCN (1634)', 'DCP1A
 (55802)', 'DCP1B (196513)', 'DCP2 (167227)', 'DCPS (28960)', 'DCST1 (149095)',
 'DCST2 (127579)', 'DCSTAMP (81501)', 'DCT (1638)', 'DCTD (1635)', 'DCTN1
 (1639)', 'DCTN2 (10540)', 'DCTN3 (11258)', 'DCTN4 (51164)', 'DCTN5 (84516)',
 'DCTN6 (10671)', 'DCTPP1 (79077)', 'DCUN1D1 (54165)', 'DCUN1D2 (55208)',
 'DCUN1D3 (123879)', 'DCUN1D4 (23142)', 'DCUN1D5 (84259)', 'DCX (1641)', 'DCXR
 (51181)', 'DDA1 (79016)', 'DDAH1 (23576)', 'DDAH2 (23564)', 'DDB1 (1642)', 'DDB2
 (1643)', 'DDC (1644)', 'DDHD1 (80821)', 'DDHD2 (23259)', 'DDI1 (414301)', 'DDI2
 (84301)', 'DDIAS (220042)', 'DDIT3 (1649)', 'DDIT4 (54541)', 'DDIT4L (115265)',
 'DDN (23109)', 'DDO (8528)', 'DDOST (1650)', 'DDR1 (780)', 'DDR2 (4921)',

'DDR GK1 (65992)', 'DDX1 (1653)', 'DDX10 (1662)', 'DDX11 (1663)', 'DDX17
 (10521)', 'DDX18 (8886)', 'DDX19A (55308)', 'DDX19B (11269)', 'DDX20 (11218)',
 'DDX21 (9188)', 'DDX23 (9416)', 'DDX24 (57062)', 'DDX25 (29118)', 'DDX27
 (55661)', 'DDX28 (55794)', 'DDX31 (64794)', 'DDX39A (10212)', 'DDX39B (7919)',
 'DDX3X (1654)', 'DDX3Y (8653)', 'DDX4 (54514)', 'DDX41 (51428)', 'DDX42
 (11325)', 'DDX43 (55510)', 'DDX46 (9879)', 'DDX47 (51202)', 'DDX49 (54555)',
 'DDX5 (1655)', 'DDX50 (79009)', 'DDX51 (317781)', 'DDX52 (11056)', 'DDX53
 (168400)', 'DDX54 (79039)', 'DDX55 (57696)', 'DDX56 (54606)', 'DDX59 (83479)',
 'DDX6 (1656)', 'DDX60 (55601)', 'DDX60L (91351)', 'DEAF1 (10522)', 'DECR1
 (1666)', 'DECR2 (26063)', 'DEDD (9191)', 'DEDD2 (162989)', 'DEF6 (50619)', 'DEF8
 (54849)', 'DEFA4 (1669)', 'DEFA5 (1670)', 'DEFA6 (1671)', 'DEFB1 (1672)',
 'DEFB108B (245911)', 'DEFB110 (245913)', 'DEFB112 (245915)', 'DEFB113 (245927)',
 'DEFB114 (245928)', 'DEFB115 (245929)', 'DEFB116 (245930)', 'DEFB118 (117285)',
 'DEFB119 (245932)', 'DEFB121 (245934)', 'DEFB123 (245936)', 'DEFB124 (245937)',
 'DEFB125 (245938)', 'DEFB126 (81623)', 'DEFB127 (140850)', 'DEFB128 (245939)',
 'DEFB129 (140881)', 'DEFB131A (644414)', 'DEFB132 (400830)', 'DEFB134 (613211)',
 'DEFB135 (613209)', 'DEFB136 (613210)', 'DEGS1 (8560)', 'DEGS2 (123099)', 'DEK
 (7913)', 'DELE1 (9812)', 'DENND10 (404636)', 'DENND11 (57189)', 'DENND1A
 (57706)', 'DENND1B (163486)', 'DENND1C (79958)', 'DENND2A (27147)', 'DENND2B
 (6764)', 'DENND2C (163259)', 'DENND2D (79961)', 'DENND3 (22898)', 'DENND4A
 (10260)', 'DENND4B (9909)', 'DENND4C (55667)', 'DENND5A (23258)', 'DENND5B
 (160518)', 'DENND6A (201627)', 'DENND6B (414918)', 'DENR (8562)', 'DEPDC1
 (55635)', 'DEPDC1B (55789)', 'DEPDC4 (120863)', 'DEPDC5 (9681)', 'DEPDC7
 (91614)', 'DEPP1 (11067)', 'DEPTOR (64798)', 'DERA (51071)', 'DERL1 (79139)',
 'DERL2 (51009)', 'DERL3 (91319)', 'DERPC (113455421)', 'DES (1674)', 'DESI1
 (27351)', 'DESI2 (51029)', 'DET1 (55070)', 'DEUP1 (159989)', 'DEXI (28955)',
 'DFFA (1676)', 'DFFB (1677)', 'DGAT1 (8694)', 'DGAT2 (84649)', 'DGAT2L6
 (347516)', 'DGCR2 (9993)', 'DGCR6 (8214)', 'DGCR6L (85359)', 'DGCR8 (54487)',
 'DGKA (1606)', 'DGKB (1607)', 'DGKD (8527)', 'DGKE (8526)', 'DGKG (1608)', 'DGKH
 (160851)', 'DGKI (9162)', 'DGKK (139189)', 'DGKQ (1609)', 'DGKZ (8525)', 'DGLUCY
 (80017)', 'DGUOK (1716)', 'DHCR24 (1718)', 'DHCR7 (1717)', 'DHDDS (79947)',
 'DHDH (27294)', 'DHFR (1719)', 'DHFR2 (200895)', 'DHH (50846)', 'DHODH (1723)',
 'DHPS (1725)', 'DHRS1 (115817)', 'DHRS11 (79154)', 'DHRS12 (79758)', 'DHRS13
 (147015)', 'DHRS2 (10202)', 'DHRS3 (9249)', 'DHRS4 (10901)', 'DHRS4L2 (317749)',
 'DHRS7 (51635)', 'DHRS7B (25979)', 'DHRS7C (201140)', 'DHRS9 (10170)', 'DHRSX
 (207063)', 'DHTKD1 (55526)', 'DHX15 (1665)', 'DHX16 (8449)', 'DHX29 (54505)',
 'DHX30 (22907)', 'DHX32 (55760)', 'DHX33 (56919)', 'DHX34 (9704)', 'DHX35
 (60625)', 'DHX36 (170506)', 'DHX37 (57647)', 'DHX38 (9785)', 'DHX40 (79665)',
 'DHX57 (90957)', 'DHX58 (79132)', 'DHX8 (1659)', 'DHX9 (1660)', 'DIABLO
 (56616)', 'DIAPH1 (1729)', 'DIAPH2 (1730)', 'DIAPH3 (81624)', 'DICER1 (23405)',
 'DID01 (11083)', 'DINT1 (27292)', 'DIO1 (1733)', 'DIO2 (1734)', 'DIO3 (1735)',
 'DIP2A (23181)', 'DIP2B (57609)', 'DIP2C (22982)', 'DIPK1A (388650)', 'DIPK1B
 (138311)', 'DIPK1C (125704)', 'DIPK2A (205428)', 'DIPK2B (79742)', 'DIRAS1
 (148252)', 'DIRAS2 (54769)', 'DIRAS3 (9077)', 'DIS3 (22894)', 'DIS3L (115752)',
 'DIS3L2 (129563)', 'DISC1 (27185)', 'DISP1 (84976)', 'DISP2 (85455)', 'DISP3
 (57540)', 'DIXDC1 (85458)', 'DKC1 (1736)', 'DKK1 (22943)', 'DKK2 (27123)', 'DKK3
 (27122)', 'DKK4 (27121)', 'DKKL1 (27120)', 'DLAT (1737)', 'DLC1 (10395)', 'DLD
 (1738)', 'DLEC1 (9940)', 'DLEU7 (220107)', 'DLG1 (1739)', 'DLG2 (1740)', 'DLG3

(1741)', 'DLG4 (1742)', 'DLG5 (9231)', 'DLGAP1 (9229)', 'DLGAP2 (9228)', 'DLGAP3 (58512)', 'DLGAP4 (22839)', 'DLGAP5 (9787)', 'DLK1 (8788)', 'DLK2 (65989)', 'DLL1 (28514)', 'DLL3 (10683)', 'DLL4 (54567)', 'DLST (1743)', 'DLX1 (1745)', 'DLX2 (1746)', 'DLX3 (1747)', 'DLX4 (1748)', 'DLX5 (1749)', 'DLX6 (1750)', 'DMAC1 (90871)', 'DMAC2 (55101)', 'DMAC2L (27109)', 'DMAP1 (55929)', 'DMBT1 (1755)', 'DMBX1 (127343)', 'DMC1 (11144)', 'DMD (1756)', 'DMGDH (29958)', 'DMKN (93099)', 'DMP1 (1758)', 'DMPK (1760)', 'DMRT1 (1761)', 'DMRT2 (10655)', 'DMRT3 (58524)', 'DMRTA1 (63951)', 'DMRTA2 (63950)', 'DMRTB1 (63948)', 'DMRTC2 (63946)', 'DMTF1 (9988)', 'DMTN (2039)', 'DMWD (1762)', 'DMXL1 (1657)', 'DMXL2 (23312)', 'DNA2 (1763)', 'DNAAF1 (123872)', 'DNAAF10 (116143)', 'DNAAF11 (23639)', 'DNAAF19 (388389)', 'DNAAF2 (55172)', 'DNAAF3 (352909)', 'DNAAF4 (161582)', 'DNAAF5 (54919)', 'DNAAF6 (139212)', 'DNAAF8 (146562)', 'DNAAF9 (25943)', 'DNAH1 (25981)', 'DNAH10 (196385)', 'DNAH11 (8701)', 'DNAH12 (201625)', 'DNAH14 (127602)', 'DNAH17 (8632)', 'DNAH2 (146754)', 'DNAH3 (55567)', 'DNAH5 (1767)', 'DNAH6 (1768)', 'DNAH7 (56171)', 'DNAH8 (1769)', 'DNAH9 (1770)', 'DNAI1 (27019)', 'DNAI2 (64446)', 'DNAI3 (126820)', 'DNAI4 (79819)', 'DNAI7 (55259)', 'DNAJA1 (3301)', 'DNAJA2 (10294)', 'DNAJA3 (9093)', 'DNAJA4 (55466)', 'DNAJB1 (3337)', 'DNAJB11 (51726)', 'DNAJB12 (54788)', 'DNAJB13 (374407)', 'DNAJB14 (79982)', 'DNAJB2 (3300)', 'DNAJB4 (11080)', 'DNAJB5 (25822)', 'DNAJB6 (10049)', 'DNAJB7 (150353)', 'DNAJB8 (165721)', 'DNAJB9 (4189)', 'DNAJC1 (64215)', 'DNAJC10 (54431)', 'DNAJC11 (55735)', 'DNAJC12 (56521)', 'DNAJC13 (23317)', 'DNAJC14 (85406)', 'DNAJC15 (29103)', 'DNAJC16 (23341)', 'DNAJC17 (55192)', 'DNAJC18 (202052)', 'DNAJC19 (131118)', 'DNAJC2 (27000)', 'DNAJC21 (134218)', 'DNAJC22 (79962)', 'DNAJC24 (120526)', 'DNAJC25 (548645)', 'DNAJC27 (51277)', 'DNAJC28 (54943)', 'DNAJC3 (5611)', 'DNAJC30 (84277)', 'DNAJC4 (3338)', 'DNAJC5 (80331)', 'DNAJC5B (85479)', 'DNAJC5G (285126)', 'DNAJC6 (9829)', 'DNAJC7 (7266)', 'DNAJC8 (22826)', 'DNAJC9 (23234)', 'DNAL1 (83544)', 'DNAL4 (10126)', 'DNALI1 (7802)', 'DNASE1 (1773)', 'DNASE1L1 (1774)', 'DNASE1L2 (1775)', 'DNASE1L3 (1776)', 'DNASE2 (1777)', 'DNASE2B (58511)', 'DND1 (373863)', 'DNER (92737)', 'DNHD1 (144132)', 'DNLZ (728489)', 'DNM1 (1759)', 'DNM1L (10059)', 'DNM2 (1785)', 'DNM3 (26052)', 'DNMBP (23268)', 'DNMT1 (1786)', 'DNMT3A (1788)', 'DNMT3B (1789)', 'DNMT3L (29947)', 'DNPEP (23549)', 'DNPH1 (10591)', 'DNTT (1791)', 'DNTTIP1 (116092)', 'DNTTIP2 (30836)', 'DOC2A (8448)', 'DOC2B (8447)', 'DOCK1 (1793)', 'DOCK10 (55619)', 'DOCK11 (139818)', 'DOCK2 (1794)', 'DOCK3 (1795)', 'DOCK4 (9732)', 'DOCK5 (80005)', 'DOCK6 (57572)', 'DOCK7 (85440)', 'DOCK8 (81704)', 'DOCK9 (23348)', 'DOHH (83475)', 'DOK1 (1796)', 'DOK2 (9046)', 'DOK3 (79930)', 'DOK4 (55715)', 'DOK5 (55816)', 'DOK6 (220164)', 'DOK7 (285489)', 'DOLK (22845)', 'DOLPP1 (57171)', 'DONSON (29980)', 'DOP1A (23033)', 'DOP1B (9980)', 'DORIP1 (122525)', 'DOT1L (84444)', 'DPAGT1 (1798)', 'DPCD (25911)', 'DPEP1 (1800)', 'DPEP2 (64174)', 'DPEP2NB (100131303)', 'DPEP3 (64180)', 'DPF1 (8193)', 'DPF2 (5977)', 'DPF3 (8110)', 'DPH1 (1801)', 'DPH2 (1802)', 'DPH3 (285381)', 'DPH5 (51611)', 'DPH6 (89978)', 'DPH7 (92715)', 'DPM1 (8813)', 'DPM2 (8818)', 'DPM3 (54344)', 'DPP10 (57628)', 'DPP3 (10072)', 'DPP4 (1803)', 'DPP6 (1804)', 'DPP7 (29952)', 'DPP8 (54878)', 'DPP9 (91039)', 'DPPA2 (151871)', 'DPPA3 (359787)', 'DPPA4 (55211)', 'DPPA5 (340168)', 'DPRX (503834)', 'DPT (1805)', 'DPY19L1 (23333)', 'DPY19L2 (283417)', 'DPY19L3 (147991)', 'DPY19L4 (286148)', 'DPY30 (84661)', 'DPYD (1806)', 'DPYS (1807)', 'DPYSL2 (1808)', 'DPYSL3 (1809)', 'DPYSL4

(10570)', 'DPYSL5 (56896)', 'DQX1 (165545)', 'DR1 (1810)', 'DRAM1 (55332)',
 'DRAM2 (128338)', 'DRAP1 (10589)', 'DRAXIN (374946)', 'DRC1 (92749)', 'DRC12
 (283152)', 'DRC3 (83450)', 'DRC7 (84229)', 'DRD1 (1812)', 'DRD2 (1813)', 'DRD3
 (1814)', 'DRD4 (1815)', 'DRD5 (1816)', 'DRG1 (4733)', 'DRG2 (1819)', 'DRGX
 (644168)', 'DRICH1 (51233)', 'DROSHA (29102)', 'DRP2 (1821)', 'DSC1 (1823)',
 'DSC2 (1824)', 'DSC3 (1825)', 'DSCAM (1826)', 'DSCAML1 (57453)', 'DSCC1
 (79075)', 'DSE (29940)', 'DSEL (92126)', 'DSG1 (1828)', 'DSG2 (1829)', 'DSG3
 (1830)', 'DSG4 (147409)', 'DSN1 (79980)', 'DSP (1832)', 'DSPP (1834)', 'DST
 (667)', 'DSTN (11034)', 'DSTYK (25778)', 'DTD1 (92675)', 'DTD2 (112487)', 'DTHD1
 (401124)', 'DTL (51514)', 'DTNA (1837)', 'DTNB (1838)', 'DTNBP1 (84062)', 'DTWD1
 (56986)', 'DTWD2 (285605)', 'DTX1 (1840)', 'DTX2 (113878)', 'DTX3 (196403)',
 'DTX3L (151636)', 'DTX4 (23220)', 'DTYMK (1841)', 'DUOX1 (53905)', 'DUOX2
 (50506)', 'DUOXA1 (90527)', 'DUOXA2 (405753)', 'DUS1L (64118)', 'DUS2 (54920)',
 'DUS3L (56931)', 'DUS4L (11062)', 'DUSP1 (1843)', 'DUSP10 (11221)', 'DUSP12
 (11266)', 'DUSP13B (51207)', 'DUSP14 (11072)', 'DUSP15 (128853)', 'DUSP16
 (80824)', 'DUSP18 (150290)', 'DUSP19 (142679)', 'DUSP2 (1844)', 'DUSP21
 (63904)', 'DUSP22 (56940)', 'DUSP23 (54935)', 'DUSP26 (78986)', 'DUSP28
 (285193)', 'DUSP29 (338599)', 'DUSP3 (1845)', 'DUSP4 (1846)', 'DUSP5 (1847)',
 'DUSP6 (1848)', 'DUSP7 (1849)', 'DUSP8 (1850)', 'DUSP9 (1852)', 'DUT (1854)',
 'DUXA (503835)', 'DUXB (100033411)', 'DVL1 (1855)', 'DVL2 (1856)', 'DVL3
 (1857)', 'DX0 (1797)', 'DYDC1 (143241)', 'DYDC2 (84332)', 'DYM (54808)', 'DYNAP
 (284254)', 'DYNC1H1 (1778)', 'DYNC1I1 (1780)', 'DYNC1I2 (1781)', 'DYNC1LI1
 (51143)', 'DYNC1LI2 (1783)', 'DYNC2H1 (79659)', 'DYNC2I1 (55112)', 'DYNC2I2
 (89891)', 'DYNC2LI1 (51626)', 'DYNLL1 (8655)', 'DYNLL2 (140735)', 'DYNLRB1
 (83658)', 'DYNLRB2 (83657)', 'DYNLT1 (6993)', 'DYNLT2 (6991)', 'DYNLT2B
 (255758)', 'DYNLT3 (6990)', 'DYNLT4 (343521)', 'DYNLT5 (200132)', 'DYRK1A
 (1859)', 'DYRK1B (9149)', 'DYRK2 (8445)', 'DYRK3 (8444)', 'DYRK4 (8798)', 'DYSF
 (8291)', 'DYTN (391475)', 'DZANK1 (55184)', 'DZIP1 (22873)', 'DZIP1L (199221)',
 'DZIP3 (9666)', 'E2F1 (1869)', 'E2F2 (1870)', 'E2F3 (1871)', 'E2F4 (1874)',
 'E2F5 (1875)', 'E2F6 (1876)', 'E2F7 (144455)', 'E2F8 (79733)', 'E4F1 (1877)',
 'EAF1 (85403)', 'EAF2 (55840)', 'EAPP (55837)', 'EARS2 (124454)', 'EBAG9
 (9166)', 'EBF1 (1879)', 'EBF2 (64641)', 'EBF3 (253738)', 'EBF4 (57593)', 'EBI3
 (10148)', 'EBLN1 (340900)', 'EBLN2 (55096)', 'EBNA1BP2 (10969)', 'EBP (10682)',
 'EBPL (84650)', 'ECD (11319)', 'ECE1 (1889)', 'ECE2 (9718)', 'ECEL1 (9427)',
 'ECH1 (1891)', 'ECHDC1 (55862)', 'ECHDC2 (55268)', 'ECHDC3 (79746)', 'ECHS1
 (1892)', 'ECI1 (1632)', 'ECI2 (10455)', 'ECM1 (1893)', 'ECM2 (1842)', 'ECPAS
 (23392)', 'ECRG4 (84417)', 'ECSCR (641700)', 'ECSIT (51295)', 'ECT2 (1894)',
 'ECT2L (345930)', 'EDA (1896)', 'EDA2R (60401)', 'EDAR (10913)', 'EDARADD
 (128178)', 'EDC3 (80153)', 'EDC4 (23644)', 'EDDM13 (100506374)', 'EDDM3A
 (10876)', 'EDDM3B (64184)', 'EDEM1 (9695)', 'EDEM2 (55741)', 'EDEM3 (80267)',
 'EDF1 (8721)', 'EDIL3 (10085)', 'EDN1 (1906)', 'EDN2 (1907)', 'EDN3 (1908)',
 'EDNRA (1909)', 'EDNRB (1910)', 'EDRF1 (26098)', 'EEA1 (8411)', 'EED (8726)',
 'EEF1A1 (1915)', 'EEF1A2 (1917)', 'EEF1AKMT1 (221143)', 'EEF1AKMT2 (399818)',
 'EEF1AKMT3 (25895)', 'EEF1AKMT4 (110599564)', 'EEF1B2 (1933)', 'EEF1D (1936)',
 'EEF1E1 (9521)', 'EEF1G (1937)', 'EEF2 (1938)', 'EEF2K (29904)', 'EEF2KMT
 (196483)', 'EEFSEC (60678)', 'EEIG1 (399665)', 'EEIG2 (284611)', 'EEDP1
 (80820)', 'EFCAB10 (100130771)', 'EFCAB11 (90141)', 'EFCAB12 (90288)', 'EFCAB13
 (124989)', 'EFCAB14 (9813)', 'EFCAB2 (84288)', 'EFCAB3 (146779)', 'EFCAB5

(374786)', 'EFCAB6 (64800)', 'EFCAB7 (84455)', 'EFCAB9 (285588)', 'EFCC1
 (79825)', 'EFEMP1 (2202)', 'EFEMP2 (30008)', 'EFHB (151651)', 'EFHC1 (114327)',
 'EFHC2 (80258)', 'EFHD1 (80303)', 'EFHD2 (79180)', 'EFL1 (79631)', 'EFNA1
 (1942)', 'EFNA2 (1943)', 'EFNA3 (1944)', 'EFNA4 (1945)', 'EFNA5 (1946)', 'EFNB1
 (1947)', 'EFNB2 (1948)', 'EFNB3 (1949)', 'EFR3A (23167)', 'EFR3B (22979)', 'EFS
 (10278)', 'EFTUD2 (9343)', 'EGF (1950)', 'EGFL6 (25975)', 'EGFL7 (51162)',
 'EGFL8 (80864)', 'EGFLAM (133584)', 'EGFR (1956)', 'EGLN1 (54583)', 'EGLN2
 (112398)', 'EGLN3 (112399)', 'EGR1 (1958)', 'EGR2 (1959)', 'EGR3 (1960)', 'EGR4
 (1961)', 'EHBP1 (23301)', 'EHBP1L1 (254102)', 'EHD1 (10938)', 'EHD2 (30846)',
 'EHD3 (30845)', 'EHD4 (30844)', 'EHF (26298)', 'EHHADH (1962)', 'EHMT1 (79813)',
 'EHMT2 (10919)', 'EI24 (9538)', 'EID1 (23741)', 'EID2 (163126)', 'EID2B
 (126272)', 'EID3 (493861)', 'EIF1 (10209)', 'EIF1AD (84285)', 'EIF1AX (1964)',
 'EIF1AY (9086)', 'EIF1B (10289)', 'EIF2A (83939)', 'EIF2AK1 (27102)', 'EIF2AK2
 (5610)', 'EIF2AK3 (9451)', 'EIF2AK4 (440275)', 'EIF2B1 (1967)', 'EIF2B2 (8892)',
 'EIF2B3 (8891)', 'EIF2B4 (8890)', 'EIF2B5 (8893)', 'EIF2D (1939)', 'EIF2S1
 (1965)', 'EIF2S2 (8894)', 'EIF2S3 (1968)', 'EIF2S3B (255308)', 'EIF3A (8661)',
 'EIF3B (8662)', 'EIF3D (8664)', 'EIF3E (3646)', 'EIF3F (8665)', 'EIF3G (8666)',
 'EIF3H (8667)', 'EIF3I (8668)', 'EIF3J (8669)', 'EIF3K (27335)', 'EIF3L
 (51386)', 'EIF3M (10480)', 'EIF4A1 (1973)', 'EIF4A2 (1974)', 'EIF4A3 (9775)',
 'EIF4B (1975)', 'EIF4E (1977)', 'EIF4E1B (253314)', 'EIF4E2 (9470)', 'EIF4E3
 (317649)', 'EIF4EBP1 (1978)', 'EIF4EBP2 (1979)', 'EIF4EBP3 (8637)', 'EIF4ENIF1
 (56478)', 'EIF4G1 (1981)', 'EIF4G2 (1982)', 'EIF4G3 (8672)', 'EIF4H (7458)',
 'EIF5 (1983)', 'EIF5A (1984)', 'EIF5A2 (56648)', 'EIF5AL1 (143244)', 'EIF5B
 (9669)', 'EIF6 (3692)', 'EIPR1 (7260)', 'ELAC1 (55520)', 'ELAC2 (60528)', 'ELANE
 (1991)', 'ELAPOR1 (57535)', 'ELAPOR2 (222223)', 'ELAVL1 (1994)', 'ELAVL2
 (1993)', 'ELAVL3 (1995)', 'ELAVL4 (1996)', 'ELF1 (1997)', 'ELF2 (1998)', 'ELF3
 (1999)', 'ELF4 (2000)', 'ELF5 (2001)', 'ELFN1 (392617)', 'ELFN2 (114794)', 'ELK1
 (2002)', 'ELK3 (2004)', 'ELK4 (2005)', 'ELL (8178)', 'ELL2 (22936)', 'ELL3
 (80237)', 'ELM01 (9844)', 'ELM02 (63916)', 'ELM03 (79767)', 'ELMOD1 (55531)',
 'ELMOD2 (255520)', 'ELMOD3 (84173)', 'ELN (2006)', 'ELOA (6924)', 'ELOA2
 (51224)', 'ELOB (6923)', 'ELOC (6921)', 'ELOF1 (84337)', 'ELOVL1 (64834)',
 'ELOVL2 (54898)', 'ELOVL3 (83401)', 'ELOVL4 (6785)', 'ELOVL5 (60481)', 'ELOVL6
 (79071)', 'ELOVL7 (79993)', 'ELP1 (8518)', 'ELP2 (55250)', 'ELP3 (55140)', 'ELP4
 (26610)', 'ELP5 (23587)', 'ELP6 (54859)', 'ELSPBP1 (64100)', 'EMB (133418)',
 'EMC1 (23065)', 'EMC10 (284361)', 'EMC2 (9694)', 'EMC3 (55831)', 'EMC4 (51234)',
 'EMC6 (83460)', 'EMC7 (56851)', 'EMC8 (10328)', 'EMC9 (51016)', 'EMCN (51705)',
 'EMD (2010)', 'EME1 (146956)', 'EME2 (197342)', 'EMG1 (10436)', 'EMID1
 (129080)', 'EMILIN1 (11117)', 'EMILIN2 (84034)', 'EMILIN3 (90187)', 'EML1
 (2009)', 'EML2 (24139)', 'EML3 (256364)', 'EML4 (27436)', 'EML5 (161436)', 'EML6
 (400954)', 'EMP1 (2012)', 'EMP2 (2013)', 'EMP3 (2014)', 'EMSY (56946)', 'EMX1
 (2016)', 'EMX2 (2018)', 'EN1 (2019)', 'EN2 (2020)', 'ENAH (55740)', 'ENAM
 (10117)', 'ENC1 (8507)', 'ENDOD1 (23052)', 'ENDOG (2021)', 'ENDOU (8909)',
 'ENDOV (284131)', 'ENG (2022)', 'ENGASE (64772)', 'ENHO (375704)', 'ENKD1
 (84080)', 'ENKUR (219670)', 'ENO1 (2023)', 'ENO2 (2026)', 'ENO3 (2027)', 'ENO4
 (387712)', 'ENOPH1 (58478)', 'ENOSF1 (55556)', 'ENOX1 (55068)', 'ENOX2 (10495)',
 'ENPEP (2028)', 'ENPP1 (5167)', 'ENPP2 (5168)', 'ENPP3 (5169)', 'ENPP4 (22875)',
 'ENPP5 (59084)', 'ENPP6 (133121)', 'ENPP7 (339221)', 'ENSA (2029)', 'ENTHD1
 (150350)', 'ENTPD1 (953)', 'ENTPD2 (954)', 'ENTPD3 (956)', 'ENTPD4 (9583)',

'ENTPD5 (957)', 'ENTPD6 (955)', 'ENTPD7 (57089)', 'ENTPD8 (377841)', 'ENTR1
 (10807)', 'ENTREP1 (9413)', 'ENTREP2 (23359)', 'ENTREP3 (10712)', 'ENY2
 (56943)', 'EOGT (285203)', 'EOLA1 (91966)', 'EOLA2 (541578)', 'EOMES (8320)',
 'EP300 (2033)', 'EP400 (57634)', 'EPAS1 (2034)', 'EPB41 (2035)', 'EPB41L1
 (2036)', 'EPB41L2 (2037)', 'EPB41L3 (23136)', 'EPB41L4A (64097)', 'EPB41L4B
 (54566)', 'EPB41L5 (57669)', 'EPB42 (2038)', 'EPC1 (80314)', 'EPC2 (26122)',
 'EPCAM (4072)', 'EPCIP (56245)', 'EPDR1 (54749)', 'EPG5 (57724)', 'EPGN
 (255324)', 'EPAH1 (2041)', 'EPAH10 (284656)', 'EPAH2 (1969)', 'EPAH3 (2042)',
 'EPAH4 (2043)', 'EPAH5 (2044)', 'EPAH6 (285220)', 'EPAH7 (2045)', 'EPAH8
 (2046)', 'EPHB1 (2047)', 'EPHB2 (2048)', 'EPHB3 (2049)', 'EPHB4 (2050)', 'EPHB6
 (2051)', 'EPHX1 (2052)', 'EPHX2 (2053)', 'EPHX3 (79852)', 'EPHX4 (253152)',
 'EPM2A (7957)', 'EPM2AIP1 (9852)', 'EPN1 (29924)', 'EPN2 (22905)', 'EPN3
 (55040)', 'EPO (2056)', 'EPOP (100170841)', 'EPOR (2057)', 'EPPIN (57119)',
 'EPRS1 (2058)', 'EPS15 (2060)', 'EPS15L1 (58513)', 'EPS8 (2059)', 'EPS8L1
 (54869)', 'EPS8L2 (64787)', 'EPS8L3 (79574)', 'EPSTI1 (94240)', 'EPX (8288)',
 'EPMC (1833)', 'EQTN (54586)', 'ERAL1 (26284)', 'ERAP1 (51752)', 'ERAP2
 (64167)', 'ERAS (3266)', 'ERBB2 (2064)', 'ERBB3 (2065)', 'ERBB4 (2066)', 'ERBIN
 (55914)', 'ERC1 (23085)', 'ERC2 (26059)', 'ERCC1 (2067)', 'ERCC2 (2068)', 'ERCC3
 (2071)', 'ERCC4 (2072)', 'ERCC5 (2073)', 'ERCC6 (2074)', 'ERCC6L (54821)',
 'ERCC6L2 (375748)', 'ERCC8 (1161)', 'EREG (2069)', 'ERF (2077)', 'ERFE
 (151176)', 'ERFL (390937)', 'ERG (2078)', 'ERG28 (11161)', 'ERGIC1 (57222)',
 'ERGIC2 (51290)', 'ERGIC3 (51614)', 'ERH (2079)', 'ERI1 (90459)', 'ERI2
 (112479)', 'ERI3 (79033)', 'ERICH1 (157697)', 'ERICH2 (285141)', 'ERICH3
 (127254)', 'ERICH4 (100170765)', 'ERICH5 (203111)', 'ERICH6 (131831)', 'ERICH6B
 (220081)', 'ERLEC1 (27248)', 'ERLIN1 (10613)', 'ERLIN2 (11160)', 'ERLN
 (100130933)', 'ERMAP (114625)', 'ERMARD (55780)', 'ERMN (57471)', 'ERMP1
 (79956)', 'ERN1 (2081)', 'ERN2 (10595)', 'ER01A (30001)', 'ER01B (56605)',
 'ERP27 (121506)', 'ERP29 (10961)', 'ERP44 (23071)', 'ERRFI1 (54206)', 'ERV3-1
 (2086)', 'ERVFRD-1 (405754)', 'ERVH48-1 (90625)', 'ERVW-1 (30816)', 'ESAM
 (90952)', 'ESCO1 (114799)', 'ESCO2 (157570)', 'ESD (2098)', 'ESF1 (51575)',
 'ESM1 (11082)', 'ESPL1 (9700)', 'ESPN (83715)', 'ESPNL (339768)', 'ESR1 (2099)',
 'ESR2 (2100)', 'ESRP1 (54845)', 'ESRP2 (80004)', 'ESRRA (2101)', 'ESRRB (2103)',
 'ESRRG (2104)', 'ESS2 (8220)', 'ESX1 (80712)', 'ESYT1 (23344)', 'ESYT2 (57488)',
 'ESYT3 (83850)', 'ETAA1 (54465)', 'ETDC (110806299)', 'ETF1 (2107)', 'ETFA
 (2108)', 'ETFB (2109)', 'ETFBKMT (254013)', 'ETFDH (2110)', 'ETFRF1 (144363)',
 'ETHE1 (23474)', 'ETNK1 (55500)', 'ETNK2 (55224)', 'ETNPPL (64850)', 'ETS1
 (2113)', 'ETS2 (2114)', 'ETV1 (2115)', 'ETV2 (2116)', 'ETV3 (2117)', 'ETV3L
 (440695)', 'ETV4 (2118)', 'ETV5 (2119)', 'ETV6 (2120)', 'ETV7 (51513)', 'EVA1A
 (84141)', 'EVA1B (55194)', 'EVA1C (59271)', 'EVC (2121)', 'EVC2 (132884)',
 'EVI2A (2123)', 'EVI2B (2124)', 'EVI5 (7813)', 'EVI5L (115704)', 'EVL (51466)',
 'EVPL (2125)', 'EVPLL (645027)', 'EVX1 (2128)', 'EVX2 (344191)', 'EWSR1 (2130)',
 'EXD1 (161829)', 'EXD2 (55218)', 'EXD3 (54932)', 'EX01 (9156)', 'EX05 (64789)',
 'EXOC1 (55763)', 'EXOC1L (644145)', 'EXOC2 (55770)', 'EXOC3 (11336)', 'EXOC3L1
 (283849)', 'EXOC3L2 (90332)', 'EXOC3L4 (91828)', 'EXOC4 (60412)', 'EXOC5
 (10640)', 'EXOC6 (54536)', 'EXOC6B (23233)', 'EXOC7 (23265)', 'EXOC8 (149371)',
 'EXOG (9941)', 'EXOSC1 (51013)', 'EXOSC10 (5394)', 'EXOSC2 (23404)', 'EXOSC3
 (51010)', 'EXOSC4 (54512)', 'EXOSC5 (56915)', 'EXOSC6 (118460)', 'EXOSC7
 (23016)', 'EXOSC8 (11340)', 'EXOSC9 (5393)', 'EXPH5 (23086)', 'EXT1 (2131)',

'EXT2 (2132)', 'EXTL1 (2134)', 'EXTL2 (2135)', 'EXTL3 (2137)', 'EYA1 (2138)',
 'EYA2 (2139)', 'EYA3 (2140)', 'EYA4 (2070)', 'EYS (346007)', 'EZH1 (2145)',
 'EZH2 (2146)', 'EZHIP (340602)', 'EZR (7430)', 'F10 (2159)', 'F11 (2160)', 'F11R
 (50848)', 'F12 (2161)', 'F13A1 (2162)', 'F13B (2165)', 'F2 (2147)', 'F2R
 (2149)', 'F2RL1 (2150)', 'F2RL2 (2151)', 'F2RL3 (9002)', 'F3 (2152)', 'F5
 (2153)', 'F7 (2155)', 'F8 (2157)', 'F9 (2158)', 'FA2H (79152)', 'FAAH (2166)',
 'FAAH2 (158584)', 'FAAP100 (80233)', 'FAAP20 (199990)', 'FAAP24 (91442)', 'FABP1
 (2168)', 'FABP12 (646486)', 'FABP2 (2169)', 'FABP3 (2170)', 'FABP4 (2167)',
 'FABP6 (2172)', 'FABP7 (2173)', 'FADD (8772)', 'FADS1 (3992)', 'FADS2 (9415)',
 'FADS3 (3995)', 'FADS6 (283985)', 'FAF1 (11124)', 'FAF2 (23197)', 'FAH (2184)',
 'FAHD1 (81889)', 'FAHD2A (51011)', 'FAHD2B (151313)', 'FAIM (55179)', 'FAIM2
 (23017)', 'FAM107A (11170)', 'FAM107B (83641)', 'FAM110A (83541)', 'FAM110B
 (90362)', 'FAM110C (642273)', 'FAM110D (79927)', 'FAM111A (63901)', 'FAM111B
 (374393)', 'FAM114A1 (92689)', 'FAM114A2 (10827)', 'FAM117A (81558)', 'FAM117B
 (150864)', 'FAM118A (55007)', 'FAM118B (79607)', 'FAM120A (23196)', 'FAM120AOS
 (158293)', 'FAM120B (84498)', 'FAM120C (54954)', 'FAM124A (220108)', 'FAM124B
 (79843)', 'FAM131A (131408)', 'FAM131B (9715)', 'FAM131C (348487)', 'FAM133A
 (286499)', 'FAM133B (257415)', 'FAM135A (57579)', 'FAM135B (51059)', 'FAM136A
 (84908)', 'FAM13A (10144)', 'FAM13B (51306)', 'FAM13C (220965)', 'FAM149A
 (25854)', 'FAM149B1 (317662)', 'FAM151A (338094)', 'FAM151B (167555)', 'FAM153A
 (285596)', 'FAM161A (84140)', 'FAM161B (145483)', 'FAM162A (26355)', 'FAM162B
 (221303)', 'FAM163A (148753)', 'FAM163B (642968)', 'FAM167A (83648)', 'FAM167B
 (84734)', 'FAM168A (23201)', 'FAM168B (130074)', 'FAM169A (26049)', 'FAM170A
 (340069)', 'FAM170B (170370)', 'FAM171A1 (221061)', 'FAM171A2 (284069)',
 'FAM171B (165215)', 'FAM174A (345757)', 'FAM174B (400451)', 'FAM174C (55009)',
 'FAM177A1 (283635)', 'FAM177B (400823)', 'FAM178B (51252)', 'FAM180A (389558)',
 'FAM180B (399888)', 'FAM181A (90050)', 'FAM181B (220382)', 'FAM184A (79632)',
 'FAM184B (27146)', 'FAM185A (222234)', 'FAM186A (121006)', 'FAM186B (84070)',
 'FAM187A (100528020)', 'FAM187B (148109)', 'FAM193A (8603)', 'FAM193B (54540)',
 'FAM199X (139231)', 'FAM200A (221786)', 'FAM200B (285550)', 'FAM200C (63920)',
 'FAM204A (63877)', 'FAM209A (200232)', 'FAM209B (388799)', 'FAM20A (54757)',
 'FAM20B (9917)', 'FAM20C (56975)', 'FAM210A (125228)', 'FAM210B (116151)',
 'FAM216A (29902)', 'FAM216B (144809)', 'FAM217A (222826)', 'FAM217B (63939)',
 'FAM219A (203259)', 'FAM219B (57184)', 'FAM220A (84792)', 'FAM221A (340277)',
 'FAM221B (392307)', 'FAM222A (84915)', 'FAM222B (55731)', 'FAM227A (646851)',
 'FAM227B (196951)', 'FAM228A (653140)', 'FAM228B (375190)', 'FAM229A
 (100128071)', 'FAM229B (619208)', 'FAM234A (83986)', 'FAM234B (57613)', 'FAM237A
 (200726)', 'FAM240A (100132146)', 'FAM241A (132720)', 'FAM241B (219738)',
 'FAM24A (118670)', 'FAM24B (196792)', 'FAM25A (643161)', 'FAM32A (26017)',
 'FAM3A (60343)', 'FAM3B (54097)', 'FAM3C (10447)', 'FAM3D (131177)', 'FAM43A
 (131583)', 'FAM43B (163933)', 'FAM47A (158724)', 'FAM47B (170062)', 'FAM47C
 (442444)', 'FAM47E (100129583)', 'FAM50A (9130)', 'FAM50B (26240)', 'FAM53A
 (152877)', 'FAM53B (9679)', 'FAM53C (51307)', 'FAM76A (199870)', 'FAM76B
 (143684)', 'FAM78A (286336)', 'FAM78B (149297)', 'FAM81A (145773)', 'FAM81B
 (153643)', 'FAM83A (84985)', 'FAM83B (222584)', 'FAM83C (128876)', 'FAM83D
 (81610)', 'FAM83E (54854)', 'FAM83F (113828)', 'FAM83G (644815)', 'FAM83H
 (286077)', 'FAM86B2 (653333)', 'FAM89A (375061)', 'FAM89B (23625)', 'FAM8A1
 (51439)', 'FAM90A1 (55138)', 'FAM91A1 (157769)', 'FAM98A (25940)', 'FAM98B

(283742)', 'FAM98C (147965)', 'FAM9A (171482)', 'FAM9B (171483)', 'FAM9C (171484)', 'FAN1 (22909)', 'FANCA (2175)', 'FANCB (2187)', 'FANCC (2176)', 'FANCD2 (2177)', 'FANCD20S (115795)', 'FANCE (2178)', 'FANCF (2188)', 'FANCG (2189)', 'FANCI (55215)', 'FANCL (55120)', 'FANCM (57697)', 'FANK1 (92565)', 'FAP (2191)', 'FAR1 (84188)', 'FAR2 (55711)', 'FARP1 (10160)', 'FARP2 (9855)', 'FARS2 (10667)', 'FARSA (2193)', 'FARSB (10056)', 'FAS (355)', 'FASLG (356)', 'FASN (2194)', 'FASTK (10922)', 'FASTKD1 (79675)', 'FASTKD2 (22868)', 'FASTKD3 (79072)', 'FASTKD5 (60493)', 'FAT1 (2195)', 'FAT2 (2196)', 'FAT3 (120114)', 'FAT4 (79633)', 'FATE1 (89885)', 'FAU (2197)', 'FAXC (84553)', 'FAXDC2 (10826)', 'FBF1 (85302)', 'FBH1 (84893)', 'FBL (2091)', 'FBLIM1 (54751)', 'FBLL1 (345630)', 'FBLN1 (2192)', 'FBLN2 (2199)', 'FBLN5 (10516)', 'FBLN7 (129804)', 'FBN1 (2200)', 'FBN2 (2201)', 'FBN3 (84467)', 'FBP1 (2203)', 'FBP2 (8789)', 'FBR5 (64319)', 'FBRSL1 (57666)', 'FBXL12 (54850)', 'FBXL13 (222235)', 'FBXL14 (144699)', 'FBXL15 (79176)', 'FBXL16 (146330)', 'FBXL17 (64839)', 'FBXL18 (80028)', 'FBXL19 (54620)', 'FBXL2 (25827)', 'FBXL20 (84961)', 'FBXL22 (283807)', 'FBXL3 (26224)', 'FBXL4 (26235)', 'FBXL5 (26234)', 'FBXL6 (26233)', 'FBXL7 (23194)', 'FBXL8 (55336)', 'FBX010 (26267)', 'FBX011 (80204)', 'FBX015 (201456)', 'FBX016 (157574)', 'FBX017 (115290)', 'FBX02 (26232)', 'FBX021 (23014)', 'FBX022 (26263)', 'FBX024 (26261)', 'FBX025 (26260)', 'FBX027 (126433)', 'FBX028 (23219)', 'FBX03 (26273)', 'FBX030 (84085)', 'FBX031 (79791)', 'FBX032 (114907)', 'FBX033 (254170)', 'FBX034 (55030)', 'FBX036 (130888)', 'FBX038 (81545)', 'FBX039 (162517)', 'FBX04 (26272)', 'FBX040 (51725)', 'FBX041 (150726)', 'FBX042 (54455)', 'FBX043 (286151)', 'FBX044 (93611)', 'FBX045 (200933)', 'FBX046 (23403)', 'FBX047 (494188)', 'FBX048 (554251)', 'FBX05 (26271)', 'FBX06 (26270)', 'FBX07 (25793)', 'FBX08 (26269)', 'FBX09 (26268)', 'FBXW10 (10517)', 'FBXW10B (374286)', 'FBXW11 (23291)', 'FBXW12 (285231)', 'FBXW2 (26190)', 'FBXW4 (6468)', 'FBXW5 (54461)', 'FBXW7 (55294)', 'FBXW8 (26259)', 'FBXW9 (84261)', 'FCAMR (83953)', 'FCAR (2204)', 'FCER1A (2205)', 'FCER1G (2207)', 'FCER2 (2208)', 'FCF1 (51077)', 'FCGR1A (2209)', 'FCGR2A (2212)', 'FCGR2B (2213)', 'FCGR3A (2214)', 'FCGR3B (2215)', 'FCGRT (2217)', 'FCH01 (23149)', 'FCH02 (115548)', 'FCHSD1 (89848)', 'FCHSD2 (9873)', 'FCMR (9214)', 'FCN1 (2219)', 'FCN2 (2220)', 'FCN3 (8547)', 'FCRL1 (115350)', 'FCRL2 (79368)', 'FCRL3 (115352)', 'FCRL4 (83417)', 'FCRL5 (83416)', 'FCRL6 (343413)', 'FCRLA (84824)', 'FCRLB (127943)', 'FCSK (197258)', 'FDCSP (260436)', 'FDFT1 (2222)', 'FDPS (2224)', 'FDX1 (2230)', 'FDX2 (112812)', 'FDXACB1 (91893)', 'FDXR (2232)', 'FECH (2235)', 'FEM1A (55527)', 'FEM1B (10116)', 'FEM1C (56929)', 'FEN1 (2237)', 'FER (2241)', 'FER1L5 (90342)', 'FER1L6 (654463)', 'FERD3L (222894)', 'FERMT1 (55612)', 'FERMT2 (10979)', 'FERMT3 (83706)', 'FERRY3 (57102)', 'FES (2242)', 'FETUB (26998)', 'FEV (54738)', 'FEZ1 (9638)', 'FEZ2 (9637)', 'FEZF1 (389549)', 'FEZF2 (55079)', 'FFAR1 (2864)', 'FFAR2 (2867)', 'FFAR4 (338557)', 'FGA (2243)', 'FGB (2244)', 'FGD1 (2245)', 'FGD2 (221472)', 'FGD3 (89846)', 'FGD4 (121512)', 'FGD5 (152273)', 'FGD6 (55785)', 'FGF1 (2246)', 'FGF10 (2255)', 'FGF11 (2256)', 'FGF12 (2257)', 'FGF13 (2258)', 'FGF14 (2259)', 'FGF16 (8823)', 'FGF17 (8822)', 'FGF18 (8817)', 'FGF19 (9965)', 'FGF2 (2247)', 'FGF20 (26281)', 'FGF21 (26291)', 'FGF22 (27006)', 'FGF23 (8074)', 'FGF3 (2248)', 'FGF4 (2249)', 'FGF5 (2250)', 'FGF6 (2251)', 'FGF7 (2252)', 'FGF8 (2253)', 'FGF9 (2254)', 'FGFBP1 (9982)', 'FGFBP2 (83888)', 'FGFBP3 (143282)', 'FGFR1 (2260)', 'FGFR10P2 (26127)', 'FGFR2 (2263)', 'FGFR3 (2261)', 'FGFR4

(2264)', 'FGFRL1 (53834)', 'FGG (2266)', 'FGGY (55277)', 'FGL1 (2267)', 'FGL2 (10875)', 'FGR (2268)', 'FH (2271)', 'FHAD1 (114827)', 'FHDC1 (85462)', 'FHIP1A (729830)', 'FHIP1B (84067)', 'FHIP2A (57700)', 'FHIP2B (64760)', 'FHIT (2272)', 'FHL1 (2273)', 'FHL2 (2274)', 'FHL3 (2275)', 'FHL5 (9457)', 'FHOD1 (29109)', 'FHOD3 (80206)', 'FIBCD1 (84929)', 'FIBIN (387758)', 'FIBP (9158)', 'FICD (11153)', 'FIG4 (9896)', 'FIGLA (344018)', 'FIGN (55137)', 'FIGNL1 (63979)', 'FIGNL2 (401720)', 'FILIP1 (27145)', 'FILIP1L (11259)', 'FIP1L1 (81608)', 'FIRRM (55732)', 'FIS1 (51024)', 'FITM1 (161247)', 'FITM2 (128486)', 'FIZ1 (84922)', 'FJX1 (24147)', 'FKBP10 (60681)', 'FKBP11 (51303)', 'FKBP14 (55033)', 'FKBP15 (23307)', 'FKBP1A (2280)', 'FKBP1B (2281)', 'FKBP2 (2286)', 'FKBP3 (2287)', 'FKBP4 (2288)', 'FKBP5 (2289)', 'FKBP6 (8468)', 'FKBP7 (51661)', 'FKBP8 (23770)', 'FKBP9 (11328)', 'FKBPL (63943)', 'FKRP (79147)', 'FKTN (2218)', 'FLACC1 (130540)', 'FLAD1 (80308)', 'FLCN (201163)', 'FLG (2312)', 'FLG2 (388698)', 'FLI1 (2313)', 'FLII (2314)', 'FLNA (2316)', 'FLNB (2317)', 'FLNC (2318)', 'FLOT1 (10211)', 'FLOT2 (2319)', 'FLRT1 (23769)', 'FLRT2 (23768)', 'FLRT3 (23767)', 'FLT1 (2321)', 'FLT3 (2322)', 'FLT3LG (2323)', 'FLT4 (2324)', 'FLVCR1 (28982)', 'FLVCR2 (55640)', 'FLYWCH1 (84256)', 'FLYWCH2 (114984)', 'FMC1 (154791)', 'FMN1 (342184)', 'FMN2 (56776)', 'FMNL1 (752)', 'FMNL2 (114793)', 'FMNL3 (91010)', 'FMO1 (2326)', 'FMO2 (2327)', 'FMO3 (2328)', 'FMO4 (2329)', 'FMO5 (2330)', 'FMOD (2331)', 'FMR1 (2332)', 'FMR1NB (158521)', 'FN1 (2335)', 'FN3K (64122)', 'FN3KRP (79672)', 'FNBP1 (23048)', 'FNBP1L (54874)', 'FNBP4 (23360)', 'FNDC1 (84624)', 'FNDC10 (643988)', 'FNDC11 (79025)', 'FNDC3A (22862)', 'FNDC3B (64778)', 'FNDC4 (64838)', 'FNDC5 (252995)', 'FNDC7 (163479)', 'FNDC8 (54752)', 'FNDC9 (408263)', 'FNIP1 (96459)', 'FNIP2 (57600)', 'FNIA (2339)', 'FNIB (2342)', 'FOCAD (54914)', 'FOLH1 (2346)', 'FOLR1 (2348)', 'FOLR2 (2350)', 'FOLR3 (2352)', 'FOS (2353)', 'FOSB (2354)', 'FOSL1 (8061)', 'FOSL2 (2355)', 'FOXA1 (3169)', 'FOXA2 (3170)', 'FOXA3 (3171)', 'FOXB1 (27023)', 'FOXB2 (442425)', 'FOXC1 (2296)', 'FOXC2 (2303)', 'FOXD1 (2297)', 'FOXD2 (2306)', 'FOXD3 (27022)', 'FOXD4 (2298)', 'FOXD4L5 (653427)', 'FOX1 (2304)', 'FOX2 (2301)', 'FOX3 (2294)', 'FOX4 (2295)', 'FOX5 (2290)', 'FOX6 (8928)', 'FOX7 (2299)', 'FOX8 (399823)', 'FOX9 (344167)', 'FOX10 (2302)', 'FOX11 (55810)', 'FOX12 (22887)', 'FOX13 (221937)', 'FOX14 (3607)', 'FOX15 (2300)', 'FOX16 (668)', 'FOX17 (401089)', 'FOX18 (116033993)', 'FOX19 (2305)', 'FOX20 (8456)', 'FOX21 (3344)', 'FOX22 (1112)', 'FOX23 (121643)', 'FOX24 (2308)', 'FOX25 (2309)', 'FOX26 (2310)', 'FOX27 (4303)', 'FOX28 (27086)', 'FOX29 (93986)', 'FOX30 (50943)', 'FOX31 (116113)', 'FOX32 (94234)', 'FOX33 (283150)', 'FOX34 (139628)', 'FOX35 (55572)', 'FOX36 (80020)', 'FOX37 (2307)', 'FOX38 (2356)', 'FOX39 (8790)', 'FOX40 (2357)', 'FOX41 (2358)', 'FOX42 (2359)', 'FOX43 (118924)', 'FOX44 (80144)', 'FOX45 (10023)', 'FOX46 (23401)', 'FOX47 (158326)', 'FOX48 (341640)', 'FOX49 (166752)', 'FOX50 (143678)', 'FOX51 (2483)', 'FOX52 (441581)', 'FOX53 (100288801)', 'FOX54 (2444)', 'FOX55 (79981)', 'FOX56 (257019)', 'FOX57 (55691)', 'FOX58 (23150)', 'FOX59 (84978)', 'FOX60 (122786)', 'FOX61 (90167)', 'FOX62 (83786)', 'FOX63 (22844)', 'FOX64 (143162)', 'FOX65 (84443)', 'FOX66 (9758)', 'FOX67 (391059)', 'FOX68 (23732)', 'FOX69 (10818)', 'FOX70 (10817)', 'FOX71 (10129)', 'FOX72 (285527)', 'FOX73 (2487)', 'FOX74 (128061)', 'FOX75 (100861412)', 'FOX76 (84075)', 'FOX77 (6624)', 'FOX78 (25794)', 'FOX79 (29999)', 'FOX80 (79187)', 'FOX81 (83856)', 'FOX82 (123722)', 'FOX83 (2488)', 'FOX84 (2492)', 'FOX85 (161835)', 'FOX86 (401024)', 'FOX87 (10468)', 'FOX88 (11167)', 'FOX89 (FSTL3

(10272)', 'FSTL4 (23105)', 'FSTL5 (56884)', 'FTCD (10841)', 'FTCDNL1 (348751)',
 'FTH1 (2495)', 'FTHL17 (53940)', 'FTL (2512)', 'FTMT (94033)', 'FTO (79068)',
 'FTSJ1 (24140)', 'FTSJ3 (117246)', 'FUBP1 (8880)', 'FUBP3 (8939)', 'FUCA1
 (2517)', 'FUCA2 (2519)', 'FUNDC1 (139341)', 'FUNDC2 (65991)', 'FUOM (282969)',
 'FURIN (5045)', 'FUS (2521)', 'FUT1 (2523)', 'FUT10 (84750)', 'FUT11 (170384)',
 'FUT2 (2524)', 'FUT3 (2525)', 'FUT4 (2526)', 'FUT5 (2527)', 'FUT6 (2528)', 'FUT7
 (2529)', 'FUT8 (2530)', 'FUT9 (10690)', 'FUZ (80199)', 'FXN (2395)', 'FXR1
 (8087)', 'FXR2 (9513)', 'FXD1 (5348)', 'FXD2 (486)', 'FXD3 (5349)', 'FXD4
 (53828)', 'FXD5 (53827)', 'FXD6 (53826)', 'FXD7 (53822)', 'FYB1 (2533)',
 'FYB2 (199920)', 'FYC01 (79443)', 'FYN (2534)', 'FYTTD1 (84248)', 'FZD1 (8321)',
 'FZD10 (11211)', 'FZD2 (2535)', 'FZD3 (7976)', 'FZD4 (8322)', 'FZD5 (7855)',
 'FZD6 (8323)', 'FZD7 (8324)', 'FZD8 (8325)', 'FZD9 (8326)', 'FZR1 (51343)',
 'GOS2 (50486)', 'G2E3 (55632)', 'G3BP1 (10146)', 'G3BP2 (9908)', 'G6PC1 (2538)',
 'G6PC2 (57818)', 'G6PC3 (92579)', 'G6PD (2539)', 'GAA (2548)', 'GAB1 (2549)',
 'GAB2 (9846)', 'GAB3 (139716)', 'GAB4 (128954)', 'GABARAP (11337)', 'GABARAPL1
 (23710)', 'GABARAPL2 (11345)', 'GABBR1 (2550)', 'GABBR2 (9568)', 'GABPA (2551)',
 'GABPB1 (2553)', 'GABPB2 (126626)', 'GABRA1 (2554)', 'GABRA2 (2555)', 'GABRA3
 (2556)', 'GABRA4 (2557)', 'GABRA5 (2558)', 'GABRA6 (2559)', 'GABRB1 (2560)',
 'GABRB2 (2561)', 'GABRB3 (2562)', 'GABRD (2563)', 'GABRE (2564)', 'GABRG1
 (2565)', 'GABRG2 (2566)', 'GABRG3 (2567)', 'GABRP (2568)', 'GABRQ (55879)',
 'GABRR1 (2569)', 'GABRR2 (2570)', 'GABRR3 (200959)', 'GAD1 (2571)', 'GAD2
 (2572)', 'GADD45A (1647)', 'GADD45B (4616)', 'GADD45G (10912)', 'GADD45GIP1
 (90480)', 'GADL1 (339896)', 'GAGE12H (729442)', 'GAK (2580)', 'GAL (51083)',
 'GAL3ST1 (9514)', 'GAL3ST2 (64090)', 'GAL3ST3 (89792)', 'GAL3ST4 (79690)', 'GALC
 (2581)', 'GALE (2582)', 'GALK1 (2584)', 'GALK2 (2585)', 'GALM (130589)', 'GALNS
 (2588)', 'GALNT1 (2589)', 'GALNT10 (55568)', 'GALNT11 (63917)', 'GALNT12
 (79695)', 'GALNT13 (114805)', 'GALNT14 (79623)', 'GALNT15 (117248)', 'GALNT16
 (57452)', 'GALNT17 (64409)', 'GALNT18 (374378)', 'GALNT2 (2590)', 'GALNT3
 (2591)', 'GALNT4 (8693)', 'GALNT5 (11227)', 'GALNT6 (11226)', 'GALNT7 (51809)',
 'GALNT8 (26290)', 'GALNT9 (50614)', 'GALNTL5 (168391)', 'GALNTL6 (442117)',
 'GALP (85569)', 'GALR1 (2587)', 'GALR2 (8811)', 'GALR3 (8484)', 'GALT (2592)',
 'GAMT (2593)', 'GAN (8139)', 'GANAB (23193)', 'GANC (2595)', 'GAP43 (2596)',
 'GAPDH (2597)', 'GAPDHS (26330)', 'GAPT (202309)', 'GAPVD1 (26130)', 'GAR1
 (54433)', 'GAREM1 (64762)', 'GAREM2 (150946)', 'GARIN1A (346653)', 'GARIN1B
 (84691)', 'GARIN2 (161142)', 'GARIN3 (153745)', 'GARIN4 (149647)', 'GARIN5A
 (112703)', 'GARIN6 (196472)', 'GARNL3 (84253)', 'GARRE1 (9710)', 'GARS1 (2617)',
 'GART (2618)', 'GAS1 (2619)', 'GAS2 (2620)', 'GAS2L1 (10634)', 'GAS2L2
 (246176)', 'GAS2L3 (283431)', 'GAS6 (2621)', 'GAS7 (8522)', 'GAS8 (2622)',
 'GASK1A (729085)', 'GASK1B (51313)', 'GAST (2520)', 'GATA1 (2623)', 'GATA2
 (2624)', 'GATA3 (2625)', 'GATA4 (2626)', 'GATA5 (140628)', 'GATA6 (2627)',
 'GATAD1 (57798)', 'GATAD2A (54815)', 'GATAD2B (57459)', 'GATB (5188)', 'GATC
 (283459)', 'GATD1 (347862)', 'GATD3 (8209)', 'GATM (2628)', 'GBA1 (2629)', 'GBA2
 (57704)', 'GBE1 (2632)', 'GBF1 (8729)', 'GBGT1 (26301)', 'GBP1 (2633)', 'GBP2
 (2634)', 'GBP3 (2635)', 'GBP4 (115361)', 'GBP5 (115362)', 'GBP6 (163351)', 'GBP7
 (388646)', 'GBX1 (2636)', 'GBX2 (2637)', 'GC (2638)', 'GCA (25801)', 'GCAT
 (23464)', 'GCC1 (79571)', 'GCC2 (9648)', 'GCDH (2639)', 'GCFC2 (6936)', 'GCG
 (2641)', 'GCCR (2642)', 'GCH1 (2643)', 'GCHFR (2644)', 'GCK (2645)', 'GCKR
 (2646)', 'GCLC (2729)', 'GCLM (2730)', 'GCM1 (8521)', 'GCM2 (9247)', 'GCN1

(10985)', 'GCNA (93953)', 'GCNT1 (2650)', 'GCNT2 (2651)', 'GCNT3 (9245)', 'GCNT4 (51301)', 'GCSAM (257144)', 'GCSAML (148823)', 'GCSH (2653)', 'GDA (9615)', 'GDAP1 (54332)', 'GDAP1L1 (78997)', 'GDAP2 (54834)', 'GDE1 (51573)', 'GDF1 (2657)', 'GDF10 (2662)', 'GDF11 (10220)', 'GDF15 (9518)', 'GDF2 (2658)', 'GDF3 (9573)', 'GDF5 (8200)', 'GDF6 (392255)', 'GDF7 (151449)', 'GDF9 (2661)', 'GDI1 (2664)', 'GDI2 (2665)', 'GDNF (2668)', 'GDPD1 (284161)', 'GDPD2 (54857)', 'GDPD3 (79153)', 'GDPD4 (220032)', 'GDPD5 (81544)', 'GDPGP1 (390637)', 'GEM (2669)', 'GEMIN2 (8487)', 'GEMIN4 (50628)', 'GEMIN5 (25929)', 'GEMIN6 (79833)', 'GEMIN7 (79760)', 'GEMIN8 (54960)', 'GEN1 (348654)', 'GET1 (7485)', 'GET3 (439)', 'GET4 (51608)', 'GFAP (2670)', 'GFER (2671)', 'GFI1 (2672)', 'GFI1B (8328)', 'GFM1 (85476)', 'GFM2 (84340)', 'GFOD1 (54438)', 'GFOD2 (81577)', 'GFPT1 (2673)', 'GFPT2 (9945)', 'GFRA1 (2674)', 'GFRA2 (2675)', 'GFRA3 (2676)', 'GFRA4 (64096)', 'GFRAL (389400)', 'GFUS (7264)', 'GFY (100507003)', 'GGA1 (26088)', 'GGA2 (23062)', 'GGA3 (23163)', 'GGACT (87769)', 'GGCT (79017)', 'GGCX (2677)', 'GGH (8836)', 'GGN (199720)', 'GGNBP2 (79893)', 'GGPS1 (9453)', 'GGT1 (2678)', 'GGT5 (2687)', 'GGT6 (124975)', 'GGT7 (2686)', 'GGTLC1 (92086)', 'GGTLC2 (91227)', 'GH1 (2688)', 'GH2 (2689)', 'GHDC (84514)', 'GHITM (27069)', 'GHR (2690)', 'GHRH (2691)', 'GHRHR (2692)', 'GHRL (51738)', 'GHSR (2693)', 'GID4 (79018)', 'GID8 (54994)', 'GIGYF1 (64599)', 'GIGYF2 (26058)', 'GIMAP1 (170575)', 'GIMAP2 (26157)', 'GIMAP4 (55303)', 'GIMAP5 (55340)', 'GIMAP6 (474344)', 'GIMAP7 (168537)', 'GIMAP8 (155038)', 'GIMD1 (100507096)', 'GIN1 (54826)', 'GINM1 (116254)', 'GINS1 (9837)', 'GINS2 (51659)', 'GINS3 (64785)', 'GINS4 (84296)', 'GIP (2695)', 'GIPC1 (10755)', 'GIPC2 (54810)', 'GIPC3 (126326)', 'GIPR (2696)', 'GIT1 (28964)', 'GIT2 (9815)', 'GJA1 (2697)', 'GJA10 (84694)', 'GJA3 (2700)', 'GJA4 (2701)', 'GJA5 (2702)', 'GJA8 (2703)', 'GJA9 (81025)', 'GJB1 (2705)', 'GJB2 (2706)', 'GJB3 (2707)', 'GJB4 (127534)', 'GJB5 (2709)', 'GJB6 (10804)', 'GJB7 (375519)', 'GJC1 (10052)', 'GJC2 (57165)', 'GJC3 (349149)', 'GJD2 (57369)', 'GJD3 (125111)', 'GJD4 (219770)', 'GJE1 (100126572)', 'GK (2710)', 'GK2 (2712)', 'GK5 (256356)', 'GKAP1 (80318)', 'GKN1 (56287)', 'GKN2 (200504)', 'GLA (2717)', 'GLB1 (2720)', 'GLB1L (79411)', 'GLB1L2 (89944)', 'GLB1L3 (112937)', 'GLCCI1 (113263)', 'GLCE (26035)', 'GLDC (2731)', 'GLDN (342035)', 'GLE1 (2733)', 'GLG1 (2734)', 'GLI1 (2735)', 'GLI2 (2736)', 'GLI3 (2737)', 'GLI4 (2738)', 'GLIPR1 (11010)', 'GLIPR1L1 (256710)', 'GLIPR1L2 (144321)', 'GLIPR2 (152007)', 'GLIS1 (148979)', 'GLIS2 (84662)', 'GLIS3 (169792)', 'GLMN (11146)', 'GLMP (112770)', 'GLO1 (2739)', 'GLOD4 (51031)', 'GLOD5 (392465)', 'GLP1R (2740)', 'GLP2R (9340)', 'GLRA1 (2741)', 'GLRA2 (2742)', 'GLRA3 (8001)', 'GLRB (2743)', 'GLRX (2745)', 'GLRX2 (51022)', 'GLRX3 (10539)', 'GLRX5 (51218)', 'GLS (2744)', 'GLS2 (27165)', 'GLT1D1 (144423)', 'GLT6D1 (360203)', 'GLT8D1 (55830)', 'GLT8D2 (83468)', 'GLTP (51228)', 'GLTPD2 (388323)', 'GLUD1 (2746)', 'GLUD2 (2747)', 'GLUL (2752)', 'GLYAT (10249)', 'GLYATL1 (92292)', 'GLYATL1B (100287520)', 'GLYATL2 (219970)', 'GLYATL3 (389396)', 'GLYCTK (132158)', 'GLYR1 (84656)', 'GM2A (2760)', 'GMCL1 (64395)', 'GMCL2 (64396)', 'GMDS (2762)', 'GMEB1 (10691)', 'GMEB2 (26205)', 'GMFB (2764)', 'GMFG (9535)', 'GMIP (51291)', 'GML (2765)', 'GMNC (647309)', 'GMNN (51053)', 'GMPPA (29926)', 'GMPPB (29925)', 'GMPR (2766)', 'GMPR2 (51292)', 'GMPS (8833)', 'GNA11 (2767)', 'GNA12 (2768)', 'GNA13 (10672)', 'GNA14 (9630)', 'GNA15 (2769)', 'GNAI1 (2770)', 'GNAI2 (2771)', 'GNAI3 (2773)', 'GNAL (2774)', 'GNAO1 (2775)', 'GNAQ (2776)', 'GNAS (2778)', 'GNAT1 (2779)', 'GNAT2 (2780)', 'GNAT3 (346562)', 'GNAZ (2781)', 'GNB1 (2782)',

'GNB1L (54584)', 'GNB2 (2783)', 'GNB3 (2784)', 'GNB4 (59345)', 'GNB5 (10681)',
 'GNE (10020)', 'GNG10 (2790)', 'GNG11 (2791)', 'GNG12 (55970)', 'GNG13 (51764)',
 'GNG14 (105372280)', 'GNG2 (54331)', 'GNG3 (2785)', 'GNG4 (2786)', 'GNG5
 (2787)', 'GNG7 (2788)', 'GNG8 (94235)', 'GNGT1 (2792)', 'GNGT2 (2793)', 'GNL1
 (2794)', 'GNL2 (29889)', 'GNL3 (26354)', 'GNL3L (54552)', 'GNLY (10578)', 'GNMT
 (27232)', 'GNPAT (8443)', 'GNPDA1 (10007)', 'GNPDA2 (132789)', 'GNPNAT1
 (64841)', 'GNPTAB (79158)', 'GNPTG (84572)', 'GNRH1 (2796)', 'GNRH2 (2797)',
 'GNRHR (2798)', 'GNS (2799)', 'GOLGA1 (2800)', 'GOLGA2 (2801)', 'GOLGA3 (2802)',
 'GOLGA4 (2803)', 'GOLGA5 (9950)', 'GOLGA6B (55889)', 'GOLGA6D (653643)',
 'GOLGA6L2 (283685)', 'GOLGA6L6 (727832)', 'GOLGA6L7 (728310)', 'GOLGA7 (51125)',
 'GOLGA7B (401647)', 'GOLGA8M (653720)', 'GOLGA8N (643699)', 'GOLGA8R
 (101059918)', 'GOLGB1 (2804)', 'GOLIM4 (27333)', 'GOLM1 (51280)', 'GOLM2
 (113201)', 'GOLPH3 (64083)', 'GOLPH3L (55204)', 'GOLT1A (127845)', 'GOLT1B
 (51026)', 'GON4L (54856)', 'GON7 (84520)', 'GOPC (57120)', 'GORAB (92344)',
 'GORASP1 (64689)', 'GORASP2 (26003)', 'GOSR1 (9527)', 'GOSR2 (9570)', 'GOT1
 (2805)', 'GOT1L1 (137362)', 'GOT2 (2806)', 'GP1BA (2811)', 'GP1BB (2812)', 'GP2
 (2813)', 'GP5 (2814)', 'GP6 (51206)', 'GP9 (2815)', 'GPA33 (10223)', 'GPAA1
 (8733)', 'GPALPP1 (55425)', 'GPAM (57678)', 'GPANK1 (7918)', 'GPAT2 (150763)',
 'GPAT3 (84803)', 'GPAT4 (137964)', 'GPATCH1 (55094)', 'GPATCH11 (253635)',
 'GPATCH2 (55105)', 'GPATCH2L (55668)', 'GPATCH3 (63906)', 'GPATCH8 (23131)',
 'GPBAR1 (151306)', 'GPBP1 (65056)', 'GPBP1L1 (60313)', 'GPC1 (2817)', 'GPC2
 (221914)', 'GPC3 (2719)', 'GPC4 (2239)', 'GPC5 (2262)', 'GPC6 (10082)', 'GPCPD1
 (56261)', 'GPD1 (2819)', 'GPD1L (23171)', 'GPD2 (2820)', 'GPER1 (2852)', 'GPHA2
 (170589)', 'GPHB5 (122876)', 'GPHN (10243)', 'GPI (2821)', 'GPIHBP1 (338328)',
 'GPKOW (27238)', 'GPLD1 (2822)', 'GPM6A (2823)', 'GPM6B (2824)', 'GPN1 (11321)',
 'GPN2 (54707)', 'GPN3 (51184)', 'GPNMB (10457)', 'GPR101 (83550)', 'GPR107
 (57720)', 'GPR108 (56927)', 'GPR119 (139760)', 'GPR12 (2835)', 'GPR132 (29933)',
 'GPR135 (64582)', 'GPR137 (56834)', 'GPR137B (7107)', 'GPR137C (283554)',
 'GPR139 (124274)', 'GPR141 (353345)', 'GPR142 (350383)', 'GPR143 (4935)',
 'GPR146 (115330)', 'GPR148 (344561)', 'GPR149 (344758)', 'GPR15 (2838)', 'GPR150
 (285601)', 'GPR151 (134391)', 'GPR152 (390212)', 'GPR153 (387509)', 'GPR155
 (151556)', 'GPR156 (165829)', 'GPR157 (80045)', 'GPR158 (57512)', 'GPR15LG
 (387695)', 'GPR160 (26996)', 'GPR161 (23432)', 'GPR162 (27239)', 'GPR17 (2840)',
 'GPR171 (29909)', 'GPR173 (54328)', 'GPR174 (84636)', 'GPR176 (11245)', 'GPR179
 (440435)', 'GPR18 (2841)', 'GPR180 (160897)', 'GPR182 (11318)', 'GPR183 (1880)',
 'GPR19 (2842)', 'GPR20 (2843)', 'GPR21 (2844)', 'GPR22 (2845)', 'GPR25 (2848)',
 'GPR26 (2849)', 'GPR27 (2850)', 'GPR3 (2827)', 'GPR31 (2853)', 'GPR32 (2854)',
 'GPR33 (2856)', 'GPR34 (2857)', 'GPR35 (2859)', 'GPR37 (2861)', 'GPR37L1
 (9283)', 'GPR39 (2863)', 'GPR4 (2828)', 'GPR45 (11250)', 'GPR50 (9248)', 'GPR52
 (9293)', 'GPR55 (9290)', 'GPR6 (2830)', 'GPR61 (83873)', 'GPR62 (118442)',
 'GPR63 (81491)', 'GPR65 (8477)', 'GPR68 (8111)', 'GPR75 (10936)', 'GPR78
 (27201)', 'GPR82 (27197)', 'GPR83 (10888)', 'GPR84 (53831)', 'GPR85 (54329)',
 'GPR87 (53836)', 'GPR88 (54112)', 'GPRASP1 (9737)', 'GPRC5A (9052)', 'GPRC5B
 (51704)', 'GPRC5C (55890)', 'GPRC5D (55507)', 'GPRC6A (222545)', 'GPRIN1
 (114787)', 'GPRIN2 (9721)', 'GPRIN3 (285513)', 'GPS1 (2873)', 'GPS2 (2874)',
 'GPSM1 (26086)', 'GPSM2 (29899)', 'GPSM3 (63940)', 'GPT (2875)', 'GPT2 (84706)',
 'GPX1 (2876)', 'GPX2 (2877)', 'GPX3 (2878)', 'GPX4 (2879)', 'GPX5 (2880)', 'GPX6
 (257202)', 'GPX7 (2882)', 'GPX8 (493869)', 'GRAMD1A (57655)', 'GRAMD1B (57476)',

'GRAMD1C (54762)', 'GRAMD2A (196996)', 'GRAMD2B (65983)', 'GRAMD4 (23151)',
 'GRAP (10750)', 'GRAP2 (9402)', 'GRB10 (2887)', 'GRB14 (2888)', 'GRB2 (2885)',
 'GRB7 (2886)', 'GREB1 (9687)', 'GREB1L (80000)', 'GREM1 (26585)', 'GREM2
 (64388)', 'GRHL1 (29841)', 'GRHL2 (79977)', 'GRHL3 (57822)', 'GRHPR (9380)',
 'GRIA1 (2890)', 'GRIA2 (2891)', 'GRIA3 (2892)', 'GRIA4 (2893)', 'GRID1 (2894)',
 'GRID2 (2895)', 'GRID2IP (392862)', 'GRIFIN (402635)', 'GRIK1 (2897)', 'GRIK2
 (2898)', 'GRIK3 (2899)', 'GRIK4 (2900)', 'GRIK5 (2901)', 'GRIN1 (2902)', 'GRIN2A
 (2903)', 'GRIN2B (2904)', 'GRIN2C (2905)', 'GRIN2D (2906)', 'GRIN3A (116443)',
 'GRIN3B (116444)', 'GRINA (2907)', 'GRIP1 (23426)', 'GRIP2 (80852)', 'GRIPAP1
 (56850)', 'GRK1 (6011)', 'GRK2 (156)', 'GRK3 (157)', 'GRK4 (2868)', 'GRK5
 (2869)', 'GRK6 (2870)', 'GRK7 (131890)', 'GRM1 (2911)', 'GRM2 (2912)', 'GRM3
 (2913)', 'GRM4 (2914)', 'GRM5 (2915)', 'GRM6 (2916)', 'GRM7 (2917)', 'GRM8
 (2918)', 'GRN (2896)', 'GRP (2922)', 'GRPEL1 (80273)', 'GRPEL2 (134266)', 'GRPR
 (2925)', 'GRSF1 (2926)', 'GRTP1 (79774)', 'GRWD1 (83743)', 'GRXCR1 (389207)',
 'GRXCR2 (643226)', 'GSAP (54103)', 'GSC (145258)', 'GSC2 (2928)', 'GSDMA
 (284110)', 'GSDMB (55876)', 'GSDMC (56169)', 'GSDMD (79792)', 'GSDME (1687)',
 'GSE1 (23199)', 'GSG1 (83445)', 'GSG1L (146395)', 'GSG1L2 (644070)', 'GSK3A
 (2931)', 'GSK3B (2932)', 'GSKIP (51527)', 'GSN (2934)', 'GSPT1 (2935)', 'GSPT2
 (23708)', 'GSR (2936)', 'GSS (2937)', 'GSTA1 (2938)', 'GSTA2 (2939)', 'GSTA3
 (2940)', 'GSTA4 (2941)', 'GSTA5 (221357)', 'GSTCD (79807)', 'GSTK1 (373156)',
 'GSTM1 (2944)', 'GSTM2 (2946)', 'GSTM3 (2947)', 'GSTM4 (2948)', 'GSTM5 (2949)',
 'GSTO1 (9446)', 'GSTO2 (119391)', 'GSTP1 (2950)', 'GSTT2B (653689)', 'GSTT4
 (25774)', 'GSTZ1 (2954)', 'GSX1 (219409)', 'GSX2 (170825)', 'GTDC1 (79712)',
 'GTF2A1 (2957)', 'GTF2A1L (11036)', 'GTF2A2 (2958)', 'GTF2B (2959)', 'GTF2E1
 (2960)', 'GTF2E2 (2961)', 'GTF2F1 (2962)', 'GTF2F2 (2963)', 'GTF2H1 (2965)',
 'GTF2H3 (2967)', 'GTF2H4 (2968)', 'GTF2H5 (404672)', 'GTF2I (2969)', 'GTF2IRD1
 (9569)', 'GTF3A (2971)', 'GTF3C1 (2975)', 'GTF3C2 (2976)', 'GTF3C3 (9330)',
 'GTF3C4 (9329)', 'GTF3C5 (9328)', 'GTF3C6 (112495)', 'GTPBP1 (9567)', 'GTPBP10
 (85865)', 'GTPBP2 (54676)', 'GTPBP3 (84705)', 'GTPBP4 (23560)', 'GTPBP6 (8225)',
 'GTPBP8 (29083)', 'GTSE1 (51512)', 'GTSF1 (121355)', 'GTSF1L (149699)',
 'GUCA1ANB-GUCA1A (118142757)', 'GUCA1B (2979)', 'GUCA1C (9626)', 'GUCA2A
 (2980)', 'GUCA2B (2981)', 'GUCD1 (83606)', 'GUCY1A1 (2982)', 'GUCY1A2 (2977)',
 'GUCY1B1 (2983)', 'GUCY2C (2984)', 'GUCY2D (3000)', 'GUCY2F (2986)', 'GUF1
 (60558)', 'GUK1 (2987)', 'GULP1 (51454)', 'GUSB (2990)', 'GVQW3 (100506127)',
 'GXYLT1 (283464)', 'GXYLT2 (727936)', 'GYG1 (2992)', 'GYG2 (8908)', 'GYPA
 (2993)', 'GYPB (2994)', 'GYPC (2995)', 'GYPE (2996)', 'GYS1 (2997)', 'GYS2
 (2998)', 'GZF1 (64412)', 'GZMA (3001)', 'GZMB (3002)', 'GZMH (2999)', 'GZMK
 (3003)', 'GZMM (3004)', 'H1-0 (3005)', 'H1-1 (3024)', 'H1-10 (8971)', 'H1-2
 (3006)', 'H1-3 (3007)', 'H1-4 (3008)', 'H1-5 (3009)', 'H1-6 (3010)', 'H1-7
 (341567)', 'H1-8 (132243)', 'H2AC1 (221613)', 'H2AC11 (8969)', 'H2AC12 (85235)',
 'H2AC15 (8330)', 'H2AC16 (8332)', 'H2AC17 (8336)', 'H2AC20 (8338)', 'H2AC21
 (317772)', 'H2AC25 (92815)', 'H2AC4 (8335)', 'H2AC6 (8334)', 'H2AC7 (3013)',
 'H2AC8 (3012)', 'H2AJ (55766)', 'H2AP (25763)', 'H2AX (3014)', 'H2AZ1 (3015)',
 'H2AZ2 (94239)', 'H2BC1 (255626)', 'H2BC10 (8346)', 'H2BC11 (8970)', 'H2BC12
 (85236)', 'H2BC13 (8340)', 'H2BC14 (8342)', 'H2BC15 (8341)', 'H2BC17 (8348)',
 'H2BC18 (440689)', 'H2BC21 (8349)', 'H2BC26 (128312)', 'H2BC3 (3018)', 'H2BC4
 (8347)', 'H2BC5 (3017)', 'H2BC6 (8344)', 'H2BC7 (8343)', 'H2BC8 (8339)', 'H2BC9
 (8345)', 'H2BK1 (114483833)', 'H2BW1 (158983)', 'H2BW2 (286436)', 'H3-3A

(3020)', 'H3-3B (3021)', 'H3-4 (8290)', 'H3-5 (440093)', 'H3C1 (8350)', 'H3C10
 (8357)', 'H3C11 (8354)', 'H3C12 (8356)', 'H3C2 (8358)', 'H3C3 (8352)', 'H3C4
 (8351)', 'H3C6 (8353)', 'H3C7 (8968)', 'H3C8 (8355)', 'H4C1 (8359)', 'H4C13
 (8368)', 'H4C16 (121504)', 'H4C2 (8366)', 'H4C3 (8364)', 'H4C4 (8360)', 'H4C5
 (8367)', 'H4C6 (8361)', 'H4C7 (8369)', 'H4C8 (8365)', 'H4C9 (8294)', 'H6PD
 (9563)', 'HAA0 (23498)', 'HABP2 (3026)', 'HABP4 (22927)', 'HACD1 (9200)', 'HACD2
 (201562)', 'HACD3 (51495)', 'HACD4 (401494)', 'HACE1 (57531)', 'HACL1 (26061)',
 'HADH (3033)', 'HADHA (3030)', 'HADHB (3032)', 'HAGH (3029)', 'HAGHL (84264)',
 'HAL (3034)', 'HAMP (57817)', 'HAND1 (9421)', 'HAND2 (9464)', 'HA01 (54363)',
 'HA02 (51179)', 'HAP1 (9001)', 'HAPLN1 (1404)', 'HAPLN2 (60484)', 'HAPLN3
 (145864)', 'HAPLN4 (404037)', 'HAPSTR1 (29035)', 'HAPSTR2 (389895)', 'HARBI1
 (283254)', 'HARS1 (3035)', 'HARS2 (23438)', 'HAS1 (3036)', 'HAS2 (3037)', 'HAS3
 (3038)', 'HASPIN (83903)', 'HAT1 (8520)', 'HAUS1 (115106)', 'HAUS2 (55142)',
 'HAUS3 (79441)', 'HAUS4 (54930)', 'HAUS5 (23354)', 'HAUS6 (54801)', 'HAUS7
 (55559)', 'HAUS8 (93323)', 'HAVCR1 (26762)', 'HAVCR2 (84868)', 'HAX1 (10456)',
 'HBB (3043)', 'HBD (3045)', 'HBE1 (3046)', 'HBEGF (1839)', 'HBM (3042)', 'HBP1
 (26959)', 'HBQ1 (3049)', 'HBS1L (10767)', 'HBZ (3050)', 'HCAR1 (27198)', 'HCAR2
 (338442)', 'HCAR3 (8843)', 'HCCS (3052)', 'HCFC1 (3054)', 'HCFC1R1 (54985)',
 'HCFC2 (29915)', 'HCK (3055)', 'HCLS1 (3059)', 'HCN1 (348980)', 'HCN2 (610)',
 'HCN3 (57657)', 'HCN4 (10021)', 'HCRT (3060)', 'HCRTR1 (3061)', 'HCRTR2 (3062)',
 'HCST (10870)', 'HDAC1 (3065)', 'HDAC10 (83933)', 'HDAC11 (79885)', 'HDAC2
 (3066)', 'HDAC3 (8841)', 'HDAC4 (9759)', 'HDAC5 (10014)', 'HDAC6 (10013)',
 'HDAC7 (51564)', 'HDAC8 (55869)', 'HDAC9 (9734)', 'HDC (3067)', 'HDDC2 (51020)',
 'HDDC3 (374659)', 'HDGF (3068)', 'HDGFL1 (154150)', 'HDGFL2 (84717)', 'HDGFL3
 (50810)', 'HDHD2 (84064)', 'HDHD3 (81932)', 'HDHD5 (27440)', 'HDLBP (3069)',
 'HDX (139324)', 'HEATR1 (55127)', 'HEATR3 (55027)', 'HEATR4 (399671)', 'HEATR5A
 (25938)', 'HEATR5B (54497)', 'HEATR6 (63897)', 'HEATR9 (256957)', 'HEBP1
 (50865)', 'HEBP2 (23593)', 'HECA (51696)', 'HECTD1 (25831)', 'HECTD2 (143279)',
 'HECTD3 (79654)', 'HECTD4 (283450)', 'HECW1 (23072)', 'HECW2 (57520)', 'HEG1
 (57493)', 'HELB (92797)', 'HELLS (3070)', 'HELQ (113510)', 'HELT (391723)',
 'HELZ (9931)', 'HELZ2 (85441)', 'HEMGN (55363)', 'HEMK1 (51409)', 'HEMK2
 (29104)', 'HENMT1 (113802)', 'HEPACAM (220296)', 'HEPACAM2 (253012)', 'HEPH
 (9843)', 'HEPHL1 (341208)', 'HERC1 (8925)', 'HERC2 (8924)', 'HERC3 (8916)',
 'HERC4 (26091)', 'HERC5 (51191)', 'HERC6 (55008)', 'HERPUD1 (9709)', 'HERPUD2
 (64224)', 'HES1 (3280)', 'HES2 (54626)', 'HES3 (390992)', 'HES4 (57801)', 'HES5
 (388585)', 'HES6 (55502)', 'HES7 (84667)', 'HESX1 (8820)', 'HEXA (3073)', 'HEXB
 (3074)', 'HEXD (284004)', 'HEXIM1 (10614)', 'HEXIM2 (124790)', 'HEY1 (23462)',
 'HEY2 (23493)', 'HEYL (26508)', 'HFE (3077)', 'HFM1 (164045)', 'HGD (3081)',
 'HGF (3082)', 'HGFAC (3083)', 'HGH1 (51236)', 'HGS (9146)', 'HGSNAT (138050)',
 'HHAT (55733)', 'HHATL (57467)', 'HHEX (3087)', 'HHIP (64399)', 'HHIPL1
 (84439)', 'HHIPL2 (79802)', 'HHLA2 (11148)', 'HIBADH (11112)', 'HIBCH (26275)',
 'HIC1 (3090)', 'HIC2 (23119)', 'HID1 (283987)', 'HIF1A (3091)', 'HIF1AN
 (55662)', 'HIF3A (64344)', 'HIGD1A (25994)', 'HIGD1B (51751)', 'HIGD1C
 (613227)', 'HIGD2A (192286)', 'HIGD2B (123346)', 'HIKESHI (51501)', 'HILPDA
 (29923)', 'HINFP (25988)', 'HINT1 (3094)', 'HINT2 (84681)', 'HINT3 (135114)',
 'HIP1 (3092)', 'HIP1R (9026)', 'HIPK1 (204851)', 'HIPK2 (28996)', 'HIPK3
 (10114)', 'HIPK4 (147746)', 'HIRA (7290)', 'HIRIP3 (8479)', 'HIVEP1 (3096)',
 'HIVEP2 (3097)', 'HIVEP3 (59269)', 'HJURP (55355)', 'HJV (148738)', 'HK1

(3098)', 'HK2 (3099)', 'HK3 (3101)', 'HKDC1 (80201)', 'HLA-A (3105)', 'HLA-B (3106)', 'HLA-C (3107)', 'HLA-DMA (3108)', 'HLA-DMB (3109)', 'HLA-DOA (3111)', 'HLA-DOB (3112)', 'HLA-DPA1 (3113)', 'HLA-DPB1 (3115)', 'HLA-DQA1 (3117)', 'HLA-DQA2 (3118)', 'HLA-DQB1 (3119)', 'HLA-DQB2 (3120)', 'HLA-DRA (3122)', 'HLA-DRB1 (3123)', 'HLA-DRB5 (3127)', 'HLA-E (3133)', 'HLA-F (3134)', 'HLA-G (3135)', 'HLCS (3141)', 'HLF (3131)', 'HLTF (6596)', 'HLX (3142)', 'HM13 (81502)', 'HMBOX1 (79618)', 'HMBS (3145)', 'HMCES (56941)', 'HMCN1 (83872)', 'HMCN2 (256158)', 'HMG20A (10363)', 'HMG20B (10362)', 'HMGA1 (3159)', 'HMGA2 (8091)', 'HMGB1 (3146)', 'HMGB2 (3148)', 'HMGB3 (3149)', 'HMGB4 (127540)', 'HMGCL (3155)', 'HMGCLL1 (54511)', 'HMGCR (3156)', 'HMGCS1 (3157)', 'HMGCS2 (3158)', 'HMGN1 (3150)', 'HMGN2 (3151)', 'HMGN3 (9324)', 'HMGN4 (10473)', 'HMGN5 (79366)', 'HMGXB3 (22993)', 'HMGXB4 (10042)', 'HMH1 (57824)', 'HMMR (3161)', 'HMOX1 (3162)', 'HMOX2 (3163)', 'HMSD (284293)', 'HMX1 (3166)', 'HMX2 (3167)', 'HMX3 (340784)', 'HNF1A (6927)', 'HNF1B (6928)', 'HNF4A (3172)', 'HNF4G (3174)', 'HNMT (3176)', 'HNRNPA0 (10949)', 'HNRNPA1 (3178)', 'HNRNPA1L2 (144983)', 'HNRNPA2B1 (3181)', 'HNRNPA3 (220988)', 'HNRNPAB (3182)', 'HNRNPC (3183)', 'HNRNPD (3184)', 'HNRNPDL (9987)', 'HNRNPF (3185)', 'HNRNPH1 (3187)', 'HNRNPH2 (3188)', 'HNRNPH3 (3189)', 'HNRNPK (3190)', 'HNRNPL (3191)', 'HNRNPLL (92906)', 'HNRNPM (4670)', 'HNRNPR (10236)', 'HNRNPU (3192)', 'HNRNPUL1 (11100)', 'HNRNPUL2 (221092)', 'HOATZ (399949)', 'HOGA1 (112817)', 'HOMER1 (9456)', 'HOMER2 (9455)', 'HOMER3 (9454)', 'HOMEZ (57594)', 'HOOK1 (51361)', 'HOOK2 (29911)', 'HOOK3 (84376)', 'HOPX (84525)', 'HORMAD1 (84072)', 'HORMAD2 (150280)', 'HOXA1 (3198)', 'HOXA10 (3206)', 'HOXA11 (3207)', 'HOXA13 (3209)', 'HOXA2 (3199)', 'HOXA3 (3200)', 'HOXA4 (3201)', 'HOXA5 (3202)', 'HOXA6 (3203)', 'HOXA7 (3204)', 'HOXA9 (3205)', 'HOXB1 (3211)', 'HOXB13 (10481)', 'HOXB2 (3212)', 'HOXB3 (3213)', 'HOXB4 (3214)', 'HOXB5 (3215)', 'HOXB6 (3216)', 'HOXB7 (3217)', 'HOXB8 (3218)', 'HOXB9 (3219)', 'HOXC10 (3226)', 'HOXC11 (3227)', 'HOXC12 (3228)', 'HOXC13 (3229)', 'HOXC4 (3221)', 'HOXC5 (3222)', 'HOXC6 (3223)', 'HOXC8 (3224)', 'HOXC9 (3225)', 'HOXD1 (3231)', 'HOXD10 (3236)', 'HOXD11 (3237)', 'HOXD12 (3238)', 'HOXD13 (3239)', 'HOXD3 (3232)', 'HOXD4 (3233)', 'HOXD8 (3234)', 'HOXD9 (3235)', 'HP (3240)', 'HP1BP3 (50809)', 'HPCA (3208)', 'HPCAL1 (3241)', 'HPCAL4 (51440)', 'HPD (3242)', 'HPDL (84842)', 'HPF1 (54969)', 'HPGD (3248)', 'HPGDS (27306)', 'HPN (3249)', 'HPR (3250)', 'HPRT1 (3251)', 'HPS1 (3257)', 'HPS3 (84343)', 'HPS4 (89781)', 'HPS5 (11234)', 'HPS6 (79803)', 'HPSE (10855)', 'HPSE2 (60495)', 'HPX (3263)', 'HR (55806)', 'HRAS (3265)', 'HRC (3270)', 'HRCT1 (646962)', 'HRG (3273)', 'HRH1 (3269)', 'HRH2 (3274)', 'HRH3 (11255)', 'HRH4 (59340)', 'HRK (8739)', 'HRNR (388697)', 'HROB (78995)', 'HS1BP3 (64342)', 'HS2ST1 (9653)', 'HS3ST1 (9957)', 'HS3ST2 (9956)', 'HS3ST3A1 (9955)', 'HS3ST3B1 (9953)', 'HS3ST4 (9951)', 'HS3ST5 (222537)', 'HS3ST6 (64711)', 'HS6ST1 (9394)', 'HS6ST2 (90161)', 'HS6ST3 (266722)', 'HSBP1 (3281)', 'HSBP1L1 (440498)', 'HSCB (150274)', 'HSD11B1 (3290)', 'HSD11B1L (374875)', 'HSD11B2 (3291)', 'HSD17B1 (3292)', 'HSD17B10 (3028)', 'HSD17B11 (51170)', 'HSD17B12 (51144)', 'HSD17B13 (345275)', 'HSD17B14 (51171)', 'HSD17B2 (3294)', 'HSD17B3 (3293)', 'HSD17B4 (3295)', 'HSD17B6 (8630)', 'HSD17B7 (51478)', 'HSD17B8 (7923)', 'HSD3B1 (3283)', 'HSD3B2 (3284)', 'HSD3B7 (80270)', 'HSDL1 (83693)', 'HSDL2 (84263)', 'HSF1 (3297)', 'HSF2 (3298)', 'HSF2BP (11077)', 'HSF4 (3299)', 'HSF5 (124535)', 'HSH2D (84941)', 'HSP90AA1 (3320)', 'HSP90AB1 (3326)', 'HSP90B1 (7184)', 'HSPA12A (259217)', 'HSPA12B (116835)',

'HSPA13 (6782)', 'HSPA14 (51182)', 'HSPA1L (3305)', 'HSPA2 (3306)', 'HSPA4 (3308)', 'HSPA4L (22824)', 'HSPA5 (3309)', 'HSPA6 (3310)', 'HSPA8 (3312)', 'HSPA9 (3313)', 'HSPB1 (3315)', 'HSPB2 (3316)', 'HSPB3 (8988)', 'HSPB6 (126393)', 'HSPB7 (27129)', 'HSPB8 (26353)', 'HSPB9 (94086)', 'HSPBAP1 (79663)', 'HSPBP1 (23640)', 'HSPD1 (3329)', 'HSPE1 (3336)', 'HSPG2 (3339)', 'HSPH1 (10808)', 'HTATIP2 (10553)', 'HTATSF1 (27336)', 'HTD2 (109703458)', 'HTN1 (3346)', 'HTN3 (3347)', 'HTR1A (3350)', 'HTR1B (3351)', 'HTR1D (3352)', 'HTR1E (3354)', 'HTR1F (3355)', 'HTR2A (3356)', 'HTR2B (3357)', 'HTR2C (3358)', 'HTR3A (3359)', 'HTR3B (9177)', 'HTR3C (170572)', 'HTR3D (200909)', 'HTR3E (285242)', 'HTR4 (3360)', 'HTR5A (3361)', 'HTR6 (3362)', 'HTR7 (3363)', 'HTRA1 (5654)', 'HTRA2 (27429)', 'HTRA3 (94031)', 'HTRA4 (203100)', 'HTT (3064)', 'HUNK (30811)', 'HUS1 (3364)', 'HUS1B (135458)', 'HUWE1 (10075)', 'HVCN1 (84329)', 'HYAL1 (3373)', 'HYAL2 (8692)', 'HYAL3 (8372)', 'HYAL4 (23553)', 'HYCC1 (84668)', 'HYCC2 (285172)', 'HYDIN (54768)', 'HYI (81888)', 'HYKK (123688)', 'HYLS1 (219844)', 'HYOU1 (10525)', 'HYPK (25764)', 'IAH1 (285148)', 'IAPP (3375)', 'IARS1 (3376)', 'IARS2 (55699)', 'IBA57 (200205)', 'IBSP (3381)', 'IBTK (25998)', 'ICA1 (3382)', 'ICA1L (130026)', 'ICAM1 (3383)', 'ICAM2 (3384)', 'ICAM3 (3385)', 'ICAM4 (3386)', 'ICAM5 (7087)', 'ICE1 (23379)', 'ICE2 (79664)', 'ICMT (23463)', 'ICOS (29851)', 'ICOSLG (23308)', 'ID1 (3397)', 'ID2 (3398)', 'ID3 (3399)', 'ID4 (3400)', 'IDE (3416)', 'IDH1 (3417)', 'IDH2 (3418)', 'IDH3A (3419)', 'IDH3B (3420)', 'IDH3G (3421)', 'IDI1 (3422)', 'IDI2 (91734)', 'IDNK (414328)', 'IDO1 (3620)', 'IDS (3423)', 'IDUA (3425)', 'IER2 (9592)', 'IER3 (8870)', 'IER3IP1 (51124)', 'IER5 (51278)', 'IER5L (389792)', 'IFFO1 (25900)', 'IFFO2 (126917)', 'IFI16 (3428)', 'IFI27 (3429)', 'IFI27L1 (122509)', 'IFI27L2 (83982)', 'IFI30 (10437)', 'IFI35 (3430)', 'IFI44 (10561)', 'IFI44L (10964)', 'IFI6 (2537)', 'IFIH1 (64135)', 'IFIT1 (3434)', 'IFIT1B (439996)', 'IFIT2 (3433)', 'IFIT3 (3437)', 'IFIT5 (24138)', 'IFITM1 (8519)', 'IFITM10 (402778)', 'IFITM2 (10581)', 'IFITM3 (10410)', 'IFITM5 (387733)', 'IFNA1 (3439)', 'IFNA10 (3446)', 'IFNA14 (3448)', 'IFNA16 (3449)', 'IFNA17 (3451)', 'IFNA2 (3440)', 'IFNA21 (3452)', 'IFNA4 (3441)', 'IFNA5 (3442)', 'IFNA6 (3443)', 'IFNA7 (3444)', 'IFNA8 (3445)', 'IFNAR1 (3454)', 'IFNAR2 (3455)', 'IFNB1 (3456)', 'IFNE (338376)', 'IFNG (3458)', 'IFNGR1 (3459)', 'IFNGR2 (3460)', 'IFNK (56832)', 'IFNL1 (282618)', 'IFNL2 (282616)', 'IFNL3 (282617)', 'IFNLR1 (163702)', 'IFNW1 (3467)', 'IFRD1 (3475)', 'IFRD2 (7866)', 'IFT122 (55764)', 'IFT140 (9742)', 'IFT172 (26160)', 'IFT20 (90410)', 'IFT22 (64792)', 'IFT25 (51668)', 'IFT27 (11020)', 'IFT43 (112752)', 'IFT46 (56912)', 'IFT52 (51098)', 'IFT56 (79989)', 'IFT57 (55081)', 'IFT70A (92104)', 'IFT70B (150737)', 'IFT74 (80173)', 'IFT80 (57560)', 'IFT81 (28981)', 'IFT88 (8100)', 'IFTAP (119710)', 'IGBP1 (3476)', 'IGDCC3 (9543)', 'IGDCC4 (57722)', 'IGF1 (3479)', 'IGF1R (3480)', 'IGF2 (3481)', 'IGF2BP1 (10642)', 'IGF2BP2 (10644)', 'IGF2BP3 (10643)', 'IGF2R (3482)', 'IGFALS (3483)', 'IGFBP1 (3484)', 'IGFBP2 (3485)', 'IGFBP3 (3486)', 'IGFBP4 (3487)', 'IGFBP5 (3488)', 'IGFBP6 (3489)', 'IGFBP7 (3490)', 'IGFBPL1 (347252)', 'IGFL1 (374918)', 'IGFL2 (147920)', 'IGFL3 (388555)', 'IGFL4 (444882)', 'IGFLR1 (79713)', 'IGFN1 (91156)', 'IGHMBP2 (3508)', 'IGIP (492311)', 'IGLL1 (3543)', 'IGLL5 (100423062)', 'IGLON5 (402665)', 'IGSF1 (3547)', 'IGSF10 (285313)', 'IGSF11 (152404)', 'IGSF21 (84966)', 'IGSF22 (283284)', 'IGSF23 (147710)', 'IGSF3 (3321)', 'IGSF5 (150084)', 'IGSF6 (10261)', 'IGSF8 (93185)', 'IGSF9 (57549)', 'IGSF9B (22997)', 'IHH (3549)', 'IH01 (339834)', 'IK (3550)', 'IKBIP

(121457)', 'IKBKB (3551)', 'IKBKE (9641)', 'IKBKG (8517)', 'IKZF1 (10320)',
 'IKZF2 (22807)', 'IKZF3 (22806)', 'IKZF4 (64375)', 'IKZF5 (64376)', 'IL10
 (3586)', 'IL10RA (3587)', 'IL10RB (3588)', 'IL11 (3589)', 'IL11RA (3590)',
 'IL12A (3592)', 'IL12B (3593)', 'IL12RB1 (3594)', 'IL12RB2 (3595)', 'IL13
 (3596)', 'IL13RA1 (3597)', 'IL13RA2 (3598)', 'IL15 (3600)', 'IL15RA (3601)',
 'IL16 (3603)', 'IL17A (3605)', 'IL17B (27190)', 'IL17C (27189)', 'IL17D
 (53342)', 'IL17F (112744)', 'IL17RA (23765)', 'IL17RB (55540)', 'IL17RC
 (84818)', 'IL17RD (54756)', 'IL17RE (132014)', 'IL17REL (400935)', 'IL18
 (3606)', 'IL18BP (10068)', 'IL18R1 (8809)', 'IL18RAP (8807)', 'IL19 (29949)',
 'IL1A (3552)', 'IL1B (3553)', 'IL1F10 (84639)', 'IL1R1 (3554)', 'IL1R2 (7850)',
 'IL1RAP (3556)', 'IL1RAPL1 (11141)', 'IL1RAPL2 (26280)', 'IL1RL1 (9173)',
 'IL1RL2 (8808)', 'IL1RN (3557)', 'IL2 (3558)', 'IL20 (50604)', 'IL20RA (53832)',
 'IL20RB (53833)', 'IL21 (59067)', 'IL21R (50615)', 'IL22 (50616)', 'IL22RA1
 (58985)', 'IL22RA2 (116379)', 'IL23A (51561)', 'IL23R (149233)', 'IL24 (11009)',
 'IL25 (64806)', 'IL26 (55801)', 'IL27 (246778)', 'IL27RA (9466)', 'IL2RA
 (3559)', 'IL2RB (3560)', 'IL2RG (3561)', 'IL3 (3562)', 'IL31 (386653)', 'IL31RA
 (133396)', 'IL32 (9235)', 'IL33 (90865)', 'IL34 (146433)', 'IL36A (27179)',
 'IL36B (27177)', 'IL36G (56300)', 'IL36RN (26525)', 'IL37 (27178)', 'IL3RA
 (3563)', 'IL4 (3565)', 'IL4I1 (259307)', 'IL4R (3566)', 'IL5 (3567)', 'IL5RA
 (3568)', 'IL6 (3569)', 'IL6R (3570)', 'IL6ST (3572)', 'IL7 (3574)', 'IL7R
 (3575)', 'IL9 (3578)', 'IL9R (3581)', 'ILDR1 (286676)', 'ILDR2 (387597)', 'ILF2
 (3608)', 'ILF3 (3609)', 'ILK (3611)', 'ILKAP (80895)', 'ILRUN (64771)', 'ILVBL
 (10994)', 'IMMP1L (196294)', 'IMMP2L (83943)', 'IMMT (10989)', 'IMP3 (55272)',
 'IMP4 (92856)', 'IMPA1 (3612)', 'IMPA2 (3613)', 'IMPACT (55364)', 'IMPDH1
 (3614)', 'IMPDH2 (3615)', 'IMPG1 (3617)', 'IMPG2 (50939)', 'INA (9118)', 'INAFM1
 (255783)', 'INAFM2 (100505573)', 'INAVA (55765)', 'INCA1 (388324)', 'INCENP
 (3619)', 'INF2 (64423)', 'ING1 (3621)', 'ING2 (3622)', 'ING3 (54556)', 'ING4
 (51147)', 'ING5 (84289)', 'INHA (3623)', 'INHBA (3624)', 'INHBB (3625)', 'INHBC
 (3626)', 'INHBE (83729)', 'INIP (58493)', 'INKA1 (389119)', 'INKA2 (55924)',
 'INMT (11185)', 'INO80 (54617)', 'INO80B (83444)', 'INO80C (125476)', 'INO80D
 (54891)', 'INO80E (283899)', 'INPP1 (3628)', 'INPP4A (3631)', 'INPP4B (8821)',
 'INPP5A (3632)', 'INPP5B (3633)', 'INPP5D (3635)', 'INPP5E (56623)', 'INPP5F
 (22876)', 'INPP5J (27124)', 'INPP5K (51763)', 'INPPL1 (3636)', 'INS (3630)',
 'INSC (387755)', 'INSIG1 (3638)', 'INSIG2 (51141)', 'INSL3 (3640)', 'INSL4
 (3641)', 'INSL5 (10022)', 'INSL6 (11172)', 'INSM1 (3642)', 'INSM2 (84684)',
 'INSR (3643)', 'INSRR (3645)', 'INSYN1 (388135)', 'INSYN2A (642938)', 'INSYN2B
 (100131897)', 'INTS1 (26173)', 'INTS10 (55174)', 'INTS11 (54973)', 'INTS12
 (57117)', 'INTS13 (55726)', 'INTS14 (81556)', 'INTS15 (79034)', 'INTS2 (57508)',
 'INTS3 (65123)', 'INTS4 (92105)', 'INTS5 (80789)', 'INTS6 (26512)', 'INTS6L
 (203522)', 'INTS7 (25896)', 'INTS8 (55656)', 'INTS9 (55756)', 'INTU (27152)',
 'INVS (27130)', 'IP6K1 (9807)', 'IP6K2 (51447)', 'IP6K3 (117283)', 'IPCEF1
 (26034)', 'IPMK (253430)', 'IP011 (51194)', 'IP013 (9670)', 'IP04 (79711)',
 'IP05 (3843)', 'IP07 (10527)', 'IP08 (10526)', 'IP09 (55705)', 'IPP (3652)',
 'IPPK (64768)', 'IQANK1 (642574)', 'IQCA1 (79781)', 'IQCA1L (392843)', 'IQCB1
 (9657)', 'IQCC (55721)', 'IQCD (115811)', 'IQCE (23288)', 'IQCF1 (132141)',
 'IQCF2 (389123)', 'IQCF3 (401067)', 'IQCF5 (389124)', 'IQCF6 (440956)', 'IQCG
 (84223)', 'IQCH (64799)', 'IQCJ (654502)', 'IQCK (124152)', 'IQCM (285423)',
 'IQCN (80726)', 'IQGAP1 (8826)', 'IQGAP2 (10788)', 'IQGAP3 (128239)', 'IQSEC1

(9922)', 'IQSEC2 (23096)', 'IQSEC3 (440073)', 'IQUB (154865)', 'IRAG1 (10335)',
 'IRAG2 (4033)', 'IRAK1 (3654)', 'IRAK1BP1 (134728)', 'IRAK2 (3656)', 'IRAK3
 (11213)', 'IRAK4 (51135)', 'IREB2 (3658)', 'IRF1 (3659)', 'IRF2 (3660)',
 'IRF2BP1 (26145)', 'IRF2BP2 (359948)', 'IRF2BPL (64207)', 'IRF3 (3661)', 'IRF4
 (3662)', 'IRF5 (3663)', 'IRF6 (3664)', 'IRF7 (3665)', 'IRF8 (3394)', 'IRF9
 (10379)', 'IRGC (56269)', 'IRGM (345611)', 'IRGQ (126298)', 'IRS1 (3667)', 'IRS2
 (8660)', 'IRS4 (8471)', 'IRX1 (79192)', 'IRX2 (153572)', 'IRX3 (79191)', 'IRX4
 (50805)', 'IRX5 (10265)', 'IRX6 (79190)', 'ISCA1 (81689)', 'ISCA2 (122961)',
 'ISCU (23479)', 'ISG15 (9636)', 'ISG20 (3669)', 'ISG20L2 (81875)', 'ISL1
 (3670)', 'ISL2 (64843)', 'ISLR (3671)', 'ISLR2 (57611)', 'ISM1 (140862)', 'ISM2
 (145501)', 'ISOC1 (51015)', 'ISOC2 (79763)', 'IST1 (9798)', 'ISX (91464)', 'ISY1
 (57461)', 'ISYNA1 (51477)', 'ITCH (83737)', 'ITFG1 (81533)', 'ITFG2 (55846)',
 'ITGA1 (3672)', 'ITGA10 (8515)', 'ITGA11 (22801)', 'ITGA2 (3673)', 'ITGA2B
 (3674)', 'ITGA3 (3675)', 'ITGA4 (3676)', 'ITGA5 (3678)', 'ITGA6 (3655)', 'ITGA7
 (3679)', 'ITGA8 (8516)', 'ITGA9 (3680)', 'ITGAD (3681)', 'ITGAE (3682)', 'ITGAL
 (3683)', 'ITGAM (3684)', 'ITGAV (3685)', 'ITGAX (3687)', 'ITGB1 (3688)',
 'ITGB1BP1 (9270)', 'ITGB1BP2 (26548)', 'ITGB2 (3689)', 'ITGB3 (3690)', 'ITGB3BP
 (23421)', 'ITGB4 (3691)', 'ITGB5 (3693)', 'ITGB6 (3694)', 'ITGB7 (3695)', 'ITGB8
 (3696)', 'ITGBL1 (9358)', 'ITIH1 (3697)', 'ITIH2 (3698)', 'ITIH3 (3699)', 'ITIH4
 (3700)', 'ITIH5 (80760)', 'ITIH6 (347365)', 'ITK (3702)', 'ITLN1 (55600)',
 'ITLN2 (142683)', 'ITM2A (9452)', 'ITM2B (9445)', 'ITM2C (81618)', 'ITPA
 (3704)', 'ITPK1 (3705)', 'ITPKA (3706)', 'ITPKB (3707)', 'ITPKC (80271)', 'ITPR1
 (3708)', 'ITPR2 (3709)', 'ITPR3 (3710)', 'ITPRID1 (223075)', 'ITPRID2 (6744)',
 'ITPRIP (85450)', 'ITPRIPL1 (150771)', 'ITPRIPL2 (162073)', 'ITSN1 (6453)',
 'ITSN2 (50618)', 'IVD (3712)', 'IVL (3713)', 'IVNS1ABP (10625)', 'IWS1 (55677)',
 'IYD (389434)', 'IZUM01 (284359)', 'IZUM01R (390243)', 'IZUM02 (126123)',
 'IZUM03 (100129669)', 'IZUM04 (113177)', 'JADE1 (79960)', 'JADE2 (23338)',
 'JADE3 (9767)', 'JAG1 (182)', 'JAG2 (3714)', 'JAGN1 (84522)', 'JAK1 (3716)',
 'JAK2 (3717)', 'JAK3 (3718)', 'JAKMIP1 (152789)', 'JAKMIP2 (9832)', 'JAKMIP3
 (282973)', 'JAM2 (58494)', 'JAM3 (83700)', 'JAML (120425)', 'JARID2 (3720)',
 'JAZF1 (221895)', 'JCAD (57608)', 'JCHAIN (3512)', 'JDP2 (122953)', 'JHY
 (79864)', 'JKAMP (51528)', 'JMJD1C (221037)', 'JMJD4 (65094)', 'JMJD6 (23210)',
 'JMJD7 (100137047)', 'JMJD8 (339123)', 'JMY (133746)', 'JOSD1 (9929)', 'JOSD2
 (126119)', 'JPH1 (56704)', 'JPH2 (57158)', 'JPH3 (57338)', 'JPH4 (84502)', 'JPT1
 (51155)', 'JPT2 (90861)', 'JRK (8629)', 'JRKL (8690)', 'JSRP1 (126306)', 'JTB
 (10899)', 'JUN (3725)', 'JUNB (3726)', 'JUND (3727)', 'JUP (3728)', 'KALRN
 (8997)', 'KANK1 (23189)', 'KANK2 (25959)', 'KANK3 (256949)', 'KANK4 (163782)',
 'KANSL1 (284058)', 'KANSL1L (151050)', 'KANSL2 (54934)', 'KANSL3 (55683)',
 'KARS1 (3735)', 'KASH5 (147872)', 'KAT14 (57325)', 'KAT2A (2648)', 'KAT2B
 (8850)', 'KAT5 (10524)', 'KAT6A (7994)', 'KAT6B (23522)', 'KAT7 (11143)', 'KAT8
 (84148)', 'KATNA1 (11104)', 'KATNAL1 (84056)', 'KATNAL2 (83473)', 'KATNB1
 (10300)', 'KATNBL1 (79768)', 'KATNIP (23247)', 'KAZALD1 (81621)', 'KAZN
 (23254)', 'KBTBD11 (9920)', 'KBTBD12 (166348)', 'KBTBD13 (390594)', 'KBTBD2
 (25948)', 'KBTBD3 (143879)', 'KBTBD4 (55709)', 'KBTBD6 (89890)', 'KBTBD7
 (84078)', 'KBTBD8 (84541)', 'KCMF1 (56888)', 'KCNA1 (3736)', 'KCNA10 (3744)',
 'KCNA2 (3737)', 'KCNA3 (3738)', 'KCNA4 (3739)', 'KCNA5 (3741)', 'KCNA6 (3742)',
 'KCNA7 (3743)', 'KCNA8 (3744)', 'KCNA9 (3745)', 'KCNA10 (3746)', 'KCNA11 (3747)',
 'KCNA12 (3748)', 'KCNA13 (3749)', 'KCNA14 (3750)', 'KCNA15 (3751)', 'KCNA16 (3752)',
 'KCNA17 (3753)', 'KCNA18 (3754)', 'KCNA19 (3755)', 'KCNA20 (3756)', 'KCNA21 (3757)',
 'KCNA22 (3758)', 'KCNA23 (3759)', 'KCNA24 (3760)', 'KCNA25 (3761)', 'KCNA26 (3762)',
 'KCNA27 (3763)', 'KCNA28 (3764)', 'KCNA29 (3765)', 'KCNA30 (3766)', 'KCNA31 (3767)',
 'KCNA32 (3768)', 'KCNA33 (3769)', 'KCNA34 (3770)', 'KCNA35 (3771)', 'KCNA36 (3772)',
 'KCNA37 (3773)', 'KCNA38 (3774)', 'KCNA39 (3775)', 'KCNA40 (3776)', 'KCNA41 (3777)',
 'KCNA42 (3778)', 'KCNA43 (3779)', 'KCNA44 (3780)', 'KCNA45 (3781)', 'KCNA46 (3782)',
 'KCNA47 (3783)', 'KCNA48 (3784)', 'KCNA49 (3785)', 'KCNA50 (3786)', 'KCNA51 (3787)',
 'KCNA52 (3788)', 'KCNA53 (3789)', 'KCNA54 (3790)', 'KCNA55 (3791)', 'KCNA56 (3792)',
 'KCNA57 (3793)', 'KCNA58 (3794)', 'KCNA59 (3795)', 'KCNA60 (3796)', 'KCNA61 (3797)',
 'KCNA62 (3798)', 'KCNA63 (3799)', 'KCNA64 (3800)', 'KCNA65 (3801)', 'KCNA66 (3802)',
 'KCNA67 (3803)', 'KCNA68 (3804)', 'KCNA69 (3805)', 'KCNA70 (3806)', 'KCNA71 (3807)',
 'KCNA72 (3808)', 'KCNA73 (3809)', 'KCNA74 (3810)', 'KCNA75 (3811)', 'KCNA76 (3812)',
 'KCNA77 (3813)', 'KCNA78 (3814)', 'KCNA79 (3815)', 'KCNA80 (3816)', 'KCNA81 (3817)',
 'KCNA82 (3818)', 'KCNA83 (3819)', 'KCNA84 (3820)', 'KCNA85 (3821)', 'KCNA86 (3822)',
 'KCNA87 (3823)', 'KCNA88 (3824)', 'KCNA89 (3825)', 'KCNA90 (3826)', 'KCNA91 (3827)',
 'KCNA92 (3828)', 'KCNA93 (3829)', 'KCNA94 (3830)', 'KCNA95 (3831)', 'KCNA96 (3832)',
 'KCNA97 (3833)', 'KCNA98 (3834)', 'KCNA99 (3835)', 'KCNA100 (3836)', 'KCNA101 (3837)',
 'KCNA102 (3838)', 'KCNA103 (3839)', 'KCNA104 (3840)', 'KCNA105 (3841)', 'KCNA106 (3842)',
 'KCNA107 (3843)', 'KCNA108 (3844)', 'KCNA109 (3845)', 'KCNA110 (3846)', 'KCNA111 (3847)',
 'KCNA112 (3848)', 'KCNA113 (3849)', 'KCNA114 (3850)', 'KCNA115 (3851)', 'KCNA116 (3852)',
 'KCNA117 (3853)', 'KCNA118 (3854)', 'KCNA119 (3855)', 'KCNA120 (3856)', 'KCNA121 (3857)',
 'KCNA122 (3858)', 'KCNA123 (3859)', 'KCNA124 (3860)', 'KCNA125 (3861)', 'KCNA126 (3862)',
 'KCNA127 (3863)', 'KCNA128 (3864)', 'KCNA129 (3865)', 'KCNA130 (3866)', 'KCNA131 (3867)',
 'KCNA132 (3868)', 'KCNA133 (3869)', 'KCNA134 (3870)', 'KCNA135 (3871)', 'KCNA136 (3872)',
 'KCNA137 (3873)', 'KCNA138 (3874)', 'KCNA139 (3875)', 'KCNA140 (3876)', 'KCNA141 (3877)',
 'KCNA142 (3878)', 'KCNA143 (3879)', 'KCNA144 (3880)', 'KCNA145 (3881)', 'KCNA146 (3882)',
 'KCNA147 (3883)', 'KCNA148 (3884)', 'KCNA149 (3885)', 'KCNA150 (3886)', 'KCNA151 (3887)',
 'KCNA152 (3888)', 'KCNA153 (3889)', 'KCNA154 (3890)', 'KCNA155 (3891)', 'KCNA156 (3892)',
 'KCNA157 (3893)', 'KCNA158 (3894)', 'KCNA159 (3895)', 'KCNA160 (3896)', 'KCNA161 (3897)',
 'KCNA162 (3898)', 'KCNA163 (3899)', 'KCNA164 (3900)', 'KCNA165 (3901)', 'KCNA166 (3902)',
 'KCNA167 (3903)', 'KCNA168 (3904)', 'KCNA169 (3905)', 'KCNA170 (3906)', 'KCNA171 (3907)',
 'KCNA172 (3908)', 'KCNA173 (3909)', 'KCNA174 (3910)', 'KCNA175 (3911)', 'KCNA176 (3912)',
 'KCNA177 (3913)', 'KCNA178 (3914)', 'KCNA179 (3915)', 'KCNA180 (3916)', 'KCNA181 (3917)',
 'KCNA182 (3918)', 'KCNA183 (3919)', 'KCNA184 (3920)', 'KCNA185 (3921)', 'KCNA186 (3922)',
 'KCNA187 (3923)', 'KCNA188 (3924)', 'KCNA189 (3925)', 'KCNA190 (3926)', 'KCNA191 (3927)',
 'KCNA192 (3928)', 'KCNA193 (3929)', 'KCNA194 (3930)', 'KCNA195 (3931)', 'KCNA196 (3932)',
 'KCNA197 (3933)', 'KCNA198 (3934)', 'KCNA199 (3935)', 'KCNA200 (3936)', 'KCNA201 (3937)',
 'KCNA202 (3938)', 'KCNA203 (3939)', 'KCNA204 (3940)', 'KCNA205 (3941)', 'KCNA206 (3942)',
 'KCNA207 (3943)', 'KCNA208 (3944)', 'KCNA209 (3945)', 'KCNA210 (3946)', 'KCNA211 (3947)',
 'KCNA212 (3948)', 'KCNA213 (3949)', 'KCNA214 (3950)', 'KCNA215 (3951)', 'KCNA216 (3952)',
 'KCNA217 (3953)', 'KCNA218 (3954)', 'KCNA219 (3955)', 'KCNA220 (3956)', 'KCNA221 (3957)',
 'KCNA222 (3958)', 'KCNA223 (3959)', 'KCNA224 (3960)', 'KCNA225 (3961)', 'KCNA226 (3962)',
 'KCNA227 (3963)', 'KCNA228 (3964)', 'KCNA229 (3965)', 'KCNA230 (3966)', 'KCNA231 (3967)',
 'KCNA232 (3968)', 'KCNA233 (3969)', 'KCNA234 (3970)', 'KCNA235 (3971)', 'KCNA236 (3972)',
 'KCNA237 (3973)', 'KCNA238 (3974)', 'KCNA239 (3975)', 'KCNA240 (3976)', 'KCNA241 (3977)',
 'KCNA242 (3978)', 'KCNA243 (3979)', 'KCNA244 (3980)', 'KCNA245 (3981)', 'KCNA246 (3982)',
 'KCNA247 (3983)', 'KCNA248 (3984)', 'KCNA249 (3985)', 'KCNA250 (3986)', 'KCNA251 (3987)',
 'KCNA252 (3988)', 'KCNA253 (3989)', 'KCNA254 (3990)', 'KCNA255 (3991)', 'KCNA256 (3992)',
 'KCNA257 (3993)', 'KCNA258 (3994)', 'KCNA259 (3995)', 'KCNA260 (3996)', 'KCNA261 (3997)',
 'KCNA262 (3998)', 'KCNA263 (3999)', 'KCNA264 (4000)', 'KCNA265 (4001)', 'KCNA266 (4002)',
 'KCNA267 (4003)', 'KCNA268 (4004)', 'KCNA269 (4005)', 'KCNA270 (4006)', 'KCNA271 (4007)',
 'KCNA272 (4008)', 'KCNA273 (4009)', 'KCNA274 (4010)', 'KCNA275 (4011)', 'KCNA276 (4012)',
 'KCNA277 (4013)', 'KCNA278 (4014)', 'KCNA279 (4015)', 'KCNA280 (4016)', 'KCNA281 (4017)',
 'KCNA282 (4018)', 'KCNA283 (4019)', 'KCNA284 (4020)', 'KCNA285 (4021)', 'KCNA286 (4022)',
 'KCNA287 (4023)', 'KCNA288 (4024)', 'KCNA289 (4025)', 'KCNA290 (4026)', 'KCNA291 (4027)',
 'KCNA292 (4028)', 'KCNA293 (4029)', 'KCNA294 (4030)', 'KCNA295 (4031)', 'KCNA296 (4032)',
 'KCNA297 (4033)', 'KCNA298 (4034)', 'KCNA299 (4035)', 'KCNA300 (4036)', 'KCNA301 (4037)',
 'KCNA302 (4038)', 'KCNA303 (4039)', 'KCNA304 (4040)', 'KCNA305 (4041)', 'KCNA306 (4042)',
 'KCNA307 (4043)', 'KCNA308 (4044)', 'KCNA309 (4045)', 'KCNA310 (4046)', 'KCNA311 (4047)',
 'KCNA312 (4048)', 'KCNA313 (4049)', 'KCNA314 (4050)', 'KCNA315 (4051)', 'KCNA316 (4052)',
 'KCNA317 (4053)', 'KCNA318 (4054)', 'KCNA319 (4055)', 'KCNA320 (4056)', 'KCNA321 (4057)',
 'KCNA322 (4058)', 'KCNA323 (4059)', 'KCNA324 (4060)', 'KCNA325 (4061)', 'KCNA326 (4062)',
 'KCNA327 (4063)', 'KCNA328 (4064)', 'KCNA329 (4065)', 'KCNA330 (4066)', 'KCNA331 (4067)',
 'KCNA332 (4068)', 'KCNA333 (4069)', 'KCNA334 (4070)', 'KCNA335 (4071)', 'KCNA336 (4072)',
 'KCNA337 (4073)', 'KCNA338 (4074)', 'KCNA339 (4075)', 'KCNA340 (4076)', 'KCNA341 (4077)',
 'KCNA342 (4078)', 'KCNA343 (4079)', 'KCNA344 (4080)', 'KCNA345 (4081)', 'KCNA346 (4082)',
 'KCNA347 (4083)', 'KCNA348 (4084)', 'KCNA349 (4085)', 'KCNA350 (4086)', 'KCNA351 (4087)',
 'KCNA352 (4088)', 'KCNA353 (4089)', 'KCNA354 (4090)', 'KCNA355 (4091)', 'KCNA356 (4092)',
 'KCNA357 (4093)', 'KCNA358 (4094)', 'KCNA359 (4095)', 'KCNA360 (4096)', 'KCNA361 (4097)',
 'KCNA362 (4098)', 'KCNA363 (4099)', 'KCNA364 (4100)', 'KCNA365 (4101)', 'KCNA366 (4102)',
 'KCNA367 (4103)', 'KCNA368 (4104)', 'KCNA369 (4105)', 'KCNA370 (4106)', 'KCNA371 (4107)',
 'KCNA372 (4108)', 'KCNA373 (4109)', 'KCNA374 (4110)', 'KCNA375 (4111)', 'KCNA376 (4112)',
 'KCNA377 (4113)', 'KCNA378 (4114)', 'KCNA379 (4115)', 'KCNA380 (4116)', 'KCNA381 (4117)',
 'KCNA382 (4118)', 'KCNA383 (4119)', 'KCNA384 (4120)', 'KCNA385 (4121)', 'KCNA386 (4122)',
 'KCNA387 (4123)', 'KCNA388 (4124)', 'KCNA389 (4125)', 'KCNA390 (4126)', 'KCNA391 (4127)',
 'KCNA392 (4128)', 'KCNA393 (4129)', 'KCNA394 (4130)', 'KCNA395 (4131)', 'KCNA396 (4132)',
 'KCNA397 (4133)', 'KCNA398 (4134)', 'KCNA399 (4135)', 'KCNA400 (4136)', 'KCNA401 (4137)',
 'KCNA402 (4138)', 'KCNA403 (4139)', 'KCNA404 (4140)', 'KCNA405 (4141)', 'KCNA406 (4142)',
 'KCNA407 (4143)', 'KCNA408 (4144)', 'KCNA409 (4145)', 'KCNA410 (4146)', 'KCNA411 (4147)',
 'KCNA412 (4148)', 'KCNA413 (4149)', 'KCNA414 (4150)', 'KCNA415 (4151)', 'KCNA416 (4152)',
 'KCNA417 (4153)', 'KCNA418 (4154)', 'KCNA419 (4155)', 'KCNA420 (4156)', 'KCNA421 (4157)',
 'KCNA422 (4158)', 'KCNA423 (4159)', 'KCNA424 (4160)', 'KCNA425 (4161)', 'KCNA426 (4162)',
 'KCNA427 (4163)', 'KCNA428 (4164)', 'KCNA429 (4165)', 'KCNA430 (4166)', 'KCNA431 (4167)',
 'KCNA432 (4168)', 'KCNA433 (4169)', 'KCNA434 (4170)', 'KCNA435 (4171)', 'KCNA436 (4172)',
 'KCNA437 (4173)', 'KCNA438 (4174)', 'KCNA439 (4175)', 'KCNA440 (4176)', 'KCNA441 (4177)',
 'KCNA442 (4178)', 'KCNA443 (4179)', 'KCNA444 (4180)', 'KCNA445 (4181)', 'KCNA446 (4182)',
 'KCNA447 (4183)', 'KCNA448 (4184)', 'KCNA449 (4185)', 'KCNA450 (4186)', 'KCNA451 (4187)',
 'KCNA452 (4188)', 'KCNA453 (4189)', 'KCNA454 (4190)', 'KCNA455 (4191)', 'KCNA456 (4192)',
 'KCNA457 (4193)', 'KCNA458 (4194)', 'KCNA459 (4195)', 'KCNA460 (4196)', 'KCNA461 (4197)',
 'KCNA462 (4198)', 'KCNA463 (4199)', 'KCNA464 (4200)', 'KCNA465 (4201)', 'KCNA466 (4202)',
 'KCNA467 (4203)', 'KCNA468 (4204)', 'KCNA469 (4205)', 'KCNA470 (4206)', 'KCNA471 (4207)',
 'KCNA472 (4208)', 'KCNA473 (4209)', 'KCNA474 (4210)', 'KCNA475 (4211)', 'KCNA476 (4212)',
 'KCNA477 (4213)', 'KCNA478 (4214)', 'KCNA479 (4215)', 'KCNA480 (4216)', 'KCNA481 (4217)',
 'KCNA482 (4218)', 'KCNA483 (4219)', 'KCNA484 (4220)', 'KCNA485 (4221)', 'KCNA486 (4222)',
 'KCNA487 (4223)', 'KCNA488 (4224)', 'KCNA489 (4225)', 'KCNA490 (4226)', 'KCNA491 (4227)',
 'KCNA492 (4228)', 'KCNA493 (4229)', 'KCNA494 (4230)', 'KCNA495 (4231)', 'KCNA496 (4232)',
 'KCNA497 (4233)', 'KCNA498 (4234)', 'KCNA499 (4235)', 'KCNA500 (4236)', 'KCNA501 (4237)',
 'KCNA502 (4238)', 'KCNA503 (4239)', 'KCNA504 (4240)', 'KCNA505 (4241)', 'KCNA506 (4242)',
 'KCNA507 (4243)', 'KCNA508 (4244)', 'KCNA509 (4245)', 'KCNA510 (4246)', 'KCNA511 (4247)',
 'KCNA512 (4248)', 'KCNA513 (4249)', 'KCNA514 (4250)', 'KCNA515 (4251)', 'KCNA516 (4252)',
 'KCNA517 (4253)', 'KCNA518 (4254)', 'KCNA519 (4255)', 'KCNA520 (4256)', 'KCNA521 (4257)',
 'KCNA522 (4258)', 'KCNA523 (4259)', 'KCNA524 (4260)', 'KCNA525 (4261)', 'KCNA526 (4262)',
 'KCNA527 (4263)', 'KCNA528 (4264)', 'KCNA529 (4265)', 'KCNA530 (4266)', 'KCNA531 (4267)',
 'KCNA532 (4268)', 'KCNA533 (4269)', 'KCNA534 (4270)', 'KCNA535 (4271)', 'KCNA536 (4272)',
 'KCNA537 (4273)', 'KCNA538 (4274)', 'KCNA539 (4275)', 'KCNA540 (4276)', 'KCNA541 (4277)',
 'KCNA542 (4278)', 'KCNA543 (4279)', 'KCNA544 (4280)', 'KCNA545 (4281)', 'KCNA546 (4282)',
 'KCNA547 (4283)', 'KCNA548 (4284)', 'KCNA549 (4285)', 'KCNA550 (4286)', 'KCNA551 (4287)',
 'KCNA552 (4288)', 'KCNA553 (4289)', 'KCNA554 (4290)', 'KCNA555 (4291)', 'KCNA556 (4292)',
 'KCNA557 (4293)', 'KCNA558 (4294)', 'KCNA559 (4295)', 'KCNA560 (4296)', 'KCNA561 (4297)',
 'KCNA562 (4298)', 'KCNA563 (4299)', 'KCNA564 (4300)', 'KCNA565 (4301)', 'KCNA566 (4302)',
 'KCNA567 (4303)', 'KCNA568 (4304)', 'KCNA569 (4305)', 'KCNA570 (4306)', 'KCNA571 (4307)',
 'KCNA572 (4308)', 'KCNA573 (4309)', 'KCNA574 (4310)', 'KCNA575 (4311)', 'KCNA576 (4312)',
 'KCNA577 (4313)', 'KCNA578 (4314)', 'KCNA579 (4315)', 'KCNA580 (4316)', 'KCNA581 (4317)',
 'KCNA582 (4318)', 'KCNA583 (4319)', 'KCNA584 (4320)', 'KCNA585 (4321)', 'KCNA586 (4322)',
 'KCNA587 (4323)', 'KCNA588 (4324)', 'KCNA589 (4325)', 'KCNA590 (4326)', 'KCNA591 (4327)',
 'KCNA592 (4328)', 'KCNA593 (4329)', 'KCNA594 (4330)', 'KCNA595 (4331)', 'KCNA596 (4332)',
 'KCNA597 (4333)', 'KCNA598 (4334)', 'KCNA599 (4335)', 'KCNA600 (4336)', 'KCNA601 (4337)',
 'KCNA602 (4338)', 'KCNA603 (4339)', 'KCNA604 (4340)', 'KCNA605 (4341)', 'KCNA606 (4342)',
 'KCNA607 (4343)', 'KCNA608 (4344)', 'KCNA609 (4345)', 'KCNA610 (4346)', 'KCNA611 (4347)',
 'KCNA612 (4348)', 'KCNA613 (4349)', 'KCNA614 (4350)', 'KCNA615 (4351)', 'KCNA616 (4352)',
 'KCNA617 (4353)', 'KCNA618 (4354)', 'KCNA619 (4355)', 'KCNA620 (4356)', 'KCNA621 (4357)',
 'KCNA622 (4358)', 'KCNA623 (4359)', 'KCNA624 (4360)', 'KCNA625 (4361)', 'KCNA626 (4362)',
 'KCNA627 (4363)', 'KCNA628 (4364)', 'KCNA629 (4365)', 'KCNA630 (4366)', 'KCNA631 (4367)',
 'KCNA632 (4368)', 'KCNA633 (4369)', 'KCNA634 (4370)', 'KCNA635 (4371)', 'KCNA636 (4372)',
 'KCNA637 (4373)', 'KCNA638 (4374)', 'KCNA639 (4375)', 'KCNA640 (4376)', 'KCNA641 (4377)',
 'KCNA642 (4378)', 'KCNA643 (4379)', 'KCNA644 (4380)', 'KCNA645 (4381)', 'KCNA646 (4382)',
 'KCNA647 (4383)', 'KCNA648 (4384)', 'KCNA649 (4385)', 'KCNA650 (4386)', 'KCNA651 (4387)',
 'KCNA652 (4388)', 'KCNA653 (4389)', 'KCNA654 (4390)', 'KCNA655 (4391)', 'KCNA656 (4392)',
 'KCNA657 (4393)', 'KCNA658 (4394)', 'KCNA659 (4395)', 'KCNA660 (4396)', 'KCNA661 (4397)',
 'KCNA662 (4398)', 'KCNA663 (4399)', 'KCNA664 (4400)', 'KCNA665 (4401)', 'KCNA666 (4402)',
 'KCNA667 (4403)', 'KCNA668 (4404)', 'KCNA669 (4405)', 'KCNA670 (4406)', 'KCNA671 (4407)',
 'KCNA672 (4408)', 'KCNA673 (4409)', 'KCNA674 (4410)', 'KCNA675 (4411)', 'KCNA676 (4412)',
 'KCNA677 (4413)', 'KCNA678 (4414)', 'KCNA679 (4415)', 'KCNA680 (4416)', 'KCNA681 (4417)',
 'KCNA682 (4418)', 'KCNA683 (4419)', 'KCNA684 (4420)', 'KCNA685 (4421)', 'KCNA686 (4422)',
 'KCNA687 (4423)', 'KCNA688 (4424)', 'KCNA689 (4425)', 'KCNA690 (4426)', 'KCNA691 (4427)',
 'KCNA692 (4428)', 'KCNA693 (4429)', 'KCNA694 (4430)', 'KCNA695 (4431)', 'KCNA696 (4432)',
 'KCNA697 (4433)', 'KCNA698 (4434)', 'KCNA699 (4435)', 'KCNA700 (4436)', 'KCNA701 (4437)',
 'KCNA702 (4438)', 'KCNA703 (4439)', 'KCNA704 (4440)', 'KCNA705 (4441)', 'KCNA706 (4442)',
 'KCNA707 (4443)', 'KCNA708 (4444)', 'KCNA709 (4445)', 'KCNA710 (4446)', 'KCNA711 (4447)',
 'KCNA712 (4448)', 'KCNA713 (4449)', 'KCNA714 (4450)', 'KCNA715 (4451)', 'KCNA716 (4452)',
 'KCNA717 (4453)', 'KCNA718 (4454)', 'KCNA719 (4455)', 'KCNA720 (4456)', 'KCNA721 (4457)',
 'KCNA722 (4458)', 'KCNA723 (4459)', 'KCNA724 (4460)', 'KCNA725 (4461)', 'KCNA726 (4462)',
 'KCNA727 (4463)', 'KCNA728 (4464)', 'KCNA729 (4465)', 'KCNA730 (4466)', 'KCNA731 (4467)',
 'KCNA732 (4468)', 'KCNA733 (4469)', 'KCNA734 (4470)', 'KCNA735 (4471)', 'KCNA736 (4472)',
 'KCNA737 (4473)', 'KCNA738 (4474)', 'KCNA739 (4475)', 'KCNA740 (4476)', 'KCNA741 (4477)',
 'KCNA742 (4478)', 'KCNA743 (4479)', 'KCNA744 (4480)', 'KCNA745 (4481)', 'KCNA746 (4482)',
 'KCNA747 (4483)', 'KCNA748 (4484)', 'KCNA749 (4485)', 'KCNA750 (4486)', 'KCNA751 (4487)',
 'KCNA752 (4488)', 'KCNA753 (4489)', 'KCNA754 (4490)', 'KCNA755 (4491)', 'KCNA756 (4492)',
 'KCNA757 (4493)', 'KCNA758 (4494)', 'KCNA759 (4495)', 'KCNA760 (4496)', 'KCNA761 (4497)',
 'KCNA762 (4498)', 'KCNA763 (4499)', 'KCNA764 (4500)', 'KCNA765 (4501)', 'KCNA766 (4502)',
 'KCNA767 (4503)', 'KCNA768 (4504)', 'KCNA769 (4505)', 'KCNA770

(3749)', 'KCND1 (3750)', 'KCND2 (3751)', 'KCND3 (3752)', 'KCNE1 (3753)', 'KCNE2
 (9992)', 'KCNE3 (10008)', 'KCNE4 (23704)', 'KCNE5 (23630)', 'KCNF1 (3754)',
 'KCG1 (3755)', 'KCG2 (26251)', 'KCG3 (170850)', 'KCG4 (93107)', 'KCNH1
 (3756)', 'KCNH2 (3757)', 'KCNH3 (23416)', 'KCNH4 (23415)', 'KCNH5 (27133)',
 'KCNH6 (81033)', 'KCNH7 (90134)', 'KCNH8 (131096)', 'KCNIP1 (30820)', 'KCNIP2
 (30819)', 'KCNIP3 (30818)', 'KCNIP4 (80333)', 'KCNJ1 (3758)', 'KCNJ10 (3766)',
 'KCNJ11 (3767)', 'KCNJ12 (3768)', 'KCNJ13 (3769)', 'KCNJ14 (3770)', 'KCNJ15
 (3772)', 'KCNJ16 (3773)', 'KCNJ2 (3759)', 'KCNJ3 (3760)', 'KCNJ4 (3761)', 'KCNJ5
 (3762)', 'KCNJ6 (3763)', 'KCNJ8 (3764)', 'KCNJ9 (3765)', 'KCNK1 (3775)', 'KCNK10
 (54207)', 'KCNK12 (56660)', 'KCNK13 (56659)', 'KCNK15 (60598)', 'KCNK16
 (83795)', 'KCNK17 (89822)', 'KCNK18 (338567)', 'KCNK2 (3776)', 'KCNK3 (3777)',
 'KCNK4 (50801)', 'KCNK5 (8645)', 'KCNK6 (9424)', 'KCNK7 (10089)', 'KCNK9
 (51305)', 'KCNMA1 (3778)', 'KCNMB1 (3779)', 'KCNMB2 (10242)', 'KCNMB3 (27094)',
 'KCNMB4 (27345)', 'KCNN1 (3780)', 'KCNN2 (3781)', 'KCNN3 (3782)', 'KCNN4
 (3783)', 'KCNQ1 (3784)', 'KCNQ2 (3785)', 'KCNQ3 (3786)', 'KCNQ4 (9132)', 'KCNQ5
 (56479)', 'KCNRG (283518)', 'KCMS1 (3787)', 'KCMS2 (3788)', 'KCMS3 (3790)',
 'KCNT1 (57582)', 'KCNT2 (343450)', 'KGNU1 (157855)', 'KCNV1 (27012)', 'KCNV2
 (169522)', 'KCP (375616)', 'KCTD1 (284252)', 'KCTD10 (83892)', 'KCTD11
 (147040)', 'KCTD12 (115207)', 'KCTD13 (253980)', 'KCTD14 (65987)', 'KCTD15
 (79047)', 'KCTD16 (57528)', 'KCTD17 (79734)', 'KCTD18 (130535)', 'KCTD19
 (146212)', 'KCTD2 (23510)', 'KCTD20 (222658)', 'KCTD21 (283219)', 'KCTD3
 (51133)', 'KCTD4 (386618)', 'KCTD5 (54442)', 'KCTD6 (200845)', 'KCTD7 (154881)',
 'KCTD8 (386617)', 'KCTD9 (54793)', 'KDEL1 (10945)', 'KDEL2 (11014)', 'KDEL3
 (11015)', 'KDF1 (126695)', 'KDM1A (23028)', 'KDM1B (221656)', 'KDM2A (22992)',
 'KDM2B (84678)', 'KDM3A (55818)', 'KDM3B (51780)', 'KDM4A (9682)', 'KDM4B
 (23030)', 'KDM4C (23081)', 'KDM4D (55693)', 'KDM4E (390245)', 'KDM5A (5927)',
 'KDM5B (10765)', 'KDM5C (8242)', 'KDM5D (8284)', 'KDM6A (7403)', 'KDM6B
 (23135)', 'KDM7A (80853)', 'KDM8 (79831)', 'KDR (3791)', 'KDSR (2531)', 'KEAP1
 (9817)', 'KEL (3792)', 'KERA (11081)', 'KGD4 (92259)', 'KHDC1 (80759)', 'KHDC1L
 (100129128)', 'KHDC3L (154288)', 'KHDC4 (22889)', 'KHDRBS1 (10657)', 'KHDRBS2
 (202559)', 'KHDRBS3 (10656)', 'KHK (3795)', 'KHNYN (23351)', 'KHSRP (8570)',
 'KIAA0040 (9674)', 'KIAA0232 (9778)', 'KIAA0319 (9856)', 'KIAA0319L (79932)',
 'KIAA0408 (9729)', 'KIAA0513 (9764)', 'KIAA0586 (9786)', 'KIAA0753 (9851)',
 'KIAA0825 (285600)', 'KIAA0930 (23313)', 'KIAA1143 (57456)', 'KIAA1191 (57179)',
 'KIAA1210 (57481)', 'KIAA1217 (56243)', 'KIAA1328 (57536)', 'KIAA1549 (57670)',
 'KIAA1549L (25758)', 'KIAA1586 (57691)', 'KIAA1614 (57710)', 'KIAA1671 (85379)',
 'KIAA1755 (85449)', 'KIAA1958 (158405)', 'KIAA2012 (100652824)', 'KIAA2013
 (90231)', 'KICS2 (144577)', 'KIDINS220 (57498)', 'KIF11 (3832)', 'KIF12
 (113220)', 'KIF13A (63971)', 'KIF13B (23303)', 'KIF14 (9928)', 'KIF15 (56992)',
 'KIF16B (55614)', 'KIF17 (57576)', 'KIF18A (81930)', 'KIF18B (146909)', 'KIF19
 (124602)', 'KIF1A (547)', 'KIF1B (23095)', 'KIF1C (10749)', 'KIF20A (10112)',
 'KIF20B (9585)', 'KIF21A (55605)', 'KIF21B (23046)', 'KIF22 (3835)', 'KIF23
 (9493)', 'KIF24 (347240)', 'KIF25 (3834)', 'KIF26A (26153)', 'KIF26B (55083)',
 'KIF27 (55582)', 'KIF2A (3796)', 'KIF2B (84643)', 'KIF2C (11004)', 'KIF3A
 (11127)', 'KIF3B (9371)', 'KIF3C (3797)', 'KIF4A (24137)', 'KIF4B (285643)',
 'KIF5A (3798)', 'KIF5B (3799)', 'KIF5C (3800)', 'KIF6 (221458)', 'KIF7
 (374654)', 'KIF9 (64147)', 'KIFAP3 (22920)', 'KIFBP (26128)', 'KIFC1 (3833)',
 'KIFC2 (90990)', 'KIFC3 (3801)', 'KIN (22944)', 'KIR2DL1 (3802)', 'KIR2DL3

(3804)', 'KIR2DL4 (3805)', 'KIR3DL1 (3811)', 'KIR3DL2 (3812)', 'KIR3DL3
 (115653)', 'KIRREL1 (55243)', 'KIRREL2 (84063)', 'KIRREL3 (84623)', 'KISS1
 (3814)', 'KISS1R (84634)', 'KIT (3815)', 'KITLG (4254)', 'KIZ (55857)', 'KL
 (9365)', 'KLB (152831)', 'KLC1 (3831)', 'KLC2 (64837)', 'KLC3 (147700)', 'KLC4
 (89953)', 'KLF1 (10661)', 'KLF10 (7071)', 'KLF11 (8462)', 'KLF12 (11278)',
 'KLF13 (51621)', 'KLF14 (136259)', 'KLF15 (28999)', 'KLF16 (83855)', 'KLF17
 (128209)', 'KLF18 (105378952)', 'KLF2 (10365)', 'KLF3 (51274)', 'KLF4 (9314)',
 'KLF5 (688)', 'KLF6 (1316)', 'KLF7 (8609)', 'KLF8 (11279)', 'KLF9 (687)',
 'KLHDC1 (122773)', 'KLHDC10 (23008)', 'KLHDC2 (23588)', 'KLHDC3 (116138)',
 'KLHDC4 (54758)', 'KLHDC7A (127707)', 'KLHDC7B (113730)', 'KLHDC8A (55220)',
 'KLHDC8B (200942)', 'KLHDC9 (126823)', 'KLHL1 (57626)', 'KLHL10 (317719)',
 'KLHL11 (55175)', 'KLHL12 (59349)', 'KLHL13 (90293)', 'KLHL14 (57565)', 'KLHL15
 (80311)', 'KLHL17 (339451)', 'KLHL18 (23276)', 'KLHL2 (11275)', 'KLHL20
 (27252)', 'KLHL21 (9903)', 'KLHL22 (84861)', 'KLHL24 (54800)', 'KLHL25 (64410)',
 'KLHL26 (55295)', 'KLHL28 (54813)', 'KLHL29 (114818)', 'KLHL3 (26249)', 'KLHL30
 (377007)', 'KLHL31 (401265)', 'KLHL32 (114792)', 'KLHL33 (123103)', 'KLHL34
 (257240)', 'KLHL35 (283212)', 'KLHL36 (79786)', 'KLHL38 (340359)', 'KLHL4
 (56062)', 'KLHL40 (131377)', 'KLHL41 (10324)', 'KLHL42 (57542)', 'KLHL5
 (51088)', 'KLHL6 (89857)', 'KLHL7 (55975)', 'KLHL8 (57563)', 'KLHL9 (55958)',
 'KLK1 (3816)', 'KLK10 (5655)', 'KLK11 (11012)', 'KLK12 (43849)', 'KLK13
 (26085)', 'KLK14 (43847)', 'KLK15 (55554)', 'KLK2 (3817)', 'KLK3 (354)', 'KLK4
 (9622)', 'KLK5 (25818)', 'KLK6 (5653)', 'KLK7 (5650)', 'KLK8 (11202)', 'KLK9
 (284366)', 'KLKB1 (3818)', 'KLLN (100144748)', 'KLRB1 (3820)', 'KLRC1 (3821)',
 'KLRC2 (3822)', 'KLRC3 (3823)', 'KLRC4 (8302)', 'KLRD1 (3824)', 'KLRF1 (51348)',
 'KLRF2 (100431172)', 'KLRG1 (10219)', 'KLRG2 (346689)', 'KMO (8564)', 'KMT2A
 (4297)', 'KMT2B (9757)', 'KMT2C (58508)', 'KMT2D (8085)', 'KMT2E (55904)',
 'KMT5A (387893)', 'KMT5B (51111)', 'KMT5C (84787)', 'KNCN (148930)', 'KNDC1
 (85442)', 'KNG1 (3827)', 'KNL1 (57082)', 'KNOP1 (400506)', 'KNSTRN (90417)',
 'KNTC1 (9735)', 'KPLCE (100129271)', 'KPNA1 (3836)', 'KPNA2 (3838)', 'KPNA3
 (3839)', 'KPNA4 (3840)', 'KPNA5 (3841)', 'KPNA6 (23633)', 'KPNA7 (402569)',
 'KPNB1 (3837)', 'KPRP (448834)', 'KPTN (11133)', 'KRAS (3845)', 'KRBA1 (84626)',
 'KRBA2 (124751)', 'KRBOX1 (100506243)', 'KRBOX4 (55634)', 'KRBOX5 (124411)',
 'KRCC1 (51315)', 'KREMEN1 (83999)', 'KREMEN2 (79412)', 'KRI1 (65095)', 'KRIT1
 (889)', 'KRR1 (11103)', 'KRT1 (3848)', 'KRT10 (3858)', 'KRT12 (3859)', 'KRT13
 (3860)', 'KRT14 (3861)', 'KRT15 (3866)', 'KRT16 (3868)', 'KRT17 (3872)', 'KRT18
 (3875)', 'KRT19 (3880)', 'KRT2 (3849)', 'KRT20 (54474)', 'KRT222 (125113)',
 'KRT23 (25984)', 'KRT24 (192666)', 'KRT25 (147183)', 'KRT26 (353288)', 'KRT27
 (342574)', 'KRT28 (162605)', 'KRT3 (3850)', 'KRT31 (3881)', 'KRT32 (3882)',
 'KRT33A (3883)', 'KRT33B (3884)', 'KRT34 (3885)', 'KRT35 (3886)', 'KRT36
 (8689)', 'KRT37 (8688)', 'KRT38 (8687)', 'KRT39 (390792)', 'KRT4 (3851)', 'KRT40
 (125115)', 'KRT5 (3852)', 'KRT6A (3853)', 'KRT6B (3854)', 'KRT6C (286887)',
 'KRT7 (3855)', 'KRT71 (112802)', 'KRT72 (140807)', 'KRT73 (319101)', 'KRT74
 (121391)', 'KRT75 (9119)', 'KRT76 (51350)', 'KRT77 (374454)', 'KRT78 (196374)',
 'KRT79 (338785)', 'KRT8 (3856)', 'KRT80 (144501)', 'KRT81 (3887)', 'KRT82
 (3888)', 'KRT83 (3889)', 'KRT84 (3890)', 'KRT85 (3891)', 'KRT86 (3892)', 'KRT9
 (3857)', 'KRTAP1-1 (81851)', 'KRTAP1-3 (81850)', 'KRTAP1-4 (728255)', 'KRTAP1-5
 (83895)', 'KRTAP10-1 (386677)', 'KRTAP10-10 (353333)', 'KRTAP10-11 (386678)',
 'KRTAP10-2 (386679)', 'KRTAP10-3 (386682)', 'KRTAP10-5 (386680)', 'KRTAP10-8

(386681)', 'KRTAP10-9 (386676)', 'KRTAP11-1 (337880)', 'KRTAP12-1 (353332)',
 'KRTAP12-2 (353323)', 'KRTAP12-3 (386683)', 'KRTAP12-4 (386684)', 'KRTAP13-1
 (140258)', 'KRTAP13-2 (337959)', 'KRTAP13-3 (337960)', 'KRTAP13-4 (284827)',
 'KRTAP15-1 (254950)', 'KRTAP16-1 (100505753)', 'KRTAP17-1 (83902)', 'KRTAP19-1
 (337882)', 'KRTAP19-2 (337969)', 'KRTAP19-3 (337970)', 'KRTAP19-4 (337971)',
 'KRTAP19-5 (337972)', 'KRTAP19-6 (337973)', 'KRTAP19-7 (337974)', 'KRTAP19-8
 (728299)', 'KRTAP20-1 (337975)', 'KRTAP20-2 (337976)', 'KRTAP20-3 (337985)',
 'KRTAP20-4 (100151643)', 'KRTAP21-1 (337977)', 'KRTAP21-2 (337978)', 'KRTAP21-3
 (100288323)', 'KRTAP22-1 (337979)', 'KRTAP22-2 (100288287)', 'KRTAP23-1
 (337963)', 'KRTAP24-1 (643803)', 'KRTAP25-1 (100131902)', 'KRTAP26-1 (388818)',
 'KRTAP27-1 (643812)', 'KRTAP29-1 (100533177)', 'KRTAP3-1 (83896)', 'KRTAP3-2
 (83897)', 'KRTAP3-3 (85293)', 'KRTAP4-1 (85285)', 'KRTAP4-11 (653240)',
 'KRTAP4-2 (85291)', 'KRTAP4-3 (85290)', 'KRTAP4-4 (84616)', 'KRTAP4-5 (85289)',
 'KRTAP4-6 (81871)', 'KRTAP4-8 (728224)', 'KRTAP4-9 (100132386)', 'KRTAP5-1
 (387264)', 'KRTAP5-10 (387273)', 'KRTAP5-11 (440051)', 'KRTAP5-2 (440021)',
 'KRTAP5-3 (387266)', 'KRTAP5-5 (439915)', 'KRTAP5-6 (440023)', 'KRTAP5-7
 (440050)', 'KRTAP5-8 (57830)', 'KRTAP5-9 (3846)', 'KRTAP6-1 (337966)', 'KRTAP6-2
 (337967)', 'KRTAP6-3 (337968)', 'KRTAP7-1 (337878)', 'KRTAP8-1 (337879)',
 'KRTAP9-1 (728318)', 'KRTAP9-3 (83900)', 'KRTAP9-6 (100507608)', 'KRTAP9-8
 (83901)', 'KRTAP9-9 (81870)', 'KRTCAP2 (200185)', 'KRTCAP3 (200634)', 'KRTDAP
 (388533)', 'KSR1 (8844)', 'KSR2 (283455)', 'KTI12 (112970)', 'KTN1 (3895)',
 'KXD1 (79036)', 'KY (339855)', 'KYAT1 (883)', 'KYAT3 (56267)', 'KYNU (8942)',
 'L1CAM (3897)', 'L1TD1 (54596)', 'L2HGDH (79944)', 'L3HYPDH (112849)', 'L3MBTL1
 (26013)', 'L3MBTL2 (83746)', 'L3MBTL3 (84456)', 'L3MBTL4 (91133)', 'LACC1
 (144811)', 'LACRT (90070)', 'LACTB (114294)', 'LACTB2 (51110)', 'LACTBL1
 (646262)', 'LAD1 (3898)', 'LAG3 (3902)', 'LAGE3 (8270)', 'LAIR1 (3903)', 'LAIR2
 (3904)', 'LALBA (3906)', 'LAMA1 (284217)', 'LAMA2 (3908)', 'LAMA3 (3909)',
 'LAMA4 (3910)', 'LAMA5 (3911)', 'LAMB1 (3912)', 'LAMB2 (3913)', 'LAMB3 (3914)',
 'LAMB4 (22798)', 'LAMC1 (3915)', 'LAMC2 (3918)', 'LAMC3 (10319)', 'LAMP1
 (3916)', 'LAMP2 (3920)', 'LAMP3 (27074)', 'LAMP5 (24141)', 'LAMTOR1 (55004)',
 'LAMTOR2 (28956)', 'LAMTOR3 (8649)', 'LAMTOR4 (389541)', 'LAMTOR5 (10542)',
 'LANCL1 (10314)', 'LANCL2 (55915)', 'LANCL3 (347404)', 'LAP3 (51056)', 'LAPTM4A
 (9741)', 'LAPTM4B (55353)', 'LAPTM5 (7805)', 'LARGE1 (9215)', 'LARGE2 (120071)',
 'LARP1 (23367)', 'LARP1B (55132)', 'LARP4 (113251)', 'LARP4B (23185)', 'LARP6
 (55323)', 'LARP7 (51574)', 'LARS1 (51520)', 'LARS2 (23395)', 'LAS1L (81887)',
 'LASP1 (3927)', 'LAT (27040)', 'LAT2 (7462)', 'LATS1 (9113)', 'LATS2 (26524)',
 'LAX1 (54900)', 'LAYN (143903)', 'LBH (81606)', 'LBHD1 (79081)', 'LBHD2
 (107984640)', 'LBP (3929)', 'LBR (3930)', 'LBX1 (10660)', 'LBX2 (85474)', 'LCA5
 (167691)', 'LCA5L (150082)', 'LCAT (3931)', 'LCE1B (353132)', 'LCE1D (353134)',
 'LCE1F (353137)', 'LCE2A (353139)', 'LCE2B (26239)', 'LCE2C (353140)', 'LCE2D
 (353141)', 'LCE3A (353142)', 'LCE3B (353143)', 'LCE3C (353144)', 'LCE3D
 (84648)', 'LCE3E (353145)', 'LCE4A (199834)', 'LCE5A (254910)', 'LCE6A
 (448835)', 'LCK (3932)', 'LCLAT1 (253558)', 'LCMT1 (51451)', 'LCMT2 (9836)',
 'LCN1 (3933)', 'LCN10 (414332)', 'LCN12 (286256)', 'LCN15 (389812)', 'LCN2
 (3934)', 'LCN6 (158062)', 'LCN8 (138307)', 'LCN9 (392399)', 'LCNL1 (401562)',
 'LCOR (84458)', 'LCORL (254251)', 'LCP1 (3936)', 'LCP2 (3937)', 'LCT (3938)',
 'LCTL (197021)', 'LDAF1 (57146)', 'LDAH (60526)', 'LDB1 (8861)', 'LDB2 (9079)',
 'LDB3 (11155)', 'LDHA (3939)', 'LDHAL6A (160287)', 'LDHAL6B (92483)', 'LDHB

(3945)', 'LDHC (3948)', 'LDHD (197257)', 'LDLR (3949)', 'LDLRAD1 (388633)',
 'LDLRAD2 (401944)', 'LDLRAD3 (143458)', 'LDLRAD4 (753)', 'LDLRAP1 (26119)',
 'LDOC1 (23641)', 'LEAP2 (116842)', 'LECT2 (3950)', 'LEF1 (51176)', 'LEFTY1
 (10637)', 'LEFTY2 (7044)', 'LEKR1 (389170)', 'LELP1 (149018)', 'LEMD1 (93273)',
 'LEMD2 (221496)', 'LEMD3 (23592)', 'LENEP (55891)', 'LENG1 (79165)', 'LENG8
 (114823)', 'LENG9 (94059)', 'LEO1 (123169)', 'LEP (3952)', 'LEPR (3953)',
 'LEPROT (54741)', 'LEPROTL1 (23484)', 'LETM1 (3954)', 'LETM2 (137994)', 'LETMD1
 (25875)', 'LEUTX (342900)', 'LFNG (3955)', 'LGALS1 (3956)', 'LGALS12 (85329)',
 'LGALS13 (29124)', 'LGALS14 (56891)', 'LGALS16 (148003)', 'LGALS2 (3957)',
 'LGALS3 (3958)', 'LGALS3BP (3959)', 'LGALS4 (3960)', 'LGALS7 (3963)', 'LGALS7B
 (653499)', 'LGALS8 (3964)', 'LGALS9 (3965)', 'LGALS9B (284194)', 'LGALS9C
 (654346)', 'LGALSL (29094)', 'LGI1 (9211)', 'LGI2 (55203)', 'LGI3 (203190)',
 'LGI4 (163175)', 'LGMN (5641)', 'LGR4 (55366)', 'LGR5 (8549)', 'LGR6 (59352)',
 'LGSN (51557)', 'LHB (3972)', 'LHCGR (3973)', 'LHFPL1 (340596)', 'LHFPL2
 (10184)', 'LHFPL3 (375612)', 'LHFPL4 (375323)', 'LHFPL5 (222662)', 'LHFPL6
 (10186)', 'LHFPL7 (255349)', 'LHPP (64077)', 'LHX1 (3975)', 'LHX2 (9355)', 'LHX3
 (8022)', 'LHX4 (89884)', 'LHX5 (64211)', 'LHX6 (26468)', 'LHX8 (431707)', 'LHX9
 (56956)', 'LIAS (11019)', 'LIAT1 (400566)', 'LIF (3976)', 'LIFR (3977)', 'LIG1
 (3978)', 'LIG3 (3980)', 'LIG4 (3981)', 'LILRA1 (11024)', 'LILRA2 (11027)',
 'LILRA4 (23547)', 'LILRA5 (353514)', 'LILRA6 (79168)', 'LILRB1 (10859)', 'LILRB2
 (10288)', 'LILRB3 (11025)', 'LILRB4 (11006)', 'LILRB5 (10990)', 'LIM2 (3982)',
 'LIMA1 (51474)', 'LIMCH1 (22998)', 'LIMD1 (8994)', 'LIMD2 (80774)', 'LIME1
 (54923)', 'LIMK1 (3984)', 'LIMK2 (3985)', 'LIMS1 (3987)', 'LIMS2 (55679)',
 'LIN28A (79727)', 'LIN28B (389421)', 'LIN37 (55957)', 'LIN52 (91750)', 'LIN54
 (132660)', 'LIN7A (8825)', 'LIN7B (64130)', 'LIN7C (55327)', 'LIN9 (286826)',
 'LINGO1 (84894)', 'LINGO2 (158038)', 'LINGO3 (645191)', 'LINGO4 (339398)',
 'LINS1 (55180)', 'LIPA (3988)', 'LIPC (3990)', 'LIPE (3991)', 'LIPF (8513)',
 'LIPG (9388)', 'LIPH (200879)', 'LIPI (149998)', 'LIPJ (142910)', 'LIPK
 (643414)', 'LIPM (340654)', 'LIPN (643418)', 'LIPT1 (51601)', 'LIPT2 (387787)',
 'LITAF (9516)', 'LIX1 (167410)', 'LIX1L (128077)', 'LKAAEAR1 (198437)', 'LLCFC1
 (135927)', 'LLGL1 (3996)', 'LLGL2 (3993)', 'LLPH (84298)', 'LMAN1 (3998)',
 'LMAN1L (79748)', 'LMAN2 (10960)', 'LMAN2L (81562)', 'LMBR1 (64327)', 'LMBR1L
 (55716)', 'LMBRD1 (55788)', 'LMBRD2 (92255)', 'LMCD1 (29995)', 'LMF1 (64788)',
 'LMF2 (91289)', 'LMNA (4000)', 'LMNB1 (4001)', 'LMNB2 (84823)', 'LMNTD1
 (160492)', 'LMNTD2 (256329)', 'LMO1 (4004)', 'LMO2 (4005)', 'LMO3 (55885)',
 'LMO4 (8543)', 'LMO7 (4008)', 'LMOD1 (25802)', 'LMOD2 (442721)', 'LMOD3
 (56203)', 'LMTK2 (22853)', 'LMTK3 (114783)', 'LMX1A (4009)', 'LMX1B (4010)',
 'LNP1 (348801)', 'LNPEP (4012)', 'LNPk (80856)', 'LNx1 (84708)', 'LNx2
 (222484)', 'LONP1 (9361)', 'LONP2 (83752)', 'LONRF1 (91694)', 'LONRF2 (164832)',
 'LONRF3 (79836)', 'LORICRIN (4014)', 'LOX (4015)', 'LOXHD1 (125336)', 'LOXL1
 (4016)', 'LOXL2 (4017)', 'LOXL3 (84695)', 'LOXL4 (84171)', 'LPA (4018)', 'LPAR1
 (1902)', 'LPAR3 (23566)', 'LPAR4 (2846)', 'LPAR5 (57121)', 'LPAR6 (10161)',
 'LPCAT1 (79888)', 'LPCAT2 (54947)', 'LPCAT3 (10162)', 'LPCAT4 (254531)', 'LPGAT1
 (9926)', 'LPIN1 (23175)', 'LPIN2 (9663)', 'LPIN3 (64900)', 'LPL (4023)', 'LPO
 (4025)', 'LPP (4026)', 'LPXN (9404)', 'LRAT (9227)', 'LRATD1 (151354)', 'LRATD2
 (157638)', 'LRBA (987)', 'LRCH1 (23143)', 'LRCH2 (57631)', 'LRCH3 (84859)',
 'LRCH4 (4034)', 'LRCOL1 (100507055)', 'LRFN1 (57622)', 'LRFN2 (57497)', 'LRFN3
 (79414)', 'LRFN4 (78999)', 'LRFN5 (145581)', 'LRG1 (116844)', 'LRGUK (136332)',

'LRIF1 (55791)', 'LRIG1 (26018)', 'LRIG2 (9860)', 'LRIG3 (121227)', 'LRIT1
 (26103)', 'LRIT2 (340745)', 'LRIT3 (345193)', 'LRMDA (83938)', 'LRP1 (4035)',
 'LRP10 (26020)', 'LRP11 (84918)', 'LRP12 (29967)', 'LRP1B (53353)', 'LRP2
 (4036)', 'LRP2BP (55805)', 'LRP3 (4037)', 'LRP4 (4038)', 'LRP5 (4041)', 'LRP6
 (4040)', 'LRP8 (7804)', 'LRPAP1 (4043)', 'LRPPRC (10128)', 'LRR1 (122769)',
 'LRRC1 (55227)', 'LRRC10 (376132)', 'LRRC10B (390205)', 'LRRC14 (9684)',
 'LRRC14B (389257)', 'LRRC15 (131578)', 'LRRC17 (10234)', 'LRRC18 (474354)',
 'LRRC19 (64922)', 'LRRC2 (79442)', 'LRRC20 (55222)', 'LRRC23 (10233)', 'LRRC24
 (441381)', 'LRRC25 (126364)', 'LRRC26 (389816)', 'LRRC27 (80313)', 'LRRC28
 (123355)', 'LRRC3 (81543)', 'LRRC30 (339291)', 'LRRC31 (79782)', 'LRRC32
 (2615)', 'LRRC34 (151827)', 'LRRC36 (55282)', 'LRRC37A2 (474170)', 'LRRC37A3
 (374819)', 'LRRC37B (114659)', 'LRRC38 (126755)', 'LRRC39 (127495)', 'LRRC3B
 (116135)', 'LRRC3C (100505591)', 'LRRC4 (64101)', 'LRRC40 (55631)', 'LRRC41
 (10489)', 'LRRC42 (115353)', 'LRRC43 (254050)', 'LRRC45 (201255)', 'LRRC46
 (90506)', 'LRRC47 (57470)', 'LRRC49 (54839)', 'LRRC4B (94030)', 'LRRC4C
 (57689)', 'LRRC51 (120356739)', 'LRRC52 (440699)', 'LRRC53 (105378803)', 'LRRC55
 (219527)', 'LRRC56 (115399)', 'LRRC57 (255252)', 'LRRC58 (116064)', 'LRRC59
 (55379)', 'LRRC61 (65999)', 'LRRC63 (220416)', 'LRRC66 (339977)', 'LRRC7
 (57554)', 'LRRC70 (100130733)', 'LRRC71 (149499)', 'LRRC72 (100506049)', 'LRRC73
 (221424)', 'LRRC74A (145497)', 'LRRC74B (400891)', 'LRRC75A (388341)', 'LRRC75B
 (388886)', 'LRRC8A (56262)', 'LRRC8B (23507)', 'LRRC8C (84230)', 'LRRC8D
 (55144)', 'LRRC8E (80131)', 'LRRC9 (341883)', 'LRRCC1 (85444)', 'LRRD1
 (401387)', 'LRRFIP1 (9208)', 'LRRFIP2 (9209)', 'LRRIQ1 (84125)', 'LRRIQ3
 (127255)', 'LRRIQ4 (344657)', 'LRRK1 (79705)', 'LRRK2 (120892)', 'LRRN1
 (57633)', 'LRRN2 (10446)', 'LRRN3 (54674)', 'LRRN4 (164312)', 'LRRN4CL
 (221091)', 'LRRTM1 (347730)', 'LRRTM2 (26045)', 'LRRTM3 (347731)', 'LRRTM4
 (80059)', 'LRSAM1 (90678)', 'LRTM1 (57408)', 'LRTM2 (654429)', 'LRWD1 (222229)',
 'LSAMP (4045)', 'LSG1 (55341)', 'LSM1 (27257)', 'LSM10 (84967)', 'LSM11
 (134353)', 'LSM12 (124801)', 'LSM14A (26065)', 'LSM14B (149986)', 'LSM2
 (57819)', 'LSM3 (27258)', 'LSM4 (25804)', 'LSM5 (23658)', 'LSM6 (11157)', 'LSM7
 (51690)', 'LSM8 (51691)', 'LSMEM1 (286006)', 'LSMEM2 (132228)', 'LSP1 (4046)',
 'LSR (51599)', 'LSS (4047)', 'LST1 (7940)', 'LTA (4049)', 'LTA4H (4048)', 'LTB
 (4050)', 'LTB4R (1241)', 'LTB4R2 (56413)', 'LTBP1 (4052)', 'LTBP2 (4053)',
 'LTBP3 (4054)', 'LTBP4 (8425)', 'LTBR (4055)', 'LTC4S (4056)', 'LTF (4057)',
 'LTK (4058)', 'LTN1 (26046)', 'LT01 (220064)', 'LTV1 (84946)', 'LUC7L (55692)',
 'LUC7L2 (51631)', 'LUC7L3 (51747)', 'LUM (4060)', 'LURAP1 (541468)', 'LURAP1L
 (286343)', 'LUZP1 (7798)', 'LUZP2 (338645)', 'LUZP4 (51213)', 'LVRN (206338)',
 'LXN (56925)', 'LY6D (8581)', 'LY6E (4061)', 'LY6G5B (58496)', 'LY6G5C (80741)',
 'LY6G6C (80740)', 'LY6H (4062)', 'LY6K (54742)', 'LY6L (101928108)', 'LY75
 (4065)', 'LY86 (9450)', 'LY9 (4063)', 'LY96 (23643)', 'LYAR (55646)', 'LYG1
 (129530)', 'LYG2 (254773)', 'LYL1 (4066)', 'LYN (4067)', 'LYNX1 (66004)', 'LYPD1
 (116372)', 'LYPD2 (137797)', 'LYPD3 (27076)', 'LYPD4 (147719)', 'LYPD5
 (284348)', 'LYPD6 (130574)', 'LYPD6B (130576)', 'LYPD8 (646627)', 'LYPLA1
 (10434)', 'LYPLA2 (11313)', 'LYPLAL1 (127018)', 'LYRM1 (57149)', 'LYRM2
 (57226)', 'LYRM4 (57128)', 'LYRM7 (90624)', 'LYRM9 (201229)', 'LYSET (26175)',
 'LYSMD1 (388695)', 'LYSMD2 (256586)', 'LYSMD3 (116068)', 'LYSMD4 (145748)',
 'LYST (1130)', 'LYVE1 (10894)', 'LYZ (4069)', 'LYZL4 (131375)', 'LYZL6 (57151)',
 'LZIC (84328)', 'LZTFL1 (54585)', 'LZTR1 (8216)', 'LZTS1 (11178)', 'LZTS2

(84445)', 'LZTS3 (9762)', 'M1AP (130951)', 'M6PR (4074)', 'MAB21L1 (4081)',
 'MAB21L2 (10586)', 'MAB21L3 (126868)', 'MAB21L4 (79919)', 'MACC1 (346389)',
 'MACF1 (23499)', 'MACIR (90355)', 'MACO1 (55219)', 'MACROD1 (28992)', 'MACROD2
 (140733)', 'MACROH2A1 (9555)', 'MACROH2A2 (55506)', 'MAD1L1 (8379)', 'MAD2L1
 (4085)', 'MAD2L1BP (9587)', 'MAD2L2 (10459)', 'MADCAM1 (8174)', 'MADD (8567)',
 'MAEA (10296)', 'MAEL (84944)', 'MAF (4094)', 'MAF1 (84232)', 'MAFA (389692)',
 'MAFB (9935)', 'MAFF (23764)', 'MAFG (4097)', 'MAFK (7975)', 'MAG (4099)',
 'MAGEA1 (4100)', 'MAGEA10 (4109)', 'MAGEA11 (4110)', 'MAGEA12 (4111)', 'MAGEA4
 (4103)', 'MAGEA6 (4105)', 'MAGEA8 (4107)', 'MAGEB1 (4112)', 'MAGEB10 (139422)',
 'MAGEB16 (139604)', 'MAGEB17 (645864)', 'MAGEB18 (286514)', 'MAGEB2 (4113)',
 'MAGEB3 (4114)', 'MAGEB4 (4115)', 'MAGEB5 (347541)', 'MAGEB6 (158809)', 'MAGEC1
 (9947)', 'MAGEC2 (51438)', 'MAGEC3 (139081)', 'MAGED1 (9500)', 'MAGED2 (10916)',
 'MAGEE1 (57692)', 'MAGEE2 (139599)', 'MAGEF1 (64110)', 'MAGEH1 (28986)', 'MAGEL2
 (54551)', 'MAGI1 (9223)', 'MAGI2 (9863)', 'MAGI3 (260425)', 'MAGIX (79917)',
 'MAGOH (4116)', 'MAGOH2 (55110)', 'MAGT1 (84061)', 'MAIP1 (79568)', 'MAJIN
 (283129)', 'MAK (4117)', 'MAK16 (84549)', 'MAL (4118)', 'MAL2 (114569)', 'MALL
 (7851)', 'MALRD1 (340895)', 'MALSU1 (115416)', 'MALT1 (10892)', 'MAMDC2
 (256691)', 'MAMDC4 (158056)', 'MAML1 (9794)', 'MAML2 (84441)', 'MAML3 (55534)',
 'MAMLD1 (10046)', 'MAMSTR (284358)', 'MAN1A1 (4121)', 'MAN1A2 (10905)', 'MAN1B1
 (11253)', 'MAN1C1 (57134)', 'MAN2A1 (4124)', 'MAN2A2 (4122)', 'MAN2B1 (4125)',
 'MAN2B2 (23324)', 'MAN2C1 (4123)', 'MANBA (4126)', 'MANBAL (63905)', 'MANEA
 (79694)', 'MANEAL (149175)', 'MANF (7873)', 'MANSC1 (54682)', 'MANSC4
 (100287284)', 'MAOA (4128)', 'MAOB (4129)', 'MAP10 (54627)', 'MAP1A (4130)',
 'MAP1B (4131)', 'MAP1LC3A (84557)', 'MAP1LC3B (81631)', 'MAP1LC3B2 (643246)',
 'MAP1LC3C (440738)', 'MAP1S (55201)', 'MAP2 (4133)', 'MAP2K1 (5604)', 'MAP2K2
 (5605)', 'MAP2K3 (5606)', 'MAP2K4 (6416)', 'MAP2K5 (5607)', 'MAP2K6 (5608)',
 'MAP2K7 (5609)', 'MAP3K1 (4214)', 'MAP3K10 (4294)', 'MAP3K11 (4296)', 'MAP3K12
 (7786)', 'MAP3K13 (9175)', 'MAP3K14 (9020)', 'MAP3K15 (389840)', 'MAP3K19
 (80122)', 'MAP3K2 (10746)', 'MAP3K20 (51776)', 'MAP3K21 (84451)', 'MAP3K3
 (4215)', 'MAP3K4 (4216)', 'MAP3K5 (4217)', 'MAP3K6 (9064)', 'MAP3K7 (6885)',
 'MAP3K7CL (56911)', 'MAP3K8 (1326)', 'MAP3K9 (4293)', 'MAP4 (4134)', 'MAP4K1
 (11184)', 'MAP4K2 (5871)', 'MAP4K3 (8491)', 'MAP4K4 (9448)', 'MAP4K5 (11183)',
 'MAP6 (4135)', 'MAP6D1 (79929)', 'MAP7 (9053)', 'MAP7D1 (55700)', 'MAP7D2
 (256714)', 'MAP7D3 (79649)', 'MAP9 (79884)', 'MAPDA (161823)', 'MAPK1 (5594)',
 'MAPK10 (5602)', 'MAPK11 (5600)', 'MAPK12 (6300)', 'MAPK13 (5603)', 'MAPK14
 (1432)', 'MAPK15 (225689)', 'MAPK1IP1L (93487)', 'MAPK3 (5595)', 'MAPK4 (5596)',
 'MAPK6 (5597)', 'MAPK7 (5598)', 'MAPK8 (5599)', 'MAPK8IP1 (9479)', 'MAPK8IP2
 (23542)', 'MAPK8IP3 (23162)', 'MAPK9 (5601)', 'MAPKAP1 (79109)', 'MAPKAPK2
 (9261)', 'MAPKAPK3 (7867)', 'MAPKAPK5 (8550)', 'MAPKBP1 (23005)', 'MAPRE1
 (22919)', 'MAPRE2 (10982)', 'MAPRE3 (22924)', 'MAPT (4137)', 'MARCF1 (55016)',
 'MARCF10 (162333)', 'MARCF11 (441061)', 'MARCF2 (51257)', 'MARCF3 (115123)',
 'MARCF4 (57574)', 'MARCF5 (54708)', 'MARCF6 (10299)', 'MARCF7 (64844)',
 'MARCF8 (220972)', 'MARCF9 (92979)', 'MARCKS (4082)', 'MARCKSL1 (65108)',
 'MARCO (8685)', 'MARCOL (105378220)', 'MARF1 (9665)', 'MARK1 (4139)', 'MARK2
 (2011)', 'MARK3 (4140)', 'MARK4 (57787)', 'MARS1 (4141)', 'MARS2 (92935)',
 'MARVELD2 (153562)', 'MARVELD3 (91862)', 'MAS1 (4142)', 'MAS1L (116511)', 'MASP1
 (5648)', 'MASP2 (10747)', 'MAST1 (22983)', 'MAST2 (23139)', 'MAST3 (23031)',
 'MAST4 (375449)', 'MASTL (84930)', 'MAT1A (4143)', 'MAT2A (4144)', 'MAT2B

(27430)', 'MATCAP1 (653319)', 'MATCAP2 (23366)', 'MATK (4145)', 'MATN1 (4146)',
 'MATN2 (4147)', 'MATN3 (4148)', 'MATN4 (8785)', 'MATR3 (9782)', 'MAU2 (23383)',
 'MAVS (57506)', 'MAX (4149)', 'MAZ (4150)', 'MB (4151)', 'MB21D2 (151963)',
 'MBD1 (4152)', 'MBD2 (8932)', 'MBD3 (53615)', 'MBD3L1 (85509)', 'MBD4 (8930)',
 'MBD5 (55777)', 'MBD6 (114785)', 'MBIP (51562)', 'MBL2 (4153)', 'MBLAC1
 (255374)', 'MBLAC2 (153364)', 'MBNL1 (4154)', 'MBNL2 (10150)', 'MBNL3 (55796)',
 'MBOAT1 (154141)', 'MBOAT2 (129642)', 'MBOAT4 (619373)', 'MBOAT7 (79143)', 'MBP
 (4155)', 'MBTD1 (54799)', 'MBTPS1 (8720)', 'MBTPS2 (51360)', 'MC1R (4157)',
 'MC2R (4158)', 'MC3R (4159)', 'MC4R (4160)', 'MC5R (4161)', 'MCAM (4162)', 'MCAT
 (27349)', 'MCC (4163)', 'MCCC1 (56922)', 'MCCC2 (64087)', 'MCCD1 (401250)',
 'MCEE (84693)', 'MCEMP1 (199675)', 'MCF2 (4168)', 'MCF2L (23263)', 'MCF2L2
 (23101)', 'MCFD2 (90411)', 'MCHR1 (2847)', 'MCHR2 (84539)', 'MCIDAS (345643)',
 'MCL1 (4170)', 'MCM10 (55388)', 'MCM2 (4171)', 'MCM3 (4172)', 'MCM3AP (8888)',
 'MCM4 (4173)', 'MCM5 (4174)', 'MCM6 (4175)', 'MCM7 (4176)', 'MCM8 (84515)',
 'MCM9 (254394)', 'MCMBP (79892)', 'MCMDC2 (157777)', 'MCOLN1 (57192)', 'MCOLN2
 (255231)', 'MCOLN3 (55283)', 'MCPH1 (79648)', 'MCRIP1 (348262)', 'MCRIP2
 (84331)', 'MCRS1 (10445)', 'MCTP1 (79772)', 'MCTP2 (55784)', 'MCTS1 (28985)',
 'MCU (90550)', 'MCUB (55013)', 'MCUR1 (63933)', 'MDC1 (9656)', 'MDFI (4188)',
 'MDFIC (29969)', 'MDFIC2 (107986096)', 'MDGA1 (266727)', 'MDGA2 (161357)', 'MDH1
 (4190)', 'MDH1B (130752)', 'MDH2 (4191)', 'MDK (4192)', 'MDM1 (56890)', 'MDM2
 (4193)', 'MDM4 (4194)', 'MDN1 (23195)', 'MDP1 (145553)', 'ME1 (4199)', 'ME2
 (4200)', 'ME3 (10873)', 'MEA1 (4201)', 'MEAF6 (64769)', 'MEAK7 (57707)', 'MECOM
 (2122)', 'MECP2 (4204)', 'MECR (51102)', 'MED1 (5469)', 'MED10 (84246)', 'MED11
 (400569)', 'MED12 (9968)', 'MED12L (116931)', 'MED13 (9969)', 'MED13L (23389)',
 'MED14 (9282)', 'MED15 (51586)', 'MED16 (10025)', 'MED17 (9440)', 'MED18
 (54797)', 'MED19 (219541)', 'MED20 (9477)', 'MED21 (9412)', 'MED22 (6837)',
 'MED23 (9439)', 'MED24 (9862)', 'MED25 (81857)', 'MED26 (9441)', 'MED27 (9442)',
 'MED28 (80306)', 'MED29 (55588)', 'MED30 (90390)', 'MED31 (51003)', 'MED4
 (29079)', 'MED6 (10001)', 'MED7 (9443)', 'MED8 (112950)', 'MED9 (55090)', 'MEDAG
 (84935)', 'MEF2A (4205)', 'MEF2B (100271849)', 'MEF2C (4208)', 'MEF2D (4209)',
 'MEFV (4210)', 'MEGF10 (84466)', 'MEGF11 (84465)', 'MEGF6 (1953)', 'MEGF8
 (1954)', 'MEGF9 (1955)', 'MEI1 (150365)', 'MEI4 (101928601)', 'MEIG1 (644890)',
 'MEIKIN (728637)', 'MEIOB (254528)', 'MEIOC (284071)', 'MEIOSIN (388553)',
 'MEIS1 (4211)', 'MEIS2 (4212)', 'MEIS3 (56917)', 'MELK (9833)', 'MELTF (4241)',
 'MEMO1 (51072)', 'MEN1 (4221)', 'MEOX1 (4222)', 'MEOX2 (4223)', 'MEP1A (4224)',
 'MEP1B (4225)', 'MEPCE (56257)', 'MEPE (56955)', 'MERTK (10461)', 'MESD
 (23184)', 'MESP1 (55897)', 'MESP2 (145873)', 'MEST (4232)', 'MET (4233)',
 'METAP1 (23173)', 'METAP1D (254042)', 'METAP2 (10988)', 'METRN (79006)', 'METRNL
 (284207)', 'METTL1 (4234)', 'METTL13 (51603)', 'METTL14 (57721)', 'METTL15
 (196074)', 'METTL16 (79066)', 'METTL17 (64745)', 'METTL18 (92342)', 'METTL21A
 (151194)', 'METTL21C (196541)', 'METTL22 (79091)', 'METTL23 (124512)', 'METTL24
 (728464)', 'METTL25 (84190)', 'METTL25B (51093)', 'METTL26 (84326)', 'METTL27
 (155368)', 'METTL2A (339175)', 'METTL2B (55798)', 'METTL3 (56339)', 'METTL4
 (64863)', 'METTL5 (29081)', 'METTL6 (131965)', 'METTL8 (79828)', 'METTL9
 (51108)', 'MEX3A (92312)', 'MEX3B (84206)', 'MEX3C (51320)', 'MEX3D (399664)',
 'MFAP1 (4236)', 'MFAP2 (4237)', 'MFAP3 (4238)', 'MFAP3L (9848)', 'MFAP4 (4239)',
 'MFAP5 (8076)', 'MFF (56947)', 'MFGE8 (4240)', 'MFHAS1 (9258)', 'MFN1 (55669)',
 'MFN2 (9927)', 'MFNG (4242)', 'MFRP (83552)', 'MFSD1 (64747)', 'MFSD10 (10227)',

'MFSD11 (79157)', 'MFSD12 (126321)', 'MFSD13A (79847)', 'MFSD14A (64645)',
'MFSD14B (84641)', 'MFSD2A (84879)', 'MFSD2B (388931)', 'MFSD3 (113655)',
'MFSD4A (148808)', 'MFSD4B (91749)', 'MFSD5 (84975)', 'MFSD6 (54842)', 'MFSD6L
(162387)', 'MFSD8 (256471)', 'MFSD9 (84804)', 'MGA (23269)', 'MGAM (8972)',
'MGAM2 (93432)', 'MGARP (84709)', 'MGAT1 (4245)', 'MGAT2 (4247)', 'MGAT3
(4248)', 'MGAT4A (11320)', 'MGAT4B (11282)', 'MGAT4C (25834)', 'MGAT4D
(152586)', 'MGAT5 (4249)', 'MGAT5B (146664)', 'MGLL (11343)', 'MGME1 (92667)',
'MGMT (4255)', 'MGP (4256)', 'MGRN1 (23295)', 'MGST1 (4257)', 'MGST2 (4258)',
'MGST3 (4259)', 'MIA (8190)', 'MIA2 (4253)', 'MIA3 (375056)', 'MIB1 (57534)',
'MIB2 (142678)', 'MICA (100507436)', 'MICAL1 (64780)', 'MICAL2 (9645)', 'MICAL3
(57553)', 'MICALL1 (85377)', 'MICALL2 (79778)', 'MICB (4277)', 'MICOS10
(440574)', 'MICOS13 (125988)', 'MICU1 (10367)', 'MICU2 (221154)', 'MICU3
(286097)', 'MID1 (4281)', 'MID1IP1 (58526)', 'MID2 (11043)', 'MIDEAS (91748)',
'MIDN (90007)', 'MIEF1 (54471)', 'MIEF2 (125170)', 'MIEN1 (84299)', 'MIER1
(57708)', 'MIER2 (54531)', 'MIER3 (166968)', 'MIF (4282)', 'MIF4GD (57409)',
'MIGA1 (374986)', 'MIGA2 (84895)', 'MIIP (60672)', 'MILR1 (284021)', 'MINAR1
(23251)', 'MINAR2 (100127206)', 'MINDY1 (55793)', 'MINDY2 (54629)', 'MINDY3
(80013)', 'MINDY4 (84182)', 'MINDY4B (646951)', 'MINK1 (50488)', 'MINPP1
(9562)', 'MIOS (54468)', 'MIOX (55586)', 'MIP (4284)', 'MIPPEP (4285)', 'MIPOL1
(145282)', 'MIS12 (79003)', 'MIS18A (54069)', 'MIS18BP1 (55320)', 'MISFA
(100506540)', 'MISP (126353)', 'MISP3 (113230)', 'MITD1 (129531)', 'MITF
(4286)', 'MIX23 (131076)', 'MIXL1 (83881)', 'MKI67 (4288)', 'MKKS (8195)',
'MKLN1 (4289)', 'MKNK1 (8569)', 'MKNK2 (2872)', 'MKRN1 (23608)', 'MKRN2
(23609)', 'MKRN2OS (100129480)', 'MKRN3 (7681)', 'MKS1 (54903)', 'MKX (283078)',
'MLANA (2315)', 'MLC1 (23209)', 'MLEC (9761)', 'MLF1 (4291)', 'MLF2 (8079)',
'MLH1 (4292)', 'MLH3 (27030)', 'MLIP (90523)', 'MLKL (197259)', 'MLLT1 (4298)',
'MLLT10 (8028)', 'MLLT11 (10962)', 'MLLT3 (4300)', 'MLLT6 (4302)', 'MLN (4295)',
'MLNR (2862)', 'MLPH (79083)', 'MLST8 (64223)', 'MLX (6945)', 'MLXIP (22877)',
'MLXIPL (51085)', 'MLYCD (23417)', 'MMAA (166785)', 'MMAB (326625)', 'MMACHC
(25974)', 'MMADHC (27249)', 'MMD (23531)', 'MMD2 (221938)', 'MME (4311)', 'MMEL1
(79258)', 'MMGT1 (93380)', 'MMP1 (4312)', 'MMP10 (4319)', 'MMP11 (4320)', 'MMP12
(4321)', 'MMP13 (4322)', 'MMP14 (4323)', 'MMP15 (4324)', 'MMP16 (4325)', 'MMP17
(4326)', 'MMP19 (4327)', 'MMP2 (4313)', 'MMP20 (9313)', 'MMP21 (118856)',
'MMP23B (8510)', 'MMP24 (10893)', 'MMP24OS (101410538)', 'MMP25 (64386)', 'MMP26
(56547)', 'MMP27 (64066)', 'MMP28 (79148)', 'MMP3 (4314)', 'MMP7 (4316)', 'MMP8
(4317)', 'MMP9 (4318)', 'MMRN1 (22915)', 'MMRN2 (79812)', 'MMS19 (64210)',
'MMS22L (253714)', 'MMUT (4594)', 'MN1 (4330)', 'MNAT1 (4331)', 'MND1 (84057)',
'MNDA (4332)', 'MNS1 (55329)', 'MNT (4335)', 'MNX1 (3110)', 'MOAP1 (64112)',
'MOB1A (55233)', 'MOB1B (92597)', 'MOB2 (81532)', 'MOB3A (126308)', 'MOB3B
(79817)', 'MOB3C (148932)', 'MOB4 (25843)', 'MOBP (4336)', 'MOCOS (55034)',
'MOCS1 (4337)', 'MOCS2 (4338)', 'MOCS3 (27304)', 'MOG (4340)', 'MOGAT1
(116255)', 'MOGAT2 (80168)', 'MOGAT3 (346606)', 'MOGS (7841)', 'MOK (5891)',
'MON1A (84315)', 'MON1B (22879)', 'MON2 (23041)', 'MORC1 (27136)', 'MORC2
(22880)', 'MORC3 (23515)', 'MORC4 (79710)', 'MORF4L1 (10933)', 'MORF4L2 (9643)',
'MORN1 (79906)', 'MORN2 (729967)', 'MORN3 (283385)', 'MORN4 (118812)', 'MORN5
(254956)', 'MOS (4342)', 'MOSMO (730094)', 'MOSPD1 (56180)', 'MOSPD2 (158747)',
'MOSPD3 (64598)', 'MOV10 (4343)', 'MOV10L1 (54456)', 'MOXD1 (26002)', 'MPC1
(51660)', 'MPC1L (347411)', 'MPC2 (25874)', 'MPDU1 (9526)', 'MPDZ (8777)',

'MPEG1 (219972)', 'MPG (4350)', 'MPHOSPH10 (10199)', 'MPHOSPH6 (10200)',
 'MPHOSPH8 (54737)', 'MPHOSPH9 (10198)', 'MPI (4351)', 'MPIG6B (80739)', 'MPL
 (4352)', 'MPLKIP (136647)', 'MPND (84954)', 'MPO (4353)', 'MPP1 (4354)', 'MPP2
 (4355)', 'MPP3 (4356)', 'MPP4 (58538)', 'MPP7 (143098)', 'MPPE1 (65258)',
 'MPPED1 (758)', 'MPPED2 (744)', 'MPRIIP (23164)', 'MPST (4357)', 'MPV17 (4358)',
 'MPV17L (255027)', 'MPV17L2 (84769)', 'MPZ (4359)', 'MPZL1 (9019)', 'MPZL2
 (10205)', 'MPZL3 (196264)', 'MR1 (3140)', 'MRAP (56246)', 'MRAP2 (112609)',
 'MRAS (22808)', 'MRC1 (4360)', 'MRC2 (9902)', 'MRE11 (4361)', 'MREG (55686)',
 'MRFAP1 (93621)', 'MRFAP1L1 (114932)', 'MRGBP (55257)', 'MRGPRD (116512)',
 'MRGPRE (116534)', 'MRGPRF (116535)', 'MRGPRG (386746)', 'MRGPRX1 (259249)',
 'MRGPRX2 (117194)', 'MRGPRX3 (117195)', 'MRGPRX4 (117196)', 'MRI1 (84245)',
 'MRLN (100507027)', 'MRM1 (79922)', 'MRM2 (29960)', 'MRM3 (55178)', 'MRNIP
 (51149)', 'MRO (83876)', 'MROH1 (727957)', 'MROH2A (339766)', 'MROH2B (133558)',
 'MROH6 (642475)', 'MROH7 (374977)', 'MROH9 (80133)', 'MRPL1 (65008)', 'MRPL10
 (124995)', 'MRPL11 (65003)', 'MRPL12 (6182)', 'MRPL13 (28998)', 'MRPL14
 (64928)', 'MRPL15 (29088)', 'MRPL16 (54948)', 'MRPL17 (63875)', 'MRPL18
 (29074)', 'MRPL19 (9801)', 'MRPL2 (51069)', 'MRPL20 (55052)', 'MRPL21 (219927)',
 'MRPL22 (29093)', 'MRPL23 (6150)', 'MRPL24 (79590)', 'MRPL27 (51264)', 'MRPL28
 (10573)', 'MRPL3 (11222)', 'MRPL30 (51263)', 'MRPL32 (64983)', 'MRPL33 (9553)',
 'MRPL34 (64981)', 'MRPL35 (51318)', 'MRPL36 (64979)', 'MRPL37 (51253)', 'MRPL38
 (64978)', 'MRPL39 (54148)', 'MRPL4 (51073)', 'MRPL40 (64976)', 'MRPL41 (64975)',
 'MRPL42 (28977)', 'MRPL43 (84545)', 'MRPL44 (65080)', 'MRPL45 (84311)', 'MRPL46
 (26589)', 'MRPL47 (57129)', 'MRPL48 (51642)', 'MRPL49 (740)', 'MRPL50 (54534)',
 'MRPL51 (51258)', 'MRPL52 (122704)', 'MRPL53 (116540)', 'MRPL54 (116541)',
 'MRPL55 (128308)', 'MRPL57 (78988)', 'MRPL58 (3396)', 'MRPL9 (65005)', 'MRPS10
 (55173)', 'MRPS11 (64963)', 'MRPS12 (6183)', 'MRPS14 (63931)', 'MRPS15 (64960)',
 'MRPS16 (51021)', 'MRPS17 (51373)', 'MRPS18A (55168)', 'MRPS18B (28973)',
 'MRPS18C (51023)', 'MRPS2 (51116)', 'MRPS21 (54460)', 'MRPS22 (56945)', 'MRPS23
 (51649)', 'MRPS24 (64951)', 'MRPS25 (64432)', 'MRPS26 (64949)', 'MRPS27
 (23107)', 'MRPS28 (28957)', 'MRPS30 (10884)', 'MRPS31 (10240)', 'MRPS33
 (51650)', 'MRPS34 (65993)', 'MRPS35 (60488)', 'MRPS5 (64969)', 'MRPS6 (64968)',
 'MRPS7 (51081)', 'MRPS9 (64965)', 'MRRF (92399)', 'MRS2 (57380)', 'MRTFA
 (57591)', 'MRTFB (57496)', 'MRT04 (51154)', 'MS4A1 (931)', 'MS4A10 (341116)',
 'MS4A12 (54860)', 'MS4A13 (503497)', 'MS4A14 (84689)', 'MS4A15 (219995)', 'MS4A2
 (2206)', 'MS4A3 (932)', 'MS4A4A (51338)', 'MS4A4E (643680)', 'MS4A5 (64232)',
 'MS4A6A (64231)', 'MS4A6E (245802)', 'MS4A7 (58475)', 'MS4A8 (83661)', 'MSANTD1
 (345222)', 'MSANTD2 (79684)', 'MSANTD3 (91283)', 'MSANTD4 (84437)', 'MSANTD5
 (102724657)', 'MSC (9242)', 'MSGN1 (343930)', 'MSH2 (4436)', 'MSH3 (4437)',
 'MSH4 (4438)', 'MSH5 (4439)', 'MSH6 (2956)', 'MSI1 (4440)', 'MSI2 (124540)',
 'MSL1 (339287)', 'MSL2 (55167)', 'MSL3 (10943)', 'MSLN (10232)', 'MSMB (4477)',
 'MSMO1 (6307)', 'MSMP (692094)', 'MSN (4478)', 'MSR1 (4481)', 'MSRA (4482)',
 'MSRB1 (51734)', 'MSRB2 (22921)', 'MSRB3 (253827)', 'MSS51 (118490)', 'MST1
 (4485)', 'MST1R (4486)', 'MSTN (2660)', 'MSTO1 (55154)', 'MSX1 (4487)', 'MSX2
 (4488)', 'MT1A (4489)', 'MT1B (4490)', 'MT1E (4493)', 'MT1F (4494)', 'MT1G
 (4495)', 'MT1H (4496)', 'MT1HL1 (645745)', 'MT1M (4499)', 'MT1X (4501)', 'MT2A
 (4502)', 'MT3 (4504)', 'MT4 (84560)', 'MTA1 (9112)', 'MTA2 (9219)', 'MTA3
 (57504)', 'MTAP (4507)', 'MTARC1 (64757)', 'MTARC2 (54996)', 'MTBP (27085)',
 'MTCH1 (23787)', 'MTCH2 (23788)', 'MTCL1 (23255)', 'MTCL2 (140710)', 'MTCP1

(4515)', 'MTDH (92140)', 'MTERF1 (7978)', 'MTERF2 (80298)', 'MTERF3 (51001)',
 'MTERF4 (130916)', 'MTF1 (4520)', 'MTF2 (22823)', 'MTFMT (123263)', 'MTFP1
 (51537)', 'MTFR1 (9650)', 'MTFR1L (56181)', 'MTFR2 (113115)', 'MTG1 (92170)',
 'MTG2 (26164)', 'MTHFD1 (4522)', 'MTHFD1L (25902)', 'MTHFD2 (10797)', 'MTHFD2L
 (441024)', 'MTHFR (4524)', 'MTHFS (10588)', 'MTHFSD (64779)', 'MTIF2 (4528)',
 'MTIF3 (219402)', 'MTM1 (4534)', 'MTMR1 (8776)', 'MTMR10 (54893)', 'MTMR11
 (10903)', 'MTMR12 (54545)', 'MTMR14 (64419)', 'MTMR2 (8898)', 'MTMR3 (8897)',
 'MTMR4 (9110)', 'MTMR6 (9107)', 'MTMR7 (9108)', 'MTMR8 (55613)', 'MTMR9
 (66036)', 'MTNAP1 (55028)', 'MTNR1A (4543)', 'MTNR1B (4544)', 'MT01 (25821)',
 'MTOR (2475)', 'MTPAP (55149)', 'MTPN (136319)', 'MTR (4548)', 'MTRES1 (51250)',
 'MTREX (23517)', 'MTRF1 (9617)', 'MTRF1L (54516)', 'MTRFR (91574)', 'MTRR
 (4552)', 'MTSS1 (9788)', 'MTSS2 (92154)', 'MTTP (4547)', 'MTURN (222166)',
 'MTUS1 (57509)', 'MTUS2 (23281)', 'MTX1 (4580)', 'MTX2 (10651)', 'MTX3
 (345778)', 'MUC1 (4582)', 'MUC12 (10071)', 'MUC13 (56667)', 'MUC15 (143662)',
 'MUC16 (94025)', 'MUC17 (140453)', 'MUC20 (200958)', 'MUC21 (394263)', 'MUC22
 (100507679)', 'MUC3A (4584)', 'MUC4 (4585)', 'MUC5AC (4586)', 'MUC5B (727897)',
 'MUC6 (4588)', 'MUC7 (4589)', 'MUCL1 (118430)', 'MUCL3 (135656)', 'MUL1
 (79594)', 'MUS81 (80198)', 'MUSK (4593)', 'MUSTN1 (389125)', 'MUTYH (4595)',
 'MVB12A (93343)', 'MVB12B (89853)', 'MVD (4597)', 'MVK (4598)', 'MVP (9961)',
 'MX1 (4599)', 'MX2 (4600)', 'MXD1 (4084)', 'MXD3 (83463)', 'MXD4 (10608)', 'MXI1
 (4601)', 'MXRA5 (25878)', 'MXRA7 (439921)', 'MXRA8 (54587)', 'MYADM (91663)',
 'MYADML2 (255275)', 'MYB (4602)', 'MYBBP1A (10514)', 'MYBL1 (4603)', 'MYBL2
 (4605)', 'MYBPC1 (4604)', 'MYBPC2 (4606)', 'MYBPC3 (4607)', 'MYBPH (4608)',
 'MYBPHL (343263)', 'MYC (4609)', 'MYCBP (26292)', 'MYCBPAP (84073)', 'MYCL
 (4610)', 'MYCN (4613)', 'MYCT1 (80177)', 'MYD88 (4615)', 'MYDGF (56005)', 'MYEF2
 (50804)', 'MYEOV (26579)', 'MYF5 (4617)', 'MYF6 (4618)', 'MYG1 (60314)', 'MYH1
 (4619)', 'MYH10 (4628)', 'MYH11 (4629)', 'MYH13 (8735)', 'MYH14 (79784)', 'MYH15
 (22989)', 'MYH2 (4620)', 'MYH3 (4621)', 'MYH4 (4622)', 'MYH6 (4624)', 'MYH7
 (4625)', 'MYH8 (4626)', 'MYH9 (4627)', 'MYL1 (4632)', 'MYL10 (93408)', 'MYL11
 (29895)', 'MYL12A (10627)', 'MYL12B (103910)', 'MYL2 (4633)', 'MYL3 (4634)',
 'MYL4 (4635)', 'MYL5 (4636)', 'MYL6 (4637)', 'MYL6B (140465)', 'MYL7 (58498)',
 'MYL9 (10398)', 'MYLIP (29116)', 'MYLK (4638)', 'MYLK2 (85366)', 'MYLK3
 (91807)', 'MYLK4 (340156)', 'MYMK (389827)', 'MYMX (101929726)', 'MYNN (55892)',
 'MYO10 (4651)', 'MYO15A (51168)', 'MYO15B (80022)', 'MYO16 (23026)', 'MYO18A
 (399687)', 'MYO18B (84700)', 'MYO19 (80179)', 'MYO1A (4640)', 'MYO1B (4430)',
 'MYO1C (4641)', 'MYO1D (4642)', 'MYO1E (4643)', 'MYO1F (4542)', 'MYO1G (64005)',
 'MYO1H (283446)', 'MYO3A (53904)', 'MYO3B (140469)', 'MYO5A (4644)', 'MYO5B
 (4645)', 'MYO5C (55930)', 'MYO6 (4646)', 'MYO7A (4647)', 'MYO7B (4648)', 'MYO9A
 (4649)', 'MYO9B (4650)', 'MYOC (4653)', 'MYOCD (93649)', 'MYOCOS (110806290)',
 'MYOD1 (4654)', 'MYOF (26509)', 'MYOG (4656)', 'MYOM1 (8736)', 'MYOM2 (9172)',
 'MYOM3 (127294)', 'MYORG (57462)', 'MYOT (9499)', 'MYOZ1 (58529)', 'MYOZ2
 (51778)', 'MYOZ3 (91977)', 'MYPN (84665)', 'MYPPOP (339344)', 'MYRF (745)',
 'MYRFL (196446)', 'MYRIP (25924)', 'MYSM1 (114803)', 'MYT1 (4661)', 'MYT1L
 (23040)', 'MYZAP (100820829)', 'MZB1 (51237)', 'MZF1 (7593)', 'MZT1 (440145)',
 'MZT2A (653784)', 'MZT2B (80097)', 'N4BP1 (9683)', 'N4BP2 (55728)', 'N4BP2L1
 (90634)', 'N4BP2L2 (10443)', 'N4BP3 (23138)', 'NAA10 (8260)', 'NAA11 (84779)',
 'NAA15 (80155)', 'NAA16 (79612)', 'NAA20 (51126)', 'NAA25 (80018)', 'NAA30
 (122830)', 'NAA35 (60560)', 'NAA38 (84316)', 'NAA40 (79829)', 'NAA50 (80218)',

'NAA60 (79903)', 'NAA80 (24142)', 'NAAA (27163)', 'NAALAD2 (10003)', 'NAALADL1 (10004)', 'NAALADL2 (254827)', 'NAB1 (4664)', 'NAB2 (4665)', 'NABP1 (64859)', 'NABP2 (79035)', 'NACA (4666)', 'NACA2 (342538)', 'NACAD (23148)', 'NACC1 (112939)', 'NACC2 (138151)', 'NADK (65220)', 'NADK2 (133686)', 'NADSYN1 (55191)', 'NAE1 (8883)', 'NAF1 (92345)', 'NAGA (4668)', 'NAGK (55577)', 'NAGLU (4669)', 'NAGPA (51172)', 'NAGS (162417)', 'NAIF1 (203245)', 'NAIP (4671)', 'NALCN (259232)', 'NALF1 (728215)', 'NALF2 (27112)', 'NAMPT (10135)', 'NANOG (79923)', 'NANOGNB (360030)', 'NANOS1 (340719)', 'NANOS2 (339345)', 'NANOS3 (342977)', 'NANP (140838)', 'NANS (54187)', 'NAP1L1 (4673)', 'NAP1L2 (4674)', 'NAP1L3 (4675)', 'NAP1L4 (4676)', 'NAP1L5 (266812)', 'NAPA (8775)', 'NAPB (63908)', 'NAPEPLD (222236)', 'NAPG (8774)', 'NAPRT (93100)', 'NAPSA (9476)', 'NARF (26502)', 'NARS1 (4677)', 'NARS2 (79731)', 'NASP (4678)', 'NAT1 (9)', 'NAT10 (55226)', 'NAT14 (57106)', 'NAT16 (375607)', 'NAT2 (10)', 'NAT8 (9027)', 'NAT8L (339983)', 'NAT9 (26151)', 'NAV1 (89796)', 'NAV2 (89797)', 'NAV3 (89795)', 'NAXD (55739)', 'NAXE (128240)', 'NBAS (51594)', 'NBDY (550643)', 'NBEA (26960)', 'NBEAL1 (65065)', 'NBEAL2 (23218)', 'NBL1 (4681)', 'NBN (4683)', 'NBPF3 (84224)', 'NBR1 (4077)', 'NCALD (83988)', 'NCAM1 (4684)', 'NCAM2 (4685)', 'NCAN (1463)', 'NCAPD2 (9918)', 'NCAPD3 (23310)', 'NCAPG (64151)', 'NCAPG2 (54892)', 'NCAPH (23397)', 'NCAPH2 (29781)', 'NCBP1 (4686)', 'NCBP2 (22916)', 'NCBP2AS2 (152217)', 'NCBP2L (392517)', 'NCBP3 (55421)', 'NCCRP1 (342897)', 'NCDN (23154)', 'NCEH1 (57552)', 'NCF1 (653361)', 'NCF2 (4688)', 'NCF4 (4689)', 'NCK1 (4690)', 'NCK2 (8440)', 'NCKAP1 (10787)', 'NCKAP1L (3071)', 'NCKAP5 (344148)', 'NCKAP5L (57701)', 'NCKIPSD (51517)', 'NCL (4691)', 'NCLN (56926)', 'NCMAP (400746)', 'NCOA1 (8648)', 'NCOA2 (10499)', 'NCOA3 (8202)', 'NCOA4 (8031)', 'NCOA5 (57727)', 'NCOA6 (23054)', 'NCOA7 (135112)', 'NCOR1 (9611)', 'NCOR2 (9612)', 'NCR1 (9437)', 'NCR2 (9436)', 'NCR3 (259197)', 'NCR3LG1 (374383)', 'NCS1 (23413)', 'NCSTN (23385)', 'NDC1 (55706)', 'NDC80 (10403)', 'NDE1 (54820)', 'NDEL1 (81565)', 'NDFIP1 (80762)', 'NDFIP2 (54602)', 'NDN (4692)', 'NDNF (79625)', 'NDOR1 (27158)', 'NDP (4693)', 'NDRG1 (10397)', 'NDRG2 (57447)', 'NDRG3 (57446)', 'NDRG4 (65009)', 'NDST1 (3340)', 'NDST2 (8509)', 'NDST3 (9348)', 'NDST4 (64579)', 'NDUFA1 (4694)', 'NDUFA10 (4705)', 'NDUFA11 (126328)', 'NDUFA12 (55967)', 'NDUFA13 (51079)', 'NDUFA2 (4695)', 'NDUFA3 (4696)', 'NDUFA4 (4697)', 'NDUFA4L2 (56901)', 'NDUFA5 (4698)', 'NDUFA6 (4700)', 'NDUFA7 (4701)', 'NDUFA8 (4702)', 'NDUFA9 (4704)', 'NDUFAB1 (4706)', 'NDUFAB1 (51103)', 'NDUFAB2 (91942)', 'NDUFAB3 (25915)', 'NDUFAB4 (29078)', 'NDUFAB5 (79133)', 'NDUFAB6 (137682)', 'NDUFAB7 (55471)', 'NDUFAB8 (284184)', 'NDUFB1 (4707)', 'NDUFB10 (4716)', 'NDUFB11 (54539)', 'NDUFB2 (4708)', 'NDUFB3 (4709)', 'NDUFB4 (4710)', 'NDUFB5 (4711)', 'NDUFB6 (4712)', 'NDUFB7 (4713)', 'NDUFB8 (4714)', 'NDUFB9 (4715)', 'NDUFC1 (4717)', 'NDUFC2 (4718)', 'NDUFS1 (4719)', 'NDUFS2 (4720)', 'NDUFS3 (4722)', 'NDUFS4 (4724)', 'NDUFS5 (4725)', 'NDUFS6 (4726)', 'NDUFS7 (374291)', 'NDUFS8 (4728)', 'NDUFV1 (4723)', 'NDUFV2 (4729)', 'NDUFV3 (4731)', 'NEB (4703)', 'NEBL (10529)', 'NECAB1 (64168)', 'NECAB2 (54550)', 'NECAB3 (63941)', 'NECAP1 (25977)', 'NECAP2 (55707)', 'NECTIN1 (5818)', 'NECTIN2 (5819)', 'NECTIN3 (25945)', 'NECTIN4 (81607)', 'NEDD1 (121441)', 'NEDD4 (4734)', 'NEDD4L (23327)', 'NEDD8 (4738)', 'NEDD9 (4739)', 'NEFH (4744)', 'NEFL (4747)', 'NEFM (4741)', 'NEGR1 (257194)', 'NEIL1 (79661)', 'NEIL2 (252969)', 'NEIL3 (55247)', 'NEK1 (4750)', 'NEK10 (152110)', 'NEK11 (79858)', 'NEK2 (4751)', 'NEK3 (4752)', 'NEK4 (6787)', 'NEK5 (341676)', 'NEK6

(10783)', 'NEK7 (140609)', 'NEK8 (284086)', 'NEK9 (91754)', 'NELFA (7469)',
 'NELFB (25920)', 'NELFCD (51497)', 'NELFE (7936)', 'NELL1 (4745)', 'NELL2
 (4753)', 'NEMF (9147)', 'NEMP1 (23306)', 'NEMP2 (100131211)', 'NENF (29937)',
 'NEO1 (4756)', 'NES (10763)', 'NET1 (10276)', 'NETO1 (81832)', 'NETO2 (81831)',
 'NEU1 (4758)', 'NEU2 (4759)', 'NEU3 (10825)', 'NEU4 (129807)', 'NEURL1 (9148)',
 'NEURL1B (54492)', 'NEURL2 (140825)', 'NEURL3 (93082)', 'NEURL4 (84461)',
 'NEUROD1 (4760)', 'NEUROD2 (4761)', 'NEUROD4 (58158)', 'NEUROD6 (63974)',
 'NEUROG1 (4762)', 'NEUROG2 (63973)', 'NEUROG3 (50674)', 'NEXMIF (340533)', 'NEXN
 (91624)', 'NF1 (4763)', 'NF2 (4771)', 'NFAM1 (150372)', 'NFASC (23114)', 'NFAT5
 (10725)', 'NFATC1 (4772)', 'NFATC2 (4773)', 'NFATC2IP (84901)', 'NFATC3 (4775)',
 'NFATC4 (4776)', 'NFE2 (4778)', 'NFE2L1 (4779)', 'NFE2L2 (4780)', 'NFE2L3
 (9603)', 'NFIA (4774)', 'NFIB (4781)', 'NFIC (4782)', 'NFIL3 (4783)', 'NFILZ
 (105372267)', 'NFIK (4784)', 'NFKB1 (4790)', 'NFKB2 (4791)', 'NFKBIA (4792)',
 'NFKBIB (4793)', 'NFKBID (84807)', 'NFKBIE (4794)', 'NFKBIL1 (4795)', 'NFKBIZ
 (64332)', 'NFRKB (4798)', 'NFS1 (9054)', 'NFU1 (27247)', 'NFX1 (4799)', 'NFXL1
 (152518)', 'NFYA (4800)', 'NFYB (4801)', 'NFYC (4802)', 'NGB (58157)', 'NGDN
 (25983)', 'NGEF (25791)', 'NGF (4803)', 'NGFR (4804)', 'NGLY1 (55768)', 'NGRN
 (51335)', 'NHEJ1 (79840)', 'NHERF1 (9368)', 'NHERF2 (9351)', 'NHERF4 (79849)',
 'NHLH1 (4807)', 'NHLH2 (4808)', 'NHLRC1 (378884)', 'NHLRC2 (374354)', 'NHLRC3
 (387921)', 'NHLRC4 (283948)', 'NHP2 (55651)', 'NHS (4810)', 'NHSL1 (57224)',
 'NHSL2 (340527)', 'NHSL3 (57648)', 'NIBAN1 (116496)', 'NIBAN2 (64855)', 'NIBAN3
 (199786)', 'NICN1 (84276)', 'NICOL1 (401115)', 'NID1 (4811)', 'NID2 (22795)',
 'NIF3L1 (60491)', 'NIFK (84365)', 'NIM1K (167359)', 'NIN (51199)', 'NINJ1
 (4814)', 'NINJ2 (4815)', 'NINL (22981)', 'NIP7 (51388)', 'NIPA1 (123606)',
 'NIPA2 (81614)', 'NIPAL1 (152519)', 'NIPAL2 (79815)', 'NIPAL3 (57185)', 'NIPAL4
 (348938)', 'NIPBL (25836)', 'NIPSNAP1 (8508)', 'NIPSNAP2 (2631)', 'NIPSNAP3A
 (25934)', 'NIPSNAP3B (55335)', 'NISCH (11188)', 'NIT1 (4817)', 'NIT2 (56954)',
 'NKAIN1 (79570)', 'NKAIN2 (154215)', 'NKAIN3 (286183)', 'NKAIN4 (128414)', 'NKAP
 (79576)', 'NKAPD1 (55216)', 'NKAPL (222698)', 'NKD1 (85407)', 'NKD2 (85409)',
 'NKG7 (4818)', 'NKIRAS1 (28512)', 'NKIRAS2 (28511)', 'NKTR (4820)', 'NKX1-1
 (54729)', 'NKX1-2 (390010)', 'NKX2-1 (7080)', 'NKX2-2 (4821)', 'NKX2-3
 (159296)', 'NKX2-4 (644524)', 'NKX2-5 (1482)', 'NKX2-8 (26257)', 'NKX3-1
 (4824)', 'NKX3-2 (579)', 'NKX6-1 (4825)', 'NKX6-2 (84504)', 'NKX6-3 (157848)',
 'NLE1 (54475)', 'NLGN1 (22871)', 'NLGN2 (57555)', 'NLGN3 (54413)', 'NLGN4X
 (57502)', 'NLGN4Y (22829)', 'NLK (51701)', 'NLN (57486)', 'NLRC3 (197358)',
 'NLRC4 (58484)', 'NLRC5 (84166)', 'NLRP1 (22861)', 'NLRP10 (338322)', 'NLRP11
 (204801)', 'NLRP12 (91662)', 'NLRP13 (126204)', 'NLRP14 (338323)', 'NLRP2
 (55655)', 'NLRP2B (286430)', 'NLRP3 (114548)', 'NLRP4 (147945)', 'NLRP5
 (126206)', 'NLRP6 (171389)', 'NLRP7 (199713)', 'NLRP8 (126205)', 'NLRP9
 (338321)', 'NLRX1 (79671)', 'NMB (4828)', 'NMBR (4829)', 'NMD3 (51068)', 'NME1
 (4830)', 'NME2 (4831)', 'NME3 (4832)', 'NME4 (4833)', 'NME5 (8382)', 'NME6
 (10201)', 'NME7 (29922)', 'NME8 (51314)', 'NME9 (347736)', 'NMI (9111)', 'NMNAT1
 (64802)', 'NMNAT2 (23057)', 'NMNAT3 (349565)', 'NMRAL1 (57407)', 'NMRK1
 (54981)', 'NMRK2 (27231)', 'NMS (129521)', 'NMT1 (4836)', 'NMT2 (9397)', 'NMU
 (10874)', 'NMUR1 (10316)', 'NMUR2 (56923)', 'NNAT (4826)', 'NNMT (4837)', 'NNT
 (23530)', 'NOA1 (84273)', 'NOB1 (28987)', 'NOBOX (135935)', 'NOC2L (26155)',
 'NOC3L (64318)', 'NOC4L (79050)', 'NOCT (25819)', 'NOD1 (10392)', 'NOD2
 (64127)', 'NODAL (4838)', 'NOG (9241)', 'NOL10 (79954)', 'NOL11 (25926)', 'NOL12

(79159)', 'NOL3 (8996)', 'NOL4 (8715)', 'NOL4L (140688)', 'NOL6 (65083)', 'NOL7 (51406)', 'NOL8 (55035)', 'NOL9 (79707)', 'NOLC1 (9221)', 'NOM1 (64434)', 'NOM01 (23420)', 'NOM02 (283820)', 'NONO (4841)', 'NOP10 (55505)', 'NOP14 (8602)', 'NOP16 (51491)', 'NOP2 (4839)', 'NOP53 (29997)', 'NOP56 (10528)', 'NOP58 (51602)', 'NOP9 (161424)', 'NOPCHAP1 (121053)', 'NOS1 (4842)', 'NOS1AP (9722)', 'NOS2 (4843)', 'NOS3 (4846)', 'NOSIP (51070)', 'NOSTRIN (115677)', 'NOTCH1 (4851)', 'NOTCH2 (4853)', 'NOTCH3 (4854)', 'NOTCH4 (4855)', 'NOTO (344022)', 'NOTUM (147111)', 'NOVA1 (4857)', 'NOVA2 (4858)', 'NOX1 (27035)', 'NOX3 (50508)', 'NOX4 (50507)', 'NOX5 (79400)', 'NOXA1 (10811)', 'NOXO1 (124056)', 'NOXRED1 (122945)', 'NPAP1 (23742)', 'NPAS1 (4861)', 'NPAS2 (4862)', 'NPAS3 (64067)', 'NPAS4 (266743)', 'NPAT (4863)', 'NPB (256933)', 'NPBWR1 (2831)', 'NPBWR2 (2832)', 'NPC1 (4864)', 'NPC1L1 (29881)', 'NPC2 (10577)', 'NPDC1 (56654)', 'NPEPL1 (79716)', 'NPEPPS (9520)', 'NPFF (8620)', 'NPFFR1 (64106)', 'NPFFR2 (10886)', 'NPHP1 (4867)', 'NPHP3 (27031)', 'NPHP4 (261734)', 'NPHS1 (4868)', 'NPHS2 (7827)', 'NPIP1 (728888)', 'NPL (80896)', 'NPLOC4 (55666)', 'NPM1 (4869)', 'NPM2 (10361)', 'NPM3 (10360)', 'NPNT (255743)', 'NPPA (4878)', 'NPPB (4879)', 'NPPC (4880)', 'NPR1 (4881)', 'NPR2 (4882)', 'NPR3 (4883)', 'NPRL2 (10641)', 'NPRL3 (8131)', 'NPS (594857)', 'NPSR1 (387129)', 'NPTN (27020)', 'NPTX1 (4884)', 'NPTX2 (4885)', 'NPTXR (23467)', 'NPVF (64111)', 'NPW (283869)', 'NPY (4852)', 'NPY1R (4886)', 'NPY2R (4887)', 'NPY5R (4889)', 'NQ01 (1728)', 'NQ02 (4835)', 'NR0B1 (190)', 'NR0B2 (8431)', 'NR1D1 (9572)', 'NR1D2 (9975)', 'NR1H2 (7376)', 'NR1H3 (10062)', 'NR1H4 (9971)', 'NR1I2 (8856)', 'NR1I3 (9970)', 'NR2C1 (7181)', 'NR2C2 (7182)', 'NR2C2AP (126382)', 'NR2E1 (7101)', 'NR2E3 (10002)', 'NR2F1 (7025)', 'NR2F2 (7026)', 'NR2F6 (2063)', 'NR3C1 (2908)', 'NR3C2 (4306)', 'NR4A1 (3164)', 'NR4A2 (4929)', 'NR4A3 (8013)', 'NR5A1 (2516)', 'NR5A2 (2494)', 'NR6A1 (2649)', 'NRAP (4892)', 'NRARP (441478)', 'NRAS (4893)', 'NRBF2 (29982)', 'NRBP1 (29959)', 'NRBP2 (340371)', 'NRCAM (4897)', 'NRDC (4898)', 'NRDE2 (55051)', 'NREP (9315)', 'NRF1 (4899)', 'NRG1 (3084)', 'NRG2 (9542)', 'NRG3 (10718)', 'NRG4 (145957)', 'NRGN (4900)', 'NRIP1 (8204)', 'NRIP2 (83714)', 'NRIP3 (56675)', 'NRK (203447)', 'NRL (4901)', 'NRM (11270)', 'NRN1 (51299)', 'NRN1L (123904)', 'NRP1 (8829)', 'NRP2 (8828)', 'NRROS (375387)', 'NRSN1 (140767)', 'NRSN2 (80023)', 'NRTN (4902)', 'NRXN1 (9378)', 'NRXN2 (9379)', 'NRXN3 (9369)', 'NSA2 (10412)', 'NSD1 (64324)', 'NSD2 (7468)', 'NSD3 (54904)', 'NSDHL (50814)', 'NSF (4905)', 'NSFL1C (55968)', 'NSG1 (27065)', 'NSG2 (51617)', 'NSL1 (25936)', 'NSMAF (8439)', 'NSMCE1 (197370)', 'NSMCE2 (286053)', 'NSMCE3 (56160)', 'NSMCE4A (54780)', 'NSMF (26012)', 'NSRP1 (84081)', 'NSUN2 (54888)', 'NSUN3 (63899)', 'NSUN4 (387338)', 'NSUN5 (55695)', 'NSUN6 (221078)', 'NSUN7 (79730)', 'NT5C (30833)', 'NT5C1A (84618)', 'NT5C1B (93034)', 'NT5C2 (22978)', 'NT5C3A (51251)', 'NT5C3B (115024)', 'NT5DC1 (221294)', 'NT5DC2 (64943)', 'NT5DC3 (51559)', 'NT5DC4 (284958)', 'NT5E (4907)', 'NT5M (56953)', 'NTAN1 (123803)', 'NTAQ1 (55093)', 'NTF3 (4908)', 'NTF4 (4909)', 'NTHL1 (4913)', 'NTM (50863)', 'NTMT1 (28989)', 'NTMT2 (149281)', 'NTN1 (9423)', 'NTN3 (4917)', 'NTN4 (59277)', 'NTN5 (126147)', 'NTNG1 (22854)', 'NTNG2 (84628)', 'NTPCR (84284)', 'NTRK1 (4914)', 'NTRK2 (4915)', 'NTRK3 (4916)', 'NTS (4922)', 'NTSR1 (4923)', 'NTSR2 (23620)', 'NUAK1 (9891)', 'NUAK2 (81788)', 'NUB1 (51667)', 'NUBP1 (4682)', 'NUBP2 (10101)', 'NUBPL (80224)', 'NUCB1 (4924)', 'NUCB2 (4925)', 'NUCKS1 (64710)', 'NUDC (10726)', 'NUDCD1 (84955)', 'NUDCD2 (134492)', 'NUDCD3 (23386)', 'NUDT1 (4521)', 'NUDT10 (170685)', 'NUDT11 (55190)', 'NUDT12

(83594)', 'NUDT13 (25961)', 'NUDT14 (256281)', 'NUDT15 (55270)', 'NUDT16
 (131870)', 'NUDT16L1 (84309)', 'NUDT17 (200035)', 'NUDT18 (79873)', 'NUDT19
 (390916)', 'NUDT2 (318)', 'NUDT21 (11051)', 'NUDT22 (84304)', 'NUDT3 (11165)',
 'NUDT4 (11163)', 'NUDT5 (11164)', 'NUDT6 (11162)', 'NUDT7 (283927)', 'NUDT8
 (254552)', 'NUDT9 (53343)', 'NUF2 (83540)', 'NUFIP1 (26747)', 'NUFIP2 (57532)',
 'NUGGC (389643)', 'NUMA1 (4926)', 'NUMB (8650)', 'NUMBL (9253)', 'NUP107
 (57122)', 'NUP133 (55746)', 'NUP153 (9972)', 'NUP155 (9631)', 'NUP160 (23279)',
 'NUP188 (23511)', 'NUP205 (23165)', 'NUP210 (23225)', 'NUP210L (91181)', 'NUP214
 (8021)', 'NUP35 (129401)', 'NUP37 (79023)', 'NUP42 (11097)', 'NUP43 (348995)',
 'NUP50 (10762)', 'NUP54 (53371)', 'NUP58 (9818)', 'NUP62 (23636)', 'NUP62CL
 (54830)', 'NUP85 (79902)', 'NUP88 (4927)', 'NUP93 (9688)', 'NUP98 (4928)',
 'NUPR1 (26471)', 'NUPR2 (389493)', 'NUS1 (116150)', 'NUSAP1 (51203)', 'NUTF2
 (10204)', 'NUTM1 (256646)', 'NUTM2A (728118)', 'NUTM2D (728130)', 'NUTM2F
 (54754)', 'NUTM2G (441457)', 'NVL (4931)', 'NWD1 (284434)', 'NWD2 (57495)',
 'NXF1 (10482)', 'NXF3 (56000)', 'NXN (64359)', 'NXNL1 (115861)', 'NXNL2
 (158046)', 'NXPE1 (120400)', 'NXPE2 (120406)', 'NXPE3 (91775)', 'NXPE4 (54827)',
 'NXPH1 (30010)', 'NXPH2 (11249)', 'NXPH3 (11248)', 'NXPH4 (11247)', 'NXT1
 (29107)', 'NXT2 (55916)', 'NYAP1 (222950)', 'NYAP2 (57624)', 'NYNRIN (57523)',
 'NYX (60506)', 'OAF (220323)', 'OARD1 (221443)', 'OAS1 (4938)', 'OAS2 (4939)',
 'OAS3 (4940)', 'OASL (8638)', 'OAT (4942)', 'OAZ1 (4946)', 'OAZ2 (4947)', 'OAZ3
 (51686)', 'OBI1 (79596)', 'OBP2A (29991)', 'OBP2B (29989)', 'OBSCN (84033)',
 'OBSL1 (23363)', 'OC90 (729330)', 'OCA2 (4948)', 'OCEL1 (79629)', 'OCIAD1
 (54940)', 'OCIAD2 (132299)', 'OCLN (100506658)', 'OCM (654231)', 'OCM2 (4951)',
 'OCRL (4952)', 'OCSTAMP (128506)', 'ODAD1 (93233)', 'ODAD2 (55130)', 'ODAD3
 (115948)', 'ODAD4 (83538)', 'ODAM (54959)', 'ODAPH (152816)', 'ODC1 (4953)',
 'ODF1 (4956)', 'ODF2 (4957)', 'ODF2L (57489)', 'ODF4 (146852)', 'ODR4 (54953)',
 'OFD1 (8481)', 'OGA (10724)', 'OGDH (4967)', 'OGDHL (55753)', 'OGFOD1 (55239)',
 'OGFOD2 (79676)', 'OGFOD3 (79701)', 'OGFR (11054)', 'OGFRL1 (79627)', 'OGG1
 (4968)', 'OGN (4969)', 'OGT (8473)', 'OIP5 (11339)', 'OIT3 (170392)', 'OLA1
 (29789)', 'OLAH (55301)', 'OLFM1 (10439)', 'OLFM2 (93145)', 'OLFM3 (118427)',
 'OLFM4 (10562)', 'OLFML1 (283298)', 'OLFML2A (169611)', 'OLFML2B (25903)',
 'OLFML3 (56944)', 'OLIG1 (116448)', 'OLIG2 (10215)', 'OLIG3 (167826)', 'OLR1
 (4973)', 'OMA1 (115209)', 'OMD (4958)', 'OMG (4974)', 'OMP (4975)', 'ONECUT1
 (3175)', 'ONECUT2 (9480)', 'ONECUT3 (390874)', 'OOEP (441161)', 'OOSP2
 (219990)', 'OOSP3 (112577461)', 'OPA1 (4976)', 'OPA3 (80207)', 'OPALIN (93377)',
 'OPCML (4978)', 'OPHN1 (4983)', 'OPLAH (26873)', 'OPN1LW (5956)', 'OPN1SW
 (611)', 'OPN3 (23596)', 'OPN4 (94233)', 'OPN5 (221391)', 'OPRD1 (4985)', 'OPRK1
 (4986)', 'OPRL1 (4987)', 'OPRM1 (4988)', 'OPRPN (58503)', 'OPTC (26254)', 'OPTN
 (10133)', 'OR10A2 (341276)', 'OR10A3 (26496)', 'OR10A4 (283297)', 'OR10A5
 (144124)', 'OR10A7 (121364)', 'OR10AD1 (121275)', 'OR10AG1 (282770)', 'OR10C1
 (442194)', 'OR10D3 (26497)', 'OR10G2 (26534)', 'OR10G3 (26533)', 'OR10G4
 (390264)', 'OR10G6 (79490)', 'OR10G7 (390265)', 'OR10G8 (219869)', 'OR10G9
 (219870)', 'OR10H1 (26539)', 'OR10H2 (26538)', 'OR10H3 (26532)', 'OR10H4
 (126541)', 'OR10H5 (284433)', 'OR10J1 (26476)', 'OR10J5 (127385)', 'OR10K1
 (391109)', 'OR10K2 (391107)', 'OR10P1 (121130)', 'OR10Q1 (219960)', 'OR10R2
 (343406)', 'OR10T2 (128360)', 'OR10V1 (390201)', 'OR10W1 (81341)', 'OR10Z1
 (128368)', 'OR11A1 (26531)', 'OR11H4 (390442)', 'OR11H6 (122748)', 'OR11L1
 (391189)', 'OR12D3 (81797)', 'OR13A1 (79290)', 'OR13C2 (392376)', 'OR13C3

(138803)', 'OR13C4 (138804)', 'OR13C5 (138799)', 'OR13C8 (138802)', 'OR13C9
 (286362)', 'OR13D1 (286365)', 'OR13F1 (138805)', 'OR13G1 (441933)', 'OR13H1
 (347468)', 'OR13J1 (392309)', 'OR14A16 (284532)', 'OR14A2 (388761)', 'OR14C36
 (127066)', 'OR14I1 (401994)', 'OR14J1 (442191)', 'OR14K1 (343170)', 'OR1A1
 (8383)', 'OR1A2 (26189)', 'OR1C1 (26188)', 'OR1D2 (4991)', 'OR1D5 (8386)',
 'OR1E1 (8387)', 'OR1E2 (8388)', 'OR1F1 (4992)', 'OR1G1 (8390)', 'OR1I1
 (126370)', 'OR1J1 (347168)', 'OR1J2 (26740)', 'OR1J4 (26219)', 'OR1K1 (392392)',
 'OR1L1 (26737)', 'OR1L3 (26735)', 'OR1L4 (254973)', 'OR1L6 (392390)', 'OR1L8
 (138881)', 'OR1M1 (125963)', 'OR1N1 (138883)', 'OR1N2 (138882)', 'OR1Q1
 (158131)', 'OR2A12 (346525)', 'OR2A14 (135941)', 'OR2A2 (442361)', 'OR2A25
 (392138)', 'OR2A5 (393046)', 'OR2AE1 (81392)', 'OR2AG1 (144125)', 'OR2AG2
 (338755)', 'OR2AJ1 (127608)', 'OR2AP1 (121129)', 'OR2AT4 (341152)', 'OR2B11
 (127623)', 'OR2B2 (81697)', 'OR2B3 (442184)', 'OR2B6 (26212)', 'OR2C1 (4993)',
 'OR2C3 (81472)', 'OR2D2 (120776)', 'OR2D3 (120775)', 'OR2F2 (135948)', 'OR2G2
 (81470)', 'OR2G3 (81469)', 'OR2G6 (391211)', 'OR2H1 (26716)', 'OR2H2 (7932)',
 'OR2J1 (442185)', 'OR2J2 (26707)', 'OR2J3 (442186)', 'OR2K2 (26248)', 'OR2L13
 (284521)', 'OR2L2 (26246)', 'OR2L3 (391192)', 'OR2L5 (81466)', 'OR2M2 (391194)',
 'OR2M3 (127062)', 'OR2M4 (26245)', 'OR2M5 (127059)', 'OR2M7 (391196)', 'OR2S2
 (56656)', 'OR2T10 (127069)', 'OR2T12 (127064)', 'OR2T2 (401992)', 'OR2T27
 (403239)', 'OR2T29 (343563)', 'OR2T35 (403244)', 'OR2T4 (127074)', 'OR2T5
 (401993)', 'OR2T6 (254879)', 'OR2T8 (343172)', 'OR2V1 (26693)', 'OR2V2
 (285659)', 'OR2W1 (26692)', 'OR2W3 (343171)', 'OR2Y1 (134083)', 'OR2Z1
 (284383)', 'OR3A1 (4994)', 'OR4A16 (81327)', 'OR4A47 (403253)', 'OR4A5 (81318)',
 'OR4B1 (119765)', 'OR4C11 (219429)', 'OR4C12 (283093)', 'OR4C13 (283092)',
 'OR4C15 (81309)', 'OR4C3 (256144)', 'OR4C46 (119749)', 'OR4C5 (79346)', 'OR4C6
 (219432)', 'OR4D1 (26689)', 'OR4D10 (390197)', 'OR4D11 (219986)', 'OR4D2
 (124538)', 'OR4D5 (219875)', 'OR4D6 (219983)', 'OR4D9 (390199)', 'OR4E1
 (26687)', 'OR4E2 (26686)', 'OR4F15 (390649)', 'OR4F6 (390648)', 'OR4K1 (79544)',
 'OR4K13 (390433)', 'OR4K14 (122740)', 'OR4K15 (81127)', 'OR4K17 (390436)',
 'OR4K2 (390431)', 'OR4K5 (79317)', 'OR4L1 (122742)', 'OR4M1 (441670)', 'OR4N2
 (390429)', 'OR4N5 (390437)', 'OR4P4 (81300)', 'OR4S1 (256148)', 'OR4S2
 (219431)', 'OR4X1 (390113)', 'OR51A2 (401667)', 'OR51A4 (401666)', 'OR51A7
 (119687)', 'OR51B4 (79339)', 'OR51B5 (282763)', 'OR51B6 (390058)', 'OR51D1
 (390038)', 'OR51E1 (143503)', 'OR51E2 (81285)', 'OR51F1 (256892)', 'OR51F2
 (119694)', 'OR51G2 (81282)', 'OR51I1 (390063)', 'OR51I2 (390064)', 'OR51L1
 (119682)', 'OR51M1 (390059)', 'OR51Q1 (390061)', 'OR51S1 (119692)', 'OR51T1
 (401665)', 'OR51V1 (283111)', 'OR52A1 (23538)', 'OR52A5 (390054)', 'OR52B2
 (255725)', 'OR52B6 (340980)', 'OR52D1 (390066)', 'OR52E2 (119678)', 'OR52E4
 (390081)', 'OR52E5 (390082)', 'OR52E6 (390078)', 'OR52E8 (390079)', 'OR52H1
 (390067)', 'OR52I1 (390037)', 'OR52J3 (119679)', 'OR52K1 (390036)', 'OR52K2
 (119774)', 'OR52L1 (338751)', 'OR52M1 (119772)', 'OR52N1 (79473)', 'OR52N2
 (390077)', 'OR52N4 (390072)', 'OR52N5 (390075)', 'OR52W1 (120787)', 'OR56A1
 (120796)', 'OR56A3 (390083)', 'OR56A4 (120793)', 'OR56A5 (390084)', 'OR56B1
 (387748)', 'OR56B4 (196335)', 'OR5A1 (219982)', 'OR5A2 (219981)', 'OR5AC2
 (81050)', 'OR5AK2 (390181)', 'OR5AN1 (390195)', 'OR5AP2 (338675)', 'OR5AS1
 (219447)', 'OR5AU1 (390445)', 'OR5B12 (390191)', 'OR5B17 (219965)', 'OR5B2
 (390190)', 'OR5B21 (219968)', 'OR5B3 (441608)', 'OR5C1 (392391)', 'OR5D14
 (219436)', 'OR5D16 (390144)', 'OR5D18 (219438)', 'OR5F1 (338674)', 'OR5H14

(403273)', 'OR5H15 (403274)', 'OR5H2 (79310)', 'OR5H6 (79295)', 'OR5I1 (10798)',
 'OR5J2 (282775)', 'OR5K1 (26339)', 'OR5K3 (403277)', 'OR5K4 (403278)', 'OR5L2
 (26338)', 'OR5M1 (390168)', 'OR5M10 (390167)', 'OR5M11 (219487)', 'OR5M3
 (219482)', 'OR5M8 (219484)', 'OR5M9 (390162)', 'OR5P2 (120065)', 'OR5P3
 (120066)', 'OR5T1 (390155)', 'OR5T2 (219464)', 'OR5T3 (390154)', 'OR5V1
 (81696)', 'OR5W2 (390148)', 'OR6A2 (8590)', 'OR6B1 (135946)', 'OR6B2 (389090)',
 'OR6B3 (150681)', 'OR6C1 (390321)', 'OR6C2 (341416)', 'OR6C3 (254786)', 'OR6C4
 (341418)', 'OR6C6 (283365)', 'OR6C65 (403282)', 'OR6C68 (403284)', 'OR6C70
 (390327)', 'OR6C74 (254783)', 'OR6C75 (390323)', 'OR6C76 (390326)', 'OR6F1
 (343169)', 'OR6J1 (79549)', 'OR6K2 (81448)', 'OR6K3 (391114)', 'OR6M1 (390261)',
 'OR6N1 (128372)', 'OR6N2 (81442)', 'OR6P1 (128366)', 'OR6Q1 (219952)', 'OR6S1
 (341799)', 'OR6T1 (219874)', 'OR6V1 (346517)', 'OR6X1 (390260)', 'OR6Y1
 (391112)', 'OR7A10 (390892)', 'OR7A17 (26333)', 'OR7A5 (26659)', 'OR7C1
 (26664)', 'OR7C2 (26658)', 'OR7D2 (162998)', 'OR7D4 (125958)', 'OR7E24 (26648)',
 'OR7G1 (125962)', 'OR7G2 (390882)', 'OR7G3 (390883)', 'OR8A1 (390275)', 'OR8B12
 (219858)', 'OR8B2 (26595)', 'OR8B3 (390271)', 'OR8B4 (283162)', 'OR8B8 (26493)',
 'OR8D1 (283159)', 'OR8D4 (338662)', 'OR8G1 (26494)', 'OR8G5 (219865)', 'OR8H1
 (219469)', 'OR8H2 (390151)', 'OR8H3 (390152)', 'OR8I2 (120586)', 'OR8J1
 (219477)', 'OR8J3 (81168)', 'OR8K1 (390157)', 'OR8K5 (219453)', 'OR8S1
 (341568)', 'OR8U1 (219417)', 'OR9A2 (135924)', 'OR9A4 (130075)', 'OR9G1
 (390174)', 'OR9G4 (283189)', 'OR9I1 (219954)', 'OR9K2 (441639)', 'OR9Q1
 (219956)', 'OR9Q2 (219957)', 'ORAI2 (80228)', 'ORAI3 (93129)', 'ORC1 (4998)',
 'ORC2 (4999)', 'ORC3 (23595)', 'ORC4 (5000)', 'ORC5 (5001)', 'ORC6 (23594)',
 'ORM1 (5004)', 'ORM2 (5005)', 'ORMDL1 (94101)', 'ORMDL2 (29095)', 'ORMDL3
 (94103)', 'OS9 (10956)', 'OSBP (5007)', 'OSBP2 (23762)', 'OSBPL10 (114884)',
 'OSBPL11 (114885)', 'OSBPL1A (114876)', 'OSBPL2 (9885)', 'OSBPL3 (26031)',
 'OSBPL5 (114879)', 'OSBPL6 (114880)', 'OSBPL7 (114881)', 'OSBPL8 (114882)',
 'OSBPL9 (114883)', 'OSCAR (126014)', 'OSCP1 (127700)', 'OSER1 (51526)', 'OSGEP
 (55644)', 'OSGEPL1 (64172)', 'OSGIN1 (29948)', 'OSGIN2 (734)', 'OSM (5008)',
 'OSMR (9180)', 'OSR1 (130497)', 'OSR2 (116039)', 'OST4 (100128731)', 'OSTC
 (58505)', 'OSTF1 (26578)', 'OSTM1 (28962)', 'OSTN (344901)', 'OTC (5009)', 'OTOA
 (146183)', 'OTOF (9381)', 'OTOG (340990)', 'OTOGL (283310)', 'OTOL1 (131149)',
 'OTOP1 (133060)', 'OTOP2 (92736)', 'OTOP3 (347741)', 'OTOR (56914)', 'OTOS
 (150677)', 'OTP (23440)', 'OTUB1 (55611)', 'OTUB2 (78990)', 'OTUD1 (220213)',
 'OTUD3 (23252)', 'OTUD4 (54726)', 'OTUD5 (55593)', 'OTUD6A (139562)', 'OTUD6B
 (51633)', 'OTUD7A (161725)', 'OTUD7B (56957)', 'OTULIN (90268)', 'OTULINL
 (54491)', 'OTX1 (5013)', 'OTX2 (5015)', 'OVCA2 (124641)', 'OVCH1 (341350)',
 'OVCH2 (341277)', 'OVGP1 (5016)', 'OVOL1 (5017)', 'OVOL2 (58495)', 'OVOL3
 (728361)', 'OXA1L (5018)', 'OXCT1 (5019)', 'OXCT2 (64064)', 'OXER1 (165140)',
 'OXGR1 (27199)', 'OXLD1 (339229)', 'OXNAD1 (92106)', 'OXR1 (55074)', 'OXSM
 (54995)', 'OXSR1 (9943)', 'OXT (5020)', 'OXTR (5021)', 'P2RX1 (5023)', 'P2RX2
 (22953)', 'P2RX3 (5024)', 'P2RX4 (5025)', 'P2RX5 (5026)', 'P2RX6 (9127)', 'P2RX7
 (5027)', 'P2RY1 (5028)', 'P2RY10 (27334)', 'P2RY11 (5032)', 'P2RY12 (64805)',
 'P2RY13 (53829)', 'P2RY14 (9934)', 'P2RY2 (5029)', 'P2RY4 (5030)', 'P2RY6
 (5031)', 'P2RY8 (286530)', 'P3H1 (64175)', 'P3H2 (55214)', 'P3H3 (10536)', 'P3H4
 (10609)', 'P3R3URF (110117498)', 'P4HA1 (5033)', 'P4HA2 (8974)', 'P4HA3
 (283208)', 'P4HB (5034)', 'P4HTM (54681)', 'PA2G4 (5036)', 'PAAF1 (80227)',
 'PABIR1 (116224)', 'PABIR2 (159090)', 'PABIR3 (159091)', 'PABPC1 (26986)',

'PABPC1L (80336)', 'PABPC3 (5042)', 'PABPC4 (8761)', 'PABPC4L (132430)', 'PABPC5 (140886)', 'PABPN1 (8106)', 'PABPN1L (390748)', 'PACC1 (55248)', 'PACRG (135138)', 'PACRGL (133015)', 'PACS1 (55690)', 'PACS2 (23241)', 'PACSIN1 (29993)', 'PACSIN2 (11252)', 'PACSIN3 (29763)', 'PADI1 (29943)', 'PADI2 (11240)', 'PADI3 (51702)', 'PADI4 (23569)', 'PADI6 (353238)', 'PAEP (5047)', 'PAF1 (54623)', 'PAFAH1B1 (5048)', 'PAFAH1B2 (5049)', 'PAFAH1B3 (5050)', 'PAFAH2 (5051)', 'PAG1 (55824)', 'PAGE1 (8712)', 'PAGE2 (203569)', 'PAGE2B (389860)', 'PAGE3 (139793)', 'PAGE4 (9506)', 'PAGE5 (90737)', 'PAGR1 (79447)', 'PAH (5053)', 'PAICS (10606)', 'PAIP1 (10605)', 'PAIP2 (51247)', 'PAIP2B (400961)', 'PAK1 (5058)', 'PAK1IP1 (55003)', 'PAK2 (5062)', 'PAK3 (5063)', 'PAK4 (10298)', 'PAK5 (57144)', 'PAK6 (56924)', 'PALB2 (79728)', 'PALD1 (27143)', 'PALLD (23022)', 'PALM (5064)', 'PALM2AKAP2 (445815)', 'PALM3 (342979)', 'PALMD (54873)', 'PALS1 (64398)', 'PALS2 (51678)', 'PAM (5066)', 'PAM16 (51025)', 'PAMR1 (25891)', 'PAN2 (9924)', 'PAN3 (255967)', 'PANK1 (53354)', 'PANK2 (80025)', 'PANK3 (79646)', 'PANK4 (55229)', 'PANX1 (24145)', 'PANX2 (56666)', 'PANX3 (116337)', 'PAOX (196743)', 'PAPLN (89932)', 'PAPOLA (10914)', 'PAPOLB (56903)', 'PAPOLG (64895)', 'PAPPA (5069)', 'PAPPA2 (60676)', 'PAPSS1 (9061)', 'PAPSS2 (9060)', 'PAQR3 (152559)', 'PAQR4 (124222)', 'PAQR5 (54852)', 'PAQR6 (79957)', 'PAQR7 (164091)', 'PAQR8 (85315)', 'PAQR9 (344838)', 'PARD3 (56288)', 'PARD3B (117583)', 'PARD6A (50855)', 'PARD6B (84612)', 'PARD6G (84552)', 'PARG (8505)', 'PARK7 (11315)', 'PARL (55486)', 'PARM1 (25849)', 'PARN (5073)', 'PARP1 (142)', 'PARP10 (84875)', 'PARP11 (57097)', 'PARP12 (64761)', 'PARP14 (54625)', 'PARP15 (165631)', 'PARP16 (54956)', 'PARP2 (10038)', 'PARP3 (10039)', 'PARP4 (143)', 'PARP6 (56965)', 'PARP8 (79668)', 'PARP9 (83666)', 'PARPBP (55010)', 'PARS2 (25973)', 'PARVA (55742)', 'PARVB (29780)', 'PARVG (64098)', 'PASD1 (139135)', 'PASK (23178)', 'PATE1 (160065)', 'PATE2 (399967)', 'PATE3 (100169851)', 'PATE4 (399968)', 'PATJ (10207)', 'PATL1 (219988)', 'PATL2 (197135)', 'PATZ1 (23598)', 'PAWR (5074)', 'PAX1 (5075)', 'PAX2 (5076)', 'PAX3 (5077)', 'PAX4 (5078)', 'PAX5 (5079)', 'PAX6 (5080)', 'PAX7 (5081)', 'PAX8 (7849)', 'PAX9 (5083)', 'PAXBP1 (94104)', 'PAXIP1 (22976)', 'PAXX (286257)', 'PBDC1 (51260)', 'PBK (55872)', 'PBLD (64081)', 'PBOV1 (59351)', 'PBRM1 (55193)', 'PBX1 (5087)', 'PBX2 (5089)', 'PBX3 (5090)', 'PBX4 (80714)', 'PBXIP1 (57326)', 'PC (5091)', 'PCARE (388939)', 'PCBD1 (5092)', 'PCBD2 (84105)', 'PCBP1 (5093)', 'PCBP2 (5094)', 'PCBP3 (54039)', 'PCBP4 (57060)', 'PCCA (5095)', 'PCCB (5096)', 'PCDH1 (5097)', 'PCDH10 (57575)', 'PCDH11X (27328)', 'PCDH11Y (83259)', 'PCDH12 (51294)', 'PCDH15 (65217)', 'PCDH17 (27253)', 'PCDH18 (54510)', 'PCDH19 (57526)', 'PCDH20 (64881)', 'PCDH7 (5099)', 'PCDH8 (5100)', 'PCDH9 (5101)', 'PCDHA1 (56147)', 'PCDHA10 (56139)', 'PCDHA11 (56138)', 'PCDHA12 (56137)', 'PCDHA13 (56136)', 'PCDHA2 (56146)', 'PCDHA3 (56145)', 'PCDHA4 (56144)', 'PCDHA5 (56143)', 'PCDHA6 (56142)', 'PCDHA7 (56141)', 'PCDHA8 (56140)', 'PCDHA9 (9752)', 'PCDHAC1 (56135)', 'PCDHAC2 (56134)', 'PCDHB1 (29930)', 'PCDHB10 (56126)', 'PCDHB11 (56125)', 'PCDHB12 (56124)', 'PCDHB13 (56123)', 'PCDHB14 (56122)', 'PCDHB15 (56121)', 'PCDHB16 (57717)', 'PCDHB2 (56133)', 'PCDHB3 (56132)', 'PCDHB4 (56131)', 'PCDHB5 (26167)', 'PCDHB6 (56130)', 'PCDHB7 (56129)', 'PCDHB8 (56128)', 'PCDHB9 (56127)', 'PCDHGA1 (56114)', 'PCDHGA10 (56106)', 'PCDHGA11 (56105)', 'PCDHGA12 (26025)', 'PCDHGA2 (56113)', 'PCDHGA3 (56112)', 'PCDHGA4 (56111)', 'PCDHGA5 (56110)', 'PCDHGA6 (56109)', 'PCDHGA7 (56108)', 'PCDHGA8 (9708)', 'PCDHGA9 (56107)', 'PCDHGB1 (56104)', 'PCDHGB2 (56103)', 'PCDHGB3

(56102)', 'PCDHGB4 (8641)', 'PCDHGB5 (56101)', 'PCDHGB6 (56100)', 'PCDHGB7
 (56099)', 'PCDHGC3 (5098)', 'PCDHGC4 (56098)', 'PCDHGC5 (56097)', 'PCED1A
 (64773)', 'PCED1B (91523)', 'PCF11 (51585)', 'PCGF1 (84759)', 'PCGF2 (7703)',
 'PCGF3 (10336)', 'PCGF5 (84333)', 'PCGF6 (84108)', 'PCID2 (55795)', 'PCIF1
 (63935)', 'PCK1 (5105)', 'PCK2 (5106)', 'PCLAF (9768)', 'PCL0 (27445)', 'PCM1
 (5108)', 'PCMT1 (5110)', 'PCMTD1 (115294)', 'PCMTD2 (55251)', 'PCNA (5111)',
 'PCNP (57092)', 'PCNT (5116)', 'PCNX1 (22990)', 'PCNX2 (80003)', 'PCNX3
 (399909)', 'PCNX4 (64430)', 'PCOLCE (5118)', 'PCOLCE2 (26577)', 'PCP2 (126006)',
 'PCP4 (5121)', 'PCP4L1 (654790)', 'PCSK1 (5122)', 'PCSK1N (27344)', 'PCSK2
 (5126)', 'PCSK4 (54760)', 'PCSK5 (5125)', 'PCSK6 (5046)', 'PCSK7 (9159)', 'PCSK9
 (255738)', 'PCTP (58488)', 'PCYOX1 (51449)', 'PCYOX1L (78991)', 'PCYT1A (5130)',
 'PCYT1B (9468)', 'PCYT2 (5833)', 'PDAP1 (11333)', 'PDC (5132)', 'PDCD1 (5133)',
 'PDCD10 (11235)', 'PDCD11 (22984)', 'PDCD1LG2 (80380)', 'PDCD2 (5134)', 'PDCD2L
 (84306)', 'PDCD4 (27250)', 'PDCD5 (9141)', 'PDCD6 (10016)', 'PDCD6IP (10015)',
 'PDCD7 (10081)', 'PDCL (5082)', 'PDCL2 (132954)', 'PDCL3 (79031)', 'PDE10A
 (10846)', 'PDE11A (50940)', 'PDE12 (201626)', 'PDE1A (5136)', 'PDE1B (5153)',
 'PDE1C (5137)', 'PDE2A (5138)', 'PDE3A (5139)', 'PDE3B (5140)', 'PDE4A (5141)',
 'PDE4B (5142)', 'PDE4C (5143)', 'PDE4D (5144)', 'PDE4DIP (9659)', 'PDE5A
 (8654)', 'PDE6A (5145)', 'PDE6B (5158)', 'PDE6C (5146)', 'PDE6D (5147)', 'PDE6G
 (5148)', 'PDE6H (5149)', 'PDE7A (5150)', 'PDE7B (27115)', 'PDE8A (5151)', 'PDE8B
 (8622)', 'PDE9A (5152)', 'PDF (64146)', 'PDGFA (5154)', 'PDGFB (5155)', 'PDGFC
 (56034)', 'PDGFD (80310)', 'PDGFRA (5156)', 'PDGFRB (5159)', 'PDGFRL (5157)',
 'PDHA1 (5160)', 'PDHA2 (5161)', 'PDHB (5162)', 'PDHX (8050)', 'PDIA2 (64714)',
 'PDIA3 (2923)', 'PDIA4 (9601)', 'PDIA5 (10954)', 'PDIA6 (10130)', 'PDIK1L
 (149420)', 'PDILT (204474)', 'PDK1 (5163)', 'PDK2 (5164)', 'PDK3 (5165)', 'PDK4
 (5166)', 'PDLIM1 (9124)', 'PDLIM2 (64236)', 'PDLIM3 (27295)', 'PDLIM4 (8572)',
 'PDLIM5 (10611)', 'PDLIM7 (9260)', 'PDP1 (54704)', 'PDP2 (57546)', 'PDPK1
 (5170)', 'PDPN (10630)', 'PDPR (55066)', 'PDRG1 (81572)', 'PDS5A (23244)',
 'PDS5B (23047)', 'PDSS1 (23590)', 'PDSS2 (57107)', 'PDX1 (3651)', 'PDXDC1
 (23042)', 'PDXK (8566)', 'PDXP (57026)', 'PDYN (5173)', 'PDZD11 (51248)', 'PDZD2
 (23037)', 'PDZD4 (57595)', 'PDZD7 (79955)', 'PDZD8 (118987)', 'PDZD9 (255762)',
 'PDZK1 (5174)', 'PDZK1IP1 (10158)', 'PDZRN3 (23024)', 'PDZRN4 (29951)', 'PEA15
 (8682)', 'PEAK1 (79834)', 'PEAK3 (374872)', 'PEAR1 (375033)', 'PEBP1 (5037)',
 'PEBP4 (157310)', 'PECAM1 (5175)', 'PECR (55825)', 'PEDS1 (387521)', 'PEF1
 (553115)', 'PEG10 (23089)', 'PEG3 (5178)', 'PELI1 (57162)', 'PELI2 (57161)',
 'PELI3 (246330)', 'PELO (53918)', 'PELP1 (27043)', 'PEMT (10400)', 'PENK
 (5179)', 'PEPD (5184)', 'PER1 (5187)', 'PER2 (8864)', 'PER3 (8863)', 'PERCC1
 (105371045)', 'PERM1 (84808)', 'PERP (64065)', 'PES1 (23481)', 'PET100
 (100131801)', 'PET117 (100303755)', 'PEX1 (5189)', 'PEX10 (5192)', 'PEX11A
 (8800)', 'PEX11B (8799)', 'PEX11G (92960)', 'PEX12 (5193)', 'PEX13 (5194)',
 'PEX14 (5195)', 'PEX16 (9409)', 'PEX19 (5824)', 'PEX2 (5828)', 'PEX26 (55670)',
 'PEX3 (8504)', 'PEX39 (441150)', 'PEX5 (5830)', 'PEX5L (51555)', 'PEX6 (5190)',
 'PEX7 (5191)', 'PF4 (5196)', 'PF4V1 (5197)', 'PFAS (5198)', 'PFDN1 (5201)',
 'PFDN2 (5202)', 'PFDN4 (5203)', 'PFDN5 (5204)', 'PFDN6 (10471)', 'PFKFB1
 (5207)', 'PFKFB2 (5208)', 'PFKFB3 (5209)', 'PFKFB4 (5210)', 'PFKL (5211)', 'PFKM
 (5213)', 'PFKP (5214)', 'PFN1 (5216)', 'PFN2 (5217)', 'PFN3 (345456)', 'PFN4
 (375189)', 'PGAM1 (5223)', 'PGAM2 (5224)', 'PGAM4 (441531)', 'PGAM5 (192111)',
 'PGAP1 (80055)', 'PGAP2 (27315)', 'PGAP3 (93210)', 'PGAP4 (84302)', 'PGAP6

(58986)', 'PGBD1 (84547)', 'PGBD2 (267002)', 'PGBD4 (161779)', 'PGBD5 (79605)',
 'PGC (5225)', 'PGCKA1 (55286)', 'PGD (5226)', 'PGF (5228)', 'PGGHG (80162)',
 'PGGT1B (5229)', 'PGK1 (5230)', 'PGK2 (5232)', 'PGLS (25796)', 'PGLYRP1 (8993)',
 'PGLYRP2 (114770)', 'PGLYRP3 (114771)', 'PGLYRP4 (57115)', 'PGM1 (5236)', 'PGM2
 (55276)', 'PGM2L1 (283209)', 'PGM3 (5238)', 'PGM5 (5239)', 'PGP (283871)',
 'PGPEP1 (54858)', 'PGPEP1L (145814)', 'PGR (5241)', 'PGRMC1 (10857)', 'PGRMC2
 (10424)', 'PGS1 (9489)', 'PHACTR1 (221692)', 'PHACTR2 (9749)', 'PHACTR3
 (116154)', 'PHACTR4 (65979)', 'PHAF1 (80262)', 'PHAX (51808)', 'PHB1 (5245)',
 'PHB2 (11331)', 'PHC1 (1911)', 'PHC2 (1912)', 'PHC3 (80012)', 'PHETA1 (144717)',
 'PHETA2 (150368)', 'PHEX (5251)', 'PHF1 (5252)', 'PHF10 (55274)', 'PHF11
 (51131)', 'PHF12 (57649)', 'PHF13 (148479)', 'PHF14 (9678)', 'PHF19 (26147)',
 'PHF2 (5253)', 'PHF20 (51230)', 'PHF20L1 (51105)', 'PHF21A (51317)', 'PHF21B
 (112885)', 'PHF23 (79142)', 'PHF24 (23349)', 'PHF3 (23469)', 'PHF5A (84844)',
 'PHF6 (84295)', 'PHF7 (51533)', 'PHF8 (23133)', 'PHGDH (26227)', 'PHGR1
 (644844)', 'PHIP (55023)', 'PHKA1 (5255)', 'PHKA2 (5256)', 'PHKB (5257)', 'PHKG1
 (5260)', 'PHKG2 (5261)', 'PHLDA1 (22822)', 'PHLDA2 (7262)', 'PHLDA3 (23612)',
 'PHLDB1 (23187)', 'PHLDB2 (90102)', 'PHLDB3 (653583)', 'PHLPP1 (23239)', 'PHLPP2
 (23035)', 'PHOSPHO1 (162466)', 'PHOSPHO2 (493911)', 'PHOX2A (401)', 'PHOX2B
 (8929)', 'PHPT1 (29085)', 'PHRF1 (57661)', 'PHTF1 (10745)', 'PHTF2 (57157)',
 'PHYH (5264)', 'PHYHD1 (254295)', 'PHYHIP (9796)', 'PHYHIPL (84457)', 'PHYKPL
 (85007)', 'PI15 (51050)', 'PI16 (221476)', 'PI3 (5266)', 'PI4K2A (55361)',
 'PI4K2B (55300)', 'PI4KA (5297)', 'PI4KB (5298)', 'PIANP (196500)', 'PIAS1
 (8554)', 'PIAS2 (9063)', 'PIAS3 (10401)', 'PIAS4 (51588)', 'PIBF1 (10464)',
 'PICALM (8301)', 'PICK1 (9463)', 'PID1 (55022)', 'PIDD1 (55367)', 'PIERCE1
 (138162)', 'PIERCE2 (145788)', 'PIEZ01 (9780)', 'PIEZ02 (63895)', 'PIF1
 (80119)', 'PIGA (5277)', 'PIGB (9488)', 'PIGBOS1 (101928527)', 'PIGC (5279)',
 'PIGF (5281)', 'PIGG (54872)', 'PIGH (5283)', 'PIGK (10026)', 'PIGL (9487)',
 'PIGM (93183)', 'PIGN (23556)', 'PIGO (84720)', 'PIGP (51227)', 'PIGQ (9091)',
 'PIGR (5284)', 'PIGS (94005)', 'PIGT (51604)', 'PIGU (128869)', 'PIGV (55650)',
 'PIGW (284098)', 'PIGX (54965)', 'PIGY (84992)', 'PIGZ (80235)', 'PIH1D1
 (55011)', 'PIH1D2 (120379)', 'PIK3AP1 (118788)', 'PIK3C2A (5286)', 'PIK3C2B
 (5287)', 'PIK3C2G (5288)', 'PIK3C3 (5289)', 'PIK3CA (5290)', 'PIK3CB (5291)',
 'PIK3CD (5293)', 'PIK3CG (5294)', 'PIK3IP1 (113791)', 'PIK3R1 (5295)', 'PIK3R2
 (5296)', 'PIK3R3 (8503)', 'PIK3R4 (30849)', 'PIK3R5 (23533)', 'PIK3R6 (146850)',
 'PIKFYVE (200576)', 'PILRA (29992)', 'PILRB (29990)', 'PIM1 (5292)', 'PIM2
 (11040)', 'PIM3 (415116)', 'PIMREG (54478)', 'PIN1 (5300)', 'PIN4 (5303)',
 'PINK1 (65018)', 'PINLYP (390940)', 'PINX1 (54984)', 'PIP (5304)', 'PIP4K2A
 (5305)', 'PIP4K2B (8396)', 'PIP4K2C (79837)', 'PIP4P1 (90809)', 'PIP4P2
 (55529)', 'PIP5K1A (8394)', 'PIP5K1B (8395)', 'PIP5K1C (23396)', 'PIP5KL1
 (138429)', 'PIPOX (51268)', 'PIR (8544)', 'PIRT (644139)', 'PISD (23761)',
 'PITHD1 (57095)', 'PITPNA (5306)', 'PITPNB (23760)', 'PITPNC1 (26207)', 'PITPNM1
 (9600)', 'PITPNM2 (57605)', 'PITPNM3 (83394)', 'PITRM1 (10531)', 'PITX1 (5307)',
 'PITX2 (5308)', 'PITX3 (5309)', 'PIWIL1 (9271)', 'PIWIL2 (55124)', 'PIWIL3
 (440822)', 'PIWIL4 (143689)', 'PJA1 (64219)', 'PJA2 (9867)', 'PJVK (494513)',
 'PKD1 (5310)', 'PKD1L1 (168507)', 'PKD1L2 (114780)', 'PKD1L3 (342372)', 'PKD2
 (5311)', 'PKD2L1 (9033)', 'PKD2L2 (27039)', 'PKDCC (91461)', 'PKDREJ (10343)',
 'PKHD1 (5314)', 'PKHD1L1 (93035)', 'PKIA (5569)', 'PKIB (5570)', 'PKIG (11142)',
 'PKLR (5313)', 'PKM (5315)', 'PKMYT1 (9088)', 'PKN1 (5585)', 'PKN2 (5586)',

'PKN3 (29941)', 'PKN0X1 (5316)', 'PKN0X2 (63876)', 'PKP1 (5317)', 'PKP2 (5318)',
 'PKP3 (11187)', 'PKP4 (8502)', 'PLA1A (51365)', 'PLA2G12A (81579)', 'PLA2G12B
 (84647)', 'PLA2G15 (23659)', 'PLA2G1B (5319)', 'PLA2G2A (5320)', 'PLA2G2C
 (391013)', 'PLA2G2D (26279)', 'PLA2G2E (30814)', 'PLA2G2F (64600)', 'PLA2G3
 (50487)', 'PLA2G4A (5321)', 'PLA2G4B (100137049)', 'PLA2G4C (8605)', 'PLA2G4D
 (283748)', 'PLA2G4E (123745)', 'PLA2G4F (255189)', 'PLA2G5 (5322)', 'PLA2G6
 (8398)', 'PLA2G7 (7941)', 'PLA2R1 (22925)', 'PLAA (9373)', 'PLAAT1 (57110)',
 'PLAAT2 (54979)', 'PLAAT3 (11145)', 'PLAAT4 (5920)', 'PLAAT5 (117245)', 'PLAC1
 (10761)', 'PLAC8 (51316)', 'PLAC8L1 (153770)', 'PLAC9 (219348)', 'PLAG1 (5324)',
 'PLAGL1 (5325)', 'PLAGL2 (5326)', 'PLAT (5327)', 'PLAU (5328)', 'PLAUR (5329)',
 'PLB1 (151056)', 'PLBD1 (79887)', 'PLBD2 (196463)', 'PLCB1 (23236)', 'PLCB2
 (5330)', 'PLCB3 (5331)', 'PLCB4 (5332)', 'PLCD1 (5333)', 'PLCD3 (113026)',
 'PLCD4 (84812)', 'PLCE1 (51196)', 'PLCG1 (5335)', 'PLCG2 (5336)', 'PLCH1
 (23007)', 'PLCH2 (9651)', 'PLCL1 (5334)', 'PLCL2 (23228)', 'PLCXD1 (55344)',
 'PLCXD2 (257068)', 'PLCXD3 (345557)', 'PLCZ1 (89869)', 'PLD1 (5337)', 'PLD2
 (5338)', 'PLD3 (23646)', 'PLD4 (122618)', 'PLD5 (200150)', 'PLD6 (201164)',
 'PLEC (5339)', 'PLEK (5341)', 'PLEK2 (26499)', 'PLEKHA1 (59338)', 'PLEKHA2
 (59339)', 'PLEKHA3 (65977)', 'PLEKHA4 (57664)', 'PLEKHA5 (54477)', 'PLEKHA6
 (22874)', 'PLEKHA7 (144100)', 'PLEKHA8 (84725)', 'PLEKHB1 (58473)', 'PLEKHB2
 (55041)', 'PLEKHD1 (400224)', 'PLEKHF1 (79156)', 'PLEKHF2 (79666)', 'PLEKHG1
 (57480)', 'PLEKHG2 (64857)', 'PLEKHG3 (26030)', 'PLEKHG4 (25894)', 'PLEKHG4B
 (153478)', 'PLEKHG5 (57449)', 'PLEKHG6 (55200)', 'PLEKHG7 (440107)', 'PLEKHH1
 (57475)', 'PLEKHH2 (130271)', 'PLEKHH3 (79990)', 'PLEKHJ1 (55111)', 'PLEKHM1
 (9842)', 'PLEKHM2 (23207)', 'PLEKHM3 (389072)', 'PLEKHN1 (84069)', 'PLEKH01
 (51177)', 'PLEKH02 (80301)', 'PLEKHS1 (79949)', 'PLET1 (349633)', 'PLG (5340)',
 'PLGRKT (55848)', 'PLIN1 (5346)', 'PLIN2 (123)', 'PLIN3 (10226)', 'PLIN4
 (729359)', 'PLIN5 (440503)', 'PLK1 (5347)', 'PLK2 (10769)', 'PLK3 (1263)', 'PLK4
 (10733)', 'PLK5 (126520)', 'PLLP (51090)', 'PLN (5350)', 'PLOD1 (5351)', 'PLOD2
 (5352)', 'PLOD3 (8985)', 'PLP1 (5354)', 'PLP2 (5355)', 'PLPBP (11212)', 'PLPP1
 (8611)', 'PLPP2 (8612)', 'PLPP3 (8613)', 'PLPP4 (196051)', 'PLPP5 (84513)',
 'PLPP6 (403313)', 'PLPP7 (84814)', 'PLPPR1 (54886)', 'PLPPR2 (64748)', 'PLPPR3
 (79948)', 'PLPPR4 (9890)', 'PLPPR5 (163404)', 'PLRG1 (5356)', 'PLS1 (5357)',
 'PLS3 (5358)', 'PLSCR1 (5359)', 'PLSCR2 (57047)', 'PLSCR3 (57048)', 'PLSCR4
 (57088)', 'PLSCR5 (389158)', 'PLTP (5360)', 'PLVAP (83483)', 'PLXDC1 (57125)',
 'PLXDC2 (84898)', 'PLXNA1 (5361)', 'PLXNA2 (5362)', 'PLXNA3 (55558)', 'PLXNA4
 (91584)', 'PLXNB1 (5364)', 'PLXNB2 (23654)', 'PLXNB3 (5365)', 'PLXNC1 (10154)',
 'PLXND1 (23129)', 'PM20D1 (148811)', 'PM20D2 (135293)', 'PMAIP1 (5366)', 'PMCH
 (5367)', 'PMEL (6490)', 'PMEPA1 (56937)', 'PMF1 (11243)', 'PMFBP1 (83449)', 'PML
 (5371)', 'PMM1 (5372)', 'PMM2 (5373)', 'PMP2 (5375)', 'PMP22 (5376)', 'PMPCA
 (23203)', 'PMPCB (9512)', 'PMS1 (5378)', 'PMS2 (5395)', 'PMVK (10654)', 'PNCK
 (139728)', 'PNISR (25957)', 'PNKD (25953)', 'PNKP (11284)', 'PNLDC1 (154197)',
 'PNLIP (5406)', 'PNLIPRP1 (5407)', 'PNLIPRP3 (119548)', 'PNMA1 (9240)', 'PNMA2
 (10687)', 'PNMA3 (29944)', 'PNMA5 (114824)', 'PNMA6A (84968)', 'PNMA6E
 (649238)', 'PNMA6F (105373377)', 'PNMA8A (55228)', 'PNMA8B (57469)', 'PNMT
 (5409)', 'PNN (5411)', 'PN01 (56902)', 'PNOC (5368)', 'PNP (4860)', 'PNPLA1
 (285848)', 'PNPLA2 (57104)', 'PNPLA3 (80339)', 'PNPLA4 (8228)', 'PNPLA5
 (150379)', 'PNPLA6 (10908)', 'PNPLA7 (375775)', 'PNPLA8 (50640)', 'PNPO
 (55163)', 'PNPT1 (87178)', 'PNRC1 (10957)', 'PNRC2 (55629)', 'POC1A (25886)',

'POC1B (282809)', 'POC5 (134359)', 'PODN (127435)', 'PODNL1 (79883)', 'PODXL
 (5420)', 'PODXL2 (50512)', 'POF1B (79983)', 'POFUT1 (23509)', 'POFUT2 (23275)',
 'POGK (57645)', 'POGLUT1 (56983)', 'POGLUT2 (79070)', 'POGLUT3 (143888)', 'POGZ
 (23126)', 'POLA1 (5422)', 'POLA2 (23649)', 'POLB (5423)', 'POLD1 (5424)', 'POLD2
 (5425)', 'POLD3 (10714)', 'POLD4 (57804)', 'POLDIP2 (26073)', 'POLDIP3 (84271)',
 'POLE (5426)', 'POLE2 (5427)', 'POLE3 (54107)', 'POLE4 (56655)', 'POLG (5428)',
 'POLG2 (11232)', 'POLH (5429)', 'POLI (11201)', 'POLK (51426)', 'POLL (27343)',
 'POLM (27434)', 'POLN (353497)', 'POLQ (10721)', 'POLR1A (25885)', 'POLR1B
 (84172)', 'POLR1C (9533)', 'POLR1D (51082)', 'POLR1E (64425)', 'POLR1F
 (221830)', 'POLR1G (10849)', 'POLR1H (30834)', 'POLR2B (5431)', 'POLR2C (5432)',
 'POLR2D (5433)', 'POLR2E (5434)', 'POLR2F (5435)', 'POLR2G (5436)', 'POLR2H
 (5437)', 'POLR2I (5438)', 'POLR2K (5440)', 'POLR2L (5441)', 'POLR2M (81488)',
 'POLR3A (11128)', 'POLR3B (55703)', 'POLR3C (10623)', 'POLR3D (661)', 'POLR3E
 (55718)', 'POLR3F (10621)', 'POLR3G (10622)', 'POLR3GL (84265)', 'POLR3H
 (171568)', 'POLR3K (51728)', 'POLRMT (5442)', 'POM121 (9883)', 'POM121C
 (100101267)', 'POM121L12 (285877)', 'POM121L2 (94026)', 'POMC (5443)', 'POMGNT1
 (55624)', 'POMGNT2 (84892)', 'POMK (84197)', 'POMP (51371)', 'POMT1 (10585)',
 'POMT2 (29954)', 'PON1 (5444)', 'PON2 (5445)', 'PON3 (5446)', 'POP1 (10940)',
 'POP4 (10775)', 'POP5 (51367)', 'POP7 (10248)', 'POPDC1 (11149)', 'POPDC2
 (64091)', 'POPDC3 (64208)', 'POR (5447)', 'PORCN (64840)', 'POSTN (10631)',
 'POT1 (25913)', 'POTEC (388468)', 'POTED (317754)', 'POTEE (445582)', 'POTEF
 (728378)', 'POTEG (404785)', 'POTEH (23784)', 'POTEI (653269)', 'POTEM
 (641455)', 'POU1F1 (5449)', 'POU2AF1 (5450)', 'POU2AF2 (341032)', 'POU2AF3
 (120376)', 'POU2F1 (5451)', 'POU2F2 (5452)', 'POU2F3 (25833)', 'POU3F1 (5453)',
 'POU3F2 (5454)', 'POU3F3 (5455)', 'POU3F4 (5456)', 'POU4F1 (5457)', 'POU4F2
 (5458)', 'POU4F3 (5459)', 'POU5F1 (5460)', 'POU5F1B (5462)', 'POU5F2 (134187)',
 'POU6F1 (5463)', 'POU6F2 (11281)', 'PP2D1 (151649)', 'PPA1 (5464)', 'PPA2
 (27068)', 'PPAN (56342)', 'PPARA (5465)', 'PPARD (5467)', 'PPARG (5468)',
 'PPARGC1A (10891)', 'PPARGC1B (133522)', 'PPAT (5471)', 'PPBP (5473)', 'PPCDC
 (60490)', 'PPCS (79717)', 'PPDPF (79144)', 'PPDPFL (492307)', 'PPEF1 (5475)',
 'PPEF2 (5470)', 'PPFIA1 (8500)', 'PPFIA2 (8499)', 'PPFIA3 (8541)', 'PPFIA4
 (8497)', 'PPFIBP1 (8496)', 'PPFIBP2 (8495)', 'PPHLN1 (51535)', 'PPIA (5478)',
 'PPIAL4G (644591)', 'PPIB (5479)', 'PPIC (5480)', 'PPID (5481)', 'PPIE (10450)',
 'PPIF (10105)', 'PPIG (9360)', 'PPIH (10465)', 'PPIL1 (51645)', 'PPIL2 (23759)',
 'PPIL3 (53938)', 'PPIL4 (85313)', 'PPIL6 (285755)', 'PPIP5K1 (9677)', 'PPIP5K2
 (23262)', 'PPL (5493)', 'PPM1A (5494)', 'PPM1B (5495)', 'PPM1D (8493)', 'PPM1E
 (22843)', 'PPM1F (9647)', 'PPM1G (5496)', 'PPM1H (57460)', 'PPM1J (333926)',
 'PPM1K (152926)', 'PPM1L (151742)', 'PPM1M (132160)', 'PPM1N (147699)', 'PPME1
 (51400)', 'PPOX (5498)', 'PPP1CA (5499)', 'PPP1CB (5500)', 'PPP1CC (5501)',
 'PPP1R10 (5514)', 'PPP1R11 (6992)', 'PPP1R12A (4659)', 'PPP1R12B (4660)',
 'PPP1R12C (54776)', 'PPP1R13B (23368)', 'PPP1R13L (10848)', 'PPP1R14A (94274)',
 'PPP1R14B (26472)', 'PPP1R14C (81706)', 'PPP1R14D (54866)', 'PPP1R15A (23645)',
 'PPP1R15B (84919)', 'PPP1R16A (84988)', 'PPP1R16B (26051)', 'PPP1R17 (10842)',
 'PPP1R18 (170954)', 'PPP1R1A (5502)', 'PPP1R1B (84152)', 'PPP1R1C (151242)',
 'PPP1R2 (5504)', 'PPP1R21 (129285)', 'PPP1R26 (9858)', 'PPP1R27 (116729)',
 'PPP1R2B (153743)', 'PPP1R2C (80316)', 'PPP1R35 (221908)', 'PPP1R36 (145376)',
 'PPP1R37 (284352)', 'PPP1R3A (5506)', 'PPP1R3B (79660)', 'PPP1R3C (5507)',
 'PPP1R3D (5509)', 'PPP1R3E (90673)', 'PPP1R3F (89801)', 'PPP1R3G (648791)',

'PPP1R42 (286187)', 'PPP1R7 (5510)', 'PPP1R8 (5511)', 'PPP1R9A (55607)',
 'PPP1R9B (84687)', 'PPP2CA (5515)', 'PPP2CB (5516)', 'PPP2R1A (5518)', 'PPP2R1B
 (5519)', 'PPP2R2A (5520)', 'PPP2R2B (5521)', 'PPP2R2C (5522)', 'PPP2R2D
 (55844)', 'PPP2R3A (5523)', 'PPP2R3B (28227)', 'PPP2R3C (55012)', 'PPP2R5A
 (5525)', 'PPP2R5B (5526)', 'PPP2R5C (5527)', 'PPP2R5D (5528)', 'PPP2R5E (5529)',
 'PPP3CA (5530)', 'PPP3CB (5532)', 'PPP3CC (5533)', 'PPP3R1 (5534)', 'PPP3R2
 (5535)', 'PPP4C (5531)', 'PPP4R1 (9989)', 'PPP4R2 (151987)', 'PPP4R3A (55671)',
 'PPP4R3B (57223)', 'PPP4R3C (139420)', 'PPP4R4 (57718)', 'PPP5C (5536)', 'PPP6C
 (5537)', 'PPP6R1 (22870)', 'PPP6R2 (9701)', 'PPP6R3 (55291)', 'PPRC1 (23082)',
 'PPT1 (5538)', 'PPT2 (9374)', 'PPTC7 (160760)', 'PPWD1 (23398)', 'PPY (5539)',
 'PQBP1 (10084)', 'PRAC1 (84366)', 'PRAC2 (360205)', 'PRADC1 (84279)', 'PRAF2
 (11230)', 'PRAG1 (157285)', 'PRAM1 (84106)', 'PRAME (23532)', 'PRAMEF1 (65121)',
 'PRAMEF11 (440560)', 'PRAMEF12 (390999)', 'PRAMEF14 (729528)', 'PRAMEF17
 (391004)', 'PRAMEF18 (391003)', 'PRAMEF19 (645414)', 'PRAMEF2 (65122)',
 'PRAMEF20 (645425)', 'PRAMEF4 (400735)', 'PRAP1 (118471)', 'PRB2 (653247)',
 'PRB3 (5544)', 'PRB4 (5545)', 'PRC1 (9055)', 'PRCC (5546)', 'PRCD (768206)',
 'PRCP (5547)', 'PRDM1 (639)', 'PRDM10 (56980)', 'PRDM11 (56981)', 'PRDM12
 (59335)', 'PRDM13 (59336)', 'PRDM14 (63978)', 'PRDM15 (63977)', 'PRDM16
 (63976)', 'PRDM2 (7799)', 'PRDM4 (11108)', 'PRDM5 (11107)', 'PRDM6 (93166)',
 'PRDM7 (11105)', 'PRDM8 (56978)', 'PRDM9 (56979)', 'PRDX1 (5052)', 'PRDX2
 (7001)', 'PRDX3 (10935)', 'PRDX4 (10549)', 'PRDX5 (25824)', 'PRDX6 (9588)',
 'PREB (10113)', 'PRELID1 (27166)', 'PRELID2 (153768)', 'PRELID3A (10650)',
 'PRELID3B (51012)', 'PRELP (5549)', 'PREP (5550)', 'PREPL (9581)', 'PREX1
 (57580)', 'PREX2 (80243)', 'PRF1 (5551)', 'PRG2 (5553)', 'PRG3 (10394)', 'PRG4
 (10216)', 'PRH2 (5555)', 'PRICKLE1 (144165)', 'PRICKLE2 (166336)', 'PRICKLE3
 (4007)', 'PRICKLE4 (29964)', 'PRIM1 (5557)', 'PRIM2 (5558)', 'PRIMA1 (145270)',
 'PRIMPOL (201973)', 'PRKAA1 (5562)', 'PRKAA2 (5563)', 'PRKAB1 (5564)', 'PRKAB2
 (5565)', 'PRKACA (5566)', 'PRKACB (5567)', 'PRKACG (5568)', 'PRKAG1 (5571)',
 'PRKAG2 (51422)', 'PRKAG3 (53632)', 'PRKAR1A (5573)', 'PRKAR1B (5575)', 'PRKAR2A
 (5576)', 'PRKAR2B (5577)', 'PRKCA (5578)', 'PRKCB (5579)', 'PRKCD (5580)',
 'PRKCE (5581)', 'PRKCG (5582)', 'PRKCH (5583)', 'PRKCI (5584)', 'PRKCQ (5588)',
 'PRKCSH (5589)', 'PRKCZ (5590)', 'PRKD1 (5587)', 'PRKD2 (25865)', 'PRKD3
 (23683)', 'PRKDC (5591)', 'PRKG1 (5592)', 'PRKG2 (5593)', 'PRKN (5071)', 'PRKRA
 (8575)', 'PRKRIP1 (79706)', 'PRKX (5613)', 'PRL (5617)', 'PRLH (51052)', 'PRLHR
 (2834)', 'PRLR (5618)', 'PRM1 (5619)', 'PRM2 (5620)', 'PRM3 (58531)', 'PRMT1
 (3276)', 'PRMT2 (3275)', 'PRMT3 (10196)', 'PRMT5 (10419)', 'PRMT6 (55170)',
 'PRMT7 (54496)', 'PRMT8 (56341)', 'PRMT9 (90826)', 'PRND (23627)', 'PRNP
 (5621)', 'PROB1 (389333)', 'PROC (5624)', 'PROCA1 (147011)', 'PROCR (10544)',
 'PRODH (5625)', 'PRODH2 (58510)', 'PROK1 (84432)', 'PROK2 (60675)', 'PROKR1
 (10887)', 'PROKR2 (128674)', 'PROM1 (8842)', 'PROM2 (150696)', 'PROP1 (5626)',
 'PRORP (9692)', 'PROS1 (5627)', 'PROSER1 (80209)', 'PROSER2 (254427)', 'PROSER3
 (148137)', 'PROX1 (5629)', 'PROX2 (283571)', 'PROZ (8858)', 'PRP4K (8899)',
 'PRPF18 (8559)', 'PRPF19 (27339)', 'PRPF3 (9129)', 'PRPF31 (26121)', 'PRPF38A
 (84950)', 'PRPF38B (55119)', 'PRPF39 (55015)', 'PRPF4 (9128)', 'PRPF40A
 (55660)', 'PRPF40B (25766)', 'PRPF6 (24148)', 'PRPF8 (10594)', 'PRPH (5630)',
 'PRPH2 (5961)', 'PRPS1 (5631)', 'PRPS1L1 (221823)', 'PRPS2 (5634)', 'PRPSAP1
 (5635)', 'PRPSAP2 (5636)', 'PRR11 (55771)', 'PRR12 (57479)', 'PRR13 (54458)',
 'PRR14 (78994)', 'PRR14L (253143)', 'PRR15 (222171)', 'PRR15L (79170)', 'PRR16

(51334)', 'PRR18 (285800)', 'PRR19 (284338)', 'PRR20G (100419008)', 'PRR22 (163154)', 'PRR23A (729627)', 'PRR23B (389151)', 'PRR23C (389152)', 'PRR27 (401137)', 'PRR29 (92340)', 'PRR3 (80742)', 'PRR30 (339779)', 'PRR32 (100130613)', 'PRR35 (146325)', 'PRR36 (80164)', 'PRR4 (11272)', 'PRR5 (55615)', 'PRR5L (79899)', 'PRR7 (80758)', 'PRR9 (574414)', 'PRRC1 (133619)', 'PRRC2A (7916)', 'PRRC2B (84726)', 'PRRC2C (23215)', 'PRRG1 (5638)', 'PRRG2 (5639)', 'PRRG3 (79057)', 'PRRG4 (79056)', 'PRRT1 (80863)', 'PRRT1B (642515)', 'PRRT2 (112476)', 'PRRT3 (285368)', 'PRRT4 (401399)', 'PRRX1 (5396)', 'PRRX2 (51450)', 'PRSS1 (5644)', 'PRSS12 (8492)', 'PRSS16 (10279)', 'PRSS2 (5645)', 'PRSS21 (10942)', 'PRSS22 (64063)', 'PRSS23 (11098)', 'PRSS27 (83886)', 'PRSS3 (5646)', 'PRSS33 (260429)', 'PRSS35 (167681)', 'PRSS36 (146547)', 'PRSS37 (136242)', 'PRSS38 (339501)', 'PRSS41 (360226)', 'PRSS48 (345062)', 'PRSS50 (29122)', 'PRSS53 (339105)', 'PRSS54 (221191)', 'PRSS55 (203074)', 'PRSS56 (646960)', 'PRSS57 (400668)', 'PRSS58 (136541)', 'PRSS8 (5652)', 'PRTFDC1 (56952)', 'PRTG (283659)', 'PRTN3 (5657)', 'PRUNE1 (58497)', 'PRUNE2 (158471)', 'PRX (57716)', 'PRXL2A (84293)', 'PRXL2B (127281)', 'PRXL2C (195827)', 'PSAP (5660)', 'PSAPL1 (768239)', 'PSAT1 (29968)', 'PSCA (8000)', 'PSD (5662)', 'PSD2 (84249)', 'PSD3 (23362)', 'PSD4 (23550)', 'PSEN1 (5663)', 'PSEN2 (5664)', 'PSENE1 (55851)', 'PSG1 (5669)', 'PSG11 (5680)', 'PSG2 (5670)', 'PSG3 (5671)', 'PSG4 (5672)', 'PSG5 (5673)', 'PSG6 (5675)', 'PSG8 (440533)', 'PSG9 (5678)', 'PSIP1 (11168)', 'PSKH1 (5681)', 'PSKH2 (85481)', 'PSMA1 (5682)', 'PSMA2 (5683)', 'PSMA3 (5684)', 'PSMA4 (5685)', 'PSMA5 (5686)', 'PSMA6 (5687)', 'PSMA7 (5688)', 'PSMA8 (143471)', 'PSMB1 (5689)', 'PSMB10 (5699)', 'PSMB11 (122706)', 'PSMB2 (5690)', 'PSMB3 (5691)', 'PSMB4 (5692)', 'PSMB5 (5693)', 'PSMB6 (5694)', 'PSMB7 (5695)', 'PSMB8 (5696)', 'PSMB9 (5698)', 'PSMC1 (5700)', 'PSMC2 (5701)', 'PSMC3 (5702)', 'PSMC3IP (29893)', 'PSMC4 (5704)', 'PSMC5 (5705)', 'PSMC6 (5706)', 'PSMD1 (5707)', 'PSMD10 (5716)', 'PSMD11 (5717)', 'PSMD12 (5718)', 'PSMD13 (5719)', 'PSMD14 (10213)', 'PSMD2 (5708)', 'PSMD3 (5709)', 'PSMD4 (5710)', 'PSMD5 (5711)', 'PSMD6 (9861)', 'PSMD7 (5713)', 'PSMD8 (5714)', 'PSMD9 (5715)', 'PSME1 (5720)', 'PSME2 (5721)', 'PSME3 (10197)', 'PSME3IP1 (80011)', 'PSME4 (23198)', 'PSMF1 (9491)', 'PSMG1 (8624)', 'PSMG2 (56984)', 'PSMG3 (84262)', 'PSMG4 (389362)', 'PSORS1C1 (170679)', 'PSORS1C2 (170680)', 'PSPC1 (55269)', 'PSPH (5723)', 'PSPN (5623)', 'PSRC1 (84722)', 'PSTK (118672)', 'PSTPIP1 (9051)', 'PSTPIP2 (9050)', 'PTAFR (5724)', 'PTAR1 (375743)', 'PTBP1 (5725)', 'PTBP2 (58155)', 'PTBP3 (9991)', 'PTCD1 (26024)', 'PTCD2 (79810)', 'PTCD3 (55037)', 'PTCH1 (5727)', 'PTCH2 (8643)', 'PTCHD1 (139411)', 'PTCHD4 (442213)', 'PTCRA (171558)', 'PTDSS1 (9791)', 'PTDSS2 (81490)', 'PTEN (5728)', 'PTER (9317)', 'PTF1A (256297)', 'PTGDR (5729)', 'PTGDR2 (11251)', 'PTGDS (5730)', 'PTGER1 (5731)', 'PTGER2 (5732)', 'PTGER3 (5733)', 'PTGER4 (5734)', 'PTGES (9536)', 'PTGES2 (80142)', 'PTGES3 (10728)', 'PTGES3L (100885848)', 'PTGFR (5737)', 'PTGFRN (5738)', 'PTGIR (5739)', 'PTGIS (5740)', 'PTGR1 (22949)', 'PTGR2 (145482)', 'PTGR3 (284273)', 'PTGS1 (5742)', 'PTGS2 (5743)', 'PTH (5741)', 'PTH1R (5745)', 'PTH2 (113091)', 'PTH2R (5746)', 'PTLH (5744)', 'PTK2 (5747)', 'PTK2B (2185)', 'PTK6 (5753)', 'PTK7 (5754)', 'PTMA (5757)', 'PTMS (5763)', 'PTN (5764)', 'PTOV1 (53635)', 'PTP4A1 (7803)', 'PTP4A2 (8073)', 'PTP4A3 (11156)', 'PTPA (5524)', 'PTPDC1 (138639)', 'PTPMT1 (114971)', 'PTPN1 (5770)', 'PTPN11 (5781)', 'PTPN12 (5782)', 'PTPN13 (5783)', 'PTPN14 (5784)', 'PTPN18 (26469)', 'PTPN2 (5771)', 'PTPN20 (26095)', 'PTPN21 (11099)', 'PTPN22 (26191)', 'PTPN23

(25930)', 'PTPN3 (5774)', 'PTPN4 (5775)', 'PTPN5 (84867)', 'PTPN6 (5777)',
 'PTPN7 (5778)', 'PTPN9 (5780)', 'PTPRA (5786)', 'PTPRB (5787)', 'PTPRC (5788)',
 'PTPRCAP (5790)', 'PTPRD (5789)', 'PTPRE (5791)', 'PTPRF (5792)', 'PTPRG
 (5793)', 'PTPRH (5794)', 'PTPRJ (5795)', 'PTPRK (5796)', 'PTPRM (5797)', 'PTPRN
 (5798)', 'PTPRN2 (5799)', 'PTPRO (5800)', 'PTPRQ (374462)', 'PTPRR (5801)',
 'PTPRS (5802)', 'PTPRT (11122)', 'PTPRU (10076)', 'PTPRZ1 (5803)', 'PTRH1
 (138428)', 'PTRH2 (51651)', 'PTRHD1 (391356)', 'PTS (5805)', 'PTTG1 (9232)',
 'PTTG1IP (754)', 'PTTG1IP2 (102723899)', 'PTTG2 (10744)', 'PTX3 (5806)', 'PTX4
 (390667)', 'PUDP (8226)', 'PUF60 (22827)', 'PUM1 (9698)', 'PUM2 (23369)', 'PUM3
 (9933)', 'PURA (5813)', 'PURB (5814)', 'PURG (29942)', 'PUS1 (80324)', 'PUS10
 (150962)', 'PUS3 (83480)', 'PUS7 (54517)', 'PUS7L (83448)', 'PUSL1 (126789)',
 'PVALB (5816)', 'PVALEF (388428)', 'PVR (5817)', 'PVRIG (79037)', 'PWP1
 (11137)', 'PWP2 (5822)', 'PWWP2A (114825)', 'PWWP2B (170394)', 'PWWP3A (84939)',
 'PWWP3B (139221)', 'PXDC1 (221749)', 'PXDN (7837)', 'PXDNL (137902)', 'P XK
 (54899)', 'PXMP2 (5827)', 'PXMP4 (11264)', 'PXN (5829)', 'PXT1 (222659)',
 'PXYLP1 (92370)', 'PYCARD (29108)', 'PYCR1 (5831)', 'PYCR2 (29920)', 'PYCR3
 (65263)', 'PYDC1 (260434)', 'PYDC5 (107181291)', 'PYGB (5834)', 'PYGL (5836)',
 'PYGM (5837)', 'PYG01 (26108)', 'PYG02 (90780)', 'PYHIN1 (149628)', 'PYM1
 (84305)', 'PYROXD1 (79912)', 'PYROXD2 (84795)', 'PYURF (100996939)', 'PYY
 (5697)', 'PZP (5858)', 'QARS1 (5859)', 'QDPR (5860)', 'QKI (9444)', 'QNG1
 (84267)', 'QPCT (25797)', 'QPCTL (54814)', 'QPRT (23475)', 'QRFP (347148)',
 'QRFP1 (84109)', 'QRICH1 (54870)', 'QRICH2 (84074)', 'QRSL1 (55278)', 'QSER1
 (79832)', 'QSOX1 (5768)', 'QSOX2 (169714)', 'QTRT1 (81890)', 'QTRT2 (79691)',
 'R3HCC1 (203069)', 'R3HCC1L (27291)', 'R3HDM1 (23518)', 'R3HDM2 (22864)',
 'R3HDM4 (91300)', 'R3HDML (140902)', 'RAB10 (10890)', 'RAB11A (8766)', 'RAB11B
 (9230)', 'RAB11FIP1 (80223)', 'RAB11FIP2 (22841)', 'RAB11FIP3 (9727)',
 'RAB11FIP4 (84440)', 'RAB11FIP5 (26056)', 'RAB12 (201475)', 'RAB13 (5872)',
 'RAB14 (51552)', 'RAB15 (376267)', 'RAB17 (64284)', 'RAB18 (22931)', 'RAB19
 (401409)', 'RAB1A (5861)', 'RAB1B (81876)', 'RAB20 (55647)', 'RAB21 (23011)',
 'RAB22A (57403)', 'RAB23 (51715)', 'RAB24 (53917)', 'RAB25 (57111)', 'RAB26
 (25837)', 'RAB27A (5873)', 'RAB27B (5874)', 'RAB28 (9364)', 'RAB29 (8934)',
 'RAB2A (5862)', 'RAB2B (84932)', 'RAB30 (27314)', 'RAB31 (11031)', 'RAB32
 (10981)', 'RAB33A (9363)', 'RAB33B (83452)', 'RAB34 (83871)', 'RAB35 (11021)',
 'RAB36 (9609)', 'RAB37 (326624)', 'RAB38 (23682)', 'RAB39A (54734)', 'RAB39B
 (116442)', 'RAB3A (5864)', 'RAB3B (5865)', 'RAB3C (115827)', 'RAB3D (9545)',
 'RAB3GAP1 (22930)', 'RAB3GAP2 (25782)', 'RAB3IL1 (5866)', 'RAB3IP (117177)',
 'RAB40A (142684)', 'RAB40AL (282808)', 'RAB40B (10966)', 'RAB40C (57799)',
 'RAB41 (347517)', 'RAB42 (115273)', 'RAB43 (339122)', 'RAB44 (401258)', 'RAB4A
 (5867)', 'RAB4B (53916)', 'RAB5A (5868)', 'RAB5B (5869)', 'RAB5C (5878)',
 'RAB5IF (55969)', 'RAB6A (5870)', 'RAB6B (51560)', 'RAB7A (7879)', 'RAB7B
 (338382)', 'RAB8A (4218)', 'RAB8B (51762)', 'RAB9A (9367)', 'RAB9B (51209)',
 'RABAC1 (10567)', 'RABEP1 (9135)', 'RABEP2 (79874)', 'RABEPK (10244)', 'RABGAP1
 (23637)', 'RABGAP1L (9910)', 'RABGEF1 (27342)', 'RABGGTA (5875)', 'RABGGTB
 (5876)', 'RABIF (5877)', 'RABL2A (11159)', 'RABL2B (11158)', 'RABL3 (285282)',
 'RABL6 (55684)', 'RAC1 (5879)', 'RAC2 (5880)', 'RAC3 (5881)', 'RACGAP1 (29127)',
 'RACK1 (10399)', 'RAD1 (5810)', 'RAD17 (5884)', 'RAD18 (56852)', 'RAD21 (5885)',
 'RAD21L1 (642636)', 'RAD23A (5886)', 'RAD23B (5887)', 'RAD50 (10111)', 'RAD51
 (5888)', 'RAD51AP1 (10635)', 'RAD51AP2 (729475)', 'RAD51B (5890)', 'RAD51C

(5889)', 'RAD51D (5892)', 'RAD52 (5893)', 'RAD54B (25788)', 'RAD54L (8438)',
 'RAD54L2 (23132)', 'RAD9A (5883)', 'RAD9B (144715)', 'RADIL (55698)', 'RADX
 (55086)', 'RAE1 (8480)', 'RAET1E (135250)', 'RAET1G (353091)', 'RAET1L
 (154064)', 'RAF1 (5894)', 'RAG1 (5896)', 'RAG2 (5897)', 'RAI1 (10743)', 'RAI14
 (26064)', 'RAI2 (10742)', 'RALA (5898)', 'RALB (5899)', 'RALBP1 (10928)',
 'RALGAPA1 (253959)', 'RALGAPA2 (57186)', 'RALGAPB (57148)', 'RALGDS (5900)',
 'RALGPS1 (9649)', 'RALGPS2 (55103)', 'RALY (22913)', 'RALYL (138046)', 'RAMP1
 (10267)', 'RAMP2 (10266)', 'RAMP3 (10268)', 'RAN (5901)', 'RANBP1 (5902)',
 'RANBP10 (57610)', 'RANBP17 (64901)', 'RANBP2 (5903)', 'RANBP3 (8498)', 'RANBP3L
 (202151)', 'RANBP6 (26953)', 'RANBP9 (10048)', 'RANGAP1 (5905)', 'RANGRF
 (29098)', 'RAP1A (5906)', 'RAP1B (5908)', 'RAP1GAP (5909)', 'RAP1GAP2 (23108)',
 'RAP1GDS1 (5910)', 'RAP2A (5911)', 'RAP2B (5912)', 'RAP2C (57826)', 'RAPGEF1
 (2889)', 'RAPGEF2 (9693)', 'RAPGEF3 (10411)', 'RAPGEF4 (11069)', 'RAPGEF5
 (9771)', 'RAPGEF6 (51735)', 'RAPGEFL1 (51195)', 'RAPH1 (65059)', 'RAPSN (5913)',
 'RARA (5914)', 'RARB (5915)', 'RARG (5916)', 'RARRES1 (5918)', 'RARRES2 (5919)',
 'RARS1 (5917)', 'RARS2 (57038)', 'RASA1 (5921)', 'RASA2 (5922)', 'RASA3
 (22821)', 'RASA4 (10156)', 'RASA4B (100271927)', 'RASAL1 (8437)', 'RASAL2
 (9462)', 'RASAL3 (64926)', 'RASD1 (51655)', 'RASD2 (23551)', 'RASEF (158158)',
 'RASGEF1A (221002)', 'RASGEF1B (153020)', 'RASGEF1C (255426)', 'RASGRF1 (5923)',
 'RASGRF2 (5924)', 'RASGRP1 (10125)', 'RASGRP2 (10235)', 'RASGRP3 (25780)',
 'RASGRP4 (115727)', 'RASIP1 (54922)', 'RASL10A (10633)', 'RASL10B (91608)',
 'RASL11A (387496)', 'RASL11B (65997)', 'RASL12 (51285)', 'RASSF1 (11186)',
 'RASSF10 (644943)', 'RASSF2 (9770)', 'RASSF3 (283349)', 'RASSF4 (83937)',
 'RASSF5 (83593)', 'RASSF6 (166824)', 'RASSF7 (8045)', 'RASSF8 (11228)', 'RASSF9
 (9182)', 'RAVER1 (125950)', 'RAVER2 (55225)', 'RAX (30062)', 'RAX2 (84839)',
 'RB1 (5925)', 'RB1CC1 (9821)', 'RBAK (57786)', 'RBBP4 (5928)', 'RBBP5 (5929)',
 'RBBP6 (5930)', 'RBBP7 (5931)', 'RBBP8 (5932)', 'RBBP8NL (140893)', 'RBBP9
 (10741)', 'RBCK1 (10616)', 'RBFA (79863)', 'RBFOX1 (54715)', 'RBFOX2 (23543)',
 'RBFOX3 (146713)', 'RBIS (401466)', 'RBKS (64080)', 'RBL1 (5933)', 'RBL2
 (5934)', 'RBM10 (8241)', 'RBM11 (54033)', 'RBM12 (10137)', 'RBM12B (389677)',
 'RBM14 (10432)', 'RBM15 (64783)', 'RBM15B (29890)', 'RBM17 (84991)', 'RBM18
 (92400)', 'RBM19 (9904)', 'RBM20 (282996)', 'RBM22 (55696)', 'RBM23 (55147)',
 'RBM24 (221662)', 'RBM25 (58517)', 'RBM26 (64062)', 'RBM27 (54439)', 'RBM28
 (55131)', 'RBM3 (5935)', 'RBM33 (155435)', 'RBM34 (23029)', 'RBM38 (55544)',
 'RBM39 (9584)', 'RBM4 (5936)', 'RBM41 (55285)', 'RBM42 (79171)', 'RBM43
 (375287)', 'RBM44 (375316)', 'RBM45 (129831)', 'RBM46 (166863)', 'RBM47
 (54502)', 'RBM48 (84060)', 'RBM4B (83759)', 'RBM5 (10181)', 'RBM6 (10180)',
 'RBM7 (10179)', 'RBM8A (9939)', 'RBMS1 (5937)', 'RBMS2 (5939)', 'RBMS3 (27303)',
 'RBMX (27316)', 'RBMX2 (51634)', 'RBMXL1 (494115)', 'RBMXL2 (27288)', 'RBMXL3
 (139804)', 'RBP1 (5947)', 'RBP2 (5948)', 'RBP3 (5949)', 'RBP4 (5950)', 'RBP5
 (83758)', 'RBP7 (116362)', 'RBPJ (3516)', 'RBPJL (11317)', 'RBPMS (11030)',
 'RBPMS2 (348093)', 'RBSN (64145)', 'RBX1 (9978)', 'RC3H1 (149041)', 'RC3H2
 (54542)', 'RCAN1 (1827)', 'RCAN2 (10231)', 'RCAN3 (11123)', 'RCBTB1 (55213)',
 'RCBTB2 (1102)', 'RCC1 (1104)', 'RCC1L (81554)', 'RCC2 (55920)', 'RCCD1
 (91433)', 'RCE1 (9986)', 'RCHY1 (25898)', 'RCL1 (10171)', 'RCN1 (5954)', 'RCN2
 (5955)', 'RCN3 (57333)', 'RCOR1 (23186)', 'RCOR2 (283248)', 'RCOR3 (55758)',
 'RCSD1 (92241)', 'RCVRN (5957)', 'RD3 (343035)', 'RD3L (647286)', 'RDH10
 (157506)', 'RDH11 (51109)', 'RDH12 (145226)', 'RDH13 (112724)', 'RDH14 (57665)',

'RDH16 (8608)', 'RDH5 (5959)', 'RDH8 (50700)', 'RDM1 (201299)', 'RDX (5962)',
 'REC114 (283677)', 'REC8 (9985)', 'RECK (8434)', 'RECQL (5965)', 'RECQL4
 (9401)', 'RECQL5 (9400)', 'REDIC1 (283461)', 'REELD1 (345051)', 'REEP1 (65055)',
 'REEP2 (51308)', 'REEP3 (221035)', 'REEP4 (80346)', 'REEP5 (7905)', 'REEP6
 (92840)', 'REG1A (5967)', 'REG1B (5968)', 'REG3A (5068)', 'REG3G (130120)',
 'REG4 (83998)', 'REL (5966)', 'RELA (5970)', 'RELB (5971)', 'RELCH (57614)',
 'RELL1 (768211)', 'RELL2 (285613)', 'RELN (5649)', 'RELT (84957)', 'REM1
 (28954)', 'REM2 (161253)', 'REN (5972)', 'RENB (5973)', 'REP15 (387849)',
 'REPIN1 (29803)', 'REPS1 (85021)', 'REPS2 (9185)', 'RER1 (11079)', 'RERE (473)',
 'RERG (85004)', 'RERGL (79785)', 'RESF1 (55196)', 'RESP18 (389075)', 'REST
 (5978)', 'RET (5979)', 'RETN (56729)', 'RETNLB (84666)', 'RETREG1 (54463)',
 'RETREG2 (79137)', 'RETREG3 (162427)', 'RETSAT (54884)', 'REV1 (51455)', 'REV3L
 (5980)', 'REX1BD (55049)', 'REX01 (57455)', 'REX02 (25996)', 'REX04 (57109)',
 'REX05 (81691)', 'RFC1 (5981)', 'RFC2 (5982)', 'RFC3 (5983)', 'RFC4 (5984)',
 'RFC5 (5985)', 'RFESD (317671)', 'RFFL (117584)', 'RFK (55312)', 'RFLNA
 (144347)', 'RFNG (5986)', 'RFPL1 (5988)', 'RFPL2 (10739)', 'RFPL3 (10738)',
 'RFPL4B (442247)', 'RFT1 (91869)', 'RFTN1 (23180)', 'RFTN2 (130132)', 'RFWD3
 (55159)', 'RFX1 (5989)', 'RFX2 (5990)', 'RFX3 (5991)', 'RFX4 (5992)', 'RFX5
 (5993)', 'RFX6 (222546)', 'RFX7 (64864)', 'RFX8 (731220)', 'RFXANK (8625)',
 'RFXAP (5994)', 'RGCC (28984)', 'RGL1 (23179)', 'RGL2 (5863)', 'RGL3 (57139)',
 'RGL4 (266747)', 'RGMA (56963)', 'RGMB (285704)', 'RGN (9104)', 'RGP1 (9827)',
 'RGPD4 (285190)', 'RGR (5995)', 'RGS1 (5996)', 'RGS10 (6001)', 'RGS11 (8786)',
 'RGS12 (6002)', 'RGS13 (6003)', 'RGS14 (10636)', 'RGS16 (6004)', 'RGS17
 (26575)', 'RGS18 (64407)', 'RGS19 (10287)', 'RGS2 (5997)', 'RGS20 (8601)',
 'RGS21 (431704)', 'RGS22 (26166)', 'RGS3 (5998)', 'RGS4 (5999)', 'RGS5 (8490)',
 'RGS6 (9628)', 'RGS7 (6000)', 'RGS7BP (401190)', 'RGS8 (85397)', 'RGS9 (8787)',
 'RGS9BP (388531)', 'RGSL1 (353299)', 'RHAG (6005)', 'RHBDD1 (84236)', 'RHBDD2
 (57414)', 'RHBDD3 (25807)', 'RHBDF1 (64285)', 'RHBDF2 (79651)', 'RHBDL1 (9028)',
 'RHBDL2 (54933)', 'RHBDL3 (162494)', 'RHBG (57127)', 'RHCE (6006)', 'RHCG
 (51458)', 'RHD (6007)', 'RHEB (6009)', 'RHEBL1 (121268)', 'RHEX (440712)',
 'RHNO1 (83695)', 'RHO (6010)', 'RHOA (387)', 'RHOB (388)', 'RHOBTB1 (9886)',
 'RHOBTB2 (23221)', 'RHOBTB3 (22836)', 'RHOC (389)', 'RHOD (29984)', 'RHOF
 (54509)', 'RHOG (391)', 'RHOH (399)', 'RHOJ (57381)', 'RHOQ (23433)', 'RHOT1
 (55288)', 'RHOT2 (89941)', 'RHOV (58480)', 'RHOV (171177)', 'RHOXF1 (158800)',
 'RHOXF2 (84528)', 'RHOXF2B (727940)', 'RHPN1 (114822)', 'RHPN2 (85415)', 'RIBC1
 (158787)', 'RIBC2 (26150)', 'RIC1 (57589)', 'RIC3 (79608)', 'RIC8A (60626)',
 'RIC8B (55188)', 'RICKOR (253260)', 'RIDA (10247)', 'RIF1 (55183)', 'RIGI
 (23586)', 'RIIAD1 (284485)', 'RILP (83547)', 'RILPL1 (353116)', 'RILPL2
 (196383)', 'RIMBP2 (23504)', 'RIMBP3 (85376)', 'RIMKLA (284716)', 'RIMKLB
 (57494)', 'RIMOC1 (285636)', 'RIMS1 (22999)', 'RIMS2 (9699)', 'RIMS3 (9783)',
 'RIMS4 (140730)', 'RIN1 (9610)', 'RIN2 (54453)', 'RIN3 (79890)', 'RING1 (6015)',
 'RINL (126432)', 'RINT1 (60561)', 'RIOK1 (83732)', 'RIOK2 (55781)', 'RIOK3
 (8780)', 'RIOX1 (79697)', 'RIOX2 (84864)', 'RIPK1 (8737)', 'RIPK2 (8767)',
 'RIPK3 (11035)', 'RIPK4 (54101)', 'RIPOR1 (79567)', 'RIPOR2 (9750)', 'RIPOR3
 (140876)', 'RIPPLY1 (92129)', 'RIPPLY2 (134701)', 'RIPPLY3 (53820)', 'RIT1
 (6016)', 'RIT2 (6014)', 'RITA1 (84934)', 'RLBP1 (6017)', 'RLF (6018)', 'RLIG1
 (91298)', 'RLIM (51132)', 'RLN1 (6013)', 'RLN2 (6019)', 'RLN3 (117579)', 'RMC1
 (29919)', 'RMDN1 (51115)', 'RMDN2 (151393)', 'RMDN3 (55177)', 'RMI1 (80010)',

'RMI2 (116028)', 'RMND1 (55005)', 'RMND5A (64795)', 'RMND5B (64777)', 'RMP24
 (83608)', 'RMP64 (25871)', 'RNASE1 (6035)', 'RNASE10 (338879)', 'RNASE11
 (122651)', 'RNASE12 (493901)', 'RNASE13 (440163)', 'RNASE2 (6036)', 'RNASE3
 (6037)', 'RNASE4 (6038)', 'RNASE6 (6039)', 'RNASE7 (84659)', 'RNASE8 (122665)',
 'RNASE9 (390443)', 'RNASEH1 (246243)', 'RNASEH2A (10535)', 'RNASEH2B (79621)',
 'RNASEH2C (84153)', 'RNASEK (440400)', 'RNASEL (6041)', 'RNASET2 (8635)', 'RND1
 (27289)', 'RND2 (8153)', 'RND3 (390)', 'RNF10 (9921)', 'RNF103 (7844)', 'RNF11
 (26994)', 'RNF111 (54778)', 'RNF112 (7732)', 'RNF113A (7737)', 'RNF113B
 (140432)', 'RNF114 (55905)', 'RNF115 (27246)', 'RNF121 (55298)', 'RNF122
 (79845)', 'RNF123 (63891)', 'RNF125 (54941)', 'RNF126 (55658)', 'RNF128
 (79589)', 'RNF13 (11342)', 'RNF130 (55819)', 'RNF133 (168433)', 'RNF135
 (84282)', 'RNF138 (51444)', 'RNF139 (11236)', 'RNF14 (9604)', 'RNF141 (50862)',
 'RNF144A (9781)', 'RNF144B (255488)', 'RNF145 (153830)', 'RNF146 (81847)',
 'RNF148 (378925)', 'RNF149 (284996)', 'RNF150 (57484)', 'RNF151 (146310)',
 'RNF152 (220441)', 'RNF157 (114804)', 'RNF166 (115992)', 'RNF167 (26001)',
 'RNF168 (165918)', 'RNF169 (254225)', 'RNF17 (56163)', 'RNF170 (81790)', 'RNF175
 (285533)', 'RNF180 (285671)', 'RNF181 (51255)', 'RNF182 (221687)', 'RNF183
 (138065)', 'RNF185 (91445)', 'RNF186 (54546)', 'RNF187 (149603)', 'RNF19A
 (25897)', 'RNF19B (127544)', 'RNF2 (6045)', 'RNF20 (56254)', 'RNF207 (388591)',
 'RNF208 (727800)', 'RNF212 (285498)', 'RNF212B (100507650)', 'RNF213 (57674)',
 'RNF214 (257160)', 'RNF215 (200312)', 'RNF216 (54476)', 'RNF217 (154214)',
 'RNF220 (55182)', 'RNF222 (643904)', 'RNF223 (401934)', 'RNF224 (643596)',
 'RNF225 (646862)', 'RNF227 (284023)', 'RNF24 (11237)', 'RNF25 (64320)', 'RNF26
 (79102)', 'RNF31 (55072)', 'RNF32 (140545)', 'RNF34 (80196)', 'RNF38 (152006)',
 'RNF39 (80352)', 'RNF4 (6047)', 'RNF40 (9810)', 'RNF41 (10193)', 'RNF43
 (54894)', 'RNF44 (22838)', 'RNF5 (6048)', 'RNF6 (6049)', 'RNF7 (9616)', 'RNF8
 (9025)', 'RNFT1 (51136)', 'RNFT2 (84900)', 'RNGTT (8732)', 'RNH1 (6050)', 'RNLS
 (55328)', 'RNMT (8731)', 'RNPC3 (55599)', 'RNPEP (6051)', 'RNPS1 (10921)', 'RO60
 (6738)', 'ROBO1 (6091)', 'ROBO2 (6092)', 'ROBO3 (64221)', 'ROBO4 (54538)',
 'ROCK1 (6093)', 'ROCK2 (9475)', 'ROGDI (79641)', 'ROM1 (6094)', 'ROM1
 (140823)', 'ROPN1 (54763)', 'ROPN1B (152015)', 'ROPN1L (83853)', 'ROR1 (4919)',
 'ROR2 (4920)', 'RORA (6095)', 'RORB (6096)', 'RORC (6097)', 'ROS1 (6098)', 'RP1
 (6101)', 'RP1L1 (94137)', 'RP2 (6102)', 'RP9 (6100)', 'RPA1 (6117)', 'RPA2
 (6118)', 'RPA3 (6119)', 'RPA4 (29935)', 'RPAIN (84268)', 'RPAP1 (26015)', 'RPAP2
 (79871)', 'RPAP3 (79657)', 'RPE (6120)', 'RPE65 (6121)', 'RPEL1 (729020)', 'RPF1
 (80135)', 'RPF2 (84154)', 'RPGR (6103)', 'RPGRIP1 (57096)', 'RPGRIP1L (23322)',
 'RPH3A (22895)', 'RPH3AL (9501)', 'RPIA (22934)', 'RPL10 (6134)', 'RPL10A
 (4736)', 'RPL10L (140801)', 'RPL11 (6135)', 'RPL12 (6136)', 'RPL13 (6137)',
 'RPL13A (23521)', 'RPL14 (9045)', 'RPL15 (6138)', 'RPL17 (6139)', 'RPL18
 (6141)', 'RPL18A (6142)', 'RPL19 (6143)', 'RPL21 (6144)', 'RPL22 (6146)',
 'RPL22L1 (200916)', 'RPL23 (9349)', 'RPL23A (6147)', 'RPL24 (6152)', 'RPL26
 (6154)', 'RPL26L1 (51121)', 'RPL27 (6155)', 'RPL27A (6157)', 'RPL28 (6158)',
 'RPL3 (6122)', 'RPL30 (6156)', 'RPL31 (6160)', 'RPL32 (6161)', 'RPL34 (6164)',
 'RPL35 (11224)', 'RPL35A (6165)', 'RPL36 (25873)', 'RPL36A (6173)', 'RPL36AL
 (6166)', 'RPL37 (6167)', 'RPL37A (6168)', 'RPL38 (6169)', 'RPL39 (6170)',
 'RPL39L (116832)', 'RPL3L (6123)', 'RPL4 (6124)', 'RPL41 (6171)', 'RPL5 (6125)',
 'RPL6 (6128)', 'RPL7 (6129)', 'RPL7A (6130)', 'RPL7L1 (285855)', 'RPL8 (6132)',
 'RPL9 (6133)', 'RPLP0 (6175)', 'RPLP1 (6176)', 'RPLP2 (6181)', 'RPN1 (6184)',

'RPN2 (6185)', 'RPP14 (11102)', 'RPP21 (79897)', 'RPP25 (54913)', 'RPP25L
 (138716)', 'RPP30 (10556)', 'RPP38 (10557)', 'RPP40 (10799)', 'RPRD1A (55197)',
 'RPRD1B (58490)', 'RPRD2 (23248)', 'RPRM (56475)', 'RPRML (388394)', 'RPS10
 (6204)', 'RPS11 (6205)', 'RPS12 (6206)', 'RPS13 (6207)', 'RPS14 (6208)', 'RPS15
 (6209)', 'RPS15A (6210)', 'RPS16 (6217)', 'RPS17 (6218)', 'RPS18 (6222)', 'RPS19
 (6223)', 'RPS19BP1 (91582)', 'RPS2 (6187)', 'RPS20 (6224)', 'RPS21 (6227)',
 'RPS23 (6228)', 'RPS24 (6229)', 'RPS25 (6230)', 'RPS27A (6233)', 'RPS27L
 (51065)', 'RPS28 (6234)', 'RPS29 (6235)', 'RPS3 (6188)', 'RPS3A (6189)', 'RPS4X
 (6191)', 'RPS4Y1 (6192)', 'RPS4Y2 (140032)', 'RPS5 (6193)', 'RPS6 (6194)',
 'RPS6KA1 (6195)', 'RPS6KA2 (6196)', 'RPS6KA3 (6197)', 'RPS6KA4 (8986)', 'RPS6KA5
 (9252)', 'RPS6KA6 (27330)', 'RPS6KB1 (6198)', 'RPS6KB2 (6199)', 'RPS6KC1
 (26750)', 'RPS6KL1 (83694)', 'RPS7 (6201)', 'RPS8 (6202)', 'RPS9 (6203)', 'RPSA
 (3921)', 'RPTN (126638)', 'RPTOR (57521)', 'RPUSD1 (113000)', 'RPUSD2 (27079)',
 'RPUSD3 (285367)', 'RPUSD4 (84881)', 'RRAD (6236)', 'RRAGA (10670)', 'RRAGB
 (10325)', 'RRAGC (64121)', 'RRAGD (58528)', 'RRAS (6237)', 'RRAS2 (22800)',
 'RRBP1 (6238)', 'RREB1 (6239)', 'RRH (10692)', 'RRM1 (6240)', 'RRM2 (6241)',
 'RRM2B (50484)', 'RRN3 (54700)', 'RRP1 (8568)', 'RRP12 (23223)', 'RRP15
 (51018)', 'RRP1B (23076)', 'RRP36 (88745)', 'RRP7A (27341)', 'RRP8 (23378)',
 'RRP9 (9136)', 'RRS1 (23212)', 'RS1 (6247)', 'RSAD1 (55316)', 'RSAD2 (91543)',
 'RSBN1 (54665)', 'RSBN1L (222194)', 'RSC1A1 (6248)', 'RSF1 (51773)', 'RSL1D1
 (26156)', 'RSL24D1 (51187)', 'RSPH1 (89765)', 'RSPH14 (27156)', 'RSPH3 (83861)',
 'RSPH4A (345895)', 'RSPH6A (81492)', 'RSPH9 (221421)', 'RSP01 (284654)', 'RSP02
 (340419)', 'RSP03 (84870)', 'RSP04 (343637)', 'RSPRY1 (89970)', 'RSRC1 (51319)',
 'RSRC2 (65117)', 'RSRP1 (57035)', 'RSU1 (6251)', 'RTBDN (83546)', 'RTCA (8634)',
 'RTCB (51493)', 'RTEL1 (51750)', 'RTF1 (23168)', 'RTF2 (51507)', 'RTKN (6242)',
 'RTKN2 (219790)', 'RTL1 (388015)', 'RTL10 (79680)', 'RTL3 (203430)', 'RTL4
 (340595)', 'RTL5 (340526)', 'RTL6 (84247)', 'RTL8A (26071)', 'RTL8B (441518)',
 'RTL8C (8933)', 'RTL9 (57529)', 'RTN1 (6252)', 'RTN2 (6253)', 'RTN3 (10313)',
 'RTN4 (57142)', 'RTN4IP1 (84816)', 'RTN4R (65078)', 'RTN4RL1 (146760)', 'RTN4RL2
 (349667)', 'RTP1 (132112)', 'RTP2 (344892)', 'RTP3 (83597)', 'RTP4 (64108)',
 'RTP5 (285093)', 'RTRAF (51637)', 'RTTN (25914)', 'RUBCN (9711)', 'RUBCNL
 (80183)', 'RUFY1 (80230)', 'RUFY2 (55680)', 'RUFY3 (22902)', 'RUNDC1 (146923)',
 'RUNDC3A (10900)', 'RUNDC3B (154661)', 'RUNX1 (861)', 'RUNX1T1 (862)', 'RUNX2
 (860)', 'RUNX3 (864)', 'RUSC1 (23623)', 'RUSC2 (9853)', 'RUSF1 (64755)', 'RUVBL1
 (8607)', 'RUVBL2 (10856)', 'RWDD1 (51389)', 'RWDD2A (112611)', 'RWDD2B (10069)',
 'RWDD3 (25950)', 'RWDD4 (201965)', 'RXFP1 (59350)', 'RXFP2 (122042)', 'RXFP3
 (51289)', 'RXFP4 (339403)', 'RXRA (6256)', 'RXRB (6257)', 'RXRG (6258)', 'RXYL1
 (10329)', 'RYK (6259)', 'RYR1 (6261)', 'RYR2 (6262)', 'RYR3 (6263)', 'S100A1
 (6271)', 'S100A10 (6281)', 'S100A11 (6282)', 'S100A12 (6283)', 'S100A13 (6284)',
 'S100A14 (57402)', 'S100A16 (140576)', 'S100A2 (6273)', 'S100A3 (6274)', 'S100A4
 (6275)', 'S100A5 (6276)', 'S100A6 (6277)', 'S100A7 (6278)', 'S100A7A (338324)',
 'S100A8 (6279)', 'S100A9 (6280)', 'S100B (6285)', 'S100G (795)', 'S100P (6286)',
 'S100PBP (64766)', 'S100Z (170591)', 'S1PR1 (1901)', 'S1PR2 (9294)', 'S1PR3
 (1903)', 'S1PR4 (8698)', 'S1PR5 (53637)', 'SAA2 (6289)', 'SAA4 (6291)', 'SAAL1
 (113174)', 'SAC3D1 (29901)', 'SACM1L (22908)', 'SACS (26278)', 'SAE1 (10055)',
 'SAFB (6294)', 'SAFB2 (9667)', 'SAG (6295)', 'SAGE1 (55511)', 'SALL1 (6299)',
 'SALL2 (6297)', 'SALL3 (27164)', 'SALL4 (57167)', 'SAMD10 (140700)', 'SAMD11
 (148398)', 'SAMD12 (401474)', 'SAMD13 (148418)', 'SAMD14 (201191)', 'SAMD15

(161394)', 'SAMD3 (154075)', 'SAMD4A (23034)', 'SAMD4B (55095)', 'SAMD5
 (389432)', 'SAMD7 (344658)', 'SAMD8 (142891)', 'SAMD9 (54809)', 'SAMD9L
 (219285)', 'SAMHD1 (25939)', 'SMM50 (25813)', 'SAMSNI (64092)', 'SAMTOR
 (154743)', 'SANBR (84542)', 'SAP130 (79595)', 'SAP18 (10284)', 'SAP25
 (100316904)', 'SAP30 (8819)', 'SAP30BP (29115)', 'SAP30L (79685)', 'SAPCD1
 (401251)', 'SAPCD2 (89958)', 'SAR1A (56681)', 'SAR1B (51128)', 'SARAF (51669)',
 'SARDH (1757)', 'SARM1 (23098)', 'SARNP (84324)', 'SARS1 (6301)', 'SARS2
 (54938)', 'SART1 (9092)', 'SART3 (9733)', 'SASH1 (23328)', 'SASH3 (54440)',
 'SASS6 (163786)', 'SAT1 (6303)', 'SAT2 (112483)', 'SATB1 (6304)', 'SATB2
 (23314)', 'SATL1 (340562)', 'SAV1 (60485)', 'SAXO1 (158297)', 'SAXO2 (283726)',
 'SAXO4 (220004)', 'SAXO5 (374877)', 'SAYS1 (55776)', 'SBDS (51119)', 'SBF1
 (6305)', 'SBF2 (81846)', 'SBK1 (388228)', 'SBK2 (646643)', 'SBK3 (100130827)',
 'SBN01 (55206)', 'SBN02 (22904)', 'SBSN (374897)', 'SBSPON (157869)', 'SC5D
 (6309)', 'SCAF1 (58506)', 'SCAF11 (9169)', 'SCAF4 (57466)', 'SCAF8 (22828)',
 'SCAI (286205)', 'SCAMP1 (9522)', 'SCAMP2 (10066)', 'SCAMP3 (10067)', 'SCAMP4
 (113178)', 'SCAMP5 (192683)', 'SCAND1 (51282)', 'SCAND3 (114821)', 'SCAP
 (22937)', 'SCAPER (49855)', 'SCARA3 (51435)', 'SCARA5 (286133)', 'SCARB1 (949)',
 'SCARB2 (950)', 'SCARF1 (8578)', 'SCARF2 (91179)', 'SCCPDH (51097)', 'SCD
 (6319)', 'SCD5 (79966)', 'SCEL (8796)', 'SCFD1 (23256)', 'SCFD2 (152579)', 'SCG2
 (7857)', 'SCG3 (29106)', 'SCG5 (6447)', 'SCGB1A1 (7356)', 'SCGB1D1 (10648)',
 'SCGB1D2 (10647)', 'SCGB1D4 (404552)', 'SCGB2A1 (4246)', 'SCGB2A2 (4250)',
 'SCGB2B2 (284402)', 'SCGB3A1 (92304)', 'SCGB3A2 (117156)', 'SCGN (10590)',
 'SCHIP1 (29970)', 'SCIMP (388325)', 'SCIN (85477)', 'SCLT1 (132320)', 'SCLY
 (51540)', 'SCMH1 (22955)', 'SCML1 (6322)', 'SCML2 (10389)', 'SCML4 (256380)',
 'SCN10A (6336)', 'SCN11A (11280)', 'SCN1A (6323)', 'SCN1B (6324)', 'SCN2A
 (6326)', 'SCN2B (6327)', 'SCN3A (6328)', 'SCN3B (55800)', 'SCN4A (6329)', 'SCN4B
 (6330)', 'SCN5A (6331)', 'SCN7A (6332)', 'SCN8A (6334)', 'SCN9A (6335)', 'SCNM1
 (79005)', 'SCNN1A (6337)', 'SCNN1B (6338)', 'SCNN1D (6339)', 'SCNN1G (6340)',
 'SC01 (6341)', 'SC02 (9997)', 'SC0C (60592)', 'SCP2 (6342)', 'SCP2D1 (140856)',
 'SCPEP1 (59342)', 'SCRG1 (11341)', 'SCRIB (23513)', 'SCRN1 (9805)', 'SCRN2
 (90507)', 'SCRN3 (79634)', 'SCRT1 (83482)', 'SCRT2 (85508)', 'SCT (6343)', 'SCTR
 (6344)', 'SCUBE1 (80274)', 'SCUBE2 (57758)', 'SCUBE3 (222663)', 'SCX (642658)',
 'SCYL1 (57410)', 'SCYL2 (55681)', 'SCYL3 (57147)', 'SDAD1 (55153)', 'SDC1
 (6382)', 'SDC2 (6383)', 'SDC3 (9672)', 'SDC4 (6385)', 'SDCBP (6386)', 'SDCBP2
 (27111)', 'SDCCAG8 (10806)', 'SDE2 (163859)', 'SDF2 (6388)', 'SDF2L1 (23753)',
 'SDF4 (51150)', 'SDHA (6389)', 'SDHAF1 (644096)', 'SDHAF2 (54949)', 'SDHAF3
 (57001)', 'SDHAF4 (135154)', 'SDHB (6390)', 'SDHC (6391)', 'SDHD (6392)', 'SDK1
 (221935)', 'SDK2 (54549)', 'SDR16C5 (195814)', 'SDR39U1 (56948)', 'SDR42E1
 (93517)', 'SDR42E2 (100288072)', 'SDR9C7 (121214)', 'SDS (10993)', 'SDSL
 (113675)', 'SEBOX (645832)', 'SEC11A (23478)', 'SEC11C (90701)', 'SEC13 (6396)',
 'SEC14L1 (6397)', 'SEC14L2 (23541)', 'SEC14L3 (266629)', 'SEC14L4 (284904)',
 'SEC14L5 (9717)', 'SEC14L6 (730005)', 'SEC16A (9919)', 'SEC16B (89866)', 'SEC22A
 (26984)', 'SEC22B (9554)', 'SEC22C (9117)', 'SEC23A (10484)', 'SEC23B (10483)',
 'SEC23IP (11196)', 'SEC24A (10802)', 'SEC24B (10427)', 'SEC24C (9632)', 'SEC24D
 (9871)', 'SEC31A (22872)', 'SEC31B (25956)', 'SEC61A1 (29927)', 'SEC61A2
 (55176)', 'SEC61B (10952)', 'SEC61G (23480)', 'SEC62 (7095)', 'SEC63 (11231)',
 'SECISBP2 (79048)', 'SECISBP2L (9728)', 'SECTM1 (6398)', 'SEH1L (81929)', 'SEL1L
 (6400)', 'SEL1L2 (80343)', 'SEL1L3 (23231)', 'SELE (6401)', 'SELENBP1 (8991)',

'SELENOF (9403)', 'SELENOH (280636)', 'SELENOI (85465)', 'SELENOK (58515)',
 'SELENOM (140606)', 'SELENON (57190)', 'SELENOO (83642)', 'SELENOP (6414)',
 'SELENOS (55829)', 'SELENOT (51714)', 'SELENOV (348303)', 'SELENOW (6415)',
 'SELL (6402)', 'SELP (6403)', 'SELPLG (6404)', 'SEM1 (7979)', 'SEMA3A (10371)',
 'SEMA3B (7869)', 'SEMA3C (10512)', 'SEMA3D (223117)', 'SEMA3E (9723)', 'SEMA3F
 (6405)', 'SEMA3G (56920)', 'SEMA4A (64218)', 'SEMA4B (10509)', 'SEMA4C (54910)',
 'SEMA4D (10507)', 'SEMA4F (10505)', 'SEMA4G (57715)', 'SEMA5A (9037)', 'SEMA5B
 (54437)', 'SEMA6A (57556)', 'SEMA6B (10501)', 'SEMA6C (10500)', 'SEMA6D
 (80031)', 'SEMA7A (8482)', 'SEMG1 (6406)', 'SEMG2 (6407)', 'SENP1 (29843)',
 'SENP2 (59343)', 'SENP3 (26168)', 'SENP5 (205564)', 'SENP6 (26054)', 'SENP7
 (57337)', 'SENP8 (123228)', 'SEPHS1 (22929)', 'SEPHS2 (22928)', 'SEPSECS
 (51091)', 'SEPTIN10 (151011)', 'SEPTIN11 (55752)', 'SEPTIN12 (124404)',
 'SEPTIN14 (346288)', 'SEPTIN2 (4735)', 'SEPTIN3 (55964)', 'SEPTIN4 (5414)',
 'SEPTIN5 (5413)', 'SEPTIN6 (23157)', 'SEPTIN7 (989)', 'SEPTIN8 (23176)',
 'SEPTIN9 (10801)', 'SERAC1 (84947)', 'SERBP1 (26135)', 'SERF2 (10169)', 'SERGEF
 (26297)', 'SERHL2 (253190)', 'SERINC1 (57515)', 'SERINC2 (347735)', 'SERINC3
 (10955)', 'SERINC4 (619189)', 'SERINC5 (256987)', 'SERP1 (27230)', 'SERP2
 (387923)', 'SERPINA1 (5265)', 'SERPINA10 (51156)', 'SERPINA11 (256394)',
 'SERPINA12 (145264)', 'SERPINA3 (12)', 'SERPINA4 (5267)', 'SERPINA5 (5104)',
 'SERPINA6 (866)', 'SERPINA7 (6906)', 'SERPINA9 (327657)', 'SERPINB1 (1992)',
 'SERPINB10 (5273)', 'SERPINB11 (89778)', 'SERPINB12 (89777)', 'SERPINB13
 (5275)', 'SERPINB2 (5055)', 'SERPINB3 (6317)', 'SERPINB4 (6318)', 'SERPINB5
 (5268)', 'SERPINB6 (5269)', 'SERPINB7 (8710)', 'SERPINB8 (5271)', 'SERPINB9
 (5272)', 'SERPINC1 (462)', 'SERPIND1 (3053)', 'SERPINE1 (5054)', 'SERPINE2
 (5270)', 'SERPINE3 (647174)', 'SERPINF1 (5176)', 'SERPINF2 (5345)', 'SERPING1
 (710)', 'SERPINH1 (871)', 'SERPINI1 (5274)', 'SERPINI2 (5276)', 'SERTAD1
 (29950)', 'SERTAD2 (9792)', 'SERTAD3 (29946)', 'SERTAD4 (56256)', 'SERTM1
 (400120)', 'SERTM2 (401613)', 'SESN1 (27244)', 'SESN2 (83667)', 'SESN3
 (143686)', 'SESTD1 (91404)', 'SET (6418)', 'SETBP1 (26040)', 'SETD1A (9739)',
 'SETD1B (23067)', 'SETD2 (29072)', 'SETD3 (84193)', 'SETD4 (54093)', 'SETD5
 (55209)', 'SETD6 (79918)', 'SETD7 (80854)', 'SETD9 (133383)', 'SETDB1 (9869)',
 'SETDB2 (83852)', 'SETMAR (6419)', 'SETX (23064)', 'SEZ6 (124925)', 'SEZ6L
 (23544)', 'SEZ6L2 (26470)', 'SF1 (7536)', 'SF3A1 (10291)', 'SF3A2 (8175)',
 'SF3A3 (10946)', 'SF3B1 (23451)', 'SF3B2 (10992)', 'SF3B3 (23450)', 'SF3B4
 (10262)', 'SF3B5 (83443)', 'SF3B6 (51639)', 'SFI1 (9814)', 'SFMBT1 (51460)',
 'SFMBT2 (57713)', 'SFN (2810)', 'SFPQ (6421)', 'SFR1 (119392)', 'SFRP1 (6422)',
 'SFRP2 (6423)', 'SFRP4 (6424)', 'SFRP5 (6425)', 'SFSWAP (6433)', 'SFT2D1
 (113402)', 'SFT2D2 (375035)', 'SFT2D3 (84826)', 'SFTA2 (389376)', 'SFTPA1
 (653509)', 'SFTPA2 (729238)', 'SFTPB (6439)', 'SFTPC (6440)', 'SFTPD (6441)',
 'SFXN1 (94081)', 'SFXN2 (118980)', 'SFXN3 (81855)', 'SFXN4 (119559)', 'SFXN5
 (94097)', 'SGCA (6442)', 'SGCB (6443)', 'SGCD (6444)', 'SGCE (8910)', 'SGCG
 (6445)', 'SGCZ (137868)', 'SGF29 (112869)', 'SGIP1 (84251)', 'SGK1 (6446)',
 'SGK2 (10110)', 'SGK3 (23678)', 'SGMS1 (259230)', 'SGMS2 (166929)', 'SGO1
 (151648)', 'SGO2 (151246)', 'SGPL1 (8879)', 'SGPP1 (81537)', 'SGPP2 (130367)',
 'SGSH (6448)', 'SGSM1 (129049)', 'SGSM2 (9905)', 'SGSM3 (27352)', 'SGTA (6449)',
 'SGTB (54557)', 'SH2B1 (25970)', 'SH2B2 (10603)', 'SH2B3 (10019)', 'SH2D1A
 (4068)', 'SH2D1B (117157)', 'SH2D2A (9047)', 'SH2D3A (10045)', 'SH2D3C (10044)',
 'SH2D4A (63898)', 'SH2D4B (387694)', 'SH2D5 (400745)', 'SH2D6 (284948)', 'SH2D7

(646892)', 'SH3BGR (6450)', 'SH3BGRL (6451)', 'SH3BGRL2 (83699)', 'SH3BGRL3
 (83442)', 'SH3BP1 (23616)', 'SH3BP2 (6452)', 'SH3BP4 (23677)', 'SH3BP5 (9467)',
 'SH3BP5L (80851)', 'SH3D19 (152503)', 'SH3D21 (79729)', 'SH3GL1 (6455)', 'SH3GL2
 (6456)', 'SH3GL3 (6457)', 'SH3GLB1 (51100)', 'SH3GLB2 (56904)', 'SH3KBP1
 (30011)', 'SH3PXD2A (9644)', 'SH3PXD2B (285590)', 'SH3RF1 (57630)', 'SH3RF2
 (153769)', 'SH3RF3 (344558)', 'SH3TC1 (54436)', 'SH3TC2 (79628)', 'SH3YL1
 (26751)', 'SHANK1 (50944)', 'SHANK2 (22941)', 'SHARPIN (81858)', 'SHB (6461)',
 'SHBG (6462)', 'SHC1 (6464)', 'SHC2 (25759)', 'SHC3 (53358)', 'SHC4 (399694)',
 'SHCBP1 (79801)', 'SHCBP1L (81626)', 'SHD (56961)', 'SHE (126669)', 'SHF
 (90525)', 'SHFL (55337)', 'SHH (6469)', 'SHISA2 (387914)', 'SHISA3 (152573)',
 'SHISA4 (149345)', 'SHISA5 (51246)', 'SHISA6 (388336)', 'SHISA7 (729956)',
 'SHISA8 (440829)', 'SHISA9 (729993)', 'SHISAL1 (85352)', 'SHISAL2A (348378)',
 'SHISAL2B (100132916)', 'SHKBP1 (92799)', 'SHLD1 (149840)', 'SHLD2 (54537)',
 'SHLD3 (112441434)', 'SHMT1 (6470)', 'SHMT2 (6472)', 'SHOC1 (158401)', 'SHOC2
 (8036)', 'SHOX (6473)', 'SHOX2 (6474)', 'SHPK (23729)', 'SHPRH (257218)', 'SHQ1
 (55164)', 'SHROOM1 (134549)', 'SHROOM2 (357)', 'SHROOM3 (57619)', 'SHROOM4
 (57477)', 'SHTN1 (57698)', 'SI (6476)', 'SIAE (54414)', 'SIAH1 (6477)', 'SIAH2
 (6478)', 'SIAH3 (283514)', 'SIDT1 (54847)', 'SIDT2 (51092)', 'SIGIRR (59307)',
 'SIGLEC1 (6614)', 'SIGLEC10 (89790)', 'SIGLEC11 (114132)', 'SIGLEC12 (89858)',
 'SIGLEC14 (100049587)', 'SIGLEC15 (284266)', 'SIGLEC5 (8778)', 'SIGLEC6 (946)',
 'SIGLEC7 (27036)', 'SIGLEC8 (27181)', 'SIGLEC9 (27180)', 'SIGLECL1 (284369)',
 'SIGMAR1 (10280)', 'SIK1 (150094)', 'SIK2 (23235)', 'SIK3 (23387)', 'SIKE1
 (80143)', 'SIL1 (64374)', 'SIM1 (6492)', 'SIM2 (6493)', 'SIMC1 (375484)', 'SIN3A
 (25942)', 'SIN3B (23309)', 'SINHCAF (58516)', 'SIPA1 (6494)', 'SIPA1L1 (26037)',
 'SIPA1L2 (57568)', 'SIPA1L3 (23094)', 'SIRPA (140885)', 'SIRPB1 (10326)',
 'SIRPB2 (284759)', 'SIRPD (128646)', 'SIRPG (55423)', 'SIRT1 (23411)', 'SIRT2
 (22933)', 'SIRT3 (23410)', 'SIRT4 (23409)', 'SIRT5 (23408)', 'SIRT6 (51548)',
 'SIRT7 (51547)', 'SIT1 (27240)', 'SIVA1 (10572)', 'SIX1 (6495)', 'SIX2 (10736)',
 'SIX3 (6496)', 'SIX4 (51804)', 'SIX5 (147912)', 'SIX6 (4990)', 'SKA1 (220134)',
 'SKA2 (348235)', 'SKA3 (221150)', 'SKAP1 (8631)', 'SKAP2 (8935)', 'SKI (6497)',
 'SKIC2 (6499)', 'SKIC3 (9652)', 'SKIC8 (80349)', 'SKIDA1 (387640)', 'SKIL
 (6498)', 'SKOR1 (390598)', 'SKOR2 (652991)', 'SKP1 (6500)', 'SKP2 (6502)', 'SLA
 (6503)', 'SLA2 (84174)', 'SLAIN1 (122060)', 'SLAIN2 (57606)', 'SLAMF1 (6504)',
 'SLAMF6 (114836)', 'SLAMF7 (57823)', 'SLAMF8 (56833)', 'SLAMF9 (89886)', 'SLBP
 (7884)', 'SLC10A1 (6554)', 'SLC10A2 (6555)', 'SLC10A3 (8273)', 'SLC10A4
 (201780)', 'SLC10A5 (347051)', 'SLC10A6 (345274)', 'SLC10A7 (84068)', 'SLC11A1
 (6556)', 'SLC11A2 (4891)', 'SLC12A1 (6557)', 'SLC12A2 (6558)', 'SLC12A3 (6559)',
 'SLC12A4 (6560)', 'SLC12A5 (57468)', 'SLC12A6 (9990)', 'SLC12A7 (10723)',
 'SLC12A8 (84561)', 'SLC12A9 (56996)', 'SLC13A1 (6561)', 'SLC13A2 (9058)',
 'SLC13A3 (64849)', 'SLC13A4 (26266)', 'SLC13A5 (284111)', 'SLC14A1 (6563)',
 'SLC14A2 (8170)', 'SLC15A1 (6564)', 'SLC15A2 (6565)', 'SLC15A3 (51296)',
 'SLC15A4 (121260)', 'SLC16A1 (6566)', 'SLC16A10 (117247)', 'SLC16A11 (162515)',
 'SLC16A12 (387700)', 'SLC16A13 (201232)', 'SLC16A14 (151473)', 'SLC16A2 (6567)',
 'SLC16A3 (9123)', 'SLC16A4 (9122)', 'SLC16A5 (9121)', 'SLC16A6 (9120)', 'SLC16A7
 (9194)', 'SLC16A8 (23539)', 'SLC16A9 (220963)', 'SLC17A1 (6568)', 'SLC17A2
 (10246)', 'SLC17A3 (10786)', 'SLC17A4 (10050)', 'SLC17A5 (26503)', 'SLC17A6
 (57084)', 'SLC17A7 (57030)', 'SLC17A8 (246213)', 'SLC17A9 (63910)', 'SLC18A1
 (6570)', 'SLC18A2 (6571)', 'SLC18A3 (6572)', 'SLC18B1 (116843)', 'SLC19A1

(6573)', 'SLC19A2 (10560)', 'SLC19A3 (80704)', 'SLC1A1 (6505)', 'SLC1A2 (6506)',
 'SLC1A3 (6507)', 'SLC1A4 (6509)', 'SLC1A5 (6510)', 'SLC1A6 (6511)', 'SLC1A7
 (6512)', 'SLC20A1 (6574)', 'SLC20A2 (6575)', 'SLC22A1 (6580)', 'SLC22A10
 (387775)', 'SLC22A11 (55867)', 'SLC22A12 (116085)', 'SLC22A13 (9390)', 'SLC22A14
 (9389)', 'SLC22A15 (55356)', 'SLC22A16 (85413)', 'SLC22A17 (51310)', 'SLC22A2
 (6582)', 'SLC22A23 (63027)', 'SLC22A24 (283238)', 'SLC22A25 (387601)', 'SLC22A3
 (6581)', 'SLC22A31 (146429)', 'SLC22A4 (6583)', 'SLC22A5 (6584)', 'SLC22A6
 (9356)', 'SLC22A7 (10864)', 'SLC22A8 (9376)', 'SLC22A9 (114571)', 'SLC23A1
 (9963)', 'SLC23A2 (9962)', 'SLC23A3 (151295)', 'SLC24A1 (9187)', 'SLC24A2
 (25769)', 'SLC24A3 (57419)', 'SLC24A4 (123041)', 'SLC24A5 (283652)', 'SLC25A1
 (6576)', 'SLC25A10 (1468)', 'SLC25A11 (8402)', 'SLC25A12 (8604)', 'SLC25A13
 (10165)', 'SLC25A14 (9016)', 'SLC25A15 (10166)', 'SLC25A16 (8034)', 'SLC25A17
 (10478)', 'SLC25A18 (83733)', 'SLC25A19 (60386)', 'SLC25A2 (83884)', 'SLC25A20
 (788)', 'SLC25A21 (89874)', 'SLC25A22 (79751)', 'SLC25A23 (79085)', 'SLC25A24
 (29957)', 'SLC25A25 (114789)', 'SLC25A26 (115286)', 'SLC25A27 (9481)', 'SLC25A28
 (81894)', 'SLC25A29 (123096)', 'SLC25A3 (5250)', 'SLC25A30 (253512)', 'SLC25A31
 (83447)', 'SLC25A32 (81034)', 'SLC25A33 (84275)', 'SLC25A34 (284723)', 'SLC25A35
 (399512)', 'SLC25A36 (55186)', 'SLC25A37 (51312)', 'SLC25A38 (54977)', 'SLC25A39
 (51629)', 'SLC25A4 (291)', 'SLC25A40 (55972)', 'SLC25A41 (284427)', 'SLC25A42
 (284439)', 'SLC25A43 (203427)', 'SLC25A44 (9673)', 'SLC25A45 (283130)',
 'SLC25A46 (91137)', 'SLC25A47 (283600)', 'SLC25A48 (153328)', 'SLC25A5 (292)',
 'SLC25A51 (92014)', 'SLC25A52 (147407)', 'SLC25A53 (401612)', 'SLC25A6 (293)',
 'SLC26A1 (10861)', 'SLC26A11 (284129)', 'SLC26A2 (1836)', 'SLC26A3 (1811)',
 'SLC26A4 (5172)', 'SLC26A5 (375611)', 'SLC26A6 (65010)', 'SLC26A7 (115111)',
 'SLC26A8 (116369)', 'SLC26A9 (115019)', 'SLC27A1 (376497)', 'SLC27A2 (11001)',
 'SLC27A3 (11000)', 'SLC27A4 (10999)', 'SLC27A5 (10998)', 'SLC27A6 (28965)',
 'SLC28A1 (9154)', 'SLC28A2 (9153)', 'SLC28A3 (64078)', 'SLC29A1 (2030)',
 'SLC29A2 (3177)', 'SLC29A3 (55315)', 'SLC29A4 (222962)', 'SLC2A1 (6513)',
 'SLC2A10 (81031)', 'SLC2A11 (66035)', 'SLC2A12 (154091)', 'SLC2A13 (114134)',
 'SLC2A14 (144195)', 'SLC2A2 (6514)', 'SLC2A3 (6515)', 'SLC2A4 (6517)', 'SLC2A4RG
 (56731)', 'SLC2A5 (6518)', 'SLC2A6 (11182)', 'SLC2A7 (155184)', 'SLC2A8
 (29988)', 'SLC2A9 (56606)', 'SLC30A1 (7779)', 'SLC30A10 (55532)', 'SLC30A2
 (7780)', 'SLC30A3 (7781)', 'SLC30A4 (7782)', 'SLC30A5 (64924)', 'SLC30A6
 (55676)', 'SLC30A7 (148867)', 'SLC30A8 (169026)', 'SLC30A9 (10463)', 'SLC31A1
 (1317)', 'SLC31A2 (1318)', 'SLC32A1 (140679)', 'SLC33A1 (9197)', 'SLC34A1
 (6569)', 'SLC34A2 (10568)', 'SLC34A3 (142680)', 'SLC35A1 (10559)', 'SLC35A2
 (7355)', 'SLC35A3 (23443)', 'SLC35A4 (113829)', 'SLC35A5 (55032)', 'SLC35B1
 (10237)', 'SLC35B2 (347734)', 'SLC35B3 (51000)', 'SLC35B4 (84912)', 'SLC35C1
 (55343)', 'SLC35C2 (51006)', 'SLC35D1 (23169)', 'SLC35D2 (11046)', 'SLC35D3
 (340146)', 'SLC35E1 (79939)', 'SLC35E2B (728661)', 'SLC35E3 (55508)', 'SLC35E4
 (339665)', 'SLC35F1 (222553)', 'SLC35F2 (54733)', 'SLC35F3 (148641)', 'SLC35F4
 (341880)', 'SLC35F5 (80255)', 'SLC35F6 (54978)', 'SLC35G1 (159371)', 'SLC35G2
 (80723)', 'SLC35G3 (146861)', 'SLC35G4 (646000)', 'SLC35G5 (83650)', 'SLC35G6
 (643664)', 'SLC36A1 (206358)', 'SLC36A2 (153201)', 'SLC36A3 (285641)', 'SLC36A4
 (120103)', 'SLC37A1 (54020)', 'SLC37A2 (219855)', 'SLC37A3 (84255)', 'SLC38A1
 (81539)', 'SLC38A10 (124565)', 'SLC38A11 (151258)', 'SLC38A2 (54407)', 'SLC38A3
 (10991)', 'SLC38A4 (55089)', 'SLC38A5 (92745)', 'SLC38A6 (145389)', 'SLC38A7
 (55238)', 'SLC38A8 (146167)', 'SLC38A9 (153129)', 'SLC39A1 (27173)', 'SLC39A10

(57181)', 'SLC39A11 (201266)', 'SLC39A12 (221074)', 'SLC39A13 (91252)',
 'SLC39A14 (23516)', 'SLC39A2 (29986)', 'SLC39A3 (29985)', 'SLC39A4 (55630)',
 'SLC39A5 (283375)', 'SLC39A6 (25800)', 'SLC39A7 (7922)', 'SLC39A8 (64116)',
 'SLC39A9 (55334)', 'SLC3A1 (6519)', 'SLC3A2 (6520)', 'SLC40A1 (30061)', 'SLC41A1
 (254428)', 'SLC41A2 (84102)', 'SLC41A3 (54946)', 'SLC43A1 (8501)', 'SLC43A2
 (124935)', 'SLC43A3 (29015)', 'SLC44A1 (23446)', 'SLC44A2 (57153)', 'SLC44A3
 (126969)', 'SLC44A4 (80736)', 'SLC44A5 (204962)', 'SLC45A1 (50651)', 'SLC45A2
 (51151)', 'SLC45A3 (85414)', 'SLC45A4 (57210)', 'SLC46A1 (113235)', 'SLC46A2
 (57864)', 'SLC46A3 (283537)', 'SLC47A1 (55244)', 'SLC47A2 (146802)', 'SLC48A1
 (55652)', 'SLC49A3 (84179)', 'SLC49A4 (84925)', 'SLC4A1 (6521)', 'SLC4A10
 (57282)', 'SLC4A11 (83959)', 'SLC4A1AP (22950)', 'SLC4A2 (6522)', 'SLC4A3
 (6508)', 'SLC4A4 (8671)', 'SLC4A5 (57835)', 'SLC4A7 (9497)', 'SLC4A8 (9498)',
 'SLC4A9 (83697)', 'SLC50A1 (55974)', 'SLC51A (200931)', 'SLC51B (123264)',
 'SLC52A1 (55065)', 'SLC52A2 (79581)', 'SLC52A3 (113278)', 'SLC5A1 (6523)',
 'SLC5A10 (125206)', 'SLC5A11 (115584)', 'SLC5A12 (159963)', 'SLC5A2 (6524)',
 'SLC5A3 (6526)', 'SLC5A4 (6527)', 'SLC5A5 (6528)', 'SLC5A6 (8884)', 'SLC5A7
 (60482)', 'SLC5A8 (160728)', 'SLC5A9 (200010)', 'SLC66A1 (54896)', 'SLC66A2
 (80148)', 'SLC66A3 (130814)', 'SLC67A1 (5002)', 'SLC6A1 (6529)', 'SLC6A11
 (6538)', 'SLC6A12 (6539)', 'SLC6A13 (6540)', 'SLC6A14 (11254)', 'SLC6A15
 (55117)', 'SLC6A16 (28968)', 'SLC6A17 (388662)', 'SLC6A18 (348932)', 'SLC6A19
 (340024)', 'SLC6A2 (6530)', 'SLC6A20 (54716)', 'SLC6A3 (6531)', 'SLC6A4 (6532)',
 'SLC6A5 (9152)', 'SLC6A6 (6533)', 'SLC6A7 (6534)', 'SLC6A8 (6535)', 'SLC6A9
 (6536)', 'SLC7A1 (6541)', 'SLC7A10 (56301)', 'SLC7A11 (23657)', 'SLC7A13
 (157724)', 'SLC7A14 (57709)', 'SLC7A2 (6542)', 'SLC7A3 (84889)', 'SLC7A4
 (6545)', 'SLC7A5 (8140)', 'SLC7A6 (9057)', 'SLC7A60S (84138)', 'SLC7A7 (9056)',
 'SLC7A8 (23428)', 'SLC7A9 (11136)', 'SLC8A1 (6546)', 'SLC8A2 (6543)', 'SLC8A3
 (6547)', 'SLC8B1 (80024)', 'SLC9A1 (6548)', 'SLC9A2 (6549)', 'SLC9A3 (6550)',
 'SLC9A4 (389015)', 'SLC9A5 (6553)', 'SLC9A6 (10479)', 'SLC9A7 (84679)', 'SLC9A8
 (23315)', 'SLC9A9 (285195)', 'SLC9B1 (150159)', 'SLC9B2 (133308)', 'SLC9C1
 (285335)', 'SLC9C2 (284525)', 'SLC01A2 (6579)', 'SLC01B1 (10599)', 'SLC01B3
 (28234)', 'SLC01C1 (53919)', 'SLC02A1 (6578)', 'SLC02B1 (11309)', 'SLC03A1
 (28232)', 'SLC04A1 (28231)', 'SLC04C1 (353189)', 'SLC05A1 (81796)', 'SLC06A1
 (133482)', 'SLF1 (84250)', 'SLF2 (55719)', 'SLFN11 (91607)', 'SLFN12 (55106)',
 'SLFN12L (100506736)', 'SLFN13 (146857)', 'SLFN14 (342618)', 'SLFN5 (162394)',
 'SLFNL1 (200172)', 'SLIRP (81892)', 'SLIT1 (6585)', 'SLIT2 (9353)', 'SLIT3
 (6586)', 'SLITRK1 (114798)', 'SLITRK2 (84631)', 'SLITRK3 (22865)', 'SLITRK4
 (139065)', 'SLITRK5 (26050)', 'SLITRK6 (84189)', 'SLK (9748)', 'SLMAP (7871)',
 'SLN (6588)', 'SLPI (6590)', 'SLTM (79811)', 'SLU7 (10569)', 'SLURP1 (57152)',
 'SLURP2 (432355)', 'SLX4 (84464)', 'SLX4IP (128710)', 'SLX9 (85395)', 'SMAD1
 (4086)', 'SMAD2 (4087)', 'SMAD3 (4088)', 'SMAD4 (4089)', 'SMAD5 (4090)', 'SMAD6
 (4091)', 'SMAD7 (4092)', 'SMAD9 (4093)', 'SMAGP (57228)', 'SMAP1 (60682)',
 'SMAP2 (64744)', 'SMARCA1 (6594)', 'SMARCA2 (6595)', 'SMARCA4 (6597)', 'SMARCA5
 (8467)', 'SMARCAD1 (56916)', 'SMARCAL1 (50485)', 'SMARCB1 (6598)', 'SMARCC1
 (6599)', 'SMARCC2 (6601)', 'SMARCD1 (6602)', 'SMARCD2 (6603)', 'SMARCD3 (6604)',
 'SMARCE1 (6605)', 'SMC1A (8243)', 'SMC1B (27127)', 'SMC2 (10592)', 'SMC3
 (9126)', 'SMC4 (10051)', 'SMC5 (23137)', 'SMC6 (79677)', 'SMCHD1 (23347)',
 'SMC01 (255798)', 'SMC02 (341346)', 'SMC03 (440087)', 'SMC04 (56935)', 'SMCP
 (4184)', 'SMCR8 (140775)', 'SMDT1 (91689)', 'SMG1 (23049)', 'SMG5 (23381)',

'SMG6 (23293)', 'SMG7 (9887)', 'SMG8 (55181)', 'SMG9 (56006)', 'SMIM1 (388588)',
 'SMIM10 (644538)', 'SMIM10L1 (100129361)', 'SMIM10L2B (644596)', 'SMIM11
 (54065)', 'SMIM12 (113444)', 'SMIM13 (221710)', 'SMIM14 (201895)', 'SMIM15
 (643155)', 'SMIM17 (147670)', 'SMIM18 (100507341)', 'SMIM19 (114926)', 'SMIM2
 (79024)', 'SMIM20 (389203)', 'SMIM21 (284274)', 'SMIM22 (440335)', 'SMIM23
 (644994)', 'SMIM24 (284422)', 'SMIM26 (388789)', 'SMIM27 (100129250)', 'SMIM28
 (110806279)', 'SMIM29 (221491)', 'SMIM3 (85027)', 'SMIM31 (100505989)', 'SMIM32
 (389332)', 'SMIM33 (111064649)', 'SMIM34 (388820)', 'SMIM38 (107984345)',
 'SMIM40 (113523636)', 'SMIM41 (113523638)', 'SMIM5 (643008)', 'SMIM7 (79086)',
 'SMIM8 (57150)', 'SMIM9 (100132963)', 'SMKR1 (100287482)', 'SMLR1 (100507203)',
 'SMNDC1 (10285)', 'SMO (6608)', 'SMOC1 (64093)', 'SMOC2 (64094)', 'SMOX
 (54498)', 'SMPD1 (6609)', 'SMPD2 (6610)', 'SMPD3 (55512)', 'SMPD4 (55627)',
 'SMPDL3A (10924)', 'SMPDL3B (27293)', 'SMPX (23676)', 'SMR3A (26952)', 'SMR3B
 (10879)', 'SMS (6611)', 'SMTN (6525)', 'SMTNL1 (219537)', 'SMTNL2 (342527)',
 'SMU1 (55234)', 'SMUG1 (23583)', 'SMURF1 (57154)', 'SMURF2 (64750)', 'SMYD1
 (150572)', 'SMYD2 (56950)', 'SMYD3 (64754)', 'SMYD4 (114826)', 'SMYD5 (10322)',
 'SNAI1 (6615)', 'SNAI2 (6591)', 'SNAI3 (333929)', 'SNAP23 (8773)', 'SNAP25
 (6616)', 'SNAP29 (9342)', 'SNAP47 (116841)', 'SNAP91 (9892)', 'SNAPC1 (6617)',
 'SNAPC2 (6618)', 'SNAPC3 (6619)', 'SNAPC4 (6621)', 'SNAPC5 (10302)', 'SNAPIN
 (23557)', 'SNCA (6622)', 'SNCAIP (9627)', 'SNCB (6620)', 'SNCG (6623)', 'SND1
 (27044)', 'SNED1 (25992)', 'SNF8 (11267)', 'SNIP1 (79753)', 'SNN (8303)', 'SNORC
 (389084)', 'SNPH (9751)', 'SNRK (54861)', 'SNRNP200 (23020)', 'SNRNP25 (79622)',
 'SNRNP27 (11017)', 'SNRNP35 (11066)', 'SNRNP40 (9410)', 'SNRNP48 (154007)',
 'SNRNP70 (6625)', 'SNRPA (6626)', 'SNRPA1 (6627)', 'SNRPB (6628)', 'SNRPB2
 (6629)', 'SNRPC (6631)', 'SNRPD1 (6632)', 'SNRPD2 (6633)', 'SNRPD3 (6634)',
 'SNRPE (6635)', 'SNRPF (6636)', 'SNRPG (6637)', 'SNRPN (6638)', 'SNTA1 (6640)',
 'SNTB1 (6641)', 'SNTB2 (6645)', 'SNTG1 (54212)', 'SNTG2 (54221)', 'SNTN
 (132203)', 'SNU13 (4809)', 'SNUPN (10073)', 'SNURF (8926)', 'SNW1 (22938)',
 'SNX1 (6642)', 'SNX10 (29887)', 'SNX11 (29916)', 'SNX12 (29934)', 'SNX13
 (23161)', 'SNX14 (57231)', 'SNX15 (29907)', 'SNX16 (64089)', 'SNX17 (9784)',
 'SNX18 (112574)', 'SNX19 (399979)', 'SNX2 (6643)', 'SNX20 (124460)', 'SNX21
 (90203)', 'SNX22 (79856)', 'SNX24 (28966)', 'SNX25 (83891)', 'SNX27 (81609)',
 'SNX29 (92017)', 'SNX3 (8724)', 'SNX30 (401548)', 'SNX31 (169166)', 'SNX32
 (254122)', 'SNX33 (257364)', 'SNX4 (8723)', 'SNX5 (27131)', 'SNX6 (58533)',
 'SNX7 (51375)', 'SNX8 (29886)', 'SNX9 (51429)', 'SOAT1 (6646)', 'SOAT2 (8435)',
 'SOBP (55084)', 'SOCS1 (8651)', 'SOCS2 (8835)', 'SOCS3 (9021)', 'SOCS4
 (122809)', 'SOCS5 (9655)', 'SOCS6 (9306)', 'SOCS7 (30837)', 'SOD1 (6647)', 'SOD2
 (6648)', 'SOD3 (6649)', 'SOHLH1 (402381)', 'SOHLH2 (54937)', 'SON (6651)',
 'SORBS1 (10580)', 'SORBS2 (8470)', 'SORBS3 (10174)', 'SORCS1 (114815)', 'SORCS2
 (57537)', 'SORCS3 (22986)', 'SORD (6652)', 'SORL1 (6653)', 'SORT1 (6272)', 'SOS1
 (6654)', 'SOS2 (6655)', 'SOST (50964)', 'SOSTDC1 (25928)', 'SOWAHA (134548)',
 'SOWAHB (345079)', 'SOWAHC (65124)', 'SOWAHD (347454)', 'SOX1 (6656)', 'SOX10
 (6663)', 'SOX11 (6664)', 'SOX12 (6666)', 'SOX13 (9580)', 'SOX14 (8403)', 'SOX15
 (6665)', 'SOX17 (64321)', 'SOX18 (54345)', 'SOX2 (6657)', 'SOX21 (11166)', 'SOX3
 (6658)', 'SOX30 (11063)', 'SOX4 (6659)', 'SOX5 (6660)', 'SOX6 (55553)', 'SOX7
 (83595)', 'SOX8 (30812)', 'SOX9 (6662)', 'SP1 (6667)', 'SP100 (6672)', 'SP110
 (3431)', 'SP140 (11262)', 'SP140L (93349)', 'SP2 (6668)', 'SP3 (6670)', 'SP4
 (6671)', 'SP5 (389058)', 'SP6 (80320)', 'SP7 (121340)', 'SP8 (221833)', 'SP9

(100131390)', 'SPA17 (53340)', 'SPAAR (158376)', 'SPACA1 (81833)', 'SPACA3
 (124912)', 'SPACA4 (171169)', 'SPACA6 (147650)', 'SPACA7 (122258)', 'SPACA9
 (11092)', 'SPACDR (402573)', 'SPADH (112577516)', 'SPAG1 (6674)', 'SPAG16
 (79582)', 'SPAG17 (200162)', 'SPAG4 (6676)', 'SPAG5 (10615)', 'SPAG6 (9576)',
 'SPAG7 (9552)', 'SPAG8 (26206)', 'SPAG9 (9043)', 'SPAM1 (6677)', 'SPANXB1
 (728695)', 'SPANXN1 (494118)', 'SPANXN2 (494119)', 'SPANXN3 (139067)', 'SPANXN4
 (441525)', 'SPANXN5 (494197)', 'SPARC (6678)', 'SPARCL1 (8404)', 'SPART
 (23111)', 'SPAST (6683)', 'SPATA1 (100505741)', 'SPATA12 (353324)', 'SPATA13
 (221178)', 'SPATA16 (83893)', 'SPATA17 (128153)', 'SPATA18 (132671)', 'SPATA19
 (219938)', 'SPATA2 (9825)', 'SPATA20 (64847)', 'SPATA21 (374955)', 'SPATA22
 (84690)', 'SPATA24 (202051)', 'SPATA25 (128497)', 'SPATA2L (124044)', 'SPATA3
 (130560)', 'SPATA31A1 (647060)', 'SPATA31A3 (727830)', 'SPATA31A6 (389730)',
 'SPATA31A7 (26165)', 'SPATA31C2 (645961)', 'SPATA31D1 (389763)', 'SPATA31E1
 (286234)', 'SPATA31F1 (259308)', 'SPATA31F3 (100129969)', 'SPATA31G1 (138724)',
 'SPATA32 (124783)', 'SPATA33 (124045)', 'SPATA4 (132851)', 'SPATA45 (149643)',
 'SPATA46 (284680)', 'SPATA6 (54558)', 'SPATA6L (55064)', 'SPATA7 (55812)',
 'SPATA9 (83890)', 'SPATC1 (375686)', 'SPATC1L (84221)', 'SPATS1 (221409)',
 'SPATS2 (65244)', 'SPATS2L (26010)', 'SPC24 (147841)', 'SPC25 (57405)', 'SPCS1
 (28972)', 'SPCS2 (9789)', 'SPCS3 (60559)', 'SPDEF (25803)', 'SPDL1 (54908)',
 'SPDYA (245711)', 'SPDYE1 (285955)', 'SPDYE3 (441272)', 'SPDYE4 (388333)',
 'SPECC1 (92521)', 'SPECC1L (23384)', 'SPEF1 (25876)', 'SPEF2 (79925)', 'SPEG
 (10290)', 'SPEM1 (374768)', 'SPEM2 (201243)', 'SPEM3 (107983988)', 'SPEN
 (23013)', 'SPESP1 (246777)', 'SPG11 (80208)', 'SPG21 (51324)', 'SPG7 (6687)',
 'SPHK1 (8877)', 'SPHK2 (56848)', 'SPHKAP (80309)', 'SPI1 (6688)', 'SPIB (6689)',
 'SPIC (121599)', 'SPICE1 (152185)', 'SPIDR (23514)', 'SPIN1 (10927)', 'SPIN3
 (169981)', 'SPIN4 (139886)', 'SPINDOC (144097)', 'SPINK1 (6690)', 'SPINK13
 (153218)', 'SPINK14 (408187)', 'SPINK2 (6691)', 'SPINK4 (27290)', 'SPINK5
 (11005)', 'SPINK6 (404203)', 'SPINK7 (84651)', 'SPINK8 (646424)', 'SPINK9
 (643394)', 'SPINT1 (6692)', 'SPINT2 (10653)', 'SPINT3 (10816)', 'SPINT4
 (391253)', 'SPIRE1 (56907)', 'SPIRE2 (84501)', 'SPMAP1 (388381)', 'SPMAP2
 (51298)', 'SPMAP2L (100506564)', 'SPMIP1 (100130705)', 'SPMIP10 (389320)',
 'SPMIP11 (255411)', 'SPMIP2 (152940)', 'SPMIP3 (200159)', 'SPMIP4 (136895)',
 'SPMIP5 (143379)', 'SPMIP6 (84688)', 'SPMIP7 (100130988)', 'SPMIP8 (374739)',
 'SPMIP9 (200523)', 'SPN (6693)', 'SPNS1 (83985)', 'SPNS2 (124976)', 'SPNS3
 (201305)', 'SP011 (23626)', 'SPOCD1 (90853)', 'SPOCK1 (6695)', 'SPOCK2 (9806)',
 'SPOCK3 (50859)', 'SPON1 (10418)', 'SPOP (8405)', 'SPOPL (339745)', 'SPOUT1
 (51490)', 'SPP1 (6696)', 'SPP2 (6694)', 'SPPL2A (84888)', 'SPPL2B (56928)',
 'SPPL2C (162540)', 'SPPL3 (121665)', 'SPR (6697)', 'SPRED1 (161742)', 'SPRED2
 (200734)', 'SPRED3 (399473)', 'SPRING1 (79794)', 'SPRN (503542)', 'SPRR1A
 (6698)', 'SPRR1B (6699)', 'SPRR2F (6705)', 'SPRR2G (6706)', 'SPRR3 (6707)',
 'SPRR4 (163778)', 'SPRTN (83932)', 'SPRY1 (10252)', 'SPRY2 (10253)', 'SPRY3
 (10251)', 'SPRY4 (81848)', 'SPRYD3 (84926)', 'SPRYD4 (283377)', 'SPRYD7
 (57213)', 'SPSB1 (80176)', 'SPSB2 (84727)', 'SPSB3 (90864)', 'SPSB4 (92369)',
 'SPTA1 (6708)', 'SPTAN1 (6709)', 'SPTB (6710)', 'SPTBN1 (6711)', 'SPTBN2
 (6712)', 'SPTBN4 (57731)', 'SPTBN5 (51332)', 'SPTLC1 (10558)', 'SPTLC2 (9517)',
 'SPTLC3 (55304)', 'SPTSSA (171546)', 'SPTSSB (165679)', 'SPTY2D1 (144108)', 'SPX
 (80763)', 'SPZ1 (84654)', 'SQLE (6713)', 'SQOR (58472)', 'SQSTM1 (8878)', 'SRA1
 (10011)', 'SRARP (149563)', 'SRBD1 (55133)', 'SRC (6714)', 'SRCAP (10847)',

'SRCIN1 (80725)', 'SRD5A1 (6715)', 'SRD5A2 (6716)', 'SRD5A3 (79644)', 'SREBF1 (6720)', 'SREBF2 (6721)', 'SREK1 (140890)', 'SREK1IP1 (285672)', 'SRF (6722)', 'SRFBP1 (153443)', 'SRGAP1 (57522)', 'SRGAP2 (23380)', 'SRGAP3 (9901)', 'SRGN (5552)', 'SRI (6717)', 'SRL (6345)', 'SRM (6723)', 'SRMS (6725)', 'SRP14 (6727)', 'SRP19 (6728)', 'SRP54 (6729)', 'SRP68 (6730)', 'SRP72 (6731)', 'SRP9 (6726)', 'SRPK1 (6732)', 'SRPK2 (6733)', 'SRPK3 (26576)', 'SRPRA (6734)', 'SRPRB (58477)', 'SRPX (8406)', 'SRPX2 (27286)', 'SRR (63826)', 'SRRD (402055)', 'SRRM1 (10250)', 'SRRM2 (23524)', 'SRRM4 (84530)', 'SRRM5 (100170229)', 'SRRT (51593)', 'SRSF1 (6426)', 'SRSF10 (10772)', 'SRSF11 (9295)', 'SRSF12 (135295)', 'SRSF2 (6427)', 'SRSF3 (6428)', 'SRSF4 (6429)', 'SRSF5 (6430)', 'SRSF6 (6431)', 'SRSF7 (6432)', 'SRSF8 (10929)', 'SRSF9 (8683)', 'SRXN1 (140809)', 'SRY (6736)', 'SS18 (6760)', 'SS18L1 (26039)', 'SS18L2 (51188)', 'SSB (6741)', 'SSBP1 (6742)', 'SSBP2 (23635)', 'SSBP3 (23648)', 'SSBP4 (170463)', 'SSC4D (136853)', 'SSC5D (284297)', 'SSH1 (54434)', 'SSH2 (85464)', 'SSH3 (54961)', 'SSMEM1 (136263)', 'SSNA1 (8636)', 'SSPN (8082)', 'SSR1 (6745)', 'SSR2 (6746)', 'SSR3 (6747)', 'SSR4 (6748)', 'SSRP1 (6749)', 'SST (6750)', 'SSTR1 (6751)', 'SSTR2 (6752)', 'SSTR3 (6753)', 'SSTR4 (6754)', 'SSTR5 (6755)', 'SSU72 (29101)', 'SSUH2 (51066)', 'SSX1 (6756)', 'SSX2IP (117178)', 'SSX3 (10214)', 'SSX5 (6758)', 'SSX7 (280658)', 'ST13 (6767)', 'ST14 (6768)', 'ST18 (9705)', 'ST3GAL1 (6482)', 'ST3GAL2 (6483)', 'ST3GAL3 (6487)', 'ST3GAL4 (6484)', 'ST3GAL5 (8869)', 'ST3GAL6 (10402)', 'ST6GAL1 (6480)', 'ST6GAL2 (84620)', 'ST6GALNAC1 (55808)', 'ST6GALNAC2 (10610)', 'ST6GALNAC3 (256435)', 'ST6GALNAC4 (27090)', 'ST6GALNAC5 (81849)', 'ST6GALNAC6 (30815)', 'ST7 (7982)', 'ST7L (54879)', 'ST8SIA1 (6489)', 'ST8SIA2 (8128)', 'ST8SIA3 (51046)', 'ST8SIA4 (7903)', 'ST8SIA5 (29906)', 'ST8SIA6 (338596)', 'STAB1 (23166)', 'STAB2 (55576)', 'STAC (6769)', 'STAC2 (342667)', 'STAC3 (246329)', 'STAG1 (10274)', 'STAG2 (10735)', 'STAG3 (10734)', 'STAM (8027)', 'STAM2 (10254)', 'STAMPB (10617)', 'STAMPBPL1 (57559)', 'STAP1 (26228)', 'STAP2 (55620)', 'STAR (6770)', 'STARD10 (10809)', 'STARD13 (90627)', 'STARD3 (10948)', 'STARD3NL (83930)', 'STARD4 (134429)', 'STARD5 (80765)', 'STARD6 (147323)', 'STARD7 (56910)', 'STARD8 (9754)', 'STARD9 (57519)', 'STAT1 (6772)', 'STAT2 (6773)', 'STAT3 (6774)', 'STAT4 (6775)', 'STAT5A (6776)', 'STAT5B (6777)', 'STAT6 (6778)', 'STATH (6779)', 'STAU1 (6780)', 'STAU2 (27067)', 'STBD1 (8987)', 'STC1 (6781)', 'STC2 (8614)', 'STEAP1 (26872)', 'STEAP1B (256227)', 'STEAP2 (261729)', 'STEAP3 (55240)', 'STEAP4 (79689)', 'STEEP1 (63932)', 'STH (246744)', 'STIL (6491)', 'STIM1 (6786)', 'STIM2 (57620)', 'STIMATE (375346)', 'STING1 (340061)', 'STIP1 (10963)', 'STK10 (6793)', 'STK11 (6794)', 'STK11IP (114790)', 'STK16 (8576)', 'STK17A (9263)', 'STK17B (9262)', 'STK24 (8428)', 'STK25 (10494)', 'STK26 (51765)', 'STK3 (6788)', 'STK31 (56164)', 'STK32A (202374)', 'STK32B (55351)', 'STK32C (282974)', 'STK33 (65975)', 'STK35 (140901)', 'STK36 (27148)', 'STK38 (11329)', 'STK38L (23012)', 'STK39 (27347)', 'STK4 (6789)', 'STK40 (83931)', 'STKLD1 (169436)', 'STMN1 (3925)', 'STMN2 (11075)', 'STMN3 (50861)', 'STMN4 (81551)', 'STMND1 (401236)', 'STMP1 (647087)', 'STN1 (79991)', 'STOM (2040)', 'STOML1 (9399)', 'STOML2 (30968)', 'STOML3 (161003)', 'STON1 (11037)', 'STON2 (85439)', 'STOX1 (219736)', 'STOX2 (56977)', 'STPG1 (90529)', 'STPG2 (285555)', 'STPG3 (441476)', 'STPG4 (285051)', 'STRA6 (64220)', 'STRA8 (346673)', 'STRADA (92335)', 'STRADB (55437)', 'STRAP (11171)', 'STRBP (55342)', 'STRC (161497)', 'STRIP1 (85369)', 'STRIP2 (57464)', 'STRIT1 (100507537)', 'STRN (6801)', 'STRN3 (29966)', 'STRN4 (29888)', 'STS (412)',

'STT3A (3703)', 'STT3B (201595)', 'STUB1 (10273)', 'STUM (375057)', 'STX10 (8677)', 'STX11 (8676)', 'STX12 (23673)', 'STX16 (8675)', 'STX17 (55014)', 'STX18 (53407)', 'STX19 (415117)', 'STX1A (6804)', 'STX1B (112755)', 'STX2 (2054)', 'STX3 (6809)', 'STX4 (6810)', 'STX5 (6811)', 'STX6 (10228)', 'STX7 (8417)', 'STX8 (9482)', 'STXBP1 (6812)', 'STXBP2 (6813)', 'STXBP3 (6814)', 'STXBP4 (252983)', 'STXBP5 (134957)', 'STXBP5L (9515)', 'STXBP6 (29091)', 'STYK1 (55359)', 'STYX (6815)', 'STYXL1 (51657)', 'STYXL2 (92235)', 'SUB1 (10923)', 'SUCLA2 (8803)', 'SUCLG1 (8802)', 'SUCLG2 (8801)', 'SUCNR1 (56670)', 'SUCO (51430)', 'SUDS3 (64426)', 'SUFU (51684)', 'SUGCT (79783)', 'SUGP1 (57794)', 'SUGP2 (10147)', 'SUGT1 (10910)', 'SULF1 (23213)', 'SULF2 (55959)', 'SULT1A1 (6817)', 'SULT1A2 (6799)', 'SULT1B1 (27284)', 'SULT1C2 (6819)', 'SULT1C3 (442038)', 'SULT1C4 (27233)', 'SULT1E1 (6783)', 'SULT2A1 (6822)', 'SULT2B1 (6820)', 'SULT4A1 (25830)', 'SULT6B1 (391365)', 'SUMF1 (285362)', 'SUMF2 (25870)', 'SUM01 (7341)', 'SUM02 (6613)', 'SUM03 (6612)', 'SUM04 (387082)', 'SUN1 (23353)', 'SUN2 (25777)', 'SUN3 (256979)', 'SUN5 (140732)', 'SUOX (6821)', 'SUPT16H (11198)', 'SUPT20H (55578)', 'SUPT20HL1 (100130302)', 'SUPT20HL2 (170067)', 'SUPT3H (8464)', 'SUPT4H1 (6827)', 'SUPT5H (6829)', 'SUPT6H (6830)', 'SUPT7L (9913)', 'SUPV3L1 (6832)', 'SURF1 (6834)', 'SURF2 (6835)', 'SURF4 (6836)', 'SURF6 (6838)', 'SUSD1 (64420)', 'SUSD2 (56241)', 'SUSD3 (203328)', 'SUSD4 (55061)', 'SUSD5 (26032)', 'SUSD6 (9766)', 'SUV39H1 (6839)', 'SUV39H2 (79723)', 'SUZ12 (23512)', 'SV2A (9900)', 'SV2B (9899)', 'SV2C (22987)', 'SVBP (374969)', 'SVEP1 (79987)', 'SVIL (6840)', 'SVIP (258010)', 'SVOP (55530)', 'SVOPL (136306)', 'SWAP70 (23075)', 'SWI5 (375757)', 'SWSAP1 (126074)', 'SWT1 (54823)', 'SYAP1 (94056)', 'SYBU (55638)', 'SYCE1 (93426)', 'SYCE1L (100130958)', 'SYCE2 (256126)', 'SYCE3 (644186)', 'SYCN (342898)', 'SYCP1 (6847)', 'SYCP2 (10388)', 'SYCP2L (221711)', 'SYCP3 (50511)', 'SYDE1 (85360)', 'SYDE2 (84144)', 'SYF2 (25949)', 'SYK (6850)', 'SYMPK (8189)', 'SYN1 (6853)', 'SYN2 (6854)', 'SYN3 (8224)', 'SYNC (81493)', 'SYNCRIP (10492)', 'SYNDIG1 (79953)', 'SYNDIG1L (646658)', 'SYNE1 (23345)', 'SYNE2 (23224)', 'SYNE3 (161176)', 'SYNE4 (163183)', 'SYNGAP1 (8831)', 'SYNGR1 (9145)', 'SYNGR2 (9144)', 'SYNGR3 (9143)', 'SYNGR4 (23546)', 'SYNJ1 (8867)', 'SYNJ2 (8871)', 'SYNJ2BP (55333)', 'SYNM (23336)', 'SYNPO (11346)', 'SYNPO2 (171024)', 'SYNPO2L (79933)', 'SYNPR (132204)', 'SYNRG (11276)', 'SYP (6855)', 'SYPL1 (6856)', 'SYPL2 (284612)', 'SYS1 (90196)', 'SYT1 (6857)', 'SYT10 (341359)', 'SYT11 (23208)', 'SYT12 (91683)', 'SYT13 (57586)', 'SYT14 (255928)', 'SYT16 (83851)', 'SYT17 (51760)', 'SYT2 (127833)', 'SYT3 (84258)', 'SYT4 (6860)', 'SYT5 (6861)', 'SYT6 (148281)', 'SYT7 (9066)', 'SYT8 (90019)', 'SYT9 (143425)', 'SYTL1 (84958)', 'SYTL2 (54843)', 'SYTL3 (94120)', 'SYTL4 (94121)', 'SYTL5 (94122)', 'SYVN1 (84447)', 'SZRD1 (26099)', 'SZT2 (23334)', 'TAAR1 (134864)', 'TAAR2 (9287)', 'TAAR5 (9038)', 'TAAR6 (319100)', 'TAAR8 (83551)', 'TAAR9 (134860)', 'TAB1 (10454)', 'TAB2 (23118)', 'TAB3 (257397)', 'TAC1 (6863)', 'TAC3 (6866)', 'TAC4 (255061)', 'TACC1 (6867)', 'TACC2 (10579)', 'TACC3 (10460)', 'TAC01 (51204)', 'TACR1 (6869)', 'TACR2 (6865)', 'TACR3 (6870)', 'TACSTD2 (4070)', 'TADA1 (117143)', 'TADA2A (6871)', 'TADA2B (93624)', 'TADA3 (10474)', 'TAF1 (6872)', 'TAF10 (6881)', 'TAF11 (6882)', 'TAF12 (6883)', 'TAF13 (6884)', 'TAF15 (8148)', 'TAF1A (9015)', 'TAF1B (9014)', 'TAF1C (9013)', 'TAF1D (79101)', 'TAF1L (138474)', 'TAF2 (6873)', 'TAF3 (83860)', 'TAF4 (6874)', 'TAF4B (6875)', 'TAF5 (6877)', 'TAF5L (27097)', 'TAF6 (6878)', 'TAF6L (10629)', 'TAF7 (6879)', 'TAF7L

(54457)', 'TAF8 (129685)', 'TAF9 (6880)', 'TAF9B (51616)', 'TAF A1 (407738)',
 'TAF A2 (338811)', 'TAF A3 (284467)', 'TAF A4 (151647)', 'TAF A5 (25817)', 'TAF AZZIN
 (6901)', 'TAGAP (117289)', 'TAGLN (6876)', 'TAGLN2 (8407)', 'TAGLN3 (29114)',
 'TAL1 (6886)', 'TAL2 (6887)', 'TALD01 (6888)', 'TAMALIN (160622)', 'TAMM41
 (132001)', 'TANC1 (85461)', 'TANC2 (26115)', 'TANGO2 (128989)', 'TANGO6
 (79613)', 'TANK (10010)', 'TAOK1 (57551)', 'TAOK2 (9344)', 'TAOK3 (51347)',
 'TAP1 (6890)', 'TAP2 (6891)', 'TAPBP (6892)', 'TAPBPL (55080)', 'TAPT1
 (202018)', 'TARBP1 (6894)', 'TARBP2 (6895)', 'TARDBP (23435)', 'TARM1 (441864)',
 'TARS1 (6897)', 'TARS2 (80222)', 'TARS3 (123283)', 'TAS1R1 (80835)', 'TAS1R2
 (80834)', 'TAS1R3 (83756)', 'TAS2R1 (50834)', 'TAS2R10 (50839)', 'TAS2R13
 (50838)', 'TAS2R14 (50840)', 'TAS2R16 (50833)', 'TAS2R19 (259294)', 'TAS2R20
 (259295)', 'TAS2R3 (50831)', 'TAS2R30 (259293)', 'TAS2R31 (259290)', 'TAS2R38
 (5726)', 'TAS2R39 (259285)', 'TAS2R4 (50832)', 'TAS2R40 (259286)', 'TAS2R41
 (259287)', 'TAS2R42 (353164)', 'TAS2R43 (259289)', 'TAS2R46 (259292)', 'TAS2R5
 (54429)', 'TAS2R50 (259296)', 'TAS2R60 (338398)', 'TAS2R7 (50837)', 'TAS2R8
 (50836)', 'TAS2R9 (50835)', 'TASL (80231)', 'TASOR (23272)', 'TASOR2 (54906)',
 'TASP1 (55617)', 'TAT (6898)', 'TATDN1 (83940)', 'TATDN2 (9797)', 'TATDN3
 (128387)', 'TAX1BP1 (8887)', 'TAX1BP3 (30851)', 'TBATA (219793)', 'TBC1D1
 (23216)', 'TBC1D10A (83874)', 'TBC1D10B (26000)', 'TBC1D10C (374403)', 'TBC1D12
 (23232)', 'TBC1D13 (54662)', 'TBC1D14 (57533)', 'TBC1D15 (64786)', 'TBC1D16
 (125058)', 'TBC1D17 (79735)', 'TBC1D19 (55296)', 'TBC1D2 (55357)', 'TBC1D20
 (128637)', 'TBC1D21 (161514)', 'TBC1D22A (25771)', 'TBC1D22B (55633)', 'TBC1D23
 (55773)', 'TBC1D24 (57465)', 'TBC1D25 (4943)', 'TBC1D26 (353149)', 'TBC1D28
 (254272)', 'TBC1D2B (23102)', 'TBC1D30 (23329)', 'TBC1D31 (93594)', 'TBC1D32
 (221322)', 'TBC1D4 (9882)', 'TBC1D5 (9779)', 'TBC1D7 (51256)', 'TBC1D8 (11138)',
 'TBC1D8B (54885)', 'TBC1D9 (23158)', 'TBC1D9B (23061)', 'TBCA (6902)', 'TBCB
 (1155)', 'TBCC (6903)', 'TBCCD1 (55171)', 'TBCD (6904)', 'TBCE (6905)', 'TBCEL
 (219899)', 'TBCK (93627)', 'TBK1 (29110)', 'TBKBP1 (9755)', 'TBL1X (6907)',
 'TBL1XR1 (79718)', 'TBL1Y (90665)', 'TBL2 (26608)', 'TBL3 (10607)', 'TBP
 (6908)', 'TBPL1 (9519)', 'TBR1 (10716)', 'TBRG1 (84897)', 'TBRG4 (9238)', 'TBX1
 (6899)', 'TBX10 (347853)', 'TBX15 (6913)', 'TBX18 (9096)', 'TBX19 (9095)', 'TBX2
 (6909)', 'TBX20 (57057)', 'TBX21 (30009)', 'TBX22 (50945)', 'TBX3 (6926)', 'TBX4
 (9496)', 'TBX5 (6910)', 'TBX6 (6911)', 'TBXA2R (6915)', 'TBXAS1 (6916)', 'TBXT
 (6862)', 'TC2N (123036)', 'TCAF1 (9747)', 'TCAF2 (285966)', 'TCAIM (285343)',
 'TCAP (8557)', 'TCEA1 (6917)', 'TCEA2 (6919)', 'TCEA3 (6920)', 'TCEAL1 (9338)',
 'TCEAL2 (140597)', 'TCEAL3 (85012)', 'TCEAL4 (79921)', 'TCEAL5 (340543)',
 'TCEAL7 (56849)', 'TCEAL8 (90843)', 'TCEAL9 (51186)', 'TCEANC (170082)',
 'TCEANC2 (127428)', 'TCERG1 (10915)', 'TCERG1L (256536)', 'TCF12 (6938)', 'TCF15
 (6939)', 'TCF19 (6941)', 'TCF20 (6942)', 'TCF21 (6943)', 'TCF23 (150921)',
 'TCF24 (100129654)', 'TCF25 (22980)', 'TCF3 (6929)', 'TCF4 (6925)', 'TCF7
 (6932)', 'TCF7L1 (83439)', 'TCF7L2 (6934)', 'TCFL5 (10732)', 'TCHH (7062)',
 'TCHHL1 (126637)', 'TCHP (84260)', 'TCIM (56892)', 'TCIRG1 (10312)', 'TCL1A
 (8115)', 'TCL1B (9623)', 'TCN1 (6947)', 'TCN2 (6948)', 'TCOF1 (6949)', 'TCP1
 (6950)', 'TCP10L (140290)', 'TCP11 (6954)', 'TCP11L1 (55346)', 'TCP11L2
 (255394)', 'TCTA (6988)', 'TCTE1 (202500)', 'TCTN1 (79600)', 'TCTN2 (79867)',
 'TCTN3 (26123)', 'TDG (6996)', 'TD02 (6999)', 'TDP1 (55775)', 'TDP2 (51567)',
 'TDRD1 (56165)', 'TDRD10 (126668)', 'TDRD12 (91646)', 'TDRD15 (100129278)',
 'TDRD3 (81550)', 'TDRD5 (163589)', 'TDRD6 (221400)', 'TDRD7 (23424)', 'TDRD9

(122402)', 'TDRKH (11022)', 'TDRP (157695)', 'TEAD1 (7003)', 'TEAD2 (8463)',
 'TEAD3 (7005)', 'TEAD4 (7004)', 'TEC (7006)', 'TECPR1 (25851)', 'TECPR2 (9895)',
 'TECR (9524)', 'TECRL (253017)', 'TECTA (7007)', 'TECTB (6975)', 'TEDC1
 (283643)', 'TEDC2 (80178)', 'TEDDM1 (127670)', 'TEF (7008)', 'TEFM (79736)',
 'TEK (7010)', 'TEKT1 (83659)', 'TEKT2 (27285)', 'TEKT3 (64518)', 'TEKT4
 (150483)', 'TEKT5 (146279)', 'TEKTIP1 (100128569)', 'TEKTL1 (126402)', 'TELO2
 (9894)', 'TEN1 (100134934)', 'TENM1 (10178)', 'TENM2 (57451)', 'TENM3 (55714)',
 'TENM4 (26011)', 'TENT2 (167153)', 'TENT4A (11044)', 'TENT4B (64282)', 'TENT5A
 (55603)', 'TENT5B (115572)', 'TENT5C (54855)', 'TENT5D (169966)', 'TEP1 (7011)',
 'TEPSIN (146705)', 'TERB2 (145645)', 'TERF1 (7013)', 'TERF2 (7014)', 'TERF2IP
 (54386)', 'TERT (7015)', 'TES (26136)', 'TESC (54997)', 'TESK1 (7016)', 'TESK2
 (10420)', 'TESMIN (9633)', 'TESPA1 (9840)', 'TET1 (80312)', 'TET2 (54790)',
 'TET3 (200424)', 'TEX10 (54881)', 'TEX101 (83639)', 'TEX11 (56159)', 'TEX12
 (56158)', 'TEX13A (56157)', 'TEX13B (56156)', 'TEX13C (100129520)', 'TEX13D
 (100132015)', 'TEX14 (56155)', 'TEX15 (56154)', 'TEX19 (400629)', 'TEX2
 (55852)', 'TEX22 (647310)', 'TEX26 (122046)', 'TEX261 (113419)', 'TEX264
 (51368)', 'TEX28 (1527)', 'TEX29 (121793)', 'TEX30 (93081)', 'TEX35 (84066)',
 'TEX36 (387718)', 'TEX38 (374973)', 'TEX44 (165100)', 'TEX46 (729059)', 'TEX47
 (219557)', 'TEX48 (100505478)', 'TEX50 (730159)', 'TEX51 (101929926)', 'TEX52
 (101929469)', 'TEX53 (105376230)', 'TEX55 (152405)', 'TEX9 (374618)', 'TF
 (7018)', 'TFAM (7019)', 'TFAP2A (7020)', 'TFAP2B (7021)', 'TFAP2C (7022)',
 'TFAP2D (83741)', 'TFAP2E (339488)', 'TFAP4 (7023)', 'TFB1M (51106)', 'TFB2M
 (64216)', 'TFCP2 (7024)', 'TFCP2L1 (29842)', 'TFDP1 (7027)', 'TFDP2 (7029)',
 'TFDP3 (51270)', 'TFE3 (7030)', 'TFEB (7942)', 'TFEC (22797)', 'TFF1 (7031)',
 'TFF2 (7032)', 'TFF3 (7033)', 'TFG (10342)', 'TFIP11 (24144)', 'TFPI (7035)',
 'TFPI2 (7980)', 'TFPT (29844)', 'TFR2 (7036)', 'TFRC (7037)', 'TG (7038)', 'TGDS
 (23483)', 'TGFA (7039)', 'TGFB1 (7040)', 'TGFB1I1 (7041)', 'TGFB2 (7042)',
 'TGFB3 (7043)', 'TGFB1 (7045)', 'TGFB1 (7046)', 'TGFB2 (7048)', 'TGFB3
 (7049)', 'TGFB3L (100507588)', 'TGFB1 (9392)', 'TGIF1 (7050)', 'TGIF2
 (60436)', 'TGIF2LX (90316)', 'TGM1 (7051)', 'TGM2 (7052)', 'TGM3 (7053)', 'TGM4
 (7047)', 'TGM5 (9333)', 'TGM6 (343641)', 'TGM7 (116179)', 'TGOLN2 (10618)',
 'TGS1 (96764)', 'TH (7054)', 'THADA (63892)', 'THAP1 (55145)', 'THAP10 (56906)',
 'THAP11 (57215)', 'THAP12 (5612)', 'THAP2 (83591)', 'THAP3 (90326)', 'THAP4
 (51078)', 'THAP5 (168451)', 'THAP6 (152815)', 'THAP7 (80764)', 'THAP8 (199745)',
 'THAP9 (79725)', 'THBD (7056)', 'THBS1 (7057)', 'THBS2 (7058)', 'THBS3 (7059)',
 'THBS4 (7060)', 'THEM4 (117145)', 'THEM5 (284486)', 'THEM6 (51337)', 'THEMIS
 (387357)', 'THEMIS2 (9473)', 'THG1L (54974)', 'THNSL1 (79896)', 'THNSL2
 (55258)', 'THOC1 (9984)', 'THOC2 (57187)', 'THOC3 (84321)', 'THOC5 (8563)',
 'THOC6 (79228)', 'THOC7 (80145)', 'THOP1 (7064)', 'THPO (7066)', 'THRA (7067)',
 'THRAP3 (9967)', 'THRB (7068)', 'THRSP (7069)', 'THSD1 (55901)', 'THSD4
 (79875)', 'THSD7A (221981)', 'THSD7B (80731)', 'THTPA (79178)', 'THUMPD1
 (55623)', 'THUMPD2 (80745)', 'THUMPD3 (25917)', 'THY1 (7070)', 'THYN1 (29087)',
 'TIA1 (7072)', 'TIAL1 (7073)', 'TIAM1 (7074)', 'TIAM2 (26230)', 'TICAM1
 (148022)', 'TICAM2 (353376)', 'TICRR (90381)', 'TIE1 (7075)', 'TIFA (92610)',
 'TIFAB (497189)', 'TIGAR (57103)', 'TIGD1 (200765)', 'TIGD2 (166815)', 'TIGD3
 (220359)', 'TIGD4 (201798)', 'TIGD5 (84948)', 'TIGD6 (81789)', 'TIGD7 (91151)',
 'TIGIT (201633)', 'TIMD4 (91937)', 'TIMELESS (8914)', 'TIMM10 (26519)', 'TIMM10B
 (26515)', 'TIMM13 (26517)', 'TIMM17A (10440)', 'TIMM17B (10245)', 'TIMM21

(29090)', 'TIMM22 (29928)', 'TIMM23 (100287932)', 'TIMM29 (90580)', 'TIMM44
 (10469)', 'TIMM50 (92609)', 'TIMM8A (1678)', 'TIMM8B (26521)', 'TIMM9 (26520)',
 'TIMMDC1 (51300)', 'TIMP1 (7076)', 'TIMP2 (7077)', 'TIMP3 (7078)', 'TIMP4
 (7079)', 'TINAG (27283)', 'TINAGL1 (64129)', 'TINF2 (26277)', 'TIPARP (25976)',
 'TIPIN (54962)', 'TIPRL (261726)', 'TIRAP (114609)', 'TJAP1 (93643)', 'TJP1
 (7082)', 'TJP2 (9414)', 'TJP3 (27134)', 'TK1 (7083)', 'TK2 (7084)', 'TKFC
 (26007)', 'TKT (7086)', 'TKTL1 (8277)', 'TKTL2 (84076)', 'TLCD1 (116238)',
 'TLCD2 (727910)', 'TLCD3A (79850)', 'TLCD3B (83723)', 'TLCD4 (148534)', 'TLCD5
 (219902)', 'TLDC2 (140711)', 'TLE1 (7088)', 'TLE2 (7089)', 'TLE3 (7090)', 'TLE4
 (7091)', 'TLE5 (166)', 'TLE6 (79816)', 'TLE7 (102723796)', 'TLK1 (9874)', 'TLK2
 (11011)', 'TLL1 (7092)', 'TLL2 (7093)', 'TLN1 (7094)', 'TLN2 (83660)', 'TLNRD1
 (59274)', 'TLR1 (7096)', 'TLR10 (81793)', 'TLR2 (7097)', 'TLR3 (7098)', 'TLR4
 (7099)', 'TLR5 (7100)', 'TLR6 (10333)', 'TLR7 (51284)', 'TLR8 (51311)', 'TLR9
 (54106)', 'TLX1 (3195)', 'TLX2 (3196)', 'TLX3 (30012)', 'TM2D1 (83941)', 'TM2D2
 (83877)', 'TM2D3 (80213)', 'TM4SF1 (4071)', 'TM4SF18 (116441)', 'TM4SF19
 (116211)', 'TM4SF20 (79853)', 'TM4SF4 (7104)', 'TM4SF5 (9032)', 'TM6SF1
 (53346)', 'TM6SF2 (53345)', 'TM7SF2 (7108)', 'TM7SF3 (51768)', 'TM9SF1 (10548)',
 'TM9SF2 (9375)', 'TM9SF3 (56889)', 'TM9SF4 (9777)', 'TMA16 (55319)', 'TMA7
 (51372)', 'TMBIM1 (64114)', 'TMBIM4 (51643)', 'TMBIM6 (7009)', 'TMC1 (117531)',
 'TMC2 (117532)', 'TMC3 (342125)', 'TMC4 (147798)', 'TMC5 (79838)', 'TMC6
 (11322)', 'TMC7 (79905)', 'TMC8 (147138)', 'TMCC1 (23023)', 'TMCC2 (9911)',
 'TMCC3 (57458)', 'TMC01 (54499)', 'TMC02 (127391)', 'TMC03 (55002)', 'TMC04
 (255104)', 'TMC05A (145942)', 'TMC06 (55374)', 'TMED1 (11018)', 'TMED10
 (10972)', 'TMED2 (10959)', 'TMED3 (23423)', 'TMED4 (222068)', 'TMED5 (50999)',
 'TMED6 (146456)', 'TMED7 (51014)', 'TMED8 (283578)', 'TMED9 (54732)', 'TMEFF1
 (8577)', 'TMEFF2 (23671)', 'TMEM100 (55273)', 'TMEM101 (84336)', 'TMEM102
 (284114)', 'TMEM104 (54868)', 'TMEM106A (113277)', 'TMEM106B (54664)', 'TMEM106C
 (79022)', 'TMEM107 (84314)', 'TMEM108 (66000)', 'TMEM109 (79073)', 'TMEM11
 (8834)', 'TMEM114 (283953)', 'TMEM115 (11070)', 'TMEM116 (89894)', 'TMEM117
 (84216)', 'TMEM119 (338773)', 'TMEM120A (83862)', 'TMEM120B (144404)', 'TMEM121
 (80757)', 'TMEM121B (27439)', 'TMEM123 (114908)', 'TMEM125 (128218)', 'TMEM126A
 (84233)', 'TMEM126B (55863)', 'TMEM127 (55654)', 'TMEM128 (85013)', 'TMEM129
 (92305)', 'TMEM130 (222865)', 'TMEM131 (23505)', 'TMEM131L (23240)', 'TMEM132A
 (54972)', 'TMEM132B (114795)', 'TMEM132C (92293)', 'TMEM132D (121256)',
 'TMEM132E (124842)', 'TMEM134 (80194)', 'TMEM135 (65084)', 'TMEM138 (51524)',
 'TMEM139 (135932)', 'TMEM140 (55281)', 'TMEM141 (85014)', 'TMEM143 (55260)',
 'TMEM144 (55314)', 'TMEM145 (284339)', 'TMEM147 (10430)', 'TMEM14A (28978)',
 'TMEM14B (81853)', 'TMEM14C (51522)', 'TMEM150A (129303)', 'TMEM150B (284417)',
 'TMEM150C (441027)', 'TMEM151A (256472)', 'TMEM151B (441151)', 'TMEM154
 (201799)', 'TMEM156 (80008)', 'TMEM158 (25907)', 'TMEM160 (54958)', 'TMEM161A
 (54929)', 'TMEM161B (153396)', 'TMEM163 (81615)', 'TMEM164 (84187)', 'TMEM165
 (55858)', 'TMEM167A (153339)', 'TMEM167B (56900)', 'TMEM168 (64418)', 'TMEM169
 (92691)', 'TMEM17 (200728)', 'TMEM170A (124491)', 'TMEM170B (100113407)',
 'TMEM171 (134285)', 'TMEM174 (134288)', 'TMEM175 (84286)', 'TMEM176A (55365)',
 'TMEM176B (28959)', 'TMEM177 (80775)', 'TMEM178A (130733)', 'TMEM178B
 (100507421)', 'TMEM179 (388021)', 'TMEM179B (374395)', 'TMEM18 (129787)',
 'TMEM181 (57583)', 'TMEM182 (130827)', 'TMEM183A (92703)', 'TMEM184A (202915)',
 'TMEM184B (25829)', 'TMEM184C (55751)', 'TMEM185A (84548)', 'TMEM185B (79134)',

'TMEM186 (25880)', 'TMEM187 (8269)', 'TMEM19 (55266)', 'TMEM190 (147744)',
 'TMEM192 (201931)', 'TMEM196 (256130)', 'TMEM198 (130612)', 'TMEM200A (114801)',
 'TMEM200B (399474)', 'TMEM200C (645369)', 'TMEM201 (199953)', 'TMEM202
 (338949)', 'TMEM203 (94107)', 'TMEM204 (79652)', 'TMEM205 (374882)', 'TMEM207
 (131920)', 'TMEM208 (29100)', 'TMEM209 (84928)', 'TMEM210 (100505993)', 'TMEM212
 (389177)', 'TMEM213 (155006)', 'TMEM214 (54867)', 'TMEM215 (401498)', 'TMEM216
 (51259)', 'TMEM217 (221468)', 'TMEM218 (219854)', 'TMEM219 (124446)', 'TMEM220
 (388335)', 'TMEM221 (100130519)', 'TMEM222 (84065)', 'TMEM223 (79064)', 'TMEM225
 (338661)', 'TMEM225B (100289187)', 'TMEM229A (730130)', 'TMEM229B (161145)',
 'TMEM230 (29058)', 'TMEM231 (79583)', 'TMEM232 (642987)', 'TMEM233 (387890)',
 'TMEM234 (56063)', 'TMEM235 (283999)', 'TMEM236 (653567)', 'TMEM237 (65062)',
 'TMEM238 (388564)', 'TMEM239 (100288797)', 'TMEM240 (339453)', 'TMEM241
 (85019)', 'TMEM242 (729515)', 'TMEM243 (79161)', 'TMEM244 (253582)', 'TMEM245
 (23731)', 'TMEM247 (388946)', 'TMEM248 (55069)', 'TMEM249 (340393)', 'TMEM25
 (84866)', 'TMEM250 (90120)', 'TMEM252 (169693)', 'TMEM253 (643382)', 'TMEM254
 (80195)', 'TMEM255A (55026)', 'TMEM255B (348013)', 'TMEM256 (254863)', 'TMEM258
 (746)', 'TMEM259 (91304)', 'TMEM26 (219623)', 'TMEM260 (54916)', 'TMEM262
 (100130348)', 'TMEM263 (90488)', 'TMEM265 (100862671)', 'TMEM266 (123591)',
 'TMEM267 (64417)', 'TMEM268 (203197)', 'TMEM269 (100129924)', 'TMEM270
 (135886)', 'TMEM271 (112441426)', 'TMEM272 (283521)', 'TMEM273 (170371)',
 'TMEM275 (105378696)', 'TMEM276 (84773)', 'TMEM278 (643965)', 'TMEM30A (55754)',
 'TMEM30B (161291)', 'TMEM31 (203562)', 'TMEM33 (55161)', 'TMEM35A (59353)',
 'TMEM35B (100506144)', 'TMEM37 (140738)', 'TMEM38A (79041)', 'TMEM38B (55151)',
 'TMEM39A (55254)', 'TMEM39B (55116)', 'TMEM40 (55287)', 'TMEM41A (90407)',
 'TMEM41B (440026)', 'TMEM42 (131616)', 'TMEM43 (79188)', 'TMEM44 (93109)',
 'TMEM45A (55076)', 'TMEM45B (120224)', 'TMEM47 (83604)', 'TMEM50A (23585)',
 'TMEM50B (757)', 'TMEM51 (55092)', 'TMEM52 (339456)', 'TMEM52B (120939)',
 'TMEM53 (79639)', 'TMEM54 (113452)', 'TMEM59 (9528)', 'TMEM59L (25789)', 'TMEM60
 (85025)', 'TMEM61 (199964)', 'TMEM62 (80021)', 'TMEM63A (9725)', 'TMEM63B
 (55362)', 'TMEM63C (57156)', 'TMEM64 (169200)', 'TMEM65 (157378)', 'TMEM67
 (91147)', 'TMEM68 (137695)', 'TMEM69 (51249)', 'TMEM70 (54968)', 'TMEM71
 (137835)', 'TMEM72 (643236)', 'TMEM74 (157753)', 'TMEM74B (55321)', 'TMEM79
 (84283)', 'TMEM80 (283232)', 'TMEM81 (388730)', 'TMEM82 (388595)', 'TMEM86A
 (144110)', 'TMEM86B (255043)', 'TMEM87A (25963)', 'TMEM87B (84910)', 'TMEM88
 (92162)', 'TMEM89 (440955)', 'TMEM8B (51754)', 'TMEM9 (252839)', 'TMEM91
 (641649)', 'TMEM92 (162461)', 'TMEM94 (9772)', 'TMEM95 (339168)', 'TMEM97
 (27346)', 'TMEM98 (26022)', 'TMEM9B (56674)', 'TMF1 (7110)', 'TMIE (259236)',
 'TMIGD1 (388364)', 'TMIGD2 (126259)', 'TMIGD3 (57413)', 'TMLHE (55217)', 'TMOD1
 (7111)', 'TMOD2 (29767)', 'TMOD3 (29766)', 'TMOD4 (29765)', 'TMP0 (7112)',
 'TMPPE (643853)', 'TMPRSS11A (339967)', 'TMPRSS11B (132724)', 'TMPRSS11D
 (9407)', 'TMPRSS11E (28983)', 'TMPRSS11F (389208)', 'TMPRSS12 (283471)',
 'TMPRSS13 (84000)', 'TMPRSS15 (5651)', 'TMPRSS2 (7113)', 'TMPRSS3 (64699)',
 'TMPRSS4 (56649)', 'TMPRSS5 (80975)', 'TMPRSS6 (164656)', 'TMPRSS7 (344805)',
 'TMPRSS9 (360200)', 'TMSB10 (9168)', 'TMSB15A (11013)', 'TMSB4X (7114)', 'TMSB4Y
 (9087)', 'TMT1A (25840)', 'TMT1B (196410)', 'TMTC1 (83857)', 'TMTC2 (160335)',
 'TMTC3 (160418)', 'TMTC4 (84899)', 'TMUB1 (83590)', 'TMUB2 (79089)', 'TMX1
 (81542)', 'TMX2 (51075)', 'TMX3 (54495)', 'TMX4 (56255)', 'TNC (3371)', 'TNF
 (7124)', 'TNFAIP1 (7126)', 'TNFAIP2 (7127)', 'TNFAIP3 (7128)', 'TNFAIP6 (7130)',

'TNFAIP8 (25816)', 'TNFAIP8L1 (126282)', 'TNFAIP8L2 (79626)', 'TNFAIP8L3 (388121)', 'TNFRSF10A (8797)', 'TNFRSF10B (8795)', 'TNFRSF10C (8794)', 'TNFRSF10D (8793)', 'TNFRSF11A (8792)', 'TNFRSF11B (4982)', 'TNFRSF12A (51330)', 'TNFRSF13B (23495)', 'TNFRSF13C (115650)', 'TNFRSF14 (8764)', 'TNFRSF17 (608)', 'TNFRSF18 (8784)', 'TNFRSF19 (55504)', 'TNFRSF1A (7132)', 'TNFRSF1B (7133)', 'TNFRSF21 (27242)', 'TNFRSF25 (8718)', 'TNFRSF4 (7293)', 'TNFRSF6B (8771)', 'TNFRSF8 (943)', 'TNFRSF9 (3604)', 'TNFSF10 (8743)', 'TNFSF11 (8600)', 'TNFSF12 (8742)', 'TNFSF13 (8741)', 'TNFSF13B (10673)', 'TNFSF14 (8740)', 'TNFSF15 (9966)', 'TNFSF18 (8995)', 'TNFSF4 (7292)', 'TNFSF8 (944)', 'TNFSF9 (8744)', 'TNIK (23043)', 'TNIP1 (10318)', 'TNIP2 (79155)', 'TNIP3 (79931)', 'TNK1 (8711)', 'TNK2 (10188)', 'TNKS (8658)', 'TNKS1BP1 (85456)', 'TNKS2 (80351)', 'TNMD (64102)', 'TNN (63923)', 'TNNC1 (7134)', 'TNNC2 (7125)', 'TNNI1 (7135)', 'TNNI2 (7136)', 'TNNI3 (7137)', 'TNNI3K (51086)', 'TNNT1 (7138)', 'TNNT2 (7139)', 'TNNT3 (7140)', 'TNP1 (7141)', 'TNP2 (7142)', 'TNPO1 (3842)', 'TNPO2 (30000)', 'TNPO3 (23534)', 'TNR (7143)', 'TNRC18 (84629)', 'TNRC6A (27327)', 'TNRC6B (23112)', 'TNRC6C (57690)', 'TNS1 (7145)', 'TNS2 (23371)', 'TNS3 (64759)', 'TNS4 (84951)', 'TNXB (7148)', 'TOB1 (10140)', 'TOB2 (10766)', 'TOE1 (114034)', 'TOGARAM1 (23116)', 'TOGARAM2 (165186)', 'TOLLIP (54472)', 'TOM1 (10043)', 'TOM1L1 (10040)', 'TOM1L2 (146691)', 'TOMM20 (9804)', 'TOMM20L (387990)', 'TOMM22 (56993)', 'TOMM34 (10953)', 'TOMM40 (10452)', 'TOMM40L (84134)', 'TOMM5 (401505)', 'TOMM6 (100188893)', 'TOMM7 (54543)', 'TOMM70 (9868)', 'TONSL (4796)', 'TOP1 (7150)', 'TOP1MT (116447)', 'TOP2A (7153)', 'TOP2B (7155)', 'TOP3A (7156)', 'TOP3B (8940)', 'TOP6BL (79703)', 'TOPAZ1 (375337)', 'TOPBP1 (11073)', 'TOPORS (10210)', 'TOR1A (1861)', 'TOR1AIP1 (26092)', 'TOR1AIP2 (163590)', 'TOR1B (27348)', 'TOR2A (27433)', 'TOR3A (64222)', 'TOR4A (54863)', 'TOX (9760)', 'TOX2 (84969)', 'TOX3 (27324)', 'TOX4 (9878)', 'TP53 (7157)', 'TP53AIP1 (63970)', 'TP53BP1 (7158)', 'TP53BP2 (7159)', 'TP53I11 (9537)', 'TP53I13 (90313)', 'TP53I3 (9540)', 'TP53INP1 (94241)', 'TP53INP2 (58476)', 'TP53RK (112858)', 'TP53TG5 (27296)', 'TP63 (8626)', 'TP73 (7161)', 'TPBG (7162)', 'TPBGL (100507050)', 'TPCN1 (53373)', 'TPCN2 (219931)', 'TPD52 (7163)', 'TPD52L1 (7164)', 'TPD52L2 (7165)', 'TPD52L3 (89882)', 'TPGS1 (91978)', 'TPGS2 (25941)', 'TPH1 (7166)', 'TPH2 (121278)', 'TPI1 (7167)', 'TPK1 (27010)', 'TPM1 (7168)', 'TPM2 (7169)', 'TPM3 (7170)', 'TPM4 (7171)', 'TPMT (7172)', 'TP0 (7173)', 'TPP1 (1200)', 'TPP2 (7174)', 'TPPP (11076)', 'TPPP2 (122664)', 'TPPP3 (51673)', 'TPR (7175)', 'TPRA1 (131601)', 'TPRG1 (285386)', 'TPRG1L (127262)', 'TPRKB (51002)', 'TPRN (286262)', 'TPRX1 (284355)', 'TPSAB1 (7177)', 'TPSD1 (23430)', 'TPSG1 (25823)', 'TPST1 (8460)', 'TPST2 (8459)', 'TPT1 (7178)', 'TPTE2 (93492)', 'TPX2 (22974)', 'TRA2A (29896)', 'TRA2B (6434)', 'TRABD (80305)', 'TRABD2A (129293)', 'TRABD2B (388630)', 'TRADD (8717)', 'TRAF1 (7185)', 'TRAF2 (7186)', 'TRAF3 (7187)', 'TRAF3IP1 (26146)', 'TRAF3IP2 (10758)', 'TRAF3IP3 (80342)', 'TRAF4 (9618)', 'TRAF5 (7188)', 'TRAF6 (7189)', 'TRAF7 (84231)', 'TRAFD1 (10906)', 'TRAIP (10293)', 'TRAK1 (22906)', 'TRAK2 (66008)', 'TRAM1 (23471)', 'TRAM1L1 (133022)', 'TRAM2 (9697)', 'TRANK1 (9881)', 'TRAP1 (10131)', 'TRAPPC1 (58485)', 'TRAPPC10 (7109)', 'TRAPPC11 (60684)', 'TRAPPC12 (51112)', 'TRAPPC13 (80006)', 'TRAPPC14 (55262)', 'TRAPPC2 (6399)', 'TRAPPC2L (51693)', 'TRAPPC3 (27095)', 'TRAPPC3L (100128327)', 'TRAPPC4 (51399)', 'TRAPPC5 (126003)', 'TRAPPC6A (79090)', 'TRAPPC6B (122553)', 'TRAPPC8 (22878)', 'TRAPPC9 (83696)', 'TRARG1 (286753)', 'TRAT1 (50852)', 'TRDMT1 (1787)', 'TRDN (10345)',

'TREH (11181)', 'TREM1 (54210)', 'TREM2 (54209)', 'TREML1 (340205)', 'TREML2 (79865)', 'TREML4 (285852)', 'TRERF1 (55809)', 'TRESX1 (11277)', 'TRESX2 (11219)', 'TRH (7200)', 'TRHDE (29953)', 'TRHR (7201)', 'TRIAP1 (51499)', 'TRIB1 (10221)', 'TRIB2 (28951)', 'TRIB3 (57761)', 'TRIL (9865)', 'TRIM10 (10107)', 'TRIM11 (81559)', 'TRIM13 (10206)', 'TRIM14 (9830)', 'TRIM15 (89870)', 'TRIM16 (10626)', 'TRIM17 (51127)', 'TRIM2 (23321)', 'TRIM21 (6737)', 'TRIM22 (10346)', 'TRIM23 (373)', 'TRIM24 (8805)', 'TRIM25 (7706)', 'TRIM26 (7726)', 'TRIM27 (5987)', 'TRIM28 (10155)', 'TRIM29 (23650)', 'TRIM3 (10612)', 'TRIM31 (11074)', 'TRIM32 (22954)', 'TRIM33 (51592)', 'TRIM34 (53840)', 'TRIM35 (23087)', 'TRIM36 (55521)', 'TRIM37 (4591)', 'TRIM38 (10475)', 'TRIM39 (56658)', 'TRIM4 (89122)', 'TRIM40 (135644)', 'TRIM41 (90933)', 'TRIM42 (287015)', 'TRIM43 (129868)', 'TRIM44 (54765)', 'TRIM45 (80263)', 'TRIM46 (80128)', 'TRIM47 (91107)', 'TRIM48 (79097)', 'TRIM49B (283116)', 'TRIM49D1 (399939)', 'TRIM5 (85363)', 'TRIM50 (135892)', 'TRIM52 (84851)', 'TRIM54 (57159)', 'TRIM55 (84675)', 'TRIM56 (81844)', 'TRIM58 (25893)', 'TRIM59 (286827)', 'TRIM6 (117854)', 'TRIM60 (166655)', 'TRIM61 (391712)', 'TRIM62 (55223)', 'TRIM63 (84676)', 'TRIM64C (646754)', 'TRIM65 (201292)', 'TRIM66 (9866)', 'TRIM67 (440730)', 'TRIM68 (55128)', 'TRIM69 (140691)', 'TRIM7 (81786)', 'TRIM71 (131405)', 'TRIM72 (493829)', 'TRIM77 (390231)', 'TRIM8 (81603)', 'TRIM9 (114088)', 'TRIML1 (339976)', 'TRIML2 (205860)', 'TRIO (7204)', 'TRIOBP (11078)', 'TRIP10 (9322)', 'TRIP11 (9321)', 'TRIP12 (9320)', 'TRIP13 (9319)', 'TRIP4 (9325)', 'TRIP6 (7205)', 'TRIQK (286144)', 'TRIR (79002)', 'TRIT1 (54802)', 'TRMO (51531)', 'TRMT1 (55621)', 'TRMT10A (93587)', 'TRMT10B (158234)', 'TRMT10C (54931)', 'TRMT11 (60487)', 'TRMT112 (51504)', 'TRMT12 (55039)', 'TRMT13 (54482)', 'TRMT1L (81627)', 'TRMT2A (27037)', 'TRMT2B (79979)', 'TRMT44 (152992)', 'TRMT5 (57570)', 'TRMT6 (51605)', 'TRMT61A (115708)', 'TRMT61B (55006)', 'TRMT9B (57604)', 'TRMU (55687)', 'TRNAU1AP (54952)', 'TRNP1 (388610)', 'TRNT1 (51095)', 'TRO (7216)', 'TROAP (10024)', 'TRPA1 (8989)', 'TRPC1 (7220)', 'TRPC3 (7222)', 'TRPC4 (7223)', 'TRPC4AP (26133)', 'TRPC5 (7224)', 'TRPC5OS (100329135)', 'TRPC6 (7225)', 'TRPC7 (57113)', 'TRPM1 (4308)', 'TRPM2 (7226)', 'TRPM3 (80036)', 'TRPM4 (54795)', 'TRPM5 (29850)', 'TRPM6 (140803)', 'TRPM7 (54822)', 'TRPM8 (79054)', 'TRPS1 (7227)', 'TRPT1 (83707)', 'TRPV1 (7442)', 'TRPV2 (51393)', 'TRPV3 (162514)', 'TRPV4 (59341)', 'TRPV5 (56302)', 'TRPV6 (55503)', 'TRRAP (8295)', 'TRUB1 (142940)', 'TRUB2 (26995)', 'TSACC (128229)', 'TSBP1 (10665)', 'TSC1 (7248)', 'TSC2 (7249)', 'TSC22D1 (8848)', 'TSC22D2 (9819)', 'TSC22D3 (1831)', 'TSC22D4 (81628)', 'TSEN15 (116461)', 'TSEN2 (80746)', 'TSEN34 (79042)', 'TSEN54 (283989)', 'TSFM (10102)', 'TSG101 (7251)', 'TSGA10 (80705)', 'TSGA10IP (254187)', 'TSGA13 (114960)', 'TSHB (7252)', 'TSHR (7253)', 'TSHZ1 (10194)', 'TSHZ2 (128553)', 'TSHZ3 (57616)', 'TSKS (60385)', 'TSKU (25987)', 'TSLP (85480)', 'TSN (7247)', 'TSNARE1 (203062)', 'TSNAX (7257)', 'TSNAXIP1 (55815)', 'TSPAN1 (10103)', 'TSPAN10 (83882)', 'TSPAN11 (441631)', 'TSPAN12 (23554)', 'TSPAN13 (27075)', 'TSPAN14 (81619)', 'TSPAN15 (23555)', 'TSPAN16 (26526)', 'TSPAN17 (26262)', 'TSPAN18 (90139)', 'TSPAN19 (144448)', 'TSPAN2 (10100)', 'TSPAN3 (10099)', 'TSPAN31 (6302)', 'TSPAN32 (10077)', 'TSPAN33 (340348)', 'TSPAN4 (7106)', 'TSPAN5 (10098)', 'TSPAN6 (7105)', 'TSPAN7 (7102)', 'TSPAN8 (7103)', 'TSPAN9 (10867)', 'TSPEAR (54084)', 'TSPO (706)', 'TSPO2 (222642)', 'TSPOAP1 (9256)', 'TSPYL1 (7259)', 'TSPYL2 (64061)', 'TSPYL4 (23270)', 'TSPYL5 (85453)', 'TSPYL6 (388951)', 'TSR1 (55720)', 'TSR2 (90121)',

'TSR3 (115939)', 'TSSC4 (10078)', 'TSSK1B (83942)', 'TSSK2 (23617)', 'TSSK3 (81629)', 'TSSK4 (283629)', 'TSSK6 (83983)', 'TST (7263)', 'TSTD1 (100131187)', 'TSTD2 (158427)', 'TSTD3 (100130890)', 'TTBK1 (84630)', 'TTBK2 (146057)', 'TTC1 (7265)', 'TTC12 (54970)', 'TTC13 (79573)', 'TTC14 (151613)', 'TTC16 (158248)', 'TTC17 (55761)', 'TTC19 (54902)', 'TTC21A (199223)', 'TTC21B (79809)', 'TTC22 (55001)', 'TTC23 (64927)', 'TTC23L (153657)', 'TTC24 (164118)', 'TTC27 (55622)', 'TTC28 (23331)', 'TTC29 (83894)', 'TTC3 (7267)', 'TTC31 (64427)', 'TTC32 (130502)', 'TTC33 (23548)', 'TTC34 (100287898)', 'TTC36 (143941)', 'TTC38 (55020)', 'TTC39A (22996)', 'TTC39B (158219)', 'TTC39C (125488)', 'TTC4 (7268)', 'TTC5 (91875)', 'TTC6 (319089)', 'TTC7A (57217)', 'TTC7B (145567)', 'TTC8 (123016)', 'TTC9 (23508)', 'TTC9B (148014)', 'TTC9C (283237)', 'TTF1 (7270)', 'TTF2 (8458)', 'TTI1 (9675)', 'TTI2 (80185)', 'TTK (7272)', 'TTL (150465)', 'TTLL1 (25809)', 'TTLL10 (254173)', 'TTLL11 (158135)', 'TTLL12 (23170)', 'TTLL2 (83887)', 'TTLL3 (26140)', 'TTLL4 (9654)', 'TTLL5 (23093)', 'TTLL6 (284076)', 'TTLL7 (79739)', 'TTLL8 (164714)', 'TTLL9 (164395)', 'TTN (7273)', 'TTPA (7274)', 'TTPAL (79183)', 'TTR (7276)', 'TTYH1 (57348)', 'TTYH2 (94015)', 'TTYH3 (80727)', 'TUB (7275)', 'TUBA1A (7846)', 'TUBA1B (10376)', 'TUBA1C (84790)', 'TUBA3C (7278)', 'TUBA3D (113457)', 'TUBA3E (112714)', 'TUBA4A (7277)', 'TUBA4B (80086)', 'TUBA8 (51807)', 'TUBAL3 (79861)', 'TUBB (203068)', 'TUBB1 (81027)', 'TUBB2A (7280)', 'TUBB2B (347733)', 'TUBB3 (10381)', 'TUBB4A (10382)', 'TUBB4B (10383)', 'TUBB6 (84617)', 'TUBB8 (347688)', 'TUBD1 (51174)', 'TUBE1 (51175)', 'TUBG1 (7283)', 'TUBG2 (27175)', 'TUBGCP2 (10844)', 'TUBGCP3 (10426)', 'TUBGCP4 (27229)', 'TUBGCP5 (114791)', 'TUBGCP6 (85378)', 'TUFM (7284)', 'TUFT1 (7286)', 'TULP1 (7287)', 'TULP2 (7288)', 'TULP3 (7289)', 'TULP4 (56995)', 'TUSC1 (286319)', 'TUSC2 (11334)', 'TUSC3 (7991)', 'TUT1 (64852)', 'TUT4 (23318)', 'TUT7 (79670)', 'TVP23A (780776)', 'TVP23C (201158)', 'TWF1 (5756)', 'TWF2 (11344)', 'TWIST1 (7291)', 'TWIST2 (117581)', 'TWNK (56652)', 'TWSG1 (57045)', 'TXK (7294)', 'TXLNA (200081)', 'TXLNB (167838)', 'TXLNG (55787)', 'TXN (7295)', 'TXN2 (25828)', 'TXNDC11 (51061)', 'TXNDC12 (51060)', 'TXNDC15 (79770)', 'TXNDC16 (57544)', 'TXNDC17 (84817)', 'TXNDC2 (84203)', 'TXNDC5 (81567)', 'TXNDC8 (255220)', 'TXNDC9 (10190)', 'TXNIP (10628)', 'TXNL1 (9352)', 'TXNL4A (10907)', 'TXNL4B (54957)', 'TXNRD1 (7296)', 'TXNRD2 (10587)', 'TXNRD3 (114112)', 'TYK2 (7297)', 'TYMP (1890)', 'TYMS (7298)', 'TYR (7299)', 'TYR03 (7301)', 'TYROBP (7305)', 'TYRP1 (7306)', 'TYSND1 (219743)', 'TYW1 (55253)', 'TYW1B (441250)', 'TYW3 (127253)', 'TYW5 (129450)', 'U2AF1 (7307)', 'U2AF1L4 (199746)', 'U2AF2 (11338)', 'U2SURP (23350)', 'UACA (55075)', 'UAP1 (6675)', 'UAP1L1 (91373)', 'UBA1 (7317)', 'UBA2 (10054)', 'UBA3 (9039)', 'UBA5 (79876)', 'UBA52 (7311)', 'UBA6 (55236)', 'UBA7 (7318)', 'UBAC1 (10422)', 'UBAC2 (337867)', 'UBALD1 (124402)', 'UBALD2 (283991)', 'UBAP1 (51271)', 'UBAP1L (390595)', 'UBAP2 (55833)', 'UBAP2L (9898)', 'UBASH3A (53347)', 'UBASH3B (84959)', 'UBB (7314)', 'UBC (7316)', 'UBD (10537)', 'UBE2A (7319)', 'UBE2B (7320)', 'UBE2C (11065)', 'UBE2D1 (7321)', 'UBE2D2 (7322)', 'UBE2D3 (7323)', 'UBE2D4 (51619)', 'UBE2E1 (7324)', 'UBE2E2 (7325)', 'UBE2E3 (10477)', 'UBE2F (140739)', 'UBE2G1 (7326)', 'UBE2G2 (7327)', 'UBE2H (7328)', 'UBE2I (7329)', 'UBE2J1 (51465)', 'UBE2J2 (118424)', 'UBE2K (3093)', 'UBE2L3 (7332)', 'UBE2L5 (171222)', 'UBE2L6 (9246)', 'UBE2M (9040)', 'UBE2N (7334)', 'UBE2O (63893)', 'UBE2Q1 (55585)', 'UBE2Q2 (92912)', 'UBE2QL1 (134111)', 'UBE2R2 (54926)', 'UBE2S (27338)', 'UBE2T (29089)', 'UBE2U (148581)', 'UBE2V1 (7335)', 'UBE2V2 (7336)',

'UBE2W (55284)', 'UBE2Z (65264)', 'UBE3A (7337)', 'UBE3B (89910)', 'UBE3C (9690)', 'UBE3D (90025)', 'UBE4A (9354)', 'UBE4B (10277)', 'UBFD1 (56061)', 'UBIAD1 (29914)', 'UBL3 (5412)', 'UBL4A (8266)', 'UBL4B (164153)', 'UBL5 (59286)', 'UBL7 (84993)', 'UBLCP1 (134510)', 'UBN1 (29855)', 'UBN2 (254048)', 'UBOX5 (22888)', 'UBP1 (7342)', 'UBQLN1 (29979)', 'UBQLN2 (29978)', 'UBQLN3 (50613)', 'UBQLN4 (56893)', 'UBQLNL (143630)', 'UBR1 (197131)', 'UBR2 (23304)', 'UBR3 (130507)', 'UBR4 (23352)', 'UBR5 (51366)', 'UBR7 (55148)', 'UBTD1 (80019)', 'UBTD2 (92181)', 'UBTF (7343)', 'UBTFL1 (642623)', 'UBXN1 (51035)', 'UBXN10 (127733)', 'UBXN11 (91544)', 'UBXN2A (165324)', 'UBXN2B (137886)', 'UBXN4 (23190)', 'UBXN6 (80700)', 'UBXN7 (26043)', 'UBXN8 (7993)', 'UCHL1 (7345)', 'UCHL3 (7347)', 'UCHL5 (51377)', 'UCK1 (83549)', 'UCK2 (7371)', 'UCKL1 (54963)', 'UCMA (221044)', 'UCN (7349)', 'UCN2 (90226)', 'UCN3 (114131)', 'UCP1 (7350)', 'UCP2 (7351)', 'UCP3 (7352)', 'UEVLD (55293)', 'UFC1 (51506)', 'UFD1 (7353)', 'UFL1 (23376)', 'UFM1 (51569)', 'UFSP1 (402682)', 'UFSP2 (55325)', 'UGCG (7357)', 'UGDH (7358)', 'UGGT1 (56886)', 'UGGT2 (55757)', 'UGP2 (7360)', 'UGT1A1 (54658)', 'UGT1A10 (54575)', 'UGT1A3 (54659)', 'UGT1A4 (54657)', 'UGT1A5 (54579)', 'UGT1A6 (54578)', 'UGT1A7 (54577)', 'UGT1A8 (54576)', 'UGT1A9 (54600)', 'UGT2A1 (10941)', 'UGT2A3 (79799)', 'UGT2B10 (7365)', 'UGT2B11 (10720)', 'UGT2B15 (7366)', 'UGT2B17 (7367)', 'UGT2B28 (54490)', 'UGT2B4 (7363)', 'UGT2B7 (7364)', 'UGT3A1 (133688)', 'UGT3A2 (167127)', 'UGT8 (7368)', 'UHMK1 (127933)', 'UHRF1 (29128)', 'UHRF2 (115426)', 'UIMC1 (51720)', 'ULBP1 (80329)', 'ULBP2 (80328)', 'ULBP3 (79465)', 'ULK1 (8408)', 'ULK2 (9706)', 'ULK3 (25989)', 'ULK4 (54986)', 'UMAD1 (729852)', 'UMOD (7369)', 'UMODL1 (89766)', 'UMPS (7372)', 'UNC119 (9094)', 'UNC119B (84747)', 'UNC13A (23025)', 'UNC13B (10497)', 'UNC13C (440279)', 'UNC13D (201294)', 'UNC45A (55898)', 'UNC45B (146862)', 'UNC50 (25972)', 'UNC5A (90249)', 'UNC5B (219699)', 'UNC5C (8633)', 'UNC5CL (222643)', 'UNC5D (137970)', 'UNC79 (57578)', 'UNC80 (285175)', 'UNC93A (54346)', 'UNC93B1 (81622)', 'UNCX (340260)', 'UNG (7374)', 'UNK (85451)', 'UNKL (64718)', 'UPB1 (51733)', 'UPF1 (5976)', 'UPF2 (26019)', 'UPF3A (65110)', 'UPF3B (65109)', 'UPK1A (11045)', 'UPK1B (7348)', 'UPK2 (7379)', 'UPK3A (7380)', 'UPK3B (105375355)', 'UPP1 (7378)', 'UPP2 (151531)', 'UPRT (139596)', 'UQCC1 (55245)', 'UQCC2 (84300)', 'UQCC3 (790955)', 'UQCC4 (283951)', 'UQCC5 (440957)', 'UQCC6 (728568)', 'UQCR10 (29796)', 'UQCR11 (10975)', 'UQCRB (7381)', 'UQCRC1 (7384)', 'UQCRC2 (7385)', 'UQCRFS1 (7386)', 'UQCRHL (440567)', 'UQCRQ (27089)', 'URAD (646625)', 'URB1 (9875)', 'URB2 (9816)', 'URGCP (55665)', 'URI1 (8725)', 'URM1 (81605)', 'UROC1 (131669)', 'UROD (7389)', 'UROS (7390)', 'USB1 (79650)', 'USE1 (55850)', 'USF1 (7391)', 'USF2 (7392)', 'USF3 (205717)', 'USH1C (10083)', 'USH1G (124590)', 'USH2A (7399)', 'USHBP1 (83878)', 'USO1 (8615)', 'USP1 (7398)', 'USP10 (9100)', 'USP11 (8237)', 'USP12 (219333)', 'USP13 (8975)', 'USP14 (9097)', 'USP15 (9958)', 'USP16 (10600)', 'USP17L10 (100287144)', 'USP17L13 (100287238)', 'USP17L7 (392197)', 'USP18 (11274)', 'USP19 (10869)', 'USP2 (9099)', 'USP20 (10868)', 'USP21 (27005)', 'USP22 (23326)', 'USP24 (23358)', 'USP25 (29761)', 'USP26 (83844)', 'USP27X (389856)', 'USP28 (57646)', 'USP29 (57663)', 'USP3 (9960)', 'USP30 (84749)', 'USP31 (57478)', 'USP32 (84669)', 'USP33 (23032)', 'USP34 (9736)', 'USP35 (57558)', 'USP36 (57602)', 'USP37 (57695)', 'USP38 (84640)', 'USP39 (10713)', 'USP4 (7375)', 'USP40 (55230)', 'USP42 (84132)', 'USP43 (124739)', 'USP44 (84101)', 'USP45 (85015)', 'USP46 (64854)', 'USP47 (55031)', 'USP48 (84196)', 'USP49 (25862)', 'USP5 (8078)',

'USP50 (373509)', 'USP51 (158880)', 'USP53 (54532)', 'USP54 (159195)', 'USP6
 (9098)', 'USP6NL (9712)', 'USP7 (7874)', 'USP8 (9101)', 'USP9X (8239)', 'USP9Y
 (8287)', 'USPL1 (10208)', 'UST (10090)', 'UTF1 (8433)', 'UTP11 (51118)', 'UTP14A
 (10813)', 'UTP14C (9724)', 'UTP15 (84135)', 'UTP18 (51096)', 'UTP20 (27340)',
 'UTP23 (84294)', 'UTP25 (27042)', 'UTP3 (57050)', 'UTP4 (84916)', 'UTP6
 (55813)', 'UTRN (7402)', 'UTS2 (10911)', 'UTS2B (257313)', 'UTS2R (2837)', 'UTY
 (7404)', 'UVRAG (7405)', 'UVSSA (57654)', 'UXS1 (80146)', 'UXT (8409)', 'VAC14
 (55697)', 'VAMP1 (6843)', 'VAMP2 (6844)', 'VAMP3 (9341)', 'VAMP4 (8674)', 'VAMP5
 (10791)', 'VAMP7 (6845)', 'VAMP8 (8673)', 'VANGL1 (81839)', 'VANGL2 (57216)',
 'VAPA (9218)', 'VAPB (9217)', 'VARs1 (7407)', 'VARs2 (57176)', 'VASH1 (22846)',
 'VASH2 (79805)', 'VASN (114990)', 'VASP (7408)', 'VAT1 (10493)', 'VAT1L
 (57687)', 'VAV1 (7409)', 'VAV2 (7410)', 'VAV3 (10451)', 'VAX1 (11023)', 'VAX2
 (25806)', 'VBP1 (7411)', 'VCAM1 (7412)', 'VCAN (1462)', 'VCF1 (84923)', 'VCF2
 (90736)', 'VCL (7414)', 'VCP (7415)', 'VCP1P1 (80124)', 'VCPKMT (79609)', 'VDAC1
 (7416)', 'VDAC2 (7417)', 'VDAC3 (7419)', 'VDR (7421)', 'VEGFA (7422)', 'VEGFB
 (7423)', 'VEGFC (7424)', 'VEGFD (2277)', 'VENTX (27287)', 'VEPH1 (79674)',
 'VEZF1 (7716)', 'VEZT (55591)', 'VGF (7425)', 'VGLL1 (51442)', 'VGLL2 (245806)',
 'VGLL3 (389136)', 'VGLL4 (9686)', 'VHL (7428)', 'VIL1 (7429)', 'VILL (50853)',
 'VIM (7431)', 'VIP (7432)', 'VIPAS39 (63894)', 'VIPR1 (7433)', 'VIPR2 (7434)',
 'VIRMA (25962)', 'VIT (5212)', 'VKORC1 (79001)', 'VKORC1L1 (154807)', 'VLDLR
 (7436)', 'VMA12 (147007)', 'VMA21 (203547)', 'VMA22 (84317)', 'VMAC (400673)',
 'VM01 (284013)', 'VMP1 (81671)', 'VN1R1 (57191)', 'VN1R2 (317701)', 'VN1R4
 (317703)', 'VNN1 (8876)', 'VNN2 (8875)', 'VOPP1 (81552)', 'VPREB1 (7441)',
 'VPREB3 (29802)', 'VPS11 (55823)', 'VPS13A (23230)', 'VPS13B (157680)', 'VPS13C
 (54832)', 'VPS13D (55187)', 'VPS16 (64601)', 'VPS18 (57617)', 'VPS25 (84313)',
 'VPS26A (9559)', 'VPS26B (112936)', 'VPS26C (10311)', 'VPS28 (51160)', 'VPS29
 (51699)', 'VPS33A (65082)', 'VPS33B (26276)', 'VPS35 (55737)', 'VPS35L (57020)',
 'VPS36 (51028)', 'VPS37A (137492)', 'VPS37B (79720)', 'VPS37C (55048)', 'VPS37D
 (155382)', 'VPS39 (23339)', 'VPS41 (27072)', 'VPS45 (11311)', 'VPS4A (27183)',
 'VPS4B (9525)', 'VPS50 (55610)', 'VPS51 (738)', 'VPS52 (6293)', 'VPS53 (55275)',
 'VPS54 (51542)', 'VPS72 (6944)', 'VPS8 (23355)', 'VPS9D1 (9605)', 'VRK1 (7443)',
 'VRK2 (7444)', 'VRK3 (51231)', 'VRTN (55237)', 'VSIG1 (340547)', 'VSIG10
 (54621)', 'VSIG10L (147645)', 'VSIG10L2 (338667)', 'VSIG2 (23584)', 'VSIG4
 (11326)', 'VSIG8 (391123)', 'VSIR (64115)', 'VSNL1 (7447)', 'VSTM1 (284415)',
 'VSTM2A (222008)', 'VSTM2B (342865)', 'VSTM2L (128434)', 'VSTM4 (196740)',
 'VSTM5 (387804)', 'VSX1 (30813)', 'VSX2 (338917)', 'VTA1 (51534)', 'VTCN1
 (79679)', 'VTI1A (143187)', 'VTI1B (10490)', 'VTN (7448)', 'VWA1 (64856)', 'VWA2
 (340706)', 'VWA3A (146177)', 'VWA3B (200403)', 'VWA5A (4013)', 'VWA5B2 (90113)',
 'VWA7 (80737)', 'VWA8 (23078)', 'VWC2 (375567)', 'VWC2L (402117)', 'VWCE
 (220001)', 'VWDE (221806)', 'VWF (7450)', 'VXN (254778)', 'WAC (51322)', 'WAPL
 (23063)', 'WARS1 (7453)', 'WARS2 (10352)', 'WAS (7454)', 'WASF1 (8936)', 'WASF2
 (10163)', 'WASF3 (10810)', 'WASHC2A (387680)', 'WASHC2C (253725)', 'WASHC3
 (51019)', 'WASHC4 (23325)', 'WASHC5 (9897)', 'WASL (8976)', 'WBP1 (23559)',
 'WBP11 (51729)', 'WBP1L (54838)', 'WBP2 (23558)', 'WBP2NL (164684)', 'WBP4
 (11193)', 'WDCP (80304)', 'WDFY1 (57590)', 'WDFY2 (115825)', 'WDFY3 (23001)',
 'WDFY4 (57705)', 'WDHD1 (11169)', 'WDPCP (51057)', 'WDR1 (9948)', 'WDR11
 (55717)', 'WDR12 (55759)', 'WDR13 (64743)', 'WDR17 (116966)', 'WDR18 (57418)',
 'WDR19 (57728)', 'WDR20 (91833)', 'WDR24 (84219)', 'WDR25 (79446)', 'WDR26

(80232)', 'WDR27 (253769)', 'WDR3 (10885)', 'WDR31 (114987)', 'WDR33 (55339)',
 'WDR35 (57539)', 'WDR36 (134430)', 'WDR37 (22884)', 'WDR38 (401551)', 'WDR4
 (10785)', 'WDR41 (55255)', 'WDR43 (23160)', 'WDR44 (54521)', 'WDR45 (11152)',
 'WDR45B (56270)', 'WDR46 (9277)', 'WDR47 (22911)', 'WDR48 (57599)', 'WDR49
 (151790)', 'WDR5 (11091)', 'WDR53 (348793)', 'WDR54 (84058)', 'WDR55 (54853)',
 'WDR59 (79726)', 'WDR5B (54554)', 'WDR6 (11180)', 'WDR62 (284403)', 'WDR64
 (128025)', 'WDR7 (23335)', 'WDR70 (55100)', 'WDR72 (256764)', 'WDR73 (84942)',
 'WDR74 (54663)', 'WDR75 (84128)', 'WDR76 (79968)', 'WDR77 (79084)', 'WDR81
 (124997)', 'WDR82 (80335)', 'WDR83 (84292)', 'WDR830S (51398)', 'WDR86
 (349136)', 'WDR87 (83889)', 'WDR88 (126248)', 'WDR89 (112840)', 'WDR90
 (197335)', 'WDR91 (29062)', 'WDR93 (56964)', 'WDR97 (340390)', 'WDSUB1
 (151525)', 'WDTCl (23038)', 'WEE1 (7465)', 'WEE2 (494551)', 'WFDC1 (58189)',
 'WFDC10A (140832)', 'WFDC10B (280664)', 'WFDC11 (259239)', 'WFDC12 (128488)',
 'WFDC13 (164237)', 'WFDC2 (10406)', 'WFDC3 (140686)', 'WFDC5 (149708)', 'WFDC6
 (140870)', 'WFDC8 (90199)', 'WFDC9 (259240)', 'WFIKKN1 (117166)', 'WFIKKN2
 (124857)', 'WFS1 (7466)', 'WHAMM (123720)', 'WHR1 (8859)', 'WHRN (25861)', 'WIF1
 (11197)', 'WIPF1 (7456)', 'WIPF2 (147179)', 'WIPF3 (644150)', 'WIPI1 (55062)',
 'WIPI2 (26100)', 'WIZ (58525)', 'WLS (79971)', 'WNK1 (65125)', 'WNK2 (65268)',
 'WNK3 (65267)', 'WNK4 (65266)', 'WNT1 (7471)', 'WNT10A (80326)', 'WNT10B
 (7480)', 'WNT11 (7481)', 'WNT16 (51384)', 'WNT2 (7472)', 'WNT2B (7482)', 'WNT3
 (7473)', 'WNT3A (89780)', 'WNT4 (54361)', 'WNT5A (7474)', 'WNT5B (81029)', 'WNT6
 (7475)', 'WNT7A (7476)', 'WNT7B (7477)', 'WNT8A (7478)', 'WNT8B (7479)', 'WNT9A
 (7483)', 'WNT9B (7484)', 'WRAP53 (55135)', 'WRAP73 (49856)', 'WRN (7486)',
 'WRNIP1 (56897)', 'WSB1 (26118)', 'WSB2 (55884)', 'WSCD1 (23302)', 'WSCD2
 (9671)', 'WT1 (7490)', 'WTAP (9589)', 'WTIP (126374)', 'WWC1 (23286)', 'WWC2
 (80014)', 'WVOX (51741)', 'WWP1 (11059)', 'WWP2 (11060)', 'WWTR1 (25937)', 'XAB2
 (56949)', 'XAF1 (54739)', 'XAGE2 (9502)', 'XAGE3 (170626)', 'XAGE5 (170627)',
 'XBP1 (7494)', 'XCL1 (6375)', 'XCL2 (6846)', 'XCR1 (2829)', 'XDH (7498)', 'XG
 (7499)', 'XIAP (331)', 'XIRP1 (165904)', 'XIRP2 (129446)', 'XK (7504)', 'XKR3
 (150165)', 'XKR4 (114786)', 'XKR5 (389610)', 'XKR6 (286046)', 'XKR7 (343702)',
 'XKR8 (55113)', 'XKR9 (389668)', 'XKRX (402415)', 'XNDC1N (100133315)', 'XPA
 (7507)', 'XPC (7508)', 'XPNPEP1 (7511)', 'XPNPEP2 (7512)', 'XPNPEP3 (63929)',
 'XP01 (7514)', 'XP04 (64328)', 'XP05 (57510)', 'XP06 (23214)', 'XP07 (23039)',
 'XPOT (11260)', 'XPR1 (9213)', 'XRCC1 (7515)', 'XRCC2 (7516)', 'XRCC3 (7517)',
 'XRCC4 (7518)', 'XRCC5 (7520)', 'XRCC6 (2547)', 'XRN1 (54464)', 'XRN2 (22803)',
 'XRAA1 (143570)', 'XXYLT1 (152002)', 'XYLB (9942)', 'XYLT1 (64131)', 'XYLT2
 (64132)', 'YAE1 (57002)', 'YAF2 (10138)', 'YAP1 (10413)', 'YARS1 (8565)', 'YARS2
 (51067)', 'YBEY (54059)', 'YBX1 (4904)', 'YBX2 (51087)', 'YBX3 (8531)', 'YDJC
 (150223)', 'YEATS2 (55689)', 'YEATS4 (8089)', 'YES1 (7525)', 'YIF1A (10897)',
 'YIF1B (90522)', 'YIPF1 (54432)', 'YIPF2 (78992)', 'YIPF3 (25844)', 'YIPF4
 (84272)', 'YIPF5 (81555)', 'YIPF6 (286451)', 'YIPF7 (285525)', 'YJEFN3
 (374887)', 'YJU2 (55702)', 'YJU2B (81576)', 'YKT6 (10652)', 'YLPM1 (56252)',
 'YME1L1 (10730)', 'YOD1 (55432)', 'YPEL1 (29799)', 'YPEL2 (388403)', 'YPEL3
 (83719)', 'YPEL4 (219539)', 'YPEL5 (51646)', 'YRDC (79693)', 'YTHDC1 (91746)',
 'YTHDC2 (64848)', 'YTHDF1 (54915)', 'YTHDF2 (51441)', 'YTHDF3 (253943)', 'YWHAB
 (7529)', 'YWHAE (7531)', 'YWHAG (7532)', 'YWHAH (7533)', 'YWHAQ (10971)', 'YWHAZ
 (7534)', 'YY1 (7528)', 'YY1AP1 (55249)', 'YY2 (404281)', 'ZACN (353174)', 'ZAN
 (7455)', 'ZAP70 (7535)', 'ZAR1 (326340)', 'ZAR1L (646799)', 'ZBBX (79740)',

'ZBED1 (9189)', 'ZBED2 (79413)', 'ZBED3 (84327)', 'ZBED4 (9889)', 'ZBED6 (100381270)', 'ZBP1 (81030)', 'ZBTB1 (22890)', 'ZBTB10 (65986)', 'ZBTB11 (27107)', 'ZBTB12 (221527)', 'ZBTB14 (7541)', 'ZBTB16 (7704)', 'ZBTB17 (7709)', 'ZBTB18 (10472)', 'ZBTB2 (57621)', 'ZBTB20 (26137)', 'ZBTB21 (49854)', 'ZBTB22 (9278)', 'ZBTB24 (9841)', 'ZBTB25 (7597)', 'ZBTB26 (57684)', 'ZBTB3 (79842)', 'ZBTB32 (27033)', 'ZBTB33 (10009)', 'ZBTB34 (403341)', 'ZBTB37 (84614)', 'ZBTB38 (253461)', 'ZBTB39 (9880)', 'ZBTB4 (57659)', 'ZBTB40 (9923)', 'ZBTB41 (360023)', 'ZBTB42 (100128927)', 'ZBTB43 (23099)', 'ZBTB44 (29068)', 'ZBTB45 (84878)', 'ZBTB46 (140685)', 'ZBTB47 (92999)', 'ZBTB48 (3104)', 'ZBTB49 (166793)', 'ZBTB5 (9925)', 'ZBTB6 (10773)', 'ZBTB7A (51341)', 'ZBTB7B (51043)', 'ZBTB7C (201501)', 'ZBTB8A (653121)', 'ZBTB8B (728116)', 'ZBTB80S (339487)', 'ZBTB9 (221504)', 'ZC2HC1A (51101)', 'ZC2HC1B (153918)', 'ZC2HC1C (79696)', 'ZC3H10 (84872)', 'ZC3H11A (9877)', 'ZC3H12A (80149)', 'ZC3H12B (340554)', 'ZC3H12C (85463)', 'ZC3H12D (340152)', 'ZC3H13 (23091)', 'ZC3H14 (79882)', 'ZC3H15 (55854)', 'ZC3H18 (124245)', 'ZC3H3 (23144)', 'ZC3H4 (23211)', 'ZC3H6 (376940)', 'ZC3H7A (29066)', 'ZC3H7B (23264)', 'ZC3H8 (84524)', 'ZC3HAV1 (56829)', 'ZC3HAV1L (92092)', 'ZC3HC1 (51530)', 'ZC4H2 (55906)', 'ZCCHC10 (54819)', 'ZCCHC12 (170261)', 'ZCCHC13 (389874)', 'ZCCHC14 (23174)', 'ZCCHC17 (51538)', 'ZCCHC18 (644353)', 'ZCCHC2 (54877)', 'ZCCHC24 (219654)', 'ZCCHC3 (85364)', 'ZCCHC4 (29063)', 'ZCCHC7 (84186)', 'ZCCHC8 (55596)', 'ZCCHC9 (84240)', 'ZCRB1 (85437)', 'ZCWPW1 (55063)', 'ZCWPW2 (152098)', 'ZDBF2 (57683)', 'ZDHC1 (29800)', 'ZDHC11 (79844)', 'ZDHC11B (653082)', 'ZDHC12 (84885)', 'ZDHC13 (54503)', 'ZDHC14 (79683)', 'ZDHC15 (158866)', 'ZDHC16 (84287)', 'ZDHC17 (23390)', 'ZDHC18 (84243)', 'ZDHC19 (131540)', 'ZDHC2 (51201)', 'ZDHC20 (253832)', 'ZDHC21 (340481)', 'ZDHC22 (283576)', 'ZDHC23 (254887)', 'ZDHC24 (254359)', 'ZDHC3 (51304)', 'ZDHC4 (55146)', 'ZDHC5 (25921)', 'ZDHC6 (64429)', 'ZDHC7 (55625)', 'ZDHC8 (29801)', 'ZDHC9 (51114)', 'ZEB1 (6935)', 'ZEB2 (9839)', 'ZER1 (10444)', 'ZFAND1 (79752)', 'ZFAND2A (90637)', 'ZFAND2B (130617)', 'ZFAND3 (60685)', 'ZFAND4 (93550)', 'ZFAND5 (7763)', 'ZFAND6 (54469)', 'ZFAT (57623)', 'ZFC3H1 (196441)', 'ZFHx2 (85446)', 'ZFHx3 (463)', 'ZFHx4 (79776)', 'ZFP1 (162239)', 'ZFP14 (57677)', 'ZFP2 (80108)', 'ZFP28 (140612)', 'ZFP3 (124961)', 'ZFP30 (22835)', 'ZFP36 (7538)', 'ZFP36L1 (677)', 'ZFP36L2 (678)', 'ZFP37 (7539)', 'ZFP41 (286128)', 'ZFP42 (132625)', 'ZFP57 (346171)', 'ZFP62 (643836)', 'ZFP64 (55734)', 'ZFP69 (339559)', 'ZFP69B (65243)', 'ZFP82 (284406)', 'ZFP90 (146198)', 'ZFP91 (80829)', 'ZFP92 (139735)', 'ZFPL1 (7542)', 'ZFPM1 (161882)', 'ZFPM2 (23414)', 'ZFR (51663)', 'ZFR2 (23217)', 'ZFTA (65998)', 'ZFTRAF1 (50626)', 'ZFX (7543)', 'ZFY (7544)', 'ZFYVE1 (53349)', 'ZFYVE16 (9765)', 'ZFYVE19 (84936)', 'ZFYVE21 (79038)', 'ZFYVE26 (23503)', 'ZFYVE27 (118813)', 'ZFYVE28 (57732)', 'ZFYVE9 (9372)', 'ZG16 (653808)', 'ZG16B (124220)', 'ZGLP1 (100125288)', 'ZGPAT (84619)', 'ZGRF1 (55345)', 'ZHX1 (11244)', 'ZHX2 (22882)', 'ZHX3 (23051)', 'ZIC1 (7545)', 'ZIC2 (7546)', 'ZIC3 (7547)', 'ZIC4 (84107)', 'ZIC5 (85416)', 'ZIK1 (284307)', 'ZIM2 (23619)', 'ZIM3 (114026)', 'ZKSCAN1 (7586)', 'ZKSCAN2 (342357)', 'ZKSCAN3 (80317)', 'ZKSCAN4 (387032)', 'ZKSCAN5 (23660)', 'ZKSCAN7 (55888)', 'ZKSCAN8 (7745)', 'ZMAT1 (84460)', 'ZMAT2 (153527)', 'ZMAT3 (64393)', 'ZMAT4 (79698)', 'ZMAT5 (55954)', 'ZMIZ1 (57178)', 'ZMIZ2 (83637)', 'ZMPSTE24 (10269)', 'ZMYM1 (79830)', 'ZMYM2 (7750)', 'ZMYM3 (9203)', 'ZMYM4 (9202)', 'ZMYM5 (9205)', 'ZMYM6 (9204)', 'ZMYND10 (51364)', 'ZMYND11 (10771)', 'ZMYND12 (84217)', 'ZMYND15 (84225)', 'ZMYND19 (116225)',

'ZMYND8 (23613)', 'ZNF10 (7556)', 'ZNF100 (163227)', 'ZNF101 (94039)', 'ZNF106 (64397)', 'ZNF107 (51427)', 'ZNF112 (7771)', 'ZNF114 (163071)', 'ZNF12 (7559)', 'ZNF121 (7675)', 'ZNF124 (7678)', 'ZNF131 (7690)', 'ZNF132 (7691)', 'ZNF133 (7692)', 'ZNF134 (7693)', 'ZNF135 (7694)', 'ZNF136 (7695)', 'ZNF138 (7697)', 'ZNF14 (7561)', 'ZNF140 (7699)', 'ZNF141 (7700)', 'ZNF142 (7701)', 'ZNF143 (7702)', 'ZNF146 (7705)', 'ZNF148 (7707)', 'ZNF154 (7710)', 'ZNF155 (7711)', 'ZNF157 (7712)', 'ZNF16 (7564)', 'ZNF160 (90338)', 'ZNF165 (7718)', 'ZNF169 (169841)', 'ZNF17 (7565)', 'ZNF174 (7727)', 'ZNF175 (7728)', 'ZNF177 (7730)', 'ZNF18 (7566)', 'ZNF180 (7733)', 'ZNF181 (339318)', 'ZNF182 (7569)', 'ZNF184 (7738)', 'ZNF185 (7739)', 'ZNF189 (7743)', 'ZNF19 (7567)', 'ZNF195 (7748)', 'ZNF2 (7549)', 'ZNF20 (7568)', 'ZNF200 (7752)', 'ZNF202 (7753)', 'ZNF205 (7755)', 'ZNF207 (7756)', 'ZNF208 (7757)', 'ZNF211 (10520)', 'ZNF212 (7988)', 'ZNF213 (7760)', 'ZNF214 (7761)', 'ZNF215 (7762)', 'ZNF217 (7764)', 'ZNF219 (51222)', 'ZNF22 (7570)', 'ZNF221 (7638)', 'ZNF222 (7673)', 'ZNF223 (7766)', 'ZNF224 (7767)', 'ZNF225 (7768)', 'ZNF226 (7769)', 'ZNF227 (7770)', 'ZNF229 (7772)', 'ZNF23 (7571)', 'ZNF230 (7773)', 'ZNF232 (7775)', 'ZNF233 (353355)', 'ZNF234 (10780)', 'ZNF235 (9310)', 'ZNF236 (7776)', 'ZNF239 (8187)', 'ZNF24 (7572)', 'ZNF248 (57209)', 'ZNF25 (219749)', 'ZNF250 (58500)', 'ZNF251 (90987)', 'ZNF253 (56242)', 'ZNF254 (9534)', 'ZNF256 (10172)', 'ZNF257 (113835)', 'ZNF26 (7574)', 'ZNF260 (339324)', 'ZNF263 (10127)', 'ZNF264 (9422)', 'ZNF266 (10781)', 'ZNF267 (10308)', 'ZNF268 (10795)', 'ZNF273 (10793)', 'ZNF274 (10782)', 'ZNF275 (10838)', 'ZNF276 (92822)', 'ZNF277 (11179)', 'ZNF28 (7576)', 'ZNF280A (129025)', 'ZNF280B (140883)', 'ZNF280C (55609)', 'ZNF280D (54816)', 'ZNF281 (23528)', 'ZNF282 (8427)', 'ZNF283 (284349)', 'ZNF284 (342909)', 'ZNF285 (26974)', 'ZNF286A (57335)', 'ZNF287 (57336)', 'ZNF292 (23036)', 'ZNF296 (162979)', 'ZNF3 (7551)', 'ZNF30 (90075)', 'ZNF300 (91975)', 'ZNF302 (55900)', 'ZNF304 (57343)', 'ZNF311 (282890)', 'ZNF316 (100131017)', 'ZNF317 (57693)', 'ZNF318 (24149)', 'ZNF319 (57567)', 'ZNF32 (7580)', 'ZNF320 (162967)', 'ZNF322 (79692)', 'ZNF324 (25799)', 'ZNF324B (388569)', 'ZNF326 (284695)', 'ZNF329 (79673)', 'ZNF330 (27309)', 'ZNF331 (55422)', 'ZNF333 (84449)', 'ZNF334 (55713)', 'ZNF335 (63925)', 'ZNF337 (26152)', 'ZNF33A (7581)', 'ZNF33B (7582)', 'ZNF34 (80778)', 'ZNF341 (84905)', 'ZNF343 (79175)', 'ZNF345 (25850)', 'ZNF346 (23567)', 'ZNF347 (84671)', 'ZNF35 (7584)', 'ZNF350 (59348)', 'ZNF354A (6940)', 'ZNF354B (117608)', 'ZNF354C (30832)', 'ZNF358 (140467)', 'ZNF362 (149076)', 'ZNF365 (22891)', 'ZNF366 (167465)', 'ZNF367 (195828)', 'ZNF37A (7587)', 'ZNF382 (84911)', 'ZNF383 (163087)', 'ZNF384 (171017)', 'ZNF385A (25946)', 'ZNF385B (151126)', 'ZNF385C (201181)', 'ZNF385D (79750)', 'ZNF391 (346157)', 'ZNF394 (84124)', 'ZNF395 (55893)', 'ZNF396 (252884)', 'ZNF397 (84307)', 'ZNF398 (57541)', 'ZNF404 (342908)', 'ZNF407 (55628)', 'ZNF408 (79797)', 'ZNF41 (7592)', 'ZNF410 (57862)', 'ZNF414 (84330)', 'ZNF415 (55786)', 'ZNF416 (55659)', 'ZNF417 (147687)', 'ZNF418 (147686)', 'ZNF419 (79744)', 'ZNF420 (147923)', 'ZNF423 (23090)', 'ZNF425 (155054)', 'ZNF426 (79088)', 'ZNF428 (126299)', 'ZNF429 (353088)', 'ZNF43 (7594)', 'ZNF430 (80264)', 'ZNF431 (170959)', 'ZNF432 (9668)', 'ZNF433 (163059)', 'ZNF436 (80818)', 'ZNF438 (220929)', 'ZNF439 (90594)', 'ZNF44 (51710)', 'ZNF440 (126070)', 'ZNF441 (126068)', 'ZNF442 (79973)', 'ZNF443 (10224)', 'ZNF444 (55311)', 'ZNF445 (353274)', 'ZNF446 (55663)', 'ZNF449 (203523)', 'ZNF45 (7596)', 'ZNF451 (26036)', 'ZNF454 (285676)', 'ZNF460 (10794)', 'ZNF461 (92283)', 'ZNF462 (58499)', 'ZNF467 (168544)', 'ZNF468

(90333)', 'ZNF469 (84627)', 'ZNF470 (388566)', 'ZNF471 (57573)', 'ZNF473
 (25888)', 'ZNF474 (133923)', 'ZNF479 (90827)', 'ZNF48 (197407)', 'ZNF480
 (147657)', 'ZNF483 (158399)', 'ZNF484 (83744)', 'ZNF485 (220992)', 'ZNF486
 (90649)', 'ZNF487 (642819)', 'ZNF488 (118738)', 'ZNF490 (57474)', 'ZNF491
 (126069)', 'ZNF492 (57615)', 'ZNF493 (284443)', 'ZNF496 (84838)', 'ZNF497
 (162968)', 'ZNF500 (26048)', 'ZNF501 (115560)', 'ZNF502 (91392)', 'ZNF503
 (84858)', 'ZNF506 (440515)', 'ZNF507 (22847)', 'ZNF510 (22869)', 'ZNF511
 (118472)', 'ZNF512 (84450)', 'ZNF512B (57473)', 'ZNF513 (130557)', 'ZNF514
 (84874)', 'ZNF516 (9658)', 'ZNF517 (340385)', 'ZNF518A (9849)', 'ZNF518B
 (85460)', 'ZNF519 (162655)', 'ZNF521 (25925)', 'ZNF524 (147807)', 'ZNF525
 (170958)', 'ZNF526 (116115)', 'ZNF527 (84503)', 'ZNF528 (84436)', 'ZNF529
 (57711)', 'ZNF530 (348327)', 'ZNF532 (55205)', 'ZNF534 (147658)', 'ZNF536
 (9745)', 'ZNF540 (163255)', 'ZNF541 (84215)', 'ZNF543 (125919)', 'ZNF544
 (27300)', 'ZNF546 (339327)', 'ZNF547 (284306)', 'ZNF548 (147694)', 'ZNF549
 (256051)', 'ZNF550 (162972)', 'ZNF551 (90233)', 'ZNF552 (79818)', 'ZNF554
 (115196)', 'ZNF555 (148254)', 'ZNF556 (80032)', 'ZNF557 (79230)', 'ZNF558
 (148156)', 'ZNF559 (84527)', 'ZNF560 (147741)', 'ZNF561 (93134)', 'ZNF562
 (54811)', 'ZNF563 (147837)', 'ZNF564 (163050)', 'ZNF565 (147929)', 'ZNF566
 (84924)', 'ZNF567 (163081)', 'ZNF568 (374900)', 'ZNF569 (148266)', 'ZNF57
 (126295)', 'ZNF570 (148268)', 'ZNF571 (51276)', 'ZNF572 (137209)', 'ZNF573
 (126231)', 'ZNF574 (64763)', 'ZNF575 (284346)', 'ZNF576 (79177)', 'ZNF577
 (84765)', 'ZNF578 (147660)', 'ZNF579 (163033)', 'ZNF580 (51157)', 'ZNF581
 (51545)', 'ZNF582 (147948)', 'ZNF583 (147949)', 'ZNF584 (201514)', 'ZNF585A
 (199704)', 'ZNF585B (92285)', 'ZNF586 (54807)', 'ZNF587 (84914)', 'ZNF587B
 (100293516)', 'ZNF589 (51385)', 'ZNF592 (9640)', 'ZNF593 (51042)', 'ZNF594
 (84622)', 'ZNF595 (152687)', 'ZNF596 (169270)', 'ZNF597 (146434)', 'ZNF599
 (148103)', 'ZNF600 (162966)', 'ZNF605 (100289635)', 'ZNF606 (80095)', 'ZNF607
 (84775)', 'ZNF608 (57507)', 'ZNF609 (23060)', 'ZNF610 (162963)', 'ZNF611
 (81856)', 'ZNF613 (79898)', 'ZNF614 (80110)', 'ZNF615 (284370)', 'ZNF616
 (90317)', 'ZNF618 (114991)', 'ZNF619 (285267)', 'ZNF620 (253639)', 'ZNF621
 (285268)', 'ZNF622 (90441)', 'ZNF623 (9831)', 'ZNF624 (57547)', 'ZNF625
 (90589)', 'ZNF626 (199777)', 'ZNF627 (199692)', 'ZNF628 (89887)', 'ZNF629
 (23361)', 'ZNF630 (57232)', 'ZNF638 (27332)', 'ZNF639 (51193)', 'ZNF641
 (121274)', 'ZNF644 (84146)', 'ZNF646 (9726)', 'ZNF648 (127665)', 'ZNF649
 (65251)', 'ZNF652 (22834)', 'ZNF653 (115950)', 'ZNF654 (55279)', 'ZNF655
 (79027)', 'ZNF658 (26149)', 'ZNF66 (7617)', 'ZNF660 (285349)', 'ZNF662
 (389114)', 'ZNF664 (144348)', 'ZNF665 (79788)', 'ZNF667 (63934)', 'ZNF668
 (79759)', 'ZNF669 (79862)', 'ZNF670 (93474)', 'ZNF671 (79891)', 'ZNF672
 (79894)', 'ZNF674 (641339)', 'ZNF675 (171392)', 'ZNF676 (163223)', 'ZNF677
 (342926)', 'ZNF678 (339500)', 'ZNF679 (168417)', 'ZNF680 (340252)', 'ZNF681
 (148213)', 'ZNF682 (91120)', 'ZNF683 (257101)', 'ZNF684 (127396)', 'ZNF687
 (57592)', 'ZNF688 (146542)', 'ZNF689 (115509)', 'ZNF69 (7620)', 'ZNF691
 (51058)', 'ZNF692 (55657)', 'ZNF695 (57116)', 'ZNF696 (79943)', 'ZNF697
 (90874)', 'ZNF699 (374879)', 'ZNF7 (7553)', 'ZNF70 (7621)', 'ZNF700 (90592)',
 'ZNF701 (55762)', 'ZNF703 (80139)', 'ZNF704 (619279)', 'ZNF705B (100132396)',
 'ZNF705G (100131980)', 'ZNF706 (51123)', 'ZNF707 (286075)', 'ZNF708 (7562)',
 'ZNF709 (163051)', 'ZNF71 (58491)', 'ZNF710 (374655)', 'ZNF711 (7552)', 'ZNF713
 (349075)', 'ZNF714 (148206)', 'ZNF716 (441234)', 'ZNF717 (100131827)', 'ZNF718

(255403)', 'ZNF721 (170960)', 'ZNF723 (646864)', 'ZNF724 (440519)', 'ZNF726 (730087)', 'ZNF727 (442319)', 'ZNF728 (388523)', 'ZNF729 (100287226)', 'ZNF730 (100129543)', 'ZNF732 (654254)', 'ZNF735 (730291)', 'ZNF736 (728927)', 'ZNF737 (100129842)', 'ZNF738 (148203)', 'ZNF74 (7625)', 'ZNF740 (283337)', 'ZNF746 (155061)', 'ZNF747 (65988)', 'ZNF749 (388567)', 'ZNF750 (79755)', 'ZNF75A (7627)', 'ZNF75D (7626)', 'ZNF76 (7629)', 'ZNF763 (284390)', 'ZNF764 (92595)', 'ZNF765 (91661)', 'ZNF766 (90321)', 'ZNF768 (79724)', 'ZNF77 (58492)', 'ZNF770 (54989)', 'ZNF771 (51333)', 'ZNF772 (400720)', 'ZNF773 (374928)', 'ZNF774 (342132)', 'ZNF775 (285971)', 'ZNF776 (284309)', 'ZNF777 (27153)', 'ZNF778 (197320)', 'ZNF780A (284323)', 'ZNF780B (163131)', 'ZNF782 (158431)', 'ZNF783 (100289678)', 'ZNF784 (147808)', 'ZNF785 (146540)', 'ZNF786 (136051)', 'ZNF787 (126208)', 'ZNF789 (285989)', 'ZNF79 (7633)', 'ZNF790 (388536)', 'ZNF791 (163049)', 'ZNF792 (126375)', 'ZNF793 (390927)', 'ZNF799 (90576)', 'ZNF8 (7554)', 'ZNF80 (7634)', 'ZNF800 (168850)', 'ZNF804A (91752)', 'ZNF804B (219578)', 'ZNF805 (390980)', 'ZNF808 (388558)', 'ZNF81 (347344)', 'ZNF813 (126017)', 'ZNF814 (730051)', 'ZNF816 (125893)', 'ZNF821 (55565)', 'ZNF823 (55552)', 'ZNF827 (152485)', 'ZNF829 (374899)', 'ZNF83 (55769)', 'ZNF830 (91603)', 'ZNF831 (128611)', 'ZNF835 (90485)', 'ZNF836 (162962)', 'ZNF837 (116412)', 'ZNF839 (55778)', 'ZNF84 (7637)', 'ZNF841 (284371)', 'ZNF843 (283933)', 'ZNF844 (284391)', 'ZNF845 (91664)', 'ZNF846 (162993)', 'ZNF85 (7639)', 'ZNF850 (342892)', 'ZNF853 (54753)', 'ZNF860 (344787)', 'ZNF862 (643641)', 'ZNF865 (100507290)', 'ZNF875 (284459)', 'ZNF878 (729747)', 'ZNF879 (345462)', 'ZNF880 (400713)', 'ZNF888 (388559)', 'ZNF891 (101060200)', 'ZNF90 (7643)', 'ZNF91 (7644)', 'ZNF92 (168374)', 'ZNF93 (81931)', 'ZNF98 (148198)', 'ZNF99 (7652)', 'ZNFX1 (57169)', 'ZNG1A (55871)', 'ZNG1B (150472)', 'ZNHIT1 (10467)', 'ZNHIT2 (741)', 'ZNHIT3 (9326)', 'ZNHIT6 (54680)', 'ZNRD2 (10534)', 'ZNRFB1 (84937)', 'ZNRFB2 (223082)', 'ZNRFB3 (84133)', 'ZNRFB4 (148066)', 'ZP1 (22917)', 'ZP2 (7783)', 'ZP3 (7784)', 'ZP4 (57829)', 'ZPBP (11055)', 'ZPBP2 (124626)', 'ZPLD1 (131368)', 'ZPR1 (8882)', 'ZRBANB1 (54764)', 'ZRBANB2 (9406)', 'ZRBANB3 (84083)', 'ZRSR2 (8233)', 'ZSCAN1 (284312)', 'ZSCAN10 (84891)', 'ZSCAN12 (9753)', 'ZSCAN16 (80345)', 'ZSCAN18 (65982)', 'ZSCAN2 (54993)', 'ZSCAN20 (7579)', 'ZSCAN21 (7589)', 'ZSCAN22 (342945)', 'ZSCAN23 (222696)', 'ZSCAN25 (221785)', 'ZSCAN26 (7741)', 'ZSCAN29 (146050)', 'ZSCAN30 (100101467)', 'ZSCAN31 (64288)', 'ZSCAN32 (54925)', 'ZSCAN4 (201516)', 'ZSCAN5A (79149)', 'ZSCAN5B (342933)', 'ZSCAN5C (649137)', 'ZSCAN9 (7746)', 'ZSWIM1 (90204)', 'ZSWIM2 (151112)', 'ZSWIM3 (140831)', 'ZSWIM4 (65249)', 'ZSWIM5 (57643)', 'ZSWIM6 (57688)', 'ZSWIM7 (125150)', 'ZSWIM8 (23053)', 'ZSWIM9 (374920)', 'ZUP1 (221302)', 'ZW10 (9183)', 'ZWILCH (55055)', 'ZWINT (11130)', 'ZXDA (7789)', 'ZXDB (158586)', 'ZXDC (79364)', 'ZYG11A (440590)', 'ZYG11B (79699)', 'ZYYX (7791)', 'ZZEF1 (23140)', 'ZZZ3 (26009)', 'AKR1C1 (1645)', 'AMY2B (280)', 'ANKRD20A1 (84210)', 'CCDC141 (285025)', 'CCL4L2 (9560)', 'CDK11A (728642)', 'CDK11B (984)', 'CT45A1 (541466)', 'CT45A10 (102723631)', 'DEFB131B (100129216)', 'DEFB4A (1673)', 'DEFB4B (100289462)', 'EPPK1 (83481)', 'FABP5 (2171)', 'FAM72B (653820)', 'FOXDL1 (200350)', 'GAGE10 (102724473)', 'GAGE12J (729396)', 'GAGE2A (729447)', 'GOLGA6A (342096)', 'GOLGA6L4 (643707)', 'GOLGA6L9 (440295)', 'GTF2IRD2 (84163)', 'GTF2IRD2B (389524)', 'HNRNPCL1 (343069)', 'HNRNPCL2 (440563)', 'KCNJ18 (100134444)', 'KRTAP4-12 (83755)', 'KRTAP4-7 (100132476)', 'LCE1C (353133)', 'LCE1E (353135)', 'LRRC69 (100130742)', 'MAGEA3

```
(4102)', 'NBPF11 (200030)', 'NBPF12 (149013)', 'NBPF14 (25832)', 'NBPF20
(100288142)', 'NBPF8 (728841)', 'NBPF9 (400818)', 'NOM3 (408050)', 'NPIP5
(100132247)', 'NPY4R (5540)', 'OR11H12 (440153)', 'OR2T3 (343173)', 'OR2T33
(391195)', 'OR4F17 (81099)', 'OR4F5 (79501)', 'OR5K2 (402135)', 'PLA2G10
(8399)', 'POLR2J2 (246721)', 'PPIAL4A (653505)', 'PRAMEF13 (400736)', 'PRAMEF27
(101929983)', 'PRH1 (5554)', 'PSG7 (5676)', 'RAB6D (150786)', 'RFPL4A (342931)',
'RFPL4AL1 (729974)', 'RGPD3 (653489)', 'SAA1 (6288)', 'SPDYE5 (442590)', 'SPRR2D
(6703)', 'SRGAP2C (653464)', 'SYT15 (83849)', 'TBC1D3E (102723859)', 'TIMM23B
(100652748)', 'TMEM191B (728229)', 'TMEM191C (645426)', 'TPTE (7179)', 'TRIM43B
(653192)', 'TVP23B (51030)', 'UQCRH (7388)', 'USP17L1 (401447)', 'USP17L2
(377630)', 'WASHC1 (100287171)', 'ZNF475 (100505841)', 'ZNF705A (440077)',
'ZNG1F (644019)', 'C12orf76 (400073)', 'C4A (720)', 'DDTL (100037417)', 'ERVV-1
(147664)', 'ERVV-2 (100271846)', 'FAM86B1 (85002)', 'FAM90A26 (100287045)',
'FKBP1C (642489)', 'OR3A2 (4995)', 'OR3A3 (8392)', 'POLR2J (5439)', 'PRAMEF10
(343071)', 'PRR33 (102724536)', 'RGPD2 (729857)', 'SMIM10L3 (122526779)', 'TPRX2
(503627)', 'TTL13 (440307)', 'VCX2 (51480)']
```

```
[481]: crispr_primary_targets = pd.DataFrame(depmap_crispr[['HSP90AA1 (3320)',
↳ 'HSP90AB1 (3326)', 'XP01 (7514)']].copy())

recurrence_model_ids = (depmap_glioma_recurrence_scores["ModelID"].
↳ drop_duplicates().values)

crispr_primary_targets_recurrence_score = crispr_primary_targets.reindex(
    recurrence_model_ids
).copy()
```

```
[482]: score_series = depmap_glioma_recurrence_scores.
↳ set_index("ModelID")["DEPMAP_RECURRENCE_SCORE"]
score_series = score_series[~score_series.index.duplicated(keep="first")]

qc_data = []

for gene in crispr_primary_targets_recurrence_score.columns:
    paired_df = pd.DataFrame({
        "GENE_EFFECT": crispr_primary_targets_recurrence_score[gene],
        "RECURRENCE_SCORE": score_series,
    }).dropna()
    gene_vals = paired_df["GENE_EFFECT"]
    qc_data.append({
        "GENE": gene,
        "N": len(paired_df),
        "MEAN_GENE_EFFECT": gene_vals.mean(),
        "SD_GENE_EFFECT": gene_vals.std(),
        "MEDIAN_GENE_EFFECT": gene_vals.median(),
        "MIN": gene_vals.min(),
        "MAX": gene_vals.max(),
    })
```



```

})

crispr_primary_targets_QC = pd.DataFrame(qc_data)

common_essentials_df = pd.read_csv(
    "../../data/raw/DepMap/CRISPRInferredCommonEssentials.csv"
)
common_essentials = set(common_essentials_df["Essentials"].astype(str).str.
    ↪split(" ").str[0])
gene_symbols = (crispr_primary_targets_QC["GENE"].astype(str).str.split(" ").
    ↪str[0])
crispr_primary_targets_QC["COMMON_ESSENTIAL_STATUS"] = gene_symbols.
    ↪isin(common_essentials)

```

```
[483]: crispr_primary_targets_QC
```

```
[483]:
```

	GENE	N	MEAN_GENE_EFFECT	SD_GENE_EFFECT	MEDIAN_GENE_EFFECT	\
0	HSP90AA1 (3320)	64	-0.194675	0.130775	-0.184037	
1	HSP90AB1 (3326)	64	-0.215668	0.193684	-0.200586	
2	XP01 (7514)	64	-2.175066	0.379663	-2.158213	

	MIN	MAX	COMMON_ESSENTIAL_STATUS
0	-0.588749	0.120093	False
1	-0.820494	0.090316	False
2	-3.029175	-1.212176	True

```
[484]: crispr_primary_targets_QC.to_excel(
    "../../results/CRISPR/crispr_primary_targets_QC.xlsx",
    index=False
)
```

- crispr_primary_targets_QC.xlsx exported to /results/CRISPR directory

```
[485]: print(
    depmap_glioma_recurrence_scores["ModelID"]
    .value_counts()
    .loc[lambda x: x > 1]
)
```

```
Series([], Name: count, dtype: int64)
```

8.2 2. Relationship of Recurrence Score and genetic dependency

```
[486]: crispr_targets_clean = crispr_primary_targets_recurrence_score[
    ~crispr_primary_targets_recurrence_score.index.duplicated(keep="first")]

combined_data = pd.concat(
    [

```

```

        score_series,
        crispr_targets_clean[
            ["HSP90AA1 (3320)", "HSP90AB1 (3326)", "XP01 (7514)"]
        ],
    ],
    axis=1,
)

sample_sizes = []
rhos = []
p_vals = []

for gene in crispr_primary_targets_QC["GENE"]:
    if gene in combined_data.columns:
        pair = combined_data[["DEPMAP_RECURRENCE_SCORE", gene]].dropna()
        n_pair = len(pair)

        if n_pair >= 3:
            rho, p = spearmanr(pair["DEPMAP_RECURRENCE_SCORE"], pair[gene])
            sample_sizes.append(n_pair)
            rhos.append(rho)
            p_vals.append(p)
        else:
            sample_sizes.append(n_pair)
            rhos.append(np.nan)
            p_vals.append(np.nan)
    else:
        sample_sizes.append(0)
        rhos.append(np.nan)
        p_vals.append(np.nan)

recurrence_score_crispr_primary_targets = pd.DataFrame({
    "GENE": crispr_primary_targets_QC["GENE"],
    "N": sample_sizes,
    "SPEARMAN_RHO": rhos,
    "P_VALUE": p_vals,
})

valid_mask = recurrence_score_crispr_primary_targets["P_VALUE"].notna()
recurrence_score_crispr_primary_targets.loc[valid_mask, "FDR"] = multipletests(
    recurrence_score_crispr_primary_targets.loc[valid_mask, "P_VALUE"],
    method="fdr_bh",
)[1]

```

[487]: recurrence_score_crispr_primary_targets

```
[487]:
```

	GENE	N	SPEARMAN_RHO	P_VALUE	FDR
0	HSP90AA1 (3320)	64	0.050962	0.689218	0.689218
1	HSP90AB1 (3326)	64	0.152747	0.228211	0.342317
2	XP01 (7514)	64	0.160989	0.203787	0.342317

```
[488]: recurrence_score_crispr_primary_targets.to_excel(
    "../../results/CRISPR/recurrence_score_crispr_primary_targets.xlsx",
    index=False
)
```

- recurrence_score_crispr_primary_targets.xlsx exported to /results/CRISPR directory

8.3 3. Relevance of CRISP results

```
[489]: targets = {
    "HSP90AA1 (3320)": "../../figures/crispr_HSP90AA1.png",
    "HSP90AB1 (3326)": "../../figures/crispr_HSP90AB1.png",
    "XP01 (7514)": "../../figures/crispr_XP01.png",
}

for gene, filename in targets.items():
    if gene not in combined_data.columns:
        print(f"Warning: {gene} not found in combined_data")
        continue

    gene_symbol = gene.split()[0]
    qc_row = crispr_primary_targets_QC[
        crispr_primary_targets_QC["GENE"].str.startswith(gene_symbol)
    ]

    if not qc_row.empty:
        is_common_essential = qc_row["COMMON_ESSENTIAL_STATUS"].values[0]
    else:
        is_common_essential = "Unknown"

    df_plot = combined_data[["DEPMAP_RECURRENCE_SCORE", gene]].dropna().copy()

    # --- Outlier Detection (Tukey's 1.5 * IQR Rule) ---
    for col in [gene, "DEPMAP_RECURRENCE_SCORE"]:
        q1 = df_plot[col].quantile(0.25)
        q3 = df_plot[col].quantile(0.75)
        iqr = q3 - q1
        df_plot[f"{col}_outlier"] = (df_plot[col] < (q1 - 1.5 * iqr)) | (
            df_plot[col] > (q3 + 1.5 * iqr)
        )

    df_plot["IS_OUTLIER"] = (
```

```

        df_plot[f"{gene}_outlier"] |
df_plot["DEPMAP_RECURRENCE_SCORE_outlier"]
    )

    x = df_plot[gene]
    y = df_plot["DEPMAP_RECURRENCE_SCORE"]
    n_samples = len(df_plot)
    n_outliers = df_plot["IS_OUTLIER"].sum()

    gene_var = x.var()
    rho, p_val = spearmanr(x, y)

    plt.figure(figsize=(7, 6))

    ax = sns.regplot(
        data=df_plot[~df_plot["IS_OUTLIER"]],
        x=gene,
        y="DEPMAP_RECURRENCE_SCORE",
        color="#2b5c8f",
        scatter_kws={"alpha": 0.7, "s": 50, "label": "Inliers"},
        line_kws={"color": "#e74c3c", "linewidth": 2},
    )

    outliers_df = df_plot[df_plot["IS_OUTLIER"]]
    if not outliers_df.empty:
        plt.scatter(
            outliers_df[gene],
            outliers_df["DEPMAP_RECURRENCE_SCORE"],
            color="#e74c3c",
            s=90,
            edgecolor="black",
            linewidth=1.2,
            zorder=5,
            label=f"Outliers (n={n_outliers})",
        )

    for model_id, row in outliers_df.iterrows():
        plt.annotate(
            str(model_id),
            (row[gene], row["DEPMAP_RECURRENCE_SCORE"]),
            textcoords="offset points",
            xytext=(5, 5),
            ha="left",
            fontsize=8,
            color="#8b0000",
            fontweight="bold",
        )

```

```

plt.axvline(
    x=-1.0,
    color="black",
    linestyle="--",
    alpha=0.5,
    label="Essentiality Threshold (-1.0)",
)

plt.title(
    f"CRISPR QC & Dependency vs Recurrence: {gene.split()[0]}",
    fontsize=13,
    fontweight="bold",
    y=1.03,
)

plt.xlabel(f"{gene} Gene Effect (Chronos Score)", fontsize=11)
plt.ylabel("DepMap Glioma Recurrence Score", fontsize=11)
plt.legend(loc="lower left")

rho_val = float(rho)
p_val_scalar = float(p_val)

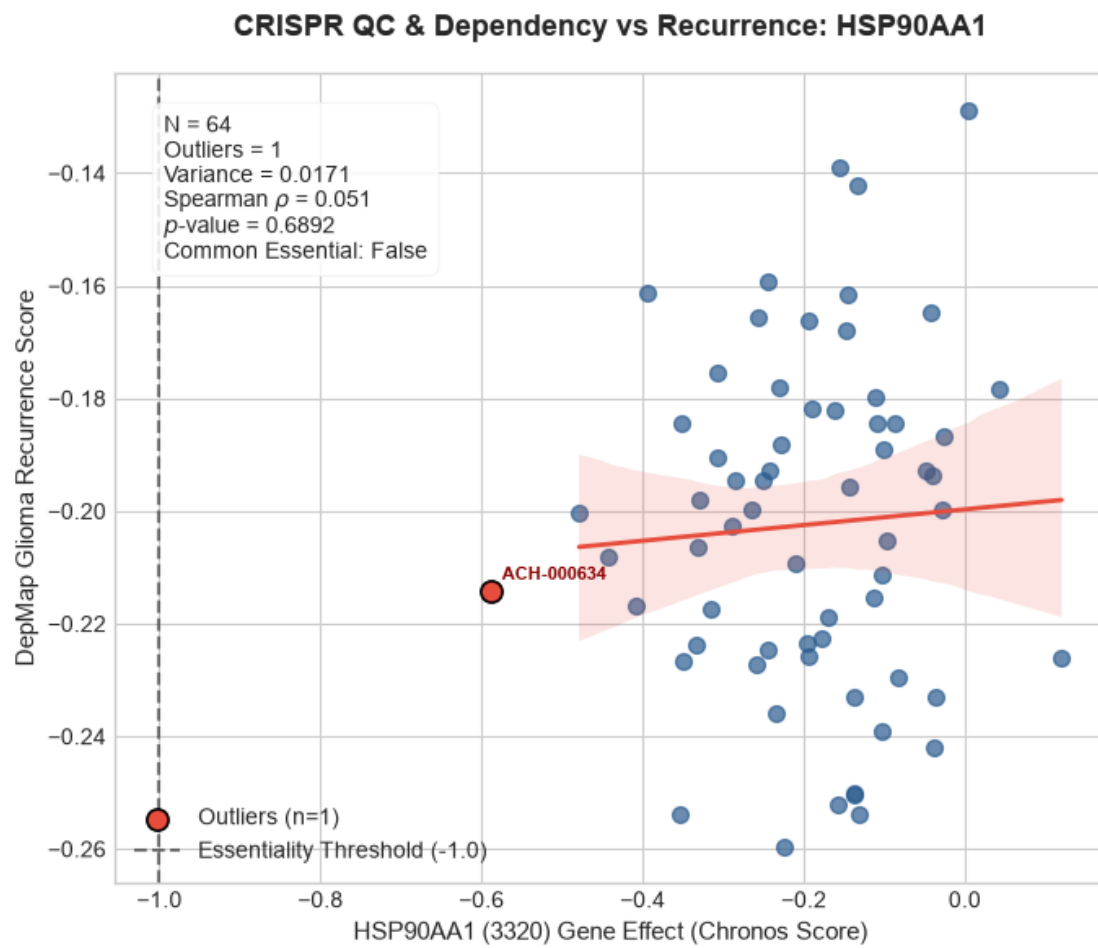
stats_text = (
    f"N = {n_samples}\n"
    f"Outliers = {n_outliers}\n"
    f"Variance = {gene_var:.4f}\n"
    f"Spearman  $\rho$  = {rho_val:.3f}\n"
    f"$p$-value = {p_val_scalar:.4f}\n"
    f"Common Essential: {is_common_essential}"
)

plt.gca().text(
    0.05,
    0.95,
    stats_text,
    transform=plt.gca().transAxes,
    fontsize=10,
    verticalalignment="top",
    bbox=dict(
        boxstyle="round,pad=0.5",
        facecolor="#ffe6e6" if is_common_essential is True else "white",
        alpha=0.85,
    ),
)

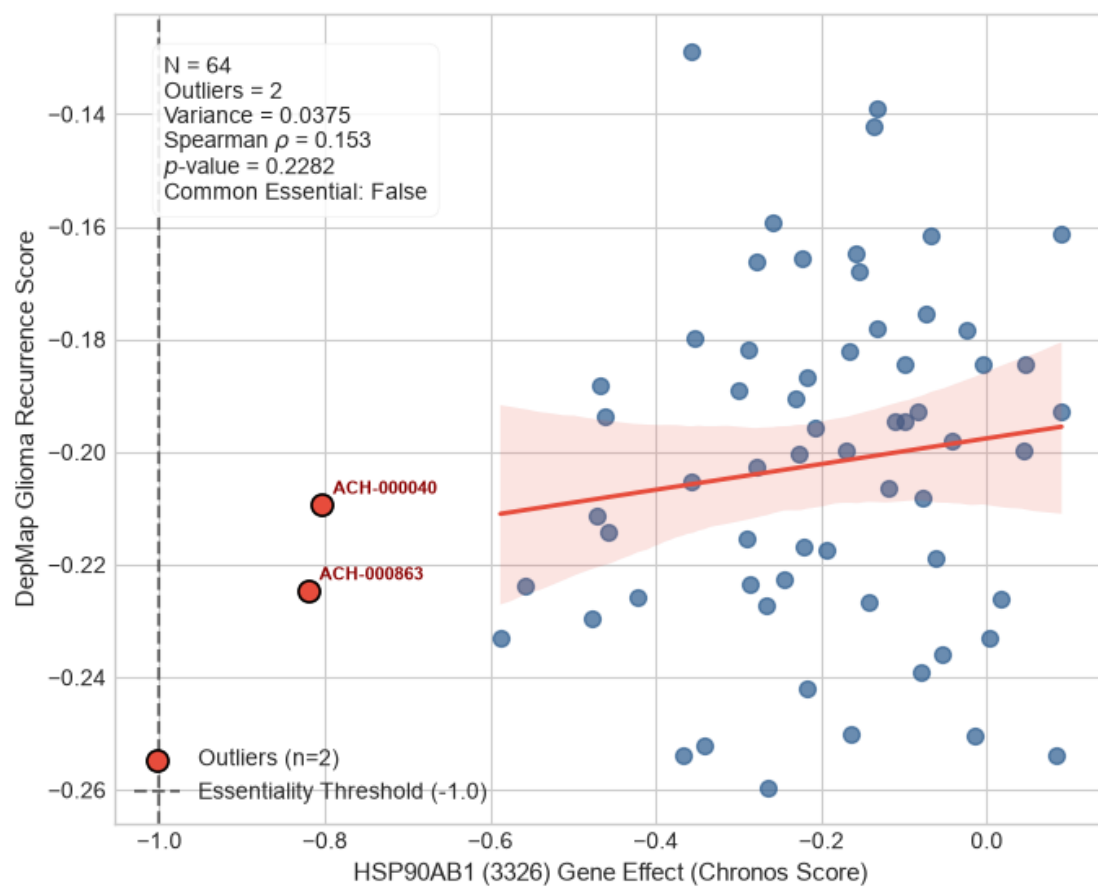
plt.tight_layout()

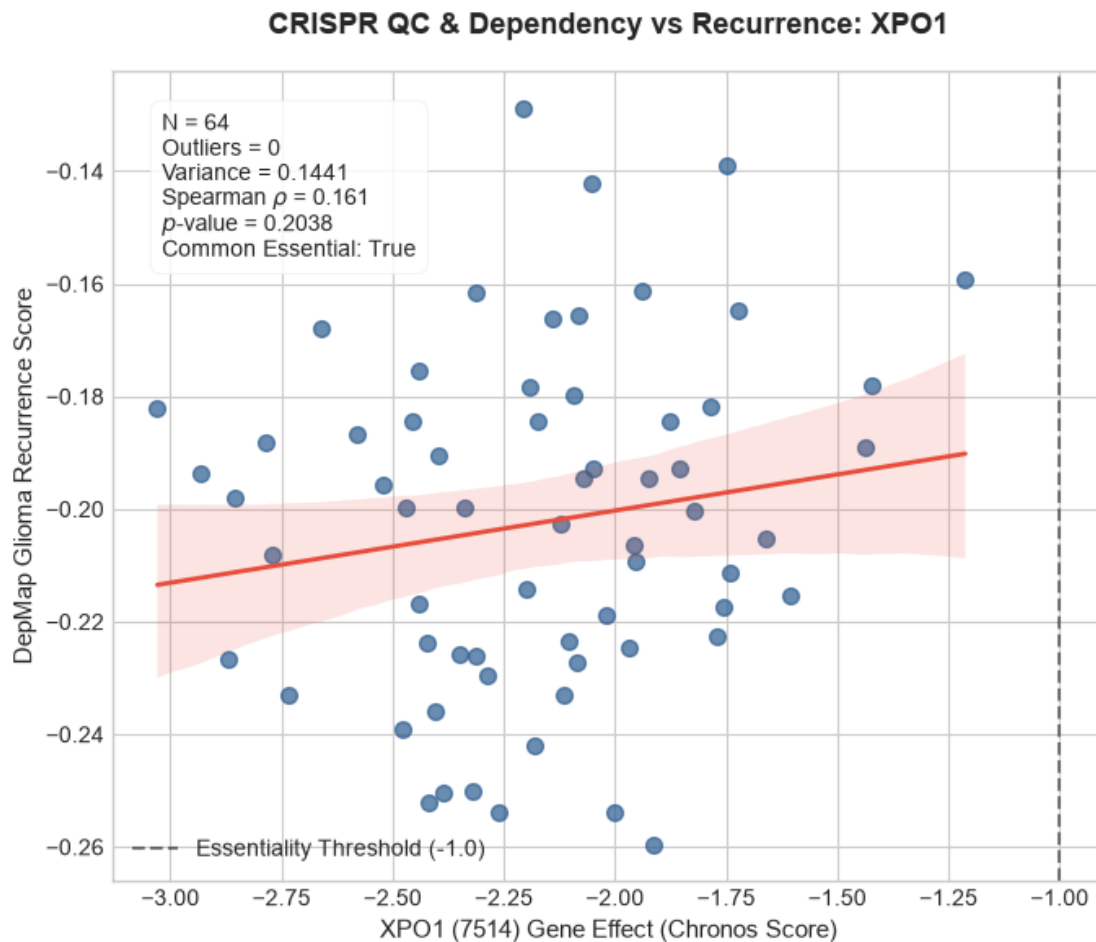
```

```
plt.savefig(filename, dpi=300)
plt.show()
```



CRISPR QC & Dependency vs Recurrence: HSP90AB1





- crispr_HSP90AA1.png, crispr_HSP90AB1.png and crispr_XPO1.png, exported to /figures directory

8.4 4. Proliferation corrected CRISPR sensitivity analysis

```
[490]: depmap_proliferation_score_clean = depmap_proliferation_score.groupby(level=0).
      ↪mean()
score_series = depmap_glioma_recurrence_scores.
      ↪set_index("ModelID")["DEPMAP_RECURRENCE_SCORE"]
score_series = score_series[~score_series.index.duplicated(keep="first")]
```

```
[491]: crispr_multivariate_results = []

for gene in crispr_primary_targets_recurrence_score.columns:
    reg_df = pd.DataFrame({
        "GENE_EFFECT": crispr_primary_targets_recurrence_score[gene],
        "RECURRENCE_SCORE": score_series,
```



```

        "PROLIFERATION_SCORE": depmap_proliferation_score_clean,
    }).dropna()

    if len(reg_df) < 10:
        continue

    X = reg_df[["RECURRENCE_SCORE", "PROLIFERATION_SCORE"]]
    X = sm.add_constant(X)
    y = reg_df["GENE_EFFECT"]

    model = sm.OLS(y, X).fit()

    crispr_multivariate_results.append({
        "GENE": gene,
        "N": len(reg_df),
        "BETA_RECURRENCE_SCORE": model.params["RECURRENCE_SCORE"],
        "P_RECURRENCE_SCORE": model.pvalues["RECURRENCE_SCORE"],
        "BETA_PROLIFERATION": model.params["PROLIFERATION_SCORE"],
        "P_PROLIFERATION": model.pvalues["PROLIFERATION_SCORE"],
        "ADJUSTED_R2": model.rsquared_adj,
    })

crispr_primary_targets_adjusted_proliferation = pd.
↳DataFrame(crispr_multivariate_results)

crispr_primary_targets_adjusted_proliferation =
↳crispr_primary_targets_adjusted_proliferation.sort_values(
    by="P_RECURRENCE_SCORE"
).reset_index(drop=True)

crispr_primary_targets_adjusted_proliferation

```

```

[491]:
      GENE      N  BETA_RECURRENCE_SCORE  P_RECURRENCE_SCORE  \
0  XP01 (7514)  64                2.352565                0.166874
1  HSP90AB1 (3326)  64                0.875435                0.313119
2  HSP90AA1 (3320)  64                0.452109                0.441945

      BETA_PROLIFERATION  P_PROLIFERATION  ADJUSTED_R2
0              0.049195              0.577354   -0.000430
1             -0.016398              0.716475   -0.007084
2              0.023726              0.439671   -0.017520

```

```

[492]: crispr_primary_targets_adjusted_proliferation.to_excel(
        ".../results/CRISPR/crispr_multivariate_results.xlsx",
        index=False
    )

```

- crispr_multivariate_results.xlsx exported to /figures directory

Summary Note: In Adult-Type Diffuse Glioma cell models ($N = 64$), the recurrence score signature does not correlate with increased genetic vulnerability to single CRISPR depletion of HSP90AA1, HSP90AB1, or XPO1. Target dependency for these three genes operates independently of the recurrence score. While individual knockouts fail to impair survival—likely due to paralog compensation between HSP90AA1 and HSP90AB1—further dual-knockout studies are necessary to evaluate whether simultaneous inactivation of both isoforms yields synthetic lethality. Nevertheless, these findings provide a biological rationale for why dual-isoform targeted therapies like alvespimycin may exert baseline antitumour activity across diffuse gliomas regardless of recurrence risk score.

9 GSE186332 - Clinical analysis of selinexor

- Is the transcriptomic program present in the primary tumour —indicative of future recurrence— associated with the clinical efficacy of selinexor at relapse?
- Reference: Lassman, A. B., Wen, P. Y., van den Bent, M. J., Plotkin, S. R., Walenkamp, A. M. E., Green, A. L., Li, K., Walker, C. J., Chang, H., Tamir, S., Henegar, L., Shen, Y., Alvarez, M. J., Califano, A., Landesman, Y., Kauffman, M. G., Shacham, S., & Mau-Sørensen, M. (2022). A Phase II Study of the Efficacy and Safety of Oral Selinexor in Recurrent Glioblastoma. *Clinical cancer research : an official journal of the American Association for Cancer Research*, 28(3), 452–460. <https://doi.org/10.1158/1078-0432.CCR-21-2225>

9.1 1. GSE186332 reconstruction of clinical metadata

- Source: <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE186332>
- Submission date: Oct 21, 2021
- Last update: Feb 04, 2022
- Organisation name: Karyopharm Therapeutics, Inc
- Platforms: GPL24676 Illumina NovaSeq 6000 (Homo sapiens)

```
[493]: gse = GEOparse.get_GEO("GSE186332", destdir="../../data/raw/GSE186332")
p = gse.phenotype_data

meta = pd.DataFrame({"PATIENT_ID": p["title"]}, index=p.index)
meta.index.name = "SAMPLE_ID"

meta["TREATMENT_ARM"] = p["characteristics_ch1.1.trial arm"].str.replace("trial_
↪arm: ", "")
meta["EXPERIMENTAL_SET"] = p["characteristics_ch1.2.experimental set"].str.
↪replace("experimental set: ", "")
meta["GEO_RESPONSE"] = p["characteristics_ch1.3.patient response"].str.
↪replace("patient response: ", "")

def parse_outcomes(r):
    es, resp = r["EXPERIMENTAL_SET"], r["GEO_RESPONSE"]
    if es == "Excluded Outlier": return ("Excluded",)*3 + (np.nan,)
    if es == "Discovery":
```

```

        return ("Clinical Benefit", "CR/PR", "CR/PR", 1) if resp == "Responder"
    else ("Resistance", "PD", "PD", 0)
    if es == "Validation":
        return ("Clinical Benefit", "durable SD", "durable SD (>140d / PFS6+)",
1) if resp == "Responder" else ("Resistance", "non-durable SD / PD",
    "non-durable SD (<100d) / PD", 0)
        return ("Unknown",)*3 + (np.nan,)

meta[["BENEFIT_STATUS", "BEST_OVERALL_RESPONSE", "PFS", "PFS6_BENEFIT_BINARY"]]
    = list(meta.apply(parse_outcomes, axis=1))
meta["RESPONSE_GROUP"] = meta["BEST_OVERALL_RESPONSE"]
meta["IDH1_STATUS"] = "Unknown (Paper: n=9 Mutated, n=43 WT)"
meta["RNA_AVAILABLE"] = True

cols = ["PATIENT_ID", "TREATMENT_ARM", "BEST_OVERALL_RESPONSE", "PFS",
    "BENEFIT_STATUS", "RESPONSE_GROUP", "EXPERIMENTAL_SET", "IDH1_STATUS",
    "RNA_AVAILABLE"]
gse186332_selinexor_clinical_metadata = meta[cols]

```

05-Oct-2026 17:22:41 DEBUG utils - Directory ../../data/raw/GSE186332 already exists. Skipping.

DEBUG:GEOparse:Directory ../../data/raw/GSE186332 already exists. Skipping.

05-Oct-2026 17:22:41 INFO GEOparse - File already exist: using local version.

INFO:GEOparse:File already exist: using local version.

05-Oct-2026 17:22:41 INFO GEOparse - Parsing

../../data/raw/GSE186332/GSE186332_family.soft.gz:

INFO:GEOparse:Parsing ../../data/raw/GSE186332/GSE186332_family.soft.gz:

05-Oct-2026 17:22:41 DEBUG GEOparse - DATABASE: GeoMiam

DEBUG:GEOparse:DATABASE: GeoMiam

05-Oct-2026 17:22:41 DEBUG GEOparse - SERIES: GSE186332

DEBUG:GEOparse:SERIES: GSE186332

05-Oct-2026 17:22:41 DEBUG GEOparse - PLATFORM: GPL24676

DEBUG:GEOparse:PLATFORM: GPL24676

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643839

DEBUG:GEOparse:SAMPLE: GSM5643839

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643840

DEBUG:GEOparse:SAMPLE: GSM5643840

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643841

DEBUG:GEOparse:SAMPLE: GSM5643841

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643842

DEBUG:GEOparse:SAMPLE: GSM5643842

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643843

DEBUG:GEOparse:SAMPLE: GSM5643843

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643844

DEBUG:GEOparse:SAMPLE: GSM5643844

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643845

DEBUG:GEOparse:SAMPLE: GSM5643845

05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643846
DEBUG:GEOParse:SAMPLE: GSM5643846
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643847
DEBUG:GEOParse:SAMPLE: GSM5643847
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643848
DEBUG:GEOParse:SAMPLE: GSM5643848
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643849
DEBUG:GEOParse:SAMPLE: GSM5643849
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643850
DEBUG:GEOParse:SAMPLE: GSM5643850
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643851
DEBUG:GEOParse:SAMPLE: GSM5643851
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643852
DEBUG:GEOParse:SAMPLE: GSM5643852
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643853
DEBUG:GEOParse:SAMPLE: GSM5643853
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643854
DEBUG:GEOParse:SAMPLE: GSM5643854
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643855
DEBUG:GEOParse:SAMPLE: GSM5643855
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643856
DEBUG:GEOParse:SAMPLE: GSM5643856
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643857
DEBUG:GEOParse:SAMPLE: GSM5643857
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643858
DEBUG:GEOParse:SAMPLE: GSM5643858
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643859
DEBUG:GEOParse:SAMPLE: GSM5643859
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643860
DEBUG:GEOParse:SAMPLE: GSM5643860
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643861
DEBUG:GEOParse:SAMPLE: GSM5643861
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643862
DEBUG:GEOParse:SAMPLE: GSM5643862
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643863
DEBUG:GEOParse:SAMPLE: GSM5643863
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643864
DEBUG:GEOParse:SAMPLE: GSM5643864
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643865
DEBUG:GEOParse:SAMPLE: GSM5643865
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643866
DEBUG:GEOParse:SAMPLE: GSM5643866
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643867
DEBUG:GEOParse:SAMPLE: GSM5643867
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643868
DEBUG:GEOParse:SAMPLE: GSM5643868
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643869
DEBUG:GEOParse:SAMPLE: GSM5643869

05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643870
DEBUG:GEOParse:SAMPLE: GSM5643870
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643871
DEBUG:GEOParse:SAMPLE: GSM5643871
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643872
DEBUG:GEOParse:SAMPLE: GSM5643872
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643873
DEBUG:GEOParse:SAMPLE: GSM5643873
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643874
DEBUG:GEOParse:SAMPLE: GSM5643874
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643875
DEBUG:GEOParse:SAMPLE: GSM5643875
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643876
DEBUG:GEOParse:SAMPLE: GSM5643876
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643877
DEBUG:GEOParse:SAMPLE: GSM5643877
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643878
DEBUG:GEOParse:SAMPLE: GSM5643878
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643879
DEBUG:GEOParse:SAMPLE: GSM5643879
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643880
DEBUG:GEOParse:SAMPLE: GSM5643880
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643881
DEBUG:GEOParse:SAMPLE: GSM5643881
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643882
DEBUG:GEOParse:SAMPLE: GSM5643882
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643883
DEBUG:GEOParse:SAMPLE: GSM5643883
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643884
DEBUG:GEOParse:SAMPLE: GSM5643884
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643885
DEBUG:GEOParse:SAMPLE: GSM5643885
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643886
DEBUG:GEOParse:SAMPLE: GSM5643886
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643887
DEBUG:GEOParse:SAMPLE: GSM5643887
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643888
DEBUG:GEOParse:SAMPLE: GSM5643888
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643889
DEBUG:GEOParse:SAMPLE: GSM5643889
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643890
DEBUG:GEOParse:SAMPLE: GSM5643890
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643891
DEBUG:GEOParse:SAMPLE: GSM5643891
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643892
DEBUG:GEOParse:SAMPLE: GSM5643892
05-Oct-2026 17:22:41 DEBUG GEOParse - SAMPLE: GSM5643893
DEBUG:GEOParse:SAMPLE: GSM5643893

05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643894
 DEBUG:GEOparse:SAMPLE: GSM5643894
 05-Oct-2026 17:22:41 DEBUG GEOparse - SAMPLE: GSM5643895
 DEBUG:GEOparse:SAMPLE: GSM5643895

[494]: gse186332_selinexor_clinical_metadata

[494]:

	PATIENT_ID	TREATMENT_ARM	BEST_OVERALL_RESPONSE	\
SAMPLE_ID				
GSM5643839	001-009	A		PD
GSM5643840	001-015	A		CR/PR
GSM5643841	001-037	C		PD
GSM5643842	001-038	D		PD
GSM5643843	001-043	D		PD
GSM5643844	001-046	D	non-durable	SD / PD
GSM5643845	001-047	D		CR/PR
GSM5643846	001-050	A	durable	SD
GSM5643847	001-055	D	non-durable	SD / PD
GSM5643848	001-064	D		CR/PR
GSM5643849	001-071	D	durable	SD
GSM5643850	001-076	D		PD
GSM5643851	001-078	D	non-durable	SD / PD
GSM5643852	002-014	B		PD
GSM5643853	002-018	B	durable	SD
GSM5643854	002-021	B		PD
GSM5643855	002-026	B	non-durable	SD / PD
GSM5643856	002-030	B		PD
GSM5643857	002-042	D	non-durable	SD / PD
GSM5643858	002-064	D		PD
GSM5643859	002-066	D	non-durable	SD / PD
GSM5643860	002-067	D	durable	SD
GSM5643861	002-069	D	durable	SD
GSM5643862	002-070	D		PD
GSM5643863	002-074	D		PD
GSM5643864	002-075	D	non-durable	SD / PD
GSM5643865	003-016	B	non-durable	SD / PD
GSM5643866	003-019	B	non-durable	SD / PD
GSM5643867	003-023	B	durable	SD
GSM5643868	003-024	B	durable	SD
GSM5643869	003-029	B		PD
GSM5643870	003-031	B		PD
GSM5643871	003-036	B		PD
GSM5643872	301-001	B	non-durable	SD / PD
GSM5643873	301-002	A		Excluded
GSM5643874	301-003	B		PD
GSM5643875	301-004	B		PD
GSM5643876	301-005	B		CR/PR

GSM5643877	301-013	B	CR/PR
GSM5643878	301-022	B	PD
GSM5643879	301-025	B	non-durable SD / PD
GSM5643880	301-028	D	Excluded
GSM5643881	301-033	D	Excluded
GSM5643882	301-039	C	CR/PR
GSM5643883	301-044	C	PD
GSM5643884	301-045	C	PD
GSM5643885	501-048	C	Excluded
GSM5643886	501-049	C	durable SD
GSM5643887	501-053	D	PD
GSM5643888	501-054	C	durable SD
GSM5643889	501-056	D	Excluded
GSM5643890	501-058	C	PD
GSM5643891	501-059	D	durable SD
GSM5643892	501-061	C	PD
GSM5643893	502-052	D	durable SD
GSM5643894	502-062	D	CR/PR
GSM5643895	502-063	C	PD

SAMPLE_ID	PFS	BENEFIT_STATUS \
GSM5643839	PD	Resistance
GSM5643840	CR/PR	Clinical Benefit
GSM5643841	PD	Resistance
GSM5643842	PD	Resistance
GSM5643843	PD	Resistance
GSM5643844	non-durable SD (<100d) / PD	Resistance
GSM5643845	CR/PR	Clinical Benefit
GSM5643846	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643847	non-durable SD (<100d) / PD	Resistance
GSM5643848	CR/PR	Clinical Benefit
GSM5643849	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643850	PD	Resistance
GSM5643851	non-durable SD (<100d) / PD	Resistance
GSM5643852	PD	Resistance
GSM5643853	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643854	PD	Resistance
GSM5643855	non-durable SD (<100d) / PD	Resistance
GSM5643856	PD	Resistance
GSM5643857	non-durable SD (<100d) / PD	Resistance
GSM5643858	PD	Resistance
GSM5643859	non-durable SD (<100d) / PD	Resistance
GSM5643860	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643861	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643862	PD	Resistance
GSM5643863	PD	Resistance

GSM5643864	non-durable SD (<100d) / PD	Resistance
GSM5643865	non-durable SD (<100d) / PD	Resistance
GSM5643866	non-durable SD (<100d) / PD	Resistance
GSM5643867	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643868	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643869	PD	Resistance
GSM5643870	PD	Resistance
GSM5643871	PD	Resistance
GSM5643872	non-durable SD (<100d) / PD	Resistance
GSM5643873	Excluded	Excluded
GSM5643874	PD	Resistance
GSM5643875	PD	Resistance
GSM5643876	CR/PR	Clinical Benefit
GSM5643877	CR/PR	Clinical Benefit
GSM5643878	PD	Resistance
GSM5643879	non-durable SD (<100d) / PD	Resistance
GSM5643880	Excluded	Excluded
GSM5643881	Excluded	Excluded
GSM5643882	CR/PR	Clinical Benefit
GSM5643883	PD	Resistance
GSM5643884	PD	Resistance
GSM5643885	Excluded	Excluded
GSM5643886	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643887	PD	Resistance
GSM5643888	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643889	Excluded	Excluded
GSM5643890	PD	Resistance
GSM5643891	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643892	PD	Resistance
GSM5643893	durable SD (>140d / PFS6+)	Clinical Benefit
GSM5643894	CR/PR	Clinical Benefit
GSM5643895	PD	Resistance

SAMPLE_ID	RESPONSE_GROUP	EXPERIMENTAL_SET \
GSM5643839	PD	Discovery
GSM5643840	CR/PR	Discovery
GSM5643841	PD	Discovery
GSM5643842	PD	Discovery
GSM5643843	PD	Discovery
GSM5643844	non-durable SD / PD	Validation
GSM5643845	CR/PR	Discovery
GSM5643846	durable SD	Validation
GSM5643847	non-durable SD / PD	Validation
GSM5643848	CR/PR	Discovery
GSM5643849	durable SD	Validation
GSM5643850	PD	Discovery

GSM5643851	non-durable SD / PD	Validation
GSM5643852	PD	Discovery
GSM5643853	durable SD	Validation
GSM5643854	PD	Discovery
GSM5643855	non-durable SD / PD	Validation
GSM5643856	PD	Discovery
GSM5643857	non-durable SD / PD	Validation
GSM5643858	PD	Discovery
GSM5643859	non-durable SD / PD	Validation
GSM5643860	durable SD	Validation
GSM5643861	durable SD	Validation
GSM5643862	PD	Discovery
GSM5643863	PD	Discovery
GSM5643864	non-durable SD / PD	Validation
GSM5643865	non-durable SD / PD	Validation
GSM5643866	non-durable SD / PD	Validation
GSM5643867	durable SD	Validation
GSM5643868	durable SD	Validation
GSM5643869	PD	Discovery
GSM5643870	PD	Discovery
GSM5643871	PD	Discovery
GSM5643872	non-durable SD / PD	Validation
GSM5643873	Excluded	Excluded Outlier
GSM5643874	PD	Discovery
GSM5643875	PD	Discovery
GSM5643876	CR/PR	Discovery
GSM5643877	CR/PR	Discovery
GSM5643878	PD	Discovery
GSM5643879	non-durable SD / PD	Validation
GSM5643880	Excluded	Excluded Outlier
GSM5643881	Excluded	Excluded Outlier
GSM5643882	CR/PR	Discovery
GSM5643883	PD	Discovery
GSM5643884	PD	Discovery
GSM5643885	Excluded	Excluded Outlier
GSM5643886	durable SD	Validation
GSM5643887	PD	Discovery
GSM5643888	durable SD	Validation
GSM5643889	Excluded	Excluded Outlier
GSM5643890	PD	Discovery
GSM5643891	durable SD	Validation
GSM5643892	PD	Discovery
GSM5643893	durable SD	Validation
GSM5643894	CR/PR	Discovery
GSM5643895	PD	Discovery

IDH1_STATUS RNA_AVAILABLE

GSM5643885	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643886	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643887	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643888	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643889	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643890	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643891	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643892	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643893	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643894	Unknown (Paper: n=9 Mutated, n=43 WT)	True
GSM5643895	Unknown (Paper: n=9 Mutated, n=43 WT)	True

```
[495]: gse186332_selinexor_clinical_metadata.to_excel(
        ".../results/GSE186332/gse186332_selinexor_clinical_metadata.xlsx",
        index=True)

print("\nDistribution of samples bases on BENEFIT_STATUS:")
print(gse186332_selinexor_clinical_metadata["BENEFIT_STATUS"].value_counts())

print("\nMDistribution of samples bases on BEST_OVERALL_RESPONSE:")
print(gse186332_selinexor_clinical_metadata["BEST_OVERALL_RESPONSE"].
      ↪value_counts())
```

Distribution of samples bases on BENEFIT_STATUS:

BENEFIT_STATUS

Resistance 34

Clinical Benefit 18

Excluded 5

Name: count, dtype: int64

MDistribution of samples bases on BEST_OVERALL_RESPONSE:

BEST_OVERALL_RESPONSE

PD 23

non-durable SD / PD 11

durable SD 11

CR/PR 7

Excluded 5

Name: count, dtype: int64

- gse186332_selinexor_clinical_metadata.xlsx exported to /results/GSE186332 directory

9.2 2. Preparing GSE186332 RNA-seq expression matrix

- Gene Identifier Type: Entrez Gene ID (Numeric IDs, e.g., '100287102', '79501')
- Genome Build: GRCh38 / hg38
- Gene Annotation: GENCODE v38

- Gene Length Definition: Non-overlapping exonic length in base pairs (sum of merged, non-overlapping exon intervals across all transcripts per gene).
- Source:
 - <https://ftp.ncbi.nlm.nih.gov/gene/DATA/gene2ensembl.gz>
 - https://ftp.ebi.ac.uk/pub/databases/gencode/Gencode_human/release_38/gencode.v38.annotation.gtf.gz

```
[496]: gse_raw_counts = pd.read_csv(
        "../../../data/raw/GSE186332/GSE186332_gbm_rawcounts.tsv",
        sep="\t",
        index_col=0,
        low_memory=False
    )

    print(gse_raw_counts.shape)
    gse_raw_counts.head()
```

(25132, 57)

```
[496]:
```

	001-009	001-015	001-037	001-038	001-043	001-046	001-047	\
NCBIGeneID								
100287102	2	0	1	1	2	4	4	
102466751	5	0	0	0	5	11	9	
100302278	0	0	0	1	3	2	4	
645520	0	0	0	0	1	0	0	
79501	0	0	0	0	0	0	0	

	001-050	001-055	001-064	...	501-049	501-053	501-054	\
NCBIGeneID				...				
100287102	1	4	3	...	4	3	0	
102466751	5	6	13	...	16	10	1	
100302278	1	3	1	...	7	1	0	
645520	0	0	0	...	0	1	0	
79501	0	0	0	...	2	0	0	

	501-056	501-058	501-059	501-061	502-052	502-062	502-063
NCBIGeneID							
100287102	1	11	1	4	4	9	3
102466751	9	11	0	1	4	12	7
100302278	7	4	0	0	1	3	2
645520	0	0	0	1	0	0	2
79501	1	1	0	0	0	0	0

[5 rows x 57 columns]

```
[497]: gene2ensembl_path = "../../../data/raw/annotation/gene2ensembl."

    with open(gene2ensembl_path, "rt") as f:
        for i in range(10):
```

```

        print(repr(next(f)))

gene2ensembl = pd.read_csv(
    gene2ensembl_path,
    sep="\t",
    dtype=str
)

print("\nColumns:")
print(gene2ensembl.columns.tolist())

```

```

'#tax_id\tGeneID\tEnsembl_gene_identifie\tRNA_nucleotide_accession.version\tEns
embl_rna_identifie\tprotein_accession.version\tEnsembl_protein_identifie\n'
'3483\t27215448\tgene-A5N79_gp34\t-\t-\tYP_009243652.1\tYP_009243652.1\n'
'3483\t27215449\tgene-A5N79_gp30\t-\t-\tYP_009243656.1\tYP_009243656.1\n'
'3483\t27215450\tgene-A5N79_gt07\t-\t-\t-\t-\n'
'3483\t27215451\tgene-A5N79_gp15\t-\t-\tYP_009243668.1\tYP_009243668.1\n'
'3483\t27215452\tgene-A5N79_gp22\t-\t-\tYP_009243663.1\tYP_009243663.1\n'
'3483\t27215453\tgene-A5N79_gp32\t-\t-\tYP_009243654.1\tYP_009243654.1\n'
'3483\t27215454\tgene-A5N79_gp29\t-\t-\tYP_009243657.1\tYP_009243657.1\n'
'3483\t27215455\tgene-A5N79_gp12\t-\t-\tYP_009243671.1\tYP_009243671.1\n'
'3483\t27215456\tgene-A5N79_gp05\t-\t-\tYP_009243678.1\tYP_009243678.1\n'

```

Columns:

```

['#tax_id', 'GeneID', 'Ensembl_gene_identifie',
'RNA_nucleotide_accession.version', 'Ensembl_rna_identifie',
'protein_accession.version', 'Ensembl_protein_identifie']

```

[498]: gene2ensembl.head()

```

[498]:   #tax_id   GeneID Ensembl_gene_identifie RNA_nucleotide_accession.version \
0      3483  27215448          gene-A5N79_gp34                        -
1      3483  27215449          gene-A5N79_gp30                        -
2      3483  27215450          gene-A5N79_gt07                        -
3      3483  27215451          gene-A5N79_gp15                        -
4      3483  27215452          gene-A5N79_gp22                        -

      Ensembl_rna_identifie protein_accession.version Ensembl_protein_identifie
0                        -                YP_009243652.1          YP_009243652.1
1                        -                YP_009243656.1          YP_009243656.1
2                        -                        -                        -
3                        -                YP_009243668.1          YP_009243668.1
4                        -                YP_009243663.1          YP_009243663.1

```

```

[499]: gene2ensembl_human = gene2ensembl[gene2ensembl["#tax_id"] == "9606"].copy()
print("\nHuman rows:")
print(gene2ensembl_human.shape)

```

Human rows:
(96204, 7)

```
[500]: mapping = gene2ensembl_human[["GeneID", "Ensembl_gene_identifier"]].copy()
mapping["GeneID"] = mapping["GeneID"].astype(str)
```

```
[501]: gse_raw_counts.index = gse_raw_counts.index.astype(str)

count_gene_ids = pd.Index(
    gse_raw_counts.index.unique(),
    name="NCBIGeneID"
)

print("\nRaw gse_raw_counts:")
print("Number of genes:", len(count_gene_ids))
```

Raw gse_raw_counts:
Number of genes: 25132

```
[502]: mapping_in_counts = mapping[mapping["GeneID"].isin(count_gene_ids)].copy()

print("\nMapping:")
print("Mapped NCBI IDs:", mapping_in_counts["GeneID"].nunique())

mapping_unique = (
    mapping_in_counts[["GeneID", "Ensembl_gene_identifier"]]
    .drop_duplicates()
    .copy()
)
```

Mapping:
Mapped NCBI IDs: 24468

```
[503]: n_total = gse_raw_counts.index.nunique()
n_mapped = mapping_unique["GeneID"].nunique()
coverage = 100 * n_mapped / n_total

print(f"Total NCBI Gene IDs: {n_total}")
print(f"Mapped NCBI Gene IDs: {n_mapped}")
print(f"Coverage: {coverage:.2f}%")
```

Total NCBI Gene IDs: 25132
Mapped NCBI Gene IDs: 24468
Coverage: 97.36%

```
[504]: multi_mapping = (
        mapping_in_counts
        .groupby("GeneID")["Ensembl_gene_identifier"]
        .nunique()
    )

    n_multi = (multi_mapping > 1).sum()
    print(f"NCBI IDs with >1 Ensembl gene: {n_multi}")
```

NCBI IDs with >1 Ensembl gene: 22

```
[505]: mapped_ids = set(mapping_in_counts["GeneID"])
    unmapped_ids = count_gene_ids[~count_gene_ids.isin(mapped_ids)]

    print(f"Unmapped NCBI IDs: {len(unmapped_ids)}")
    print("\nFirst unmapped IDs:")
    unmapped_ids[:20].tolist()
```

Unmapped NCBI IDs: 664

First unmapped IDs:

```
[505]: ['102723897',
        '102465432',
        '79854',
        '9906',
        '100996583',
        '100616489',
        '102724450',
        '102724539',
        '100422970',
        '105376736',
        '100133301',
        '729574',
        '284729',
        '102723383',
        '101928017',
        '100616177',
        '101928728',
        '101929464',
        '101929626',
        '105378721']
```

```
[506]: multi_mapping_unique = mapping_unique.
        ↪groupby("GeneID")["Ensembl_gene_identifier"].nunique()
    multi_mapping_unique = multi_mapping_unique[multi_mapping_unique > 1]
```

```

multi_mapping_table = mapping_unique[mapping_unique["GeneID"].
↳isin(multi_mapping_unique.index)].sort_values(["GeneID",
↳"Ensembl_gene_identifier"])

print(f"NCBI IDs with >1 unique Ensembl gene:
↳{len(multi_mapping_unique)}\n\n{multi_mapping_table.to_string(index=False)}")

```

NCBI IDs with >1 unique Ensembl gene: 22

GeneID	Ensembl_gene_identifier
100359394	ENSG00000230542
100359394	ENSG00000292347
100500894	ENSG00000265658
100500894	ENSG00000292355
102464837	ENSG00000277120
102464837	ENSG00000292346
1438	ENSG00000198223
1438	ENSG00000292357
207063	ENSG00000169084
207063	ENSG00000292338
28227	ENSG00000167393
28227	ENSG00000292327
286530	ENSG00000182162
286530	ENSG00000292333
293	ENSG00000169100
293	ENSG00000292334
3563	ENSG00000185291
3563	ENSG00000292332
3581	ENSG00000124334
3581	ENSG00000292373
401577	ENSG00000291040
401577	ENSG00000292362
4267	ENSG00000002586
4267	ENSG00000292348
438	ENSG00000196433
438	ENSG00000292336
55344	ENSG00000182378
55344	ENSG00000292344
64109	ENSG00000205755
64109	ENSG00000292363
6473	ENSG00000185960
6473	ENSG00000292354
6845	ENSG00000124333
6845	ENSG00000292366
751580	ENSG00000236871
751580	ENSG00000292341
8225	ENSG00000178605
8225	ENSG00000292358

8227	ENSG000000197976
8227	ENSG000000292343
8623	ENSG000000169093
8623	ENSG000000292339
9189	ENSG000000214717
9189	ENSG000000292345

```
[507]: ambiguous_ensembl = set(multi_mapping_table["Ensembl_gene_identifier"])
gencode_ids = set()
with gzip.open("../data/raw/annotation/gencode.v38.annotation.gtf.gz", "rt") as f:
    for line in f:
        if line.startswith("#"): continue
        fields = line.rstrip("\n").split("\t")
        if len(fields) != 9: continue
        attributes = fields[8]
        gene_id = None
        for item in attributes.split(";"):
            item = item.strip()
            if item.startswith("gene_id "):
                gene_id = item.split("'")[1].split(".")[0]
                break
        if gene_id is not None: gencode_ids.add(gene_id)

multi_mapping_check = multi_mapping_table.copy()
multi_mapping_check["in_gencode"] =
    multi_mapping_check["Ensembl_gene_identifier"].isin(gencode_ids)
print(multi_mapping_check.to_string(index=False))
print("\nSummary:")
print(multi_mapping_check.groupby("GeneID")["in_gencode"].agg(["sum", "count"]))
```

GeneID	Ensembl_gene_identifier	in_gencode
100359394	ENSG000000230542	True
100359394	ENSG000000292347	False
100500894	ENSG000000265658	True
100500894	ENSG000000292355	False
102464837	ENSG000000277120	True
102464837	ENSG000000292346	False
1438	ENSG000000198223	True
1438	ENSG000000292357	False
207063	ENSG000000169084	True
207063	ENSG000000292338	False
28227	ENSG000000167393	True
28227	ENSG000000292327	False
286530	ENSG000000182162	True
286530	ENSG000000292333	False
293	ENSG000000169100	True
293	ENSG000000292334	False

3563	ENSG000000185291	True
3563	ENSG000000292332	False
3581	ENSG000000124334	True
3581	ENSG000000292373	False
401577	ENSG000000291040	False
401577	ENSG000000292362	False
4267	ENSG000000002586	True
4267	ENSG000000292348	False
438	ENSG000000196433	True
438	ENSG000000292336	False
55344	ENSG000000182378	True
55344	ENSG000000292344	False
64109	ENSG000000205755	True
64109	ENSG000000292363	False
6473	ENSG000000185960	True
6473	ENSG000000292354	False
6845	ENSG000000124333	True
6845	ENSG000000292366	False
751580	ENSG000000236871	True
751580	ENSG000000292341	False
8225	ENSG000000178605	True
8225	ENSG000000292358	False
8227	ENSG000000197976	True
8227	ENSG000000292343	False
8623	ENSG000000169093	True
8623	ENSG000000292339	False
9189	ENSG000000214717	True
9189	ENSG000000292345	False

Summary:

GeneID	sum	count
100359394	1	2
100500894	1	2
102464837	1	2
1438	1	2
207063	1	2
28227	1	2
286530	1	2
293	1	2
3563	1	2
3581	1	2
401577	0	2
4267	1	2
438	1	2
55344	1	2
64109	1	2
6473	1	2

6845	1	2
751580	1	2
8225	1	2
8227	1	2
8623	1	2
9189	1	2

```
[508]: mapping_gencode = mapping_unique[mapping_unique["Ensembl_gene_identifier"].
↳isin(gencode_ids)].copy()
mapping_gencode_counts = mapping_gencode.
↳groupby("GeneID")["Ensembl_gene_identifier"].nunique()

print("NCBI IDs with exactly 1 GENCODE Ensembl match:", (mapping_gencode_counts_
↳== 1).sum())
print("NCBI IDs with >1 GENCODE Ensembl match:", (mapping_gencode_counts > 1).
↳sum())
print("NCBI IDs with 0 GENCODE Ensembl matches:",
↳len(set(mapping_unique["GeneID"]) - set(mapping_gencode_counts.index)))
```

NCBI IDs with exactly 1 GENCODE Ensembl match: 24052
 NCBI IDs with >1 GENCODE Ensembl match: 0
 NCBI IDs with 0 GENCODE Ensembl matches: 416

```
[509]: raw_gene_ids = pd.Index(gse_raw_counts.index.astype(str))
unique_gencode_mapping = mapping_gencode.
↳groupby("GeneID")["Ensembl_gene_identifier"].agg(list)
unambiguous_mapping = unique_gencode_mapping[unique_gencode_mapping.str.len()_
↳== 1].copy().apply(lambda x: x[0])
mapped_ids = raw_gene_ids.intersection(unambiguous_mapping.index)

print("Raw genes:", len(raw_gene_ids))
print("Mapped to exactly one GENCODE gene:", len(mapped_ids))
print("Unmapped/ambiguous:", len(raw_gene_ids) - len(mapped_ids))
print("Coverage:", round(100 * len(mapped_ids) / len(raw_gene_ids), 2), "%")
```

Raw genes: 25132
 Mapped to exactly one GENCODE gene: 24052
 Unmapped/ambiguous: 1080
 Coverage: 95.7 %

```
[510]: exons_by_gene = defaultdict(list)
with gzip.open("../data/raw/annotation/gencode.v38.annotation.gtf.gz", "rt")_
↳as f:
    for line in f:
        if line.startswith("#"): continue
        fields = line.rstrip("\n").split("\t")
        if len(fields) != 9: continue
```

```

    chrom, source, feature, start, end, score, strand, frame, attributes = fields
    if feature != "exon": continue
    gene_id = None
    for item in attributes.split(";"):
        item = item.strip()
        if item.startswith("gene_id "):
            gene_id = item.split(' ')[1].split(".")[0]
            break
    if gene_id is None: continue
    start0 = int(start) - 1
    end0 = int(end)
    exons_by_gene[gene_id].append((chrom, start0, end0))

```

```

[511]: def merged_exon_length(intervals):
    """
    Calculate the number of non-overlapping bases
    covered by exon intervals.
    """
    if not intervals: return 0
    intervals = sorted(intervals)
    total = 0
    current_chrom, current_start, current_end = intervals[0]
    for chrom, start, end in intervals[1:]:
        if chrom != current_chrom:
            total += current_end - current_start
            current_chrom, current_start, current_end = chrom, start, end
            continue
        if start <= current_end: current_end = max(current_end, end)
        else:
            total += current_end - current_start
            current_start, current_end = start, end
    total += current_end - current_start
    return total

gene_lengths = pd.Series({gene_id: merged_exon_length(intervals) for gene_id, intervals in exons_by_gene.items()}, name="gene_length_bp")
print("Genes with exon-derived lengths:", len(gene_lengths))
gene_lengths.head()

```

Genes with exon-derived lengths: 60605

```

[511]: ENSG00000223972    1735
      ENSG00000227232    1351
      ENSG00000278267     68
      ENSG00000243485    1021
      ENSG00000284332    138

```

Name: gene_length_bp, dtype: int64

```
[512]: mapping_length_check = pd.DataFrame({"GeneID": unambiguous_mapping.index.  
      ↳ astype(str), "Ensembl_gene_id": unambiguous_mapping.values})  
mapping_length_check["gene_length_bp"] =  
      ↳ mapping_length_check["Ensembl_gene_id"].map(gene_lengths)  
  
print("Mapped genes:", len(mapping_length_check))  
print("Missing gene lengths:", mapping_length_check["gene_length_bp"].isna().  
      ↳ sum())  
print("Zero-length genes:", (mapping_length_check["gene_length_bp"] == 0).sum())
```

Mapped genes: 24052

Missing gene lengths: 0

Zero-length genes: 0

```
[513]: mapping_table = pd.DataFrame({"GeneID": unambiguous_mapping.index.astype(str),  
      ↳ "Ensembl_gene_id": unambiguous_mapping.values})  
mapping_table["gene_length_bp"] = mapping_table["Ensembl_gene_id"].  
      ↳ map(gene_lengths)  
mapping_table["gene_length_kb"] = mapping_table["gene_length_bp"] / 1000  
mapping_table = mapping_table.set_index("GeneID")  
mapping_table.head()
```

```
[513]:
```

	Ensembl_gene_id	gene_length_bp	gene_length_kb
GeneID			
1	ENSG00000121410	3999	3.999
10	ENSG00000156006	1285	1.285
100	ENSG00000196839	2809	2.809
1000	ENSG00000170558	8583	8.583
10000	ENSG00000117020	16643	16.643

```
[514]: mapping_for_counts = mapping_table.reset_index()[["GeneID", "Ensembl_gene_id",  
      ↳ "gene_length_bp", "gene_length_kb"]].copy()  
mapping_for_counts["GeneID"] = mapping_for_counts["GeneID"].astype(str)  
gse_raw_counts.index = gse_raw_counts.index.astype(str)  
  
counts_mapped = gse_raw_counts.loc[gse_raw_counts.index.  
      ↳ intersection(mapping_for_counts["GeneID"])]  
mapping_for_counts = mapping_for_counts.set_index("GeneID").loc[counts_mapped.  
      ↳ index]  
assert mapping_for_counts.index.equals(counts_mapped.index)
```

```
[515]: counts_ensembl = counts_mapped.copy()  
counts_ensembl.index = mapping_for_counts["Ensembl_gene_id"].values  
counts_ensembl.index.name = "Ensembl_gene_id"  
counts_ensembl = counts_ensembl.groupby(level="Ensembl_gene_id").sum()
```

```
print("Raw count matrix after Ensembl aggregation:")
print(counts_ensembl.shape)
```

Raw count matrix after Ensembl aggregation:
(23978, 57)

```
[516]: gene_metadata = mapping_for_counts[["Ensembl_gene_id", "gene_length_bp", "gene_length_kb"]].drop_duplicates(subset=["Ensembl_gene_id"]).set_index("Ensembl_gene_id")
gene_metadata = gene_metadata.reindex(counts_ensembl.index)
assert gene_metadata.index.equals(counts_ensembl.index)
assert gene_metadata.index.is_unique
assert gene_metadata["gene_length_bp"].notna().all()
assert (gene_metadata["gene_length_bp"] > 0).all()

print(gene_metadata.shape)
print(gene_metadata["gene_length_bp"].isna().sum())
print((gene_metadata["gene_length_bp"] <= 0).sum())
print()
print("Aggregated counts:", counts_ensembl.shape)
print("Gene metadata:", gene_metadata.shape)
print("Unique metadata genes:", gene_metadata.index.nunique())
```

(23978, 2)

0

0

Aggregated counts: (23978, 57)
Gene metadata: (23978, 2)
Unique metadata genes: 23978

$$RPK_i = \frac{\text{counts_mapped}_i}{\text{gene_length_kb}_i}$$

```
[517]: gene_length_kb = gene_metadata["gene_length_kb"]
rpk = counts_ensembl.div(gene_length_kb, axis=0)
print("RPK matrix:", rpk.shape)
```

RPK matrix: (23978, 57)

$$TPM_i = \frac{RPK_i}{\sum_j RPK_j} \times 10^6$$

```
[518]: tpm = rpk.div(rpk.sum(axis=0), axis=1) * 1e6
print("TPM matrix:", tpm.shape)
```

TPM matrix: (23978, 57)

```
[519]: tpm_sums = tpm.sum(axis=0)
      tpm_sums
```

```
[519]: 001-009      1000000.0
      001-015      1000000.0
      001-037      1000000.0
      001-038      1000000.0
      001-043      1000000.0
      001-046      1000000.0
      001-047      1000000.0
      001-050      1000000.0
      001-055      1000000.0
      001-064      1000000.0
      001-071      1000000.0
      001-076      1000000.0
      001-078      1000000.0
      002-014      1000000.0
      002-018      1000000.0
      002-021      1000000.0
      002-026      1000000.0
      002-030      1000000.0
      002-042      1000000.0
      002-064      1000000.0
      002-066      1000000.0
      002-067      1000000.0
      002-069      1000000.0
      002-070      1000000.0
      002-074      1000000.0
      002-075      1000000.0
      003-016      1000000.0
      003-019      1000000.0
      003-023      1000000.0
      003-024      1000000.0
      003-029      1000000.0
      003-031      1000000.0
      003-036      1000000.0
      301-001      1000000.0
      301-002      1000000.0
      301-003      1000000.0
      301-004      1000000.0
      301-005      1000000.0
      301-013      1000000.0
      301-022      1000000.0
      301-025      1000000.0
      301-028      1000000.0
      301-033      1000000.0
      301-039      1000000.0
```

```

301-044    1000000.0
301-045    1000000.0
501-048    1000000.0
501-049    1000000.0
501-053    1000000.0
501-054    1000000.0
501-056    1000000.0
501-058    1000000.0
501-059    1000000.0
501-061    1000000.0
502-052    1000000.0
502-062    1000000.0
502-063    1000000.0
dtype: float64

```

```
[520]: assert ((tpm_sums - 1e6).abs() < 1e-6).all()
```

$$\log_2_tpm_i = \log_2(TPM + 1)$$

```
[521]: gse_log2_tpm = np.log2(tpm + 1)
print("Final matrix:", gse_log2_tpm.shape)
gse_log2_tpm.iloc[:5, :5]
```

Final matrix: (23978, 57)

```
[521]:
```

	001-009	001-015	001-037	001-038	001-043
Ensembl_gene_id					
ENSG000000000003	2.900600	3.569328	3.922517	5.466130	5.116568
ENSG000000000005	2.957185	0.000000	0.000000	3.509839	2.671874
ENSG000000000419	2.253435	3.015734	2.158003	2.333923	2.919734
ENSG000000000457	4.084080	2.801146	3.906966	4.176606	4.315053
ENSG000000000460	3.737365	3.586487	4.033272	4.157177	4.283577

```
[522]: gse_log2_tpm.index = gene_metadata.index
gse_log2_tpm.index.name = "Ensembl_gene_id"
```

```
[523]: N_SAMPLES = gse_log2_tpm.shape[1]
N_GENES = gse_log2_tpm.shape[0]
N_DUPLICATED_GENES = gse_log2_tpm.index.duplicated().sum()
N_MISSING_VALUES = gse_log2_tpm.isna().sum().sum()
N_ALL_ZERO_GENES = (gse_log2_tpm.eq(0).all(axis=1)).sum()

gse_expr_qc = pd.DataFrame({
    "Metric": [
        "N_SAMPLES",
        "N_GENES",
        "N_DUPLICATED_GENES",

```



```

        "N_MISSING_VALUES",
        "N_ALL_ZERO_GENES"
    ],
    "Value": [
        N_SAMPLES,
        N_GENES,
        N_DUPLICATED_GENES,
        N_MISSING_VALUES,
        N_ALL_ZERO_GENES
    ]
})

gse_expr_qc

```

```
[523]:
```

	Metric	Value
0	N_SAMPLES	57
1	N_GENES	23978
2	N_DUPLICATED_GENES	0
3	N_MISSING_VALUES	0
4	N_ALL_ZERO_GENES	500

```
[698]: gene_metadata.to_csv("../data/processed/GSE186332/
↳GSE186332_gene_mapping_lengths.tsv", sep="\t")
gse_log2_tpm.to_csv("../data/processed/GSE186332/GSE186332_log2TPM.tsv",
↳sep="\t")
gse_expr_qc.to_excel("../results/GSE186332/GSE186332_expression_QC.xlsx")

```

- GSE186332_gene_mapping_lengths.tsv, GSE186332_log2TPM.tsv and GSE186332_expression_QC.xlsx exported to /results/GSE186332 directory

9.3 3. GLASS-GSE186332 common gene universe and technical transfer

- Source: https://storage.googleapis.com/public-download-files/hgnc/tsv/tsv/hgnc_complete_set.txt

```
[525]: GLASS_FILTERED_GENES = set(glass_idhwt_gbm_primary_R1_log2TPM_filtered.index.
↳astype(str))
GSE186332_GENES = set(gse_log2_tpm.index.astype(str))

```

```
[526]: hgnc = pd.read_csv(
    "../data/raw/annotation/hgnc_complete_set.txt",
    sep="\t",
    dtype=str
)

print("HGNC shape:", hgnc.shape)
print("HGNC columns:")
print(hgnc.columns.tolist())

```

HGNC shape: (45083, 53)

HGNC columns:

```
['hgnc_id', 'symbol', 'name', 'locus_group', 'locus_type', 'status', 'location',  
'alias_symbol', 'alias_name', 'prev_symbol', 'prev_name', 'gene_group',  
'gene_group_id', 'date_approved_reserved', 'date_symbol_changed',  
'date_name_changed', 'date_modified', 'entrez_id', 'ensembl_gene_id', 'vega_id',  
'ucsc_id', 'ena', 'refseq_accession', 'ccds_id', 'uniprot_ids', 'pubmed_id',  
'mgd_id', 'rgd_id', 'lsdb', 'cosmic', 'omim_id', 'mirbase', 'homeodb',  
'snornabase', 'bioparadigms_slc', 'orphanet', 'pseudogene.org', 'horde_id',  
'merops', 'imgt', 'iuphar', 'kznf_gene_catalog', 'mamit-trnadb', 'cd',  
'lncrnadb', 'enzyme_id', 'intermediate_filament_db', 'rna_central_id',  
'lncipedia', 'gtrnadb', 'agr', 'mane_select', 'gencc']
```

```
[527]: hgnc_mapping = hgnc[["symbol", "entrez_id", "ensembl_gene_id"]].copy().rename(  
        columns={"symbol": "Hugo_Symbol",  
                "entrez_id": "NCBIGeneID",  
                "ensembl_gene_id": "Ensembl_gene_id"}  
    ).dropna(subset=["Hugo_Symbol", "Ensembl_gene_id"])  
hgnc_mapping = hgnc_mapping[hgnc_mapping["Ensembl_gene_id"].str.  
    ↪startswith("ENSG")].copy()  
  
print("\nHGNC mapping shape:", hgnc_mapping.shape)  
print(hgnc_mapping.head())
```

HGNC mapping shape: (42399, 3)

	Hugo_Symbol	NCBIGeneID	Ensembl_gene_id
0	A1BG	1	ENSG00000121410
1	A1BG-AS1	503538	ENSG00000268895
2	A1CF	29974	ENSG00000148584
3	A2M	2	ENSG00000175899
4	A2M-AS1	144571	ENSG00000245105

```
[528]: glass_symbols = pd.Index(  
        glass_idhwt_gbm_primary_R1_log2TPM_filtered.index.astype(str),  
        name="Hugo_Symbol"  
    )  
  
glass_hgnc_mapping = hgnc_mapping[  
    hgnc_mapping["Hugo_Symbol"].isin(glass_symbols)  
].copy()
```

```
[529]: symbol_counts = glass_hgnc_mapping.groupby("Hugo_Symbol")["Ensembl_gene_id"].  
    ↪nunique()  
unambiguous_glass_mapping = glass_hgnc_mapping[  
    glass_hgnc_mapping["Hugo_Symbol"].isin(symbol_counts[symbol_counts == 1].  
    ↪index)
```

```

].drop_duplicates(subset=["Hugo_Symbol"]).
↳set_index("Hugo_Symbol")["Ensembl_gene_id"]

print("\nGLASS symbols:", len(glass_symbols))
print("Mapped GLASS symbols:", symbol_counts.size)
print("GLASS symbols with >1 Ensembl:", (symbol_counts > 1).sum())
print("GLASS symbols with 0 mapping:", len(glass_symbols.
↳difference(symbol_counts.index)))
print("Unambiguous GLASS mappings:", len(unambiguous_glass_mapping))
print("\nExample Hugo -> Ensembl mapping:")
print(unambiguous_glass_mapping.head())

```

```

GLASS symbols: 17789
Mapped GLASS symbols: 15041
GLASS symbols with >1 Ensembl: 0
GLASS symbols with 0 mapping: 2748
Unambiguous GLASS mappings: 15041

```

Example Hugo -> Ensembl mapping:

```

Hugo_Symbol
A1BG      ENSG00000121410
A2M       ENSG00000175899
A2ML1     ENSG00000166535
A4GALT    ENSG00000128274
AAAS      ENSG00000094914
Name: Ensembl_gene_id, dtype: str

```

```

[530]: unmapped_glass_symbols = sorted(set(glass_symbols) - set(symbol_counts.index))
print("\nUnmapped GLASS symbols:", len(unmapped_glass_symbols))
print(unmapped_glass_symbols[:100])

```

```

Unmapped GLASS symbols: 2748
['AAED1', 'AARS', 'AATK', 'ABC13-48559800H2.1', 'ABC7-42389800N19.1',
'AC000041.8', 'AC000095.9', 'AC000367.1', 'AC002310.17', 'AC002321.2',
'AC002398.9', 'AC002472.1', 'AC002472.13', 'AC002553.1', 'AC002985.3',
'AC003002.6', 'AC003006.7', 'AC003029.1', 'AC003070.1', 'AC003079.1',
'AC003080.4', 'AC003102.1', 'AC003989.3', 'AC004017.1', 'AC004057.1',
'AC004076.7', 'AC004076.9', 'AC004166.6', 'AC004381.6', 'AC004383.3',
'AC004466.1', 'AC004706.1', 'AC004797.1', 'AC004824.2', 'AC004878.3',
'AC004951.5', 'AC004967.7', 'AC004980.10', 'AC004980.7', 'AC004980.8',
'AC004980.9', 'AC004985.12', 'AC005003.1', 'AC005013.1', 'AC005077.12',
'AC005082.1', 'AC005086.1', 'AC005086.2', 'AC005086.5', 'AC005086.8',
'AC005154.7', 'AC005154.8', 'AC005280.1', 'AC005326.2', 'AC005336.4',
'AC005363.9', 'AC005399.1', 'AC005488.1', 'AC005517.3', 'AC005618.1',
'AC005776.1', 'AC005779.1', 'AC005786.3', 'AC005831.1', 'AC006011.4',
'AC006014.1', 'AC006014.7', 'AC006014.8', 'AC006035.1', 'AC006042.8',

```

```
'AC006115.6', 'AC006116.20', 'AC006116.27', 'AC006449.1', 'AC006466.5',
'AC006486.9', 'AC006509.7', 'AC006538.4', 'AC006547.14', 'AC006548.19',
'AC006960.5', 'AC006978.6', 'AC006987.4', 'AC006995.8', 'AC007000.10',
'AC007000.11', 'AC007000.12', 'AC007040.11', 'AC007204.1', 'AC007216.2',
'AC007228.8', 'AC007283.4', 'AC007347.1', 'AC007362.1', 'AC007382.1',
'AC007461.1', 'AC007560.1', 'AC007875.2', 'AC007875.3', 'AC007969.5']
```

```
[531]: hgnc_alias_lookup = {}
for _, row in hgnc.iterrows():
    symbol = row["symbol"]
    if pd.notna(row["alias_symbol"]):
        for alias in str(row["alias_symbol"]).split("|"):
            hgnc_alias_lookup.setdefault(alias, set()).add(symbol)
    if pd.notna(row["prev_symbol"]):
        for prev in str(row["prev_symbol"]).split("|"):
            hgnc_alias_lookup.setdefault(prev, set()).add(symbol)

alias_resolved = {gene: hgnc_alias_lookup.get(gene, set()) for gene in
    ↪unmapped_glass_symbols}
n_alias_resolved = sum(len(v) > 0 for v in alias_resolved.values())
print("\nUnmapped symbols resolvable via HGNC alias/previous symbol:",
    ↪n_alias_resolved)

alias_resolved_examples = {gene: sorted(targets) for gene, targets in
    ↪alias_resolved.items() if targets}
print("\nFirst alias-resolved examples:")
print(list(alias_resolved_examples.items())[:50])
```

Unmapped symbols resolvable via HGNC alias/previous symbol: 1206

First alias-resolved examples:

```
(('AAED1', ['PRXL2C']), ('AARS', ['AARS1']), ('AATK', ['LMTK1']), ('AC010642.1',
['ZNF8-ERVK3-1']), ('AC015660.1', ['LINC03080']), ('AC079776.2', ['LINC02572']),
('AC096677.1', ['CSRP1-AS1']), ('ACN9', ['SDHAF3']), ('ACPL2', ['PXYP1']),
('ACPP', ['ACP3']), ('ACRC', ['GCNA']), ('ADC', ['AZIN2']), ('ADCK3',
['COQ8A']), ('ADCK4', ['COQ8B']), ('ADPRHL2', ['ADPRS']), ('ADRBK1', ['GRK2']),
('ADRBK2', ['GRK3']), ('ADSS', ['ADSS2']), ('ADSSL1', ['ADSS1']), ('AES',
['TLE5']), ('AGAP10', ['AGAP10P']), ('AGAP8', ['AGAP4']), ('AGPAT6', ['GPAT4']),
('AGPAT9', ['GPAT3', 'LPCAT1']), ('AHS2', ['AHS2P']), ('AIM1', ['CRYBG1']),
('AL139147.1', ['SGIP1-AS1']), ('AL139385.1', ['NAXD-AS1']), ('AL591806.1',
['NECTIN4-AS1']), ('ALMS1P', ['ALMS1P1']), ('ALS2CR11', ['CATSPERT']),
('ALS2CR12', ['FLACC1']), ('AMICA1', ['JAML']), ('ANKRD20A2', ['ANKRD20A2P']),
('ANKRD20A3', ['ANKRD20A3P']), ('ANKRD20A4', ['ANKRD20A4P']), ('ANKRD32',
['SLF1']), ('ANO7P1', ['ANO7L1']), ('APO01007.1', ['PKNX2-AS1']), ('APITD1',
['CENPS']), ('APITD1-CORT', ['CENPS-CORT']), ('APOA1BP', ['NAXE']), ('APOPT1',
['COA8']), ('AQPEP', ['LVRN']), ('ARMC4', ['ODAD2']), ('ARSE', ['ARSL']),
('ASNA1', ['GET3']), ('ASUN', ['INTS13']), ('ATHL1', ['PGGHG']), ('ATP5A1',
```

```
['ATP5F1A']])
```

```
[532]: GLASS_FILTERED_GENES_ENSEMBL = set(unambiguous_glass_mapping.values)
print("\nGLASS Ensembl gene universe:", len(GLASS_FILTERED_GENES_ENSEMBL))

GSE186332_COMMON_GENE_UNIVERSE = (GLASS_FILTERED_GENES_ENSEMBL &
    ↪GSE186332_GENES)
print("GSE186332_COMMON_GENE_UNIVERSE:", len(GSE186332_COMMON_GENE_UNIVERSE))
```

```
GLASS Ensembl gene universe: 15041
GSE186332_COMMON_GENE_UNIVERSE: 14229
```

```
[533]: common_gene_mapping = glass_hgnc_mapping[
    glass_hgnc_mapping["Hugo_Symbol"].isin(unambiguous_glass_mapping.index) &
    glass_hgnc_mapping["Ensembl_gene_id"].isin(GSE186332_COMMON_GENE_UNIVERSE)
].drop_duplicates(subset=["Hugo_Symbol", "Ensembl_gene_id"]).
    ↪sort_values(["Ensembl_gene_id", "Hugo_Symbol"])

print("\nCommon gene mapping shape:", common_gene_mapping.shape)
print(common_gene_mapping.head())
```

```
Common gene mapping shape: (14229, 3)
   Hugo_Symbol  NCBIGeneID  Ensembl_gene_id
37314      TSPAN6        7105  ENSG00000000003
43815        DPM1        8813  ENSG000000000419
30081       SCYL3       57147  ENSG000000000457
1951         FGR        2268  ENSG000000000938
41171        CFH        3075  ENSG000000000971
```

```
[534]: gse186332_common = gse_log2_tpm.loc[gse_log2_tpm.index.
    ↪isin(GSE186332_COMMON_GENE_UNIVERSE)].sort_index().copy()

print("\nGSE186332 common matrix:", gse186332_common.shape)
```

```
GSE186332 common matrix: (14229, 57)
```

```
[535]: gse_common_ensembl_ids = set(gse186332_common.index.astype(str))
glass_common_ensembl_ids = set(GSE186332_COMMON_GENE_UNIVERSE)

assert (gse_common_ensembl_ids == GSE186332_COMMON_GENE_UNIVERSE)
assert (glass_common_ensembl_ids == gse_common_ensembl_ids)
assert all(gene.startswith("ENSG")
    for gene in GSE186332_COMMON_GENE_UNIVERSE)

print("\nGLASS common Ensembl genes:", len(glass_common_ensembl_ids))
print("GSE186332 common Ensembl genes:", len(gse_common_ensembl_ids))
```

GLASS common Ensembl genes: 14229
GSE186332 common Ensembl genes: 14229

```
[536]: common_ensembl = pd.Index(sorted(GSE186332_COMMON_GENE_UNIVERSE),  
    ↪name="Ensembl_gene_id")  
print("Common universe:", len(common_ensembl))
```

Common universe: 14229

```
[537]: consensus_hugo_genes = set(consensus_genes.dropna().astype(str))  
consensus_up_hugo_genes = set(consensus_up_genes.dropna().astype(str))  
consensus_down_hugo_genes = set(consensus_down_genes.dropna().astype(str))  
  
assert consensus_up_hugo_genes <= consensus_hugo_genes  
assert consensus_down_hugo_genes <= consensus_hugo_genes
```

```
[538]: consensus_to_ensembl = unambiguous_glass_mapping.  
    ↪reindex(sorted(consensus_hugo_genes)).dropna()  
consensus_up_to_ensembl = unambiguous_glass_mapping.  
    ↪reindex(sorted(consensus_up_hugo_genes)).dropna()  
consensus_down_to_ensembl = unambiguous_glass_mapping.  
    ↪reindex(sorted(consensus_down_hugo_genes)).dropna()
```

```
[539]: common_ensembl_genes = set(GSE186332_COMMON_GENE_UNIVERSE)  
consensus_ensembl_genes = set(consensus_to_ensembl.values)  
consensus_up_ensembl_genes = set(consensus_up_to_ensembl.values)  
consensus_down_ensembl_genes = set(consensus_down_to_ensembl.values)
```

```
[540]: common_consensus_genes = (consensus_ensembl_genes & common_ensembl_genes)  
common_consensus_up_genes = (consensus_up_ensembl_genes & common_ensembl_genes)  
common_consensus_down_genes = (consensus_down_ensembl_genes &  
    ↪common_ensembl_genes)  
N_COMMON_GENES = len(common_consensus_genes)  
N_SIGNATURE_UP_AVAILABLE = len(common_consensus_up_genes)  
N_SIGNATURE_DOWN_AVAILABLE = len(common_consensus_down_genes)
```

```
[541]: coverage = pd.DataFrame({  
    "metric": [  
        "N_COMMON_GENES",  
        "N_SIGNATURE_UP_AVAILABLE",  
        "N_SIGNATURE_DOWN_AVAILABLE",  
    ],  
    "value": [  
        N_COMMON_GENES,  
        N_SIGNATURE_UP_AVAILABLE,  
        N_SIGNATURE_DOWN_AVAILABLE,  
    ],  
})
```

```
})

coverage
```

```
[541]:
```

	metric	value
0	N_COMMON_GENES	253
1	N_SIGNATURE_UP_AVAILABLE	161
2	N_SIGNATURE_DOWN_AVAILABLE	92

```
[542]: coverage.to_excel(
    "../../results/GSE186332/glass_gse186332_gene_coverage.xlsx",
    index=False
)
```

- glass_gse186332_gene_coverage.xlsx exported to /results/GSE186332 directory

```
[543]: glass_ensembl_expr = glass_idhwt_gbm_primary_R1_log2TPM_filtered.copy()
glass_ensembl_expr.index = glass_ensembl_expr.index.astype(str).
    ↪map(unambiguous_glass_mapping)
glass_ensembl_expr.index.name = "Ensembl_gene_id"
glass_ensembl_expr = glass_ensembl_expr.loc[glass_ensembl_expr.index.notna()].
    ↪copy()

print("Shape:", glass_ensembl_expr.shape)
print("Duplicated Ensembl IDs:", glass_ensembl_expr.index.duplicated().sum())
```

```
Shape: (15041, 206)
Duplicated Ensembl IDs: 0
```

```
[544]: glass_common = glass_ensembl_expr.loc[glass_ensembl_expr.index.
    ↪isin(GSE186332_COMMON_GENE_UNIVERSE)].copy()
glass_recurrence_score_gse186332_universe =
    ↪calculate_recurrence_score(glass_common,
    ↪common_consensus_up_genes, common_consensus_down_genes)
scores = glass_recurrence_score_gse186332_universe.copy()
pairs = glass_idhwt_gbm_primary_R1_pairs

sample_info = pd.concat([
    pairs[["PATIENT_ID", "PRIMARY_SAMPLE_ID"]]
    .rename(columns={"PRIMARY_SAMPLE_ID": "SAMPLE_ID"})
    .assign(SAMPLE_TYPE="Tumor Primary"),

    pairs[["PATIENT_ID", "FIRST_RECURRENCE_SAMPLE_ID"]]
    .rename(columns={"FIRST_RECURRENCE_SAMPLE_ID": "SAMPLE_ID"})
    .assign(SAMPLE_TYPE="First Recurrence")
], ignore_index=True)

sample_info
```

```
[544]:
```

	PATIENT_ID	SAMPLE_ID	SAMPLE_TYPE
0	GLSS-19-0267	GLSS-19-0267-TP	Tumor Primary
1	GLSS-19-0268	GLSS-19-0268-TP	Tumor Primary
2	GLSS-19-0269	GLSS-19-0269-TP	Tumor Primary
3	GLSS-19-0271	GLSS-19-0271-TP	Tumor Primary
4	GLSS-19-0272	GLSS-19-0272-TP	Tumor Primary
..	...	...	...
201	TCGA-06-0210	TCGA-06-0210-R1	First Recurrence
202	TCGA-06-0211	TCGA-06-0211-R1	First Recurrence
203	TCGA-14-1034	TCGA-14-1034-R1	First Recurrence
204	TCGA-19-0957	TCGA-19-0957-R1	First Recurrence
205	TCGA-19-4065	TCGA-19-4065-R1	First Recurrence

[206 rows x 3 columns]

```
[545]: score_info = sample_info.merge(
        scores[["RECURRENCE_SCORE"]],
        left_on="SAMPLE_ID",
        right_index=True,
        how="inner")

print(score_info.head())
print(score_info.shape)
```

	PATIENT_ID	SAMPLE_ID	SAMPLE_TYPE	RECURRENCE_SCORE
0	GLSS-19-0267	GLSS-19-0267-TP	Tumor Primary	-0.171299
1	GLSS-19-0268	GLSS-19-0268-TP	Tumor Primary	-0.298567
2	GLSS-19-0269	GLSS-19-0269-TP	Tumor Primary	-0.264337
3	GLSS-19-0271	GLSS-19-0271-TP	Tumor Primary	-0.175296
4	GLSS-19-0272	GLSS-19-0272-TP	Tumor Primary	-0.312530

(206, 4)

```
[546]: paired_scores = (
    score_info
    .pivot_table(
        index="PATIENT_ID",
        columns="SAMPLE_TYPE",
        values="RECURRENCE_SCORE"
    ).dropna(subset=["Tumor Primary", "First Recurrence"]))
```

```
[547]: paired_scores = paired_scores.rename(columns={
    "Tumor Primary": "PRIMARY_SCORE",
    "First Recurrence": "R1_SCORE"})

paired_scores["DELTA"] = (paired_scores["R1_SCORE"] -
    paired_scores["PRIMARY_SCORE"])
```



```
[548]: N_PATIENTS = int(len(paired_scores))
MEAN_DELTA = paired_scores["DELTA"].mean()
MEDIAN_DELTA = paired_scores["DELTA"].median()
PAIRED_T_PVALUE = ttest_rel(paired_scores["R1_SCORE"],
    ↪paired_scores["PRIMARY_SCORE"]).pvalue
WILCOXON_PVALUE = wilcoxon(paired_scores["R1_SCORE"],
    ↪paired_scores["PRIMARY_SCORE"]).pvalue
FRACTION_R1_GREATER_THAN_PRIMARY = (paired_scores["R1_SCORE"] >
    ↪paired_scores["PRIMARY_SCORE"]).mean()
```

```
[549]: paired_scores
```

```
[549]: SAMPLE_TYPE  R1_SCORE  PRIMARY_SCORE    DELTA
PATIENT_ID
GLSS-19-0267    0.255018      -0.171299    0.426317
GLSS-19-0268    0.455626      -0.298567    0.754193
GLSS-19-0269    0.104326      -0.264337    0.368663
GLSS-19-0271    0.121060      -0.175296    0.296356
GLSS-19-0272    0.316930      -0.312530    0.629460
...
TCGA-06-0210   -0.126675      -0.173507    0.046833
TCGA-06-0211   -0.339076      -0.405362    0.066285
TCGA-14-1034   -0.227140      -0.347053    0.119913
TCGA-19-0957    0.103943      -0.199312    0.303255
TCGA-19-4065    0.024457      -0.282202    0.306659
```

```
[103 rows x 3 columns]
```

```
[550]: glass_recurrence_score_gse186332_universe_summary = pd.DataFrame({
    "Metric": [
        "N_PATIENTS",
        "MEAN_DELTA",
        "MEDIAN_DELTA",
        "PAIRED_T_PVALUE",
        "WILCOXON_PVALUE",
        "FRACTION_R1_GREATER_THAN_PRIMARY",
    ],
    "Value": [
        f"{N_PATIENTS:d}",
        f"{MEAN_DELTA:.4f}",
        f"{MEDIAN_DELTA:.4f}",
        f"{PAIRED_T_PVALUE:.2e}",
        f"{WILCOXON_PVALUE:.2e}",
        f"{FRACTION_R1_GREATER_THAN_PRIMARY * 100:.2f}%",
    ],
})
```

```
[551]: glass_recurrence_score_gse186332_universe_summary
```

```
[551]:
```

	Metric	Value
0	N_PATIENTS	103
1	MEAN_DELTA	0.2325
2	MEDIAN_DELTA	0.2357
3	PAIRED_T_PVALUE	5.87e-15
4	WILCOXON_PVALUE	1.62e-12
5	FRACTION_R1_GREATER_THAN_PRIMARY	79.61%

```
[552]: output_file = "../results/GSE186332/  
        ↪glass_recurrence_score_gse186332_universe.xlsx"  
  
with pd.ExcelWriter(output_file, engine="openpyxl") as writer:  
    glass_recurrence_score_gse186332_universe_summary.to_excel(  
        writer,  
        sheet_name="Summary",  
        index=False  
    )  
  
    glass_recurrence_score_gse186332_universe.to_excel(  
        writer,  
        sheet_name="Paired_Scores",  
        index=True  
    )
```

- glass_recurrence_score_gse186332_universe.xlsx exported to /results/GSE186332 directory

9.4 4. Calculating Frozen Recurrence Score in GSE186332

```
[553]: gse186332_common = gse_log2_tpm.loc[gse_log2_tpm.index.  
        ↪isin(GSE186332_COMMON_GENE_UNIVERSE)].sort_index().copy()  
assert set(gse186332_common.index) == GSE186332_COMMON_GENE_UNIVERSE  
assert not gse186332_common.index.duplicated().any()  
assert not gse186332_common.isna().any().any()  
  
print("Frozen common gene universe:", gse186332_common.shape)
```

Frozen common gene universe: (14229, 57)

```
[554]: gse186332_frozen_recurrence_scores =  
        ↪calculate_recurrence_score(gse186332_common, common_consensus_up_genes,  
        ↪common_consensus_down_genes)  
gse186332_frozen_recurrence_scores.index.name = "SAMPLE_ID"  
gse186332_frozen_recurrence_scores = gse186332_frozen_recurrence_scores.  
        ↪rename(columns={"RECURRENCE_SCORE": "GSE186332_RECURRENCE_SCORE"})
```

```

assert gse186332_frozen_recurrence_scores.index.equals(gse186332_common.columns)
assert not gse186332_frozen_recurrence_scores.isna().any().any()

print(gse186332_frozen_recurrence_scores.shape)
gse186332_frozen_recurrence_scores.head()

```

(57, 3)

```

[554]:          UP_SCORE  DOWN_SCORE  GSE186332_RECURRENCE_SCORE
SAMPLE_ID
001-009      0.674123      0.370553                0.303569
001-015      0.311839      0.402374               -0.090535
001-037      0.521839      0.522477               -0.000638
001-038      0.686762      0.337676                0.349085
001-043      0.297213      0.551267               -0.254054

```

```

[555]: gse186332_frozen_recurrence_scores.to_excel(
        ".../results/GSE186332/gse186332_frozen_recurrence_scores.xlsx",
        index=True
    )

```

- gse186332_frozen_recurrence_scores.xlsx exported to /results/GSE186332 directory

9.5 5. Primary clinical selinexor-analysis

```

[556]: gse186332_clinical_for_join = gse186332_selinexor_clinical_metadata.
        ↪set_index("PATIENT_ID").copy()
assert gse186332_clinical_for_join.index.is_unique

selinexor_analysis = gse186332_frozen_recurrence_scores.
        ↪join(gse186332_clinical_for_join, how="inner").copy()
selinexor_analysis = selinexor_analysis[selinexor_analysis["BENEFIT_STATUS"].
        ↪isin(["Clinical Benefit", "Resistance"])]

print("N samples:", len(selinexor_analysis))
print(selinexor_analysis["BENEFIT_STATUS"].value_counts())

```

```

N samples: 52
BENEFIT_STATUS
Resistance      34
Clinical Benefit  18
Name: count, dtype: int64

```

```

[557]: selinexor_analysis["BENEFIT_BINARY"] = (selinexor_analysis["BENEFIT_STATUS"] ==
        ↪"Clinical Benefit").astype(int)

```

```

[558]: rs_benefit = selinexor_analysis.loc[selinexor_analysis["BENEFIT_BINARY"] == 1,
        ↪"GSE186332_RECURRENCE_SCORE"]

```

```

rs_resistance = selinexor_analysis.loc[selinexor_analysis["BENEFIT_BINARY"] == 0, "GSE186332_RECURRENCE_SCORE"]

N_BENEFIT, N_RESISTANCE = len(rs_benefit), len(rs_resistance)
MEDIAN_RS_BENEFIT, MEDIAN_RS_RESISTANCE = rs_benefit.median(), rs_resistance.
    median()

mw = mannwhitneyu(rs_benefit, rs_resistance, alternative="two-sided")
MANN_WHITNEY_P = mw.pvalue
RANK_BISERIAL_CORRELATION = (2 * mw.statistic / (N_BENEFIT * N_RESISTANCE)) - 1

```

```

[559]: y, scores = selinexor_analysis["BENEFIT_BINARY"].to_numpy(),
    selinexor_analysis["GSE186332_RECURRENCE_SCORE"].to_numpy()
ROC_AUC = roc_auc_score(y, scores)

rng = np.random.default_rng(42)
bootstrap_aucs = []

for _ in range(2000):
    idx = rng.integers(0, len(y), size=len(y))
    y_boot, scores_boot = y[idx], scores[idx]
    if len(np.unique(y_boot)) < 2: continue
    bootstrap_aucs.append(roc_auc_score(y_boot, scores_boot))

ROC_AUC_CI_LOWER, ROC_AUC_CI_UPPER = np.percentile(bootstrap_aucs, [2.5, 97.5])

```

```

[560]: rs = selinexor_analysis["GSE186332_RECURRENCE_SCORE"]
selinexor_analysis["RS_Z"] = (rs - rs.mean()) / rs.std(ddof=1)
clean_df = selinexor_analysis[["RS_Z", "BENEFIT_BINARY"]].dropna()
X = sm.add_constant(clean_df["RS_Z"])
y = clean_df["BENEFIT_BINARY"]
logit_model = sm.Logit(y, X).fit(disp=False)
beta = logit_model.params["RS_Z"]
beta_ci = logit_model.conf_int().loc["RS_Z"]
OR_PER_1SD = float(np.exp(beta))
OR_CI_LOWER = float(np.exp(beta_ci.iloc[0]))
OR_CI_UPPER = float(np.exp(beta_ci.iloc[1]))
LOGISTIC_P_VALUE = float(logit_model.pvalues["RS_Z"])

print(f"OR per +1 SD: {OR_PER_1SD:.3f} (95% CI: {OR_CI_LOWER:.3f} -
    {OR_CI_UPPER:.3f}), p = {LOGISTIC_P_VALUE:.4e}")

```

OR per +1 SD: 0.713 (95% CI: 0.389 - 1.307), p = 2.7387e-01

```

[561]: selinexor_recurrence_score_summary = pd.DataFrame({
    "metric": [
        "N_BENEFIT",

```

```

        "N_RESISTANCE",
        "MEDIAN_RS_BENEFIT",
        "MEDIAN_RS_RESISTANCE",
        "MANN_WHITNEY_P",
        "RANK_BISERIAL_CORRELATION",
        "ROC_AUC",
        "ROC_AUC_CI_LOWER",
        "ROC_AUC_CI_UPPER",
        "OR_PER_1SD",
        "OR_CI_LOWER",
        "OR_CI_UPPER",
        "LOGISTIC_P_VALUE",
    ],
    "value": [
        N_BENEFIT,
        N_RESISTANCE,
        MEDIAN_RS_BENEFIT,
        MEDIAN_RS_RESISTANCE,
        MANN_WHITNEY_P,
        RANK_BISERIAL_CORRELATION,
        ROC_AUC,
        ROC_AUC_CI_LOWER,
        ROC_AUC_CI_UPPER,
        OR_PER_1SD,
        OR_CI_LOWER,
        OR_CI_UPPER,
        LOGISTIC_P_VALUE,
    ]
}

```

```
[562]: print(selinexor_recurrence_score_summary)
```

	metric	value
0	N_BENEFIT	18.000000
1	N_RESISTANCE	34.000000
2	MEDIAN_RS_BENEFIT	-0.161232
3	MEDIAN_RS_RESISTANCE	-0.155900
4	MANN_WHITNEY_P	0.371110
5	RANK_BISERIAL_CORRELATION	-0.153595
6	ROC_AUC	0.423203
7	ROC_AUC_CI_LOWER	0.262448
8	ROC_AUC_CI_UPPER	0.604701
9	OR_PER_1SD	0.712863
10	OR_CI_LOWER	0.388781
11	OR_CI_UPPER	1.307097
12	LOGISTIC_P_VALUE	0.273871

```

[563]: plt.rcParams.update({"font.family": "sans-serif", "font.sans-serif":
    ↳ ["Helvetica Neue", "Helvetica", "Arial", "DejaVu Sans"], "axes.edgecolor":
    ↳ "#333333", "axes.linewidth": 0.8})

rng = np.random.default_rng(42)
groups = [np.asarray(rs_benefit), np.asarray(rs_resistance)]
group_names = ["Clinical Benefit", "No Benefit / Resistance"]
colors = ["#2e7d32", "#c62828"]

fig, ax = plt.subplots(figsize=(5.5, 5.5), dpi=300)
bp = ax.boxplot(groups, label=group_names, patch_artist=True, widths=0.45,
    ↳ showfliers=False, zorder=2)

for patch, color in zip(bp["boxes"], colors):
    patch.set(facecolor=color, alpha=0.25, edgecolor=color, linewidth=1.5)

for element in ["whiskers", "caps"]:
    plt.setp(bp[element], color="#555555", linewidth=1.2)

for median, color in zip(bp["medians"], colors):
    plt.setp(median, color=color, linewidth=2.5)

for i, (vals, color) in enumerate(zip(groups, colors), start=1):
    vals_clean = vals[~np.isnan(vals)]
    x_jitter = rng.uniform(-0.08, 0.08, size=len(vals_clean)) + i
    ax.scatter(x_jitter, vals_clean, color=color, s=40, alpha=0.7,
    ↳ edgecolors="white", linewidths=0.5, zorder=3)

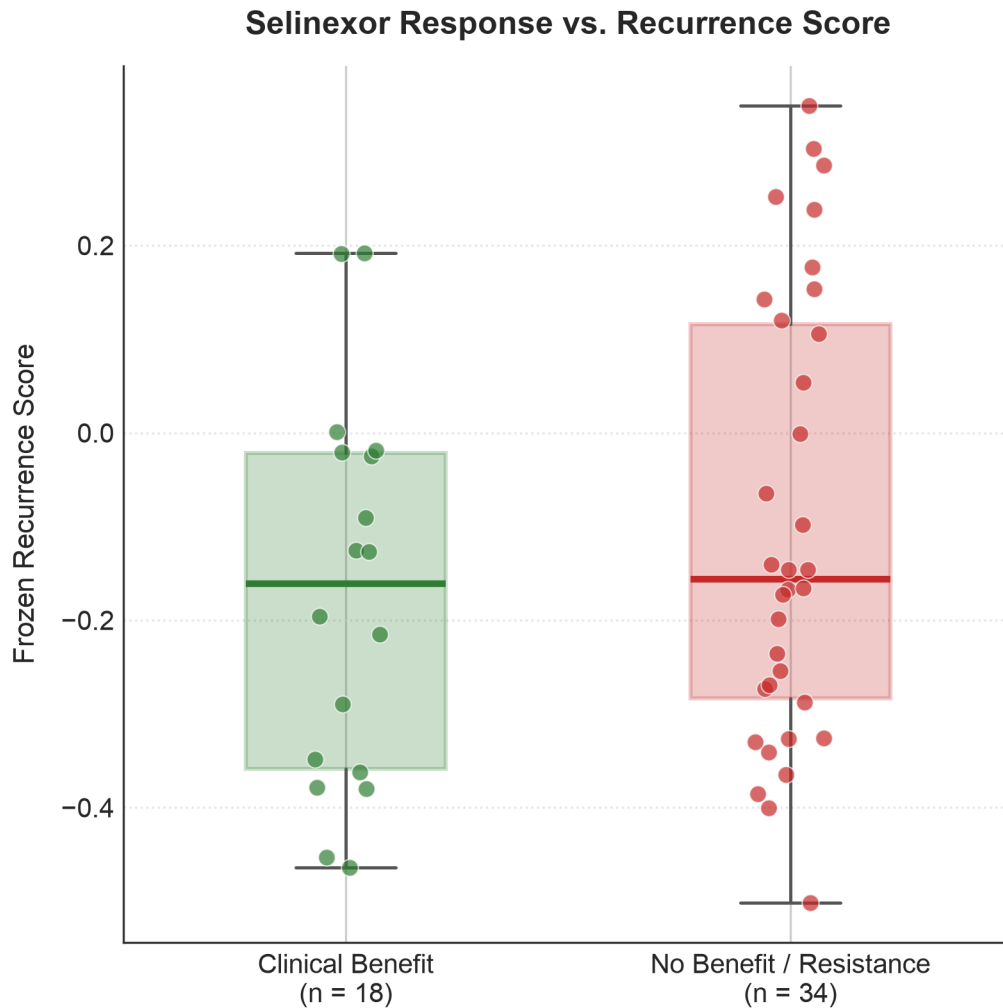
ax.set_ylabel("Frozen Recurrence Score", fontsize=10.5, labelpad=8)
ax.set_title("Selinexor Response vs. Recurrence Score", fontsize=12,
    ↳ fontweight="bold", pad=12)

counts = [len(g[~np.isnan(g)]) for g in groups]
ax.set_xticklabels([f"{name}\n(n = {n})" for name, n in zip(group_names,
    ↳ counts)], fontsize=10)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(axis="y", color="#e5e5e5", linestyle=":", linewidth=0.8, zorder=1)

fig.tight_layout()
fig.savefig("../figures/selinexor_recurrence_score_benefit.png", dpi=300,
    ↳ bbox_inches="tight")
plt.show()

```



- selinexor_recurrence_score_benefit.png exported to /figures directory

```
[564]: y_eval = selinexor_analysis["BENEFIT_BINARY"].to_numpy()
scores_eval = selinexor_analysis["GSE186332_RECURRENCE_SCORE"].to_numpy()
fpr, tpr, thresholds = roc_curve(y_eval, scores_eval)
ROC_AUC = roc_auc_score(y_eval, scores_eval)

plt.rcParams.update({"font.family": "sans-serif", "font.sans-serif":
    ["Helvetica Neue", "Helvetica", "Arial", "DejaVu Sans"], "axes.edgecolor":
    "#333333", "axes.linewidth": 0.8})
fig, ax = plt.subplots(figsize=(5.5, 5.5), dpi=300)

ax.plot(fpr, tpr, color="#c62828", linewidth=2, label=f"Selinexor Response (AUC_
    {ROC_AUC:.3f})", zorder=3)
```

```

ax.plot([0, 1], [0, 1], color="#888888", linestyle="--", linewidth=1.2,
        label="Random Chance (AUC = 0.500)", zorder=2)

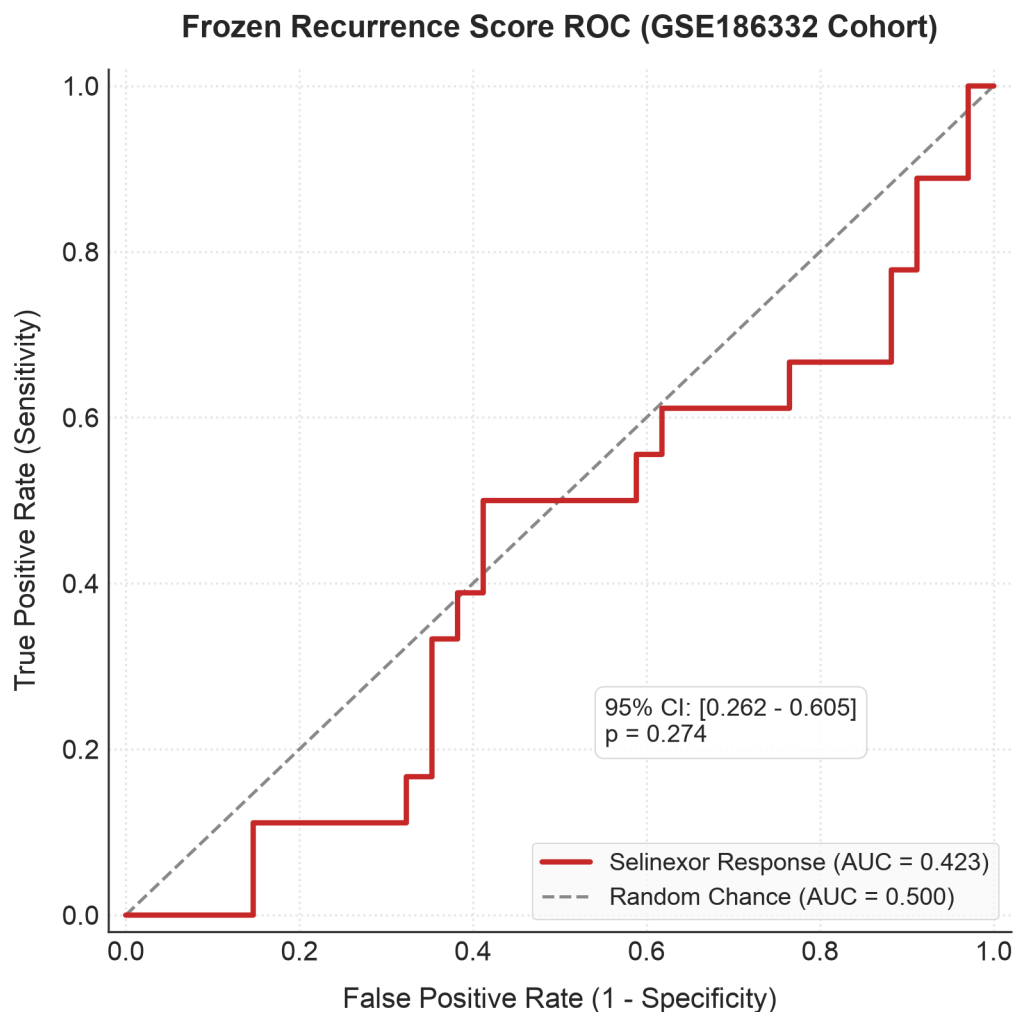
ax.set_xlim([-0.02, 1.02])
ax.set_ylim([-0.02, 1.02])
ax.set_xlabel("False Positive Rate (1 - Specificity)", fontsize=10.5,
              labelpad=8)
ax.set_ylabel("True Positive Rate (Sensitivity)", fontsize=10.5, labelpad=8)
ax.set_title("Frozen Recurrence Score ROC (GSE186332 Cohort)", fontsize=11.5,
             fontweight="bold", pad=12)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(color="#e5e5e5", linestyle=":", linewidth=0.8, zorder=1)

ax.text(
    0.55, 0.22,
    f"95% CI: [{ROC_AUC_CI_LOWER:.3f} - {ROC_AUC_CI_UPPER:.3f}]\np =\n{LOGISTIC_P_VALUE:.3f}",
    transform=ax.transAxes, fontsize=9,
    bbox=dict(boxstyle="round,pad=0.4", facecolor="ffffff",
              edgecolor="#d3d3d3", alpha=0.9)
)
ax.legend(loc="lower right", frameon=True, facecolor="#f9f9f9",
         edgecolor="#d3d3d3", fontsize=9.0)

fig.tight_layout()
fig.savefig("../figures/selinexor_recurrence_score_ROC.png", dpi=300,
            bbox_inches="tight")
plt.show()

```

- selinexor_recurrence_score_ROC.png exported to /figures directory

```
[565]: output_file = "../../../results/GSE186332/
        ↪selinexor_recurrence_score_clinical_association.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    selinexor_recurrence_score_summary.to_excel(writer, sheet_name="Summary",
    ↪index=False)
    selinexor_analysis.to_excel(writer, sheet_name="Patient_Level", index=True)
```

Summary Note: In the 52-patient evaluable GSE186332 cohort (18 clinical benefit, 34 resistance), the frozen Recurrence Score was not significantly associated with selinexor clinical benefit (Mann–Whitney $p=0.371$; ROC AUC=0.423, 95% CI 0.262–0.605; OR per 1-SD=0.713, 95% CI 0.389–1.307, $p=0.274$).

- selinexor_recurrence_score_clinical_association.xlsx exported to /results/GSE186332 directory

9.6 6. Predefined clinical robustness analysis

9.6.1 6.1 Clear-response cohort

```
[566]: gse186332_clinical_for_join = gse186332_selinexor_clinical_metadata.  
        ↪set_index("PATIENT_ID").copy()  
assert gse186332_clinical_for_join.index.is_unique  
clear_response_analysis = gse186332_frozen_recurrence_scores.  
        ↪join(gse186332_clinical_for_join, how="inner").copy()
```

```
[567]: clear_response_analysis =  
        ↪clear_response_analysis[clear_response_analysis["EXPERIMENTAL_SET"] ==  
        ↪"Discovery"].copy()  
clear_response_analysis =  
        ↪clear_response_analysis[clear_response_analysis["BEST_OVERALL_RESPONSE"].  
        ↪isin(["CR/PR", "PD"])].copy()  
  
print(clear_response_analysis["BEST_OVERALL_RESPONSE"].value_counts())  
  
assert clear_response_analysis["BEST_OVERALL_RESPONSE"].value_counts().  
        ↪to_dict() == {"PD": 23, "CR/PR": 7}
```

```
BEST_OVERALL_RESPONSE  
PD          23  
CR/PR        7  
Name: count, dtype: int64
```

```
[568]: clear_response_analysis["BENEFIT_BINARY"] =  
        ↪(clear_response_analysis["BEST_OVERALL_RESPONSE"] == "CR/PR").astype(int)  
  
rs_crpr = clear_response_analysis.loc[clear_response_analysis["BENEFIT_BINARY"]  
        ↪== 1, "GSE186332_RECURRENCE_SCORE"]  
rs_pd = clear_response_analysis.loc[clear_response_analysis["BENEFIT_BINARY"]  
        ↪== 0, "GSE186332_RECURRENCE_SCORE"]  
  
N_CRPR, N_PD = len(rs_crpr), len(rs_pd)  
MEDIAN_RS_CRPR, MEDIAN_RS_PD = rs_crpr.median(), rs_pd.median()
```

```
[569]: mw = mannwhitneyu(rs_crpr, rs_pd, alternative="two-sided")  
MANN_WHITNEY_U = mw.statistic  
MANN_WHITNEY_P = mw.pvalue  
RANK_BISERIAL_CORRELATION = (2 * MANN_WHITNEY_U / (N_CRPR * N_PD) - 1)
```

```
[570]: y = clear_response_analysis["BENEFIT_BINARY"].to_numpy()  
scores = clear_response_analysis["GSE186332_RECURRENCE_SCORE"].to_numpy()  
ROC_AUC = roc_auc_score(y, scores)  
  
rng = np.random.default_rng(42)
```

```

bootstrap_auc = []

for _ in range(2000):
    idx = rng.integers(0, len(y), size=len(y))
    y_boot, scores_boot = y[idx], scores[idx]
    if len(np.unique(y_boot)) < 2:
        continue
    bootstrap_auc.append(roc_auc_score(y_boot, scores_boot))

ROC_AUC_CI_LOWER = np.percentile(bootstrap_auc, 2.5)
ROC_AUC_CI_UPPER = np.percentile(bootstrap_auc, 97.5)

```

```

[571]: clear_response_summary = pd.DataFrame({
    "metric": [
        "N_CR_PR",
        "N_PD",
        "MEDIAN_RS_CR_PR",
        "MEDIAN_RS_PD",
        "MANN_WHITNEY_U",
        "MANN_WHITNEY_P",
        "RANK_BISERIAL_CORRELATION",
        "ROC_AUC",
        "ROC_AUC_CI_LOWER",
        "ROC_AUC_CI_UPPER",
    ],
    "value": [
        N_CRPR,
        N_PD,
        MEDIAN_RS_CRPR,
        MEDIAN_RS_PD,
        MANN_WHITNEY_U,
        MANN_WHITNEY_P,
        RANK_BISERIAL_CORRELATION,
        ROC_AUC,
        ROC_AUC_CI_LOWER,
        ROC_AUC_CI_UPPER,
    ],
})

clear_response_summary

```

```

[571]:

```

	metric	value
0	N_CR_PR	7.000000
1	N_PD	23.000000
2	MEDIAN_RS_CR_PR	-0.090535
3	MEDIAN_RS_PD	-0.166923
4	MANN_WHITNEY_U	84.000000

5	MANN_WHITNEY_P	0.885738
6	RANK_BISERIAL_CORRELATION	0.043478
7	ROC_AUC	0.521739
8	ROC_AUC_CI_LOWER	0.264095
9	ROC_AUC_CI_UPPER	0.776536

```
[572]: clear_response_patient_level = clear_response_analysis[
    [
        "GSE186332_RECURRENCE_SCORE",
        "TREATMENT_ARM",
        "BEST_OVERALL_RESPONSE",
        "PFS",
        "BENEFIT_STATUS",
        "RESPONSE_GROUP",
        "IDH1_STATUS",
        "RNA_AVAILABLE",
    ]
].copy()

clear_response_patient_level
```

```
[572]:
```

SAMPLE_ID	GSE186332_RECURRENCE_SCORE	TREATMENT_ARM	BEST_OVERALL_RESPONSE \
001-009	0.303569	A	PD
001-015	-0.090535	A	CR/PR
001-037	-0.000638	C	PD
001-038	0.349085	D	PD
001-043	-0.254054	D	PD
001-047	0.190930	D	CR/PR
001-064	-0.362106	D	CR/PR
001-076	-0.501270	D	PD
002-014	-0.166923	B	PD
002-021	-0.385560	B	PD
002-030	0.053960	B	PD
002-064	0.285845	D	PD
002-070	0.119757	D	PD
002-074	-0.326396	D	PD
003-029	-0.140542	B	PD
003-031	-0.097839	B	PD
003-036	-0.365135	B	PD
301-003	-0.287187	B	PD
301-004	-0.235351	B	PD
301-005	0.001004	B	CR/PR
301-013	-0.018753	B	CR/PR
301-022	0.238637	B	PD
301-039	-0.125333	C	CR/PR
301-044	-0.172339	C	PD

301-045	0.251858	C	PD
501-053	-0.165610	D	PD
501-058	-0.272957	C	PD
501-061	-0.268843	C	PD
502-062	-0.348191	D	CR/PR
502-063	-0.329929	C	PD

SAMPLE_ID	PFS	BENEFIT_STATUS	RESPONSE_GROUP	\
001-009	PD	Resistance		PD
001-015	CR/PR	Clinical Benefit		CR/PR
001-037	PD	Resistance		PD
001-038	PD	Resistance		PD
001-043	PD	Resistance		PD
001-047	CR/PR	Clinical Benefit		CR/PR
001-064	CR/PR	Clinical Benefit		CR/PR
001-076	PD	Resistance		PD
002-014	PD	Resistance		PD
002-021	PD	Resistance		PD
002-030	PD	Resistance		PD
002-064	PD	Resistance		PD
002-070	PD	Resistance		PD
002-074	PD	Resistance		PD
003-029	PD	Resistance		PD
003-031	PD	Resistance		PD
003-036	PD	Resistance		PD
301-003	PD	Resistance		PD
301-004	PD	Resistance		PD
301-005	CR/PR	Clinical Benefit		CR/PR
301-013	CR/PR	Clinical Benefit		CR/PR
301-022	PD	Resistance		PD
301-039	CR/PR	Clinical Benefit		CR/PR
301-044	PD	Resistance		PD
301-045	PD	Resistance		PD
501-053	PD	Resistance		PD
501-058	PD	Resistance		PD
501-061	PD	Resistance		PD
502-062	CR/PR	Clinical Benefit		CR/PR
502-063	PD	Resistance		PD

SAMPLE_ID	IDH1_STATUS	RNA_AVAILABLE
001-009	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-015	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-037	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-038	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-043	Unknown (Paper: n=9 Mutated, n=43 WT)	True

001-047	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-064	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-076	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-014	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-021	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-030	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-064	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-070	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-074	Unknown (Paper: n=9 Mutated, n=43 WT)	True
003-029	Unknown (Paper: n=9 Mutated, n=43 WT)	True
003-031	Unknown (Paper: n=9 Mutated, n=43 WT)	True
003-036	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-003	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-004	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-005	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-013	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-022	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-039	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-044	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-045	Unknown (Paper: n=9 Mutated, n=43 WT)	True
501-053	Unknown (Paper: n=9 Mutated, n=43 WT)	True
501-058	Unknown (Paper: n=9 Mutated, n=43 WT)	True
501-061	Unknown (Paper: n=9 Mutated, n=43 WT)	True
502-062	Unknown (Paper: n=9 Mutated, n=43 WT)	True
502-063	Unknown (Paper: n=9 Mutated, n=43 WT)	True

```
[573]: output_file = ("../results/GSE186332/selinexor_clear_response_cohort.xlsx")
```

```
with pd.ExcelWriter(
    output_file,
    engine="openpyxl"
) as writer:
    clear_response_summary.to_excel(
        writer,
        sheet_name="Summary",
        index=False
    )

    clear_response_patient_level.to_excel(
        writer,
        sheet_name="Patient_Level",
        index=True
    )
```

- selinexor_clear_response_cohort.xlsx exported to /results/GSE186332 directory

Summary Note: In the predefined clear-response cohort (7 CR/PR, 23 PD), the frozen Recurrence Score did not differ between CR/PR and PD groups (median -0.091)

vs -0.167; Mann-Whitney $p=0.264$; rank-biserial correlation=-0.886). ROC analysis showed no meaningful discrimination (AUC=0.522, 95% bootstrap CI 0.264-0.777).

9.6.2 6.2 Internal-validation group

```
[574]: gse186332_clinical_for_join = gse186332_selinexor_clinical_metadata.  
      ↪ set_index("PATIENT_ID").copy()  
assert gse186332_clinical_for_join.index.is_unique  
validation_analysis = gse186332_frozen_recurrence_scores.  
      ↪ join(gse186332_clinical_for_join, how="inner").copy()
```

```
[575]: validation_analysis =  
      ↪ validation_analysis[validation_analysis["EXPERIMENTAL_SET"] == "Validation"].  
      ↪ copy()  
validation_analysis = validation_analysis[validation_analysis["BENEFIT_STATUS"].  
      ↪ isin(["Clinical Benefit", "Resistance"])].copy()  
  
print(validation_analysis["BENEFIT_STATUS"].value_counts())  
  
assert validation_analysis["BENEFIT_STATUS"].value_counts().to_dict() ==  
      ↪ {"Clinical Benefit": 11, "Resistance": 11}  
  
validation_analysis["BENEFIT_BINARY"] = (validation_analysis["BENEFIT_STATUS"]  
      ↪ == "Clinical Benefit").astype(int)  
  
rs_benefit = validation_analysis.loc[validation_analysis["BENEFIT_BINARY"] ==  
      ↪ 1, "GSE186332_RECURRENCE_SCORE"]  
rs_resistance = validation_analysis.loc[validation_analysis["BENEFIT_BINARY"]  
      ↪ == 0, "GSE186332_RECURRENCE_SCORE"]  
  
N_BENEFIT, N_RESISTANCE = len(rs_benefit), len(rs_resistance)  
MEDIAN_RS_BENEFIT, MEDIAN_RS_RESISTANCE = rs_benefit.median(), rs_resistance.  
      ↪ median()
```

```
BENEFIT_STATUS  
Resistance          11  
Clinical Benefit    11  
Name: count, dtype: int64
```

```
[576]: mw = mannwhitneyu(rs_benefit, rs_resistance, alternative="two-sided")  
MANN_WHITNEY_U, MANN_WHITNEY_P = mw.statistic, mw.pvalue  
RANK_BISERIAL_CORRELATION = (2 * MANN_WHITNEY_U / (N_BENEFIT * N_RESISTANCE)) -  
      ↪ 1
```

```
[577]: y = validation_analysis["BENEFIT_BINARY"].to_numpy()  
scores = validation_analysis["GSE186332_RECURRENCE_SCORE"].to_numpy()  
ROC_AUC = roc_auc_score(y, scores)
```

```

rng = np.random.default_rng(42)

bootstrap_aucs = []

for _ in range(2000):
    idx = rng.integers(0, len(y), size=len(y))
    y_boot = y[idx]
    scores_boot = scores[idx]

    if len(np.unique(y_boot)) < 2:
        continue

    bootstrap_aucs.append(roc_auc_score(y_boot, scores_boot))

ROC_AUC_CI_LOWER = np.percentile(bootstrap_aucs, 2.5)
ROC_AUC_CI_UPPER = np.percentile(bootstrap_aucs, 97.5)

```

```

[578]: rs = validation_analysis["GSE186332_RECURRENCE_SCORE"]
validation_analysis["RS_Z"] = ((rs - rs.mean()) / rs.std(ddof=1))

X = sm.add_constant(validation_analysis["RS_Z"])
y = validation_analysis["BENEFIT_BINARY"]

logit_model = sm.Logit(y, X).fit(dis= False)
beta = logit_model.params["RS_Z"]
beta_ci = logit_model.conf_int().loc["RS_Z"]

OR_PER_1SD = np.exp(beta)

OR_CI_LOWER = np.exp(beta_ci[0])
OR_CI_UPPER = np.exp(beta_ci[1])

LOGISTIC_P_VALUE = (logit_model.pvalues["RS_Z"])

```

```

[579]: validation_summary = pd.DataFrame({
    "metric": [
        "N_BENEFIT",
        "N_RESISTANCE",
        "MEDIAN_RS_BENEFIT",
        "MEDIAN_RS_RESISTANCE",
        "MANN_WHITNEY_U",
        "MANN_WHITNEY_P",
        "RANK_BISERIAL_CORRELATION",
        "ROC_AUC",
        "ROC_AUC_CI_LOWER",
        "ROC_AUC_CI_UPPER",
        "OR_PER_1SD",
    ]
})

```



```

        "OR_CI_LOWER",
        "OR_CI_UPPER",
        "LOGISTIC_P_VALUE",
    ],
    "value": [
        N_BENEFIT,
        N_RESISTANCE,
        MEDIAN_RS_BENEFIT,
        MEDIAN_RS_RESISTANCE,
        MANN_WHITNEY_U,
        MANN_WHITNEY_P,
        RANK_BISERIAL_CORRELATION,
        ROC_AUC,
        ROC_AUC_CI_LOWER,
        ROC_AUC_CI_UPPER,
        OR_PER_1SD,
        OR_CI_LOWER,
        OR_CI_UPPER,
        LOGISTIC_P_VALUE,
    ],
})

```

[580]: validation_summary

```

[580]:
      metric      value
0      N_BENEFIT  11.000000
1      N_RESISTANCE  11.000000
2  MEDIAN_RS_BENEFIT  -0.214849
3  MEDIAN_RS_RESISTANCE  -0.145697
4      MANN_WHITNEY_U  43.000000
5      MANN_WHITNEY_P   0.264292
6  RANK_BISERIAL_CORRELATION  -0.289256
7           ROC_AUC   0.355372
8      ROC_AUC_CI_LOWER   0.132131
9      ROC_AUC_CI_UPPER   0.607173
10          OR_PER_1SD   0.539867
11          OR_CI_LOWER   0.213735
12          OR_CI_UPPER   1.363630
13      LOGISTIC_P_VALUE   0.192262

```

[581]: validation_analysis

```

[581]:
      SAMPLE_ID  UP_SCORE  DOWN_SCORE  GSE186332_RECURRENCE_SCORE  TREATMENT_ARM  \
001-046      0.260791    0.586478                    -0.325687             D
001-050      0.441750    0.466316                    -0.024566             A
001-055      0.574812    0.468945                     0.105867             D

```

001-071	0.250864	0.629391	-0.378527	D
001-078	0.242691	0.642719	-0.400028	D
002-018	0.282814	0.497662	-0.214849	B
002-026	0.429652	0.494545	-0.064894	B
002-042	0.369308	0.515005	-0.145697	D
002-066	0.301080	0.499993	-0.198913	D
002-067	0.602109	0.410542	0.191567	D
002-069	0.217555	0.597072	-0.379517	D
002-075	0.192285	0.533216	-0.340931	D
003-016	0.610159	0.467781	0.142378	B
003-019	0.372768	0.518957	-0.146189	B
003-023	0.229274	0.425047	-0.195774	B
003-024	0.272618	0.562390	-0.289772	B
301-001	0.539479	0.385701	0.153778	B
301-025	0.562089	0.385288	0.176801	B
501-049	0.292119	0.418810	-0.126691	C
501-054	0.480939	0.501689	-0.020749	C
501-059	0.199050	0.652157	-0.453108	D
502-052	0.184569	0.648551	-0.463983	D

BEST_OVERALL_RESPONSE

PFS \

SAMPLE_ID

001-046	non-durable SD / PD	non-durable SD (<100d) / PD
001-050	durable SD	durable SD (>140d / PFS6+)
001-055	non-durable SD / PD	non-durable SD (<100d) / PD
001-071	durable SD	durable SD (>140d / PFS6+)
001-078	non-durable SD / PD	non-durable SD (<100d) / PD
002-018	durable SD	durable SD (>140d / PFS6+)
002-026	non-durable SD / PD	non-durable SD (<100d) / PD
002-042	non-durable SD / PD	non-durable SD (<100d) / PD
002-066	non-durable SD / PD	non-durable SD (<100d) / PD
002-067	durable SD	durable SD (>140d / PFS6+)
002-069	durable SD	durable SD (>140d / PFS6+)
002-075	non-durable SD / PD	non-durable SD (<100d) / PD
003-016	non-durable SD / PD	non-durable SD (<100d) / PD
003-019	non-durable SD / PD	non-durable SD (<100d) / PD
003-023	durable SD	durable SD (>140d / PFS6+)
003-024	durable SD	durable SD (>140d / PFS6+)
301-001	non-durable SD / PD	non-durable SD (<100d) / PD
301-025	non-durable SD / PD	non-durable SD (<100d) / PD
501-049	durable SD	durable SD (>140d / PFS6+)
501-054	durable SD	durable SD (>140d / PFS6+)
501-059	durable SD	durable SD (>140d / PFS6+)
502-052	durable SD	durable SD (>140d / PFS6+)

BENEFIT_STATUS

RESPONSE_GROUP EXPERIMENTAL_SET \

SAMPLE_ID

001-046	Resistance	non-durable SD / PD	Validation
001-050	Clinical Benefit	durable SD	Validation
001-055	Resistance	non-durable SD / PD	Validation
001-071	Clinical Benefit	durable SD	Validation
001-078	Resistance	non-durable SD / PD	Validation
002-018	Clinical Benefit	durable SD	Validation
002-026	Resistance	non-durable SD / PD	Validation
002-042	Resistance	non-durable SD / PD	Validation
002-066	Resistance	non-durable SD / PD	Validation
002-067	Clinical Benefit	durable SD	Validation
002-069	Clinical Benefit	durable SD	Validation
002-075	Resistance	non-durable SD / PD	Validation
003-016	Resistance	non-durable SD / PD	Validation
003-019	Resistance	non-durable SD / PD	Validation
003-023	Clinical Benefit	durable SD	Validation
003-024	Clinical Benefit	durable SD	Validation
301-001	Resistance	non-durable SD / PD	Validation
301-025	Resistance	non-durable SD / PD	Validation
501-049	Clinical Benefit	durable SD	Validation
501-054	Clinical Benefit	durable SD	Validation
501-059	Clinical Benefit	durable SD	Validation
502-052	Clinical Benefit	durable SD	Validation

	IDH1_STATUS	RNA_AVAILABLE \
SAMPLE_ID		
001-046	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-050	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-055	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-071	Unknown (Paper: n=9 Mutated, n=43 WT)	True
001-078	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-018	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-026	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-042	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-066	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-067	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-069	Unknown (Paper: n=9 Mutated, n=43 WT)	True
002-075	Unknown (Paper: n=9 Mutated, n=43 WT)	True
003-016	Unknown (Paper: n=9 Mutated, n=43 WT)	True
003-019	Unknown (Paper: n=9 Mutated, n=43 WT)	True
003-023	Unknown (Paper: n=9 Mutated, n=43 WT)	True
003-024	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-001	Unknown (Paper: n=9 Mutated, n=43 WT)	True
301-025	Unknown (Paper: n=9 Mutated, n=43 WT)	True
501-049	Unknown (Paper: n=9 Mutated, n=43 WT)	True
501-054	Unknown (Paper: n=9 Mutated, n=43 WT)	True
501-059	Unknown (Paper: n=9 Mutated, n=43 WT)	True
502-052	Unknown (Paper: n=9 Mutated, n=43 WT)	True

SAMPLE_ID	BENEFIT_BINARY	RS_Z
001-046	0	-0.800579
001-050	1	0.607835
001-055	0	1.217898
001-071	1	-1.047723
001-078	0	-1.148288
002-018	1	-0.282161
002-026	0	0.419213
002-042	0	0.041277
002-066	0	-0.207626
002-067	1	1.618738
002-069	1	-1.052353
002-075	0	-0.871880
003-016	0	1.388669
003-019	0	0.038974
003-023	1	-0.192943
003-024	1	-0.632595
301-001	0	1.441989
301-025	0	1.549671
501-049	1	0.130172
501-054	1	0.625684
501-059	1	-1.396553
502-052	1	-1.447418

```
[582]: validation_patient_level = validation_analysis[
    [
        "GSE186332_RECURRENCE_SCORE",
        "TREATMENT_ARM",
        "BEST_OVERALL_RESPONSE",
        "PFS",
        "BENEFIT_STATUS",
        "RESPONSE_GROUP",
        "IDH1_STATUS",
        "RNA_AVAILABLE",
        "RS_Z",
    ]
].copy()
```

```
[583]: validation_patient_level
```

```
[583]: GSE186332_RECURRENCE_SCORE TREATMENT_ARM BEST_OVERALL_RESPONSE \
SAMPLE_ID
001-046 -0.325687 D non-durable SD / PD
001-050 -0.024566 A durable SD
001-055 0.105867 D non-durable SD / PD
```

001-071	-0.378527	D	durable SD
001-078	-0.400028	D	non-durable SD / PD
002-018	-0.214849	B	durable SD
002-026	-0.064894	B	non-durable SD / PD
002-042	-0.145697	D	non-durable SD / PD
002-066	-0.198913	D	non-durable SD / PD
002-067	0.191567	D	durable SD
002-069	-0.379517	D	durable SD
002-075	-0.340931	D	non-durable SD / PD
003-016	0.142378	B	non-durable SD / PD
003-019	-0.146189	B	non-durable SD / PD
003-023	-0.195774	B	durable SD
003-024	-0.289772	B	durable SD
301-001	0.153778	B	non-durable SD / PD
301-025	0.176801	B	non-durable SD / PD
501-049	-0.126691	C	durable SD
501-054	-0.020749	C	durable SD
501-059	-0.453108	D	durable SD
502-052	-0.463983	D	durable SD

	PFS	BENEFIT_STATUS	RESPONSE_GROUP \
SAMPLE_ID			
001-046	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
001-050	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
001-055	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
001-071	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
001-078	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
002-018	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
002-026	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
002-042	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
002-066	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
002-067	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
002-069	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
002-075	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
003-016	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
003-019	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
003-023	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
003-024	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
301-001	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
301-025	non-durable SD (<100d) / PD	Resistance	non-durable SD / PD
501-049	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
501-054	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
501-059	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD
502-052	durable SD (>140d / PFS6+)	Clinical Benefit	durable SD

	IDH1_STATUS	RNA_AVAILABLE	RS_Z
SAMPLE_ID			

001-046	Unknown (Paper: n=9 Mutated, n=43 WT)	True -0.800579
001-050	Unknown (Paper: n=9 Mutated, n=43 WT)	True 0.607835
001-055	Unknown (Paper: n=9 Mutated, n=43 WT)	True 1.217898
001-071	Unknown (Paper: n=9 Mutated, n=43 WT)	True -1.047723
001-078	Unknown (Paper: n=9 Mutated, n=43 WT)	True -1.148288
002-018	Unknown (Paper: n=9 Mutated, n=43 WT)	True -0.282161
002-026	Unknown (Paper: n=9 Mutated, n=43 WT)	True 0.419213
002-042	Unknown (Paper: n=9 Mutated, n=43 WT)	True 0.041277
002-066	Unknown (Paper: n=9 Mutated, n=43 WT)	True -0.207626
002-067	Unknown (Paper: n=9 Mutated, n=43 WT)	True 1.618738
002-069	Unknown (Paper: n=9 Mutated, n=43 WT)	True -1.052353
002-075	Unknown (Paper: n=9 Mutated, n=43 WT)	True -0.871880
003-016	Unknown (Paper: n=9 Mutated, n=43 WT)	True 1.388669
003-019	Unknown (Paper: n=9 Mutated, n=43 WT)	True 0.038974
003-023	Unknown (Paper: n=9 Mutated, n=43 WT)	True -0.192943
003-024	Unknown (Paper: n=9 Mutated, n=43 WT)	True -0.632595
301-001	Unknown (Paper: n=9 Mutated, n=43 WT)	True 1.441989
301-025	Unknown (Paper: n=9 Mutated, n=43 WT)	True 1.549671
501-049	Unknown (Paper: n=9 Mutated, n=43 WT)	True 0.130172
501-054	Unknown (Paper: n=9 Mutated, n=43 WT)	True 0.625684
501-059	Unknown (Paper: n=9 Mutated, n=43 WT)	True -1.396553
502-052	Unknown (Paper: n=9 Mutated, n=43 WT)	True -1.447418

```
[584]: output_file = "../../../results/GSE186332/selinexor_predefined_subgroup_validation.
        ↪xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    validation_summary.to_excel(writer, sheet_name="Summary", index=False)
    validation_patient_level.to_excel(writer, sheet_name="Patient_Level",
    ↪index=True)
```

- selinexor_predefined_subgroup_validation.xlsx exported to /results/GSE186332 directory

Summary Note: In the predefined internal-validation cohort (11 clinical benefit, 11 resistance), the frozen Recurrence Score did not show a statistically significant association with clinical benefit (median -0.215 vs -0.146 ; Mann-Whitney $p=0.264$; rank-biserial correlation= -0.289). ROC analysis showed limited discrimination (AUC= 0.355 , 95% bootstrap CI 0.132 – 0.607). Logistic regression estimated an OR of 0.540 per 1-SD increase in Recurrence Score (95% CI 0.214 – 1.364 ; $p=0.192$).

9.7 7. IDH-wildtype sensitivity analysis

The Recurrence Signature was derived from IDH-wildtype GBM, therefore an IDH-wildtype-only sensitivity analysis was planned for the GSE186332 selinexor cohort.

However, patient-level IDH1 mutation status is not available in the reconstructed GSE186332 clinical metadata. The publication reports only cohort-level information (9 IDH1-mutated and 43 IDH1-wildtype tumours), which does not identify which individual samples belong to each IDH

group.

Therefore, an IDH-wildtype-only sensitivity analysis cannot be performed reliably without introducing unsupported assumptions about individual patient IDH status.

Status: Not performed / not estimable from the available patient-level data.

No samples were excluded based on inferred IDH status, and no IDH-wildtype-specific statistical results were generated.

If patient-level IDH1 status becomes available, the predefined sensitivity analysis should be repeated using:

- group medians;
- Mann–Whitney U test;
- rank-biserial effect size;
- ROC AUC.

The analysis should remain a **sensitivity analysis** and should not replace the primary clinical analysis.

9.8 8. Proliferation confounding analysis in selinexor cohort

```
[585]: target_pathways = ["E2F Targets", "G2-M Checkpoint"]
prolif_df = hallmark_results[hallmark_results["PATHWAY"].isin(target_pathways)]

proliferation_genes = set()
for raw_str in prolif_df["LEADING_EDGE_GENES"]:
    genes = [g.strip() for g in str(raw_str).replace(",", ";").split(";") if g.
↳strip()]
    proliferation_genes.update(genes)

valid_prolif_mapping = unambiguous_glass_mapping.
↳reindex(sorted(proliferation_genes)).dropna()
valid_prolif_ensembl = set(valid_prolif_mapping.values) & set(gse_log2_tpm.
↳index)

print("Total proliferation genes:", len(proliferation_genes))
print("Mapped to GSE Ensembl:", len(valid_prolif_ensembl))

prolif_expr = gse_log2_tpm.loc[sorted(valid_prolif_ensembl)].copy()
prolif_z = prolif_expr.sub(prolif_expr.mean(axis=1), axis=0).div(prolif_expr.
↳std(axis=1, ddof=1), axis=0)
gse_proliferation_score = prolif_z.mean(axis=0)
gse_proliferation_score.name = "PROLIFERATION_SCORE"
```

Total proliferation genes: 217

Mapped to GSE Ensembl: 215

```
[586]: gse186332_clinical_for_join = gse186332_selinexor_clinical_metadata.
        ↪set_index("PATIENT_ID").copy()
        assert gse186332_clinical_for_join.index.is_unique

        prolif_analysis = gse186332_frozen_recurrence_scores.
        ↪join(gse_proliferation_score, how="inner").join(gse186332_clinical_for_join,
        ↪how="inner").copy()
        prolif_analysis = prolif_analysis[prolif_analysis["BENEFIT_STATUS"].
        ↪isin(["Clinical Benefit", "Resistance"])] .copy()
```

```
[587]: spearman_rho, spearman_p =
        ↪spearmanr(prolif_analysis["GSE186332_RECURRENCE_SCORE"],
        ↪prolif_analysis["PROLIFERATION_SCORE"])
        rs = prolif_analysis["GSE186332_RECURRENCE_SCORE"]
        prolif_analysis["RS_Z"] = (rs - rs.mean()) / rs.std(ddof=1)
        prolif_analysis["BENEFIT_BINARY"] = (prolif_analysis["BENEFIT_STATUS"] ==
        ↪"Clinical Benefit").astype(int)
        X = sm.add_constant(prolif_analysis[["RS_Z", "PROLIFERATION_SCORE"]].copy())
        y = prolif_analysis["BENEFIT_BINARY"]
        logit_model = sm.Logit(y, X).fit(dis= False)
```

```
[588]: model_results = pd.DataFrame({
        "term": logit_model.params.index,
        "BETA": logit_model.params.values,
        "OR": np.exp(logit_model.params.values),
        "CI_LOWER": np.exp(logit_model.conf_int()[0].values),
        "CI_UPPER": np.exp(logit_model.conf_int()[1].values),
        "P_VALUE": logit_model.pvalues.values,
    })
```

```
[589]: sensitivity_summary = pd.DataFrame({
        "metric": [
            "N_SAMPLES",
            "N_BENEFIT",
            "N_RESISTANCE",
            "N_PROLIFERATION_GENES",
            "N_PROLIFERATION_GENES_MAPPED",
            "SPEARMAN_RHO_RS_PROLIFERATION",
            "SPEARMAN_P_RS_PROLIFERATION",
            "ADJUSTED_RS_OR_PER_1SD",
            "ADJUSTED_RS_CI_LOWER",
            "ADJUSTED_RS_CI_UPPER",
            "ADJUSTED_RS_P_VALUE",
            "PROLIFERATION_OR",
            "PROLIFERATION_CI_LOWER",
            "PROLIFERATION_CI_UPPER",
            "PROLIFERATION_P_VALUE",
        ]
    })
```



```

],
"value": [
    len(prolif_analysis),
    (prolif_analysis["BENEFIT_BINARY"] == 1).sum(),
    (prolif_analysis["BENEFIT_BINARY"] == 0).sum(),
    len(proliferation_genes),
    len(valid_prolif_ensembl),
    spearman_rho,
    spearman_p,
    np.exp(logit_model.params["RS_Z"]),
    np.exp(logit_model.conf_int().loc["RS_Z", 0]),
    np.exp(logit_model.conf_int().loc["RS_Z", 1]),
    logit_model.pvalues["RS_Z"],
    np.exp(logit_model.params["PROLIFERATION_SCORE"]),
    np.exp(logit_model.conf_int().loc["PROLIFERATION_SCORE", 0]),
    np.exp(logit_model.conf_int().loc["PROLIFERATION_SCORE", 1]),
    logit_model.pvalues["PROLIFERATION_SCORE"],
]
})

sensitivity_summary

```

```

[589]:

```

	metric	value
0	N_SAMPLES	52.000000
1	N_BENEFIT	18.000000
2	N_RESISTANCE	34.000000
3	N_PROLIFERATION_GENES	217.000000
4	N_PROLIFERATION_GENES_MAPPED	215.000000
5	SPEARMAN_RHO_RS_PROLIFERATION	-0.596175
6	SPEARMAN_P_RS_PROLIFERATION	0.000003
7	ADJUSTED_RS_OR_PER_1SD	0.265792
8	ADJUSTED_RS_CI_LOWER	0.101701
9	ADJUSTED_RS_CI_UPPER	0.694640
10	ADJUSTED_RS_P_VALUE	0.006865
11	PROLIFERATION_OR	0.070469
12	PROLIFERATION_CI_LOWER	0.012036
13	PROLIFERATION_CI_UPPER	0.412600
14	PROLIFERATION_P_VALUE	0.003264

```

[590]: patient_level = prolif_analysis[
    [
        "GSE186332_RECURRENCE_SCORE",
        "PROLIFERATION_SCORE",
        "RS_Z",
        "TREATMENT_ARM",
        "BEST_OVERALL_RESPONSE",
        "PFS",
    ]
]

```

```

        "BENEFIT_STATUS",
        "RESPONSE_GROUP",
        "IDH1_STATUS",
    ]
].copy()

patient_level

```

```

[590]:
      GSE186332_RECURRENCE_SCORE  PROLIFERATION_SCORE    RS_Z  \
SAMPLE_ID
001-009             0.303569             -0.485815    1.890328
001-015            -0.090535             -1.437722    0.153602
001-037            -0.000638              0.179187    0.549756
001-038             0.349085             -0.612808    2.090907
001-043            -0.254054              0.398856   -0.566988
001-046            -0.325687              0.793878   -0.882659
001-047             0.190930             -1.009661    1.393955
001-050            -0.024566             -0.574646    0.444314
001-055             0.105867              0.065906    1.019099
001-064            -0.362106             -0.302943   -1.043150
001-071            -0.378527             -0.216133   -1.115512
001-076            -0.501270              0.901593   -1.656413
001-078            -0.400028              0.554772   -1.210262
002-014            -0.166923              0.377807   -0.183023
002-018            -0.214849              1.819470   -0.394219
002-021            -0.385560              0.199834   -1.146503
002-026            -0.064894              0.475732    0.266598
002-030             0.053960              0.729422    0.790359
002-042            -0.145697              0.569191   -0.089484
002-064             0.285845             -0.525900    1.812222
002-066            -0.198913              0.346214   -0.323994
002-067             0.191567             -1.546183    1.396761
002-069            -0.379517              0.169653   -1.119875
002-070             0.119757             -0.226810    1.080311
002-074            -0.326396              0.211829   -0.885782
002-075            -0.340931             -0.168949   -0.949837
003-016             0.142378             -0.368276    1.179995
003-019            -0.146189             -0.029686   -0.091653
003-023            -0.195774             -0.000662   -0.310160
003-024            -0.289772              0.496116   -0.724388
003-029            -0.140542              0.235188   -0.066767
003-031            -0.097839              0.303867    0.121415
003-036            -0.365135              0.415701   -1.056496
301-001             0.153778             -0.882181    1.230233
301-003            -0.287187              0.621514   -0.712997
301-004            -0.235351              0.627173   -0.484570
301-005             0.001004             -0.720308    0.556993

```

301-013	-0.018753	-0.117882	0.469927
301-022	0.238637	-0.364682	1.604185
301-025	0.176801	-0.653013	1.331688
301-039	-0.125333	-0.588704	0.000256
301-044	-0.172339	1.332979	-0.206889
301-045	0.251858	-0.456210	1.662446
501-049	-0.126691	-0.309909	-0.005729
501-053	-0.165610	0.074261	-0.177236
501-054	-0.020749	-0.112033	0.461131
501-058	-0.272957	0.411334	-0.650288
501-059	-0.453108	0.819050	-1.444171
501-061	-0.268843	0.154432	-0.632161
502-052	-0.463983	-0.097875	-1.492095
502-062	-0.348191	-0.161334	-0.981830
502-063	-0.329929	0.397953	-0.901351

SAMPLE_ID	TREATMENT_ARM	BEST_OVERALL_RESPONSE	PFS \
001-009	A	PD	PD
001-015	A	CR/PR	CR/PR
001-037	C	PD	PD
001-038	D	PD	PD
001-043	D	PD	PD
001-046	D	non-durable SD / PD	non-durable SD (<100d) / PD
001-047	D	CR/PR	CR/PR
001-050	A	durable SD	durable SD (>140d / PFS6+)
001-055	D	non-durable SD / PD	non-durable SD (<100d) / PD
001-064	D	CR/PR	CR/PR
001-071	D	durable SD	durable SD (>140d / PFS6+)
001-076	D	PD	PD
001-078	D	non-durable SD / PD	non-durable SD (<100d) / PD
002-014	B	PD	PD
002-018	B	durable SD	durable SD (>140d / PFS6+)
002-021	B	PD	PD
002-026	B	non-durable SD / PD	non-durable SD (<100d) / PD
002-030	B	PD	PD
002-042	D	non-durable SD / PD	non-durable SD (<100d) / PD
002-064	D	PD	PD
002-066	D	non-durable SD / PD	non-durable SD (<100d) / PD
002-067	D	durable SD	durable SD (>140d / PFS6+)
002-069	D	durable SD	durable SD (>140d / PFS6+)
002-070	D	PD	PD
002-074	D	PD	PD
002-075	D	non-durable SD / PD	non-durable SD (<100d) / PD
003-016	B	non-durable SD / PD	non-durable SD (<100d) / PD
003-019	B	non-durable SD / PD	non-durable SD (<100d) / PD
003-023	B	durable SD	durable SD (>140d / PFS6+)

003-024	B	durable SD	durable SD (>140d / PFS6+)
003-029	B	PD	PD
003-031	B	PD	PD
003-036	B	PD	PD
301-001	B	non-durable SD / PD	non-durable SD (<100d) / PD
301-003	B	PD	PD
301-004	B	PD	PD
301-005	B	CR/PR	CR/PR
301-013	B	CR/PR	CR/PR
301-022	B	PD	PD
301-025	B	non-durable SD / PD	non-durable SD (<100d) / PD
301-039	C	CR/PR	CR/PR
301-044	C	PD	PD
301-045	C	PD	PD
501-049	C	durable SD	durable SD (>140d / PFS6+)
501-053	D	PD	PD
501-054	C	durable SD	durable SD (>140d / PFS6+)
501-058	C	PD	PD
501-059	D	durable SD	durable SD (>140d / PFS6+)
501-061	C	PD	PD
502-052	D	durable SD	durable SD (>140d / PFS6+)
502-062	D	CR/PR	CR/PR
502-063	C	PD	PD

SAMPLE_ID	BENEFIT_STATUS	RESPONSE_GROUP \
001-009	Resistance	PD
001-015	Clinical Benefit	CR/PR
001-037	Resistance	PD
001-038	Resistance	PD
001-043	Resistance	PD
001-046	Resistance	non-durable SD / PD
001-047	Clinical Benefit	CR/PR
001-050	Clinical Benefit	durable SD
001-055	Resistance	non-durable SD / PD
001-064	Clinical Benefit	CR/PR
001-071	Clinical Benefit	durable SD
001-076	Resistance	PD
001-078	Resistance	non-durable SD / PD
002-014	Resistance	PD
002-018	Clinical Benefit	durable SD
002-021	Resistance	PD
002-026	Resistance	non-durable SD / PD
002-030	Resistance	PD
002-042	Resistance	non-durable SD / PD
002-064	Resistance	PD
002-066	Resistance	non-durable SD / PD

002-067	Clinical Benefit	durable SD
002-069	Clinical Benefit	durable SD
002-070	Resistance	PD
002-074	Resistance	PD
002-075	Resistance	non-durable SD / PD
003-016	Resistance	non-durable SD / PD
003-019	Resistance	non-durable SD / PD
003-023	Clinical Benefit	durable SD
003-024	Clinical Benefit	durable SD
003-029	Resistance	PD
003-031	Resistance	PD
003-036	Resistance	PD
301-001	Resistance	non-durable SD / PD
301-003	Resistance	PD
301-004	Resistance	PD
301-005	Clinical Benefit	CR/PR
301-013	Clinical Benefit	CR/PR
301-022	Resistance	PD
301-025	Resistance	non-durable SD / PD
301-039	Clinical Benefit	CR/PR
301-044	Resistance	PD
301-045	Resistance	PD
501-049	Clinical Benefit	durable SD
501-053	Resistance	PD
501-054	Clinical Benefit	durable SD
501-058	Resistance	PD
501-059	Clinical Benefit	durable SD
501-061	Resistance	PD
502-052	Clinical Benefit	durable SD
502-062	Clinical Benefit	CR/PR
502-063	Resistance	PD

IDH1_STATUS

SAMPLE_ID	
001-009	Unknown (Paper: n=9 Mutated, n=43 WT)
001-015	Unknown (Paper: n=9 Mutated, n=43 WT)
001-037	Unknown (Paper: n=9 Mutated, n=43 WT)
001-038	Unknown (Paper: n=9 Mutated, n=43 WT)
001-043	Unknown (Paper: n=9 Mutated, n=43 WT)
001-046	Unknown (Paper: n=9 Mutated, n=43 WT)
001-047	Unknown (Paper: n=9 Mutated, n=43 WT)
001-050	Unknown (Paper: n=9 Mutated, n=43 WT)
001-055	Unknown (Paper: n=9 Mutated, n=43 WT)
001-064	Unknown (Paper: n=9 Mutated, n=43 WT)
001-071	Unknown (Paper: n=9 Mutated, n=43 WT)
001-076	Unknown (Paper: n=9 Mutated, n=43 WT)
001-078	Unknown (Paper: n=9 Mutated, n=43 WT)

002-014 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-018 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-021 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-026 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-030 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-042 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-064 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-066 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-067 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-069 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-070 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-074 Unknown (Paper: n=9 Mutated, n=43 WT)
 002-075 Unknown (Paper: n=9 Mutated, n=43 WT)
 003-016 Unknown (Paper: n=9 Mutated, n=43 WT)
 003-019 Unknown (Paper: n=9 Mutated, n=43 WT)
 003-023 Unknown (Paper: n=9 Mutated, n=43 WT)
 003-024 Unknown (Paper: n=9 Mutated, n=43 WT)
 003-029 Unknown (Paper: n=9 Mutated, n=43 WT)
 003-031 Unknown (Paper: n=9 Mutated, n=43 WT)
 003-036 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-001 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-003 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-004 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-005 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-013 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-022 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-025 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-039 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-044 Unknown (Paper: n=9 Mutated, n=43 WT)
 301-045 Unknown (Paper: n=9 Mutated, n=43 WT)
 501-049 Unknown (Paper: n=9 Mutated, n=43 WT)
 501-053 Unknown (Paper: n=9 Mutated, n=43 WT)
 501-054 Unknown (Paper: n=9 Mutated, n=43 WT)
 501-058 Unknown (Paper: n=9 Mutated, n=43 WT)
 501-059 Unknown (Paper: n=9 Mutated, n=43 WT)
 501-061 Unknown (Paper: n=9 Mutated, n=43 WT)
 502-052 Unknown (Paper: n=9 Mutated, n=43 WT)
 502-062 Unknown (Paper: n=9 Mutated, n=43 WT)
 502-063 Unknown (Paper: n=9 Mutated, n=43 WT)

[591]: model_results

	term	BETA	OR	CI_LOWER	CI_UPPER	P_VALUE
0	const	-0.766645	0.464569	0.235447	0.91666	0.027042
1	RS_Z	-1.325041	0.265792	0.101701	0.69464	0.006865
2	PROLIFERATION_SCORE	-2.652582	0.070469	0.012036	0.41260	0.003264

```
[592]: output_file = "../results/GSE186332/
↳selinexor_proliferation_sensitivity_analysis.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    sensitivity_summary.to_excel(writer, sheet_name="Summary", index=False)
    model_results.to_excel(writer, sheet_name="Logistic_Regression",
↳index=False)
    patient_level.to_excel(writer, sheet_name="Patient_Level", index=True)
```

- selinexor_proliferation_sensitivity_analysis.xlsx exported to
/results/GSE186332 directory

```
[593]: sensitivity_summary
```

```
[593]:
```

	metric	value
0	N_SAMPLES	52.000000
1	N_BENEFIT	18.000000
2	N_RESISTANCE	34.000000
3	N_PROLIFERATION_GENES	217.000000
4	N_PROLIFERATION_GENES_MAPPED	215.000000
5	SPEARMAN_RHO_RS_PROLIFERATION	-0.596175
6	SPEARMAN_P_RS_PROLIFERATION	0.000003
7	ADJUSTED_RS_OR_PER_1SD	0.265792
8	ADJUSTED_RS_CI_LOWER	0.101701
9	ADJUSTED_RS_CI_UPPER	0.694640
10	ADJUSTED_RS_P_VALUE	0.006865
11	PROLIFERATION_OR	0.070469
12	PROLIFERATION_CI_LOWER	0.012036
13	PROLIFERATION_CI_UPPER	0.412600
14	PROLIFERATION_P_VALUE	0.003264

- selinexor_proliferation_sensitivity_analysis.xlsx exported to
/results/GSE186332 directory

Summary Note: In the 52-patient evaluable cohort, the frozen Recurrence Score was strongly inversely correlated with the proliferation score (Spearman $= -0.596$, $p=3 \times 10^{-5}$). After adjustment for proliferation, the Recurrence Score remained significantly associated with clinical benefit (OR per 1-SD=0.266, 95% CI 0.102–0.695, $p=0.0069$), while the proliferation score was also independently associated with benefit (OR=0.070, 95% CI 0.012–0.413, $p=0.0033$). Of 217 proliferation genes, 215 mapped to the GSE Ensembl gene universe.

9.9 9. Treatment-arm sensitivity analysis

```
[594]: gse186332_clinical_for_join = gse186332_selinexor_clinical_metadata.
↳set_index("PATIENT_ID").copy()
assert gse186332_clinical_for_join.index.is_unique
```

```
treatment_analysis = gse186332_frozen_recurrence_scores.  
    ↪join(gse186332_clinical_for_join, how="inner").copy()
```

```
[595]: treatment_analysis = treatment_analysis[treatment_analysis["BENEFIT_STATUS"].  
    ↪isin(["Clinical Benefit", "Resistance"])]
```

```
[596]: arm_counts = treatment_analysis["TREATMENT_ARM"].value_counts().sort_index()  
  
print("Treatment-arm counts:")  
print(arm_counts)  
  
assert treatment_analysis["TREATMENT_ARM"].notna().all()
```

Treatment-arm counts:

TREATMENT_ARM

```
A      3  
B     19  
C      9  
D     21
```

Name: count, dtype: int64

```
[597]: arm_score_summary = treatment_analysis.  
    ↪groupby("TREATMENT_ARM")["GSE186332_RECURRENCE_SCORE"].agg(  
        N="count", MEDIAN="median", MEAN="mean", SD="std", MIN="min", MAX="max"  
    ).reset_index()  
  
arm_score_summary
```

```
[597]:
```

	TREATMENT_ARM	N	MEDIAN	MEAN	SD	MIN	MAX
0	A	3	-0.024566	0.062823	0.211086	-0.090535	0.303569
1	B	19	-0.140542	-0.096958	0.186762	-0.385560	0.238637
2	C	9	-0.126691	-0.118402	0.178461	-0.329929	0.251858
3	D	21	-0.325687	-0.180998	0.270875	-0.501270	0.349085

```
[598]: arm_groups = [group["GSE186332_RECURRENCE_SCORE"].to_numpy() for _, group in  
    ↪treatment_analysis.groupby("TREATMENT_ARM")]  
valid_arm_groups = [x for x in arm_groups if len(x) >= 2]  
  
if len(valid_arm_groups) >= 2:  
    kruskal_result = kruskal(*valid_arm_groups)  
    KRUSKAL_H, KRUSKAL_P = kruskal_result.statistic, kruskal_result.pvalue  
else:  
    KRUSKAL_H, KRUSKAL_P = np.nan, np.nan
```

```
[599]: # Only exploratory!  
treatment_analysis["BENEFIT_BINARY"] = (treatment_analysis["BENEFIT_STATUS"] ==  
    ↪"Clinical Benefit").astype(int)
```



```

rs = treatment_analysis["GSE186332_RECURRENCE_SCORE"]
treatment_analysis["RS_Z"] = (rs - rs.mean()) / rs.std(ddof=1)

arm_dummies = pd.get_dummies(treatment_analysis["TREATMENT_ARM"], prefix="ARM",
    ↪drop_first=True, dtype=float)
X = sm.add_constant(pd.concat([treatment_analysis[["RS_Z"]], arm_dummies],
    ↪axis=1))
y = treatment_analysis["BENEFIT_BINARY"]

print("\nModel variables:")
print(X.columns.tolist())

logit_model = sm.Logit(y, X).fit(dis=False)

```

Model variables:

```
['const', 'RS_Z', 'ARM_B', 'ARM_C', 'ARM_D']
```

```

[600]: logistic_results = pd.DataFrame({
    "term": logit_model.params.index,
    "BETA": logit_model.params.values,
    "OR": np.exp(logit_model.params.values),
    "CI_LOWER": np.exp(logit_model.conf_int()[0].values),
    "CI_UPPER": np.exp(logit_model.conf_int()[1].values),
    "P_VALUE": logit_model.pvalues.values,
})
logistic_results

```

```

[600]:      term      BETA      OR  CI_LOWER  CI_UPPER  P_VALUE
0  const  1.059261  2.884239  0.236623  35.156496  0.406390
1   RS_Z -0.422156  0.655632  0.340219   1.263458  0.207208
2  ARM_B -2.063077  0.127062  0.008493   1.900918  0.135015
3  ARM_C -1.756790  0.172598  0.009758   3.052909  0.230710
4  ARM_D -1.682128  0.185978  0.012103   2.857780  0.227548

```

```

[601]: RS_ADJUSTED_OR_PER_1SD = np.exp(logit_model.params["RS_Z"])
RS_ADJUSTED_CI_LOWER = np.exp(logit_model.conf_int().loc["RS_Z", 0])
RS_ADJUSTED_CI_UPPER = np.exp(logit_model.conf_int().loc["RS_Z", 1])
RS_ADJUSTED_P_VALUE = (logit_model.pvalues["RS_Z"])

```

```

[602]: treatment_summary = pd.DataFrame({
    "metric": [
        "N_SAMPLES",
        "N_BENEFIT",
        "N_RESISTANCE",
        "N_TREATMENT_ARMS",
        "KRUSKAL_WALLIS_H",
        "KRUSKAL_WALLIS_P",
    ]
})

```

```

        "RS_ADJUSTED_OR_PER_1SD",
        "RS_ADJUSTED_CI_LOWER",
        "RS_ADJUSTED_CI_UPPER",
        "RS_ADJUSTED_P_VALUE",
        "REFERENCE_TREATMENT_ARM",
    ],
    "value": [
        len(treatment_analysis),
        (treatment_analysis["BENEFIT_BINARY"] == 1).sum(),
        (treatment_analysis["BENEFIT_BINARY"] == 0).sum(),
        treatment_analysis["TREATMENT_ARM"].nunique(),
        KRUSKAL_H,
        KRUSKAL_P,
        RS_ADJUSTED_OR_PER_1SD,
        RS_ADJUSTED_CI_LOWER,
        RS_ADJUSTED_CI_UPPER,
        RS_ADJUSTED_P_VALUE,
        sorted(treatment_analysis["TREATMENT_ARM"].unique())[0],
    ]
})

treatment_summary

```

```

[602]:

```

	metric	value
0	N_SAMPLES	52
1	N_BENEFIT	18
2	N_RESISTANCE	34
3	N_TREATMENT_ARMS	4
4	KRUSKAL_WALLIS_H	5.134356
5	KRUSKAL_WALLIS_P	0.162219
6	RS_ADJUSTED_OR_PER_1SD	0.655632
7	RS_ADJUSTED_CI_LOWER	0.340219
8	RS_ADJUSTED_CI_UPPER	1.263458
9	RS_ADJUSTED_P_VALUE	0.207208
10	REFERENCE_TREATMENT_ARM	A

```

[603]: patient_level = treatment_analysis[
    [
        "GSE186332_RECURRENCE_SCORE",
        "RS_Z",
        "TREATMENT_ARM",
        "BEST_OVERALL_RESPONSE",
        "PFS",
        "BENEFIT_STATUS",
        "RESPONSE_GROUP",
    ]
].copy()

```

patient_level

[603] :

SAMPLE_ID	GSE186332_RECURRENCE_SCORE	RS_Z	TREATMENT_ARM \
001-009	0.303569	1.890328	A
001-015	-0.090535	0.153602	A
001-037	-0.000638	0.549756	C
001-038	0.349085	2.090907	D
001-043	-0.254054	-0.566988	D
001-046	-0.325687	-0.882659	D
001-047	0.190930	1.393955	D
001-050	-0.024566	0.444314	A
001-055	0.105867	1.019099	D
001-064	-0.362106	-1.043150	D
001-071	-0.378527	-1.115512	D
001-076	-0.501270	-1.656413	D
001-078	-0.400028	-1.210262	D
002-014	-0.166923	-0.183023	B
002-018	-0.214849	-0.394219	B
002-021	-0.385560	-1.146503	B
002-026	-0.064894	0.266598	B
002-030	0.053960	0.790359	B
002-042	-0.145697	-0.089484	D
002-064	0.285845	1.812222	D
002-066	-0.198913	-0.323994	D
002-067	0.191567	1.396761	D
002-069	-0.379517	-1.119875	D
002-070	0.119757	1.080311	D
002-074	-0.326396	-0.885782	D
002-075	-0.340931	-0.949837	D
003-016	0.142378	1.179995	B
003-019	-0.146189	-0.091653	B
003-023	-0.195774	-0.310160	B
003-024	-0.289772	-0.724388	B
003-029	-0.140542	-0.066767	B
003-031	-0.097839	0.121415	B
003-036	-0.365135	-1.056496	B
301-001	0.153778	1.230233	B
301-003	-0.287187	-0.712997	B
301-004	-0.235351	-0.484570	B
301-005	0.001004	0.556993	B
301-013	-0.018753	0.469927	B
301-022	0.238637	1.604185	B
301-025	0.176801	1.331688	B
301-039	-0.125333	0.000256	C
301-044	-0.172339	-0.206889	C

301-045	0.251858	1.662446	C
501-049	-0.126691	-0.005729	C
501-053	-0.165610	-0.177236	D
501-054	-0.020749	0.461131	C
501-058	-0.272957	-0.650288	C
501-059	-0.453108	-1.444171	D
501-061	-0.268843	-0.632161	C
502-052	-0.463983	-1.492095	D
502-062	-0.348191	-0.981830	D
502-063	-0.329929	-0.901351	C

SAMPLE_ID	BEST_OVERALL_RESPONSE	PFS \
001-009	PD	PD
001-015	CR/PR	CR/PR
001-037	PD	PD
001-038	PD	PD
001-043	PD	PD
001-046	non-durable SD / PD	non-durable SD (<100d) / PD
001-047	CR/PR	CR/PR
001-050	durable SD	durable SD (>140d / PFS6+)
001-055	non-durable SD / PD	non-durable SD (<100d) / PD
001-064	CR/PR	CR/PR
001-071	durable SD	durable SD (>140d / PFS6+)
001-076	PD	PD
001-078	non-durable SD / PD	non-durable SD (<100d) / PD
002-014	PD	PD
002-018	durable SD	durable SD (>140d / PFS6+)
002-021	PD	PD
002-026	non-durable SD / PD	non-durable SD (<100d) / PD
002-030	PD	PD
002-042	non-durable SD / PD	non-durable SD (<100d) / PD
002-064	PD	PD
002-066	non-durable SD / PD	non-durable SD (<100d) / PD
002-067	durable SD	durable SD (>140d / PFS6+)
002-069	durable SD	durable SD (>140d / PFS6+)
002-070	PD	PD
002-074	PD	PD
002-075	non-durable SD / PD	non-durable SD (<100d) / PD
003-016	non-durable SD / PD	non-durable SD (<100d) / PD
003-019	non-durable SD / PD	non-durable SD (<100d) / PD
003-023	durable SD	durable SD (>140d / PFS6+)
003-024	durable SD	durable SD (>140d / PFS6+)
003-029	PD	PD
003-031	PD	PD
003-036	PD	PD
301-001	non-durable SD / PD	non-durable SD (<100d) / PD

301-003	PD	PD
301-004	PD	PD
301-005	CR/PR	CR/PR
301-013	CR/PR	CR/PR
301-022	PD	PD
301-025	non-durable SD / PD	non-durable SD (<100d) / PD
301-039	CR/PR	CR/PR
301-044	PD	PD
301-045	PD	PD
501-049	durable SD	durable SD (>140d / PFS6+)
501-053	PD	PD
501-054	durable SD	durable SD (>140d / PFS6+)
501-058	PD	PD
501-059	durable SD	durable SD (>140d / PFS6+)
501-061	PD	PD
502-052	durable SD	durable SD (>140d / PFS6+)
502-062	CR/PR	CR/PR
502-063	PD	PD

SAMPLE_ID	BENEFIT_STATUS	RESPONSE_GROUP
001-009	Resistance	PD
001-015	Clinical Benefit	CR/PR
001-037	Resistance	PD
001-038	Resistance	PD
001-043	Resistance	PD
001-046	Resistance	non-durable SD / PD
001-047	Clinical Benefit	CR/PR
001-050	Clinical Benefit	durable SD
001-055	Resistance	non-durable SD / PD
001-064	Clinical Benefit	CR/PR
001-071	Clinical Benefit	durable SD
001-076	Resistance	PD
001-078	Resistance	non-durable SD / PD
002-014	Resistance	PD
002-018	Clinical Benefit	durable SD
002-021	Resistance	PD
002-026	Resistance	non-durable SD / PD
002-030	Resistance	PD
002-042	Resistance	non-durable SD / PD
002-064	Resistance	PD
002-066	Resistance	non-durable SD / PD
002-067	Clinical Benefit	durable SD
002-069	Clinical Benefit	durable SD
002-070	Resistance	PD
002-074	Resistance	PD
002-075	Resistance	non-durable SD / PD

003-016	Resistance	non-durable SD / PD
003-019	Resistance	non-durable SD / PD
003-023	Clinical Benefit	durable SD
003-024	Clinical Benefit	durable SD
003-029	Resistance	PD
003-031	Resistance	PD
003-036	Resistance	PD
301-001	Resistance	non-durable SD / PD
301-003	Resistance	PD
301-004	Resistance	PD
301-005	Clinical Benefit	CR/PR
301-013	Clinical Benefit	CR/PR
301-022	Resistance	PD
301-025	Resistance	non-durable SD / PD
301-039	Clinical Benefit	CR/PR
301-044	Resistance	PD
301-045	Resistance	PD
501-049	Clinical Benefit	durable SD
501-053	Resistance	PD
501-054	Clinical Benefit	durable SD
501-058	Resistance	PD
501-059	Clinical Benefit	durable SD
501-061	Resistance	PD
502-052	Clinical Benefit	durable SD
502-062	Clinical Benefit	CR/PR
502-063	Resistance	PD

```
[604]: output_file = "../../../results/GSE186332/selinexor_treatment_arm_sensitivity.xlsx"
```

```
with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    treatment_summary.to_excel(writer, sheet_name="Summary", index=False)
    arm_score_summary.to_excel(writer, sheet_name="Arm_Score_Distribution",
    ↪index=False)
    logistic_results.to_excel(writer, sheet_name="Logistic_Regression",
    ↪index=False)
    patient_level.to_excel(writer, sheet_name="Patient_Level", index=True)
```

- selinexor_treatment_arm_sensitivity.xlsx exported to /results/GSE186332

Summary Note: Recurrence Score distributions did not differ significantly across treatment arms (Kruskal–Wallis $p=0.162$). In an exploratory logistic regression adjusted for treatment arm, the Recurrence Score coefficient remained directionally consistent with the primary analysis but was not statistically significant (OR per 1-SD increase=0.656, 95% CI 0.340–1.263, $p=0.207$). Treatment arms were substantially imbalanced (A=3, B=19, C=9, D=21), so the adjusted model should be interpreted cautiously.

10 Integration of evidence

10.1 1. Integrated evidence matrix.

```
[605]: integrated_recurrence_vulnerability_matrix = pd.DataFrame(  
    {  
        "Target / Intervention": ["HSP90 inhibition", "XP01 / selinexor"],  
        "PRISM_PHARMACOLOGIC_SIGNAL": ["SUPPORTED", "NO SUPPORT"],  
        "CLASS_LEVEL_PHARMACOLOGIC_SUPPORT": ["SUPPORTED", "PARTIAL SUPPORT"],  
        "CRISPR_DEPENDENCY_SUPPORT": ["PARTIAL SUPPORT", "NO SUPPORT"],  
        "PROLIFERATION_ROBUSTNESS": ["SUPPORTED", "NO SUPPORT"],  
        "CLINICAL_PATIENT_SUPPORT": ["PARTIAL SUPPORT", "NO SUPPORT"],  
        "CNS_TRANSLATIONAL_RELEVANCE": ["SUPPORTED", "PARTIAL SUPPORT"],  
        "FINAL_INTERPRETATION": ["SUPPORTED", "NO SUPPORT"],  
    }  
)  
.set_index("Target / Intervention")  
  
integrated_recurrence_vulnerability_matrix.to_excel("../results/evidence/  
↪integrated_recurrence_vulnerability_matrix.xlsx", index=True)
```

- integrated_recurrence_vulnerability_matrix.xlsx exported to /results directory

```
[606]: integrated_recurrence_vulnerability_matrix
```

```
[606]:
```

	PRISM_PHARMACOLOGIC_SIGNAL \	
Target / Intervention		
HSP90 inhibition	SUPPORTED	
XP01 / selinexor	NO SUPPORT	

	CLASS_LEVEL_PHARMACOLOGIC_SUPPORT \	
Target / Intervention		
HSP90 inhibition	SUPPORTED	
XP01 / selinexor	PARTIAL SUPPORT	

	CRISPR_DEPENDENCY_SUPPORT	PROLIFERATION_ROBUSTNESS \	
Target / Intervention			
HSP90 inhibition	PARTIAL SUPPORT	SUPPORTED	
XP01 / selinexor	NO SUPPORT	NO SUPPORT	

	CLINICAL_PATIENT_SUPPORT	CNS_TRANSLATIONAL_RELEVANCE \	
Target / Intervention			
HSP90 inhibition	PARTIAL SUPPORT	SUPPORTED	
XP01 / selinexor	NO SUPPORT	PARTIAL SUPPORT	

	FINAL_INTERPRETATION	
Target / Intervention		
HSP90 inhibition	SUPPORTED	
XP01 / selinexor	NO SUPPORT	

```

[607]: plt.rcParams["font.family"] = "sans-serif"
plt.rcParams["font.sans-serif"] = ["Helvetica Neue", "Helvetica", "Arial",
    ↪ "DejaVu Sans"]

fig, ax = plt.subplots(figsize=(10, 14))
ax.set_xlim(0, 10)
ax.set_ylim(0, 16)
ax.axis("off")

def draw_box(x, y, w, h, text, facecolor="#f8f9fa", edgecolor="#333333",
    ↪ text_color="#222222", style="round,pad=0.35", ls="solid"):
    box = patches.FancyBboxPatch((x - w / 2, y - h / 2), w, h, boxstyle=style,
    ↪ facecolor=facecolor, edgecolor=edgecolor, linewidth=1.5, linestyle=ls)
    ax.add_patch(box)
    ax.text(x, y, text, ha="center", va="center", fontsize=9.5,
    ↪ fontweight="bold", color=text_color, wrap=True)

def draw_connector(x1, y1, x2, y2, ls="solid", color="#555555"):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1),
    ↪ arrowprops=dict(arrowstyle="->", color=color, lw=1.5, linestyle=ls,
    ↪ mutation_scale=12))

draw_box(5, 14.5, 6.2, 0.8, "GLASS longitudinal patient samples\n(Primary ↪
    ↪ First Recurrence)", facecolor="#ffffff", edgecolor="#2c3e50")
draw_box(5, 12.9, 4.8, 0.6, "265-gene Recurrence Signature",
    ↪ facecolor="#ffffff", edgecolor="#2c3e50")
draw_box(5, 11.4, 3.8, 0.6, "Recurrence Score", facecolor="#ffffff",
    ↪ edgecolor="#2c3e50")
draw_box(5, 9.9, 3.8, 0.6, "DepMap glioma models", facecolor="#ffffff",
    ↪ edgecolor="#2c3e50")
draw_box(5, 8.4, 4.5, 0.6, "PRISM pharmacogenomic screen", facecolor="#ffffff",
    ↪ edgecolor="#2c3e50")

draw_connector(5, 14.1, 5, 13.2)
draw_connector(5, 12.6, 5, 11.7)
draw_connector(5, 11.1, 5, 10.2)
draw_connector(5, 9.6, 5, 8.7)

ax.text(5, 7.3, "Main Translational Directions", ha="center", va="center",
    ↪ fontsize=11, fontstyle="italic", fontweight="bold", color="#2980b9")
draw_connector(5, 8.0, 5, 7.6)
draw_connector(5, 7.0, 5, 6.6)

```



```

draw_box(2.5, 5.5, 3.6, 0.6, "HSP90 / Alvespimycin\n(Candidate Hit)",
    ↳facecolor="#e8f5e9", edgecolor="#2e7d32")
draw_box(2.5, 4.2, 3.8, 0.6, "HSP90 inhibitor class\n(Consistent Signal)",
    ↳facecolor="#e8f5e9", edgecolor="#2e7d32")
draw_box(2.5, 2.9, 3.6, 0.6, "CRISPR dependency\n(Orthogonal Support: NOT_
    ↳Met)", facecolor="#fff3e0", edgecolor="#f57c00", text_color="#e65100",
    ↳ls="dashed")

draw_connector(5, 6.6, 2.5, 5.9, color="#2e7d32")
draw_connector(2.5, 5.2, 2.5, 4.5, color="#2e7d32")
draw_connector(2.5, 3.9, 2.5, 3.2, color="#f57c00", ls="dashed")

draw_box(7.5, 5.5, 3.6, 0.6, "XP01 / Selinexor\n(No PRISM Signal)",
    ↳facecolor="#f5f5f5", edgecolor="#9e9e9e", text_color="#555555", ls="dashed")
draw_box(7.5, 4.2, 3.8, 0.6, "GSE186332\n(Low AUC = 0.423)",
    ↳facecolor="#f5f5f5", edgecolor="#9e9e9e", text_color="#555555", ls="dashed")
draw_box(7.5, 2.9, 3.8, 0.6, "Clinical Benefit: NOT Met\n(Deprioritized)",
    ↳facecolor="#ffebee", edgecolor="#c62828", text_color="#c62828", ls="dashed")

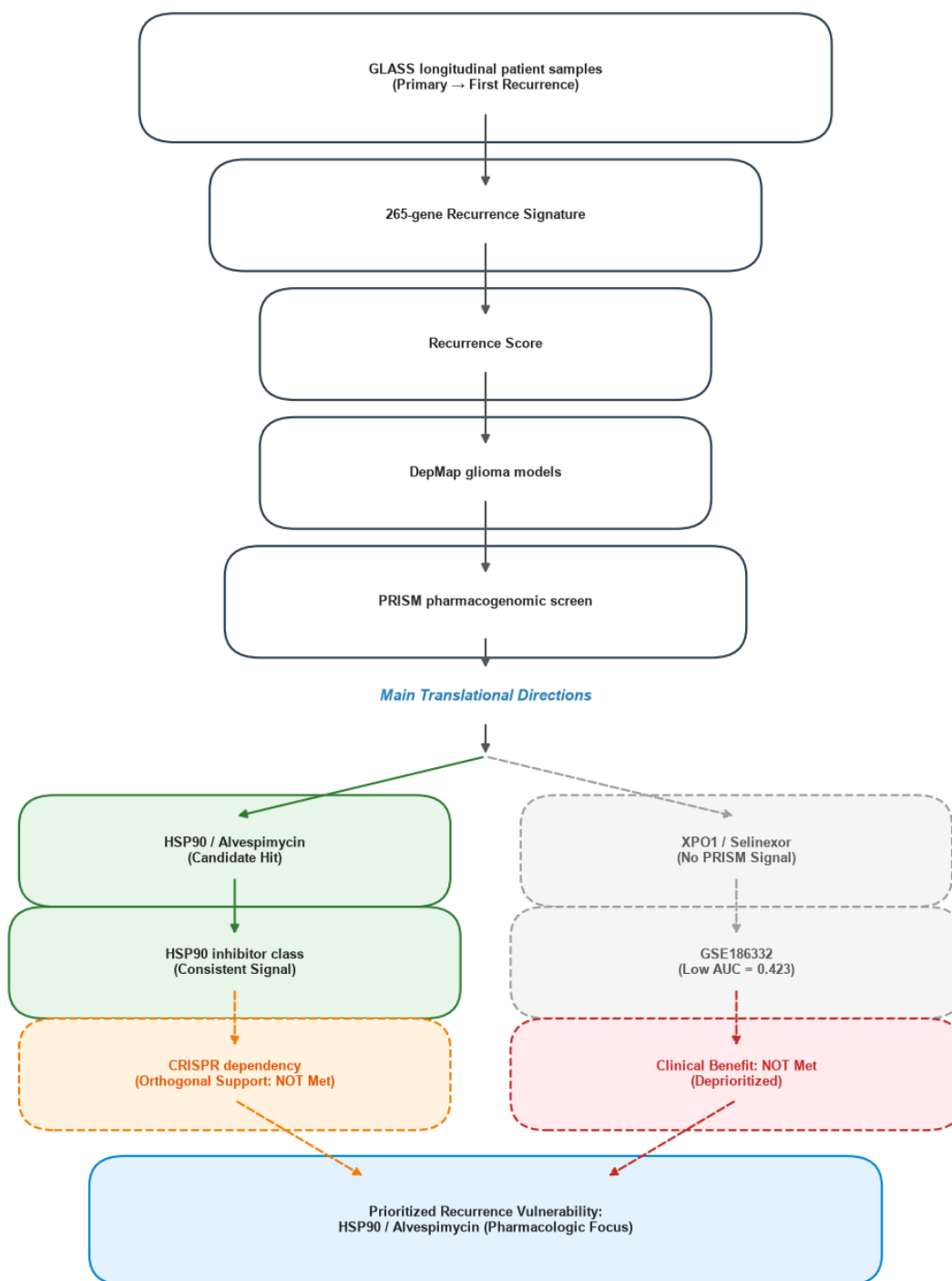
draw_connector(5, 6.6, 7.5, 5.9, color="#9e9e9e", ls="dashed")
draw_connector(7.5, 5.2, 7.5, 4.5, color="#9e9e9e", ls="dashed")
draw_connector(7.5, 3.9, 7.5, 3.2, color="#c62828", ls="dashed")

draw_box(5, 1.2, 7.2, 0.8, "Prioritized Recurrence Vulnerability:\nHSP90 /
    ↳Alvespimycin (Pharmacologic Focus)", facecolor="#e3f2fd",
    ↳edgecolor="#0288d1")
draw_connector(2.5, 2.6, 3.8, 1.7, color="#f57c00", ls="dashed")
draw_connector(7.5, 2.6, 6.2, 1.7, color="#c62828", ls="dashed")

plt.title("TDK Integrated Workflow & Pathway Prioritization", fontsize=20,
    ↳fontweight="bold", pad=15, color="#1a1a1a")
plt.tight_layout()
output_filename = "../figures/tdk_integrated_workflow_figure.png"
plt.savefig(output_filename, dpi=300)
plt.show()

```

TDK Integrated Workflow & Pathway Prioritization



- `tdk_integrated_workflow_figure.png` exported to `/figures` directory

11 Validation of HSP90 drug-class specificity and pharmacogenomic robustness

11.1 1. Creating HSP90 class sensitivity score

11.1.1 1.1 Drug-wise sensitivity score

```
[608]: hsp90_drugs = frozen_prism_hsp90_drugs[["DRUG", "BRD_ID"]].drop_duplicates().
        ↪copy()
drug_sensitivity_percentiles = []

for _, row in hsp90_drugs.iterrows():
    drug, brd_id = row["DRUG"], row["BRD_ID"]
    cohort_col = f"{drug} (BRD:{brd_id})"

    if cohort_col not in prism_cohort.columns:
        continue

    sensitivity = -prism_cohort[cohort_col].copy()
    sensitivity_percentile = sensitivity.rank(method="average", pct=True)

    drug_result = pd.DataFrame({
        "DRUG": drug,
        "BRD_ID": brd_id,
        "SENSITIVITY": sensitivity,
        "SENSITIVITY_PERCENTILE": sensitivity_percentile,
    })
    drug_sensitivity_percentiles.append(drug_result)
```

```
[609]: hsp90_percentile_matrix = pd.DataFrame(index=prism_cohort.index)
hsp90_auc_matrix = pd.DataFrame(index=prism_cohort.index)

for _, row in hsp90_drugs.iterrows():
    drug, brd_id = row["DRUG"], row["BRD_ID"]
    cohort_col = f"{drug} (BRD:{brd_id})"

    if cohort_col not in prism_cohort.columns:
        continue

    auc = prism_cohort[cohort_col]
    hsp90_auc_matrix[drug] = auc
    sensitivity = -auc
    hsp90_percentile_matrix[drug] = sensitivity.rank(method="average", pct=True)
```

```
[610]: hsp90_percentile_matrix
```

```
[610]:          ALVESPIMYCIN  BIIB021  KW-2478  XL888  AT13387  GANETESPIB  \
ACH-000027      0.162791  0.209302  0.333333  0.535714  0.357143  0.232558
```

ACH-000036	0.441860	0.325581	NaN	NaN	0.166667	0.162791
ACH-000040	0.465116	0.930233	0.533333	0.500000	0.976190	0.697674
ACH-000067	0.093023	0.162791	NaN	NaN	0.047619	0.116279
ACH-000075	0.906977	0.953488	NaN	NaN	0.785714	0.976744
ACH-000098	0.697674	0.674419	0.700000	0.571429	0.571429	0.604651
ACH-000102	0.302326	0.627907	0.766667	0.750000	0.500000	0.395349
ACH-000137	0.372093	0.860465	0.666667	0.785714	0.642857	0.511628
ACH-000152	0.255814	0.372093	NaN	NaN	0.476190	0.255814
ACH-000208	0.627907	0.348837	NaN	NaN	0.404762	0.418605
ACH-000215	0.651163	0.418605	NaN	NaN	0.523810	0.581395
ACH-000231	NaN	NaN	0.733333	0.285714	NaN	NaN
ACH-000232	0.744186	0.651163	0.933333	0.678571	0.833333	0.674419
ACH-000244	0.720930	0.139535	0.600000	0.250000	0.142857	0.488372
ACH-000269	0.813953	1.000000	NaN	NaN	0.952381	0.953488
ACH-000283	0.674419	0.581395	NaN	NaN	0.690476	0.720930
ACH-000289	0.953488	0.697674	0.966667	0.428571	0.666667	0.906977
ACH-000323	0.767442	0.837209	1.000000	0.464286	0.714286	0.441860
ACH-000329	0.488372	0.279070	0.233333	0.214286	0.119048	0.139535
ACH-000368	0.581395	0.441860	0.833333	0.642857	0.333333	0.372093
ACH-000376	0.116279	0.023256	0.200000	0.321429	0.071429	0.023256
ACH-000389	0.976744	0.744186	0.166667	0.714286	0.452381	0.837209
ACH-000437	0.558140	0.511628	0.500000	1.000000	0.809524	0.534884
ACH-000445	0.883721	0.976744	0.433333	0.821429	0.738095	0.651163
ACH-000464	0.279070	0.069767	NaN	NaN	NaN	0.093023
ACH-000469	0.395349	0.488372	0.633333	0.928571	0.261905	0.465116
ACH-000479	0.232558	0.093023	0.133333	0.142857	0.309524	0.279070
ACH-000504	0.604651	0.534884	0.466667	0.178571	0.095238	0.069767
ACH-000543	1.000000	0.604651	NaN	NaN	0.547619	0.186047
ACH-000558	0.511628	0.302326	0.266667	0.392857	0.380952	0.627907
ACH-000570	0.837209	0.906977	0.866667	NaN	0.928571	0.767442
ACH-000571	0.860465	0.883721	0.400000	0.964286	0.761905	0.813953
ACH-000592	0.069767	0.395349	0.066667	0.607143	0.428571	0.558140
ACH-000595	0.534884	0.232558	0.033333	0.107143	0.238095	0.209302
ACH-000609	0.023256	0.720930	0.800000	NaN	0.619048	0.883721
ACH-000622	0.418605	0.116279	0.566667	0.071429	0.190476	0.046512
ACH-000623	0.186047	0.255814	NaN	NaN	0.214286	0.348837
ACH-000631	0.348837	0.186047	NaN	NaN	0.285714	0.325581
ACH-000738	0.046512	0.046512	0.100000	0.857143	0.023810	0.302326
ACH-000760	0.790698	0.790698	NaN	NaN	0.857143	0.790698
ACH-000819	0.325581	0.767442	0.300000	0.035714	0.880952	0.860465
ACH-000863	0.209302	0.558140	0.366667	0.892857	0.595238	0.930233
ACH-000883	0.139535	0.465116	NaN	NaN	1.000000	0.744186
ACH-001016	0.930233	0.813953	0.900000	0.357143	0.904762	1.000000
	NMS-E973	TANESPIMYCIN	SNX 2112	SNX-5422	VER-49009	PU-H71
ACH-000027	0.566667	0.214286	0.200000	0.307692	0.100000	0.433333
ACH-000036	NaN	0.928571	NaN	NaN	NaN	NaN

ACH-000040	0.666667	0.476190	0.900000	0.769231	0.633333	0.966667
ACH-000067	NaN	0.761905	NaN	NaN	NaN	NaN
ACH-000075	NaN	0.738095	NaN	NaN	NaN	NaN
ACH-000098	0.633333	0.571429	0.233333	0.384615	0.466667	0.233333
ACH-000102	0.733333	NaN	0.666667	0.730769	0.300000	0.800000
ACH-000137	0.500000	0.119048	0.566667	0.500000	0.600000	0.666667
ACH-000152	NaN	0.071429	NaN	NaN	NaN	NaN
ACH-000208	NaN	0.523810	NaN	NaN	NaN	NaN
ACH-000215	NaN	0.880952	NaN	NaN	NaN	NaN
ACH-000231	0.900000	NaN	0.533333	0.846154	0.866667	0.466667
ACH-000232	0.833333	0.642857	0.800000	0.884615	0.900000	0.733333
ACH-000244	0.233333	0.166667	0.300000	NaN	0.266667	0.333333
ACH-000269	NaN	0.619048	NaN	NaN	NaN	NaN
ACH-000283	NaN	0.976190	NaN	NaN	NaN	NaN
ACH-000289	0.866667	0.833333	0.833333	0.576923	0.033333	0.133333
ACH-000323	0.966667	0.666667	0.933333	1.000000	0.766667	0.600000
ACH-000329	0.033333	0.809524	0.333333	0.115385	0.166667	0.400000
ACH-000368	0.366667	0.238095	0.733333	0.346154	0.666667	0.633333
ACH-000376	0.100000	0.547619	0.600000	0.038462	0.366667	0.533333
ACH-000389	0.133333	0.952381	0.166667	0.230769	0.233333	0.300000
ACH-000437	0.700000	0.428571	0.500000	0.692308	0.833333	0.866667
ACH-000445	0.600000	0.357143	0.633333	NaN	0.800000	0.900000
ACH-000464	NaN	0.261905	NaN	NaN	NaN	NaN
ACH-000469	0.466667	0.309524	0.466667	0.653846	0.400000	0.700000
ACH-000479	0.333333	0.142857	0.100000	NaN	0.200000	0.200000
ACH-000504	0.266667	0.904762	0.033333	0.076923	0.066667	0.166667
ACH-000543	NaN	1.000000	NaN	NaN	NaN	NaN
ACH-000558	0.533333	0.333333	0.400000	0.538462	0.500000	0.266667
ACH-000570	0.933333	0.380952	0.766667	0.423077	0.333333	0.566667
ACH-000571	0.200000	0.785714	0.433333	0.461538	0.966667	0.766667
ACH-000592	0.066667	0.095238	0.266667	NaN	0.733333	0.033333
ACH-000595	0.166667	0.500000	0.066667	0.192308	0.433333	0.066667
ACH-000609	1.000000	0.035714	1.000000	0.961538	1.000000	1.000000
ACH-000622	0.300000	0.285714	0.366667	0.153846	0.933333	0.100000
ACH-000623	NaN	0.714286	NaN	NaN	NaN	NaN
ACH-000631	NaN	0.857143	NaN	NaN	NaN	NaN
ACH-000738	0.433333	0.035714	0.133333	0.269231	0.133333	0.366667
ACH-000760	NaN	0.595238	NaN	NaN	NaN	NaN
ACH-000819	0.400000	0.452381	0.700000	0.807692	0.700000	0.933333
ACH-000863	0.766667	0.190476	0.866667	0.923077	0.566667	0.833333
ACH-000883	NaN	0.690476	NaN	NaN	NaN	NaN
ACH-001016	0.800000	0.404762	0.966667	0.615385	0.533333	0.500000

11.1.2 1.2 HSP90 class score

$$\text{HSP90 CLASS SENSITIVITY}_i = \text{median}_{d \in \mathcal{D}_i} (\text{Sensitivity Percentile}_{i,d})$$

```
[611]: n_hsp90_drugs_available = hsp90_percentile_matrix.notna().sum(axis=1)
hsp90_class_sensitivity = hsp90_percentile_matrix.median(axis=1, skipna=True)
hsp90_class_sensitivity = hsp90_class_sensitivity.where(n_hsp90_drugs_available_
↳ >= 5)
```

```
[612]: recurrence_info = (
    depmap_glioma_recurrence_scores[
        ["ModelID", "DEPMAP_RECURRENCE_SCORE"]
    ]
    .drop_duplicates("ModelID")
    .set_index("ModelID")
)

cell_line_info = glioma_cell_line_master[["DEPMAP_ID"]].copy()
cell_line_info["CELL_LINE_NAME"] = cell_line_info.index
cell_line_info = (
    cell_line_info
    .drop_duplicates("DEPMAP_ID")
    .set_index("DEPMAP_ID")
)
```

```
[613]: prism_hsp90_class_sensitivity = pd.DataFrame({
    "DEPMAP_ID": hsp90_class_sensitivity.index,
    "N_HSP90_DRUGS_AVAILABLE": n_hsp90_drugs_available.values,
    "HSP90_CLASS_SENSITIVITY": hsp90_class_sensitivity.values,
})

prism_hsp90_class_sensitivity = (
    prism_hsp90_class_sensitivity
    .set_index("DEPMAP_ID")
    .join(recurrence_info.rename(columns={"DEPMAP_RECURRENCE_SCORE":
↳ "RECURRENCE_SCORE"}), how="left")
    .join(cell_line_info, how="left")
    .reset_index()
)

prism_hsp90_class_sensitivity = prism_hsp90_class_sensitivity[[
    "DEPMAP_ID",
    "CELL_LINE_NAME",
    "RECURRENCE_SCORE",
    "N_HSP90_DRUGS_AVAILABLE",
    "HSP90_CLASS_SENSITIVITY",
]]
```

```
[614]: prism_hsp90_class_sensitivity.
↳ loc[prism_hsp90_class_sensitivity["N_HSP90_DRUGS_AVAILABLE"] <
↳ 5, "HSP90_CLASS_SENSITIVITY"] = np.nan
```

```
[615]: prism_hsp90_class_sensitivity.to_excel(
        ".../results/class_sensitivity/prism_hsp90_class_sensitivity.xlsx",
        index=False
    )
```

- prism_hsp90_class_sensitivity.xlsx exported to /results/class_sensitivity

```
[616]: print(prism_hsp90_class_sensitivity["N_HSP90_DRUGS_AVAILABLE"].describe())
print("Cell lines meeting >=5-drug requirement:",
      ↪(prism_hsp90_class_sensitivity["N_HSP90_DRUGS_AVAILABLE"] >= 5).sum())
print("Cell lines failing requirement:",
      ↪(prism_hsp90_class_sensitivity["N_HSP90_DRUGS_AVAILABLE"] < 5).sum())
```

```
count      44.000000
mean        9.477273
std         3.245667
min         4.000000
25%         5.000000
50%        11.500000
75%        12.000000
max         12.000000
Name: N_HSP90_DRUGS_AVAILABLE, dtype: float64
Cell lines meeting >=5-drug requirement: 43
Cell lines failing requirement: 1
```

11.2 2. Relationship between Recurrence Score and HSP90 class sensitivity

```
[617]: correlation_df = prism_hsp90_class_sensitivity[["DEPMAP_ID",
      ↪"RECURRENCE_SCORE", "HSP90_CLASS_SENSITIVITY"]].dropna().copy()

N = len(correlation_df)

rho, p_value = spearmanr(
    correlation_df["RECURRENCE_SCORE"],
    correlation_df["HSP90_CLASS_SENSITIVITY"],
)

B = 2000
RANDOM_SEED = 42

rng = np.random.default_rng(RANDOM_SEED)

x = correlation_df["RECURRENCE_SCORE"].to_numpy()
y = correlation_df["HSP90_CLASS_SENSITIVITY"].to_numpy()

bootstrap_rhos = []

for _ in range(B):
```

```

idx = rng.integers(0, N, size=N)
x_boot = x[idx]
y_boot = y[idx]
if len(np.unique(x_boot)) < 2:
    continue
if len(np.unique(y_boot)) < 2:
    continue
rho_boot, _ = spearmanr(x_boot, y_boot)
if np.isfinite(rho_boot):
    bootstrap_rhos.append(rho_boot)

bootstrap_rhos = np.asarray(bootstrap_rhos)

BOOTSTRAP_RHO_CI_LOWER = np.percentile(bootstrap_rhos, 2.5)
BOOTSTRAP_RHO_CI_UPPER = np.percentile(bootstrap_rhos, 97.5)

```

```

[618]: recurrence_vs_hsp90_class_sensitivity = pd.DataFrame({
    "Metric": [
        "N",
        "SPEARMAN_RHO",
        "P_VALUE",
        "BOOTSTRAP_B",
        "RANDOM_SEED",
        "BOOTSTRAP_RHO_CI_LOWER",
        "BOOTSTRAP_RHO_CI_UPPER",
        "BOOTSTRAP_N_VALID",
    ],
    "Value": [
        N,
        rho,
        p_value,
        B,
        RANDOM_SEED,
        BOOTSTRAP_RHO_CI_LOWER,
        BOOTSTRAP_RHO_CI_UPPER,
        len(bootstrap_rhos),
    ],
})

```

```

[619]: recurrence_vs_hsp90_class_sensitivity

```

```

[619]:

```

	Metric	Value
0	N	43.000000
1	SPEARMAN_RHO	0.354101
2	P_VALUE	0.019826
3	BOOTSTRAP_B	2000.000000
4	RANDOM_SEED	42.000000


```

5  BOOTSTRAP_RHO_CI_LOWER      0.020572
6  BOOTSTRAP_RHO_CI_UPPER      0.636370
7      BOOTSTRAP_N_VALID  2000.000000

```

```
[620]: print(recurrence_vs_hsp90_class_sensitivity)
```

```

          Metric      Value
0              N    43.000000
1    SPEARMAN_RHO    0.354101
2        P_VALUE    0.019826
3    BOOTSTRAP_B  2000.000000
4    RANDOM_SEED    42.000000
5  BOOTSTRAP_RHO_CI_LOWER    0.020572
6  BOOTSTRAP_RHO_CI_UPPER    0.636370
7    BOOTSTRAP_N_VALID  2000.000000

```

```
[621]: output_file = "../results/class_sensitivity/
        ↪recurrence_vs_hsp90_class_sensitivity.xlsx"

with pd.ExcelWriter(output_file,engine="openpyxl") as writer:
    recurrence_vs_hsp90_class_sensitivity.to_excel(
        writer,
        sheet_name="Summary",
        index=False
    )

    correlation_df.to_excel(
        writer,
        sheet_name="Correlation_Data",
        index=False
    )

```

- recurrence_vs_hsp90_class_sensitivity.xlsx exported to
/results/class_sensitivity

Summary Note: The Recurrence Score was positively associated with HSP90 class sensitivity (N = 43, Spearman's $\rho = 0.354$, $p = 0.0198$). This direction supports the predefined hypothesis that higher Recurrence Scores are associated with greater HSP90 inhibitor sensitivity. However, the 95% bootstrap CI (0.0226 to 0.636) included zero, indicating uncertainty around the effect estimate.

11.3 3. Control of general drug sensitivity

11.3.1 3.1 Global non-HSP90 sensitivity score

$$\text{GLOBAL NONHSP90 SENSITIVITY}_i = \text{median}_{d \in D_i} (\text{Sensitivity Percentile}_{i,d})$$

```
[622]: hsp90_drug_names = set(frozen_prism_hsp90_drugs["DRUG"].dropna().astype(str))
```

```

prism_non_hsp90_drugs = (
    prism_recurrence_drug_screen[
        ~prism_recurrence_drug_screen["DRUG"].astype(str).isin(hsp90_drug_names)
    ][["DRUG", "BRD_ID"]]
    .drop_duplicates()
    .copy()
)

```

```

[623]: non_hsp90_percentile_matrix, non_hsp90_auc_matrix = pd.
        ↪DataFrame(index=prism_cohort.index), pd.DataFrame(index=prism_cohort.index)
auc_columns, percentile_columns = {}, {}

for _, row in prism_non_hsp90_drugs.iterrows():
    drug, brd_id = row["DRUG"], row["BRD_ID"]
    cohort_col = f"{drug} (BRD:{brd_id})"

    if cohort_col not in prism_cohort.columns:
        continue

    auc = prism_cohort[cohort_col]
    auc_columns[drug] = auc
    sensitivity = -auc
    percentile_columns[drug] = sensitivity.rank(method="average", pct=True)

non_hsp90_auc_matrix = pd.DataFrame(auc_columns, index=prism_cohort.index)
non_hsp90_percentile_matrix = pd.DataFrame(percentile_columns,
        ↪index=prism_cohort.index)

```

```

[624]: n_non_hsp90_drugs_available = non_hsp90_percentile_matrix.notna().sum(axis=1)
global_non_hsp90_sensitivity = non_hsp90_percentile_matrix.median(axis=1,
        ↪skipna=True)
MIN_NON_HSP90_DRUGS = 20
global_non_hsp90_sensitivity = global_non_hsp90_sensitivity.
        ↪where(n_non_hsp90_drugs_available >= MIN_NON_HSP90_DRUGS)

```

```

[625]: recurrence_info = depmap_glioma_recurrence_scores[["ModelID",
        ↪"DEPMAP_RECURRENCE_SCORE"]].drop_duplicates("ModelID").set_index("ModelID")

cell_line_info = glioma_cell_line_master[["DEPMAP_ID"]].copy()
cell_line_info["CELL_LINE_NAME"] = cell_line_info.index
cell_line_info = cell_line_info.drop_duplicates("DEPMAP_ID").
        ↪set_index("DEPMAP_ID")

global_non_hsp90_sensitivity_df = (
    global_non_hsp90_sensitivity
    .rename("GLOBAL_NON_HSP90_SENSITIVITY")
    .to_frame()
)

```

```

.join(n_non_hsp90_drugs_available.rename("N_NON_HSP90_DRUGS_AVAILABLE"))
.join(recurrence_info.rename(columns={"DEPMAP_RECURRENCE_SCORE":
↪"RECURRENCE_SCORE"}), how="left")
.join(cell_line_info, how="left")
.reset_index()
.rename(columns={"index": "DEPMAP_ID"})
)

```

```

[626]: print("Total non-HSP90 PRISM drugs:", len(prism_non_hsp90_drugs))
print("Non-HSP90 drugs with response columns:", non_hsp90_percentile_matrix.
↪shape[1])
print("Cell lines:", len(global_non_hsp90_sensitivity_df))
print("Cell lines meeting minimum drug coverage:",
↪global_non_hsp90_sensitivity_df["GLOBAL_NON_HSP90_SENSITIVITY"].notna().
↪sum())
print("Minimum non-HSP90 drug coverage:", MIN_NON_HSP90_DRUGS)
print("Median number of available non-HSP90 drugs:",
↪n_non_hsp90_drugs_available.median())

```

Total non-HSP90 PRISM drugs: 1397
 Non-HSP90 drugs with response columns: 1397
 Cell lines: 44
 Cell lines meeting minimum drug coverage: 44
 Minimum non-HSP90 drug coverage: 20
 Median number of available non-HSP90 drugs: 1309.0

```

[627]: output_file = "../results/class_sensitivity/global_non_hsp90_sensitivity.
↪xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    global_non_hsp90_sensitivity_df.to_excel(writer,
↪sheet_name="Global_Sensitivity", index=False)
    non_hsp90_percentile_matrix.to_excel(writer, sheet_name="Drug_Percentiles")
    pd.DataFrame({
        "metric": [
            "N_NON_HSP90_DRUGS_UNIVERSE", "N_NON_HSP90_DRUGS_WITH_DATA",
            "MIN_NON_HSP90_DRUGS_REQUIRED", "N_CELL_LINES",
↪"N_CELL_LINES_MEETING_COVERAGE"
        ],
        "value": [
            len(prism_non_hsp90_drugs), non_hsp90_percentile_matrix.shape[1],
            MIN_NON_HSP90_DRUGS, len(global_non_hsp90_sensitivity_df),
            global_non_hsp90_sensitivity_df["GLOBAL_NON_HSP90_SENSITIVITY"].
↪notna().sum()
        ]
    }).to_excel(writer, sheet_name="QC", index=False)

```

- global_non_hsp90_sensitivity.xlsx exported to /results/class_sensitivity direc-

tory

11.3.2 3.2 Recurrence score and global sensitivity

```
[628]: correlation_df = global_non_hsp90_sensitivity_df[
        ["DEPMAP_ID", "RECURRENCE_SCORE", "GLOBAL_NON_HSP90_SENSITIVITY"]
    ].dropna().copy()

N = len(correlation_df)

rho, p_value = spearmanr(
    correlation_df["RECURRENCE_SCORE"],
    correlation_df["GLOBAL_NON_HSP90_SENSITIVITY"],
)
```

```
[629]: B, RANDOM_SEED = 2000, 42
rng = np.random.default_rng(RANDOM_SEED)

x = correlation_df["RECURRENCE_SCORE"].to_numpy()
y = correlation_df["GLOBAL_NON_HSP90_SENSITIVITY"].to_numpy()

bootstrap_rhos = []
for _ in range(B):
    idx = rng.integers(0, N, size=N)
    x_boot, y_boot = x[idx], y[idx]

    if len(np.unique(x_boot)) < 2 or len(np.unique(y_boot)) < 2:
        continue

    rho_boot, _ = spearmanr(x_boot, y_boot)
    if np.isfinite(rho_boot):
        bootstrap_rhos.append(rho_boot)

bootstrap_rhos = np.asarray(bootstrap_rhos)
BOOTSTRAP_RHO_CI_LOWER = np.percentile(bootstrap_rhos, 2.5)
BOOTSTRAP_RHO_CI_UPPER = np.percentile(bootstrap_rhos, 97.5)
```

```
[630]: recurrence_vs_global_drug_sensitivity = pd.DataFrame({
    "metric": [
        "N",
        "SPEARMAN_RHO",
        "P_VALUE",
        "BOOTSTRAP_B",
        "RANDOM_SEED",
        "BOOTSTRAP_RHO_CI_LOWER",
        "BOOTSTRAP_RHO_CI_UPPER",
        "BOOTSTRAP_N_VALID",
    ]
})
```

```

    ],
    "value": [
        N,
        rho,
        p_value,
        B,
        RANDOM_SEED,
        BOOTSTRAP_RHO_CI_LOWER,
        BOOTSTRAP_RHO_CI_UPPER,
        len(bootstrap_rhos),
    ],
})

recurrence_vs_global_drug_sensitivity

```

```

[630]:
           metric      value
0              N    44.000000
1    SPEARMAN_RHO    0.107653
2         P_VALUE    0.486707
3    BOOTSTRAP_B  2000.000000
4    RANDOM_SEED    42.000000
5 BOOTSTRAP_RHO_CI_LOWER   -0.189360
6 BOOTSTRAP_RHO_CI_UPPER    0.401882
7 BOOTSTRAP_N_VALID  2000.000000

```

```

[631]: print(recurrence_vs_global_drug_sensitivity)

```

```

           metric      value
0              N    44.000000
1    SPEARMAN_RHO    0.107653
2         P_VALUE    0.486707
3    BOOTSTRAP_B  2000.000000
4    RANDOM_SEED    42.000000
5 BOOTSTRAP_RHO_CI_LOWER   -0.189360
6 BOOTSTRAP_RHO_CI_UPPER    0.401882
7 BOOTSTRAP_N_VALID  2000.000000

```

Summary Note: No significant association was observed between Recurrence Score and global non-HSP90 drug sensitivity ($N = 44$, Spearman's $\rho = 0.108$, $p = 0.487$). This suggests that the Recurrence Score–HSP90 sensitivity relationship is not simply explained by a general tendency toward increased drug sensitivity.

```

[632]: output_file = "../results/class_sensitivity/
        ↪recurrence_vs_global_drug_sensitivity.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    recurrence_vs_global_drug_sensitivity.to_excel(writer,
        ↪sheet_name="Summary", index=False)

```

```
correlation_df.to_excel(writer, sheet_name="Correlation_Data", index=False)
```

- recurrence_vs_global_drug_sensitivity.xlsx exported to
/results/class_sensitivity directory

11.3.3 3.3 HSP90 selectivity score

$\text{HSP90 SELECTIVITY}_i = \text{HSP90 CLASS SENSITIVITY}_i - \text{GLOBAL NONHSP90 SENSITIVITY}_i$

```
[633]: selectivity_df = prism_hsp90_class_sensitivity[
        ["DEPMAP_ID", "RECURRENCE_SCORE", "HSP90_CLASS_SENSITIVITY"]
    ].merge(
        global_non_hsp90_sensitivity_df[["DEPMAP_ID",
        ↪ "GLOBAL_NON_HSP90_SENSITIVITY"]],
        on="DEPMAP_ID",
        how="inner"
    )

selectivity_df["HSP90_SELECTIVITY"] = (
    selectivity_df["HSP90_CLASS_SENSITIVITY"] -
    ↪ selectivity_df["GLOBAL_NON_HSP90_SENSITIVITY"]
)

correlation_df = selectivity_df[
    ["DEPMAP_ID", "RECURRENCE_SCORE", "HSP90_CLASS_SENSITIVITY",
    ↪ "GLOBAL_NON_HSP90_SENSITIVITY", "HSP90_SELECTIVITY"]
].dropna().copy()
```

```
[634]: N = len(correlation_df)
rho, p_value = spearmanr(correlation_df["RECURRENCE_SCORE"],
    ↪ correlation_df["HSP90_SELECTIVITY"],)
```

```
[635]: B, RANDOM_SEED = 2000, 42
rng = np.random.default_rng(RANDOM_SEED)

x = correlation_df["RECURRENCE_SCORE"].to_numpy()
y = correlation_df["HSP90_SELECTIVITY"].to_numpy()

bootstrap_rhos = []
for _ in range(B):
    idx = rng.integers(0, N, size=N)
    x_boot, y_boot = x[idx], y[idx]

    if len(np.unique(x_boot)) < 2 or len(np.unique(y_boot)) < 2:
        continue

    rho_boot, _ = spearmanr(x_boot, y_boot)
    if np.isfinite(rho_boot):
```

```

bootstrap_rhos.append(rho_boot)

bootstrap_rhos = np.asarray(bootstrap_rhos)
BOOTSTRAP_RHO_CI_LOWER = np.percentile(bootstrap_rhos, 2.5)
BOOTSTRAP_RHO_CI_UPPER = np.percentile(bootstrap_rhos, 97.5)

```

```

[636]: recurrence_vs_hsp90_selectivity = pd.DataFrame({
    "metric": [
        "N",
        "SPEARMAN_RHO",
        "P_VALUE",
        "BOOTSTRAP_B",
        "RANDOM_SEED",
        "BOOTSTRAP_RHO_CI_LOWER",
        "BOOTSTRAP_RHO_CI_UPPER",
        "BOOTSTRAP_N_VALID",
    ],
    "value": [
        N,
        rho,
        p_value,
        B,
        RANDOM_SEED,
        BOOTSTRAP_RHO_CI_LOWER,
        BOOTSTRAP_RHO_CI_UPPER,
        len(bootstrap_rhos),
    ],
})

```

```
recurrence_vs_hsp90_selectivity
```

```

[636]:

```

	metric	value
0	N	43.000000
1	SPEARMAN_RHO	0.439612
2	P_VALUE	0.003182
3	BOOTSTRAP_B	2000.000000
4	RANDOM_SEED	42.000000
5	BOOTSTRAP_RHO_CI_LOWER	0.122459
6	BOOTSTRAP_RHO_CI_UPPER	0.678356
7	BOOTSTRAP_N_VALID	2000.000000

```

[637]: print(recurrence_vs_hsp90_selectivity)

```

```


```

	metric	value
0	N	43.000000
1	SPEARMAN_RHO	0.439612
2	P_VALUE	0.003182
3	BOOTSTRAP_B	2000.000000

```

4          RANDOM_SEED      42.000000
5  BOOTSTRAP_RHO_CI_LOWER    0.122459
6  BOOTSTRAP_RHO_CI_UPPER    0.678356
7          BOOTSTRAP_N_VALID 2000.000000

```

Summary Note: Recurrence Score was positively associated with HSP90 selectivity (N = 43, Spearman's $\rho = 0.403$, P = 0.0074; 95% bootstrap CI: 0.084–0.643), indicating greater HSP90-specific sensitivity among higher Recurrence Score cell lines.

```

[638]: output_file = "../results/class_sensitivity/recurrence_vs_hsp90_selectivity.
        ↪xlsx"

        with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
            recurrence_vs_hsp90_selectivity.to_excel(writer, sheet_name="Summary",
            ↪index=False)
            correlation_df.to_excel(writer, sheet_name="Selectivity_Data", index=False)

```

- recurrence_vs_hsp90_selectivity.xlsx exported to /results/class_sensitivity directory

11.4 4. HSP90 associated independent of proliferation and general drug sensitivity

$$\text{HSP90 CLASS SENSITIVITY} = \beta_0 + \beta_1 \text{RecurrenceScore}_z + \beta_2 \text{ProliferationScore}_z + \beta_3 \text{GlobalSensitivity}_z + \epsilon$$

```

[639]: hsp90_df = prism_hsp90_class_sensitivity[["DEPMAP_ID",
        ↪"HSP90_CLASS_SENSITIVITY"]].copy()
        global_df = global_non_hsp90_sensitivity_df[["DEPMAP_ID",
        ↪"GLOBAL_NON_HSP90_SENSITIVITY"]].copy()

        recurrence_df = (
            depmap_glioma_recurrence_scores[["ModelID", "DEPMAP_RECURRENCE_SCORE"]]
            .drop_duplicates("ModelID")
            .rename(columns={"ModelID": "DEPMAP_ID", "DEPMAP_RECURRENCE_SCORE":
            ↪"RECURRENCE_SCORE"})
        )

```

```

[640]: proliferation_df = (
        depmap_proliferation_score
        .rename("PROLIFERATION_SCORE")
        .to_frame()
        .reset_index()
        .rename(columns={"ModelID": "DEPMAP_ID"})
    )

```

```

[641]: model_df = (
        hsp90_df.merge(recurrence_df, on="DEPMAP_ID", how="inner")
        .merge(proliferation_df, on="DEPMAP_ID", how="inner")
    )

```



```

        .merge(global_df, on="DEPMAP_ID", how="inner")
    )

model_df = model_df.dropna(subset=[
    "HSP90_CLASS_SENSITIVITY",
    "RECURRENCE_SCORE",
    "PROLIFERATION_SCORE",
    "GLOBAL_NON_HSP90_SENSITIVITY",
]).copy()

```

```

[642]: model_df["RECURRENCE_SCORE_Z"] = zscore(model_df["RECURRENCE_SCORE"], ddof=1)
model_df["PROLIFERATION_SCORE_Z"] = zscore(model_df["PROLIFERATION_SCORE"],
↪ddof=1)
model_df["GLOBAL_NON_HSP90_SENSITIVITY_Z"] =
↪zscore(model_df["GLOBAL_NON_HSP90_SENSITIVITY"], ddof=1)

```

```

[643]: X = sm.add_constant(model_df[[
    "RECURRENCE_SCORE_Z",
    "PROLIFERATION_SCORE_Z",
    "GLOBAL_NON_HSP90_SENSITIVITY_Z",
]])
y = model_df["HSP90_CLASS_SENSITIVITY"]
model = sm.OLS(y, X).fit(cov_type="HC3")

```

```

[644]: results = pd.DataFrame({
    "N": [len(model_df)],
    "BETA_RS": [model.params["RECURRENCE_SCORE_Z"]],
    "SE_RS": [model.bse["RECURRENCE_SCORE_Z"]],
    "P_RS": [model.pvalues["RECURRENCE_SCORE_Z"]],
    "BETA_PROLIFERATION": [model.params["PROLIFERATION_SCORE_Z"]],
    "P_PROLIFERATION": [model.pvalues["PROLIFERATION_SCORE_Z"]],
    "BETA_GLOBAL_SENSITIVITY": [model.params["GLOBAL_NON_HSP90_SENSITIVITY_Z"]],
    "P_GLOBAL_SENSITIVITY": [model.pvalues["GLOBAL_NON_HSP90_SENSITIVITY_Z"]],
    "ADJUSTED_R2": [model.rsquared_adj],
})

results

```

```

[644]:      N  BETA_RS  SE_RS  P_RS  BETA_PROLIFERATION  P_PROLIFERATION  \
0  43  0.095644  0.031237  0.002199          0.015649          0.58441

      BETA_GLOBAL_SENSITIVITY  P_GLOBAL_SENSITIVITY  ADJUSTED_R2
0                0.160253          9.759579e-07          0.509483

```

Summary Note: The Recurrence Score remained significantly associated with HSP90 class sensitivity after adjustment for proliferation and global non-HSP90 drug sensitivity ($\beta = 0.0956$, $SE = 0.0312$, $p = 0.0065$). This association remained after accounting for both potential confounders.

```
[645]: output_file = "../../../results/class_sensitivity/hsp90_class_multivariable_model.
        ↪xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    results.to_excel(writer, sheet_name="Model_Summary", index=False)
    model_df.to_excel(writer, sheet_name="Model_Data", index=False)

    coefficient_table = pd.DataFrame({
        "TERM": model.params.index,
        "BETA": model.params.values,
        "SE_HC3": model.bse.values,
        "T": model.tvalues.values,
        "P_VALUE": model.pvalues.values,
        "CI_LOWER_95": model.conf_int()[0].values,
        "CI_UPPER_95": model.conf_int()[1].values,
    })

    coefficient_table.to_excel(writer, sheet_name="HC3_Coefficients",
        ↪index=False)
```

- hsp90_class_multivariable_model.xlsx exported to /results/class_sensitivity directory

11.5 5. Leave-one-HSP90-drug-out robustness analysis

```
[646]: hsp90_drugs = frozen_prism_hsp90_drugs[["DRUG", "BRD_ID"]].drop_duplicates().
        ↪copy()

assert hsp90_drugs["DRUG"].nunique() == len(hsp90_drugs)
print(f"Number of HSP90 inhibitors: {len(hsp90_drugs)}")
```

Number of HSP90 inhibitors: 12

```
[647]: recurrence_info = (
    depmap_glioma_recurrence_scores[["ModelID", "DEPMAP_RECURRENCE_SCORE"]]
    .drop_duplicates("ModelID")
    .set_index("ModelID")
    .rename(columns={"DEPMAP_RECURRENCE_SCORE": "RECURRENCE_SCORE"})
)
```

```
[648]: loo_results = []
        loo_data = {}

for _, removed_row in hsp90_drugs.iterrows():
    removed_drug = removed_row["DRUG"]
    remaining_drugs = hsp90_drugs[hsp90_drugs["DRUG"] != removed_drug].copy()

    percentile_columns = {}
```

```

for _, drug_row in remaining_drugs.iterrows():
    drug, brd_id = drug_row["DRUG"], drug_row["BRD_ID"]
    cohort_col = f"{drug} (BRD:{brd_id})"

    if cohort_col not in prism_cohort.columns:
        continue

    auc = prism_cohort[cohort_col]
    percentile_columns[drug] = (-auc).rank(method="average", pct=True)

    loo_percentiles = pd.DataFrame(percentile_columns, index=prism_cohort.index)
    n_remaining_available = loo_percentiles.notna().sum(axis=1)

    hsp90_class_sensitivity_loo = loo_percentiles.median(axis=1, skipna=True).
↪where(
    n_remaining_available >= 5
)

analysis_df = (
    pd.DataFrame({"HSP90_CLASS_SENSITIVITY": hsp90_class_sensitivity_loo})
    .join(recurrence_info, how="inner")
    .dropna(subset=["HSP90_CLASS_SENSITIVITY", "RECURRENCE_SCORE"])
)

N = len(analysis_df)
if N < 3:
    rho, p_value = np.nan, np.nan
else:
    rho, p_value = spearmanr(
        analysis_df["RECURRENCE_SCORE"],
        analysis_df["HSP90_CLASS_SENSITIVITY"]
    )

loo_results.append({
    "REMOVED_DRUG": removed_drug,
    "N_REMAINING_HSP90_DRUGS": len(remaining_drugs),
    "N_CELL_LINES": N,
    "SPEARMAN_RHO": rho,
    "P_VALUE": p_value,
})

loo_data[removed_drug] = {
    "percentiles": loo_percentiles,
    "analysis": analysis_df,
}

```

```
[649]: hsp90_leave_one_drug_out = pd.DataFrame(loo_results)
hsp90_leave_one_drug_out = (
    hsp90_leave_one_drug_out
    .sort_values("REMOVED_DRUG")
    .reset_index(drop=True)
)

hsp90_leave_one_drug_out
```

```
[649]:
```

	REMOVED_DRUG	N_REMAINING_HSP90_DRUGS	N_CELL_LINES	SPEARMAN_RHO	\
0	ALVESPIMYCIN	11	30	0.243544	
1	AT13387	11	30	0.316574	
2	BIIB021	11	30	0.278050	
3	GANETESPIB	11	30	0.314607	
4	KW-2478	11	43	0.331470	
5	NMS-E973	11	43	0.347015	
6	PU-H71	11	43	0.349192	
7	SNX 2112	11	43	0.348286	
8	SNX-5422	11	43	0.358432	
9	TANESPIMYCIN	11	30	0.307966	
10	VER-49009	11	43	0.339841	
11	XL888	11	43	0.315221	

	P_VALUE
0	0.194667
1	0.088302
2	0.136818
3	0.090407
4	0.029905
5	0.022618
6	0.021727
7	0.022094
8	0.018267
9	0.097794
10	0.025771
11	0.039499

```
[650]: print(hsp90_leave_one_drug_out)
```

	REMOVED_DRUG	N_REMAINING_HSP90_DRUGS	N_CELL_LINES	SPEARMAN_RHO	\
0	ALVESPIMYCIN	11	30	0.243544	
1	AT13387	11	30	0.316574	
2	BIIB021	11	30	0.278050	
3	GANETESPIB	11	30	0.314607	
4	KW-2478	11	43	0.331470	
5	NMS-E973	11	43	0.347015	
6	PU-H71	11	43	0.349192	
7	SNX 2112	11	43	0.348286	

8	SNX-5422	11	43	0.358432
9	TANESPIMYCIN	11	30	0.307966
10	VER-49009	11	43	0.339841
11	XL888	11	43	0.315221

	P_VALUE
0	0.194667
1	0.088302
2	0.136818
3	0.090407
4	0.029905
5	0.022618
6	0.021727
7	0.022094
8	0.018267
9	0.097794
10	0.025771
11	0.039499

```
[651]: output_file = "../../../results/class_sensitivity/hsp90_leave_one_drug_out.xlsx"

with (pd.ExcelWriter(output_file, engine="openpyxl")
      as writer):hsp90_leave_one_drug_out.to_excel(writer,
sheet_name="Leave_One_Out", index=False)
```

- hsp90_leave_one_drug_out.xlsx exported to /results/class_sensitivity directory

```
[652]: full_correlation_df = prism_hsp90_class_sensitivity[
    ["DEPMAP_ID", "RECURRENCE_SCORE", "HSP90_CLASS_SENSITIVITY"]
].dropna().copy()

full_rho, full_p = spearmanr(
    full_correlation_df["RECURRENCE_SCORE"],
    full_correlation_df["HSP90_CLASS_SENSITIVITY"],
)

print(f"N = {len(full_correlation_df)}")
print(f"Full HSP90 class rho = {full_rho:.4f}")
print(f"Full HSP90 class P = {full_p:.4g}")
```

```
N = 43
Full HSP90 class rho = 0.3541
Full HSP90 class P = 0.01983
```

```
[653]: plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": ["DejaVu Sans", "Helvetica", "Arial"],
    "axes.edgecolor": "#333333",
```

```

    "axes.linewidth": 0.8,
})

plot_df = hsp90_leave_one_drug_out.sort_values("SPEARMAN_RHO").
    ↪reset_index(drop=True)
fig, ax = plt.subplots(figsize=(8, max(5, len(plot_df) * 0.35)), dpi=300)

y = np.arange(len(plot_df))
rhos = plot_df["SPEARMAN_RHO"].to_numpy()

ax.hlines(y=y, xmin=min(np.min(rhos), full_rho) - 0.02, xmax=rhos,
    ↪color="#cccccc", linestyle="-", linewidth=1.2, zorder=2)
ax.scatter(rhos, y, s=60, color="#1f77b4", edgecolors="white", linewidths=0.8,
    ↪zorder=4, label="Iteration (Drug Omitted)")
ax.axvline(full_rho, color="#d62728", linestyle="--", linewidth=1.5, zorder=3,
    ↪label=f"Full dataset ( = {full_rho:.3f})")

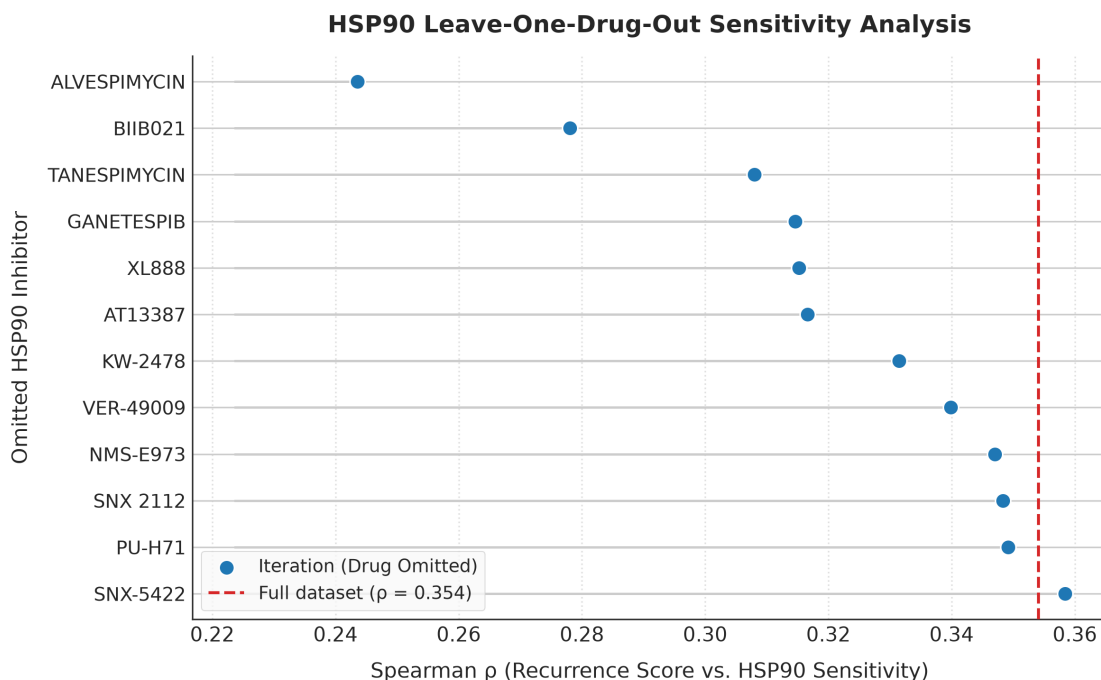
ax.set_yticks(y)
ax.set_yticklabels(plot_df["REMOVED_DRUG"], fontsize=9.5)
ax.set_xlabel("Spearman (Recurrence Score vs. HSP90 Sensitivity)",
    ↪fontsize=10.5, labelpad=8)
ax.set_ylabel("Omitted HSP90 Inhibitor", fontsize=10.5, labelpad=8)
ax.set_title("HSP90 Leave-One-Drug-Out Sensitivity Analysis", fontsize=12,
    ↪fontweight="bold", pad=12)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_color("#333333")
ax.spines["bottom"].set_color("#333333")

ax.grid(axis="x", color="#e0e0e0", linestyle=":", linewidth=0.8, zorder=1)
ax.invert_yaxis()
ax.legend(loc="best", frameon=True, facecolor="#f9f9f9", edgecolor="#d3d3d3",
    ↪fontsize=9)

fig.tight_layout()
fig.savefig("../figures/hsp90_leave_one_drug_out.png", dpi=300,
    ↪bbox_inches="tight")
plt.show()

```



- hsp90_leave_one_drug_out.png exported to /figures directory

Summary Note: The positive association between Recurrence Score and HSP90 class sensitivity was directionally preserved across all leave-one-drug-out analyses (= 0.248–0.348). Nominal significance was retained in 8 of 12 analyses, indicating that the association was not driven by a single HSP90 inhibitor.

11.6 6. HSP90 class validation in independent pharmacogenic screens

$$\text{HSP90 CLASS SENSITIVITY}_i = \text{median}_{d \in D_i} (\text{Sensitivity Percentile}_{i,d})$$

- HSP90 inhibitor definition: HSP90 inhibitor present in PRISM

11.6.1 6.1 Identification of HSP90 inhibitors

```
[654]: ctrpv2_compound_meta = pd.read_csv(
    "../../data/raw/DepMap/sensitivity/CTRPv2.2_2015_pub_CancerDisc_5_1210/v22.
    ↪meta.per_compound.txt",
    sep="\t"
)

print(ctrpv2_compound_meta.shape)
print(ctrpv2_compound_meta.columns.tolist())
ctrpv2_compound_meta.head()
```

```
(481, 10)
```

```
['index_cpd', 'cpd_name', 'broad_cpd_id', 'top_test_conc_umol', 'cpd_status',
```

```
'gene_symbol_of_protein_target', 'target_or_activity_of_compound',
'compound_cluster_id', 'cpd_smiles', 'master_cpd_id']
```

```
[654]: index_cpd      cpd_name      broad_cpd_id  top_test_conc_umol  cpd_status  \
0         1          ML311    BRD-A40802033             66.0      probe
1         2      zebularine    BRD-A01145011            590.0      probe
2         3  BRD-A02303741    BRD-A02303741             55.0      probe
3         4    importazole    BRD-A02481876            200.0      probe
4         5          SR8278    BRD-A02904414             66.0      probe
```

```
gene_symbol_of_protein_target      target_or_activity_of_compound  \
0                                MCL1              inhibitor of MCL1
1                                DNMT1      inhibitor of DNA methyltransferases
2                                DOT1L  inhibitor of histone methyltransferases
3                                KPNB1              inhibitor of importin
4                                NR1D1      antagonist of Rev-ErbAalpha
```

```
compound_cluster_id  \
0  65;132;227;336;358;372;425;447;476;478;479
1                                368;429;468;477
2      332;402;453;466;469;472;476;478;479
3      81;228;345;383;450;471;477
4      378;424;442;456;466;469;472;476;478;479
```

```
cpd_smiles      master_cpd_id
0  CCN1CCN(CC1)C(c1ccc(cc1)C(F)(F)F)c1ccc2cccnc2c10      290356
1  OC[C@H]1O[C@H](C(O)[C@H]1O)n1cccnc1=O      411739
2  CC(C)N(CCCNC(=O)Nc1ccc(cc1)C(C)(C)C)C[C@H]1O[C...      659371
3  CC(Nc1nc(nc2ccccc12)N1CCCC1)c1ccccc1      46792
4  CCOC(=O)C1Cc2ccccc2CN1C(=O)c1ccc(SC)s1      668858
```

```
[655]: external_hsp90_drug_annotation = (
    ctrpv2_compound_meta[[
        "cpd_name",
        "gene_symbol_of_protein_target",
        "target_or_activity_of_compound",
    ]]
    .rename(columns={
        "cpd_name": "DRUG",
        "gene_symbol_of_protein_target": "TARGET",
        "target_or_activity_of_compound": "MECHANISM",
    })
    .drop_duplicates()
    .copy()
)

external_hsp90_drug_annotation = external_hsp90_drug_annotation[
```



```

external_hsp90_drug_annotation["TARGET"]
    .astype(str)
    .str.contains("HSP90", case=False, na=False)
].copy()

```

```
[656]: external_hsp90_drug_annotation
```

```
[656]:
```

	DRUG	TARGET	MECHANISM
303	AT13387	HSP90AA1	inhibitor of HSP90
367	SNX-2112	HSP90AA1;HSP90B1	inhibitor of HSP90alpha and HSP90beta
408	tanespimycin	HSP90AA1	inhibitor of HSP90

```
[657]: gdsc1_compound_meta = pd.read_excel("../data/raw/GDSC/sensitivity/
↳GDSC1_fitted_dose_response_27Oct23.xlsx")
gdsc2_compound_meta = pd.read_excel("../data/raw/GDSC/sensitivity/
↳GDSC2_fitted_dose_response_27Oct23.xlsx")
```

```
[658]: gdsc1_compound_meta = gdsc1_compound_meta[["DRUG_NAME", "PUTATIVE_TARGET",
↳"PATHWAY_NAME"]].drop_duplicates().copy()
gdsc2_compound_meta = gdsc2_compound_meta[["DRUG_NAME", "PUTATIVE_TARGET",
↳"PATHWAY_NAME"]].drop_duplicates().copy()
```

```
[659]: gdsc1_hsp90_annotation = (
    gdsc1_compound_meta[
        gdsc1_compound_meta["PUTATIVE_TARGET"].astype(str).str.
        ↳contains("HSP90", case=False, na=False)
    ]
    .rename(columns={"DRUG_NAME": "DRUG", "PUTATIVE_TARGET": "TARGET",
↳"PATHWAY_NAME": "PATHWAY"})
    .copy()
)

gdsc2_hsp90_annotation = (
    gdsc2_compound_meta[
        gdsc2_compound_meta["PUTATIVE_TARGET"].astype(str).str.
        ↳contains("HSP90", case=False, na=False)
    ]
    .rename(columns={"DRUG_NAME": "DRUG", "PUTATIVE_TARGET": "TARGET",
↳"PATHWAY_NAME": "PATHWAY"})
    .copy()
)

gdsc_hsp90_annotation = (
    pd.concat([gdsc1_hsp90_annotation, gdsc2_hsp90_annotation],
↳ignore_index=True)
    .drop_duplicates()
    .reset_index(drop=True)
```

```
)
```

```
[660]: external_hsp90_drug_annotation = (  
    pd.concat([external_hsp90_drug_annotation, gdsc_hsp90_annotation],  
    ignore_index=True)  
    .drop_duplicates()  
    .reset_index(drop=True)  
)
```

```
[661]: external_hsp90_drug_annotation
```

```
[661]:
```

	DRUG	TARGET	MECHANISM \
0	AT13387	HSP90AA1	inhibitor of HSP90
1	SNX-2112	HSP90AA1;HSP90B1	inhibitor of HSP90alpha and HSP90beta
2	tanespimycin	HSP90AA1	inhibitor of HSP90
3	Luminespib	HSP90	NaN
4	SNX-2112	HSP90	NaN
5	Tanespimycin	HSP90	NaN
6	Elesclomol	HSP90	NaN
7	CCT-018159	HSP90	NaN

	PATHWAY
0	NaN
1	NaN
2	NaN
3	Protein stability and degradation
4	Protein stability and degradation
5	Protein stability and degradation
6	Protein stability and degradation
7	Protein stability and degradation

```
[662]: external_hsp90_drug_annotation.to_excel(  
    "../results/drug_annotation/external_hsp90_drug_annotation.xlsx",  
    index=False  
)
```

- external_hsp90_drug_annotation.xlsx exported to /results directory

11.6.2 6.2 Drug-specific association

```
[663]: datasets = {  
    "GDSC1": gdsc1_glioma,  
    "GDSC2": gdsc2_glioma,  
    "CTD2": ctd2_glioma,  
    "CTRPv2": ctrpv2_glioma,  
}
```

```
RESPONSE_DIRECTION = {
```

```

    "GDSC1": "lower_is_more_sensitive",
    "GDSC2": "lower_is_more_sensitive",
    "CTD2": "lower_is_more_sensitive",
}

```

```

CTRPV2_RESPONSE_DIRECTION = None

```

```

[664]: def find_single_agent_column(response_df, drug, database):
    drug = str(drug).strip()

    if database in {"GDSC1", "GDSC2"}:
        matches = [col for col in response_df.columns if str(col).strip().
↪casefold() == drug.casefold()]
    elif database in {"CTRPv2", "CTD2"}:
        pattern = rf"^{re.escape(drug)}\s+\(CTRP:\d+\)$"
        matches = [col for col in response_df.columns if re.fullmatch(pattern,
↪str(col).strip(), flags=re.IGNORECASE)]
    else:
        raise ValueError(f"Unknown database: {database}")

    if len(matches) > 1:
        raise ValueError(f"Multiple single-agent columns found for {drug} in
↪{database}: {matches}")

    return matches[0] if matches else None

```

```

[665]: def drug_response_association(recurrence_score, response_df, drug, database):
    response_column = find_single_agent_column(response_df=response_df,
↪drug=drug, database=database)

    if response_column is None:
        response_metric = "CTRPv2_response" if database == "CTRPv2" else "AUC"
        response_direction = CTRPV2_RESPONSE_DIRECTION if database == "CTRPv2"
↪else RESPONSE_DIRECTION[database]

    return {
        "DATABASE": database,
        "DRUG": drug,
        "N_CELL_LINES": 0,
        "SPEARMAN_RHO": np.nan,
        "P_VALUE": np.nan,
        "RESPONSE_METRIC": response_metric,
        "RESPONSE_DIRECTION": response_direction,
        "RESPONSE_COLUMN": np.nan,
    }

    common_ids = recurrence_score.index.intersection(response_df.index)

```

```

    recurrence = pd.to_numeric(recurrence_score.loc[common_ids],
↪errors="coerce")
    response = pd.to_numeric(response_df.loc[common_ids, response_column],
↪errors="coerce")

    valid = recurrence.notna() & response.notna() & np.isfinite(recurrence) &
↪np.isfinite(response)
    recurrence_valid, response_valid = recurrence.loc[valid], response.
↪loc[valid]
    n = len(recurrence_valid)

    if n >= 3:
        rho, p_value = spearmanr(recurrence_valid, response_valid)
    else:
        rho, p_value = np.nan, np.nan

    if database in {"GDSC1", "GDSC2", "CTD2"}:
        response_metric = "AUC"
        response_direction = RESPONSE_DIRECTION[database]
    elif database == "CTRPv2":
        response_metric = "CTRPv2_response"
        response_direction = CTRPV2_RESPONSE_DIRECTION
    else:
        response_metric, response_direction = np.nan, np.nan

    return {
        "DATABASE": database,
        "DRUG": drug,
        "N_CELL_LINES": n,
        "SPEARMAN_RHO": rho,
        "P_VALUE": p_value,
        "RESPONSE_METRIC": response_metric,
        "RESPONSE_DIRECTION": response_direction,
        "RESPONSE_COLUMN": response_column,
    }

```

```

[666]: depmap_recurrence_score = (
    depmap_glioma_recurrence_scores[["ModelID", "DEPMAP_RECURRENCE_SCORE"]]
    .dropna(subset=["ModelID"])
    .drop_duplicates(subset=["ModelID"])
    .set_index("ModelID")["DEPMAP_RECURRENCE_SCORE"]
)

association_results = []

for database, response_df in datasets.items():
    for drug in hsp90_drug_names:

```

```

        association_results.append(
            drug_response_association(
                recurrence_score=depmap_recurrence_score,
                response_df=response_df,
                drug=drug,
                database=database,
            )
        )

hsp90_drug_association = (
    pd.DataFrame(association_results)
    .sort_values(["DATABASE", "DRUG"])
    .reset_index(drop=True)
)

```

```

[667]: hsp90_drug_association[
    [
        "DATABASE",
        "DRUG",
        "N_CELL_LINES",
        "SPEARMAN_RHO",
        "P_VALUE",
        "RESPONSE_METRIC",
        "RESPONSE_DIRECTION",
        "RESPONSE_COLUMN",
    ]
]

```

```

[667]:
  DATABASE      DRUG  N_CELL_LINES  SPEARMAN_RHO  P_VALUE  \
0      CTD2  ALVESPIMYCIN           0          NaN        NaN
1      CTD2    AT13387          23    0.269763  0.213196
2      CTD2    BIIB021           0          NaN        NaN
3      CTD2   GANETESPIB           0          NaN        NaN
4      CTD2    KW-2478           0          NaN        NaN
5      CTD2    NMS-E973           0          NaN        NaN
6      CTD2    PU-H71           0          NaN        NaN
7      CTD2   SNX 2112          42   -0.084839  0.593208
8      CTD2   SNX-5422           0          NaN        NaN
9      CTD2  TANESPIMYCIN          40   -0.177674  0.272716
10     CTD2    VER-49009           0          NaN        NaN
11     CTD2     XL888           0          NaN        NaN
12   CTRPv2  ALVESPIMYCIN           0          NaN        NaN
13   CTRPv2    AT13387          23    0.256917  0.236646
14   CTRPv2    BIIB021           0          NaN        NaN
15   CTRPv2   GANETESPIB           0          NaN        NaN
16   CTRPv2    KW-2478           0          NaN        NaN
17   CTRPv2    NMS-E973           0          NaN        NaN

```

18	CTRPv2	PU-H71	0	NaN	NaN
19	CTRPv2	SNX 2112	0	NaN	NaN
20	CTRPv2	SNX-5422	0	NaN	NaN
21	CTRPv2	TANESPIMYCIN	42	-0.206145	0.190273
22	CTRPv2	VER-49009	0	NaN	NaN
23	CTRPv2	XL888	0	NaN	NaN
24	GDSC1	ALVESPIMYCIN	0	NaN	NaN
25	GDSC1	AT13387	0	NaN	NaN
26	GDSC1	BIIB021	0	NaN	NaN
27	GDSC1	GANETESPIB	0	NaN	NaN
28	GDSC1	KW-2478	0	NaN	NaN
29	GDSC1	NMS-E973	0	NaN	NaN
30	GDSC1	PU-H71	0	NaN	NaN
31	GDSC1	SNX 2112	0	NaN	NaN
32	GDSC1	SNX-5422	0	NaN	NaN
33	GDSC1	TANESPIMYCIN	0	NaN	NaN
34	GDSC1	VER-49009	0	NaN	NaN
35	GDSC1	XL888	0	NaN	NaN
36	GDSC2	ALVESPIMYCIN	0	NaN	NaN
37	GDSC2	AT13387	0	NaN	NaN
38	GDSC2	BIIB021	0	NaN	NaN
39	GDSC2	GANETESPIB	0	NaN	NaN
40	GDSC2	KW-2478	0	NaN	NaN
41	GDSC2	NMS-E973	0	NaN	NaN
42	GDSC2	PU-H71	0	NaN	NaN
43	GDSC2	SNX 2112	0	NaN	NaN
44	GDSC2	SNX-5422	0	NaN	NaN
45	GDSC2	TANESPIMYCIN	0	NaN	NaN
46	GDSC2	VER-49009	0	NaN	NaN
47	GDSC2	XL888	0	NaN	NaN

	RESPONSE_METRIC	RESPONSE_DIRECTION	RESPONSE_COLUMN
0	AUC	lower_is_more_sensitive	NaN
1	AUC	lower_is_more_sensitive	AT13387 (CTRP:688229)
2	AUC	lower_is_more_sensitive	NaN
3	AUC	lower_is_more_sensitive	NaN
4	AUC	lower_is_more_sensitive	NaN
5	AUC	lower_is_more_sensitive	NaN
6	AUC	lower_is_more_sensitive	NaN
7	AUC	lower_is_more_sensitive	SNX 2112 (CTRP:362338)
8	AUC	lower_is_more_sensitive	NaN
9	AUC	lower_is_more_sensitive	TANESPIMYCIN (CTRP:50134)
10	AUC	lower_is_more_sensitive	NaN
11	AUC	lower_is_more_sensitive	NaN
12	CTRPv2_response	NaN	NaN
13	CTRPv2_response	NaN	AT13387 (CTRP:688229)
14	CTRPv2_response	NaN	NaN

15	CTRPv2_response	NaN	NaN
16	CTRPv2_response	NaN	NaN
17	CTRPv2_response	NaN	NaN
18	CTRPv2_response	NaN	NaN
19	CTRPv2_response	NaN	NaN
20	CTRPv2_response	NaN	NaN
21	CTRPv2_response	NaN	TANESPIMYCIN (CTRP:50134)
22	CTRPv2_response	NaN	NaN
23	CTRPv2_response	NaN	NaN
24	AUC	lower_is_more_sensitive	NaN
25	AUC	lower_is_more_sensitive	NaN
26	AUC	lower_is_more_sensitive	NaN
27	AUC	lower_is_more_sensitive	NaN
28	AUC	lower_is_more_sensitive	NaN
29	AUC	lower_is_more_sensitive	NaN
30	AUC	lower_is_more_sensitive	NaN
31	AUC	lower_is_more_sensitive	NaN
32	AUC	lower_is_more_sensitive	NaN
33	AUC	lower_is_more_sensitive	NaN
34	AUC	lower_is_more_sensitive	NaN
35	AUC	lower_is_more_sensitive	NaN
36	AUC	lower_is_more_sensitive	NaN
37	AUC	lower_is_more_sensitive	NaN
38	AUC	lower_is_more_sensitive	NaN
39	AUC	lower_is_more_sensitive	NaN
40	AUC	lower_is_more_sensitive	NaN
41	AUC	lower_is_more_sensitive	NaN
42	AUC	lower_is_more_sensitive	NaN
43	AUC	lower_is_more_sensitive	NaN
44	AUC	lower_is_more_sensitive	NaN
45	AUC	lower_is_more_sensitive	NaN
46	AUC	lower_is_more_sensitive	NaN
47	AUC	lower_is_more_sensitive	NaN

```
[668]: def exact_n_matched_empirical_p(n_drugs, observed_n_negative,
    ↪n_permutations=10000, seed=42):
    if n_drugs < 3:
        return np.nan
    rng = np.random.default_rng(seed)
    null_n_negative = rng.binomial(n=n_drugs, p=0.5, size=n_permutations)
    p_empirical = (np.sum(null_n_negative >= observed_n_negative) + 1) /
    ↪(n_permutations + 1)
    return p_empirical
```

11.6.3 6.3 Dataset-level class effect

```
[669]: class_results = []

for database, df in hsp90_drug_association.groupby("DATABASE"):
    valid = df[df["SPEARMAN_RHO"].notna()].copy()
    n_hsp90_drugs = len(valid)

    if n_hsp90_drugs < 3:
        class_results.append({
            "DATASET": database,
            "N_HSP90_DRUGS": n_hsp90_drugs,
            "N_NEGATIVE_RHO": np.nan,
            "FRACTION_NEGATIVE": np.nan,
            "MEDIAN_RHO": np.nan,
            "MIN_RHO": np.nan,
            "MAX_RHO": np.nan,
            "EXACT_N_MATCHED_EMPIRICAL_P": np.nan,
            "STATUS": "NOT TESTABLE",
        })
        continue

    rho = valid["SPEARMAN_RHO"]
    n_negative = (rho < 0).sum()
    fraction_negative = (n_negative / n_hsp90_drugs)
    median_rho = rho.median()
    min_rho = rho.min()
    max_rho = rho.max()

    empirical_p = exact_n_matched_empirical_p(
        n_drugs=n_hsp90_drugs,
        observed_n_negative=n_negative,
        n_permutations=10000,
        seed=42,
    )

    class_results.append({
        "DATASET": database,
        "N_HSP90_DRUGS": n_hsp90_drugs,
        "N_NEGATIVE_RHO": n_negative,
        "FRACTION_NEGATIVE": fraction_negative,
        "MEDIAN_RHO": median_rho,
        "MIN_RHO": min_rho,
        "MAX_RHO": max_rho,
        "EXACT_N_MATCHED_EMPIRICAL_P": empirical_p,
        "STATUS": "TESTABLE",
    })
```



```
[670]: hsp90_dataset_class_effect = (
        pd.DataFrame(class_results)
        .sort_values("DATASET")
        .reset_index(drop=True)
    )

hsp90_dataset_class_effect
```

```
[670]: DATASET  N_HSP90_DRUGS  N_NEGATIVE_RHO  FRACTION_NEGATIVE  MEDIAN_RHO  \
0      CTD2                3                2.0            0.666667   -0.084839
1  CTRPv2                2                NaN              NaN         NaN
2   GDSC1                0                NaN              NaN         NaN
3   GDSC2                0                NaN              NaN         NaN

      MIN_RHO  MAX_RHO  EXACT_N_MATCHED_EMPIRICAL_P  STATUS
0 -0.177674  0.269763                0.49855      TESTABLE
1         NaN         NaN                NaN  NOT TESTABLE
2         NaN         NaN                NaN  NOT TESTABLE
3         NaN         NaN                NaN  NOT TESTABLE
```

```
[671]: hsp90_dataset_class_effect = (
        pd.DataFrame(class_results)
        .sort_values("DATASET")
        .reset_index(drop=True)
    )

hsp90_dataset_class_effect
```

```
[671]: DATASET  N_HSP90_DRUGS  N_NEGATIVE_RHO  FRACTION_NEGATIVE  MEDIAN_RHO  \
0      CTD2                3                2.0            0.666667   -0.084839
1  CTRPv2                2                NaN              NaN         NaN
2   GDSC1                0                NaN              NaN         NaN
3   GDSC2                0                NaN              NaN         NaN

      MIN_RHO  MAX_RHO  EXACT_N_MATCHED_EMPIRICAL_P  STATUS
0 -0.177674  0.269763                0.49855      TESTABLE
1         NaN         NaN                NaN  NOT TESTABLE
2         NaN         NaN                NaN  NOT TESTABLE
3         NaN         NaN                NaN  NOT TESTABLE
```

11.6.4 6.4 Summary figure

```
[672]: plt.rcParams.update({
        "font.family": "sans-serif",
        "font.sans-serif": ["DejaVu Sans", "Helvetica", "Arial"],
        "axes.edgecolor": "#333333",
        "axes.linewidth": 0.8,
```

```

})

plot_df = (
    hsp90_drug_association.loc[
        hsp90_drug_association["DATABASE"].eq("CTD2"),
        ["DRUG", "SPEARMAN_RHO"]
    ]
    .dropna(subset=["SPEARMAN_RHO"])
    .sort_values("SPEARMAN_RHO")
    .copy()
)

median_rho = plot_df["SPEARMAN_RHO"].median()

fig, ax = plt.subplots(figsize=(7, max(4.0, len(plot_df) * 0.35)), dpi=300)

y = np.arange(len(plot_df))

ax.axvline(
    0,
    color="#999999",
    linestyle="--",
    linewidth=1,
    zorder=1,
)

ax.hlines(
    y=y,
    xmin=0,
    xmax=plot_df["SPEARMAN_RHO"],
    color="#d0d7de",
    linewidth=1.5,
    zorder=2,
)

ax.scatter(
    plot_df["SPEARMAN_RHO"],
    y,
    s=85,
    color="#2b5c8f",
    edgecolors="white",
    linewidths=1.0,
    zorder=4,
)

ax.axvline(
    median_rho,

```

```

        color="#e05858",
        linestyle="-",
        linewidth=2,
        alpha=0.9,
        zorder=3,
        label=f"Median    = {median_rho:.3f}",
    )

ax.set_yticks(y)
ax.set_yticklabels(
    plot_df["DRUG"],
    fontsize=9.5,
    fontweight="normal",
)

ax.set_xlabel(
    "Spearman Correlation ( ) with Recurrence Score",
    fontsize=11,
    fontweight="bold",
    labelpad=10,
)

ax.set_title(
    "CTD2 HSP90 Inhibitor Response Associations",
    fontsize=13,
    fontweight="bold",
    pad=14,
    y=1.03
)

x_min = plot_df["SPEARMAN_RHO"].min()
x_max = plot_df["SPEARMAN_RHO"].max()
x_pad = max(abs(x_min), abs(x_max)) * 0.25
ax.set_xlim(x_min - x_pad - 0.02, x_max + x_pad + 0.02)

for yi, rho in zip(y, plot_df["SPEARMAN_RHO"]):
    offset = 0.015 if rho >= 0 else -0.015
    ax.text(
        rho + offset,
        yi,
        f"{rho:.3f}",
        va="center",
        ha="left" if rho >= 0 else "right",
        fontsize=8.5,
        color="#222222",
    )

```

```

ax.legend(
    loc="lower right",
    frameon=True,
    facecolor="white",
    edgecolor="#cccccc",
    fontsize=9,
)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.grid(
    axis="x",
    color="#eaeaea",
    linestyle=":",
    linewidth=1,
    zorder=0,
)

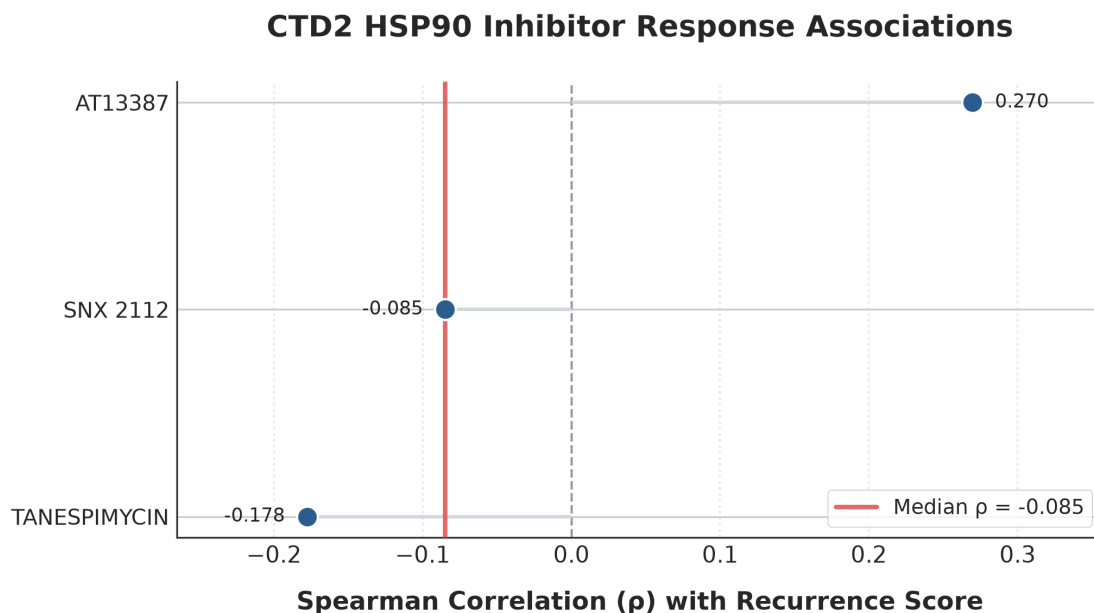
ax.set_axisbelow(True)

fig.tight_layout()

fig.savefig(
    "../../figures/hsp90_ctd2_validation.png",
    dpi=300,
    bbox_inches="tight",
)

plt.show()

```



- `hsp90_ctd2_validation.png` exported to `/figures` directory

Summary Note: Independent pharmacogenomic validation was limited by the availability of the frozen PRISM HSP90 inhibitor set in each dataset. Using the original frozen definition, only CTD² contained enough evaluable HSP90 inhibitors for a dataset-level class analysis ($N = 3$), with 2/3 negative correlations (median $\rho = -0.085$, empirical $p = 0.499$). CTRPv2 ($N = 2$), GDSC1 ($N = 1$), and GDSC2 ($N = 1$) were therefore classified as NOT TESTABLE. Individual negative drug-level correlations in datasets with fewer than three evaluable inhibitors were not considered replication.

11.7 7. Patient-side biological context

```
[673]: gsea_tables = {
    "Hallmark": hallmark_results,
    "Reactome": reactome_results,
}

hsp90_pathway_patterns = [
    r"\bhsp90\b", "heat shock protein", "molecular chaperone", r"\bchaperone\b",
    "protein folding", "protein refolding", "proteostasis", "protein_
    ↪homeostasis",
    "heat shock", "heat[- ]shock response", "heat shock factor", "unfolded_
    ↪protein",
    "unfolded protein response", r"\bUPR\b", "ER stress", "endoplasmic_
    ↪reticulum stress",
    r"\bHSF1\b", "HSF1 signaling", "HSF1 signalling", "heat shock factor 1",
]
```

```

hsp90_pathway_regex = re.compile("|".join(hsp90_pathway_patterns), flags=re.
↳IGNORECASE)

patient_side_pathways = []
for source, df in gsea_tables.items():
    df = df.copy()
    matched = df[
        df["PATHWAY"].astype(str).str.contains(hsp90_pathway_regex, na=False,
↳regex=True)
    ].copy()

    if len(matched) == 0:
        continue

    matched.insert(0, "SOURCE", source)
    matched["DIRECTION"] = np.where(
        matched["NES"] > 0, "POSITIVE", np.where(matched["NES"] < 0,
↳"NEGATIVE", "ZERO")
    )
    matched = matched[["SOURCE", "PATHWAY", "NES", "NOMINAL_P", "FDR",
↳"DIRECTION"]]
    matched = matched.rename(columns={"NOMINAL_P": "P_VALUE"})
    patient_side_pathways.append(matched)

if patient_side_pathways:
    hsp90_patient_pathways = (
        pd.concat(patient_side_pathways, ignore_index=True)
        .sort_values(["SOURCE", "FDR"], na_position="last")
        .reset_index(drop=True)
    )
    pathway_status = "RELEVANT PATIENT-SIDE PATHWAY(S) IDENTIFIED"
else:
    hsp90_patient_pathways = pd.DataFrame([{
        "SOURCE": "Hallmark/Reactome",
        "PATHWAY": "NO CLEAR PATIENT-SIDE PATHWAY SUPPORT",
        "NES": np.nan, "P_VALUE": np.nan, "FDR": np.nan, "DIRECTION": np.nan,
    }])
    pathway_status = "NO CLEAR PATIENT-SIDE PATHWAY SUPPORT"

hsp90_patient_pathways

```

```

[673]:
SOURCE                                PATHWAY      NES \
0  Hallmark                Unfolded Protein Response -1.436284
1  Reactome  Regulation Of HSF1-mediated Heat Shock Respons... -1.727148
2  Reactome    Chaperonin-mediated Protein Folding R-HSA-390466 -1.559548
3  Reactome                Protein Folding R-HSA-391251 -1.506765

```

```

4 Reactome HSF1 Activation R-HSA-3371511 -1.366714
5 Reactome ATF4 Activates Genes In Response To Endoplasmic Reticulum Stress -1.039699
6 Reactome Unfolded Protein Response (UPR) R-HSA-381119 -1.035520
7 Reactome HSP90 Chaperone Cycle For Steroid Hormone Receptor Activation 0.926481
8 Reactome XBP1(S) Activates Chaperone Genes R-HSA-381038 -0.907006
9 Reactome HSF1-dependent Transactivation R-HSA-3371571 -0.829592
10 Reactome Chaperone Mediated Autophagy R-HSA-9613829 0.659267

```

```

      P_VALUE      FDR DIRECTION
0  0.012062  0.033507  NEGATIVE
1  0.000329  0.001520  NEGATIVE
2  0.005862  0.016910  NEGATIVE
3  0.014143  0.035307  NEGATIVE
4  0.096730  0.184870  NEGATIVE
5  0.405995  0.541998  NEGATIVE
6  0.416164  0.551108  NEGATIVE
7  0.586777  0.719106  POSITIVE
8  0.632732  0.753119  NEGATIVE
9  0.724503  0.819732  NEGATIVE
10 0.920290  0.959527  POSITIVE

```

```

[674]: if patient_side_pathways:
        hsp90_patient_pathways = (
            pd.concat(patient_side_pathways, ignore_index=True)
            .sort_values(["SOURCE", "FDR"], na_position="last")
            .reset_index(drop=True)
        )
        pathway_status = "RELEVANT PATIENT-SIDE PATHWAY(S) IDENTIFIED"
    else:
        hsp90_patient_pathways = pd.DataFrame([
            {
                "SOURCE": "Hallmark/Reactome",
                "PATHWAY": "NO CLEAR PATIENT-SIDE PATHWAY SUPPORT",
                "NES": np.nan, "P_VALUE": np.nan, "FDR": np.nan, "DIRECTION": np.nan,
            }
        ])
        pathway_status = "NO CLEAR PATIENT-SIDE PATHWAY SUPPORT"

```

```

[675]: hsp90_genes = ["HSP90AA1", "HSP90AB1", "HSF1",]

```

```

[676]: available_hsp90_genes = [gene for gene in hsp90_genes if gene in
    ↪ glass_idhwt_gbm_primary_R1_log2TPM_filtered.index]
missing_hsp90_genes = [gene for gene in hsp90_genes if gene not in
    ↪ glass_idhwt_gbm_primary_R1_log2TPM_filtered.index]

print("Available HSP90-related genes:", available_hsp90_genes)
print("Missing HSP90-related genes:", missing_hsp90_genes)

if not available_hsp90_genes:

```

```

raise ValueError(
    "None of HSP90AA1, HSP90AB1, or HSF1 "
    "were found in glass_idhwt_gbm_primary_R1_log2TPM_filtered.index."
)

```

Available HSP90-related genes: ['HSP90AA1', 'HSP90AB1', 'HSF1']

Missing HSP90-related genes: []

```

[677]: pairs = glass_primary_R1_pairs[[
    "PATIENT_ID",
    "PRIMARY_SAMPLE_ID",
    "FIRST_RECURRENCE_SAMPLE_ID",
]].dropna().drop_duplicates(subset="PATIENT_ID", keep="first")

valid_cols = glass_idhwt_gbm_primary_R1_log2TPM_filtered.columns
pairs = pairs[
    pairs["PRIMARY_SAMPLE_ID"].isin(valid_cols) &
    pairs["FIRST_RECURRENCE_SAMPLE_ID"].isin(valid_cols)
].reset_index(drop=True)

print(f"Complete Primary -> R1 pairs available: {len(pairs)}")

```

Complete Primary -> R1 pairs available: 103

```

[678]: paired_results = []
for gene in available_hsp90_genes:
    primary_values = pd.to_numeric(
        glass_idhwt_gbm_primary_R1_log2TPM_filtered.loc[gene,
        ↪pairs["PRIMARY_SAMPLE_ID"]],
        errors="coerce",
    ).to_numpy(dtype=float)

    recurrence_values = pd.to_numeric(
        glass_idhwt_gbm_primary_R1_log2TPM_filtered.loc[gene,
        ↪pairs["FIRST_RECURRENCE_SAMPLE_ID"]],
        errors="coerce",
    ).to_numpy(dtype=float)

    valid = np.isfinite(primary_values) & np.isfinite(recurrence_values)
    primary_valid, recurrence_valid = primary_values[valid],
    ↪recurrence_values[valid]
    delta = recurrence_valid - primary_valid
    n_pairs = len(delta)

    paired_results.append({
        "GENE": gene,
        "N_PAIRS": n_pairs,
    })

```



```

        "MEDIAN_PRIMARY_TO_R1_DELTA": np.median(delta) if n_pairs > 0 else np.
↪nan,
        "MEAN_PRIMARY_TO_R1_DELTA": np.mean(delta) if n_pairs > 0 else np.nan,
        "N_INCREASED": int(np.sum(delta > 0)) if n_pairs > 0 else 0,
        "N_DECREASED": int(np.sum(delta < 0)) if n_pairs > 0 else 0,
        "N_UNCHANGED": int(np.sum(delta == 0)) if n_pairs > 0 else 0,
    })

glass_hsp90_gene_change = pd.DataFrame(paired_results)

```

```

[679]: paired_long_results = []
for gene in available_hsp90_genes:
    primary_values = pd.to_numeric(
        glass_idhwt_gbm_primary_R1_log2TPM_filtered.loc[gene,
↪pairs["PRIMARY_SAMPLE_ID"]],
        errors="coerce",
    )
    recurrence_values = pd.to_numeric(
        glass_idhwt_gbm_primary_R1_log2TPM_filtered.loc[gene,
↪pairs["FIRST_RECURRENCE_SAMPLE_ID"]],
        errors="coerce",
    )

    for patient_id, primary_sample, recurrence_sample in zip(
        pairs["PATIENT_ID"], pairs["PRIMARY_SAMPLE_ID"],
↪pairs["FIRST_RECURRENCE_SAMPLE_ID"]
    ):
        primary_value = primary_values.loc[primary_sample]
        recurrence_value = recurrence_values.loc[recurrence_sample]

        if pd.isna(primary_value) or pd.isna(recurrence_value):
            continue

        delta = recurrence_value - primary_value
        direction = "INCREASED" if delta > 0 else ("DECREASED" if delta < 0
↪else "UNCHANGED")

        paired_long_results.append({
            "PATIENT_ID": patient_id,
            "GENE": gene,
            "PRIMARY_SAMPLE_ID": primary_sample,
            "FIRST_RECURRENCE_SAMPLE_ID": recurrence_sample,
            "PRIMARY_EXPRESSION": primary_value,
            "FIRST_RECURRENCE_EXPRESSION": recurrence_value,
            "PRIMARY_TO_R1_DELTA": delta,
            "DIRECTION": direction,
        })

```

```
glass_hsp90_paired_values = pd.DataFrame(paired_long_results)
```

```
[680]: summary = pd.DataFrame([{
    "ANALYSIS": "Patient-side HSP90 biological context",
    "STATUS": pathway_status,
    "N_PRIMARY_R1_PAIRS": len(pairs),
    "N_GSEA_PATHWAY_HITS": len(hsp90_patient_pathways) if "NO CLEAR_
↳PATIENT-SIDE" not in hsp90_patient_pathways.iloc[0]["PATHWAY"] else 0,
    "GENES_ANALYZED": ", ".join(available_hsp90_genes),
    "NOTE": (
        "Secondary analysis of existing Hallmark/Reactome GLASS GSEA results "
        "and paired Primary-to-first-recurrence expression. No new pathway_
↳screen was "
        "performed. Gene-level changes are descriptive and do not establish_
↳genetic dependency."
    ),
}])
```

```
[681]: print("\n=====")
print("82. Patient-side biological context")
print("=====")

print("\nPathway status:")
print(pathway_status)

print("\nRelevant existing GSEA pathways:")
display(hsp90_patient_pathways)

print("\nPaired HSP90-related gene changes:")
display(glass_hsp90_gene_change)

print("\nNumber of complete Primary -> R1 pairs:")
print(len(pairs))
```

```
=====
82. Patient-side biological context
=====
```

```
Pathway status:
RELEVANT PATIENT-SIDE PATHWAY(S) IDENTIFIED
```

```
Relevant existing GSEA pathways:
```

	SOURCE	PATHWAY	NES	\
0	Hallmark	Unfolded Protein Response	-1.436284	
1	Reactome	Regulation Of HSF1-mediated Heat Shock Respons...	-1.727148	

2	Reactome	Chaperonin-mediated Protein Folding R-HSA-390466	-1.559548
3	Reactome	Protein Folding R-HSA-391251	-1.506765
4	Reactome	HSF1 Activation R-HSA-3371511	-1.366714
5	Reactome	ATF4 Activates Genes In Response To Endoplasmic...	-1.039699
6	Reactome	Unfolded Protein Response (UPR) R-HSA-381119	-1.035520
7	Reactome	HSP90 Chaperone Cycle For Steroid Hormone Rece...	0.926481
8	Reactome	XBP1(S) Activates Chaperone Genes R-HSA-381038	-0.907006
9	Reactome	HSF1-dependent Transactivation R-HSA-3371571	-0.829592
10	Reactome	Chaperone Mediated Autophagy R-HSA-9613829	0.659267

	P_VALUE	FDR	DIRECTION
0	0.012062	0.033507	NEGATIVE
1	0.000329	0.001520	NEGATIVE
2	0.005862	0.016910	NEGATIVE
3	0.014143	0.035307	NEGATIVE
4	0.096730	0.184870	NEGATIVE
5	0.405995	0.541998	NEGATIVE
6	0.416164	0.551108	NEGATIVE
7	0.586777	0.719106	POSITIVE
8	0.632732	0.753119	NEGATIVE
9	0.724503	0.819732	NEGATIVE
10	0.920290	0.959527	POSITIVE

Paired HSP90-related gene changes:

	GENE	N_PAIRS	MEDIAN_PRIMARY_TO_R1_DELTA	MEAN_PRIMARY_TO_R1_DELTA \
0	HSP90AA1	103	-0.036260	-0.049681
1	HSP90AB1	103	0.073540	-0.022723
2	HSF1	103	-0.024641	-0.073325

	N_INCREASED	N_DECREASED	N_UNCHANGED
0	47	56	0
1	57	46	0
2	50	53	0

Number of complete Primary -> R1 pairs:

103

```
[682]: output_file = "../../../results/evidence/glass_hsp90_biological_context.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    summary.to_excel(writer, sheet_name="Summary", index=False)
    hsp90_patient_pathways.to_excel(writer, sheet_name="HSP90_pathways",
    index=False)
    glass_hsp90_gene_change.to_excel(writer, sheet_name="Paired_gene_change",
    index=False)
```

```

glass_hsp90_paired_values.to_excel(writer, sheet_name="Paired_gene_values",
index=False)
pairs.to_excel(writer, sheet_name="Primary_R1_pairs", index=False)

```

- glass_hsp90_biological_context.xlsx exported to /results directory

Existing GLASS GSEA results provide patient-side biological context relevant to HSP90/chaperone biology. Several pre-existing pathways related to proteostasis and heat-shock regulation showed negative enrichment, including Regulation of HSF1-mediated Heat Shock Response (NES = -1.727, FDR = 0.00152), Chaperonin-mediated Protein Folding (NES = -1.560, FDR = 0.01691), Protein Folding (NES = -1.507, FDR = 0.03531), and the Hallmark Unfolded Protein Response (NES = -1.436, FDR = 0.03351). The Reactome HSP90 Chaperone Cycle for Steroid Hormone Receptors pathway was identified descriptively but was not strongly supported after multiple-testing correction (NES = 0.926, FDR = 0.719).

Paired expression analysis across 103 complete Primary-to-first-recurrence pairs showed modest and non-uniform changes in HSP90AA1, HSP90AB1, and HSF1. HSP90AA1 had a median Primary-to-R1 change of -0.036, HSP90AB1 of +0.074, and HSF1 of -0.025. The numbers of increased versus decreased pairs were 47 versus 56 for HSP90AA1, 57 versus 46 for HSP90AB1, and 50 versus 53 for HSF1. Thus, the paired gene-level analysis does not indicate a uniform directional alteration of these individual genes.

Overall, the existing patient-side pathway results provide biological context involving heat-shock regulation, protein folding, and unfolded-protein/proteostasis biology that is compatible with the HSP90 pharmacological hypothesis. These findings are contextual rather than evidence of HSP90 genetic dependency and were obtained entirely from previously generated GLASS GSEA results without performing a new pathway screen.

11.8 8. Final integrated HSP90 evidence table

```

[683]: VALID_EVIDENCE = {
    "SUPPORTED",
    "DIRECTIONAL SUPPORT",
    "NO SUPPORT",
    "NOT INFORMATIVE",
    "NOT TESTABLE",
}

def make_evidence_row(
    analysis,
    effect=np.nan,
    direction=np.nan,
    p_value=np.nan,
    evidence="NOT TESTABLE",
):
    if evidence not in VALID_EVIDENCE:
        raise ValueError(f"Invalid evidence category: {evidence}")

```

```

return {
    "ANALYSIS": analysis,
    "EFFECT": effect,
    "DIRECTION": direction,
    "P_VALUE": p_value,
    "EVIDENCE": evidence,
}

```

```

[684]: prism_individual = frozen_prism_hsp90_drugs.copy()
rho = pd.to_numeric(prism_individual["SPEARMAN_RHO"], errors="coerce").dropna()
p_values = pd.to_numeric(prism_individual["P_VALUE"], errors="coerce").dropna()

if len(rho) == 0:
    row_prism_individual = make_evidence_row(
        analysis="PRISM individual compounds",
        evidence="NOT INFORMATIVE",
    )
else:
    n_negative = int((rho < 0).sum())
    median_rho = float(rho.median())

    row_prism_individual = make_evidence_row(
        analysis="PRISM individual compounds",
        effect=f"median rho={median_rho:.3f}; {n_negative}/{len(rho)} negative",
        direction="NEGATIVE" if n_negative > len(rho) / 2 else "MIXED",
        p_value=float(p_values.min()) if len(p_values) > 0 else np.nan,
        evidence="DIRECTIONAL SUPPORT" if n_negative > len(rho) / 2 else "NO_
↳SUPPORT",
    )

```

```

[685]: prism_class = hsp90_class_enrichment_summary.iloc[0]
n_hsp90_drugs = int(prism_class["N_HSP90_DRUGS"])
n_negative = int(prism_class["N_NEGATIVE_RHO"])
fraction_negative = float(prism_class["FRACTION_NEGATIVE"])
mean_rho = float(prism_class["MEAN_RHO"])
median_rho = float(prism_class["MEDIAN_RHO"])
empirical_p = float(prism_class["EMPIRICAL_P_VALUE"])

row_prism_class = make_evidence_row(
    analysis="PRISM class enrichment",
    effect=f"median rho={median_rho:.3f}; mean rho={mean_rho:.3f}; {n_negative}/
↳{n_hsp90_drugs} negative",
    direction="NEGATIVE",
    p_value=empirical_p,
    evidence="SUPPORTED" if empirical_p < 0.05 and fraction_negative > 0.5 else_
↳"DIRECTIONAL SUPPORT",
)

```

```
)
```

```
[686]: loo = hsp90_leave_one_out_summary.iloc[0]
loo_n_drugs = int(loo["N_HSP90_DRUGS"])
loo_n_negative = int(loo["N_NEGATIVE_RHO"])
loo_fraction_negative = loo_n_negative / loo_n_drugs if loo_n_drugs > 0 else np.
↳nan
loo_median_rho = float(loo["MEDIAN_RHO"])
loo_p = float(loo["EXACT_N_MATCHED_EMPIRICAL_P"])

row_leave_alvespimycin = make_evidence_row(
    analysis="Leave-alvespimycin-out",
    effect=f"median rho={loo_median_rho:.3f}; {loo_n_negative}/{loo_n_drugs}␣
↳negative",
    direction="NEGATIVE",
    p_value=loo_p,
    evidence="SUPPORTED" if loo_p < 0.05 and loo_fraction_negative > 0.5 else␣
↳"DIRECTIONAL SUPPORT",
)
```

```
[687]: full_correlation_df = (
    prism_hsp90_class_sensitivity[[
        "DEPMAP_ID",
        "RECURRENCE_SCORE",
        "HSP90_CLASS_SENSITIVITY",
    ]]
    .dropna()
    .copy()
)

if len(full_correlation_df) >= 3:
    full_rho, full_p = spearmanr(
        full_correlation_df["RECURRENCE_SCORE"],
        full_correlation_df["HSP90_CLASS_SENSITIVITY"],
    )

    row_cell_line_class = make_evidence_row(
        analysis="Cell-line HSP90 class phenotype",
        effect=f"rho={full_rho:.3f}; N={len(full_correlation_df)}",
        direction="POSITIVE" if full_rho > 0 else ("NEGATIVE" if full_rho < 0␣
↳else "ZERO"),
        p_value=float(full_p),
        evidence=(
            "SUPPORTED" if full_p < 0.05 and full_rho > 0
            else ("DIRECTIONAL SUPPORT" if full_rho > 0 else "NO SUPPORT")
        ),
    )
)
```

```

else:
    row_cell_line_class = make_evidence_row(
        analysis="Cell-line HSP90 class phenotype",
        evidence="NOT TESTABLE",
    )

```

```

[688]: global_df = (
    global_non_hsp90_sensitivity_df[[
        "DEPMAP_ID",
        "RECURRENCE_SCORE",
        "GLOBAL_NON_HSP90_SENSITIVITY",
    ]]
    .dropna()
    .copy()
)

if len(global_df) >= 3:
    global_rho, global_p = spearmanr(
        global_df["RECURRENCE_SCORE"],
        global_df["GLOBAL_NON_HSP90_SENSITIVITY"],
    )

    row_global_adjustment = make_evidence_row(
        analysis="Global drug sensitivity adjustment",
        effect=f"rho={global_rho:.3f}; N={len(global_df)}",
        direction="POSITIVE" if global_rho > 0 else ("NEGATIVE" if global_rho < 0
↪ else "ZERO"),
        p_value=float(global_p),
        evidence="DIRECTIONAL SUPPORT" if global_p < 0.05 else "NO SUPPORT",
    )
else:
    row_global_adjustment = make_evidence_row(
        analysis="Global drug sensitivity adjustment",
        evidence="NOT TESTABLE",
    )

```

```

[689]: prolifer_df = (
    crispr_primary_targets_adjusted_proliferation[
        crispr_primary_targets_adjusted_proliferation["GENE"].isin(
            ["HSP90AA1 (3320)", "HSP90AB1 (3326)"]
        )
    ]
    .copy()
)

if not prolifer_df.empty:

```

```

    beta_values = pd.to_numeric(prolif_df["BETA_RECURRENCE_SCORE"],
↳errors="coerce").dropna()
    p_values = pd.to_numeric(prolif_df["P_RECURRENCE_SCORE"], errors="coerce").
↳dropna()

    if len(beta_values) > 0:
        median_beta = float(beta_values.median())
        row_proliferation = make_evidence_row(
            analysis="Proliferation adjustment",
            effect=f"median HSP90 beta={median_beta:.3f}; {len(beta_values)}
↳genes",
            direction="POSITIVE" if median_beta > 0 else ("NEGATIVE" if
↳median_beta < 0 else "ZERO"),
            p_value=float(p_values.min()) if len(p_values) > 0 else np.nan,
            evidence=(
                "DIRECTIONAL SUPPORT" if median_beta < 0 and len(p_values) > 0
↳and p_values.min() < 0.05
                else "NO SUPPORT"
            ),
        )
    else:
        row_proliferation = make_evidence_row(
            analysis="Proliferation adjustment",
            evidence="NOT INFORMATIVE",
        )
else:
    row_proliferation = make_evidence_row(
        analysis="Proliferation adjustment",
        evidence="NOT TESTABLE",
    )

```

```

[690]: loo_drug_df = hsp90_leave_one_drug_out.copy()

rho_columns = [
    "SPEARMAN_RHO",
    "RHO",
    "LEAVE_ONE_OUT_RHO",
    "CLASS_RHO",
    "RHO_LEAVE_ONE_OUT",
]

loo_rho_col = next(
    (col for col in rho_columns if col in loo_drug_df.columns),
    None,
)

if loo_rho_col is None:

```



```

row_leave_one_drug = make_evidence_row(
    analysis="Leave-one-drug-out",
    evidence="NOT TESTABLE",
)

else:
    loo_drug_rho = pd.to_numeric(
        loo_drug_df[loo_rho_col],
        errors="coerce",
    ).dropna()

    if len(loo_drug_rho) == 0:
        row_leave_one_drug = make_evidence_row(
            analysis="Leave-one-drug-out",
            evidence="NOT INFORMATIVE",
        )

    else:
        n_positive = int((loo_drug_rho > 0).sum())
        n_total = len(loo_drug_rho)
        fraction_positive = n_positive / n_total
        median_loo_drug_rho = float(loo_drug_rho.median())

        row_leave_one_drug = make_evidence_row(
            analysis="Leave-one-drug-out",
            effect=(
                f"median rho={median_loo_drug_rho:.3f}; "
                f"{n_positive}/{n_total} positive"
            ),
            direction=(
                "POSITIVE"
                if fraction_positive > 0.5
                else "MIXED"
            ),
            p_value=np.nan,
            evidence=(
                "SUPPORTED"
                if n_positive == n_total
                else (
                    "DIRECTIONAL SUPPORT"
                    if fraction_positive > 0.5
                    else "NO SUPPORT"
                )
            ),
        )

```

```
[691]: def external_dataset_summary(association_df, database):
    df = association_df[association_df["DATABASE"] == database].copy()
    rho = pd.to_numeric(df["SPEARMAN_RHO"], errors="coerce").dropna()

    if len(rho) == 0:
        return make_evidence_row(
            analysis=database,
            evidence="NOT INFORMATIVE",
        )

    median_rho = float(rho.median())
    n_negative = int((rho < 0).sum())

    return make_evidence_row(
        analysis=database,
        effect=f"median rho={median_rho:.3f}; {n_negative}/{len(rho)} negative",
        direction="NEGATIVE" if n_negative > len(rho) / 2 else "MIXED",
        p_value=np.nan,
        evidence="NOT TESTABLE" if len(rho) < 3 else "DIRECTIONAL SUPPORT",
    )

row_gdsc1 = external_dataset_summary(hsp90_drug_association, "GDSC1")
row_gdsc2 = external_dataset_summary(hsp90_drug_association, "GDSC2")
row_ctd2 = external_dataset_summary(hsp90_drug_association, "CTD2")
row_ctrpv2 = external_dataset_summary(hsp90_drug_association, "CTRPv2")
```

```
[692]: crispr_df = (
    recurrence_score_crispr_primary_targets[
        recurrence_score_crispr_primary_targets["GENE"].isin(["HSP90AA1_
↪(3320)", "HSP90AB1 (3326)"])
    ]
    .copy()
)

def crispr_row(df, gene):
    row = df[df["GENE"] == gene]

    if row.empty:
        return make_evidence_row(
            analysis=f"CRISPR {gene.split()[0]}",
            evidence="NOT TESTABLE",
        )

    row = row.iloc[0]
    rho = float(row["SPEARMAN_RHO"])
```

```

p = float(row["P_VALUE"])

return make_evidence_row(
    analysis=f"CRISPR {gene.split()[0]}",
    effect=f"rho={rho:.3f}; N={int(row['N'])}",
    direction="POSITIVE" if rho > 0 else ("NEGATIVE" if rho < 0 else
↪ "ZERO"),
    p_value=p,
    evidence="NO SUPPORT" if rho >= 0 else "DIRECTIONAL SUPPORT",
)

row_crispr_hsp90aa1 = crispr_row(
    crispr_df,
    "HSP90AA1 (3320)",
)

row_crispr_hsp90ab1 = crispr_row(
    crispr_df,
    "HSP90AB1 (3326)",
)

```

```

[693]: patient_pathways = hsp90_patient_pathways.copy()

patient_pathways = patient_pathways[
    ~patient_pathways["PATHWAY"].astype(str).str.contains(
        "NO CLEAR PATIENT-SIDE",
        case=False,
        na=False,
    )
].copy()

if not patient_pathways.empty:
    patient_pathways["FDR_NUMERIC"] = pd.to_numeric(
        patient_pathways["FDR"],
        errors="coerce",
    )

    strongest_pathway = (
        patient_pathways
        .sort_values(
            "FDR_NUMERIC",
            na_position="last",
        )
        .iloc[0]
    )

```

```

row_patient_context = make_evidence_row(
    analysis="Patient-side pathway context",
    effect=(
        f"{strongest_pathway['PATHWAY']}; "
        f"NES={float(strongest_pathway['NES']):.3f}"
    ),
    direction=strongest_pathway["DIRECTION"],
    p_value=float(strongest_pathway["P_VALUE"]),
    evidence=(
        "SUPPORTED"
        if float(strongest_pathway["FDR"]) < 0.05
        else "DIRECTIONAL SUPPORT"
    ),
)
else:
    row_patient_context = make_evidence_row(
        analysis="Patient-side pathway context",
        evidence="NO SUPPORT",
    )

```

```

[694]: final_rows = [
    row_prism_individual,
    row_prism_class,
    row_leave_alvespimycin,
    row_cell_line_class,
    row_global_adjustment,
    row_proliferation,
    row_leave_one_drug,
    row_gdsc1,
    row_gdsc2,
    row_ctd2,
    row_ctrpv2,
    row_crispr_hsp90aa1,
    row_crispr_hsp90ab1,
    row_patient_context,
]

final_hsp90_evidence_matrix = pd.DataFrame(
    final_rows,
    columns=[
        "ANALYSIS",
        "EFFECT",
        "DIRECTION",
        "P_VALUE",
        "EVIDENCE",
    ],
)

```

```
[695]: analysis_order = [
    "PRISM individual compounds",
    "PRISM class enrichment",
    "Leave-alvespimycin-out",
    "Cell-line HSP90 class phenotype",
    "Global drug sensitivity adjustment",
    "Proliferation adjustment",
    "Leave-one-drug-out",
    "GDSC1",
    "GDSC2",
    "CTD2",
    "CTRPv2",
    "CRISPR HSP90AA1",
    "CRISPR HSP90AB1",
    "Patient-side pathway context",
]

final_hsp90_evidence_matrix["ANALYSIS"] = (
    pd.Categorical(final_hsp90_evidence_matrix["ANALYSIS"],
        categories=analysis_order,
        ordered=True,
    )
)

final_hsp90_evidence_matrix = (
    final_hsp90_evidence_matrix
    .sort_values("ANALYSIS")
    .reset_index(drop=True)
)

[696]: assert set(final_hsp90_evidence_matrix["EVIDENCE"]).issubset(VALID_EVIDENCE)

[697]: output_file = "../results/evidence/final_hsp90_evidence_matrix.xlsx"

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    final_hsp90_evidence_matrix.to_excel(writer,
    ↪sheet_name="HSP90_Evidence_Matrix", index=False)
    hsp90_drug_association.to_excel(writer, sheet_name="External_Drug_Level",
    ↪index=False)
    hsp90_class_enrichment_summary.to_excel(writer, sheet_name="PRISM_Class",
    ↪index=False)
    hsp90_leave_one_out_summary.to_excel(writer,
    ↪sheet_name="Leave_Alvespimycin", index=False)
    hsp90_leave_one_drug_out.to_excel(writer, sheet_name="Leave_One_Drug",
    ↪index=False)
    recurrence_score_crispr_primary_targets.to_excel(writer,
    ↪sheet_name="CRISPR_Primary", index=False)
```

```

    crispr_primary_targets_adjusted_proliferation.to_excel(writer,
↪sheet_name="CRISPR_Adjusted", index=False)
    hsp90_patient_pathways.to_excel(writer, sheet_name="Patient_Pathways",
↪index=False)

display(final_hsp90_evidence_matrix)

```

```

                                ANALYSIS \
0      PRISM individual compounds
1      PRISM class enrichment
2      Leave-alvespimycin-out
3      Cell-line HSP90 class phenotype
4      Global drug sensitivity adjustment
5      Proliferation adjustment
6      Leave-one-drug-out
7      GDSC1
8      GDSC2
9      CTD2
10     CTRPv2
11     CRISPR HSP90AA1
12     CRISPR HSP90AB1
13     Patient-side pathway context

                                EFFECT DIRECTION  P_VALUE \
0      median rho=-0.251; 11/12 negative  NEGATIVE  0.000017
1      median rho=-0.251; mean rho=-0.248; 11/12 nega...  NEGATIVE  0.000100
2      median rho=-0.251; 11/12 negative  NEGATIVE  0.000100
3      rho=0.354; N=43  POSITIVE  0.019826
4      rho=0.108; N=44  POSITIVE  0.486707
5      median HSP90 beta=0.664; 2 genes  POSITIVE  0.313119
6      median rho=0.324; 12/12 positive  POSITIVE      NaN
7      NaN  NaN  NaN
8      NaN  NaN  NaN
9      median rho=-0.085; 2/3 negative  NEGATIVE      NaN
10     median rho=0.025; 1/2 negative  MIXED      NaN
11     rho=0.051; N=64  POSITIVE  0.689218
12     rho=0.153; N=64  POSITIVE  0.228211
13     Regulation Of HSF1-mediated Heat Shock Respons...  NEGATIVE  0.000329

                                EVIDENCE
0      DIRECTIONAL SUPPORT
1      SUPPORTED
2      SUPPORTED
3      SUPPORTED
4      NO SUPPORT
5      NO SUPPORT
6      SUPPORTED

```

7	NOT INFORMATIVE
8	NOT INFORMATIVE
9	DIRECTIONAL SUPPORT
10	NOT TESTABLE
11	NO SUPPORT
12	NO SUPPORT
13	SUPPORTED

- `final_hsp90_evidence_matrix.xlsx` exported to `/results` directory

Summary Note: The integrated evidence supports a reproducible association between the Recurrence Score and HSP90-related sensitivity. PRISM class-level enrichment and robustness analyses were supported, while the cell-line HSP90 phenotype remained significant after adjustment for global drug sensitivity and proliferation. Independent GDSC1, GDSC2, CTD², and CTRPv2 datasets showed directionally consistent effects but provided only directional support because the available HSP90 inhibitor sets were limited. CRISPR analyses did not detect recurrence-associated single-gene dependency for HSP90AA1 or HSP90AB1, whereas patient-side HSF1/heat-shock pathway activity provided independent biological context. The pre-specified clinical validation in the independent GSE186332 selinexor-treated recurrent glioblastoma cohort did not support the predictive value of the frozen Recurrence Score. Importantly, neither the selinexor signature nor the response cutoff was modified following this negative validation result.